Digitalteknik

Digitalteknik

Från ettor och nollor till en mikrodator som styr något i verkligheten

Jonas Granath och Erik Pihl

19 oktober 2026

Digitalteknik
Copyright © 2026 Jonas Granath och Erik Pihl.

Bokens text och figurer, liksom kursmaterialet den är satt från, är licensierade under Creative Commons Erkännande 4.0 Internationell (CC BY 4.0). Du får kopiera och sprida materialet i vilket medium och format som helst, och bearbeta det, för vilket ändamål som helst, även kommersiellt, förutsatt att du anger upphovspersonen, länkar till licensen och anger om du har gjort ändringar. Licensen finns på https://creativecommons.org/licenses/by/4.0/deed.sv.

Programkoden i boken, assemblerprogrammen och deras kommentarer, får också användas under MIT-licensen, som är licensen för all källkod i kursmaterialet: programmen, skripten som kontrollerar dem och verktygen som ritar figurerna och sätter boken.

Programmen i boken är skrivna för ATmega328P med klockfrekvensen 16 MHz, och assembleras i Microchip Studio. ATmega328P, AVR och Microchip Studio är varumärken som tillhör Microchip Technology Inc.; Arduino och Arduino Uno är varumärken som tillhör Arduino S.r.l.; Windows är ett varumärke som tillhör Microsoft Corporation.

Satt med TeX Gyre Pagella och DejaVu Sans Mono i LuaLaTeX.

Satt från kursens tolv pass, deras övningar och lösningsförslag, de tre laborationshandledningarna och övningsprovet. Den här utgåvan: 19 oktober 2026.

Innehåll

Förord **ix**

1 Talsystem och binära koder **1**
1.1 Talsystem . . . 1
1.2 Binära koder . . . 10
1.3 Sammanfattning . . . 14
1.4 Övningar . . . 15

2 Logiska grindar, boolesk algebra och kontaktnät **21**
2.1 Logiska grindar och boolesk algebra . . . 22
2.2 Kontaktnät . . . 33
2.3 Sammanfattning . . . 43
2.4 Övningar . . . 44

3 Analys och felsökning av kontaktnät **51**
3.1 Analys och felsökning av kontaktnät . . . 52
3.2 Sammanfattning . . . 58
3.3 Övningar . . . 59

4 Karnaughdiagram och IC-kretsar **65**
4.1 Karnaughdiagram . . . 65
4.2 IC-kretsar . . . 80
4.3 Sammanfattning . . . 90
4.4 Övningar . . . 91

5 Felsökning av grindnät **97**
5.1 Felsökning av grindnät . . . 98
5.2 Sammanfattning . . . 103
5.3 Övningar . . . 104

6 Sekvensnät, vippor och mikrodatorns uppbyggnad **109**
6.1 Sekvensnät och vippor . . . 110

6.2 Mikrodatorns uppbyggnad . . . 118
6.3 Sammanfattning . . . 126
6.4 Övningar . . . 127

7 Mikrodatorn och Microchip Studio **133**
7.1 Mikrodatorn ATmega328P . . . 134
7.2 Assemblerspråk . . . 139
7.3 Simulering i Microchip Studio . . . 144
7.4 Sammanfattning . . . 147
7.5 Övningar . . . 148

8 Aritmetik, logik och flaggor **155**
8.1 Aritmetik och flaggor . . . 156
8.2 Logiska operationer, skift och rotation . . . 162
8.3 Utporten . . . 168
8.4 Sammanfattning . . . 173
8.5 Övningar . . . 174

9 Programflöde, loopar och tidsfördröjningar **179**
9.1 Flödesplaner och villkorliga hopp . . . 180
9.2 Loopar och tidsfördröjningar . . . 187
9.3 Sammanfattning . . . 193
9.4 Övningar . . . 194

10 Inporten, subrutiner och stacken **199**
10.1 ASCII-koder och 16-bitarstal . . . 200
10.2 Inporten . . . 208
10.3 Subrutiner och stacken . . . 213
10.4 Sammanfattning . . . 221
10.5 Övningar . . . 222

11 Styrobjekt på riktig hårdvara **227**
11.1 Styrobjekt: in- och utgångar i verkligheten . . . 228
11.2 Genomarbetat exempel: från flödesplan till program . . . 234
11.3 Sammanfattning . . . 240
11.4 Övningar . . . 241

12 Repetition inför det skriftliga provet **245**
12.1 Repetition . . . 245
12.2 Sammanfattning . . . 251
12.3 Övningar . . . 252

A Labb 1: Kopplingsnät **257**
A.1 Mål . . . 257
A.2 Utrustning . . . 257
A.3 Säkerhet . . . 258
A.4 Förberedelser . . . 258
A.5 Arbetsgång . . . 259
A.6 Uppgifterna . . . 260
A.7 Redovisning . . . 268

B Labb 2: Grindnät med IC-kretsar **269**
B.1 Mål . . . 269
B.2 Utrustning . . . 269
B.3 Säkerhet . . . 274
B.4 Förberedelser . . . 275
B.5 Labb 2-1 . . . 276
B.6 Labb 2-2 . . . 281
B.7 Redovisning . . . 283

C Labb 3: Mikrodatorn **285**
C.1 Mål . . . 285
C.2 Upplägg . . . 285
C.3 Programmallen . . . 286
C.4 Delarna . . . 286
C.5 Redovisning och bedömning . . . 287
C.6 När något inte fungerar . . . 287
C.7 Del 1.1, 1.2 och 2: Första stegen . . . 288
C.8 Del 3–10: Aritmetik, logik och flaggor . . . 292
C.9 Del 11, 12 och 14–17: Programflöde, loopar och tidsfördröjningar . . . 298
C.10 Del 18–23: Text, 16 bitar, inporten och subrutiner . . . 304
C.11 Slutuppgift: trafikljuset och övergångsstället . . . 312

D Microchip Studio **319**
D.1 Installera Microchip Studio . . . 319
D.2 Skapa ett assemblerprojekt . . . 319
D.3 Assemblera . . . 320
D.4 Simulera . . . 321
D.5 Programmera ett Arduino-kort från Microchip Studio . . . 323
D.6 Vanliga fel . . . 336

E Referensblad: AVR-instruktionerna **337**

F Det skriftliga provet **341**

G Övningsprov **345**

H Lösningsförslag till övningsprovet **349**
Fråga 1: Talsystem och koder . . . 349
Fråga 2: Grindar och boolesk algebra . . . 350
Fråga 3: Karnaughdiagram . . . 351
Fråga 4: Analys av ett kontaktnät . . . 352
Fråga 5: Mikrodatorns uppbyggnad . . . 353
Fråga 6: Assemblerprogrammering . . . 353

I Lösningsförslag till övningarna **355**
1 Talsystem och binära koder . . . 355
2 Logiska grindar, boolesk algebra och kontaktnät . . . 362
3 Analys och felsökning av kontaktnät . . . 369
4 Karnaughdiagram och IC-kretsar . . . 373
5 Felsökning av grindnät . . . 388
6 Sekvensnät, vippor och mikrodatorns uppbyggnad . . . 393
7 Mikrodatorn och Microchip Studio . . . 398
8 Aritmetik, logik och flaggor . . . 403
9 Programflöde, loopar och tidsfördröjningar . . . 408
10 Inporten, subrutiner och stacken . . . 411
11 Styrobjekt på riktig hårdvara . . . 417
12 Repetition inför det skriftliga provet . . . 421

Förord

Den här boken tar dig från ettor och nollor till en mikrodator som styr något i verkligheten. Först talsystem, logiska grindar och boolesk algebra, byggda både med strömkontakter på 24 V och med IC-kretsar på kopplingsdäck. Sedan vippan, den byggsten som ger en krets minne, och därifrån mikrodatorn: en ATmega328P som programmeras i assembler, först i simulatorn och till sist på ett riktigt kort där den styr lysdioder och läser av en knapp.

Bokens text är på svenska. All kod och alla kommentarer i koden är på engelska, vilket är det du kommer att möta i datablad, instruktionslistor och på nätet.

Bokens tre delar

Del 1: Digitala kombinatoriska nät, kapitel 1–5. Binära, hexadecimala och decimala tal och omvandling mellan dem. Logiska grindar, sanningstabeller och boolesk algebra, och hur ett uttryck förenklas, algebraiskt och med Karnaughdiagram. Samma logik realiseras på två sätt: med tryckknappar och en lampa på 24 V i **Labb 1**, och med IC-kretsar ur 74-serien på kopplingsdäck i **Labb 2**.

Del 2: Från vippa till mikrodator, kapitel 6. Sekvensnät och vippor: hur en krets kan minnas. Register, räknare och minne är vippor, och mikrodatorn är register och minne kring en aritmetisk enhet, byggd av de grindar del 1 handlade om. Kapitlet slutar i mikrodatorns blockschema.

Del 3: Mikrodatorn och assemblerprogrammering, kapitel 7–11. ATmega328P programmeras i assembler i Microchip Studio: talformat, aritmetik och flaggor, logiska operationer, villkorliga hopp, loopar och tidsfördröjningar, in- och utportar, subrutiner och stacken. **Labb 3** löper genom alla fem kapitlen, och avslutas med ett styrobjekt, ett trafikljus, som körs på ett Arduino Uno-kort.

Repetition, kapitel 12. Hela kursen i sammandrag, med de vanligaste felen och övningar i provets form, inför det skriftliga provet.

Det här ska du kunna efteråt

När du har arbetat dig igenom boken ska du kunna:

- Förklara grindar och logiska funktioner, och beskriva logisk algebra och förenkling av booleska uttryck.
- Beskriva det binära, det hexadecimala och det decimala talsystemet, och omvandla tal mellan dem.
- Syntetisera ett digitalt kombinatoriskt nät, med IC-kretsar och med kontakter, och använda Karnaughdiagram för att analysera och minimera det.
- Analysera ett digitalt kombinatoriskt nät: från krets till uttryck till sanningstabell.
- Redogöra för mikrodatorns uppbyggnad på blockschemanivå.
- Använda assemblerspråk för att programmera en mikrodator, och programmera den att styra ett styrobjekt.

Vad du behöver kunna innan

Inga förkunskaper i elektronik eller programmering förutsätts. Det räcker med matematik på gymnasienivå: potenser, och de fyra räknesätten utan miniräknare.

Kursen boken är skriven för går snabbt. Tolv pass om tre timmar rymmer fyra ämnen som var och ett skulle kunna fylla en egen kurs, så **varje kapitel förutsätter att det föregående är läst och att dess övningar är gjorda.**

Hur boken är upplagd

Boken är satt från en kurs med tolv pass om tre timmar, och varje kapitel är ett pass. Kapitlets avsnitt är passets appendix, i samma ordning och med samma underavsnitt, så att ett kapitel här och ett pass i kursmaterialet kan läsas sida vid sida:

Kapitel	Pass	Ämne
1	`L01`	Talsystem och binära koder
2	`L02`	Logiska grindar, boolesk algebra och kontaktnät
3	`L03`	Analys och felsökning av kontaktnät, med **Labb 1**
4	`L04`	Karnaughdiagram och IC-kretsar
5	`L05`	Felsökning av grindnät, med **Labb 2**
6	`L06`	Sekvensnät, vippor och mikrodatorns uppbyggnad
7	`L07`	Mikrodatorn och Microchip Studio; **Labb 3** börjar
8	`L08`	Aritmetik, logik och flaggor
9	`L09`	Programflöde, loopar och tidsfördröjningar
10	`L10`	Inporten, subrutiner och stacken
11	`L11`	Styrobjekt på riktig hårdvara; **Labb 3** slutförs
12	`L12`	Repetition inför det skriftliga provet

Varje kapitel börjar med vad det innehåller, arbetar sig igenom materialet, sammanfattar vad du nu ska kunna förklara, och slutar med sina övningar. Bilagorna håller det som används vid sidan av kapitlen: de tre laborationshandledningarna, en guide till Microchip

Studio, referensbladet med AVR-instruktionerna, en beskrivning av det skriftliga provet med ett övningsprov och dess lösningsförslag, och sist lösningsförslagen till övningarna.

Övningarnas sorter

Varje övning är märkt med sin sort:

Sort	Vad den ber dig göra
FÖRSTÅELSE	Förklara något från kapitlet med egna ord. Inga verktyg.
RÄKNA FÖR HAND	Räkna ut något på papper: en omvandling, en förenkling, en cykelräkning.
KONSTRUKTION	Ta fram en lösning: ett grindnät, ett kontaktnät, en flödesplan.
PROGRAM	Skriv assembler, och kör det i simulatorn.
KONTROLL	En per kapitel. Se nedan.
FÖRDJUPNING	En övning för den som vill gå längre. Den kan hoppas över.

Kontrolluppgiften är kursens signaturövning. Du räknar ut ett svar för hand, du kontrollerar det med ett verktyg, i CircuitVerse, i simulatorn eller på labbstationen, och du förklarar varför de två stämmer, eller varför de inte gör det. Att förutsäga ett beteende och sedan kontrollera det är den färdighet resten av kursen vilar på.

Lösningsförslagen

Lösningsförslagen till övningarna finns i bilaga I, sist i boken, kapitel för kapitel. Där står svaret på varje övning som räknas, konstrueras eller förklaras, med uträkningen, och ofta med de vanligaste felaktiga svaren och varför de är fel.

Två slags uppgifter har inga lösningsförslag i boken. **Programuppgifterna**, där du skriver assembler: ett program lär du dig skriva genom att skriva det, och simulatorn visar om det gör vad det ska. Delar av en programuppgift som räknas eller förklaras, en förutsägelse av vad registren innehåller eller en cykelräkning, har däremot sina svar i bilagan. **Laborationerna** redovisas för läraren vid stationen, och deras svar är det du visar där.

Försök alltid själv först: ett svar du har läst är mycket mindre värt än ett du har räknat fram och sedan kontrollerat.

Laborationerna

Kursen har tre laborationer, som görs i par. Labb 1 och Labb 2 görs på varsitt pass; Labb 3 löper över de fem mikrodatorkapitlen. Handledningarna finns i bilaga A till C.

- **Förbered dig hemma.** Varje handledning börjar med förberedelseuppgifter: sanningstabeller, uttryck och kopplingsscheman som ska vara klara när passet börjar. Utan dem hinner man inte klart, och det är den enskilt vanligaste orsaken till att en laboration inte blir godkänd på passet.

- **Arbeta i par**, och byt roll: den ena kopplar eller skriver, den andra kontrollerar mot schemat. Båda ska kunna förklara lösningen.
- **Förutsäg, sedan kontrollera.** Skriv ned vad du förväntar dig innan du slår på strömmen eller kör programmet. En avvikelse mellan förutsägelse och resultat är information: den säger att schemat, kopplingen eller programmet har ett fel, och att du nu vet var du ska leta.
- **Redovisa för läraren** varje uppgift som är markerad REDOVISA. Läraren prickar av den i redovisningslistan.

> **Säkerhet.**
>
> - Koppla **alltid** med spänningen frånslagen, och slå på först när kopplingen är kontrollerad.
> - 24 V DC är ingen farlig spänning att beröra, men en kortslutning kan ge stora strömmar, heta sladdar och en bränd säkring i aggregatet. En glödlampa blir dessutom varm.
> - Koppla aldrig 24 V till kopplingsdäcket eller till Arduino-kortet. IC-kretsarna och mikrodatorn ska matas med 5 V, och 24 V förstör dem på ett ögonblick.
> - IC-kretsar vänds lätt fel. En krets som blir varm är felvänd eller felkopplad: bryt matningen direkt.

Programvara

- **CircuitVerse**, `https://circuitverse.org/simulator`: en gratis, webbläsarbaserad logiksimulator, där grindnät byggs och testas innan de kopplas upp. Inget att installera.
- **Microchip Studio 7**, `https://www.microchip.com/en-us/tools-resources/develop/microchip-studio`: en gratis utvecklingsmiljö för AVR, där programmen i Labb 3 skrivs, assembleras och simuleras. Den installeras före kapitel 7 enligt bilaga D. Microchip Studio finns bara för Windows; den som har en Mac använder skolans datorer eller en virtuell Windowsmaskin.
- **Arduino IDE**, `https://www.arduino.cc/en/software`: används en enda gång, i kapitel 11, för att ta fram kommandot som programmerar kortet (se bilaga D).

Under kursen går det bra att kontrollera omvandlingar med kalkylatorn i Windows, i läget **Programmerare**, men räkna alltid för hand först. På provet finns ingen miniräknare.

Utrustning

Labb 1 görs vid en station med ett 24 V DC-aggregat, två 4-poliga tryckknappar med både slutande (NO) och brytande (NC) kontakter, en glödlampa för 24 V, kopplingssladdar och en multimeter för felsökning.

Labb 2 görs vid ett kopplingsdäck med kringutrustning: 5 V-matning, strömbrytare eller knappar för ingångarna och lysdioder för utgångarna. Till det kommer IC-kretsarna 74HC08

(AND), 74HC32 (OR) och 74HC04 (inverterare) till Labb 2-1, 74HC00 (NAND) och 74HC02 (NOR) till Labb 2-2, kopplingstråd och en multimeter.

Labb 3 görs med ett Arduino Uno-kort (ATmega328P) med USB-kabel, ett kopplingsdäck, tre lysdioder (röd, gul och grön) med förkopplingsmotstånd på 220–330 Ω, en tryckknapp och kopplingstråd. Det mesta av Labb 3 görs i simulatorn i Microchip Studio; kortet behövs först i kapitel 11.

Referenser

Två dokument från tillverkaren används som referens i mikrodatordelen:

- *ATmega328P datasheet*: `https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf`
- *AVR Instruction Set Manual*: `https://ww1.microchip.com/downloads/en/devicedoc/atmel-0856-avr-instruction-set-manual.pdf`

Det du behöver av dem under kursen, instruktionerna, flaggorna och cykeltiderna, finns samlat på referensbladet i bilaga E.

KAPITEL 1

Talsystem och binära koder

Varför en dator räknar med ettor och nollor, och hur man räknar som den.

I DET HÄR KAPITLET

- Digitalt och analogt: varför nästan all teknik i dag räknar med ettor och nollor.
- Positionssystem: det decimala, det binära och det hexadecimala talsystemet.
- Omvandling mellan decimalt, binärt och hexadecimalt, för hand.
- Bitar, byte och talområden: hur stora tal som ryms i 4, 8 och 16 bitar.
- Binär addition, och vad som händer när summan inte ryms.
- Binära koder: BCD, Graykod, ASCII, 7-segmentkod och paritetsbit.

Allt i den här boken vilar på ett enda faktum: en digital signal har bara två tillstånd. Det här kapitlet visar varför det är en styrka och inte en begränsning, och hur många sådana signaler tillsammans kan bära ett tal, en bokstav eller en siffra på en display.

Arbeta igenom kapitlet i ordning. Avsnitt 1.1 om talsystemen är grunden för resten av boken: viktraden 1, 2, 4, 8, 16, 32, 64, 128 och omvandlingarna mellan decimalt, binärt och hexadecimalt används i varje kapitel som följer, och de sitter först när du har räknat många själv. Avsnitt 1.2 går igenom de binära koder du kommer att möta, både i kursen och i yrkeslivet. Övningarna i avsnitt 1.4 räknas med papper och penna, och deras lösningsförslag finns i bilaga I.

1.1 Talsystem

1.1.1 Digitalt och analogt

En **analog** signal kan anta vilket värde som helst inom ett intervall. Spänningen från en temperaturgivare kan vara 1,2 V, 1,21 V eller 1,2137 V, och varje liten ändring betyder något. Det gör också varje liten störning: brus från en motor i närheten flyttar signalen, och mottagaren kan inte veta vilken del av värdet som är mätning och vilken som är störning.

En **digital** signal antar bara två värden, som vi kallar `0` och `1`. I en krets som matas med 5 V betyder ungefär 0 V en nolla och ungefär 5 V en etta, och allt som ligger tillräckligt nära

någon av dem läses som exakt `0` eller exakt `1`. En störning på några tiondels volt ändrar därför ingenting alls.

Den tåligheten mot brus är skälet till att nästan all modern elektronik är digital, från mikrokontrollern i en kaffebryggare till styrsystemet i en fabrik. Priset är att en digital signal bara bär en enda uppgift, ja eller nej, tänd eller släckt. För att kunna representera ett tal, en temperatur eller en bokstav behövs många digitala signaler tillsammans, och ett sätt att tolka dem. Det är vad talsystemen och koderna i det här kapitlet handlar om.

En digital signal, eller en siffra i ett binärt tal, kallas en **bit**, av engelskans *binary digit*.

1.1.2 Positionssystem och talbas

Det talsystem vi använder till vardags, det **decimala** talsystemet, har tio siffror, 0 till 9. Antalet siffror kallas talsystemets **talbas**, så det decimala talsystemet har talbasen 10.

Det är också ett **positionssystem**: en siffras värde beror på var i talet den står. I talet 25 är femman värd 5 och tvåan värd 20. Lägg till en nolla sist, 250, så flyttar båda ett steg åt vänster och blir värda tio gånger mer: femman 50 och tvåan 200.

Varje position har alltså en **vikt**, en potens av talbasen. Positionerna numreras från höger med början på **noll**:

Position	2	1	0
Vikt	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
Siffra i talet 250	2	5	0
Värde	200	50	0

Talets värde är summan av varje siffra gånger vikten för dess position:

$$250 = 2 \cdot 10^2 + 5 \cdot 10^1 + 0 \cdot 10^0 = 200 + 50 + 0$$

Att numreringen börjar på noll är ingen godtycklig vana. Den högra positionen har vikten $B^0 = 1$ i **alla** talsystem, oavsett talbas B, och det är skälet till att den kallas position noll.

Samma formel gäller alltså för varje talsystem. En siffra x_n på position n i ett tal med talbasen B är värd

$$x_n \cdot B^n$$

och talets värde är summan av alla siffrornas värden. Den högra siffran, med lägst vikt, kallas den **minst signifikanta** siffran, och den vänstra den **mest signifikanta**.

När det inte framgår av sammanhanget vilket talsystem ett tal är skrivet i, skrivs talbasen som ett index efter talet: 25_{10} är tjugofem decimalt, 11001_2 är samma tal binärt. I kod och i den här bokens text används i stället ett prefix, se avsnitt 1.1.8.

1.1.3 Det binära talsystemet

En digital signal har två tillstånd, så den naturliga talbasen i digitaltekniken är 2. Det **binära** talsystemet har bara siffrorna 0 och 1, och varje position är värd dubbelt så mycket som positionen till höger om den:

Position	7	6	5	4	3	2	1	0
Vikt	128	64	32	16	8	4	2	1

Den raden med vikter är det viktigaste i hela det här kapitlet. Lär dig den utantill, från höger: 1, 2, 4, 8, 16, 32, 64, 128.

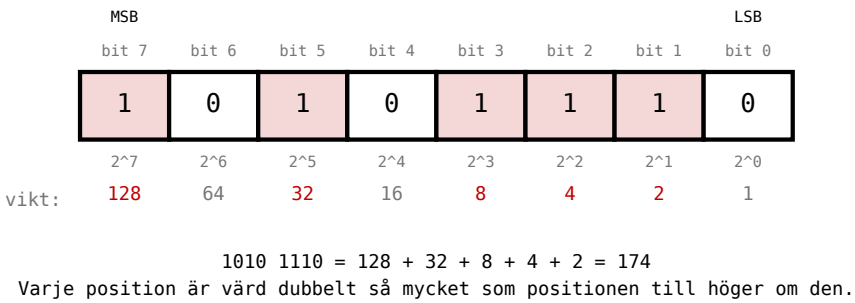
Från binärt till decimalt

För att omvandla ett binärt tal till decimalt summerar du vikterna för de positioner där det står en etta. Nollorna bidrar inte med något och kan hoppas över.

Exempel: 1011_2 har ettor på position 3, 1 och 0:

$$1011_2 = 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 8 + 2 + 1 = 11_{10}$$

Exempel: $1010\,1110_2$ har ettor på position 7, 5, 3, 2 och 1:



Figur 1.1 Ett 8-bitars tal med bitnumren ovanför och vikterna under: talet `1010 1110` summeras till 174.

$$1010\,1110_2 = 128 + 32 + 8 + 4 + 2 = 174_{10}$$

Binära tal skrivs ofta i grupper om fyra siffror, med ett mellanrum emellan, på samma sätt som man skriver 1 000 000 i stället för 1000000. Mellanrummet betyder ingenting; det gör bara talet lättare att läsa.

Bitar, nibblar och byte

Några namn på grupper av bitar som återkommer genom hela boken:

Namn	Antal bitar	Exempel
bit	1	`1`
nibble	4	`1011`
byte	8	`1010 1110`
ord (på AVR-processorn)	16	`0000 0011 1110 1000`

Den vänstra biten, med störst vikt, kallas **MSB** (*most significant bit*). Den högra, med vikten 1, kallas **LSB** (*least significant bit*). I en byte är MSB bit 7 och LSB bit 0.

Två saker går att se direkt på LSB och MSB:

- **LSB avgör om talet är udda eller jämnt.** Alla andra vikter är jämna tal, så bara LSB kan göra summan udda.
- **MSB avgör om talet är minst hälften av det största talet.** I en byte är MSB värd 128, och resten tillsammans som mest 127.

Talområden

Med n bitar finns det 2^n olika kombinationer av ettor och nollor, eftersom varje bit fördubblar antalet möjligheter. Den minsta är noll, så den största är en mindre än antalet kombinationer:

$$\text{antal värden} = 2^n \qquad \text{största värde} = 2^n - 1$$

Antal bitar	Antal värden	Talområde
1	2	0–1
4	16	0–15
8	256	0–255
10	1024	0–1023
16	65 536	0–65 535

Du kan kontrollera den största: åtta ettor är $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$, vilket är $2^8 - 1$.

Det här är ingen teoretisk detalj. Mikrodatorn i den här boken, ATmega328P, räknar med 8 bitar åt gången, så ett av dess register kan bara hålla talen 0–255. Vad som händer när ett tal inte ryms är ämnet för avsnitt 1.1.7, och för en stor del av kapitel 8.

1.1.4 Från decimalt till binärt

Åt andra hållet finns två metoder. Båda ger samma svar; välj den du tycker är enklast.

Metod 1: vikttabellen

Skriv upp vikterna, och ta för varje vikt från vänster ställning till om den ryms i det som är kvar av talet. Ryms den skriver du en etta och drar bort vikten; annars skriver du en nolla.

Exempel: omvandla 23_{10} till binärt.

Vikt	16	8	4	2	1
Ryms den i det som är kvar?	23: ja	7: nej	7: ja	3: ja	1: ja
Kvar efteråt	7	7	3	1	0
Bit	1	0	1	1	1

Alltså är $23_{10} = 10111_2$. Kontrollera baklänges: $16 + 4 + 2 + 1 = 23$.

Metod 2: division med 2

Dela talet med 2 och skriv upp resten, som alltid är 0 eller 1. Dela sedan kvoten med 2 igen, och fortsätt tills kvoten är 0. Resterna är bitarna, med **den första resten som LSB**.

Exempel: omvandla 23_{10} till binärt igen.

Division	**Kvot**	**Rest**
23 / 2	11	1 (bit 0, LSB)
11 / 2	5	1 (bit 1)
5 / 2	2	1 (bit 2)
2 / 2	1	0 (bit 3)
1 / 2	0	1 (bit 4, MSB)

Resterna läses **nedifrån och upp**: 10111_2, samma svar som med tabellen.

Det vanligaste felet med den här metoden är att läsa resterna uppifrån och ned, vilket ger talet baklänges, $11101_2 = 29$. Kontrollera därför alltid svaret genom att räkna tillbaka till decimalt.

Att fylla ut till en hel byte

10111_2 har fem bitar. Ska talet ligga i ett 8-bitars register fylls det ut med nollor till vänster: `0001 0111`. Nollor till vänster ändrar inte värdet, precis som 023 är samma sak som 23.

1.1.5 Det hexadecimala talsystemet

Binära tal blir snabbt långa och svårlästa. Ett 16-bitars tal som `0100 1010 1000 1100` är lätt att skriva av fel, och det är nästan omöjligt att se vilket tal det är. Därför skrivs binära tal nästan alltid i det **hexadecimala** talsystemet, med talbasen 16.

Det behövs 16 siffror, så efter 0–9 används bokstäverna A–F:

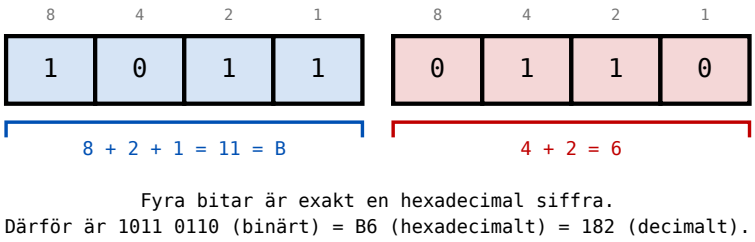
Decimalt	Binärt	Hexadecimalt	Decimalt	Binärt	Hexadecimalt
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Poängen med just talbasen 16 är att $16 = 2^4$**: en hexadecimal siffra motsvarar exakt fyra bitar**, en nibble. Omvandlingen mellan binärt och hexadecimalt kräver därför ingen räkning alls, bara tabellen ovan.

Från binärt till hexadecimalt

Dela upp det binära talet i grupper om fyra bitar, **med början från höger**, och byt varje grupp mot sin hexadecimala siffra.

Exempel: $1011\,0110_2$.



Figur 1.2 Talet `1011 0110` uppdelat i två grupper om fyra bitar: den vänstra gruppen, `1011`, blir B och den högra, `0110`, blir 6.

Alltså är $1011\,0110_2 = \mathrm{B6}_{16}$.

Om den sista gruppen till vänster har färre än fyra bitar fyller du ut den med nollor: $1\,1110_2 = 0001\,1110_2 = \mathrm{1E}_{16}$. Det är därför grupperingen börjar från höger. Börjar man från vänster hamnar de utfyllande nollorna mitt i talet, där de ändrar värdet.

Från hexadecimalt till binärt

Byt varje hexadecimal siffra mot sina fyra bitar.

Exempel: $\mathrm{4A8C}_{16}$.

Hexadecimal siffra	4	A	8	C
Fyra bitar	`0100`	`1010`	`1000`	`1100`

Alltså är $4A8C_{16} = 0100\,1010\,1000\,1100_2$. Fyra tecken i stället för sexton, och omvandlingen tar några sekunder när tabellen väl sitter.

1.1.6 Omvandling mellan alla tre

Mellan binärt och hexadecimalt är det bara att gruppera, som i avsnitt 1.1.5. Mellan decimalt och de andra två finns två vägar.

Från hexadecimalt till decimalt

Använd formeln från avsnitt 1.1.2 med talbasen 16. Vikterna är $16^0 = 1$, $16^1 = 16$, $16^2 = 256$ och $16^3 = 4096$.

Exempel: $F6_{16}$, där F är 15:

$$F6_{16} = 15 \cdot 16 + 6 \cdot 1 = 240 + 6 = 246_{10}$$

Exempel: $2A5_{16}$, där A är 10:

$$2A5_{16} = 2 \cdot 256 + 10 \cdot 16 + 5 \cdot 1 = 512 + 160 + 5 = 677_{10}$$

Den andra vägen går via binärt: $F6_{16} = 1111\,0110_2 = 128 + 64 + 32 + 16 + 4 + 2 = 246$. Den är ofta lika snabb för tvåsiffriga tal, och bra som kontroll.

Från decimalt till hexadecimalt

Också här finns två vägar.

Via binärt: omvandla till binärt enligt avsnitt 1.1.4, och gruppera sedan i fyror. 200_{10} blir $1100\,1000_2$, som är $C8_{16}$.

Division med 16: som divisionsmetoden i avsnitt 1.1.4, men med 16 i stället för 2. Resterna är hexadecimala siffror, och även här är den första resten den minst signifikanta.

Division	**Kvot**	**Rest**
200 / 16	12	8
12 / 16	0	12 = C

Nedifrån och upp: $C8_{16}$. Kontroll: $12 \cdot 16 + 8 = 192 + 8 = 200$.

En sammanfattning

Från \ Till	Decimalt	Binärt	Hexadecimalt
Decimalt	–	vikttabellen eller division med 2	via binärt, eller division med 16
Binärt	summera vikterna	–	gruppera i fyror från höger
Hexadecimalt	siffra gånger 16^n	fyra bitar per siffra	–

1.1.7 Binär addition

Binära tal adderas med samma uppställning som decimala: kolumn för kolumn från höger, med en **minnessiffra** (en *carry*) som förs vidare till nästa kolumn när summan inte ryms i en siffra.

Skillnaden är bara att det ”blir tio” redan vid två. Fyra regler räcker:

Summa	Resultat	Minnessiffra
$0 + 0$	0	0
$0 + 1$	1	0
$1 + 1$	0	1
$1 + 1 + 1$ (med minnessiffra)	1	1

Den tredje raden är den som känns ovan: $1 + 1 = 10_2$, alltså två, skrivet som en nolla med en etta i minnet.

Exempel: 90 + 55, med båda talen som 8-bitars binära tal.

```
minne:   1111 1100
           0101 1010     (90)
       + 0011 0111     (55)
       -----------
           1001 0001     (145)
```

Kontrollera: $1001\,0001_2 = 128 + 16 + 1 = 145$, och $90 + 55 = 145$.

När summan inte ryms

Ett 8-bitars register rymmer 0–255. Vad händer med 240 + 17 = 257?

```
minne: 1 1110 0000
           1111 0000     (240)
       + 0001 0001     (17)
       -----------
       1 0000 0001
```

Summan behöver nio bitar. I ett 8-bitars register finns bara plats för de åtta högra, `0000 0001`, alltså 1, och den nionde biten, minnessiffran ut ur bit 7, hamnar utanför. Resultatet blir $257 - 256 = 1$.

Det är inte ett fel i datorn; det är vad åtta bitar betyder. Talet slår runt, som vägmätaren i en gammal bil som går från 99 999 till 00 000. Mikrodatorn sparar den nionde biten i en särskild flagga, *carry*, så att programmet kan upptäcka att det hände. Det är ämnet för kapitel 8, där samma sak kallas **aritmetisk rundgång**.

Hexadecimal addition

Hexadecimala tal kan adderas direkt, med regeln att det ”blir tio” vid 16. Oftast är det lika enkelt att gå via binärt eller decimalt.

Exempel: $3C_{16} + 0A_{16}$. Entalskolumnen: $C + A = 12 + 10 = 22 = 16 + 6$, så siffran blir 6 med en etta i minnet. Sextontalskolumnen: $3 + 0 + 1 = 4$. Summan är 46_{16}, och kontrollen i decimalt är $60 + 10 = 70 = 4 \cdot 16 + 6$.

1.1.8 Skrivsätt

Samma tal kan skrivas på flera sätt, och vilket som används beror på sammanhanget.

Skrivsätt	Exempel (tjugofem)	Används
Index efter talet	25_{10}, 11001_2, 19_{16}	i matematik och i läroböcker
Prefix `0b`	`0b00011001`	i kod, för binära tal
Prefix `0x`	`0x19`	i kod, för hexadecimala tal
Prefix `$`	`$19`	i äldre AVR-assembler, för hexadecimala tal
Utan prefix	`25`	alltid decimalt

Den här boken använder prefixen `0b` och `0x` i löptext och i kod, eftersom det är så talen skrivs i assemblerprogrammen från kapitel 7 och framåt. Indexen används bara i härledningar som den här, där båda skrivsätten behövs sida vid sida.

Två saker är värda att notera:

- `0x19` och `19` är olika tal: tjugofem respektive nitton. Ett tal utan prefix är decimalt.
- Nollor till vänster ändrar inte värdet: `0x09` och `0x9` är samma tal. I kod skriver man ofta ut dem ändå, så att det syns hur många bitar talet är tänkt att ha.

Att kontrollera med kalkylatorn

Kalkylatorn i Windows har ett läge som heter **Programmerare**. Där visas samma tal samtidigt som `HEX`, `DEC`, `OCT` (oktalt, talbas 8, som inte används i boken) och `BIN`. Knappen där det står `QWORD`, under talen, väljer hur många bitar kalkylatorn räknar med; varje klick byter till nästa: `QWORD` (64), `DWORD` (32), `WORD` (16) och `BYTE` (8).

Använd den för att kontrollera dina omvandlingar, aldrig för att ta fram dem: på provet finns ingen kalkylator, och en omvandling du inte kan göra för hand kan du inte heller felsöka i ett program.

1.2 Binära koder

1.2.1 Vad en kod är

Ett binärt tal är ett sätt att tolka en rad bitar: som en summa av vikter. Men samma rad bitar kan tolkas på andra sätt, och vilket sätt som gäller är en överenskommelse mellan den som skriver och den som läser. En sådan överenskommelse kallas en **binär kod**.

Bitmönstret `0100 0001` är talet 65 om det tolkas som ett binärt tal, bokstaven `A` om det tolkas som ASCII, och talet 41 om det tolkas som BCD. Bitarna vet inte vilket de är. **Det är alltid sammanhanget, och den som skrev programmet, som bestämmer vad ett bitmönster betyder.**

Det här avsnittet går igenom de koder som förekommer i boken och som du kommer att möta i yrkeslivet:

Kod	Kodar	Används till exempel i
BCD	decimala siffror	displayer, klockkretsar, äldre styrsystem
Graykod	tal, så att bara en bit ändras åt gången	lägesgivare, Karnaughdiagram (kapitel 4)
ASCII	bokstäver, siffertecken och skiljetecken	all text i datorer, seriell kommunikation
7-segmentkod	vilka segment som ska lysa i en sifferdisplay	displayer på instrument och apparater
Paritetsbit	en extra bit som avslöjar ett överföringsfel	seriell kommunikation, minnen

1.2.2 BCD

BCD, *binary coded decimal*, kodar varje decimal siffra för sig med fyra bitar, med vikterna 8, 4, 2 och 1. Koden kallas därför också **8421-kod**.

Decimal siffra	0	1	2	3	4	5	6	7	8	9
BCD	`0000`	`0001`	`0010`	`0011`	`0100`	`0101`	`0110`	`0111`	`1000`	`1001`

Ett tal med flera siffror kodas siffra för siffra, fyra bitar per siffra:

```
59 (decimalt)  ->  5 = 0101, 9 = 1001  ->  0101 1001 (BCD)
```

Jämför med det binära talet 59, som är `0011 1011`. **BCD och binärt är olika koder**, och förväxlar man dem blir det fel: `0101 1001` tolkat som ett binärt tal är 89.

Fyra bitar har 16 kombinationer, men BCD använder bara tio av dem. Kombinationerna `1010` till `1111` är **ogiltiga** i BCD. Det är slöseri med bitar, men priset är värt det när talet ska visas: varje nibble är redan en decimal siffra, redo att skickas till sin egen display, utan någon omräkning. Därför används BCD i klockkretsar och sifferdisplayer. Att koden lämnar sex kombinationer oanvända återkommer i kapitel 4, där de blir *don't care* i ett Karnaughdiagram.

1.2.3 Graykod

Tänk dig en givare som mäter vinkeln på en axel med tre bitar, till exempel i en robotarm. När axeln vrids från läge 3 till läge 4 ska givarens utsignal gå från `011` till `100`. Men alla tre bitarna ändras, och de ändras aldrig på exakt samma gång. Under det korta ögonblick då bara några av dem har hunnit ändras läser styrsystemet något helt annat, kanske `111`, alltså läge 7. Axeln har inte rört sig mer än en bråkdel av ett steg, men systemet tror att den har gått halvvägs runt.

Graykoden löser det genom att ordna kombinationerna så att **exakt en bit ändras mellan två på varandra följande tal**, också från det sista talet tillbaka till det första:

Tal	Binärt	Graykod
0	`000`	`000`
1	`001`	`001`
2	`010`	`011`
3	`011`	`010`
4	`100`	`110`
5	`101`	`111`
6	`110`	`101`
7	`111`	`100`

Mellan läge 3 (`010`) och läge 4 (`110`) ändras nu bara den vänstra biten. Under övergången kan givaren alltså bara visa 3 eller 4, aldrig något orimligt.

Att bygga en Graykod

En Graykod med en bit fler byggs genom att **spegla** den man har:

1. Skriv koden med två bitar: `00`, `01`, `11`, `10`.
2. Skriv samma lista en gång till, i omvänd ordning: `10`, `11`, `01`, `00`.
3. Sätt en nolla framför den första halvan och en etta framför den andra.

Resultatet är tabellen ovan. Därför kallas koden också **reflekterad binärkod**.

Tvåbitarsversionen, `00`, `01`, `11`, `10`, är den du kommer att använda mest. Den är ordningen på rader och kolumner i ett Karnaughdiagram, där just egenskapen att grannar skiljer sig i en enda bit är hela poängen (kapitel 4).

1.2.4 ASCII

En dator lagrar text som tal: varje tecken har en kod. Den kod som nästan all text bygger på heter **ASCII**, *American Standard Code for Information Interchange*. Den använder 7 bitar och har alltså 128 tecken: bokstäver, siffertecken, skiljetecken och några styrtecken.

Ett urval:

Tecken	Hexadecimalt	Decimalt	Tecken	Hexadecimalt	Decimalt
mellanslag	`0x20`	32	`A`	`0x41`	65
`0`	`0x30`	48	`B`	`0x42`	66
`1`	`0x31`	49	`Z`	`0x5A`	90
`9`	`0x39`	57	`a`	`0x61`	97
`:`	`0x3A`	58	`b`	`0x62`	98
`?`	`0x3F`	63	`z`	`0x7A`	122

Styrtecknen ligger på `0x00`–`0x1F` och skrivs inte ut. De två du oftast möter är radmatning (`0x0A`) och vagnretur (`0x0D`), som avslutar en rad text.

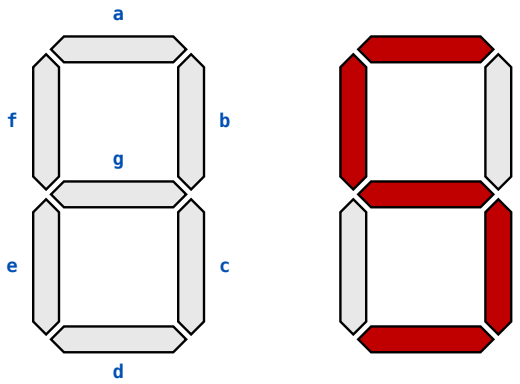
Tre saker i tabellen är värda att lägga på minnet:

- **Sifferteckнen `0`–`9` ligger på `0x30`–`0x39`.** Sifferтecknet `7` är alltså inte talet 7 utan talet `0x37` = 55. För att få talet ur tecknet drar man bort `0x30`, och för att få tecknet ur talet lägger man till `0x30`. Det är precis vad ett program gör när det ska visa ett tal som text, och det gör du själv i kapitel 10.
- **Bokstäverna ligger i ordning**, så `B` är ett mer än `A`. Det gör att ett program kan räkna fram bokstäver.
- **Stora och små bokstäver skiljer sig med `0x20`**, alltså bara i bit 5: `A` är `0100 0001` och `a` är `0110 0001`.

ASCII har inga svenska bokstäver. Å, ä och ö kom med senare koder som utökar ASCII, i dag nästan alltid Unicode i formatet UTF-8, där de första 128 tecknen är exakt ASCII.

1.2.5 7-segmentkod

En sifferdisplay på en mikrovågsugn, en multimeter eller en klocka består ofta av sju avlånga lysdioder, **segment**, som kallas a till g. Varje siffra visas genom att rätt segment tänds.



Sju segment, a till g, som tänds var för sig. Till höger siffran 5:
segmenten a, c, d, f och g tända, alltså koden gfedcba = 1101101.

Figur 1.3 En 7-segmentsdisplay med segmenten a till g, och samma display när segmenten a, c, d, f och g lyser och bildar siffran 5.

7-segmentkoden för en siffra är en bit per segment: `1` för tänt och `0` för släckt. Skrivs bitarna i ordningen g, f, e, d, c, b, a, med a som bit 0, blir koderna för siffrorna 0–9:

Siffra	Tända segment	g	f	e	d	c	b	a	Hexadecimalt
0	a b c d e f	0	1	1	1	1	1	1	0x3F
1	b c	0	0	0	0	1	1	0	0x06
2	a b d e g	1	0	1	1	0	1	1	0x5B
3	a b c d g	1	0	0	1	1	1	1	0x4F
4	b c f g	1	1	0	0	1	1	0	0x66
5	a c d f g	1	1	0	1	1	0	1	0x6D
6	a c d e f g	1	1	1	1	1	0	1	0x7D
7	a b c	0	0	0	0	1	1	1	0x07
8	a b c d e f g	1	1	1	1	1	1	1	0x7F
9	a b c d f g	1	1	0	1	1	1	1	0x6F

En krets som tar emot en BCD-siffra och tänder rätt segment kallas en **avkodare**, och den är ett kombinatoriskt nät: sju utgångar, en per segment, som var och en är en funktion av de fyra ingångsbitarna. Sådana nät är ämnet för kapitel 2 och kapitel 4.

Tabellen gäller en display där en etta tänder segmentet (gemensam katod). Det finns också displayer där en nolla tänder segmentet (gemensam anod), och då är varje bit inverterad. Vilken sort man har står i databladet.

1.2.6 Paritetsbit

När bitar skickas över en kabel kan en störning ibland vända en etta till en nolla eller tvärtom. Det enklaste sättet att upptäcka det är en **paritetsbit**: en extra bit som läggs till så att det totala antalet ettor alltid är jämnt.

Exempel: tecknet `A` är `100 0001` i ASCII. Det har två ettor, ett jämnt antal, så paritetsbiten blir `0`, och det som skickas är `0100 0001`. Tecknet `C` är `100 0011` med tre ettor, så paritetsbiten blir `1`, och det som skickas är `1100 0011`, med fyra ettor.

Mottagaren räknar ettorna. Är antalet udda har något gått fel, och tecknet kan begäras på nytt.

Paritetsbiten har en tydlig begränsning: **den upptäcker en felaktig bit, men inte två.** Vänds två bitar är antalet ettor jämnt igen, och felet passerar obemärkt. Den säger inte heller vilken bit som är fel. Den är ändå vanlig, eftersom den kostar en enda bit och fångar det vanligaste felet.

Att räkna ut en paritetsbit är samma sak som att använda XOR-grinden från kapitel 2 på alla bitar: XOR av ett jämnt antal ettor är `0`, och av ett udda antal `1`.

Det finns också **udda paritet**, där paritetsbiten väljs så att antalet ettor blir udda. Sändare och mottagare måste vara överens om vilken som gäller; det är ännu en överenskommelse av det slag som avsnitt 1.2.1 handlade om.

1.3 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Förklara skillnaden mellan en digital och en analog signal, och varför digitala signaler tål brus.
- Beskriva det decimala, det binära och det hexadecimala talsystemet: talbas, siffror och positionernas vikter.
- Omvandla tal mellan decimalt, binärt och hexadecimalt för hand, i alla riktningar.
- Ange hur många olika värden ett tal med n bitar kan anta, och vilket det största är.
- Addera två binära tal, och avgöra om summan ryms i åtta bitar.
- Beskriva BCD, Graykod, ASCII och 7-segmentkod, och koda och avkoda tal och tecken i dem.
- Beräkna en jämn paritetsbit, och förklara vilka fel den upptäcker.

Frågor att testa dig själv med

1. Varför har den minst signifikanta positionen vikten 1 i *alla* talsystem?
2. Hur ser du direkt på ett binärt tal om det är jämnt eller udda?
3. Varför används hexadecimala tal till att skriva binära tal, och inte decimala?

4. Talet `0x100` är ett större tal än `0xFF`, men bara med ett. Hur många bitar behövs för var och ett av dem?
5. Vad händer med ett binärt tal när du lägger till en nolla längst till höger? Jämför med vad som händer med ett decimalt tal.
6. Tecknet `'7'` och talet 7 är olika saker i en dator. Vad skiljer dem åt, och hur gör du om det ena till det andra?
7. I vilken kod ändras exakt en bit mellan två på varandra följande tal, och varför är det användbart?

Talsystemen används i resten av boken, och de sitter först när du har räknat många omvandlingar själv. Räkna hellre för många än för få. Kalkylatorn i Windows, i läget **Programmerare**, visar samma tal decimalt, binärt och hexadecimalt samtidigt. Använd den för att kontrollera dina svar, aldrig för att ta fram dem.

Nästa kapitel

Kapitel 2 handlar om logiska grindar och boolesk algebra:

- Logiska grindar: NOT, AND, OR, NAND, NOR, XOR och XNOR, och deras sanningstabeller.
- Boolesk algebra: notationen, räknelagarna och De Morgans lagar.
- Från sanningstabell till grindnät, och från grindnät tillbaka till sanningstabell.
- Samma logik byggd med strömkontakter på 24 V, inför Labb 1.

1.4 Övningar

Så kontrollerar du ditt arbete. Alla övningar här räknas med papper och penna. Räkna varje omvandling för hand först, och kontrollera den sedan genom att räkna tillbaka åt andra hållet: ett binärt tal du har räknat fram ur ett decimalt ska ge samma decimala tal när du summerar vikterna. Den kontrollen hittar nästan alla fel, och den finns alltid till hands, också på provet. Lösningsförslagen finns i bilaga I. Där visas också de vanligaste felaktiga svaren, och varför de är fel.
Varje övning är märkt med sin sort. Övning 1.13 är kapitlets **kontrolluppgift**: där räknar du för hand och kontrollerar med kalkylatorn i Windows.

Övning 1.1 Positionssystem

FÖRSTÅELSE

a) Talet 4072 är decimalt. Ange för varje siffra dess position, dess vikt och det värde den bidrar med.

b) I ett talsystem med talbasen 5 finns bara siffrorna 0–4. Vilken vikt har position 2 där? Vilket decimalt tal är 243_5?

c) Förklara med egna ord varför den högra positionen har vikten 1 oavsett talbas.

Övning 1.2 Från binärt till decimalt

RÄKNA FÖR HAND

Omvandla till decimalt. Använd viktraden 128, 64, 32, 16, 8, 4, 2, 1.

a) `1101`

b) `1 0000`

c) `0110 0100`

d) `1000 0001`

e) `1111 1111`

f) `1101 0110`

Övning 1.3 Från decimalt till binärt

RÄKNA FÖR HAND

Omvandla till binärt. Använd vikttabellen för a)–c) och division med 2 för d)–f), och kontrollera varje svar genom att räkna tillbaka.

a) 13

b) 37

c) 100

d) 200

e) 255

f) 1000

Övning 1.4 Binärt och hexadecimalt

RÄKNA FÖR HAND

Omvandla från binärt till hexadecimalt:

a) `1010 1111`

b) `0011 1100`

c) `1 1110`

d) `11 1110 1000`

Omvandla från hexadecimalt till binärt:

e) `0x7F`

f) `0xC3`

g) `0x2A5`

Övning 1.5 Hexadecimalt och decimalt

RÄKNA FÖR HAND

Omvandla från hexadecimalt till decimalt:

a) `0x2A`

b) `0xFF`

c) `0x100`

d) `0x3E8`

Omvandla från decimalt till hexadecimalt, med valfri metod:

e) 75

f) 160

g) 4095

Övning 1.6 Bitar och talområden

FÖRSTÅELSE

a) Hur många olika värden kan ett tal med 4, 8, 12 respektive 16 bitar anta, och vilket är det största?

b) Hur många bitar behövs minst för att kunna representera 1000 olika värden?

c) En nivågivare i en tank ska rapportera nivån i hela steg från 0 till 200. Hur många bitar behövs? Hur många av kombinationerna blir då oanvända?

d) Utan att räkna ut värdet: är `1011 0111` ett jämnt eller ett udda tal? Är det större eller mindre än 128? Motivera.

Övning 1.7 Binär addition

RÄKNA FÖR HAND

Addera de 8-bitars binära talen. Skriv ut minnessiffrorna, och kontrollera varje summa genom att göra om talen till decimalt.

a) `0011 1010 + 0001 0110`

b) `0111 1111 + 0000 0001`

c) `1100 1000 + 0110 0100`

d) Vilken av summorna i a)–c) ryms inte i åtta bitar? Vad blir kvar i ett 8-bitars register, och vad hände med resten?

Övning 1.8 BCD

RÄKNA FÖR HAND

a) Koda talet 47 i BCD. Koda det sedan som ett 8-bitars binärt tal, och jämför.

b) Koda årtalet 2026 i BCD. Hur många bitar behövs?

c) Vilket decimalt tal är `1001 0011` i BCD? Vilket tal är samma bitmönster som binärt tal?

d) Är `0110 1010` ett giltigt BCD-tal? Motivera.

Övning 1.9 Graykod

FÖRSTÅELSE

a) Bygg Graykoden med tre bitar genom att spegla tvåbitarskoden `00, 01, 11, 10`, enligt avsnitt 1.2.3.

b) Hur många bitar ändras när en binär räknare går från 7 till 0 (`111` till `000`)? Hur många ändras mellan motsvarande Graykoder?

c) En vinkelgivare med tre bitar står mellan läge 1 och läge 2. Förklara vad styrsystemet kan läsa under övergången om givaren använder vanlig binärkod, och vad det kan läsa om den använder Graykod.

Övning 1.10 ASCII

RÄKNA FÖR HAND

Använd tabellen i avsnitt 1.2.4.

a) Skriv texten `IC74` som ASCII-koder, hexadecimalt.

b) Vilken text är ASCII-koderna `0x48 0x65 0x6A`? (`e` ligger på `0x65`, och bokstäverna ligger i ordning.)

c) Hur mycket skiljer `a` och `A`? Vilken enda bit skiljer dem åt?

d) Ett program har läst in siffertecknet `7` från ett tangentbord. Hur får programmet fram talet 7 ur det? Och hur gör det om talet 3 till tecknet `3`?

Övning 1.11 7-segmentkod

KONSTRUKTION

Använd bitordningen g f e d c b a från avsnitt 1.2.5.

a) Vilka segment ska lysa för den hexadecimala siffran `E`? Ange koden binärt och hexadecimalt.

b) En display får koden `0x6D`. Vilken siffra visas?

c) Segment g har gått sönder och lyser aldrig. Vilka siffror går då inte längre att skilja från någon annan siffra? Vilka ser bara konstiga ut?

Övning 1.12 Paritetsbit

RÄKNA FÖR HAND

Använd jämn paritet: paritetsbiten väljs så att det totala antalet ettor blir jämnt, och den skickas som bit 7 framför de sju ASCII-bitarna.

a) Ange den byte som skickas för tecknen `B`, `C` och `G`.

b) En mottagare får byten `1100 0001`. Har ett fel inträffat? Kan mottagaren veta vilket tecken som skulle ha skickats?

c) Visa med ett eget exempel att två felaktiga bitar i samma byte inte upptäcks.

Övning 1.13 Kontroll: för hand, och sedan med kalkylatorn

KONTROLL

Räkna för hand, kontrollera med ett verktyg, och förklara det du ser.

Gör delarna i ordning, och skriv ned varje svar innan du går vidare. Poängen går förlorad om du tittar i kalkylatorn först.

a) För hand

Omvandla 173 till binärt och hexadecimalt, `0x3E8` till decimalt och binärt, och `0b1100 1010` till decimalt och hexadecimalt.

b) Med kalkylatorn

Öppna kalkylatorn i Windows och välj läget **Programmerare**. Skriv in vart och ett av talen i det talsystem det är givet i (klicka först på `DEC`, `HEX` eller `BIN`), och läs av de andra två. Stämmer dina svar? Om inte: hitta felet i din uträkning, inte bara rätt svar.

c) Något du inte kan förklara än

Välj `DEC`, skriv `-1` (tryck `1` och sedan `+/-`), och läs av `HEX` och `BIN`. Byt sedan ordlängd med knappen där det står `QWORD` tills det står `BYTE`, och läs av igen. Skriv ned vad du ser i båda fallen.

d)

Vilket positivt tal har samma bitmönster som det kalkylatorn visade för `-1` i läget `BYTE`? Gissa varför datorn skriver `-1` just så. Svaret, som heter **tvåkomplement**, kommer i kapitel 8.

Övning 1.14 Att lägga till en nolla

FÖRSTÅELSE FÖRDJUPNING

a) `0b1011` är 11. Vilket tal är `0b10110`, där en nolla har lagts till längst till höger? Och `0b101100`?

b) Formulera en regel: vad händer med värdet på ett binärt tal när alla bitar flyttas ett steg åt vänster och en nolla fylls på från höger?

c) Vad händer i stället med värdet om den högra biten tas bort och alla bitar flyttas ett steg åt höger? Pröva med `0b1011` och `0b1010`.

d) Vad motsvarar det att lägga till en nolla längst till höger i ett hexadecimalt tal? Pröva med `0x2A`.

Mikrodatorn i boken har instruktioner som gör exakt det här med ett register, och du kommer att använda dem i kapitel 8.

KAPITEL 2

Logiska grindar, boolesk algebra och kontaktnät

Hur ettor och nollor fattar beslut, med grindar och med kontakter.

I DET HÄR KAPITLET

- De logiska grindarna NOT, AND, OR, NAND, NOR, XOR och XNOR, deras sanningstabeller och symboler.
- Boolesk algebra: notationen, räknelagarna och De Morgans lagar.
- Från sanningstabell till uttryck, och från uttryck till grindnät.
- Analys: från ett färdigt grindnät till dess sanningstabell.
- CircuitVerse: att bygga och simulera ett grindnät i webbläsaren.
- Kontaktnät: samma logik byggd med tryckknappar, en lampa och 24 V, inför Labb 1.
- Reläet, och en krets som minns: självhållningen.

Kapitel 1 visade hur bitar bär tal och tecken. Det här kapitlet handlar om hur de fattar beslut: hur en utsignal bestäms av insignalerna, och hur en sådan funktion skrivs, förenklas och byggs. Kapitlet byggs kring ett enda exempel, bromsassistenten, som först blir ett grindnät och sedan ett kontaktnät.

Arbeta igenom kapitlet i ordning. Avsnitt 2.1 går igenom grindarna och den booleska algebran, och hur ett grindnät tas fram ur en sanningstabell och analyseras; avsnitt 2.1.1–2.1.6 är grunden för allt annat. När du kommer till bromsassistenten i avsnitt 2.1.10: läs kravet och försök ta fram ett uttryck för den själv, innan du läser härledningen. Grindnäten byggs och simuleras i CircuitVerse, som körs i webbläsaren utan att något behöver installeras (avsnitt 2.1.9).

Avsnitt 2.2 om kontaktnät är förberedelsen inför Labb 1 (bilaga A). Gör förberedelseuppgifterna i labbhandledningen innan labben; utan dem hinner du inte klart. Övningarna i avsnitt 2.4 har sina lösningsförslag i bilaga I.

2.1 Logiska grindar och boolesk algebra

2.1.1 Digitala signaler och logiska nivåer

Avsnitt 1.1.1 konstaterade att en digital signal bara har två värden, `0` och `1`. I en verklig krets är de två värdena två spänningsintervall:

- I en krets som matas med 5 V, som IC-kretsarna i Labb 2 (bilaga B) och mikrodatorn i Labb 3, läses en spänning nära 0 V som `0` och en spänning nära 5 V som `1`. Exakt var gränserna går står i kretsens datablad (kapitel 4).
- I ett kontaktnät på 24 V, som i Labb 1 (bilaga A), är en tryckknapp `1` när den är nedtryckt och `0` när den är släppt, och lampan är `1` när den lyser.

Det är en överenskommelse att hög spänning betyder `1`. Den kallas **positiv logik** och är den som används i hela boken.

En **logisk grind** är en krets med en eller flera ingångar och en utgång, där utgången är en fast funktion av ingångarna. Ett nät av grindar, där utgångarna bara beror på vad ingångarna är just nu, kallas ett **kombinatoriskt nät**. Det är ämnet för det här kapitlet och de tre följande. En krets som också minns vad som har hänt tidigare kallas ett **sekvensnät**; den typen får du en glimt av i avsnitt 2.2.7, och den är ämnet för kapitel 6.

2.1.2 Grindarna

Det finns sju grundgrindar. NOT har en ingång; de andra har två eller fler.

Grind	Utgången är `1` när ...	Uttryck
NOT (inverterare)	ingången är `0`	`X = A'`
AND	**alla** ingångar är `1`	`X = AB`
OR	**minst en** ingång är `1`	`X = A + B`
NAND	inte alla ingångar är `1` (inverterad AND)	`X = (AB)'`
NOR	ingen ingång är `1` (inverterad OR)	`X = (A + B)'`
XOR	ingångarna är **olika**	`X = A ⊕ B`
XNOR	ingångarna är **lika**	`X = (A ⊕ B)'`

Sanningstabellerna för alla grindar med två ingångar, sida vid sida:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

A	NOT A
0	1
1	0

Värt att lägga på minnet direkt ur tabellerna:

- NAND, NOR och XNOR är inverserna av AND, OR och XOR: varje kolumn är den andras med ettor och nollor bytta.
- AND och OR går att utvidga till fler ingångar. En AND med tre ingångar är 1 bara om alla tre är 1; en OR med tre ingångar är 1 om minst en är 1.
- XOR med fler än två ingångar är 1 när ett **udda** antal ingångar är 1. Det är samma sak som paritetsbiten i avsnitt 1.2.6.

Två sätt att rita samma grind

Grindarna ritas med två olika symbolstandarder, och du kommer att möta båda:

- **ANSI** (amerikansk standard) har en egen form för varje grind: AND är D-formad, OR har en böjd ingångssida. Den används i CircuitVerse och i de flesta datablad.
- **IEC** (internationell standard) ritar varje grind som en rektangel med en symbol i: `&` för AND, `≥1` för OR (minst en ingång ska vara 1), `=1` för XOR (exakt en ingång ska vara 1) och `1` för NOT. Den förekommer ofta i svenska läroböcker och kopplingsscheman.

I båda standarderna betyder en liten ring på utgången att utsignalen är inverterad (figur 2.1).

2.1.3 Boolesk algebra som notation

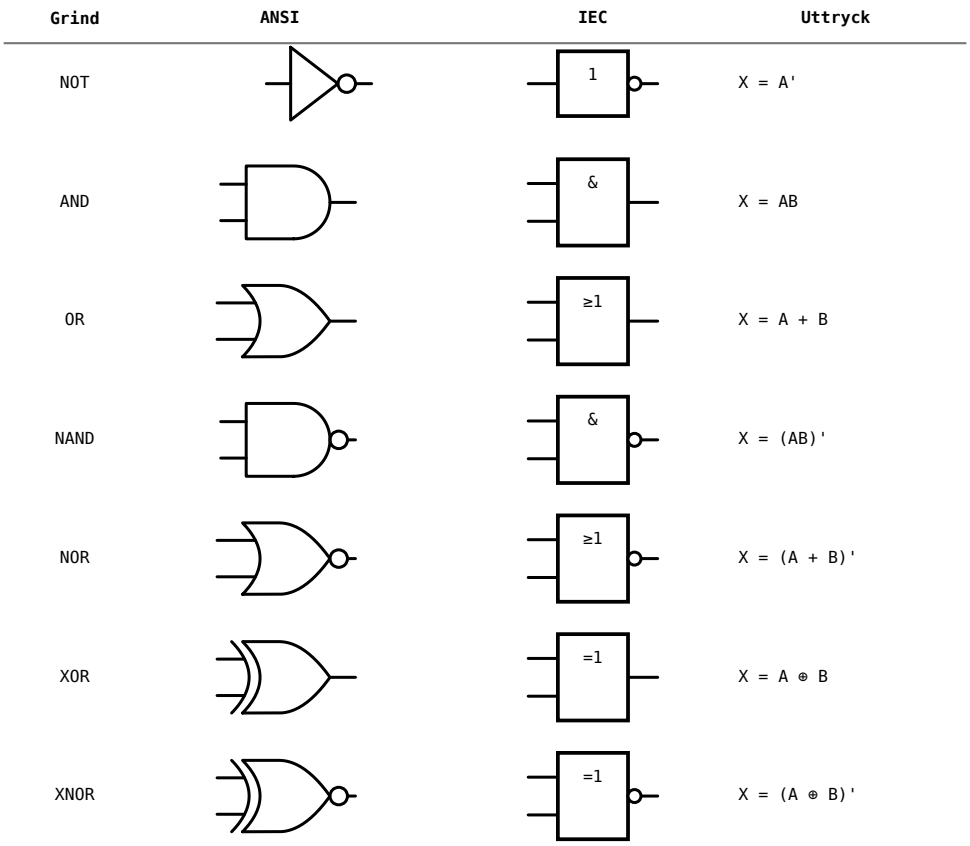
Att skriva ”A AND (NOT B) OR C” blir snabbt otympligt. **Boolesk algebra**, efter matematikern George Boole, ger en kort notation som ser ut som vanlig algebra:

Operation	Skrivs	Utläses
NOT	`A'`	”A inverterad”, ”icke A”
AND	`AB` eller `A·B`	”A och B”
OR	`A + B`	”A eller B”
XOR	`A ⊕ B`	”A exklusivt eller B”

Du kommer att se NOT skrivet på andra sätt också, framför allt som ett streck över bokstaven, $\overline{A}$. Boken använder primtecknet, `A'`, eftersom det går att skriva på ett tangentbord.

Prioritetsordningen är NOT först, sedan AND, sist OR, precis som potenser, multiplikation och addition i vanlig algebra:

- `AB + C` betyder `(AB) + C`.
- `A + BC` betyder `A + (BC)`, och det är **inte** samma sak som `(A + B)C`.
- `AB'` betyder `A(B')`: bara B är inverterad. `(AB)'` inverterar hela produkten.



Samma sju grindar i båda symbolstandarderna. CircuitVerse och de flesta datablad använder ANSI; svenska kopplingsscheman ofta IEC.

Figur 2.1 De sju grundgrindarna med ANSI-symbol, IEC-symbol och booleskt uttryck.

Sätt ut parenteser när du är osäker. De kostar ingenting.

Ett uttryck som är en OR av AND-termer, till exempel `X = AB' + CD`, sägs stå på **SP-form**, summa av produkter. Det är den form nästan alla uttryck i boken skrivs på, eftersom den läses av direkt ur en sanningstabell (avsnitt 2.1.6) och översätts direkt till ett grindnät: en AND-grind per term, och en OR-grind som samlar dem.

2.1.4 Räknelagarna

Lagarna nedan gäller för alla värden på A, B och C, och varje lag går att kontrollera genom att skriva ut sanningstabellen för båda sidorna. Många av dem ser ut som vanlig algebra; några gör det inte, och de är markerade.

Lag	Med OR	Med AND
Identitet	`A + 0 = A`	`A · 1 = A`
Dominans	`A + 1 = 1` (*inte som vanlig algebra*)	`A · 0 = 0`
Idempotens (*inte som vanlig algebra*)	`A + A = A`	`A · A = A`
Komplement	`A + A' = 1`	`A · A' = 0`
Dubbel invertering	`(A')' = A`	
Kommutativ	`A + B = B + A`	`AB = BA`
Associativ	`A + (B + C) = (A + B) + C`	`A(BC) = (AB)C`
Distributiv	`A(B + C) = AB + AC`	`A + BC = (A + B)(A + C)` (*inte som vanlig algebra*)
Absorption (*inte som vanlig algebra*)	`A + AB = A`	`A(A + B) = A`
Förenkling	`A + A'B = A + B`	`A(A' + B) = AB`

Några av lagarna blir självklara när man tänker på vad de betyder:

- `A + 1 = 1`: ”A eller sant” är alltid sant.
- `A + A' = 1`: en signal är alltid antingen `1` eller `0`, så ”A eller icke A” är alltid sant.
- `A + AB = A`: om A är `1` är uttrycket `1` oavsett B, och om A är `0` är båda termerna `0`. Termen `AB` tillför alltså ingenting.

Den sista raden, **förenklingslagen** `A + A'B = A + B`, är den som oftast behövs och oftast missas. Den går att härleda ur den andra distributiva lagen:

```
A + A'B = (A + A')(A + B) = 1 · (A + B) = A + B
```

Med ord: i termen `A'B` behövs inte villkoret `A'`, för när A är `1` är uttrycket redan `1`.

Att bevisa en lag med en sanningstabell

Varje lag kan kontrolleras genom att räkna ut båda sidorna för alla kombinationer. Förenklingslagen:

A	B	A′	A′B	A + A′B	A + B
0	0	1	0	0	0
0	1	1	1	1	1
1	0	0	0	1	1
1	1	0	0	1	1

De två sista kolumnerna är lika på varje rad, så lagen gäller. Med två variabler är det fyra rader och med tre är det åtta, så det finns ingen ursäkt för att inte kontrollera en förenkling man är osäker på.

2.1.5 De Morgans lagar

Två lagar är så viktiga att de har ett eget namn, efter matematikern Augustus De Morgan:

```
(A + B)' = A'B'
(AB)'    = A' + B'
```

Med ord: **invertera hela uttrycket genom att invertera varje variabel och byta AND mot OR** (och tvärtom). En minnesregel är ”bryt strecket, byt tecknet”.

Kontrollera den första med en tabell:

A	B	A + B	(A + B)′	A′	B′	A′B′
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Kolumnerna `(A + B)'` och `A'B'` är lika. Med ord: ”inte (A eller B)” är samma sak som ”varken A eller B”, alltså ”inte A och inte B”.

Lagarna gäller för fler variabler också:

```
(A + B + C)' = A'B'C'
(ABC)'       = A' + B' + C'
```

Vad De Morgan betyder för grindarna

De Morgan är mer än en räkneregel. Den säger att en grind kan bytas mot en annan grind med inverterade in- och utgångar:

- En **NAND**, `(AB)'`, är samma sak som en **OR med båda ingångarna inverterade**, `A' + B'`.
- En **NOR**, `(A + B)'`, är samma sak som en **AND med båda ingångarna inverterade**, `A'B'`.

Det har två viktiga följder. Den första är att **varje funktion går att bygga med enbart NAND-grindar**, eller med enbart NOR-grindar. En NAND med båda ingångarna ihopkopplade är en NOT, eftersom `(AA)' = A'`; en NAND följd av en NOT är en AND; och en OR är en NAND med inverterade ingångar. Det är därför NAND-kretsen är den vanligaste IC-kretsen av alla, och det är vad Labb 2-2 (avsnitt B.6) går ut på.

Den andra följden syns i kontaktnät: en NAND är två brytande kontakter parallellt, och en NOR två brytande kontakter i serie (avsnitt 2.2.5).

2.1.6 Från sanningstabell till uttryck

Ofta börjar ett konstruktionsarbete med en beskrivning av vad kretsen ska göra, och den beskrivningen blir en sanningstabell. Varje sanningstabell kan läsas av direkt som ett uttryck på SP-form:

1. Leta upp varje rad där utgången är `1`.
2. Skriv för varje sådan rad en AND-term med **alla** ingångar: ingången som den är om den är `1` på raden, inverterad om den är `0`.
3. Sätt ihop termerna med OR.

En sådan AND-term, med alla ingångar och som är `1` på exakt en rad, kallas en **minterm**.

Exempel:

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Raden `AB = 01` ger mintermen `A'B`, och raden `AB = 10` ger `AB'`. Alltså:

```
X = A'B + AB'
```

Det är XOR-funktionen. Samma nät går alltså att bygga av en enda XOR-grind, i stället för två AND, två NOT och en OR.

Exempel med tre ingångar, en **majoritetsgrind**: utgången är `1` när minst två av de tre ingångarna är `1`. Den används där tre givare mäter samma sak och systemet ska lita på det två av dem är överens om, så att en enstaka trasig givare inte kan bestämma något på egen hand.

A	B	C	X	Minterm
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	`A'BC`
1	0	0	0	
1	0	1	1	`AB'C`
1	1	0	1	`ABC'`
1	1	1	1	`ABC`

```
X = A'BC + AB'C + ABC' + ABC
```

Uttrycket är **korrekt**, men det är inte det **minsta**: fyra AND-grindar med tre ingångar och en OR-grind med fyra. Att förenkla det är nästa avsnitts ämne.

2.1.7 Algebraisk förenkling

Ett uttryck som läses av rakt ur en sanningstabell har en term per etta. Ofta delar flera termer det mesta av sina variabler, och då går de att slå ihop. Verktyget är nästan alltid samma: **bryt ut det gemensamma, och använd `A + A' = 1`**.

Exempel 1:

```
X = AB + AB' = A(B + B') = A · 1 = A
```

Värdet på B spelar ingen roll, så B försvinner.

Exempel 2: en sanningstabell där X är `1` på raderna 001, 011, 101 och 111.

```
X = A'B'C + A'BC + AB'C + ABC
  = A'C(B' + B) + AC(B' + B)
  = A'C + AC
  = C(A' + A)
  = C
```

Fyra termer blev en enda variabel. Titta på tabellen igen: X är `1` precis när C är `1`. Det hade gått att se direkt, men uträkningen visar varför.

Exempel 3: majoritetsgrinden från avsnitt 2.1.6. Här behövs ett knep: termen `ABC` kan användas flera gånger, eftersom `ABC + ABC = ABC` (idempotens). Skriv den tre gånger, och para ihop den med var och en av de andra termerna:

```
X = A'BC + AB'C + ABC' + ABC
  = (A'BC + ABC) + (AB'C + ABC) + (ABC' + ABC)
  = BC(A' + A)  + AC(B' + B)  + AB(C' + C)
  = BC + AC + AB
```

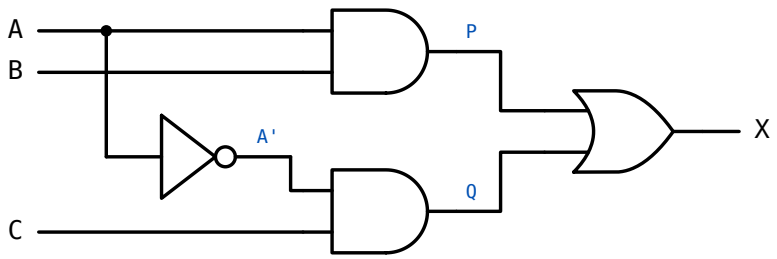
Tre AND-grindar med två ingångar och en OR-grind med tre, i stället för fyra AND-grindar med tre ingångar.

Exempel 4: inte allt går att förenkla. `X = A'B + AB'` har inga gemensamma variabler att bryta ut. Uttrycket är redan minimalt på SP-form, och det bästa man kan göra är att känna igen det som en XOR.

Algebraisk förenkling kräver att man ser vilka termer som går att para ihop, och det är lätt att missa ett par, eller att inte veta när man är klar. Kapitel 4 introducerar **Karnaughdiagram**, som gör samma sak med en ritning, systematiskt, och med ett tydligt svar på när man är klar.

2.1.8 Analys: från grindnät till sanningstabell

Åt andra hållet: du har ett färdigt grindnät och ska ta reda på vad det gör. Metoden är att ge varje grinds utgång ett namn, en **mellansignal**, och räkna fram dem en i taget, rad för rad.




```
Namnge varje mellansignal, räkna ut dem rad för rad,
                och läs sedan av X.
```

Figur 2.2 Ett grindnät att analysera, med mellansignalerna A′, P och Q.

Mellansignalerna är `A'`, `P = AB` och `Q = A'C`, och utgången är `X = P + Q`. Uttrycket för hela nätet blir alltså

```
X = AB + A'C
```

och sanningstabellen räknas fram kolumn för kolumn:

A	B	C	A′	P = AB	Q = A′C	X = P + Q
0	0	0	1	0	0	0
0	0	1	1	0	1	1
0	1	0	1	0	0	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	0	1	0	1
1	1	1	0	1	0	1

Läs tabellen i två halvor. När A är `0` är X samma som C; när A är `1` är X samma som B. Nätet är alltså en **väljare**: A väljer vilken av B och C som ska släppas fram till utgången. En sådan krets kallas en **multiplexer**, och den är ett av de viktigaste byggblocken i en dator. Den återkommer i övning 4.13.

Två råd som gäller för all analys:

- **Skriv ut mellansignalerna som egna kolumner.** Att räkna fram X direkt ur A, B och C i huvudet fungerar för små nät och ger fel i större.
- **Sammanfatta sedan tabellen med ord.** En tabell säger vad kretsen gör rad för rad; en mening som ”A väljer mellan B och C” säger vad den är till för.

2.1.9 CircuitVerse

CircuitVerse, `https://circuitverse.org/simulator`, är en gratis logiksimulator som körs i webbläsaren. Varje grindnät i boken byggs och testas där innan det byggs med riktiga kretsar:

1. Öppna simulatorn och starta ett nytt projekt.
2. Placera ett **Input**-element per insignal och ett **Output**-element per utsignal, och döp om dem så att de heter som dina variabler.
3. Placera de grindar ditt uttryck kräver, från panelen **Gates**.
4. Koppla ihop nätet en grind i taget, enligt uttrycket. Du ska redan veta exakt vilka grindar du behöver innan du öppnar simulatorn, och det är därför avsnitt 2.1.6 och 2.1.7 kommer först.
5. Klicka på varje ingång för att växla den mellan `0` och `1`, och kontrollera att utgången stämmer med din sanningstabell på **varje** rad.
6. Spara under menyn **Project**, antingen online med ett konto eller som en fil på datorn.

Steg 5 är inte valfritt, och ingenting annat gör det åt dig. Tre regler gör det till en kontroll snarare än en formalitet:

- **Alla rader, inte ett urval.** Två ingångar är fyra rader och tre är åtta. Vid de storlekarna finns ingen ursäkt för att bara prova några.

- **Förutsäg först, simulera sedan.** Skriv sanningstabellen innan du bygger. Annars läser du av simuleringen och tror att du har kontrollerat den, när du bara har skrivit av den.
- **En avvikelse är information.** Den säger att ritningen och uttrycket är oense, och att en av dem har ett fel.

Den vanan, att förutsäga ett beteende och sedan kontrollera det, är den du kommer att använda vid varje laboration i boken, med kontakter, med IC-kretsar och med mikrodatorn.

2.1.10 Kapitlets krets: bromsassistenten

Kapitlet bygger en krets från ett krav till ett simulerat grindnät, och i avsnitt 2.2.6 samma krets som kontaktnät.

Kravet: **bilen bromsar om föraren bromsar, eller om assistanssystemet bestämmer sig för det.** Assistanssystemet bestämmer sig för det när endera av två detektorer ser ett hinder, om det inte samtidigt rapporterar ett fel, i vilket fall det inte anförtros att bromsa alls.

Signal	Betydelse
`D`	Föraren står på bromspedalen.
`S`	Avståndssensorn ser ett hinder.
`R`	Radarn ser ett hinder.
`F`	Assistanssystemet rapporterar ett fel.
`B`	Utgång: lägg an bromsarna.

Tre meningar i kravet bestämmer tillsammans hela strukturen:

- Endera detektorn räcker på egen hand, och ingen av dem väger tyngre än den andra.
- Ett fel utlöser inget larm; det *hindrar* assistanssystemet från att bromsa.
- Föraren kan alltid bromsa, även när assistanssystemet är trasigt.

Den sista är ett säkerhetskrav, och det är den att hålla fast vid: pedalen är inte en ingång till assistanslogiken, den går förbi den.

Sanningstabellen

Sexton rader, i den ordning en fyrabitars räknare skulle producera dem. Kolumnen `P` är ingen utgång; den är assistanssystemets eget beslut, visad för att det ska synas var strukturen kommer ifrån.

D	S	R	F	P	B
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	0	0
1	0	0	0	0	1
1	0	0	1	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	1	0	1	1
1	1	1	1	0	1

Den nedre halvan är genomgående `1`: så snart D är `1` ändrar ingenting annat svaret. Det är hur ”åsidosätter allt” ser ut i en tabell.

> **Försök själv först.** Innan du läser vidare: läs av uttrycket på SP-form, förenkla det, och räkna grindarna. Det raka uttrycket har elva termer; det förenklade får plats på en rad.

Härledningen

Att läsa av tabellen rakt av ger elva mintermer, en per etta, var och en med fyra variabler. Korrekt, men ohanterligt. Dela i stället upp tabellen efter D.

Den nedre halvan (`D = 1`) är `1` på alla åtta raderna. De åtta mintermerna tillsammans täcker alla kombinationer av S, R och F, så de förenklas till bara `D`, precis som i exempel 2 i avsnitt 2.1.7.

Den övre halvan (`D = 0`) är `1` på tre rader: SRF = 010, 100 och 110, alltså när F är `0` och minst en av S och R är `1`:

```
D'S'RF' + D'SR'F' + D'SRF' = D'F'(S'R + SR' + SR) = D'F'(S + R)
```

(`S'R + SR' + SR` är `1` när minst en av S och R är `1`, alltså `S + R`; övning 2.6 c) låter dig visa det algebraiskt.)

Tillsammans:

```
B = D + D'(S + R)F'
```

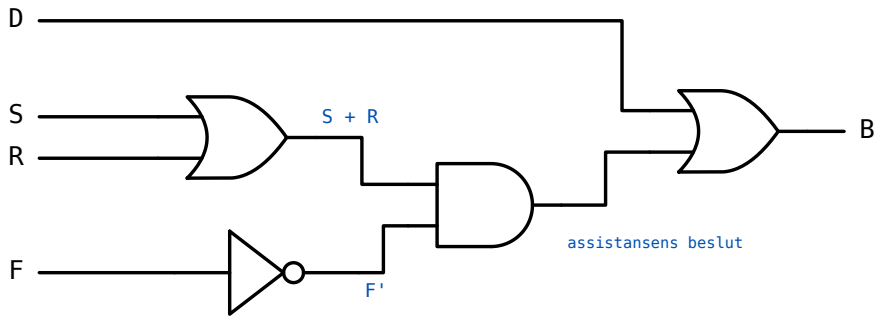
och förenklingslagen `A + A'X = A + X` från avsnitt 2.1.4 tar bort `D'`:

```
B = D + (S + R)F'
```

Uttrycket läses som kravet: bromsa om föraren bromsar, eller om (sensorn eller radarn) ser något och det inte finns något fel.

Grindnätet

Fyra grindar: en OR för `S + R`, en NOT för `F'`, en AND för assistanssystemets beslut, och en OR som lägger till förarens pedal.




```
B = D + (S + R)F'. Förarens broms D går förbi hela assistanslogiken
                   och läggs in i den sista OR-grinden.
```

Figur 2.3 Bromsassistentens grindnät.

Säkerhetsargumentet vilar på den sista OR-grinden. Förarens pedal går direkt in där, förbi allt som har med felet att göra. Ett nät som i stället matar in pedalen före fellogiken, `B = (D + S + R)F'`, är en grind enklare och ser rimligt ut, men det bromsar inte när föraren trampar på pedalen samtidigt som assistanssystemet rapporterar fel. Övning 2.13 låter dig hitta exakt vilka rader som går fel.

Bygg nätet i CircuitVerse och kontrollera det mot alla sexton raderna i tabellen.

2.2 Kontaktnät

2.2.1 Kontakter och symboler

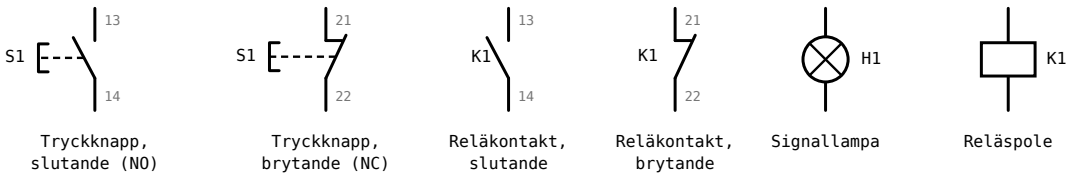
Långt innan det fanns IC-kretsar byggdes styrlogik av kontakter: tryckknappar, gränslägesbrytare och reläer, kopplade så att en lampa, en motor eller en ventil fick ström precis när den skulle. Så fungerar fortfarande en stor del av säkerhetskretsarna i industrin, och det är samma logik som en PLC programmeras med i dag (avsnitt 2.2.8).

En **kontakt** har två lägen, sluten eller öppen, och är därför en logisk variabel. Det finns två sorter:

- En **slutande kontakt** (NO, *normally open*) är öppen i vila och sluts när den påverkas. Tryck på knappen, så leder den.
- En **brytande kontakt** (NC, *normally closed*) är sluten i vila och bryts när den påverkas. Tryck på knappen, så slutar den leda.

En tryckknapp innehåller ofta flera **kontaktelement** som påverkas samtidigt, till exempel ett slutande och ett brytande. Kontaktelementens anslutningar har nummer enligt en standard: den sista siffran är **3–4 för ett slutande** element och **1–2 för ett brytande**, och den första siffran är elementets platsnummer. Ett slutande element kan alltså heta 13–14 och ett brytande 21–22.

Kontakterna ritas i ett **strömvägsschema**: vertikala strömvägar mellan en matningsskena upptill (+24 V) och en återledare nedtill (0 V), med kontakterna ritade vertikalt i vägen och det som ska styras, en lampa eller en reläspole, längst ned.



Figur 2.4 Symbolerna för tryckknappar, reläkontakter, signallampa och reläspole, med anslutningsnummer.

Symbolerna ritas alltid i **viloläge**: den slutande kontakten öppen, den brytande sluten, som när ingen rör något. Den streckade linjen till vänster är tryckknappens påverkansdon; en reläkontakt har ingen sådan, eftersom den påverkas av sin spole. Varje komponent har en beteckning: S för tryckknappar och andra manöverdon, H för lampor och K för reläer.

Kontakten som logisk variabel

Översättningen till logik är:

- en **nedtryckt knapp** är `1`, en släppt knapp `0`;
- en **tänd lampa** är `1`, en släckt lampa `0`;
- **lampan lyser när det finns en sluten väg från +24 V till 0 V.**

Den sista punkten är hela bron mellan kontakter och logik. Allt i resten av det här avsnittet följer av den.

2.2.2 Seriekoppling = AND

Två slutande kontakter efter varandra i samma strömväg. Strömmen måste passera **båda**, så lampan lyser bara om S1 **och** S2 är nedtryckta:

```
H1 = S1 · S2
```

S1	S2	H1
0	0	0
0	1	0
1	0	0
1	1	1

Det är AND-grindens sanningstabell. Ett vardagsexempel är **tvåhandsmanövern** på en press: den går bara om båda knapparna trycks samtidigt, så att båda händerna garanterat är utanför.

2.2.3 Parallellkoppling = OR

Två slutande kontakter bredvid varandra, var och en en egen väg mellan samma två punkter. Det räcker att **en** av vägarna är sluten, så lampan lyser om S1 **eller** S2 är nedtryckt, eller båda:

```
H1 = S1 + S2
```

S1	S2	H1
0	0	0
0	1	1
1	0	1
1	1	1

Det är OR-grindens sanningstabell. Ett vardagsexempel är en ringklocka med en knapp vid framdörren och en vid bakdörren: vilken som helst av dem får den att ringa.

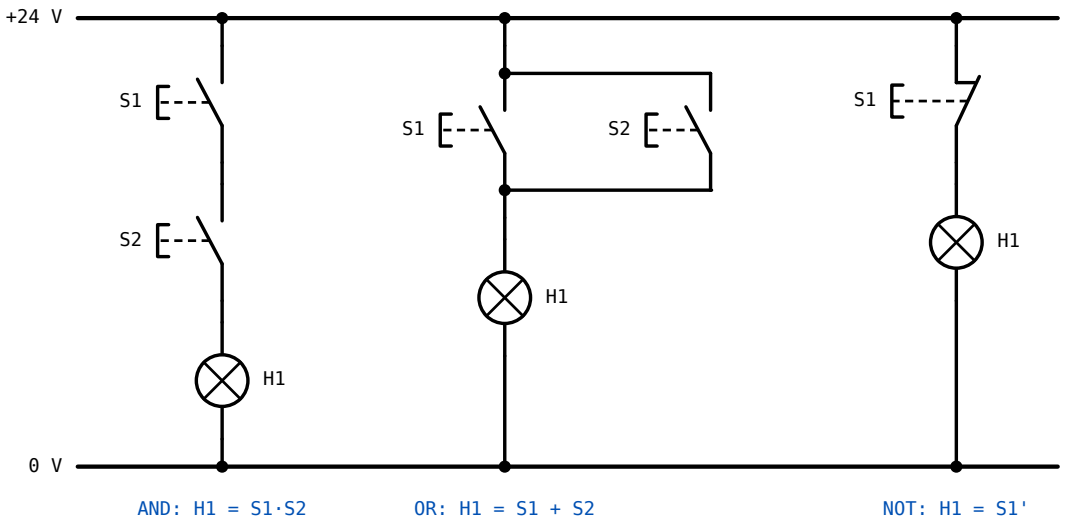
2.2.4 Brytande kontakt = NOT

En brytande kontakt ensam i strömvägen. Lampan lyser **när knappen inte är nedtryckt**, och slocknar när den trycks:

```
H1 = S1'
```

Det är NOT. Ett vardagsexempel är en stoppknapp: den ska bryta, inte sluta, en krets, och varför den ska vara brytande just av säkerhetsskäl återkommer i avsnitt 2.2.7.

Här är de tre grundkopplingarna i samma figur:



Figur 2.5 AND, OR och NOT som strömvägar mellan +24 V och 0 V.

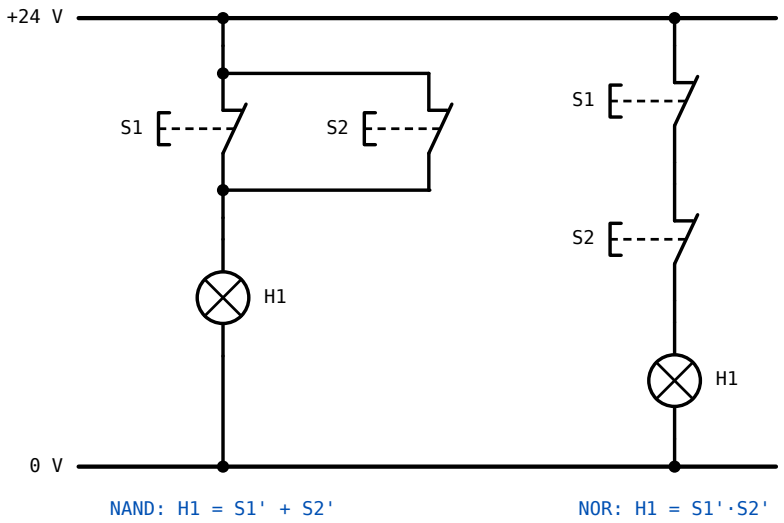
2.2.5 Sammansatta kontaktnät

Med serie, parallell och brytande kontakter går det att bygga varje logisk funktion.

NAND och NOR

De Morgans lagar från avsnitt 2.1.5 visar hur:

- NAND: `(S1 · S2)' = S1' + S2'`. Två **brytande** kontakter **parallellt**: lampan slocknar bara om båda vägarna bryts, alltså om båda knapparna trycks.
- NOR: `(S1 + S2)' = S1' · S2'`. Två **brytande** kontakter **i serie**: lampan slocknar så snart någon av knapparna trycks.



Figur 2.6 NAND som två brytande kontakter parallellt, och NOR som två brytande kontakter i serie.

Det är De Morgan uttryckt i koppar: inverterade ingångar (brytande kontakter) och bytt operation (parallell i stället för serie).

XOR och XNOR: trappkopplingen

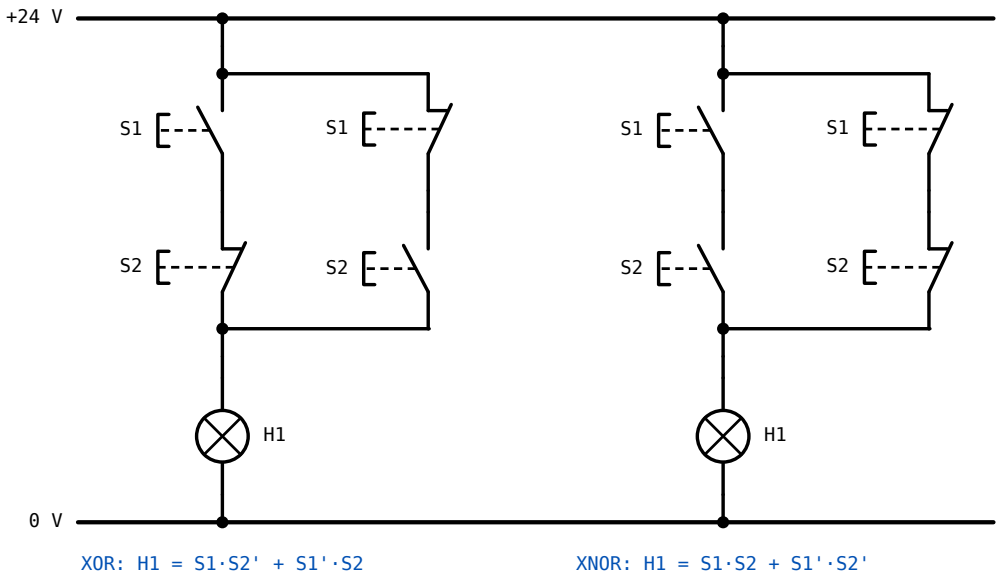
En lampa i en trappa ska kunna tändas och släckas från både övervåningen och nedervåningen. Varje tryck på vilken knapp som helst ska ändra lampans läge. Med två knappar är det XOR:

```
H1 = S1 · S2' + S1' · S2
```

Varje term är en strömväg med två kontakter i serie, och de två vägarna ligger parallellt (figur 2.7). Termen `S1 · S2'` behöver S1:s **slutande** kontakt och S2:s **brytande**; termen `S1' · S2` tvärtom. **Varje knapp används alltså med båda sina kontaktelement**, och det är skälet till att labbstationens knappar har både slutande och brytande kontakter.

Kontakterna med samma beteckning, till exempel de två som heter S1, sitter i samma knapp och påverkas samtidigt. I schemat ritas de där de gör nytta i strömvägen, inte bredvid varandra.

(En riktig trappbelysning använder vippströmbrytare, som stannar i det läge man lämnar dem, i stället för tryckknappar. Logiken är densamma.)



Figur 2.7 XOR och XNOR som kontaktnät.

2.2.6 Från uttryck till kontaktnät och tillbaka

Översättningen går åt båda hållen med fyra regler:

Uttryck	Kontaktnät
en variabel `S1`	en slutande kontakt S1
en inverterad variabel `S1'`	en brytande kontakt S1
en produkt, `AB`	kontakterna i **serie**
en summa, `A + B`	kontakterna **parallellt**

Parenteser blir grupper: `(S2 + S3) · S4'` är en parallellkoppling av S2 och S3, i serie med en brytande kontakt S4.

Från uttryck till nät: börja med den yttersta operationen och arbeta dig inåt. Är uttrycket en summa av termer, rita en parallell gren per term; är det en produkt, rita faktorerna i serie.

Från nät till uttryck: börja med de innersta grupperna, de minsta serie- och parallellkopplingarna, och arbeta dig utåt. Kapitel 3 går igenom det i detalj.

Bromsassistenten som kontaktnät

Uttrycket från avsnitt 2.1.10, `B = D + (S + R)F'`, blir med tryckknappar i stället för signaler (S1 för D, S2 för S, S3 för R och S4 för F) och en lampa H1 som bromsljus:

```
H1 = S1 + (S2 + S3) · S4'
```

Den yttersta operationen är en summa: två parallella grenar. Den vänstra är S1 ensam. Den högra är en produkt: parallellkopplingen av S2 och S3, i serie med S4:s brytande kontakt.

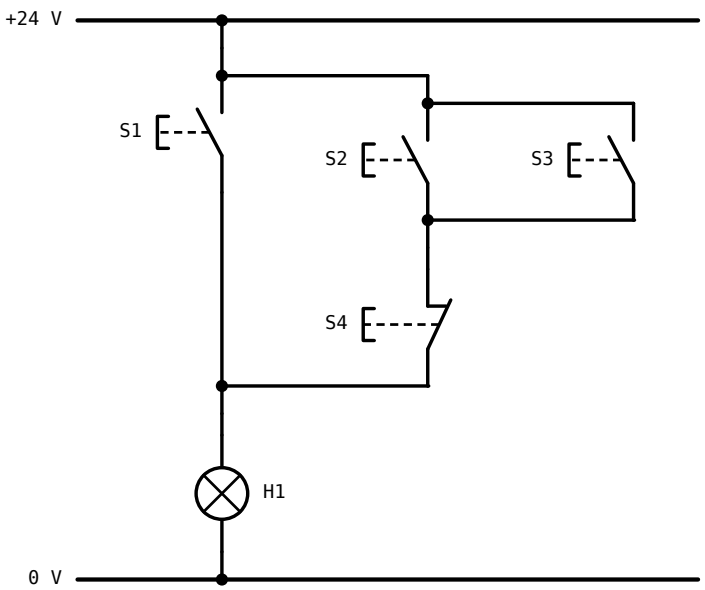
Säkerhetsargumentet syns direkt i ritningen (figur 2.8): förarens knapp S1 har en egen väg till lampan, och felsignalen S4 kan bara bryta assistanssystemets gren. Ingen kombination av de andra knapparna kan hindra S1 från att tända H1.

2.2.7 Reläet

Ett **relä** är en elektriskt styrd strömbrytare: en **spole** som, när den får ström, drar till sig ett ankare som påverkar reläets **kontakter**. En reläkontakt kan vara slutande eller brytande, precis som en tryckknapps. Spolen och kontakterna har samma beteckning, till exempel K1, men ritas där de hör hemma i schemat: spolen längst ned i en strömväg, kontakterna i andra strömvägar.

Två egenskaper gör reläet användbart:

- **En liten ström styr en stor.** Spolen kan drivas av en svag signal, till exempel från en mikrodator, medan kontakterna kan koppla en motor eller en lampa på 24 V eller 230 V. Det återkommer i kapitel 11, när mikrodatorn ska styra något i verkligheten.
- **En signal kan användas många gånger.** Ett relä kan ha flera kontakter, så en logisk variabel, ”K1 har dragit”, kan användas i så många strömvägar som behövs, slutande i några och brytande i andra.

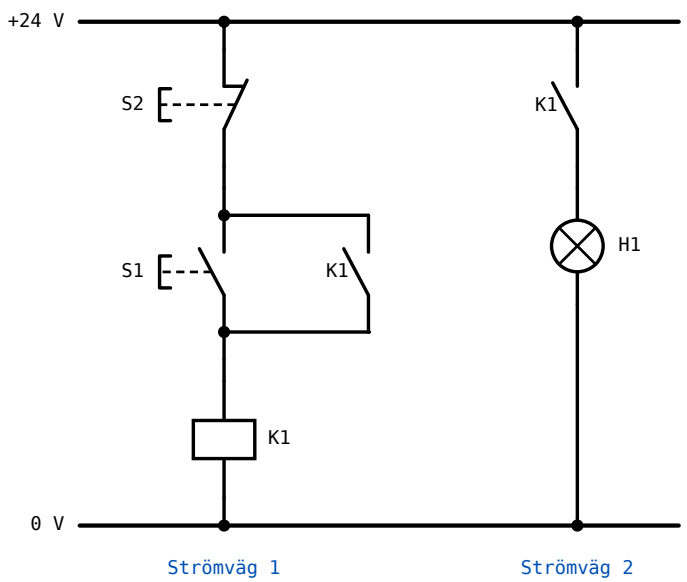


Bromsassistenten som kontaktnät: S1 förarens pedal, S2 sensorn, S3 radarn, S4 felsignalen. H1 = S1 + (S2 + S3)·S4'.

Figur 2.8 Bromsassistenten som kontaktnät.

Självhållning: en krets som minns

Den viktigaste reläkopplingen av alla är **självhållningen**, som finns i nästan varje maskin med en start- och en stoppknapp:




```
S1 = start, S2 = stopp. När K1 har dragit håller dess egen kontakt spolen
  strömsatt, även efter att S1 släppts. Kretsen minns att den startades.
```

Figur 2.9 Självhållning med startknappen S1, stoppknappen S2 och reläet K1.

Följ vad som händer:

1. I vila är S1 och reläkontakten K1 öppna. Spolen får ingen ström, och lampan är släckt.
2. **Tryck på start, S1.** Strömväg 1 sluts genom S2 (brytande, alltså sluten) och S1. Spolen K1 får ström och drar. Då sluts båda K1-kontakterna: lampan tänds, och K1-kontakten parallellt med S1 sluts.
3. **Släpp S1.** Strömmen går nu genom K1:s egen kontakt i stället, så spolen håller sig själv strömsatt. Lampan fortsätter lysa.
4. **Tryck på stopp, S2.** Den brytande kontakten bryter strömväg 1, spolen släpper, och båda K1-kontakterna öppnar. När S2 släpps är S1 och K1 öppna igen, och kretsen är tillbaka i vila.

Lägg märke till vad som hände i steg 3: **ingångarna är desamma som i steg 1**, båda knapparna släppta, men lampan lyser. Utgången beror inte bara på ingångarna nu, utan också på vad som har hänt tidigare. Kretsen **minns** att den startades.

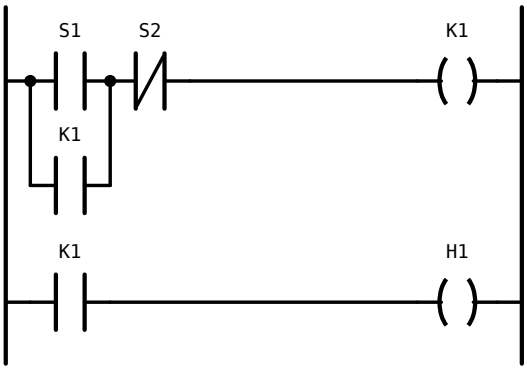
Det är inte ett kombinatoriskt nät längre. Ett nät där en utgång återkopplas till sin egen ingång, och som därför har minne, kallas ett **sekvensnät**, och det är grunden för allt minne i en dator, från ett enda register till hela arbetsminnet. Kapitel 6 bygger samma sak med grindar i stället för reläer.

Stoppknappen är brytande av ett säkerhetsskäl: går en ledning till den av, bryts kretsen och maskinen stannar. Med en slutande stoppknapp skulle en avbruten ledning i stället göra det omöjligt att stoppa maskinen.

2.2.8 Kontaktnät och PLC:ns stegdiagram

I dag byggs styrlogik sällan av reläer, utan programmeras i en **PLC**, en industridator gjord för att styra maskiner. Det vanligaste språket för att programmera en PLC, **stegdiagram** (LD, *ladder diagram*), är ritat för att se ut som ett kontaktnät vänt på sidan:

- strömvägarna ligger ned, som stegpinnar mellan två lodräta skenor;
- en slutande kontakt ritas `-| |-` och en brytande `-|/|-`;
- utgången, en spole eller en lampa, ritas `-( )-` längst till höger.



Samma självhållning som stegdiagram i ett PLC-program: strömvägarna ligger ned, kontakterna ritas -| |- och -|/|-, utgångarna -( )-.

Figur 2.10 Självhållningen som stegdiagram.

Allt i det här avsnittet gäller oförändrat: serie är AND, parallell är OR, en brytande kontakt är NOT, och självhållningen fungerar på samma sätt. Den som kan läsa ett kontaktnät kan läsa ett stegdiagram.

2.2.9 Förbered Labb 1

I Labb 1 (bilaga A) bygger du kontaktnät med två tryckknappar, en lampa och ett aggregat på 24 V. Innan labben ska du ha gjort förberedelseuppgifterna F1–F4 i labbhandledningen. De går ut på det som det här avsnittet har visat:

- skriva sanningstabeller för AND, OR, NOT, NAND, NOR, XOR och XNOR med två knappar;
- rita strömvägsscheman för dem, med rätt kontaktsort för varje variabel;
- ta fram ett uttryck ur en sanningstabell, och förenkla det;
- analysera ett färdigt kontaktnät, som i avsnitt 2.2.6.

Rita alltid schemat innan du kopplar. Ett schema på papper går att kontrollera mot sanningstabellen på en minut. En felkopplad sladd på stationen tar mycket längre tid att hitta.

2.3 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Rita symbolen för varje grundgrind, i ANSI- och i IEC-form, och ange dess sanningstabell och uttryck.
- Skriva ett booleskt uttryck med bokens notation, och förenkla det med räknelagarna och De Morgans lagar.
- Ta fram ett uttryck i SP-form ur en sanningstabell, och realisera det som ett grindnät.
- Analysera ett grindnät: ta fram dess uttryck och sanningstabell med hjälp av mellansignaler.
- Bygga och simulera ett grindnät i CircuitVerse, och kontrollera det mot en sanningstabell.
- Översätta mellan ett uttryck och ett kontaktnät: serie är AND, parallell är OR, brytande kontakt är NOT.
- Förklara hur ett relä med självhållning kan minnas att det har startats.

Frågor att testa dig själv med

1. Vilken grind ger `1` bara när alla ingångar är `1`? Vilken ger `0` bara när alla ingångar är `0`?
2. Varför är `A + A'B` lika med `A + B`? Visa det både med en sanningstabell och med ord.
3. Hur flyttar De Morgan en inversion genom en grind, och varför betyder det att en NAND-grind kan bygga vilken funktion som helst?
4. I bromsassistenten läggs förarens pedal in med OR sist, i stället för att gå genom felhanteringslogiken. Vad går sönder om du kopplar den tvärtom?

5. Varför behöver en XOR-funktion med tryckknappar både den slutande och den brytande kontakten i varje knapp?
6. Vad händer i självhållningskretsen om stoppknappen av misstag kopplas med sin slutande kontakt i stället för sin brytande?

CircuitVerse, `https://circuitverse.org/simulator`, är simulatorn som grindnäten byggs och testas i, i det här kapitlet och fram till och med kapitel 6. Innan Labb 1: läs avsnitt 2.2 om kontaktnät, och gör förberedelseuppgifterna i bilaga A.

Nästa kapitel

Kapitel 3 hör ihop med Labb 1, kopplingsnät:

- **Labb 1 – Kopplingsnät**: logiska funktioner byggda med två tryckknappar och en lampa på 24 V.
- Att analysera ett kontaktnät man inte har ritat själv, och att hitta felet när lampan inte gör som den ska.
- Förberedelseuppgifterna i labbhandledningen ska vara klara när du börjar labben.

2.4 Övningar

Så kontrollerar du ditt arbete. Varje uttryck du förenklar kan kontrolleras med en sanningstabell: räkna ut det ursprungliga och det förenklade uttrycket för alla kombinationer av ingångarna, och jämför kolumnerna. Är de lika på varje rad är förenklingen rätt, oavsett vilken väg du tog dit. Varje grindnät kan dessutom byggas i CircuitVerse och kontrolleras rad för rad, enligt avsnitt 2.1.9.
Lösningsförslagen finns i bilaga I.
Varje övning är märkt med sin sort. Övning 2.7 är kapitlets **kontrolluppgift**.

Övning 2.1 Grindar med tre ingångar FÖRSTÅELSE

a) Skriv sanningstabellen för en AND-grind, en OR-grind och en XOR-grind med tre ingångar A, B och C, i samma tabell.

b) Beskriv med en mening var för varje grind när utgången är 1.

c) På hur många av de åtta raderna är AND-grinden 1? OR-grinden? Förklara varför svaren är så olika.

Övning 2.2 Från ord till uttryck

FÖRSTÅELSE

Skriv ett booleskt uttryck för varje beskrivning. Välj själv bokstäver för signalerna, och ange vad de betyder.

a) Larmet ska ljuda om dörren är öppen och larmet är aktiverat.

b) Pumpen ska gå om nivån i tanken är låg och ingen har tryckt på det manuella stoppet.

c) Varningslampan ska lysa om minst en av tre dörrar är öppen.

d) Beskriv med ord vad `X = AB' + C` betyder, om A är ”maskinen är påslagen”, B är ”skyddet är på plats” och C är ”nödlarmet är utlöst”.

e) Beräkna `AB + C` och `A(B + C)` för $A = 0$, $B = 0$ och $C = 1$. Vad visar resultatet om prioritetsordningen?

Övning 2.3 Räknelagarna

RÄKNA FÖR HAND

Förenkla med lagarna i avsnitt 2.1.4, och ange vilken lag du använder.

a) `A + A`

b) `A · A'`

c) `A + 1`

d) `(A')'`

e) `A + AB`

f) `A(A + B)`

g) `A + A'B`

h) `AB + A'B`

Övning 2.4 De Morgan

RÄKNA FÖR HAND

a) Skriv om `(AB)'` utan någon inversion av ett helt uttryck.

b) Skriv om `(A + B + C)'` på samma sätt.

c) Förenkla `(A'B)'`. Tänk på att `(A')' = A`.

d) Förenkla `(A + B')'`.

e) Visa med en sanningstabell att `(AB)' = A' + B'`.

Övning 2.5 Från sanningstabell till uttryck

RÄKNA FÖR HAND

A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

a) Läs av uttrycket för X på SP-form, med en minterm per etta.

b) Förenkla uttrycket algebraiskt. Du ska få två termer med två variabler var.

c) Kontrollera ditt förenklade uttryck mot tabellen, rad för rad.

d) Hur många grindar behöver det oförenklade respektive det förenklade uttrycket, om du har NOT-grindar och AND- och OR-grindar med så många ingångar du vill?

Övning 2.6 Algebraisk förenkling

RÄKNA FÖR HAND

Förenkla så långt det går, och kontrollera varje svar med en sanningstabell.

a) `AB + AB' + A'B`

b) `(A + B)(A + B')`

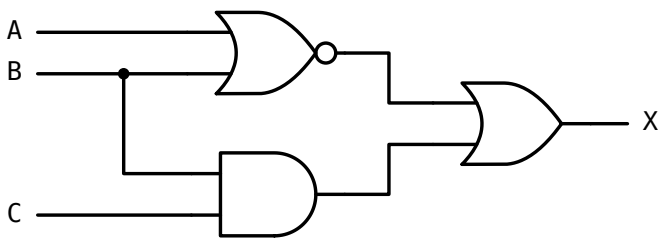
c) `S'R + SR' + SR`, som behövdes i härledningen av bromsassistenten i avsnitt 2.1.10. Visa att det är `S + R`.

d) `A'B'C + A'BC + ABC`

Övning 2.7 Kontroll: analys av ett grindnät

KONTROLL

Analysera för hand, bygg och simulera, jämför.



Figur 2.11 Grindnätet i övning 2.7.

Gör delarna i ordning, och skriv ned varje svar innan du går vidare.

a) För hand
Ge NOR-grindens utgång namnet P och AND-grindens utgång namnet Q. Skriv uttrycken för P, Q och X.

b)
Räkna fram sanningstabellen med kolumnerna A, B, C, P, Q och X, som i avsnitt 2.1.8.

c) I CircuitVerse
Bygg nätet precis som det är ritat, och gå igenom alla åtta kombinationerna. Stämmer simuleringen med din tabell?

d)
Om inte: är det tabellen eller bygget som är fel? Kontrollera den rad som skiljer genom att räkna ut P och Q för just den raden en gång till, och genom att följa varje ledning i bygget.

e)
Beskriv med en mening när X är `1`.

Övning 2.8 Från uttryck till grindnät

KONSTRUKTION

a) Rita grindnätet för `X = A'B + BC'` med NOT-, AND- och OR-grindar. Hur många grindar går det åt?

b) Bryt ut B ur uttrycket, och använd De Morgan på det som blir kvar. Visa att samma funktion går att bygga med en NAND-grind och en AND-grind.

c) Rita grindnätet för `Y = (A + B)C'`. Hur många grindar går det åt?

d) Bygg nätet från b) i CircuitVerse och kontrollera det mot en sanningstabell för `A'B + BC'`.

Övning 2.9 Allt av NAND

KONSTRUKTION

Avsnitt 2.1.5 påstår att varje funktion går att bygga med enbart NAND-grindar. Visa det för de tre grundoperationerna.

a) Visa algebraiskt att en NAND-grind med båda ingångarna ihopkopplade är en NOT-grind.

b) Hur bygger du en AND av NAND-grindar? Hur många behövs?

c) Utgå från `A + B = ((A + B)')'` och använd De Morgan för att skriva OR med enbart NAND-operationer. Hur många NAND-grindar behövs?

d) Bygg alla tre i CircuitVerse med enbart NAND-grindar, och kontrollera var och en mot sin sanningstabell.

Övning 2.10 Från uttryck till kontaktnät

KONSTRUKTION

Rita ett strömvägsschema mellan +24 V och 0 V för varje uttryck, med tryckknappar och lampan H1. Ange för varje kontakt om den är slutande eller brytande.

a) `H1 = S1 · S2'`

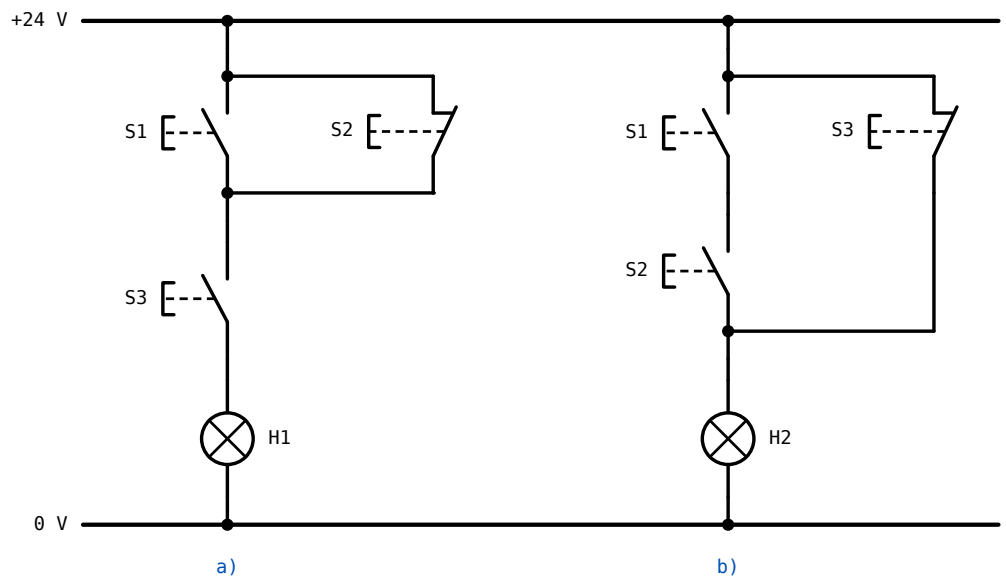
b) `H1 = S1 + S2 · S3`

c) `H1 = (S1 + S2) · S3'`

d) Hur många kontaktelement behöver varje knapp ha i a)–c)? Och hur många i XOR-kopplingen i avsnitt 2.2.5?

Övning 2.11 Analys av kontaktnät

RÄKNA FÖR HAND



Figur 2.12 Kontaktnäten a) och b) i övning 2.11.

a) Skriv uttrycket för H1 i nät a), och ta fram sanningstabellen för S1, S2 och S3.

b) Gör samma sak för H2 i nät b).

c) I nät b): vilka kombinationer tänder H2 utan att S1 eller S2 är nedtryckt? Förklara med ritningen varför.

Övning 2.12 Självhållning

FÖRSTÅELSE

Utgå från självhållningskretsen i avsnitt 2.2.7, figur 2.9.

a) Beskriv vad som händer med spolen K1 och lampan H1 när du trycker på S1, och sedan när du släpper den.

b) Beskriv vad som händer när du sedan trycker på S2, och när du släpper den.

c) Vad händer om någon trycker på S1 och S2 samtidigt? Vilken av knapparna ”vinner”?

d) Vid monteringen kopplas stoppknappen S2 av misstag med sitt slutande kontaktelement i stället för sitt brytande. Beskriv hur maskinen beter sig.

e) Förklara varför kretsen inte är ett kombinatoriskt nät. Använd ordet minne.

Övning 2.13 Bromsassistenten, felkopplad

RÄKNA FÖR HAND FÖRDJUPNING

En konstruktör vill spara en grind och kopplar bromsassistenten från avsnitt 2.1.10 som

```
B = (D + S + R)F'
```

Det är en OR-grind med tre ingångar följd av en AND med den inverterade felsignalen, och det ser rimligt ut.

a) Ta fram sanningstabellen för det nya uttrycket, och jämför den rad för rad med tabellen i avsnitt 2.1.10. På vilka rader skiljer de sig?

b) Vad har de raderna gemensamt? Beskriv vad som händer fysiskt i bilen i en sådan situation.

c) Vad säger det här om värdet av en sanningstabell som är härledd ur kravet, jämfört med en som är avläst ur en krets man redan har byggt?

KAPITEL 3

Analys och felsökning av kontaktnät

Från schema till lampa och tillbaka: att bygga, läsa och felsöka ett kontaktnät.

I DET HÄR KAPITLET

- Labbstationen: aggregatet, tryckknapparnas kontaktelement och lampan.
- Säkerhet och arbetssätt: rita först, koppla med spänningen frånslagen, förutsäg innan du mäter.
- **Labb 1**: AND, OR, NOT, NAND, NOR, XOR och XNOR med två tryckknappar och en lampa på 24 V.
- Syntes: från en sanningstabell till ett kontaktnät med så få kontakter som möjligt.
- Analys: från ett färdigt kontaktnät till dess uttryck, och vidare till ett mindre nät.
- Felsökning: vad man gör när lampan inte gör som schemat säger.

I kapitel 2 blev ett uttryck ett kontaktnät på papperet. Det här kapitlet hör ihop med Labb 1, vars handledning finns i bilaga A: där bygger du näten med två tryckknappar, en lampa och ett aggregat på 24 V. Kapitlet ger dig det du behöver för att gå åt andra hållet också, från ett färdigt nät till dess uttryck och till ett mindre nät som gör samma sak, och från en lampa som inte gör som schemat säger till felet som orsakar det.

Förbered dig innan labben. Läs avsnitt 2.2 om kontaktnät igen om det behövs, läs hela labbhandledningen, och gör dess förberedelseuppgifter F1–F4: de ska vara klara innan du börjar koppla. Läs också avsnitt 3.1, åtminstone avsnitt 3.1.1–3.1.4; resten, om felsökning, är det du slår upp när lampan inte lyser.

Gör sedan uppgift 1–8 i labben i ordning, och redovisa varje uppgift när den är klar, inte alla på en gång i slutet. Uppgift 9 är frivillig. Övningarna i avsnitt 3.3 tränar samma analys, syntes och felsökning på papper, och deras lösningsförslag finns i bilaga I.

3.1 Analys och felsökning av kontaktnät

3.1.1 Kontaktnätet som uttryck

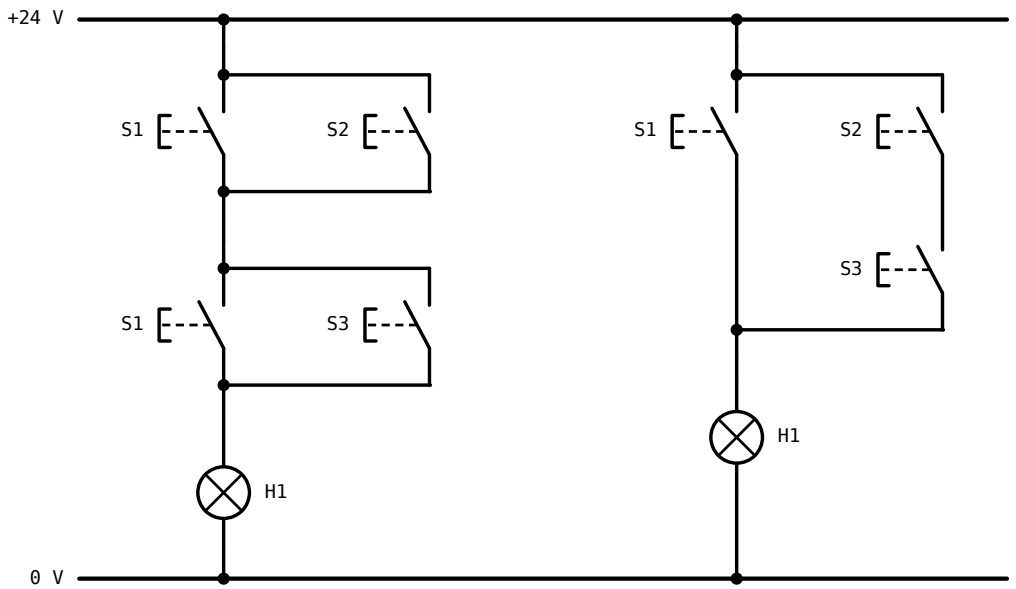
Avsnitt 2.2.6 visade hur ett uttryck blir ett kontaktnät. Att **analysera** ett nät är samma sak baklänges: du har ett schema, och ska ta reda på vilken funktion det realiserar.

Metoden är att arbeta **inifrån och ut**:

1. Leta upp de minsta grupperna: två eller flera kontakter som sitter direkt i serie eller direkt parallellt, utan något annat emellan. Skriv uttrycket för var och en: en seriekoppling blir en produkt, en parallellkoppling en summa. En slutande kontakt är variabeln, en brytande kontakt variabeln inverterad.
2. Betrakta varje sådan grupp som en enda kontakt, och upprepa: nu syns nästa nivå av serie- och parallellkopplingar.
3. När hela strömvägen från +24 V till lampan har blivit ett enda uttryck är du klar.
4. Räkna fram sanningstabellen ur uttrycket, och sammanfatta den med en mening.

Sätt parenteser runt varje grupp, också när de inte behövs. En parallellkoppling i serie med något annat är alltid en parentes, `(S1 + S2) · S3`, och den som glömmer den får `S1 + S2 · S3`, som är en helt annan funktion.

3.1.2 Genomarbetat exempel



Två nät, samma sanningstabell. Nät b) behöver en kontakt mindre, och S1 bara en gång. Lagen bakom är A + BC = (A + B)(A + C).

Figur 3.1 Två kontaktnät med samma sanningstabell.

Nät a). De minsta grupperna är de två parallellkopplingarna: `S1 + S2` överst och `S1 + S3` under. De två grupperna sitter i serie med varandra, och därefter lampan:

```
H1 = (S1 + S2) · (S1 + S3)
```

Sanningstabellen, med varje grupp som en egen kolumn:

S1	S2	S3	S1 + S2	S1 + S3	H1
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

Sammanfattat med ord: **lampan lyser om S1 är nedtryckt, eller om både S2 och S3 är det.**

Men den meningen är ett annat uttryck, `S1 + S2 · S3`, och det är nät b). Att de två är samma funktion går att visa algebraiskt genom att multiplicera ut:

```
(S1 + S2)(S1 + S3) = S1·S1 + S1·S3 + S2·S1 + S2·S3
                   = S1 + S1·S3 + S1·S2 + S2·S3
                   = S1(1 + S3 + S2) + S2·S3
                   = S1 + S2·S3
```

Det är den andra distributiva lagen från avsnitt 2.1.4, `A + BC = (A + B)(A + C)`, den som inte ser ut som vanlig algebra. I kontakter är den lätt att se: i nät a) behövs S1 i båda parallellkopplingarna, och i nät b) räcker det med en S1 som går förbi allt annat.

Nät b) har tre kontakter i stället för fyra, och S1 behöver bara ett kontaktelement i stället för två. Färre kontakter betyder färre sladdar att koppla, färre ställen där något kan gå sönder, och i en maskin lägre kostnad.

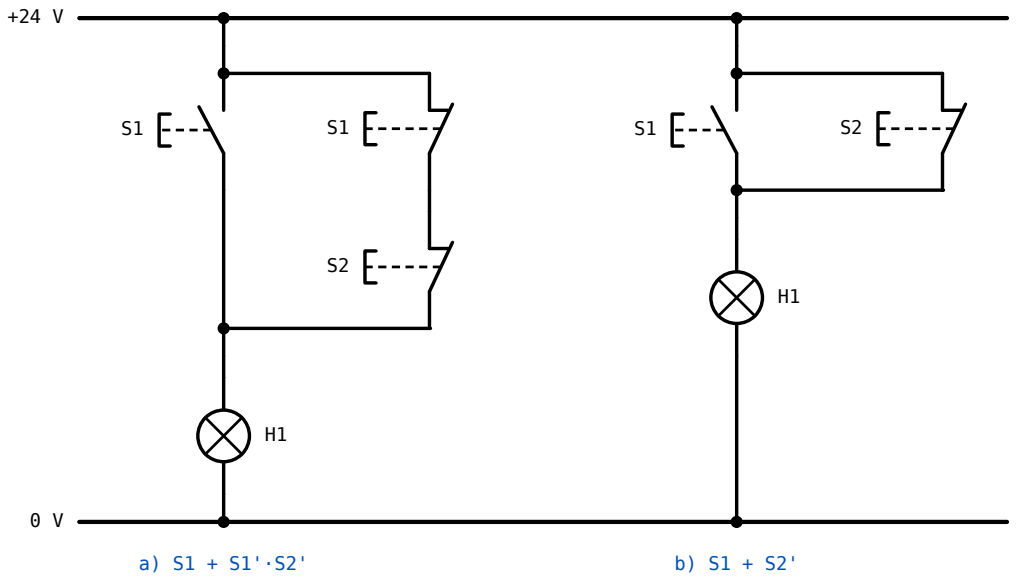
3.1.3 Likvärdiga nät

Två nät är **likvärdiga** om de har samma sanningstabell. De kan se helt olika ut, ha olika antal kontakter och vara kopplade på olika sätt; om lampan gör samma sak för varje kombination av knapparna är de, för den som trycker på knapparna, samma nät.

Det finns två sätt att visa att två nät är likvärdiga:

- **Med sanningstabeller.** Räkna fram tabellen för båda och jämför dem rad för rad. Det fungerar alltid, och med två eller tre knappar är det bara fyra eller åtta rader.
- **Med algebra.** Förenkla det ena uttrycket tills det blir det andra, som i avsnitt 3.1.2. Det är snabbare när man ser vägen, men lätt att räkna fel.

Ett exempel på ett nät med en onödig kontakt:



Figur 3.2 Två likvärdiga nät, det ena med en onödig kontakt.

Nät a) är `S1 + S1' · S2'`. Förenklingslagen från avsnitt 2.1.4, `A + A'B = A + B` med `S2'` i B:s roll, säger direkt att det är `S1 + S2'`, alltså nät b).

Varför kan den brytande kontakten S1 tas bort? Titta på de två fallen:

- När S1 är **nedtryckt** leder den vänstra grenen, och lampan lyser vad den högra grenen än gör.
- När S1 är **släppt** är den brytande kontakten S1 sluten, och den högra grenen är bara S2:s brytande kontakt.

Den brytande kontakten S1 avgör alltså aldrig något: i det enda fall där den är öppen spelar den högra grenen ingen roll. Den **påverkar strömmen**, men den **påverkar inte lampan**, och en kontakt som inte påverkar utgången kan ersättas med en ledning. Labb 1 har i uppgift 8 ett nät av samma slag, som du analyserar och förenklar själv.

3.1.4 Förutsäg innan du mäter

Varje uppgift i Labb 1 följer samma arbetsgång:

1. **Rita schemat** på papper, med beteckning och kontaktsort för varje kontakt.
2. **Förutsäg** lampans läge för varje kombination av knapparna, och skriv in det i tabellen.
3. **Koppla** efter schemat, med spänningen frånslagen.

4. **Mät**: slå på spänningen, pröva varje kombination, och skriv in vad lampan gör.
5. **Jämför** de två kolumnerna.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

Det kan kännas som en omväg att fylla i den förutsagda kolumnen när svaret ändå kommer om en minut. Det är det inte. **Den som bara mäter har inget att jämföra med**, och kan inte avgöra om lampan gör rätt eller fel. En avvikelse mellan de två kolumnerna är information: den säger att schemat, förutsägelsen eller kopplingen är fel, och vilken rad som avslöjar det. Det är där felsökningen börjar.

3.1.5 Felsökning

Lampan gör inte som den ska. Det finns två sätt att hitta felet, och båda bygger på samma idé: **följ strömvägen från +24 V mot 0 V, och hitta det första stället där den är bruten.**

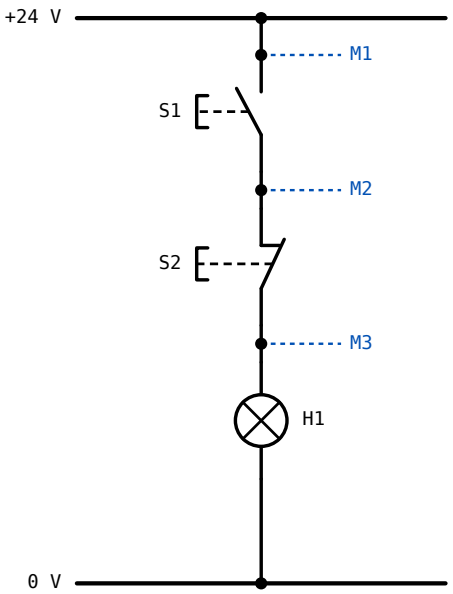
Med spänningen påslagen: mät spänning

Ställ multimetern på likspänning (V DC), sätt den svarta mätsladden på 0 V, och mät med den röda i varje mätpunkt längs strömvägen, uppifrån och ned, som i figur 3.3. Tryck på de knappar som ska få lampan att lysa.

Så länge vägen ovanför mätpunkten är sluten visar mätaren 24 V. Efter det element som bryter vägen visar den 0 V. Med S1 nedtryckt och S2 släppt ska lampan lysa, och mätningen avslöjar var den inte gör det:

M1	M2	M3	Slutsats
0 V	0 V	0 V	Ingen spänning alls: aggregatet är av, eller ledningen från +24 V är bruten.
24 V	0 V	0 V	S1 leder inte: fel kontaktelement, en lös sladd, eller en trasig kontakt.
24 V	24 V	0 V	S2 leder inte: troligen kopplad till de slutande anslutningarna i stället för de brytande.
24 V	24 V	24 V	Vägen fram till lampan är hel: felet sitter i lampan eller i ledningen till 0 V.

I det sista fallet: mät direkt över lampan. 24 V över en släckt lampa betyder att lampan är trasig; 0 V betyder att ledningen från lampan till 0 V är bruten.



Mät spänningen mellan varje mätpunkt och 0 V, uppifrån och ned. Så länge vägen är sluten visar mätaren 24 V; efter det element som bryter vägen visar den 0 V.

Figur 3.3 En strömväg med mätpunkterna M1, M2 och M3.

Med spänningen frånslagen: mät ledning

Slå av aggregatet, ställ multimetern på genomgångsprovning (ofta med en summer) eller resistans, och mät över ett element i taget. En sluten kontakt eller en hel sladd ger nästan 0 Ω och ett pip; en öppen kontakt eller en bruten sladd ger inget utslag. Tryck på knappen medan du mäter över dess kontakt, för att se att den ändrar sig som den ska.

> **Mät aldrig resistans eller genomgång med spänningen påslagen.** Mätaren mäter resistans genom att skicka en egen liten ström, och en yttre spänning ger i bästa fall ett meningslöst värde och i värsta fall en trasig mätare.

Halveringsmetoden

I en lång strömväg lönar det sig att inte börja uppifrån, utan i **mitten**. Visar mätpunkten i mitten 24 V sitter felet i den nedre halvan; visar den 0 V sitter det i den övre. Mät sedan i mitten av den halva som återstår, och så vidare. Med åtta element hittar du felet på tre mätningar i stället för upp till sju.

3.1.6 Vanliga fel

Symptom	Trolig orsak
Lampan lyser aldrig	Aggregatet är av; en sladd sitter i fel hål; lampan är trasig; en kontakt i serie är kopplad som slutande när den skulle vara brytande, eller omvänt.
Lampan lyser alltid	En kontakt är förbikopplad av en extra sladd; en parallell gren har fått en brytande kontakt i stället för en slutande.
Lampan gör precis tvärtom mot tabellen	En kontakt är kopplad till fel kontaktelement: slutande och brytande har bytt plats.
Lampan fungerar för en knapp men inte för den andra	Den andra knappens kontakt är inte med i strömvägen; kontrollera att båda ändarna av den är anslutna.
Aggregatet går i strömbegränsning eller säkringen löser ut	Kortslutning: +24 V har fått en väg till 0 V utan lampan emellan. Slå av och kontrollera schemat.

Den vanligaste orsaken av alla är att schemat och kopplingen inte stämmer överens. Kontrollera kopplingen mot schemat sladd för sladd, gärna med en kamrat som läser schemat högt medan du pekar på sladdarna, innan du börjar mäta.

3.2 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet och Labb 1 ska du kunna:

- Avgöra vilka anslutningar på en tryckknapp som är slutande och vilka som är brytande, både med hjälp av märkningen och genom mätning.
- Koppla upp ett kontaktnät efter ett strömvägsschema, och kontrollera det mot en sanningstabell.
- Realisera AND, OR, NOT, NAND, NOR, XOR och XNOR med kontakter.
- Ta fram ett kontaktnät ur en sanningstabell, via ett förenklat uttryck.
- Analysera ett kontaktnät: ta fram dess uttryck och sanningstabell, och hitta ett mindre nät med samma funktion.
- Felsöka ett kontaktnät systematiskt, med en lampa eller en multimeter.

Frågor att testa dig själv med

1. Varför ska ett kontaktnät kopplas med spänningen frånslagen, fast 24 V inte är farligt att röra?
2. Hur avgör du med en lampa och ett aggregat om ett kontaktelement är slutande eller brytande?
3. Två kontaktnät ser helt olika ut. Hur avgör du om de gör samma sak?

4. I uppgift 8 i Labb 1 går det att ta bort en kontakt utan att funktionen ändras. Hur kan en kontakt vara onödig, när den ju faktiskt påverkar strömmen?
5. Lampan lyser aldrig, vad du än trycker på. Var börjar du leta, och varför just där?
6. Varför används en stoppknapps brytande kontakt, och inte dess slutande, i en maskins stoppkrets?

Nästa kapitel

Kapitel 4 handlar om Karnaughdiagram och IC-kretsar:

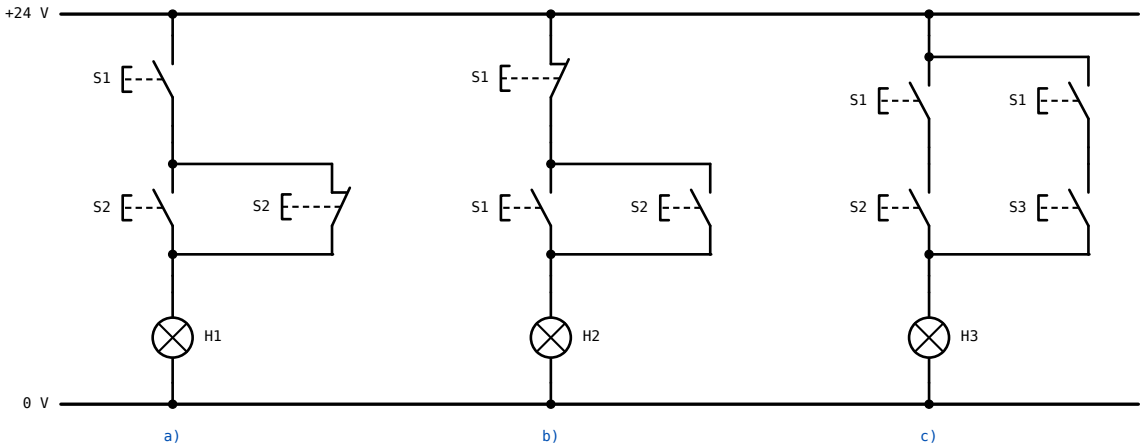
- Karnaughdiagram: ett systematiskt sätt att hitta det minsta nätet, i stället för att leta med räknelagarna.
- Syntes av kombinatoriska nät, från specifikation till grindnät.
- NAND- och NOR-nät.
- IC-kretsar ur 74-serien: benplacering, datablad och kopplingsdäck, inför Labb 2.

3.3 Övningar

Så kontrollerar du ditt arbete. Varje uttryck du läser av ur ett nät kan kontrolleras med nätet självt: välj en rad i sanningstabellen, tänk dig knapparna i det läget, och följ strömvägen i ritningen. Finns det en sluten väg från +24 V till lampan ska tabellen säga 1. Övning 3.3 kontrolleras på labbstationen.
Lösningsförslagen finns i bilaga I.
Varje övning är märkt med sin sort. Övning 3.3 är kapitlets **kontrolluppgift**.

Övning 3.1 Läs av uttrycket

RÄKNA FÖR HAND



Figur 3.4 Tre kontaktnät att analysera.

För vart och ett av näten a), b) och c):

a) Skriv uttrycket för lampan, med parenteser runt varje grupp, enligt avsnitt 3.1.1.

b) Ta fram sanningstabellen.

c) Sammanfatta tabellen med en mening.

Övning 3.2 Det minsta nätet

KONSTRUKTION

a) Förenkla vart och ett av de tre uttrycken från övning 3.1, och ange vilken räknelag du använder.

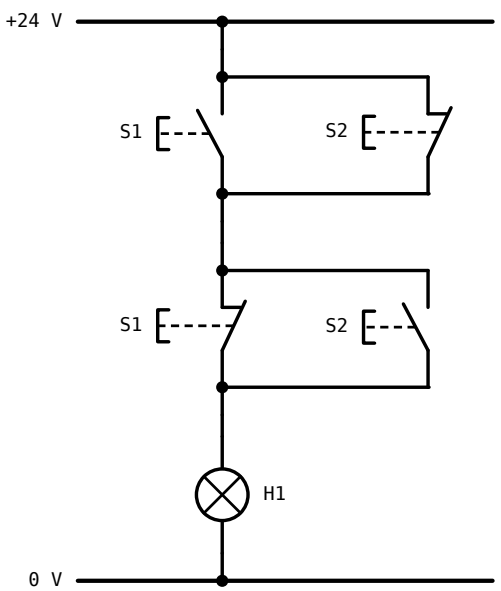
b) Beskriv det minsta nätet för varje uttryck: vilka kontakter, och hur de sitter.

c) Hur många kontakter sparar du i varje nät? Hur många kontaktelement behöver varje knapp före och efter?

Övning 3.3 Kontroll: förutsäg och bygg

KONTROLL

Förutsäg för hand, bygg på stationen, jämför.



Varje knapp används med en slutande och en brytande kontakt.

Figur 3.5 Kontaktnätet i kontrolluppgiften.

Gör delarna i ordning, och skriv ned varje svar innan du går vidare.

a) För hand
Skriv uttrycket för H1, och förutsäg lampans läge för alla fyra kombinationerna av S1 och S2.

b)
Multiplicera ut uttrycket och förenkla. Vilken grind från kapitel 2 är det?

c) På stationen
Koppla nätet och pröva alla fyra kombinationerna. Stämmer lampan med din förutsägelse?

d)
Jämför med nätet för samma funktion i Labb 1, uppgift 6. Hur många kontakter har de två näten? Varför kan två så olika nät göra samma sak?

Övning 3.4 Syntes

KONSTRUKTION

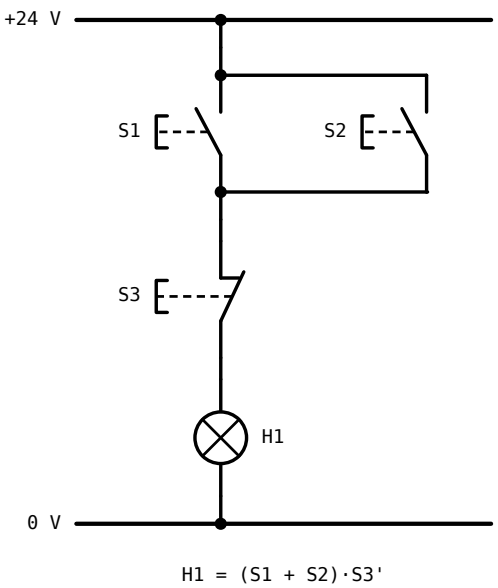
a) Ta fram det minsta kontaktnätet med två knappar för sanningstabellen nedan:

S1	S2	H1
0	0	1
0	1	1
1	0	0
1	1	1

b) Ett transportband drivs av en motor som styrs av reläet K1. Motorn får gå om skyddsgrinden är stängd (givaren S3 påverkad), och samtidigt antingen startknappen på plats (S1) eller fjärrstarten (S2) är intryckt. Skriv uttrycket för K1 och rita strömvägen.

Övning 3.5 Felsökning

FÖRSTÅELSE



Figur 3.6 Nätet som ska felsökas.

Nätet ska realisera `H1 = (S1 + S2) · S3'`.

a) Du trycker på S1 men inte på S3, och lampan förblir släckt. Du mäter 24 V mellan 0 V och punkten ovanför parallellkopplingen, 24 V mellan 0 V och punkten mellan

parallellkopplingen och S3, och 0 V mellan 0 V och punkten mellan S3 och lampan. Var sitter felet, och vad är den troligaste orsaken?

b) Efter en ny koppling lyser lampan när S1 eller S2 är nedtryckt, men den slocknar inte när du dessutom trycker på S3. Vad är fel?

c) Efter ytterligare en ny koppling lyser lampan bara när S3 **och** minst en av S1 och S2 är nedtryckta. Vad är fel nu?

Övning 3.6 Stoppknappen

FÖRSTÅELSE

a) I en maskins stoppkrets används stoppknappens brytande kontakt. Beskriv vad som händer med maskinen om ledningen till stoppknappen går av.

b) Beskriv vad som skulle hända om stoppknappens slutande kontakt i stället användes, och kretsen var gjord så att maskinen stannar när kontakten sluts. Vad händer då om ledningen går av?

c) Formulera en regel för hur en stoppfunktion ska kopplas, med ordet *säker*.

Övning 3.7 Alla funktioner av två knappar

FÖRSTÅELSE

a) En sanningstabell med två ingångar har fyra rader. Hur många olika sanningstabeller finns det, alltså hur många olika funktioner av två variabler?

b) Två av funktionerna behöver inte någon kontakt alls. Vilka, och hur realiseras de?

c) Vilka funktioner behöver både det slutande och det brytande kontaktelementet i båda knapparna?

Övning 3.8 En väljare av kontakter

RÄKNA FÖR HAND FÖRDJUPNING

Ett nät har kontakterna S1 och S2 slutande parallellt, i serie med S1 brytande och S3 slutande parallellt: `H1 = (S1 + S2) · (S1' + S3)`.

a) Ta fram sanningstabellen.

b) Dela tabellen i två halvor efter S1, och beskriv med ord vad nätet gör.

c) Visa algebraiskt att `(S1 + S2)(S1' + S3) = S1' · S2 + S1 · S3`. Tänk på att `S1 · S1' = 0`, och att en term som `S2 · S3` kan visa sig vara onödig: kontrollera med tabellen.

d) Jämför med grindnätet i avsnitt 2.1.8. Vad har de gemensamt?

KAPITEL 4

Karnaughdiagram och IC-kretsar

Hur man hittar det minsta nätet utan att gissa, och hur man kopplar upp det.

I DET HÄR KAPITLET

- Varför ett korrekt nät inte räcker: det ska också vara litet.
- Karnaughdiagram för två, tre och fyra ingångar: Graykoden, grupperna och avläsningen.
- Kanter, hörn och don't care, de tre ställen där minimering oftast går fel.
- Syntes av ett kombinatoriskt nät: från specifikation till minimerat grindnät.
- Nät med enbart NAND eller enbart NOR, och varför de sparar kretsar.
- IC-kretsar ur 74-serien: kapsel, benplacering, datablad och logiknivåer.
- Kopplingsdäcket, lysdioder och ingångar som aldrig får flyta.

Ett grindnät som ger rätt sanningstabell är inte färdigt förrän det också är litet. Det här kapitlet visar hur ett Karnaughdiagram hittar det minsta nätet utan gissningar, och hur nätet sedan byggs med riktiga IC-kretsar ur 74-serien på ett kopplingsdäck, inför Labb 2.

Repetera först räknelagarna, De Morgans lagar och hur ett uttryck läses av ur en sanningstabell, i avsnitt 2.1. Läs sedan avsnitt 4.1 om Karnaughdiagram, där avsnitt 4.1.1–4.1.4 är grunden för resten, och avsnitt 4.2 om IC-kretsar, datablad och kopplingsdäck. Övningarna i avsnitt 4.4 görs med papper och penna och kontrolleras i CircuitVerse, och deras lösningsförslag finns i bilaga I. Gör förberedelserna till Labb 2 i bilaga B, som avsnitt 4.2.8 går igenom, innan labben i kapitel 5.

4.1 Karnaughdiagram

4.1.1 Varför minimera

I kapitel 2 lärde du dig läsa av ett uttryck direkt ur en sanningstabell: en AND-term för varje rad där utgången är `1`, och alla termerna ihop med OR. Den metoden fungerar **alltid**, och det är dess stora styrka. Men den ger nästan aldrig det **minsta** nätet.

Ta sanningstabellen nedan, med tre ingångar `A`, `B`, `C` och en utgång `X`:

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Fem rader har `X = 1`, så den direkta avläsningen ger fem AND-termer med tre ingångar var:

```
X = A'B'C + A'BC + AB'C + ABC' + ABC
```

Det är fem AND-grindar med tre ingångar, en OR-grind med fem ingångar och tre inverterare. Samma funktion kan skrivas `X = AB + C`: en AND-grind och en OR-grind. Båda uttrycken ger exakt samma sanningstabell, men det ena kostar nio grindar och det andra två.

Skillnaden är inte akademisk. Varje grind är en krets att köpa, en plats på kopplingsdäcket och ett dussin sladdar att dra, och varje sladd är ett ställe där något kan bli fel. I Labb 1 var varje term dessutom en kontakt i en tryckknapp, och en tryckknapp har bara så många kontakter.

Du kan förenkla med räknelagarna från kapitel 2, och det är precis vad du gjorde där. Problemet med algebra är att du måste **se** vilken lag som ska användas härnäst, och att du aldrig riktigt vet när du är klar. Ett **Karnaughdiagram** (uttalas ”karnå”) gör samma förenkling visuell: du ritar tabellen som ett rutnät, ringar in ettor som ligger bredvid varandra, och läser av svaret. För funktioner med två till fyra ingångar är det den snabbaste och säkraste metoden som finns.

4.1.2 Diagrammet och Graykoden

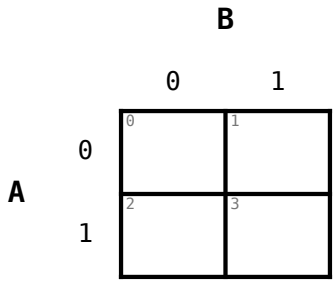
Ett Karnaughdiagram är sanningstabellen omskriven till ett rutnät: **en ruta per rad** i tabellen. Tre ingångar ger åtta rader och därför åtta rutor; fyra ingångar ger sexton.

Idén bakom rutnätet är en enda regel: **två rutor som ligger bredvid varandra får bara skilja sig i en enda ingång.** Då betyder två ettor bredvid varandra alltid att utgången inte beror på den ingång som skiljer dem åt, och den ingången kan strykas. Det är hela metoden; resten är teknik.

För att regeln ska hålla skrivs rubrikerna i **Graykod**, inte i vanlig binär ordning. Med två bitar är ordningen `00, 01, 11, 10`: bara en bit ändras i varje steg, också i steget från den sista tillbaka till den första. Graykoden presenterades i avsnitt 1.2.3; här är den till nytta. Vanlig binär ordning, `00, 01, 10, 11`, fungerar **inte**: i steget från `01` till `10` ändras båda bitarna, och två rutor som ser ut som grannar är det inte.

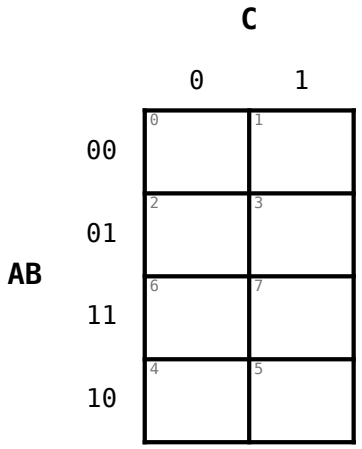
Boken använder alltid samma uppställning:

- **två ingångar**: `A` nedåt, `B` åt sidan;
- **tre ingångar**: `AB` nedåt i Graykod, `C` åt sidan;
- **fyra ingångar**: `AB` nedåt och `CD` åt sidan, båda i Graykod.



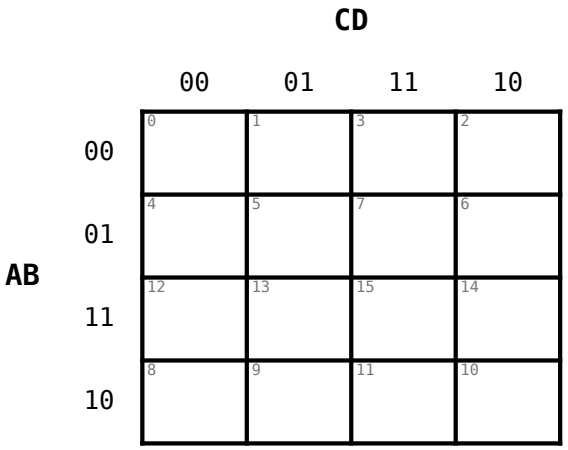

```
Två variabler: A nedåt, B åt sidan.
```

Figur 4.1 Tomt Karnaughdiagram för två ingångar, med rutornas nummer.




```
Tre variabler: AB nedåt i Graykod, C åt sidan.
```

Figur 4.2 Tomt Karnaughdiagram för tre ingångar, med rutornas nummer.



Figur 4.3 Tomt Karnaughdiagram för fyra ingångar, med rutornas nummer.

Det lilla grå talet i hörnet av varje ruta är radens nummer i sanningstabellen: ingångarna läst som ett binärt tal, med `A` som mest signifikant bit. Rutan på raden `AB = 11` och i kolumnen `CD = 01` är alltså `ABCD = 1101`, rad **13**. Ett sådant radnummer kallas en **minterm**, och det är det enklaste sättet att föra över en tabell till diagrammet utan att fastna på Graykoden.

Lägg märke till ordningen: `2` står till höger om `3`, och raden med `12`–`15` står ovanför raden med `8`–`11`. Det är Graykoden som syns. Den som fyller i diagrammet i tabellens ordning, rad för rad från vänster, gör just det fel som gör resten av arbetet värdelöst.

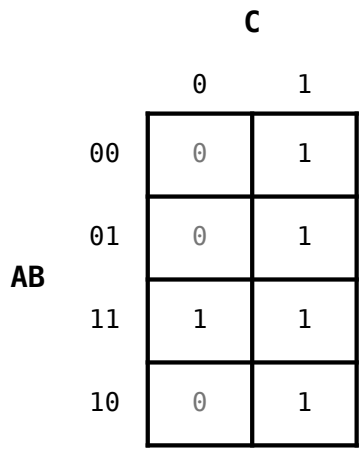
4.1.3 Att gruppera

Arbetsgången, för varje diagram:

1. **Fyll i** en `1` i varje ruta där utgången är `1`. Lämna övriga rutor tomma; nollorna behövs inte för att hitta grupperna.
2. **Ringa in grupper av ettor**, efter tre regler:
   - en grupp är en **rektangel**, aldrig ett L eller en diagonal;
   - gruppens storlek är en **tvåpotens**: 1, 2, 4, 8 eller 16 rutor. En grupp om 4 kan vara en rad, en kolumn eller en kvadrat om 2×2, men aldrig tre rutor och aldrig sex;
   - en grupp får bara innehålla ettor.
3. **Gör varje grupp så stor som möjligt.** En större grupp stryker fler ingångar och ger en kortare term.

4. **Täck varje etta minst en gång**, med så få grupper som möjligt. Grupper får överlappa, och en etta får alltså ingå i flera grupper. En grupp vars alla ettor redan täcks av andra grupper är däremot överflödig: stryk den.

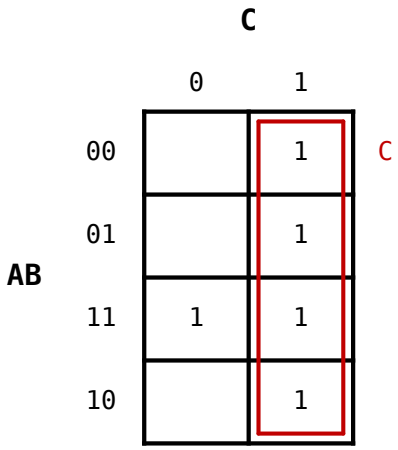
Diagrammet för tabellen i avsnitt 4.1.1, ifyllt:




```
Sanningstabellen inlagd: en etta per rad där X = 1.
```

Figur 4.4 Diagrammet för `X` ifyllt med en etta för varje rad där `X = 1`.

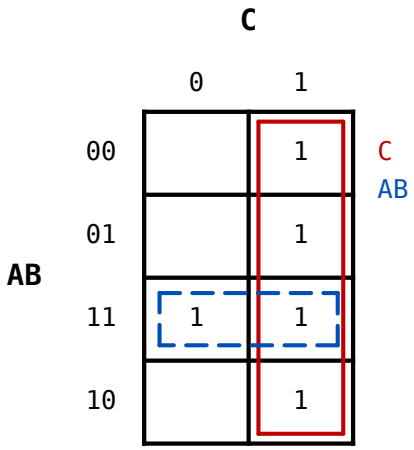
Hela kolumnen `C = 1` är fylld med ettor: fyra rutor, en tillåten grupp.



Första gruppen: hela kolumnen C = 1.

Figur 4.5 Samma diagram med gruppen C = 1 inringad.

En etta är ännu inte täckt, på raden AB = 11. Den största grupp den kan ingå i är paret med sin granne till höger, som redan ingår i C-gruppen. Det gör inget: överlapp är tillåtet, och en grupp om två är bättre än en grupp om en.



Alla ettor täckta: X = AB + C.

Figur 4.6 Samma diagram med också gruppen AB = 11 inringad.

Varje etta är nu täckt, och båda grupperna är så stora de kan bli.

4.1.4 Att läsa av grupperna

Varje grupp blir en AND-term. För att hitta den, gå igenom ingångarna en i taget och fråga: **har ingången samma värde i varje ruta i gruppen?**

- Om ja, och värdet är `1`: ingången står med i termen, som den är.
- Om ja, och värdet är `0`: ingången står med, inverterad (med prim).
- Om nej: ingången stryks. Utgången bryr sig inte om den inom gruppen.

Gruppen i kolumnen `C = 1`: `A` är både 0 och 1 i gruppen, och stryks. `B` är både 0 och 1, och stryks. `C` är 1 i alla fyra rutorna. Termen blir **`C`**.

Gruppen på raden `AB = 11`: `A` är 1 i båda rutorna, `B` är 1 i båda, och `C` är 0 i den ena och 1 i den andra. Termen blir **`AB`**.

Svaret är termerna ihop med OR:

```
X = AB + C
```

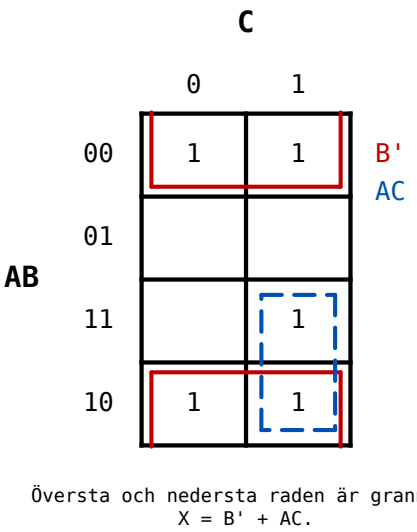
Två tumregler gör avläsningen snabb, och de är värda att lära sig utantill:

Gruppens storlek	Antal ingångar som stryks	Kvar i termen, av 3	Kvar i termen, av 4
1 ruta	0	3	4
2 rutor	1	2	3
4 rutor	2	1	2
8 rutor	3	0	1

- En grupp om 2^k rutor stryker exakt k ingångar. Om din avläsning ger en annan längd på termen har du läst fel.
- **Kontrollera alltid svaret mot tabellen.** Sätt in varje rad i ditt uttryck och se att det ger rätt `X`. Med tre ingångar är det åtta rader, och det tar en minut. Det är samma disciplin som i CircuitVerse: förutsäg, kontrollera, och lita inte på något du inte kontrollerat.

4.1.5 Kanter och hörn

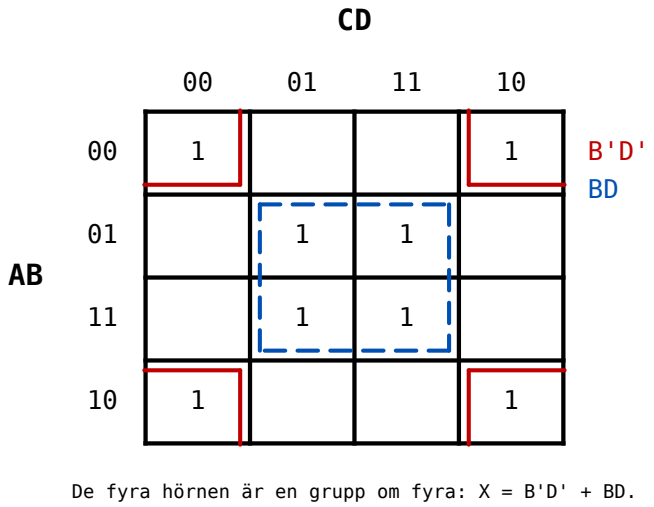
Graykoden gäller också runt kanterna: den översta raden (`AB = 00`) och den nedersta (`AB = 10`) skiljer sig bara i `A`, och är därför grannar. Likadant är vänstra och högra kolumnen grannar i ett diagram med fyra ingångar. Tänk dig att diagrammet är ritat på en cylinder, eller att rutnätet fortsätter på andra sidan, som i ett tv-spel där man kommer ut på motsatt sida.



Figur 4.7 En grupp om fyra som går över kanten mellan översta och nedersta raden.

Gruppen `B'` består av raderna `AB = 00` och `AB = 10`, båda kolumnerna. I alla fyra rutorna är `B = 0`, och den läses av precis som en grupp i mitten av diagrammet. Den ritas som två halva ringar som är öppna mot kanten de går över; det är så en grupp över kanten ritas för hand. Den sista ettan, `ABC = 111`, paras med sin granne nedanför och ger `AC`, så `X = B' + AC`.

I ett diagram med fyra ingångar är de fyra **hörnen** grannar till varandra, två och två: översta och nedersta raden är grannar, och vänstra och högra kolumnen är grannar. Hörnen bildar alltså en kvadrat om 2 × 2, bara uppdelad på fyra ställen.



Figur 4.8 De fyra hörnen som en grupp, och mitten som en annan.

Hörngruppen: `A` varierar (rad `00` och `10`), `B = 0` i alla fyra, `C` varierar (kolumn `00` och `10`), `D = 0` i alla fyra. Termen blir `B'D'`. Mittgruppen har `B = 1` och `D = 1` överallt och blir `BD`. Svaret är `X = B'D' + BD`, vilket för övrigt är XNOR av `B` och `D`: `A` och `C` spelar ingen roll alls.

Hörnen är det ställe där flest missar en grupp. När du tror att du är klar med ett diagram med fyra ingångar, titta på hörnen en extra gång.

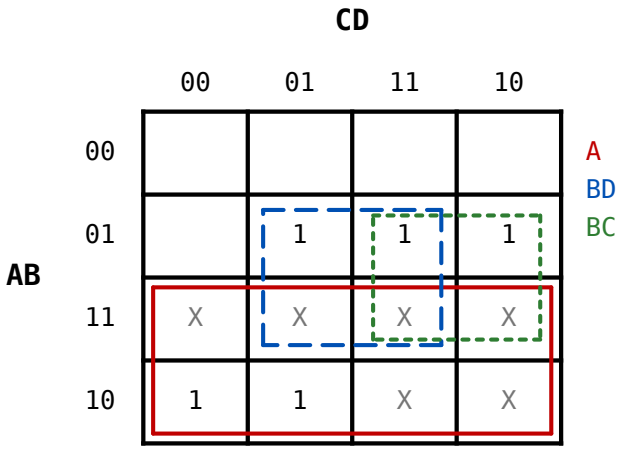
4.1.6 Don't care

Ibland finns det ingångskombinationer som **aldrig förekommer**. Då spelar det ingen roll vad utgången blir för dem, och det kan du utnyttja.

Det vanligaste exemplet är **BCD** från avsnitt 1.2.2: en decimal siffra 0–9 kodad med fyra bitar `ABCD`. Kombinationerna `1010` till `1111`, alltså 10–15, är inga siffror och kommer aldrig in i kretsen. I diagrammet markeras de med ett **X**, ”don't care”.

Regeln är enkel: **ett X får räknas som en etta om det gör en grupp större, och som en nolla annars.** Det behöver inte täckas.

Exempel: en krets som ska tända en lampa när siffran är **minst 5**. Ettor för 5, 6, 7, 8 och 9, X för 10–15:



Figur 4.9 BCD-siffran minst 5, med don't care i rutorna 10–15.

Tack vare X-en kan de två nedersta raderna bli en enda grupp om åtta: `A`. Utan X-en hade ettorna 8 och 9 bara kunnat bilda gruppen `AB'C'`. De andra två grupperna, `BD` och `BC`, använder också X-rutor för att växa från två till fyra rutor. Svaret är:

```
X = A + BD + BC
```

Kontrollera mot siffrorna: 5 är `0101`, där `BD = 1`. 6 är `0110`, där `BC = 1`. 7 är `0111`, där båda är 1. 8 och 9 har `A = 1`. Siffrorna 0–4 har `A = 0` och aldrig både `B` och `C` eller `B` och `D`, så `X = 0`. Stämmer.

4.1.7 Från specifikation till grindnät

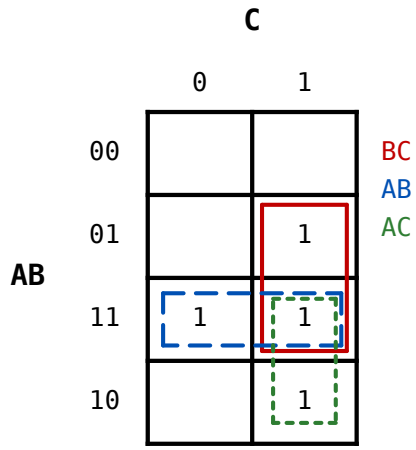
Nu hela vägen, från en beskrivning i ord till ett nät som kan kopplas upp. Det är arbetsgången du använder i Labb 2, och den har fem steg.

Specifikationen. En tank har tre nivågivare, `A`, `B` och `C`, som var och en ger `1` när de känner vätska. Givare kan gå sönder, så ett larm för hög nivå ska inte lita på en enda givare. Larmet `X` ska gå när **minst två** av givarna ger `1`. En sådan krets kallas en **majoritetskrets**, och samma idé används i flygplan och kraftverk, där tre datorer räknar samma sak och systemet gör det två av dem är överens om.

Steg 1: sanningstabellen. Tre ingångar, åtta rader. `X = 1` där minst två ingångar är `1`:

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Steg 2: Karnaughdiagrammet. Ettor i rutorna 3, 5, 6 och 7:



Tre grupper om två som överlappar i ABC = 111:
X = AB + AC + BC.

Figur 4.10 Majoritetskretsen: tre överlappande grupper om två.

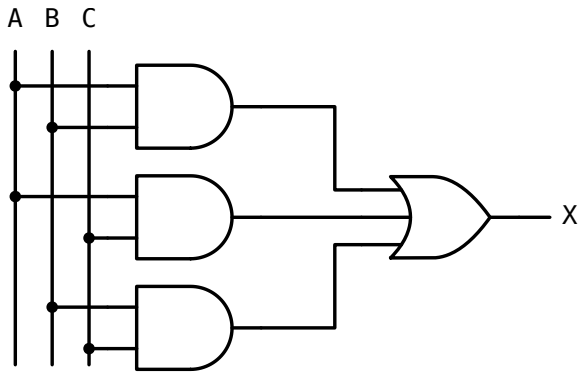
Ingen grupp om fyra är möjlig, men varje etta kan paras med rutan `ABC = 111`. Tre par, som överlappar i mitten.

Steg 3: det minimerade uttrycket.

```
X = AB + AC + BC
```

Läs det högt och jämför med specifikationen: ”A och B, eller A och C, eller B och C”. Det är ”minst två” skrivet som logik, och det är en bra kontroll att uttrycket går att läsa så.

Steg 4: grindnätet. Tre AND-grindar med två ingångar och en OR-grind med tre:



Figur 4.11 Grindnät för majoritetskretsen, `X = AB + AC + BC`: tre AND-grindar och en OR-grind med tre ingångar.

Steg 5: kontrollera. Bygg nätet i CircuitVerse (`https://circuitverse.org/simulator`) och gå igenom alla åtta raderna, eller sätt in raderna i uttrycket för hand. Först därefter kopplas det upp. En direkt avläsning av tabellen hade gett fyra termer med tre ingångar var; diagrammet gav tre termer med två.

4.1.8 Nät med flera utgångar

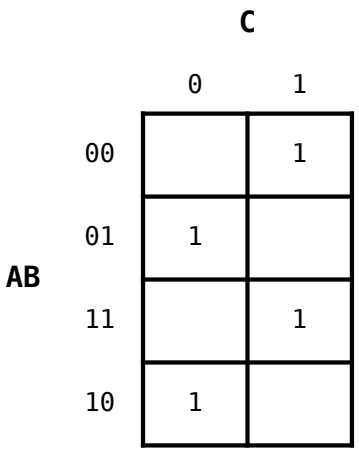
Många kretsar har mer än en utgång. Då ritas **ett diagram per utgång**, och varje utgång minimeras för sig.

Ett exempel som kommer tillbaka i kapitel 6 (avsnitt 6.2.3), när vi tittar inuti mikrodatorns räkneenhet, är **heladderaren**: den adderar två bitar `A` och `B` och en minnessiffra `C` från föregående position, och ger en summa `S` och en ny minnessiffra `Cut`.

A	B	C	Cut	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Kolumnen `Cut` är precis majoritetskretsen från avsnitt 4.1.7: minnessiffran blir 1 när minst två av de tre bitarna är 1. Samma diagram, samma svar, `Cut = AB + AC + BC`.

Kolumnen `S` är mer intressant:



Summan S i en heladderare: inga två ettor är grannar, så varje grupp blir en enda ruta.

Figur 4.12 Summan `S` i en heladderare: ettorna bildar ett schackmönster.

Ettorna bildar ett schackbräde. **Ingen etta har en etta som granne**, så den största möjliga gruppen är en enda ruta, och diagrammet kan inte förenkla något. Det är inget misslyckande, det är ett svar: ett schackmönster betyder **XOR**. `S = 1` när ett udda antal av bitarna är 1, vilket är `S = A xor B xor C`, två XOR-grindar.

Ett Karnaughdiagram hittar det minsta nätet av AND-, OR- och NOT-grindar. Det ser inte XOR. När du får ett schackmönster, eller en del av ett, är det dags att tänka på XOR i stället.

4.1.9 NAND- och NOR-nät

I avsnitt 2.1.5 såg du att **NAND** ensam räcker för att bygga vilken funktion som helst, och likaså **NOR**. Det är mer än en kuriositet: en krets som 74HC00 innehåller fyra NAND-grindar, och ett nät som bara använder NAND kan ofta byggas med en enda krets i stället för två eller tre.

Från summa av produkter till NAND-NAND

Ett uttryck på summa av produkter-form, som de ett Karnaughdiagram ger, kan alltid göras om till enbart NAND i tre steg:

1. Rita nätet som AND-grindar som matar en OR-grind.
2. Invertera två gånger på varje ledning mellan AND och OR. Två inversioner tar ut varandra, så funktionen är densamma.

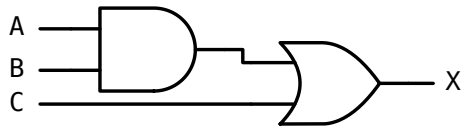
3. Flytta den ena inversionen in i AND-grinden (som då blir NAND), och den andra in i OR-grinden. En OR-grind med inverterade ingångar är, enligt De Morgan, en NAND-grind: `A' + B' = (AB)'`.

Resultatet: **varje AND blir en NAND, och OR-grinden blir en NAND.** En ensam ingång som går direkt till OR-grinden, utan AND framför, måste inverteras, och en inverterare är en NAND med ingångarna ihopkopplade.

För `X = AB + C`:

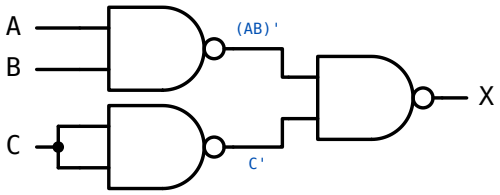
```
X = AB + C = ((AB)' · C')'
```

Kontrollera med De Morgan: `((AB)' · C')' = (AB)'' + C'' = AB + C`. Stämmer.



AND + OR: 2 grindar, 2 kretsar

Figur 4.13 `X = AB + C` med AND och OR: två grindar i två kretsar.



enbart NAND: 3 grindar, 1 krets

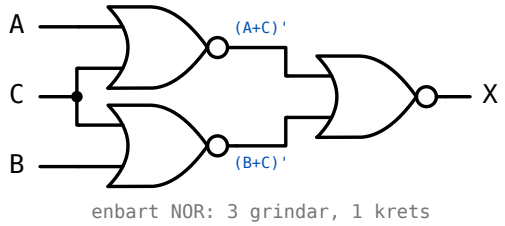
Figur 4.14 Samma funktion med enbart NAND: tre grindar i en krets.

Nätet har en grind mer, men alla tre ryms i **en** 74HC00. AND-OR-versionen kräver en 74HC08 och en 74HC32. Färre kretsar betyder färre matningar att dra och färre ställen att göra fel, och därför är NAND-NAND ofta det bästa valet på ett kopplingsdäck.

Från produkt av summor till NOR-NOR

Samma sak fungerar spegelvänt: ett uttryck på **produkt av summor**-form (avsnitt 4.1.10) blir enbart NOR. Varje OR blir en NOR, och AND-grinden som samlar ihop dem blir en NOR:

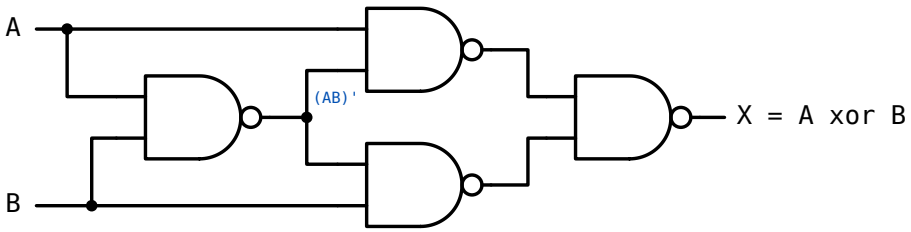
```
X = (A + C)(B + C) = ((A + C)' + (B + C)')'
```



Figur 4.15 Samma funktion med enbart NOR: tre grindar i en krets.

XOR av fyra NAND

En XOR-grind finns som egen krets (74HC86), men den kan också byggas av fyra NAND:



Figur 4.16 XOR byggd av fyra NAND-grindar.

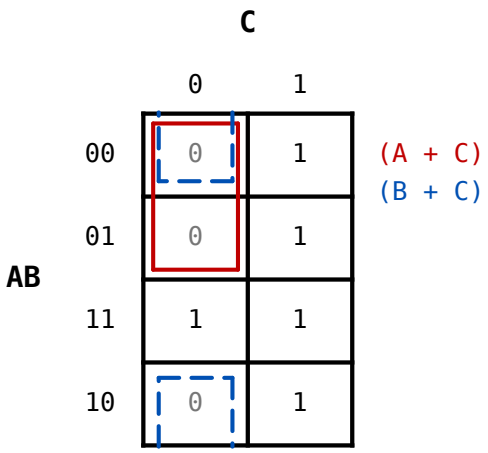
Följ signalerna: den första grinden ger `(AB)'`. Den övre i mitten ger `(A · (AB)')'`, den nedre `(B · (AB)')'`, och den sista kombinerar dem. Sätt in alla fyra kombinationerna av `A` och `B` så ser du att utgången är `1` precis när `A` och `B` skiljer sig åt. Det är en frivillig uppgift i Labb 2-2 (avsnitt B.6).

4.1.10 Produkt av summor

Hittills har grupperna ringat in **ettor**, och svaret har blivit en summa av produkter (SP-form): AND-termer ihop med OR. Man kan lika gärna ringa in **nollor**. Då blir svaret en **produkt av summor** (PS-form): OR-termer ihop med AND.

Varje grupp av nollor läses av som en OR-term, med omvänd regel: en ingång som är `0` i hela gruppen står med som den är, och en ingång som är `1` står inverterad. Tanken är att termen ska bli `0` precis i gruppens rutor.

Samma funktion som i avsnitt 4.1.3, nu med nollorna inringade:



Grupperna ringar in nollor. Varje grupp ger en OR-term:
X = (A + C)(B + C), samma funktion som AB + C.

Figur 4.17 Två grupper av nollor, som ger `X = (A + C)(B + C)`.

Gruppen `AB = 00, 01` i kolumnen `C = 0` har `A = 0` och `C = 0`: termen `(A + C)`, som är 0 bara när både `A` och `C` är 0. Gruppen över kanten, raderna `00` och `10` i samma kolumn, har `B = 0` och `C = 0`: termen `(B + C)`.

```
X = (A + C)(B + C)
```

Multiplicera ut, så får du `AB + AC + BC + C`, och eftersom `AC + BC + C = C` enligt absorptionslagen blir det `AB + C`. Samma funktion, två skrivsätt.

Du behöver PS-formen främst för NOR-nät (avsnitt 4.1.9). För allt annat i boken räcker SP-formen, och den är den du ska välja om inget annat sägs.

4.2 IC-kretsar

4.2.1 Från grind till krets

En logisk grind är ett begrepp. För att kunna koppla upp den behöver du en **IC-krets** (*integrated circuit*): en liten kiselbit med transistorer, ingjuten i en plastkapsel med metallben. Den mest spridda familjen av enkla logikkretsar är **74-serien**, som funnits sedan 1960-talet och fortfarande tillverkas. Varje krets i serien har ett nummer som säger vad den innehåller:

Krets	Innehåll	Används i
74HC00	fyra NAND-grindar med två ingångar	Labb 2-2
74HC02	fyra NOR-grindar med två ingångar	Labb 2-2
74HC04	sex inverterare (NOT)	Labb 2-1
74HC08	fyra AND-grindar med två ingångar	Labb 2-1
74HC32	fyra OR-grindar med två ingångar	Labb 2-1
74HC86	fyra XOR-grindar med två ingångar	–

Namnet har tre delar. **74** betyder standardserien, för normala temperaturer. **HC** är kretsfamiljen, alltså hur transistorerna inuti är byggda: *High-speed CMOS*. **00** är funktionen. En 74HC00 och en 74LS00 innehåller samma fyra NAND-grindar, men i olika teknik.

Tillverkarna lägger ofta till egna bokstäver före och efter. `SN74HC00N` från Texas Instruments och `74HC00N` från Nexperia är samma krets; `N` på slutet betyder här kapseln DIP, den som passar i ett kopplingsdäck. Leta efter `74`, familjen och numret, och bry dig inte om resten.

Två familjer du kommer att möta:

- **HC** (CMOS) arbetar på 2–6 V, drar nästan ingen ström i vila, och är den boken använder.
- **LS** (TTL) är äldre, kräver 5 V, och drar mer ström. En LS-ingång som lämnas okopplad läses som 1, vilket har lärt generationer av elektroniker den dåliga vanan att lämna ingångar okopplade. På en HC-krets är det ett fel (avsnitt 4.2.4).

4.2.2 Kapsel och benplacering

Kretsarna i boken sitter i en **DIP-14**-kapsel: 14 ben i två rader, 2,54 mm mellan benen, så att de passar i kopplingsdäckets hål.

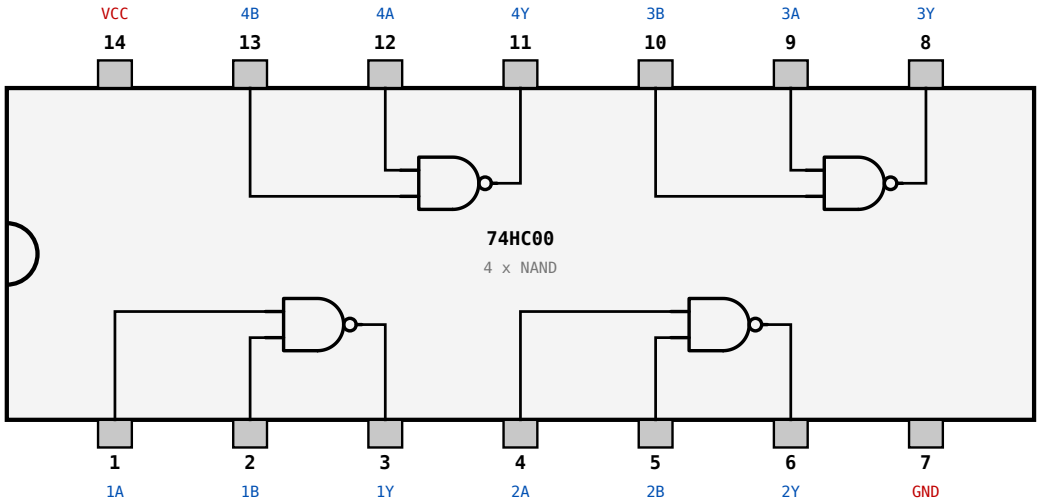
Ben 1 hittar du med hjälp av markeringen. I ena kortsidan finns ett halvrunt **hack**, och ofta också en liten prick vid ben 1. Håll kretsen med hacket åt vänster och texten rättvänd: då sitter ben 1 längst ned till vänster. Benen räknas sedan **moturs, sett ovanifrån**: 1 till 7 längs nederkanten från vänster till höger, och 8 till 14 längs överkanten från höger till vänster.

Alla kretsar i tabellen ovan har matningen på samma ben:

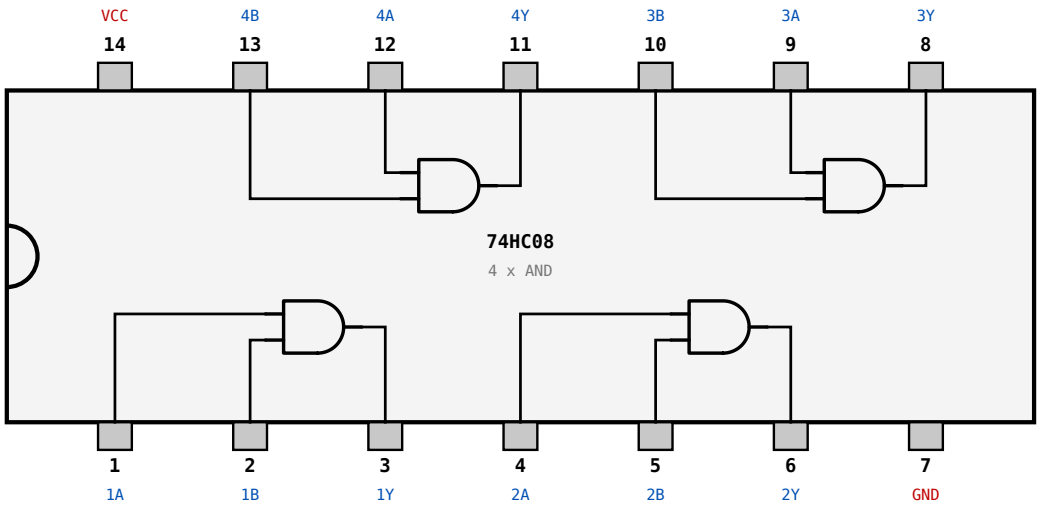
- **ben 14: VCC**, plus 5 V;
- **ben 7: GND**, jord (0 V).

Utan matning fungerar ingenting. Och en krets som sitter felvänd får plus på ben 7 och jord på ben 14: den blir varm, fungerar inte, och kan förstöras. Känn efter med ett finger om en krets blir varm, och bryt matningen direkt om den blir det.

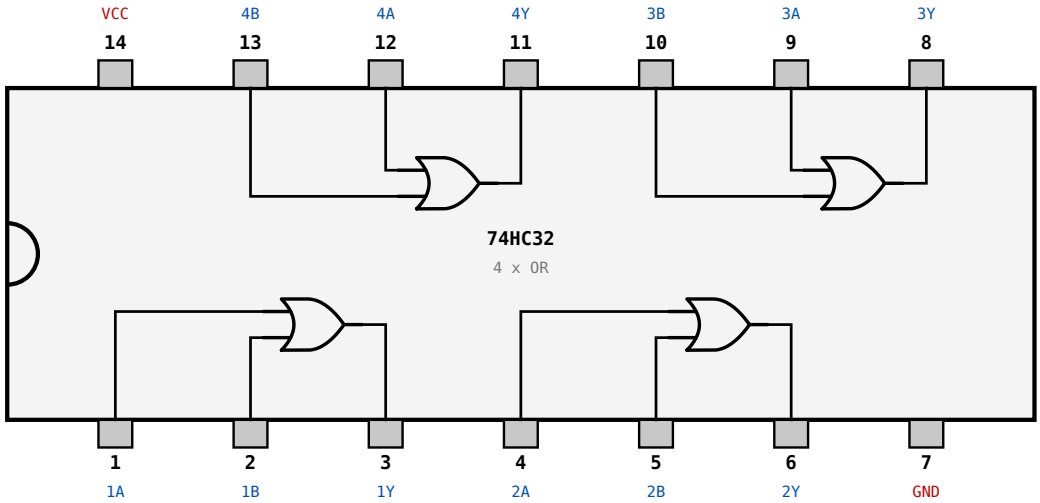
Benplaceringen finns i databladet. Här är de kretsar boken använder, ritade ovanifrån med grindarna inuti:



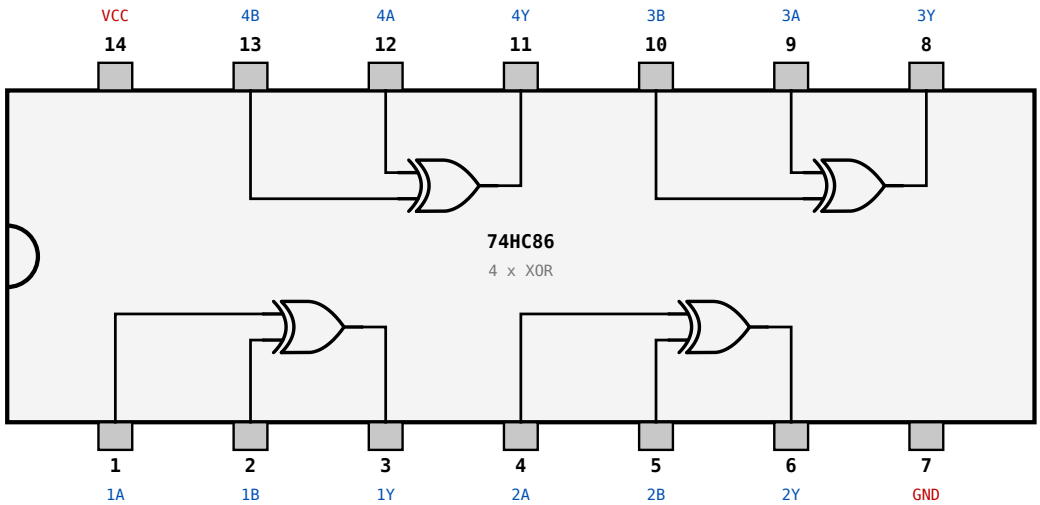
Figur 4.18 Benplacering för 74HC00, fyra NAND-grindar.



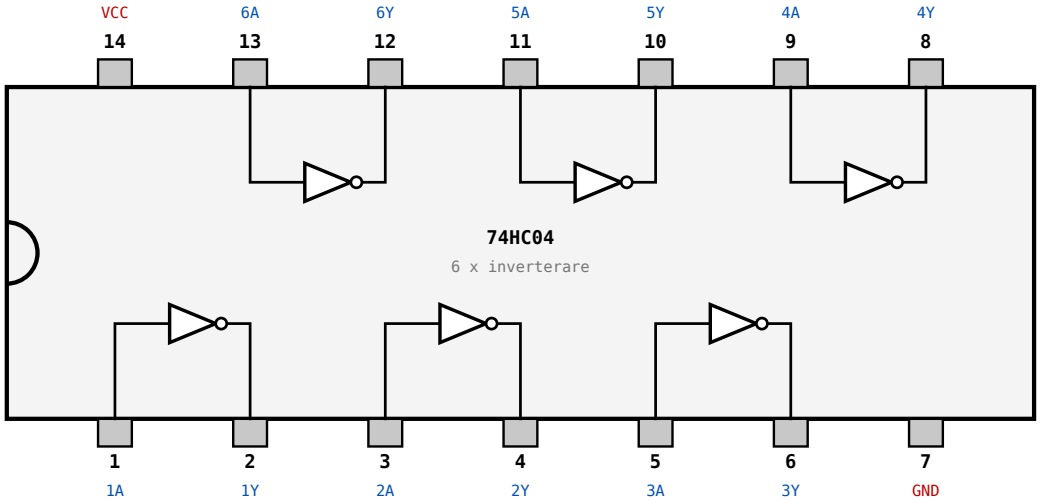
Figur 4.19 Benplacering för 74HC08, fyra AND-grindar: samma benplacering som 74HC00.



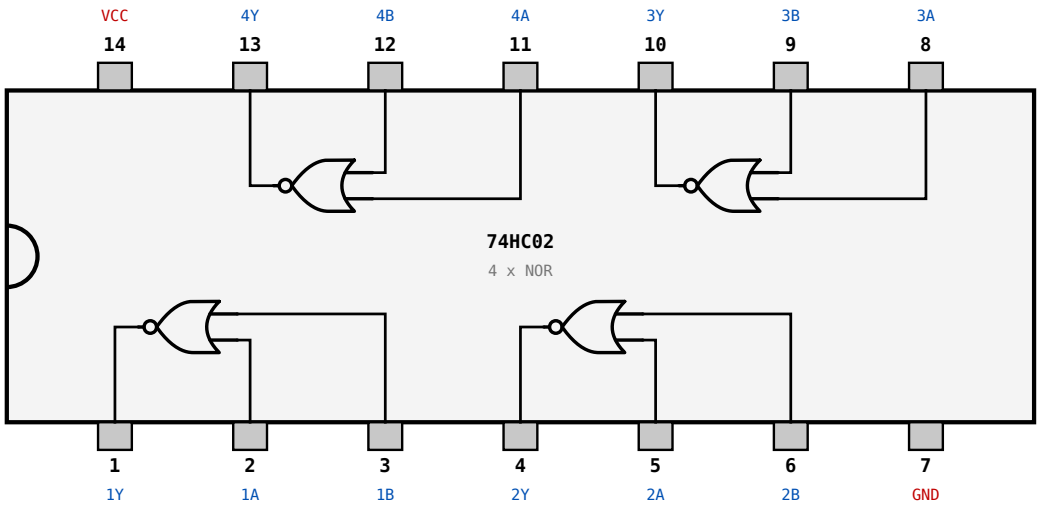
Figur 4.20 Benplacering för 74HC32, fyra OR-grindar: samma benplacering som 74HC00.



Figur 4.21 Benplacering för 74HC86, fyra XOR-grindar: samma benplacering som 74HC00.



Figur 4.22 Benplacering för 74HC04, sex inverterare.



Figur 4.23 Benplacering för 74HC02, fyra NOR-grindar: utgången kommer först i varje grupp.

74HC00, 08, 32 och 86 har samma benplacering. Grind 1 har ingångarna på ben 1 och 2 och utgången på ben 3, grind 2 på ben 4, 5 och 6, och så vidare. Byt en 74HC08 mot en 74HC32 så byts alla fyra AND-grindarna mot OR-grindar, utan att en enda sladd flyttas.

74HC02 är undantaget, och det är den krets flest kopplar fel. I varje grupp om tre ben kommer **utgången först**: ben 1 är utgång och ben 2 och 3 ingångar. Den som kopplar en 74HC02 ur minnet av en 74HC00 kopplar sina insignaler till en utgång. Titta på benplaceringen varje gång.

4.2.3 Databladet

Varje krets har ett **datablad**, ett dokument från tillverkaren som beskriver den i detalj. Sök på kretsens namn och ”datasheet”, och välj en stor tillverkare som Texas Instruments, Nexperia eller onsemi. Ett datablad är ofta 15–20 sidor, men för att koppla upp en krets behöver du bara tre saker ur det:

1. **Benplaceringen** (*pin configuration* eller *pinout*): vilket ben som är vad, som i figurerna ovan.
2. **Funktionstabellen** (*function table*): kretsens sanningstabell, skriven med `H` för hög och `L` för låg, och ibland `X` för ”spelar ingen roll”.
3. **De elektriska nivåerna**: vilka spänningar som räknas som 0 och 1.

Logiknivåer

En digital krets delar in spänningen i tre områden: låg, hög, och ett förbjudet område däremellan. För en 74HC-krets matad med 4,5 V anger databladet:

Storhet	Betydelse	Värde vid VCC = 4,5 V
V_{IH}	lägsta spänning på en ingång som garanterat läses som 1	3,15 V
V_{IL}	högsta spänning på en ingång som garanterat läses som 0	1,35 V
V_{OH}	lägsta spänning en hög utgång ger, vid 4 mA last	3,84 V
V_{OL}	högsta spänning en låg utgång ger, vid 4 mA last	0,33 V

En tumregel för HC-kretsar: under **30 %** av matningen är 0, över **70 %** är 1. Vid 5 V betyder det under 1,5 V och över 3,5 V. En ingång mellan de två gränserna kan läsas som vad som helst, och får kretsen att dra onödigt mycket ström.

Lägg märke till marginalen: en utgång ger minst 3,84 V när den är hög, och en ingång behöver bara 3,15 V. Skillnaden, ungefär 0,7 V, är **brusmarginalen**. Den gör att en störning på upp till 0,7 V på en ledning inte ändrar något. Det är brusimmuniteten från avsnitt 1.1.1, uttryckt i volt.

Gränsvärden

Databladet har också en tabell med **absoluta maxvärden** (*absolute maximum ratings*). De är inte arbetsvärden, utan gränser som förstör kretsen om de överskrids. För 74HC:

- matningen får vara högst 7 V (och kretsen är specificerad för 2–6 V);

- en utgång får leverera högst 25 mA;
- hela kretsen, alla utgångar tillsammans, högst 50 mA genom VCC eller GND.

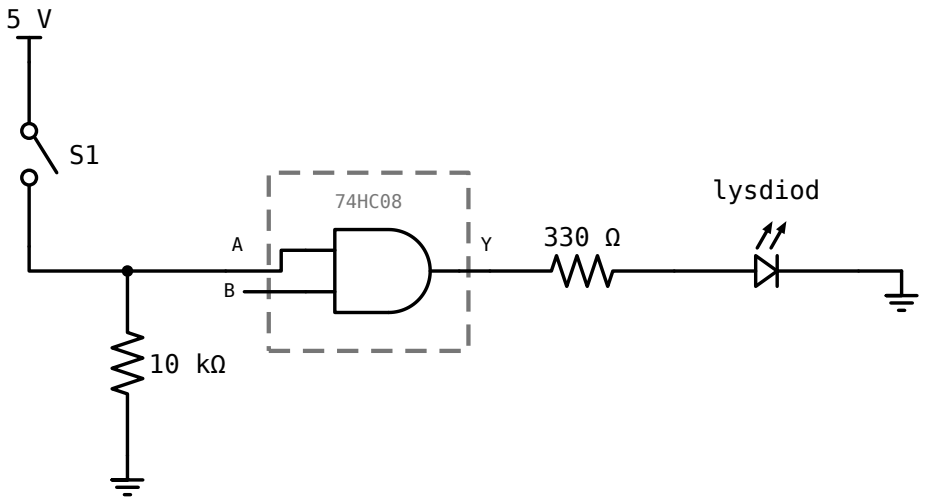
Koppla aldrig 24 V från Labb 1 till en 74HC-krets. Den överlever det inte.

4.2.4 Ingångar: aldrig flytande

En CMOS-ingång drar nästan ingen ström, under en mikroampere. Det låter bra, men det betyder att en ingång som inte är ansluten till något, en **flytande ingång**, inte har någon bestämd spänning alls. Den plockar upp laddning från omgivningen och kan läsas som 0 i ena sekunden och 1 i nästa, beroende på om din hand är nära. Kretsen verkar fungera ibland, och det är den värsta sortens fel.

Regeln: varje ingång ska vara ansluten till något. Det gäller också ingångarna på grindar du inte använder: koppla dem till GND (eller VCC).

En strömbrytare eller tryckknapp ger bara en förbindelse när den är sluten. När den är öppen är ingången flytande, om inte ett motstånd bestämmer nivån:



Släppt S1: motståndet drar A till 0 V (logisk 0). Tryckt S1: A = 5 V (1).
Ingång B kopplas likadant, med en egen knapp och ett eget motstånd.
Motståndet på utgången begränsar strömmen genom lysdioden.

Figur 4.24 Tryckknapp med pull-down-motstånd på en grindingång, och lysdiod med förkopplingsmotstånd på utgången.

- Knappen sitter mellan **5 V** och ingången.
- Ett motstånd på **10 kΩ** sitter mellan ingången och **jord**. Det kallas ett **pull-down-motstånd**: det drar ned ingången till 0 när knappen är öppen.

- När knappen trycks ned kopplar den ingången direkt till 5 V. Motståndet ligger då mellan 5 V och jord och leder 0,5 mA, men det spelar ingen roll; ingången ser 5 V och läser 1.

Det omvända, knappen mot jord och motståndet mot 5 V, kallas ett **pull-up-motstånd** och ger omvänd logik: nedtryckt knapp läses som 0. Det är den kopplingen mikrokontrollern använder i kapitel 10, där motståndet dessutom sitter inbyggt i kretsen.

Kopplingsdäckets stationer i Labb 2 har strömbrytare som redan ger 0 eller 5 V. Då behövs inga egna motstånd, men regeln om ingångar du inte använder gäller fortfarande.

4.2.5 Utgångar: lysdiod med förkopplingsmotstånd

För att se vad en utgång gör kopplar du en **lysdiod** till den. En lysdiod är en diod som lyser när ström går igenom den i rätt riktning: från det långa benet (**anoden**) till det korta (**katoden**, som också är märkt med en platt kant på kapseln).

En lysdiod måste alltid ha ett **förkopplingsmotstånd** i serie. Över en röd lysdiod ligger ungefär 2 V när den lyser, nästan oberoende av strömmen, så utan motstånd skulle strömmen bara begränsas av hur mycket utgången orkar leverera, och det är mer än lysdioden tål.

Motståndet räknas ut med Ohms lag. Resten av spänningen, det som inte ligger över lysdioden, ska ligga över motståndet:

$$R = \frac{U_{\text{matning}} - U_{\text{lysdiod}}}{I}$$

Med 5 V matning, en röd lysdiod på 2 V och en önskad ström på 10 mA:

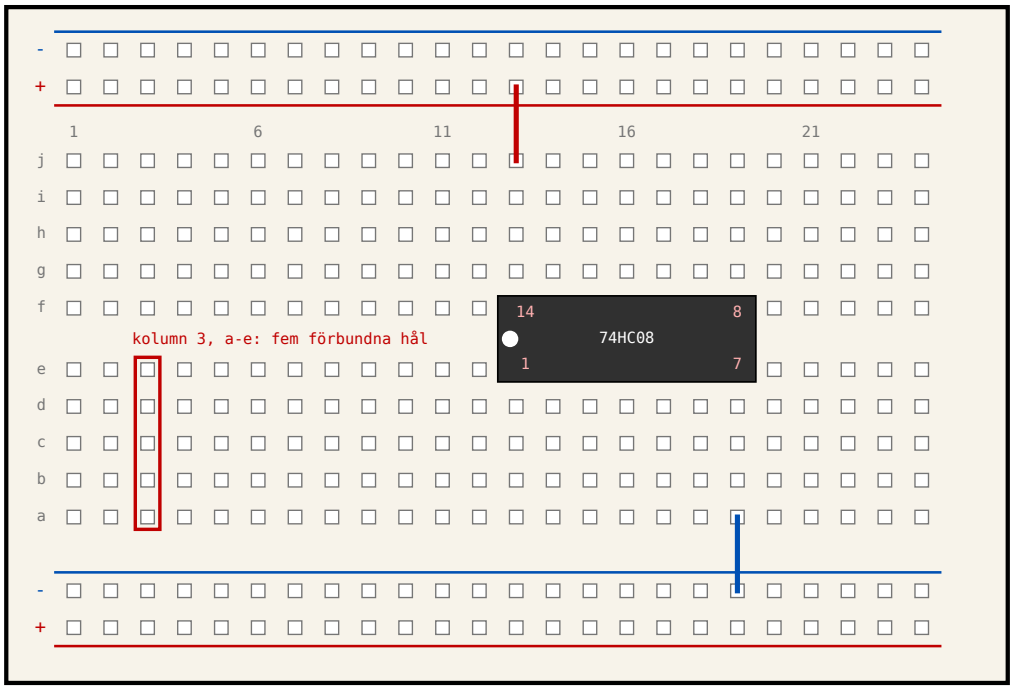
$$R = \frac{5 - 2}{0{,}010} = 300\ \Omega$$

Välj närmaste standardvärde uppåt, **330 Ω**, som ger 9 mA. En lysdiod lyser tydligt redan vid några milliampere, så 330–470 Ω är ett bra val till en 74HC-utgång. Håll strömmen under 10 mA: då ligger utgången väl inom sina gränser, och en hög utgång håller sin spänning.

Lysdioden kopplas från utgången, genom motståndet, till jord. Då lyser den när utgången är **1**. Vänd den, eller koppla den mot 5 V i stället för mot jord, så lyser den när utgången är 0: det fungerar också, men gör varje avläsning baklänges.

4.2.6 Kopplingsdäcket

Ett **kopplingsdäck** (*breadboard*) låter dig koppla upp en krets utan att löda. Under hålen finns metallfjädrar som förbinder hålen i ett bestämt mönster:



Matningsskenorna går längs hela kanten; hålen i mitten är förbundna fem och fem i kolumner. Kretsen sitter över mittspåret, så att varje ben får en egen kolumn. Ben 14 går till +5 V och ben 7 till jord.

Figur 4.25 Kopplingsdäck med matningsskenor, femhålskolumner och en IC-krets över mittspåret.

- **Matningsskenorna** längs kanterna, märkta + (röd linje) och - (blå linje), är förbundna längs hela kanten. Anslut 5 V och jord till dem en gång, så har du matning överallt.
- **Hålen i mitten** är förbundna **fem och fem i kolumner**, a–e på ena sidan av mittspåret och f–j på den andra. Allt som sitter i samma kolumn på samma sida är kopplat ihop.
- **Mittspåret** skiljer de två halvorna åt. En IC-krets sätts **över** mittspåret, så att varje ben hamnar i en egen kolumn, och du får fyra lediga hål bredvid varje ben att koppla till.

Tre vanliga fel, som alla ger en krets som ”nästan” fungerar:

- Två ledningar i **samma kolumn** som inte skulle vara ihopkopplade. De är det nu.
- En krets som **inte** sitter över mittspåret: då kortsluts benen parvis.
- Matningsskenor som är **delade på mitten**. På en del kopplingsdäck går skenan inte hela vägen, vilket syns som ett avbrott i den röda eller blå linjen. Koppla i så fall ihop halvorna.

Använd gärna **röd tråd för 5 V och svart eller blå för jord**, och korta trådar som ligger platt mot däcket. En uppkoppling som går att följa med blicken går också att felsöka.

Sätt en **avkopplingskondensator** på 100 nF mellan VCC och GND, så nära varje krets som möjligt. När en grind slår om drar den en kort strömpuls, och kondensatorn levererar den lokalt, i stället för att pulsen ska störa matningen till resten av kretsarna. I små uppkopplingar som i Labb 2 fungerar det ofta utan, men det är god vana, och i en verklig konstruktion är den inte valfri.

4.2.7 Att planera en uppkoppling

Den som börjar koppla direkt från ett grindnät tappar bort sig efter tredje sladden. Planera i stället i tre steg, med papper och penna.

1. Räkna grindarna, välj kretsar. Räkna hur många grindar av varje sort nätet behöver, och hur många kretsar det blir. Varje 74HC08, 32, 00, 02 och 86 har fyra grindar, 74HC04 har sex.

2. Gör en grindtilldelning. Bestäm vilken grind i vilken krets som gör vad, och skriv ut benens nummer. Kretsarna får beteckningarna U1, U2 och så vidare. För `X = AB + C`:

Grind i nätet	Krets	Grind i kretsen	Ingångar (ben)	Utgång (ben)
AND: `AB`	U1 = 74HC08	grind 1	A → 1, B → 2	3
OR: `AB + C`	U2 = 74HC32	grind 1	U1 ben 3 → 1, C → 2	3 → X

3. Gör en kopplingslista. En rad per ledning, i den ordning du kopplar. Börja alltid med matningen:

Nr	Från	Till
1	U1 ben 14	+5 V
2	U1 ben 7	GND
3	U2 ben 14	+5 V
4	U2 ben 7	GND
5	strömbrytare A	U1 ben 1
6	strömbrytare B	U1 ben 2
7	U1 ben 3	U2 ben 1
8	strömbrytare C	U2 ben 2
9	U2 ben 3	lysdiod X
10	oanvända ingångar (U1 ben 4, 5, 9, 10, 12, 13; U2 ben 4, 5, 9, 10, 12, 13)	GND

Bocka av varje rad när ledningen sitter. Om något inte fungerar har du nu en lista att kontrollera rad för rad, och det är precis vad felsökningen i avsnitt 5.1 bygger på.

4.2.8 Förbered Labb 2

Labb 2 (bilaga B) hör till kapitel 5, och du hinner inte klart om förberedelserna görs vid labbplatsen. Före labben ska du ha:

- läst hela labbhandledningen, och tagit fram sanningstabeller, Karnaughdiagram och minimerade uttryck för varje uppgift;
- ritat grindnätet för varje uppgift, och byggt och testat det i CircuitVerse;
- gjort en grindtilldelning och en kopplingslista (avsnitt 4.2.7) för varje uppgift;
- för Labb 2-2: gjort om näten till enbart NAND (avsnitt 4.1.9), och gjort grindtilldelningen för 74HC00.

Ta med allt på papper eller i datorn. Förberedelserna är en del av redovisningen.

4.3 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Rita ett Karnaughdiagram för en funktion med två, tre eller fyra ingångar, med rubrikerna i Graykod.
- Ringa in grupper av ettor efter reglerna, också över kanter och i hörn, och läsa av ett minimerat uttryck på summa av produkter-form.
- Använda don't care för kombinationer som aldrig förekommer.
- Syntetisera ett kombinatoriskt nät från en beskrivning i ord: sanningstabell, Karnaughdiagram, minimerat uttryck, grindnät, kontroll.
- Göra om ett nät till enbart NAND-grindar eller enbart NOR-grindar.
- Hitta ben 1, VCC, GND och varje grinds ben på en 74HC-krets, med hjälp av databladet.
- Räkna ut ett förkopplingsmotstånd för en lysdiod, och förklara varför en ingång aldrig får lämnas flytande.
- Planera en uppkoppling med en grindtilldelning och en kopplingslista.

Frågor att testa dig själv med

1. Varför måste rubrikerna i ett Karnaughdiagram stå i Graykod? Vad går fel med vanlig binär ordning?
2. En grupp om fyra rutor i ett diagram med fyra ingångar: hur många ingångar blir kvar i termen?
3. Ett diagram där ettorna bildar ett schackmönster går inte att förenkla. Vilken grind beräknar en sådan funktion?
4. När får en X-ruta ingå i en grupp, och måste den täckas?
5. Ett NAND-nät har ofta fler grindar än AND-OR-nätet för samma funktion. Varför kan det ändå vara billigare att bygga?
6. Vilket ben är VCC och vilket är GND på en 74HC08? Och på en 74HC02? Vad skiljer 74HC02 från de andra?
7. Varför är en okopplad ingång på en HC-krets ett fel, även om kretsen verkar fungera?

Nästa kapitel

Kapitel 5 hör ihop med Labb 2 (bilaga B):

- **Labb 2-1**: grindnät med AND-, OR- och inverterarkretsar på kopplingsdäck.
- **Labb 2-2**: samma nät med enbart NAND och enbart NOR.
- Felsökning av ett grindnät som inte gör vad det ska, systematiskt i stället för på måfå.

4.4 Övningar

> **Så kontrollerar du ditt arbete.** Varje uttryck du tar fram med ett Karnaughdiagram kan du kontrollera på två sätt, och du ska göra minst ett av dem varje gång: sätt in varje rad i sanningstabellen i ditt uttryck och se att det ger rätt utgång, eller bygg nätet i CircuitVerse och gå igenom alla kombinationer av ingångarna. Ett uttryck som inte är kontrollerat är en gissning.
> Lösningsförslagen finns i bilaga I. Gör varje övning innan du läser lösningen. Där det finns ett vanligt felsvar visar lösningen det också, och varför det är fel.
> Varje övning är märkt med sin sort. Kontrolluppgiften är övning 4.12.

Övning 4.1 Graykod och grannar

FÖRSTÅELSE

a) Varför skrivs rubrikerna i ett Karnaughdiagram i ordningen `00, 01, 11, 10` och inte `00, 01, 10, 11`? Svara med en mening om vad två grannrutor måste ha gemensamt.

b) I ett diagram med fyra ingångar, `ABCD`: vilka fyra rutor är grannar till ruta 5 (`ABCD = 0101`)? Ange dem som mintermnummer. Använd figuren med rutornas nummer i avsnitt 4.1.2 (figur 4.3) om du vill, men kontrollera svaret genom att se att varje granne skiljer sig från 0101 i exakt en bit.

c) Samma fråga för ruta 0. Två av grannarna ligger på andra sidan en kant. Vilka?

d) Varför får en grupp inte bestå av tre rutor, ens när tre ettor ligger på rad?

Övning 4.2 Två ingångar

RÄKNA FÖR HAND

A	B	X
0	0	0
0	1	1
1	0	0
1	1	1

a) Rita Karnaughdiagrammet och ringa in grupperna.

b) Läs av det minimerade uttrycket. Vad säger det om ingången `A`?

c) Läs av uttrycket direkt ur tabellen, som i avsnitt 2.1.6, och förenkla det algebraiskt. Får du samma svar?

Övning 4.3 Tre ingångar

RÄKNA FÖR HAND

ABC	X
000	1
001	1
010	0
011	0
100	1
101	1
110	0
111	0

a) Fyll i diagrammet. Två av raderna med ettor ligger inte bredvid varandra på papperet. Är de grannar ändå?

b) Ringa in och läs av det minimerade uttrycket.

c) Hur många grindar behövs för det minimerade nätet, och hur många hade den direkta avläsningen ur tabellen krävt?

Övning 4.4 Från mintermer

RÄKNA FÖR HAND

`X` har tre ingångar `A`, `B`, `C` och är `1` för mintermerna 0, 2, 5 och 7.

a) Skriv sanningstabellen, och fyll i diagrammet.

b) Ringa in och läs av det minimerade uttrycket.

c) Uttrycket beror bara på två av ingångarna. Vilken grind från kapitel 2 beräknar det med en enda grind?

Övning 4.5 Två utgångar

RÄKNA FÖR HAND

En krets har fyra ingångar `ABCD` och två utgångar `X` och `Y`:

ABCD	XY	ABCD	XY
0000	01	1000	11
0001	00	1001	10
0010	00	1010	10
0011	01	1011	11
0100	11	1100	01
0101	10	1101	00
0110	10	1110	00
0111	11	1111	01

a) Rita ett diagram för `X` och ett för `Y`, och läs av båda uttrycken.

b) Två av ingångarna påverkar inte `X` alls, och två andra påverkar inte `Y`. Vilka? Hur syns det i diagrammen?

c) Båda uttrycken kan skrivas med en enda grind var. Vilka grindar?

Övning 4.6 Hörnen

RÄKNA FÖR HAND

`X` har fyra ingångar och är `1` för mintermerna 0, 2, 8, 10 och 15.

a) Fyll i diagrammet, ringa in och läs av.

b) Minterm 15 får ingen granne att gå ihop med. Är det ett fel i din gruppering? Motivera.

Övning 4.7 Segment e i en sifferdisplay

RÄKNA FÖR HAND

En 7-segmentsdisplay visar en BCD-siffra 0–9 som kommer in på fyra ledningar `ABCD` (avsnitt 1.2.5). Segment **e**, det nedre vänstra, ska lysa för siffrorna 0, 2, 6 och 8, och vara släckt för 1, 3, 4, 5, 7 och 9. Kombinationerna 10–15 förekommer aldrig.

a) Fyll i diagrammet med ettor, och med X för 10–15.

b) Ringa in och läs av ett minimerat uttryck för `e`. Använd X-rutorna där de gör grupperna större.

c) Kontrollera ditt uttryck för alla tio siffrorna.

d) Vad visar segment e om kretsen ändå får `1010` på ingången? Är det ett problem?

Övning 4.8 Fläkten

KONSTRUKTION

En ventilationsfläkt `F` styrs av fyra signaler:

- `T`: temperaturen är hög;
- `W`: fönstret är öppet;
- `C`: koldioxidhalten är hög;
- `B`: brandlarmet har löst ut.

Fläkten ska gå om temperaturen är hög och fönstret är stängt, eller om koldioxidhalten är hög. Men när brandlarmet har löst ut ska fläkten **aldrig** gå, eftersom den då skulle mata elden med luft.

a) Skriv sanningstabellen, med ingångarna i ordningen `T W C B`.

b) Fyll i Karnaughdiagrammet, med `TW` nedåt och `CB` åt sidan, och läs av ett minimerat uttryck.

c) Rita grindnätet.

d) Gör en grindtilldelning (avsnitt 4.2.7): vilka kretsar ur 74HC08, 74HC32 och 74HC04 behövs, och hur många av varje?

Övning 4.9 Enbart NAND

KONSTRUKTION

`X = AB' + C`.

a) Rita nätet med AND, OR och NOT.

b) Gör om det till enbart NAND-grindar, enligt avsnitt 4.1.9. Glöm inte inverteringen av `B`, och ingången `C` som går direkt till OR-grinden.

c) Kontrollera med De Morgan att ditt NAND-uttryck är lika med `AB' + C`.

d) Hur många NAND-grindar behövs, och hur många 74HC00?

Övning 4.10 Enbart NOR

KONSTRUKTION

`X = (A + B)C`.

a) Vilken form är uttrycket på, SP eller PS?

b) Gör om det till enbart NOR-grindar. Rita nätet.

c) Kontrollera ditt nät för alla åtta kombinationerna av `A`, `B` och `C`.

Övning 4.11 Kretsar och ben

FÖRSTÅELSE

Använd benplaceringarna i avsnitt 4.2.2.

a) Vilka ben har grind 3 i en 74HC00: ingångarna och utgången? Samma fråga för en 74HC02.

b) Ett nät behöver sex AND-grindar och en inverterare. Hur många kretsar av vilka sorter?

c) En grön lysdiod har spänningsfallet 2,1 V och ska drivas med ungefär 5 mA från en utgång på 5 V. Räkna ut förkopplingsmotståndet och välj ett standardvärde ur E12-serien (…, 330, 390, 470, 560, 680, 820, 1000 Ω). Vilken ström blir det?

d) En labbkamrat har lämnat ingångarna på de oanvända grindarna okopplade, och säger att det fungerar. Förklara varför det ändå är fel, och vad som kan hända.

Övning 4.12 Kontroll: minimerat och ominimerat nät

KONTROLL

Räkna för hand, bygg och jämför, förklara.

`X` har fyra ingångar och är `1` för mintermerna 1, 3, 5, 7, 9 och 11.

a) För hand

Skriv den direkta avläsningen ur tabellen: sex AND-termer med fyra ingångar var. Minimera sedan med ett Karnaughdiagram.

b) Förutsäg

Hur många grindar och hur många kretsar behövs för vart och ett av de två uttrycken? Skriv ned siffrorna innan du bygger något.

c) Bygg båda i CircuitVerse

Bygg dem med samma fyra ingångar, och varsin utgång. Gå igenom alla sexton kombinationerna och anteckna båda utgångarna.

d) Jämför

Utgångarna ska vara lika på varje rad. Om de skiljer sig på någon rad: vilken rad, och vad säger det om vilket av uttrycken som är fel?

e)

Det minimerade uttrycket kan skrivas `D(A' + B')`, och `A' + B'` är enligt De Morgan `(AB)'`. Hur få grindar och kretsar klarar du dig med då? Jämför med ditt svar på b).

Övning 4.13 Multiplexern

KONSTRUKTION FÖRDJUPNING

En **multiplexer** väljer vilken av två signaler som ska släppas fram. Den har två dataingångar `A` och `B`, en väljare `S` och en utgång `X`: när `S = 0` ska `X = B`, och när `S = 1` ska `X = A`.

a) Skriv sanningstabellen med ingångarna i ordningen `S A B`.

b) Minimera med ett Karnaughdiagram, med `SA` nedåt och `B` åt sidan.

c) Rita nätet, och förklara med egna ord varför högst en av de två AND-termerna kan vara 1 åt gången.

KAPITEL 5

Felsökning av grindnät

Förutsäg, mät, jämför: från uttryck till fungerande krets på kopplingsdäcket.

I DET HÄR KAPITLET

- Kopplingsdäcket, kretsens riktning och matningen.
- **Labb 2-1**: grindnät med AND-, OR- och inverterarkretsar, från minimerat uttryck till fungerande krets.
- **Labb 2-2**: samma sorts nät med enbart NAND, och en titt på NOR.
- Felsökning: förutsäg, mät, jämför, i stället för att flytta sladdar på måfå.

I kapitel 4 minimerade du grindnät med Karnaughdiagram och lärde dig läsa en benplacering. Det här kapitlet hör ihop med Labb 2, vars handledning finns i bilaga B: där kopplar du upp näten med 74HC-kretsar på ett kopplingsdäck, först med AND, OR och inverterare i Labb 2-1 och sedan med enbart NAND i Labb 2-2. Kapitlets eget ämne är vad du gör när ett nät inte fungerar: hur du hittar felet systematiskt, med sanningstabellen och en multimeter, i stället för att flytta sladdar på måfå.

Förbered dig innan labben. Gör alla förberedelser F1–F5 i labbhandledningen: sanningstabeller, Karnaughdiagram, NAND-nät, simulering i CircuitVerse och en kopplingslista för varje uppgift. De är en del av redovisningen, och utan dem hinner du inte klart. Repetera avsnitt 4.2 om benplaceringen, logiknivåerna och kopplingsdäcket, och läs avsnitt 5.1 om felsökning.

I labben gör du uppgift 1–4 i Labb 2-1 och uppgift 1–3 i Labb 2-2, och redovisar varje uppgift när den är klar; uppgift 4 i Labb 2-2 är frivillig. Övningarna i avsnitt 5.3 är till största delen felsökningsfall. Kontrolluppgiften, övning 5.10, bygger på mätningen i Labb 2-1 uppgift 1. Lösningsförslagen finns i bilaga I.

5.1 Felsökning av grindnät

5.1.1 Varför felsöka systematiskt

Förr eller senare kopplar du upp ett nät som inte gör vad det ska. Det är inget misslyckande, det händer alla, och det är ingen slump heller: ett nät med tre kretsar och tjugo sladdar har tjugo ställen där en sladd kan sitta i fel hål, och du behöver bara en.

Den vanligaste reaktionen är att börja flytta sladdar, trycka till kretsar och prova igen. Ibland fungerar det, och då vet du inte varför, vilket betyder att felet kan komma tillbaka. Oftast gör det saken värre, eftersom en ny felkoppling läggs till den gamla.

Det finns ett bättre sätt, och det bygger på samma idé som resten av boken: **förutsäg, mät, jämför**. Du vet vad varje ledning ska ha för värde för en viss kombination av ingångarna, eftersom du har sanningstabellen och grindnätet. Mät vad den faktiskt har. Det första stället där mätt och förväntat skiljer sig är felet, eller precis bredvid det.

Figur 5.1 visar arbetsgången i en figur, och resten av det här avsnittet går igenom stegen ett i taget.

5.1.2 Att mäta en logisk nivå

Allt du behöver är en **multimeter** inställd på likspänning (V med ett rakt streck, eller `DCV`), med ett mätområde på 20 V om den inte väljer själv.

- Sätt den **svarta** mätsladden på **jord**, till exempel matningsskenan märkt `-`. Den får sitta kvar där hela tiden.
- Peka med den **röda** mätsladden på det ben eller den ledning du vill mäta.

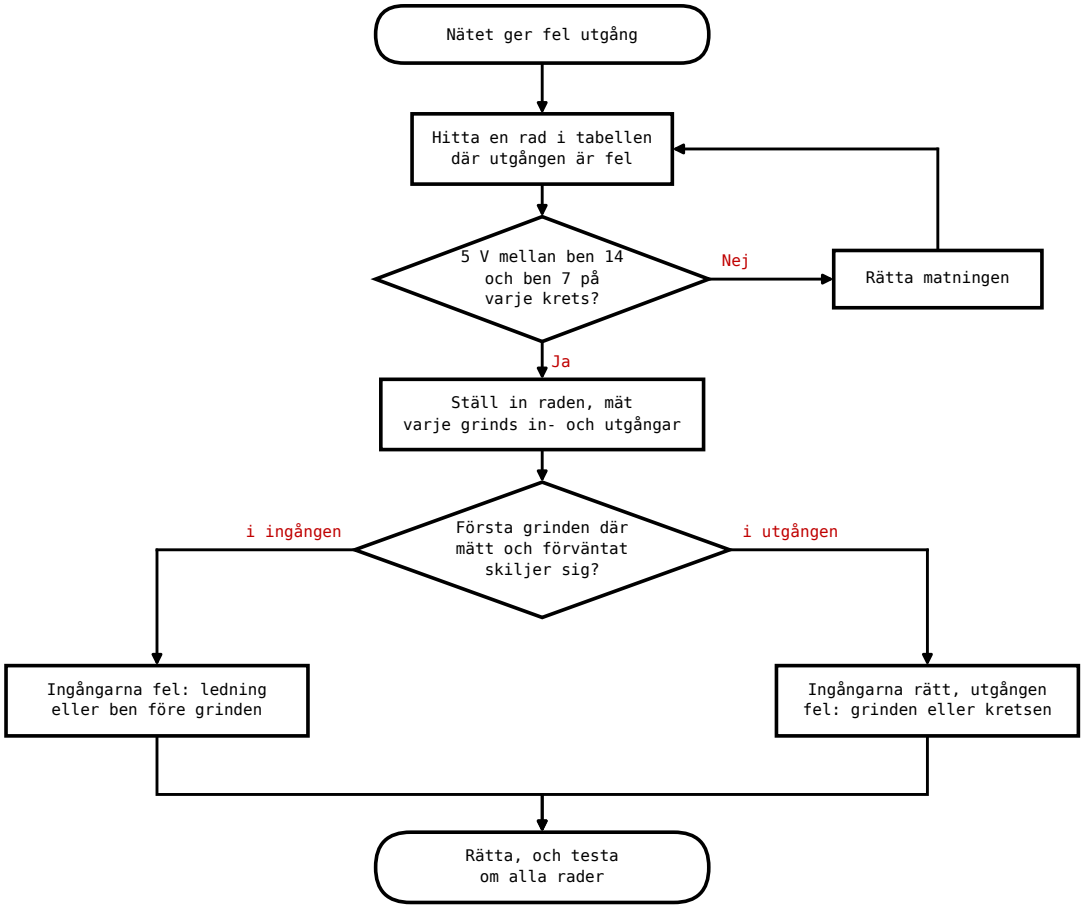
Tolka värdet med logiknivåerna från avsnitt 4.2.3:

Mätt spänning	Betyder
under 1,5 V	logisk **0**
över 3,5 V	logisk **1**
mellan 1,5 och 3,5 V	**fel**: ingen giltig nivå

En giltig utgång ligger nästan alltid mycket nära 0 V eller 5 V: under 0,3 V när den är låg och över 4 V när den är hög, även med en lysdiod ansluten. Ett värde mitt emellan betyder nästan alltid något av tre saker:

- en **flytande ingång** (avsnitt 5.1.7), som kan visa vad som helst och ofta ändrar sig när du rör handen;
- **två utgångar** som är kopplade mot varandra, där den ena driver hög och den andra låg;
- en **krets utan matning**, där ingenting ligger på en bestämd nivå.

En lysdiod på utgången är en utmärkt indikator, men den visar bara slutresultatet. Multimetern låter dig titta på varje ledning i nätet, och det är vad felsökning kräver.



Figur 5.1 Flödesplan för felsökning av ett grindnät.

5.1.3 Börja med matningen

Mät alltid matningen först, på varje krets: röd mätsladd på ben 14, svart på ben 7. Du ska se ungefär 5 V.

Det låter onödigt, och det är det inte. En krets utan matning ger inte ett tydligt fel; den ger konstiga, halvt fungerande resultat, eftersom den kan få en del ström genom sina ingångar. Den enskilt vanligaste orsaken till att ett nät ”nästan” fungerar är en krets där ben 7 eller ben 14 sitter i fel kolumn, eller en matningsskena som är delad på mitten av kopplingsdäcket.

Mät mellan benen, inte mellan skenorna: det är spänningen **på kretsen** som räknas. En skena med 5 V hjälper inte om sladden till ben 14 sitter i kolumnen bredvid.

Om kretsen **blir varm**: bryt matningen direkt. En varm 74HC-krets är nästan alltid felvänd, eller har en utgång som är kortsluten mot matningen eller mot en annan utgång.

5.1.4 Följ signalen

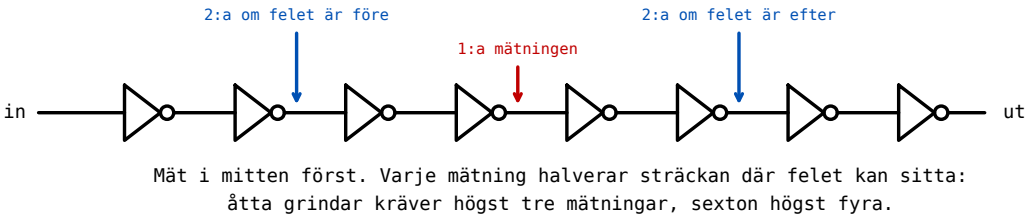
När matningen är kontrollerad:

1. **Välj en rad i sanningstabellen där utgången blir fel**, och ställ in ingångarna på den. Använd inte vilken rad som helst; använd en rad där du vet att resultatet är fel.
2. **Skriv upp vad varje ledning i nätet ska ha för värde** för just den raden. Gå från ingångarna genom varje grind fram till utgången.
3. **Mät grind för grind**, i samma ordning: först ingångarna på den första grinden, sedan dess utgång, sedan nästa grind.
4. **Stanna vid den första grinden där mätt och förväntat skiljer sig.** Då finns två fall:
   - **En ingång är fel.** Felet sitter före grinden: i ledningen dit, i den föregående grindens utgång, eller i att sladden sitter i fel kolumn. Mät i andra änden av ledningen.
   - **Alla ingångar är rätt men utgången är fel.** Då är det grinden: fel ben (du mäter på en annan grind än du tror), fel krets (en 74HC32 där det skulle sitta en 74HC08), eller i sällsynta fall en trasig krets.

Kom ihåg att mäta **på kretsens ben**, inte bara på sladden du tror går dit. Poängen är att hitta skillnaden mellan det du tror att du har kopplat och det du faktiskt har kopplat.

5.1.5 Halvering

I ett nät med många grindar i rad går det snabbare att börja i mitten än i början:



Figur 5.2 Halvering i en kedja av åtta inverterare.

Mät efter halva kedjan. Om värdet där är rätt sitter felet i den andra halvan, annars i den första. Mät sedan i mitten av den halva som återstår, och så vidare. Varje mätning **halverar** området där felet kan sitta, så åtta grindar kräver högst tre mätningar och sexton högst fyra, i stället för sju respektive femton om du börjar från början och mäter efter varje grind.

I ett litet nät som i Labb 2, med tre eller fyra grindar, är det ingen stor vinst. Men metoden gäller för allt som går att dela i två: en lång kabel, ett program, en maskin med många steg. Det är den viktigaste felsökningsmetoden som finns, och den kommer tillbaka i bokens mikrodatordel, när ett program ska felsökas i simulatorn.

5.1.6 Läs felet ur sanningstabellen

Innan du mäter något alls kan tabellen berätta mycket. Jämför den uppmätta tabellen med den förväntade, och titta på **vilka rader** som blir fel:

- **Utgången ändras aldrig**, den är alltid 0 eller alltid 1: utgången är inte kopplad till det du tror, kretsen saknar matning, eller lysdioden sitter fel.
- **Utgången är inverterad** på varje rad: en NAND där det skulle vara en AND, eller en lysdiod kopplad mot 5 V i stället för mot jord.
- **Utgången beror inte alls på en viss ingång**: den ingången når inte fram. Sladden sitter i fel kolumn, eller går till en oanvänd grind.
- **Bara några rader fel**, och de har något gemensamt: de pekar ut en grind. Om `X = AB + C` blir fel precis när `C = 0` och `A = B = 1`, är det AND-grindens bidrag som saknas, och den ska mätas först.

Det sista fallet är värt att öva på, för det sparar mest tid. Frågan att ställa är: **vilken del av uttrycket är det enda som kan göra de här raderna 1?**

5.1.7 Vanliga fel

De fel du med störst sannolikhet kommer att göra, och hur de ser ut:

Symptom	Trolig orsak	Kontrollera
Ingenting fungerar alls	Matningen saknas	Mät mellan ben 14 och ben 7
Kretsen blir varm	Kretsen felvänd, eller en utgång kortsluten	Bryt matningen; kontrollera hacket och ben 1
Utgången flimrar eller ändras när du rör handen	Flytande ingång	Mät alla ingångar, också på oanvända grindar
En ingång påverkar ingenting	Sladden i fel kolumn eller på fel grind	Mät på kretsens ben, inte på sladden
Mätt värde mellan 1,5 och 3,5 V	Flytande ingång eller två utgångar mot varandra	Följ ledningen, hitta vad som driver den
Lysdioden lyser aldrig, fast utgången är 5 V	Lysdioden felvänd	Långa benet mot utgången
74HC02 fungerar inte alls	Kopplad som en 74HC00	Benplaceringen: utgången först
Kretsen fungerar ibland	Glappkontakt, eller en flytande ingång	Tryck till sladdarna; mät igen

5.1.8 Ett felsökningsexempel

Ett par i Labb 2-1 har kopplat upp `X = AB + C` med en 74HC08 och en 74HC32. De går igenom tabellen och får:

A	B	C	X, förväntat	X, mätt
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	1	1
1	1	0	1	**0**
1	1	1	1	1

Läs tabellen först (avsnitt 5.1.6). Bara raden `110` är fel. Det är den enda raden där `X` ska bli 1 tack vare `AB` och inte tack vare `C`. Så `C` når fram genom OR-grinden, men `AB` gör det inte. Misstanken faller på AND-grinden eller ledningen från den.

Matningen (avsnitt 5.1.3). Ben 14 till ben 7 på båda kretsarna: 5,0 V. Den är inte felet.

Följ signalen (avsnitt 5.1.4), för raden `ABC = 110`:

Mätpunkt	Förväntat	Mätt
U1 ben 1 (A)	1	4,9 V: 1
U1 ben 2 (B)	1	4,9 V: 1
U1 ben 3 (AB)	1	4,8 V: 1
U2 ben 1 (AB)	1	**0,0 V: 0**
U2 ben 2 (C)	0	0,0 V: 0
U2 ben 3 (X)	1	0,0 V: 0

AND-grindens utgång är rätt, men OR-grindens ingång är fel. Felet sitter **i ledningen mellan dem**. En titt på kopplingsdäcket visar att sladden från U1 ben 3 sitter i kolumnen **bredvid** U2 ben 1, i en tom kolumn. Den flyttas ett hål, och alla åtta raderna stämmer.

Lägg märke till vad paret **inte** gjorde: de bytte inte kretsen, och de kopplade inte om hela nätet. Två minuter och sex mätningar pekade ut en enda sladd.

5.1.9 Hur mycket en utgång orkar

Den spänning en utgång ger beror på hur mycket ström den levererar. Utan last ligger en hög 74HC-utgång mycket nära 5 V; med en lysdiod som drar 9 mA sjunker den en bit, eftersom utgångstransistorerna har en viss inre resistans. Databladet lovar minst 3,84 V vid 4 mA och matningen 4,5 V (avsnitt 4.2.3), och i praktiken ser du något i stil med 4,3–4,7 V med en lysdiod ansluten.

Det är därför förkopplingsmotståndet ska dimensioneras för några få milliampere. Ju mer ström en utgång levererar, desto närmare det förbjudna området hamnar dess nivå, och desto mindre marginal har nästa grind som läser den. Kontrolluppgiften, övning 5.10, låter dig mäta precis det.

5.2 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet och Labb 2 ska du kunna:

- Koppla upp ett grindnät med 74HC-kretsar på ett kopplingsdäck, med matning, ingångar och utgång, enligt en grindtilldelning och en kopplingslista.
- Verifiera ett nät mot dess sanningstabell, rad för rad, med förväntat och uppmätt värde.
- Bygga NOT, AND och OR av enbart NAND-grindar, och bygga ett nät med enbart NAND.
- Mäta en logisk nivå med en multimeter och avgöra om den är 0, 1 eller ogiltig.
- Felsöka ett nät systematiskt: matningen först, sedan signalen grind för grind, och läsa ut ur sanningstabellen var felet troligen sitter.

Frågor att testa dig själv med

1. Varför ska matningen mätas först, även när nätet delvis verkar fungera?
2. En utgång visar 2,4 V. Vad kan det betyda?
3. Ett nät är fel på exakt de rader där en viss AND-term ska göra utgången till 1. Var mäter du först?
4. Varför kan en 74HC32 sitta där det ska vara en 74HC08 utan att du märker det på alla rader?
5. Varför är 74HC02 den krets flest kopplar fel, och vad händer om den kopplas som en 74HC00?
6. `X = AB + C` med enbart NAND har en grind mer än med AND och OR. Vad vinner du ändå?

Nästa kapitel

Kapitel 6 handlar om sekvensnät, vippor och mikrodatorns uppbyggnad:

- Sekvensnät: kretsar som minns, från reläets självhållning till SR-låset.
- D-låset och D-vippan, klockan och flanken.
- Register, räknare och skiftregister, byggda av vippor.
- Mikrodatorns uppbyggnad på blockschemanivå, och hur varje block går tillbaka till grindar och vippor.

5.3 Övningar

Så kontrollerar du ditt arbete. De flesta övningarna här är felsökningsfall: en beskrivning av ett nät, vad som mättes, och frågan var felet sitter. Svara alltid med **var du mäter härnäst och varför**, inte bara med en gissning om orsaken. En gissning som råkar vara rätt lär dig ingenting om nästa fel.
Kontrolluppgiften, övning 5.10, görs på labbstationen, gärna under Labb 2-1.
Lösningsförslagen finns i bilaga I.
Varje övning är märkt med sin sort.

Övning 5.1 Matningen först FÖRSTÅELSE

a) Mellan vilka ben mäter du matningen på en 74HC08, och vilket värde ska du se?

b) Varför ska du mäta på kretsens ben och inte på matningsskenorna?

c) En krets saknar matning helt, men nätet ”fungerar nästan”. Hur kan en krets utan matning alls ge några utsignaler?

Övning 5.2 Vad betyder mätvärdet? FÖRSTÅELSE

Du mäter på ingångar och utgångar i ett nät med 74HC-kretsar på 5 V. Vilken logisk nivå motsvarar varje mätvärde, och vilka av dem tyder på ett fel?

a) 4,7 V på en utgång.

b) 0,1 V på en utgång.

c) 2,3 V på en ingång.

d) 2,5 V på en utgång, som dessutom är ansluten till en annan grinds utgång.

Övning 5.3 En grind som ljuger RÄKNA FÖR HAND

På en krets som ska vara en 74HC08 mäter du på grind 1: ben 1 = 4,9 V, ben 2 = 0,1 V och ben 3 = 4,8 V.

a) Vad borde ben 3 ha visat?

b) Grinden beter sig som en annan grind. Vilken? Ge en trolig förklaring, och vad du kontrollerar för att bekräfta den.

Övning 5.4 Den varma kretsen

FÖRSTÅELSE

Direkt efter att du slagit på matningen känner du att en av kretsarna blir varm.

a) Vad gör du först?

b) Nämn två troliga orsaker.

c) Hur kontrollerar du, utan att slå på matningen igen, vilken av dem det är?

Övning 5.5 Läs felet ur tabellen

RÄKNA FÖR HAND

Nätet `X = AB + C` ger den här tabellen:

A	B	C	X, mätt
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

a) Vilka rader är fel?

b) Vad har de gemensamt, och vilken del av nätet pekar det ut?

c) Var mäter du först?

Övning 5.6 Bromsassistenten som inte bromsar

RÄKNA FÖR HAND

Ett par har byggt bromsassistenten i Labb 2-1 uppgift 3. Nätet stämmer på tolv av sexton rader. De fyra felaktiga raderna är alla rader där `D = 1` och `F = 1`: där blir `B = 0`, men ska vara 1. Beteckningarna är desamma som i avsnitt 2.1.10.

a) Vilket krav i specifikationen bryter nätet mot, och varför är just det felet farligt?

b) Paret har kopplat förarens pedal på ett ställe i nätet där den inte hör hemma. Var sitter den, och vilket uttryck har de byggt i stället för det rätta?

c) Var mäter du för att bekräfta det?

Övning 5.7 Halvering FÖRSTÅELSE

En signal går genom sexton grindar i rad, och den är fel vid utgången. Den är rätt vid ingången.

a) Var gör du den första mätningen, och varför just där?

b) Hur många mätningar behöver du högst för att hitta den grind där felet uppstår?

c) Hur många hade du behövt, i värsta fall, om du i stället börjat från ingången och mätt efter varje grind?

Övning 5.8 Den flimrande lysdioden FÖRSTÅELSE

En lysdiod på utgången av ett nät tänds och släcks när du för handen nära kopplingsdäcket, fast du inte rör strömbrytarna.

a) Vad är den troliga orsaken?

b) Hur hittar du exakt vilket ben det gäller?

c) Varför händer det här med HC-kretsar, men inte på samma sätt med de äldre LS-kretsarna?

Övning 5.9 Planera fläkten KONSTRUKTION

Fläkten från övning 4.8 har uttrycket `F = TW'B' + CB'`.

a) Gör en fullständig grindtilldelning med kretsbeteckningar och bennummer, med 74HC04, 74HC08 och 74HC32.

b) Skriv kopplingslistan, med matningen först och de oanvända ingångarna sist.

c) Nätet fungerar, utom att fläkten aldrig startar på hög temperatur. Vilka rader i tabellen är fel, och vilken del av nätet mäter du först?

Övning 5.10 Kontroll: en utgång under last KONTROLL

Förutsäg, mät på labbstationen, förklara skillnaden.

En utgång på en 74HC08 driver en röd lysdiod genom ett förkopplingsmotstånd på 330 Ω till jord.

a) Förutsäg

Räkna ut strömmen genom lysdioden som i avsnitt 4.2.5, med antagandet att utgången ger exakt 5 V och att lysdioden har 2 V över sig.

b) Mät

Med utgången hög: mät spänningen på utgången och spänningen över lysdioden. Med utgången låg: mät spänningen på utgången.

c) Räkna igen

Nu med de uppmätta spänningarna: spänningen över motståndet är utgångens spänning minus lysdiodens. Vilken ström går det?

d) Förklara skillnaden

Förklara skillnaden mellan a) och c). Vilket av de två antagandena i a) var fel, och varför? Stämmer den uppmätta höga och låga nivån med databladets V_{OH} och V_{OL}?

Övning 5.11 Fel krets i sockeln

FÖRSTÅELSE FÖRDJUPNING

Någon har satt en 74HC02 där det skulle sitta en 74HC00, och kopplat den enligt benplaceringen för 74HC00: strömbrytarna A och B till ben 1 och 2, och ben 3 till lysdioden.

a) Vad är ben 1 på en 74HC02?

b) Vad händer elektriskt när strömbrytaren på ben 1 står i läge 0 medan NOR-grinden försöker ge en etta på samma ben? Varför kan kretsen bli varm?

c) Vad visar lysdioden på ben 3?

KAPITEL 6

Sekvensnät, vippor och mikrodatorns uppbyggnad

Hur grindar som minns blir register och räknare, och till sist en hel dator.

I DET HÄR KAPITLET

- Kretsar som minns: från reläets självhållning till SR-låset av två NOR-grindar.
- D-låset och D-vippan: nivåkänsligt mot flanktriggat, och vad klockan gör.
- Register, räknare och skiftregister, byggda av vippor.
- Mikrodatorns blockschema: CPU, register, ALU, styrenhet, programräknare, minnen, bussar, portar och klocka.
- Instruktionscykeln: hämta, avkoda, utför, sexton miljoner gånger i sekunden.
- ATmega328P på ett Arduino Uno-kort: mikrodatorn resten av boken handlar om.

Alla nät hittills i boken har varit kombinatoriska: samma ingångar ger alltid samma utgång. Det här kapitlet visar hur ett nät kan minnas, och hur minnet byggs ut till register, räknare och skiftregister. Av de byggstenarna, och av de kombinatoriska näten från de tidigare kapitlen, sätts sedan en hel mikrodator ihop: den ATmega328P som resten av boken handlar om.

Repetera självhållningen i avsnitt 2.2.7 innan du börjar, och läs sedan avsnitt 6.1 om sekvensnät och vippor. Avsnitt 6.2 går igenom mikrodatorns uppbyggnad block för block. Blockschemat där är ett lärandemål i kursplanen och kommer på provet, så öva på att rita det och att förklara vad varje block gör. Övningarna finns i avsnitt 6.4, och deras lösningsförslag i bilaga I.

Från och med nästa kapitel programmeras mikrodatorn i Microchip Studio. **Installera programmet redan nu**, enligt avsnitt D.1 i bilaga D: installationen tar tid och ska vara klar när du börjar på kapitel 7.

6.1 Sekvensnät och vippor

6.1.1 Från kombinatoriskt till sekventiellt

Alla nät hittills har varit **kombinatoriska**: utgången beror bara på ingångarnas värden just nu. Samma ingångar ger alltid samma utgång, och nätet har ingen aning om vad som hände för en sekund sedan. Bromsassistenten, majoritetskretsen och segmentavkodaren är alla sådana.

Många verkliga uppgifter kräver mer. Ett transportband som startas med en knapp och stoppas med en annan måste **minnas** att startknappen har tryckts, även efter att den släppts. En räknare måste minnas hur långt den räknat. Ett trafikljus måste minnas vilken färg som lyser, för att veta vilken som kommer härnäst. En mikrodator måste minnas sitt program, sina variabler och vilken instruktion den är på.

Ett nät som minns kallas ett **sekvensnät**: utgången beror på ingångarna **och** på vad som hänt tidigare, alltså på ett lagrat **tillstånd**. Det här avsnittet visar hur minne byggs av vanliga grindar, och vad som byggs av minnet: register, räknare och skiftregister. Det är de byggstenar som mikrodatorn i avsnitt 6.2 består av.

Sekvensnät ingår inte i kursplanens lärandemål för sin egen skull. De finns med för att mikrodatorn annars blir en svart låda: dess register, programräknare och minnen är vippor.

6.1.2 Minne genom återkoppling

Du har redan byggt ett sekvensnät, i kapitel 2: reläets **självhållning** (avsnitt 2.2.7).

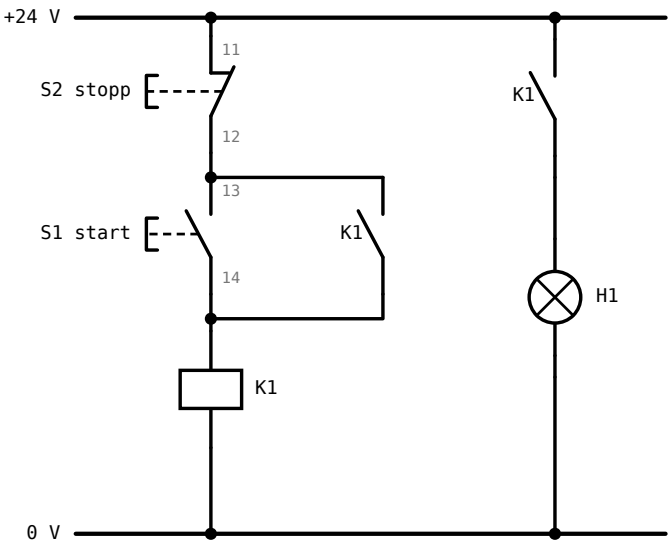
Tryck på start, `S1`, så får reläspolen `K1` ström och drar. Reläets egen kontakt `K1` sitter parallellt med `S1`, så när `S1` släpps går strömmen i stället genom `K1`:s kontakt, och reläet håller sig självt draget. Först när stopp, `S2`, trycks bryts strömmen, reläet släpper, och kretsen är tillbaka i utgångsläget.

Samma knapptryckning, `S1` släppt och `S2` släppt, ger alltså olika resultat beroende på vad som hänt innan: lampan lyser om start senast trycktes, och är släckt om stopp senast trycktes. Kretsen minns.

Nyckeln är **återkopplingen**: reläets utgång, kontakten, är kopplad tillbaka till dess egen ingång. Samma idé fungerar med grindar.

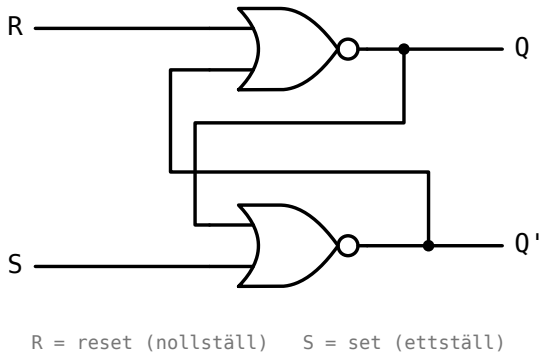
6.1.3 SR-låset

Koppla två NOR-grindar så att varje grinds utgång matar den andra grindens ingång:



S1 drar K1, och K1:s egen kontakt håller sedan K1 dragen när S1 släpps.
Först S2 bryter. Kretsen minns att S1 har tryckts: det är ett SR-lås.

Figur 6.1 Självhållning med startknappen S1, stoppknappen S2 och reläet K1.



Figur 6.2 SR-lås av två korskopplade NOR-grindar.

Det här är ett **SR-lås** (*SR latch*), den enklaste minnescellen som finns. Den har två ingångar:

- **S** (*set*, ettställ): gör `Q = 1`;
- **R** (*reset*, nollställ): gör `Q = 0`.

Utgången `Q'` är alltid inversen av `Q`, utom i ett fall (nedan).

S	R	Q efteråt	Betydelse
0	0	samma som före	**minns**
1	0	1	ettställ
0	1	0	nollställ
1	1	ogiltigt	båda utgångarna 0

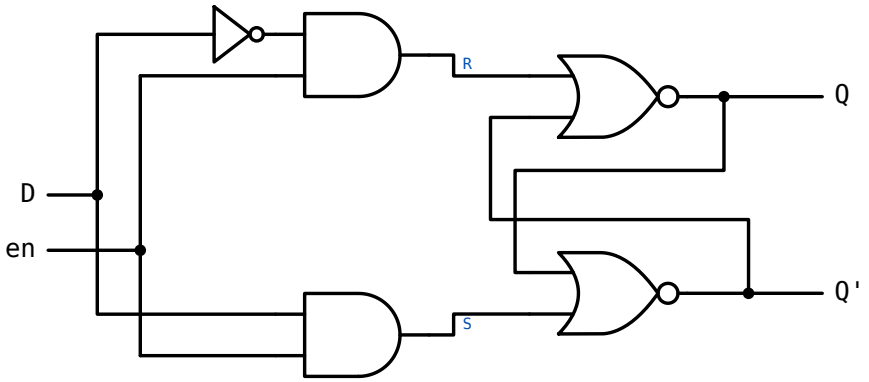
Följ signalerna en gång för att se varför det minns. Anta att `Q = 1`, så `Q' = 0`, och att `S = R = 0`. Den övre grinden får `R = 0` och `Q' = 0`, och en NOR av två nollor är 1: `Q` förblir 1. Den nedre grinden får `Q = 1` och `S = 0`, och en NOR med en etta in är 0: `Q'` förblir 0. Tillståndet håller sig självt, precis som reläet. Ett kort tryck på `R` gör `Q = 0`, och sedan håller sig det tillståndet lika bra.

Jämför med självhållningen: `S1` är **S**, `S2` är **R**, och reläet är låset. Skillnaden är fallet `S = R = 1`. I reläkretsen vinner stopp, eftersom `S2` sitter i serie med allt; i NOR-låset blir båda utgångarna 0, och vad som händer när båda släpps samtidigt går inte att förutsäga. Därför är kombinationen förbjuden, och därför används SR-låset sällan direkt.

Bygg låset i CircuitVerse (`https://circuitverse.org/simulator`) och prova: tryck `S`, släpp, tryck `R`, släpp. Lägg märke till att utgången inte går att förutsäga från ingångarna ensamma. Du måste veta vad som hände innan.

6.1.4 D-låset

D-låset tar bort den förbjudna kombinationen. I stället för `S` och `R` har det en dataingång `D`, värdet som ska lagras, och en ingång `en` (*enable*) som bestämmer när låset lyssnar:



Figur 6.3 D-lås: en inverterare och två AND-grindar framför ett SR-lås.

De två AND-grindarna gör `S = D · en` och `R = D' · en`. När `en = 0` är både `S` och `R` noll, och låset minns. När `en = 1` blir antingen `S` eller `R` en etta, beroende på `D`, men aldrig båda.

en	Beteende
1	**genomsläppligt**: `Q` följer `D` direkt
0	**låst**: `Q` behåller det värde det hade när `en` gick till 0

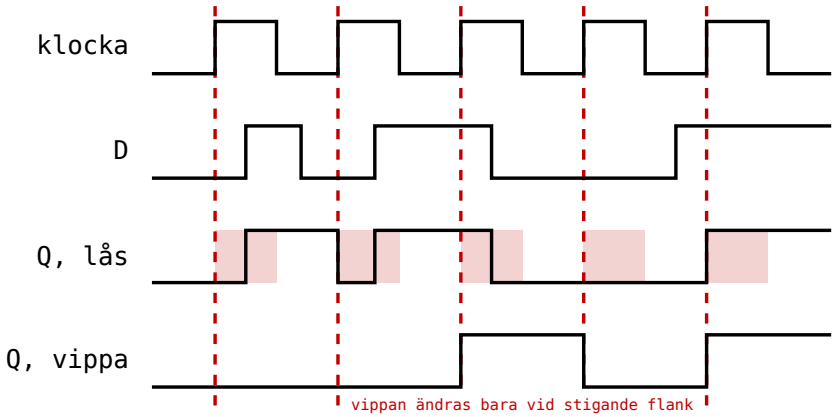
D-låset är **nivåkänsligt**: det släpper igenom allt som händer på `D` så länge `en` är 1. Det är också dess svaghet. En störning på `D` medan `en = 1` går rakt igenom till `Q`, och i ett system med tusentals minnesceller är det svårt att garantera att varje `D` är stabilt under hela den tid `en` är 1.

6.1.5 D-vippan och klockan

D-vippan (*D flip-flop*) löser problemet. Den lagrar också en bit, men den läser `D` bara i ett enda **ögonblick**: när en **klocksignal** går från 0 till 1, den **stigande flanken**. Resten av tiden, oavsett vad `D` gör, behåller `Q` sitt värde.

Invändigt är en D-vippa två D-lås i rad, där det ena är öppet när klockan är låg och det andra när den är hög. Nettoeffekten är att bara värdet i själva ögonblicket när klockan stiger kommer igenom.

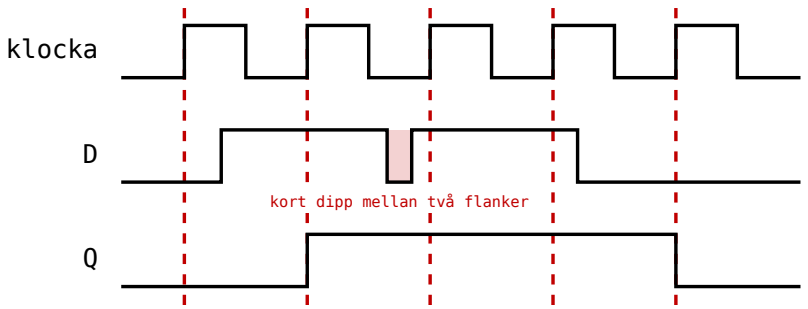
Kör ett lås och en vippa med samma klocka och samma `D`, så syns skillnaden:



Låset är genomsläppligt medan klockan är hög (skuggat) och följer D.
Vippan läser D bara i ögonblicket klockan går från 0 till 1.

Figur 6.4 Ett D-lås och en D-vippa med samma klocka och samma D.

Låset ändrar sig flera gånger, varje gång D ändras medan klockan är hög. Vippan ändrar sig bara vid de stigande flankerna, och bara om D då har ett annat värde än det Q redan har. En störning mellan två flanker syns inte alls:



D dippar och återhämtar sig mellan två stigande flanker. Q rör sig inte,
eftersom ingen flank inträffade medan D var låg.

Figur 6.5 En kort störning på D mellan två stigande flanker.

Klockan är en fyrkantvåg som alla vippor i ett system delar. Två tal beskriver den: **perioden** `T`, tiden för en hel svängning, och **frekvensen** `f = 1/T`. En Arduino Uno har en klocka på **16 MHz**, sexton miljoner svängningar i sekunden, så perioden är

$$T = \frac{1}{16\,000\,000\ \text{Hz}} = 62{,}5\ \text{ns}$$

Varje vippa i mikrodatorn uppdateras vid samma flank. Det gör att allt i datorn tar ett steg i taget, i takt, och det är därför det går att säga att en instruktion tar ett bestämt, helt antal **klockcykler**. Det talet kommer tillbaka i kapitel 9, där tidsfördröjningar räknas ut i klockcykler.

I scheman ritas en D-vippa som en ruta med ingången `D`, utgången `Q` och en liten triangel vid klockingången. Triangeln betyder ”flanktriggad”.

6.1.6 Register

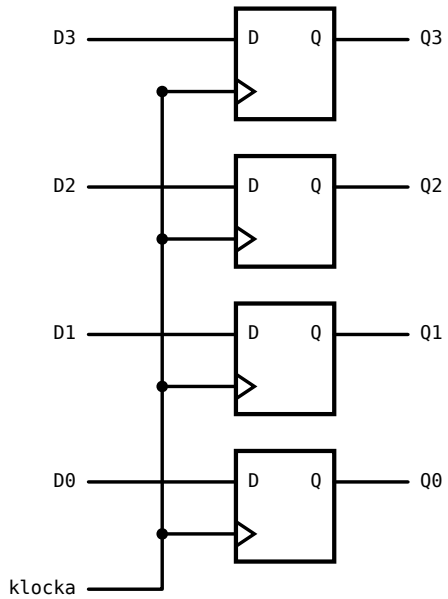
En vippa lagrar en bit. Ett **register** lagrar flera: det är N vippor sida vid sida, som delar samma klocka.

Vid varje stigande flank kopieras alla ingångarna `D3`–`D0` till utgångarna `Q3`–`Q0` samtidigt, och resten av tiden håller registret sitt värde.

Mikrodatorn i resten av boken är full av register. De 32 **arbetsregistren** `r0`–`r31`, där programmet räknar, är 8-bitars register: åtta vippor vardera. **Statusregistret** `SREG`, där räkneenheten sparar sina flaggor, är ett register. Och **utporten** `PORTB`, som tänder lysdioderna i Labb 3, är ett register vars utgångar är kopplade direkt till kretsens stift: skriv en etta i en bit, så blir stiftet 5 V.

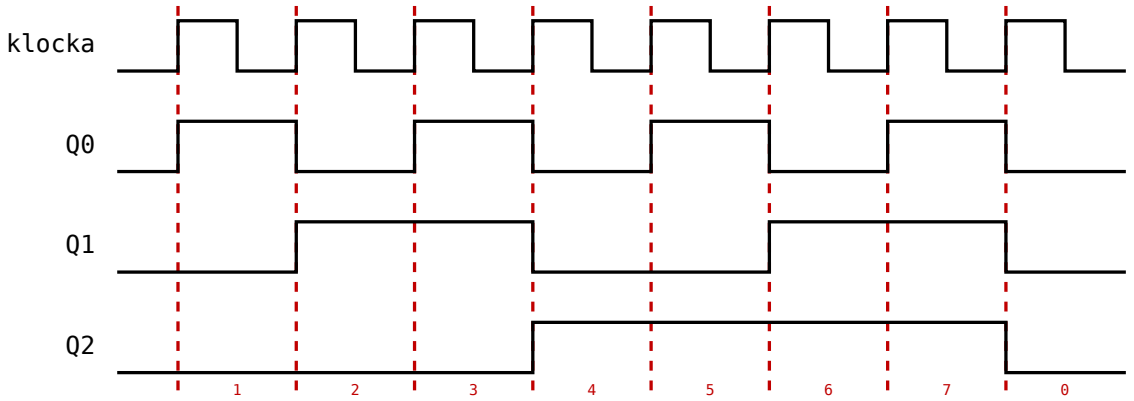
6.1.7 Räknare

Koppla en vippa så att den **växlar** vid varje flank: `D` får inversen av `Q`. Då blir `Q` en fyrkantvåg med halva klockans frekvens. Kedja flera sådana, så att varje vippa växlar när den föregående går från 1 till 0, så får du en **räknare**:



Fyra D-vippor med gemensam klocka: ett 4-bitars register.
Vid varje stigande flank kopieras D3-D0 till Q3-Q0.

Figur 6.6 Ett 4-bitars register av fyra D-vippor med gemensam klocka.



Figur 6.7 Tidsdiagram för en 3-bitars räknare.

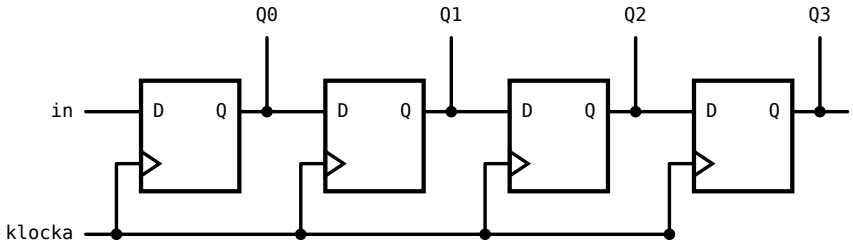
Läs `Q2 Q1 Q0` som ett binärt tal efter varje flank: 001, 010, 011, 100 och så vidare upp till 111, och sedan 000 igen. Tre vippor räknar till 7; `n` vippor räknar från 0 till `2^n - 1` och börjar sedan om. Att räknaren slår om från 111 till 000 är ingen särskild logik; det är vad som händer när den fjärde biten inte finns. Samma sak kallas **rundgång** i kapitel 8, när ett 8-bitars register räknar förbi 255.

Två saker att lägga märke till:

- **Varje bit har halva frekvensen av biten före.** `Q0` växlar vid varje flank, `Q1` vid varannan, `Q2` vid var fjärde. En räknare är alltså också en frekvensdelare.
- **Mikrodatorns programräknare är en räknare.** Den håller adressen till nästa instruktion och räknar upp ett steg för varje instruktion som hämtas (avsnitt 6.2).

6.1.8 Skiftregister

Koppla vipporna i en kedja, så att varje vippas `Q` matar nästa vippas `D`:



Figur 6.8 Ett 4-bitars skiftregister: fyra D-vippor i kedja.

Vid varje flank tar varje vippa över värdet från vippan före, och den första tar in en ny bit. Innehållet **flyttas**, skiftas, ett steg per klockflank. Det kallas ett **skiftregister**.

Efter flank	in	Q0	Q1	Q2	Q3
start		0	0	0	0
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	0
4	0	0	0	0	1

En enda etta vandrar genom registret, som ett rinnande ljus. Skiftregister används för att skicka data en bit i taget över en ledning (seriell överföring), och de har en direkt motsvarighet i mikrodatorn: instruktionen `lsl` flyttar alla bitar i ett register ett steg åt vänster, vilket är samma sak som att multiplicera med 2. Den kommer i kapitel 8. Att figuren flyttar åt höger är bara ritsättet: skrivs innehållet som ett binärt tal, `Q3 Q2 Q1 Q0`, flyttas varje bit till en mer signifikant plats, alltså åt vänster i talet, precis som med `lsl`.

6.2 Mikrodatorns uppbyggnad

6.2.1 Vad en mikrodator är

En **dator** är en maskin som utför en lista av instruktioner, ett **program**, och som kan läsa och skriva data medan den gör det. Den består av en centralenhet som utför instruktionerna, minne som håller programmet och datat, och in- och utenheter som förbinder den med omvärlden.

En **mikrodator** är en dator där allt detta sitter i en eller några få kretsar. När hela datorn, centralenhet, minnen och portar, sitter på **ett enda chip** kallas kretsen en **mikrokontroller**.

Den ATmega328P som sitter på ett Arduino Uno-kort, och som du programmerar från kapitel 7, är en mikrokontroller.

Mikrokontroller finns överallt där något ska styras: i tvättmaskiner, kaffebryggare, bilar, hissar, fjärrkontroller och industrirobotar. En modern bil har hundratals. De kallas **inbyggda system**, eftersom datorn är inbyggd i en produkt och sällan syns. Till skillnad från en persondator kör en mikrokontroller oftast ett enda program, från det att strömmen slås på tills den slås av, och programmet handlar om att läsa givare och styra saker: det som kursplanen kallar **styrobjekt**.

6.2.2 Blockschemat

Alla datorer, från den minsta mikrokontroller till en server, är uppbyggda av samma block:

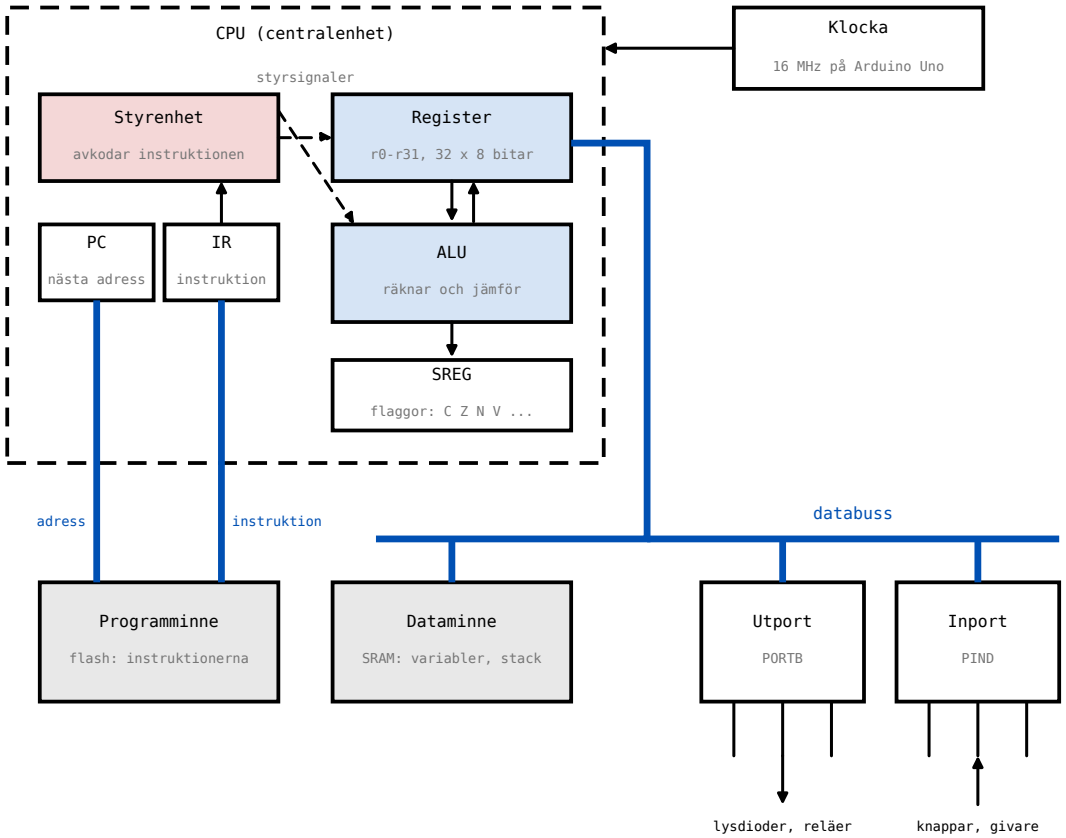
Block	Uppgift
CPU (centralenhet)	Hämtar instruktioner och utför dem. Består av styrenhet, register och ALU.
Styrenhet	Tolkar (avkodar) varje instruktion och styr de andra blocken så att den utförs.
Programräknare, PC	Håller adressen till nästa instruktion.
Instruktionsregister, IR	Håller den instruktion som utförs just nu.
Register	Snabba lagringsplatser inuti CPU:n, där räkningarna görs.
ALU	Aritmetisk-logisk enhet: adderar, subtraherar, jämför och gör logiska operationer.
SREG	Statusregistret: flaggor som beskriver resultatet av den senaste räkningen.
Programminne	Håller programmet, instruktion för instruktion.
Dataminne	Håller variabler och annat data medan programmet kör.
In- och utportar	Förbinder datorn med omvärlden: knappar och givare in, lysdioder och reläer ut.
Bussar	Ledningarna mellan blocken, som adresser och data går på.
Klocka	Ger takten som alla vippor i datorn uppdateras i.

Resten av det här avsnittet går igenom blocken ett i taget, och slutar med hur de ser ut i ATmega328P.

6.2.3 CPU:n: ALU, register och styrenhet

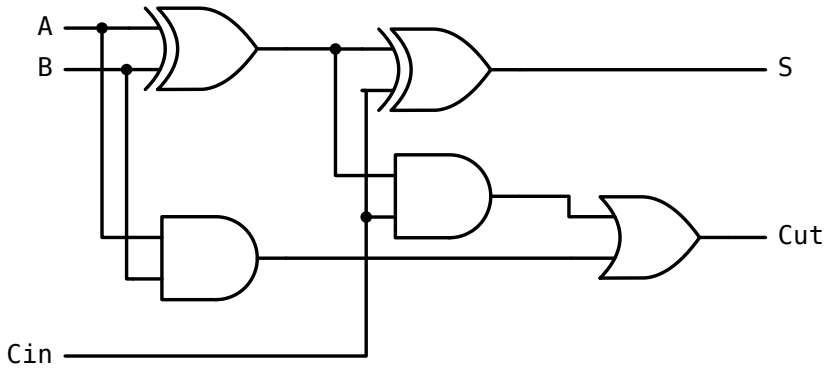
Registren är små, snabba minnen inuti CPU:n. I ATmega328P finns 32 stycken, `r0` till `r31`, med 8 bitar vardera: varje register är åtta D-vippor med gemensam klocka (avsnitt 6.1.6). All räkning sker i registren. Ett tal i dataminnet måste först hämtas in i ett register innan något kan göras med det.

ALU:n, den aritmetisk-logiska enheten, är det enda ställe i datorn där något faktiskt räknas. Den tar ett eller två tal från registren, utför en operation och lämnar resultatet i ett register. Den är ett stort **kombinatoriskt nät**: samma sorts nät som i kapitel 2 och 4, bara större. Kärnan i den är en **adderare**, byggd av heladderare (avsnitt 4.1.8):



Styrenheten hämtar instruktioner ur programminnet, där PC pekar, och låter ALU:n räkna med registren. Data går över databussen till minnet och portarna.

Figur 6.9 Mikrodatorns blockschema.




```
Heladderaren: S = A xor B xor Cin, Cut = AB + (A xor B)Cin.
Åtta sådana i rad, där varje Cut blir nästa Cin, adderar två byte.
```

Figur 6.10 En heladderare av två XOR-grindar, två AND-grindar och en OR-grind.

En heladderare adderar tre bitar: två siffror och minnessiffran från positionen till höger. Åtta heladderare i rad, där varje `Cut` blir nästa positions `Cin`, adderar två 8-bitars tal, precis som när du adderar två binära tal för hand i avsnitt 1.1.7. Den sista minnessiffran, den som inte får plats i 8 bitar, är **carry-flaggan**.

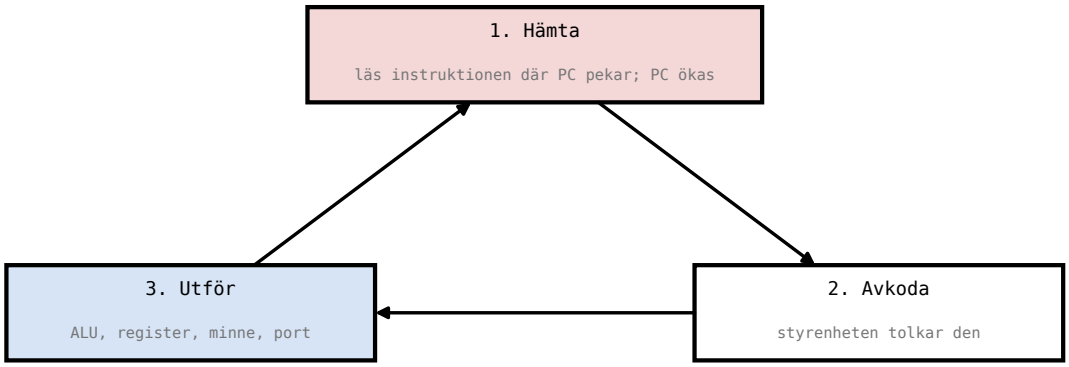
Statusregistret, SREG, är ett register där ALU:n sparar information om resultatet: blev det noll, blev det en minnessiffra över, blev det negativt? De här bitarna kallas **flaggor**. Programmet använder dem för att fatta beslut, ”hoppa om resultatet blev noll”, vilket är ämnet för kapitel 8 och 9.

Styrenheten läser instruktionen, avgör vad den betyder, och skickar **styrsignaler** till de andra blocken: vilka register ALU:n ska läsa, vilken operation den ska göra, vart resultatet ska. Den är i grunden ett sekvensnät, som stegar sig igenom varje instruktion.

6.2.4 Programräknaren och instruktionscykeln

Ett program är en lista av instruktioner i programminnet, en efter en, var och en på sin adress. **Programräknaren**, PC (*program counter*), håller adressen till den instruktion som står på tur. Den är en räknare (avsnitt 6.1.7), och det är den som gör att datorn går framåt i programmet.

Allt en CPU gör är att upprepa samma tre steg, **instruktionscykeln**:



Figur 6.11 Instruktionscykeln: hämta, avkoda och utför.

1. **Hämta** (*fetch*): läs instruktionen på den adress PC pekar på, lägg den i instruktionsregistret, och räkna upp PC så att den pekar på nästa instruktion.
2. **Avkoda** (*decode*): styrenheten tolkar bitarna i instruktionen. Vilken operation? Vilka register?
3. **Utför** (*execute*): ALU:n räknar, ett register skrivs, eller data flyttas till eller från minnet eller en port.

Sedan börjar det om, med nästa instruktion. Ett litet exempel, tre instruktioner i programminnet:

Adress	Instruktion	Betyder
0	`ldi r16, 5`	lägg talet 5 i register r16
1	`inc r16`	öka r16 med 1
2	`rjmp 1`	hoppa till adress 1

Steg	PC före	Hämtas	Utförs	r16 efter	PC efter
1	0	`ldi r16, 5`	r16 = 5	5	1
2	1	`inc r16`	r16 = r16 + 1	6	2
3	2	`rjmp 1`	PC = 1	6	1
4	1	`inc r16`	r16 = r16 + 1	7	2
5	2	`rjmp 1`	PC = 1	7	1

Lägg märke till steg 3. Ett **hopp** är ingenting annat än en instruktion som skriver ett nytt värde i programräknaren. Normalt räknar PC uppåt ett steg i taget; ett hopp låter

programmet gå tillbaka och upprepa något, eller hoppa över något. Alla loopar och alla beslut i ett program är byggda på det, och det är vad kapitel 9 handlar om.

På ATmega328P tar de flesta instruktioner **en klockcykel**: hämtningen av nästa instruktion sker samtidigt som den förra utförs. Vid 16 MHz blir det upp till sexton miljoner instruktioner i sekunden.

6.2.5 Minnen: programminne, dataminne och EEPROM

En mikrodator har flera sorters minne, för olika ändamål:

Minne	Håller	Bevaras utan ström?	I ATmega328P
Programminne (flash)	programmet	ja	32 kB
Dataminne (SRAM)	variabler och stacken	nej	2 kB
EEPROM	inställningar som ska sparas	ja	1 kB

Flashminnet håller programmet. Det skrivs när du programmerar kretsen, i kapitel 11, och finns kvar när strömmen slås av, så att programmet startar igen nästa gång. Det kan skrivas om ungefär tiotusen gånger, vilket räcker gott för utveckling men inte för data som ändras hela tiden.

SRAM (*static RAM*) är dataminnet: variabler, mellanresultat och **stacken**, där datorn sparar returadresser när den anropar en subrutin (kapitel 10). Det är snabbt och kan skrivas hur många gånger som helst, men innehållet försvinner när strömmen bryts. Varje bit i ett SRAM är i princip ett lås.

EEPROM bevarar sitt innehåll utan ström, som flashminnet, men kan skrivas en byte i taget. Det används till sådant som ska överleva ett strömavbrott: en kalibrering, en inställning, en räknare av driftstimmar. Boken använder det inte.

Lägg märke till att programminnet och dataminnet är **separata** minnen med var sin buss. Det kallas en **Harvardarkitektur**, och det betyder att CPU:n kan hämta nästa instruktion samtidigt som den läser eller skriver data. De flesta persondatorer har i stället program och data i samma minne, en **von Neumann-arkitektur**. För dig som programmerar betyder skillnaden att ett tal i programminnet och ett tal i dataminnet ligger på olika ställen och nås med olika instruktioner, vilket du märker i kapitel 7.

6.2.6 Bussar

Blocken är förbundna med **bussar**: grupper av ledningar som bär ett tal i taget, en bit per ledning.

- **Adressbussen** säger **var**: vilken minnesplats eller vilket register som ska läsas eller skrivas. Med `n` ledningar kan den peka ut `2^n` olika adresser. Programminnet i ATmega328P har 16 384 instruktionsplatser, och $2^{14} = 16\,384$, så programräknaren behöver 14 bitar.

- **Databussen** bär **vad**: själva värdet som läses eller skrivs. I en 8-bitars dator som ATmega328P är den 8 bitar bred, och det är därför den kallas 8-bitars: den hanterar en byte i taget.
- **Styrsignalerna** säger **hur**: läs eller skriv, och när.

En buss är ingen ny sorts komponent. Den är ledningar, och i en mikrokontroller sitter de inuti chipet. Men begreppet förklarar mycket: allt som ska flyttas mellan två block går över en buss, en sak i taget, och det är därför en instruktion som flyttar data mellan minnet och ett register ibland tar två klockcykler i stället för en.

6.2.7 In- och utportar

En dator som inte kan påverka något eller känna av något är ganska meningslös för den som ska styra en maskin. **Portarna** är förbindelsen med omvärlden.

En **utport** är ett register vars utgångar är kopplade till kretsens **stift**. Skriver programmet en etta i en bit, blir motsvarande stift 5 V; skriver det en nolla, blir stiftet 0 V. Koppla en lysdiod med ett förkopplingsmotstånd till stiftet, precis som till en grindutgång i Labb 2, så styr programmet lysdioden. Koppla ett relä via en transistor, så styr programmet en lampa på 24 V eller en motor.

En **inport** är motsatsen: ett sätt för programmet att läsa av spänningen på ett antal stift, som ettor och nollor. En knapp, en gränslägesbrytare eller en nivågivare blir då en bit som programmet kan fatta beslut utifrån.

I ATmega328P kan varje stift vara antingen in- eller utgång, och det väljer programmet själv. Tre register per port styr det: `DDRx` (riktningen), `PORTx` (vad en utgång ska ge) och `PINx` (vad en ingång läser). Hur de används är ämnet för kapitel 8 och 10.

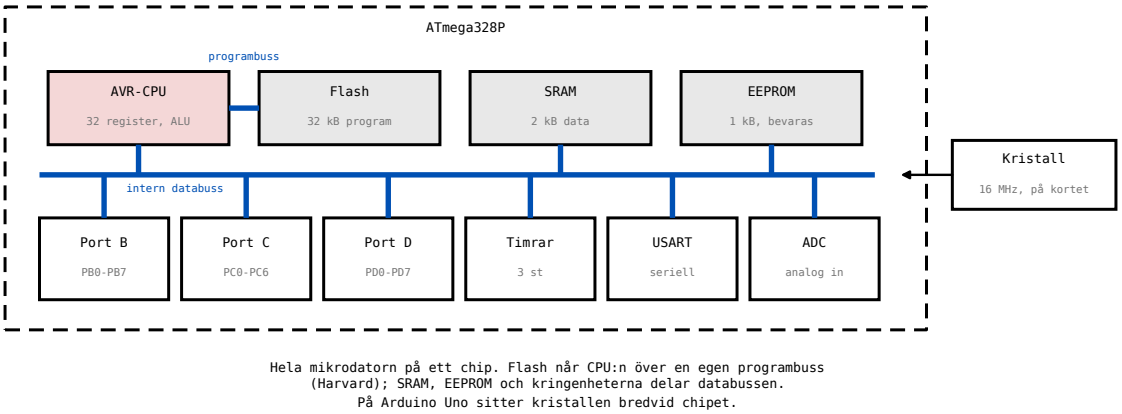
6.2.8 Klockan

Allt i mikrodatorn går i takt med **klockan**, samma sorts fyrkantvåg som styr vipporna i avsnitt 6.1.5. Vid varje stigande flank tar varje register, programräknaren och styrenheten ett steg.

På ett Arduino Uno-kort kommer klockan från en **keramisk resonator** på 16 MHz bredvid chipet. Det ger:

Storhet	Värde
Klockfrekvens	16 MHz = 16 000 000 perioder per sekund
En klockcykel	1 / 16 000 000 s = 62,5 ns
Instruktioner per sekund	upp till 16 miljoner (de flesta tar en cykel)

Att varje instruktion tar ett känt antal klockcykler gör att du kan räkna ut exakt hur lång tid en del av ett program tar. Det är så en tidsfördröjning byggs, utan någon klocka på väggen: låt datorn räkna ett visst antal varv, och räkna ut hur många cykler det blir. Det gör du i kapitel 9.



Figur 6.12 Förenklat blockschema för ATmega328P.

6.2.9 ATmega328P och Arduino Uno

Mikrokontrollern i resten av boken är **ATmega328P** från Microchip, samma krets som sitter på ett **Arduino Uno**-kort. Den är en 8-bitars mikrokontroller i familjen **AVR**.

Egenskap	ATmega328P
Arkitektur	8-bitars AVR, Harvard
Arbetsregister	32 st, `r0`–`r31`, 8 bitar vardera
Programminne (flash)	32 kB, varav en liten del upptas av Arduinos bootloader
Dataminne (SRAM)	2 kB
EEPROM	1 kB
Klocka på Arduino Uno	16 MHz (keramisk resonator)
Portar	B, C och D; 23 stift som kan vara in- eller utgångar
Kringenheter	tre timrar, seriekommunikation (USART), analog-till-digital-omvandlare (ADC)
Matning	5 V på Arduino Uno

Blocken från avsnitt 6.2.2 finns alla där. CPU:n med sina 32 register, sin ALU och sitt SREG är kärnan. Flashminnet håller programmet, SRAM variablerna. Portarna B, C och D är utportar och inportar, och kortets numrerade stift (0–13, A0–A5) är kopplade till dem. Därtill kommer **kringenheter**, egna små hårdvarublock för sådant som är vanligt i styrning: timrar som räknar tid, en seriekanal för att prata med en dator, och en omvandlare som mäter analoga spänningar. Boken använder portarna; resten får vänta till en fortsättningskurs.

Två saker är värda att veta innan du börjar programmera:

- Arduino-kortets stiftnummer är **kortets** numrering, inte kretsens. Stift 13 på kortet, där den inbyggda lysdioden sitter, är bit 5 i port B, `PB5`. Kopplingen mellan dem kommer i kapitel 8.
- Boken programmerar kretsen direkt i **assembler**, utan Arduinos egna programbibliotek. Varje instruktion du skriver blir en instruktion i flashminnet, och du ser exakt vad CPU:n gör.

6.2.10 Från grind till dator

Varje block i mikrodatorn går tillbaka till något du redan har byggt:

Block	Byggt av	Där du mötte det
ALU	kombinatoriska nät: adderare, AND, OR, XOR	kapitel 2 och 4
Heladderare	XOR, AND, OR	avsnitt 4.1.8
Register	D-vippor med gemensam klocka	avsnitt 6.1.6
Programräknare	en räknare av vippor	avsnitt 6.1.7
Instruktionsregister	ett register	avsnitt 6.1.6
Statusregistret	ett register, en vippa per flagga	avsnitt 6.1.6
Dataminne (SRAM)	tusentals lås, och avkodare som väljer ut rätt byte	avsnitt 6.1.4, kapitel 4
Utport	ett register kopplat till stiften	avsnitt 6.1.6
Adressavkodning	kombinatoriska nät som väljer en adress	kapitel 4
Styrenhet	ett sekvensnät	avsnitt 6.1.1

Ingenting i en dator är magiskt. Den är grindar och vippor, i stora mängder, i takt med en klocka. Det du lär dig i resten av boken är hur man styr dem med ett program.

6.3 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Förklara skillnaden mellan ett kombinatoriskt nät och ett sekvensnät, och känna igen vilket en krets är.
- Förklara hur återkoppling ger minne, med självhållningen och SR-låset som exempel.
- Beskriva skillnaden mellan ett D-lås och en D-vippa, och rita utgången för båda i ett tidsdiagram.
- Räkna ut klockperioden från klockfrekvensen, och tvärtom.
- Förklara vad ett register, en räknare och ett skiftregister är, och vad de är byggda av.
- Rita mikrodatorns blockschema och redogöra för vad varje block gör: CPU, styrenhet, register, ALU, statusregister, programräknare, programminne, dataminne, bussar, portar och klocka.
- Beskriva instruktionscykeln och följa ett kort program instruktion för instruktion.

- Redogöra för ATmega328P:s minnen och portar, och vad som skiljer flash, SRAM och EEPROM åt.

Frågor att testa dig själv med

1. Varför går det inte att skriva en vanlig sanningstabell för ett SR-lås?
2. Vad vinner man på att en vippa bara läser `D` vid den stigande flanken?
3. Hur lång är en klockcykel vid 16 MHz?
4. En räknare med 8 vippor står på sitt högsta värde och får en klockflank till. Vad händer?
5. Vilket block i mikrodatorn räknar, och vilket bestämmer vad som ska räknas?
6. Vad är ett hopp i ett program, sett från programräknarens håll?
7. Vilket av ATmega328P:s minnen tappar sitt innehåll när strömmen bryts, och varför gör inte de andra det?
8. Vad är en utport, sett som en byggsten av vippor?

Nästa kapitel

Kapitel 7 handlar om mikrodatorn på nära håll och om hur den programmeras:

- ATmega328P på nära håll: de 32 registren, statusregistret och de tre minnena.
- Assemblerspråket: instruktioner, operander, direktiv och etiketter, och hur en rad blir maskinkod.
- Microchip Studio: skapa ett projekt, assemblera, och stega genom programmet i simulatorn.
- **Labb 3** börjar, bilaga C.

6.4 Övningar

> **Så kontrollerar du ditt arbete.** Ett sekvensnät kan inte kontrolleras rad för rad i en sanningstabell, eftersom samma ingångar kan ge olika utgång beroende på vad som hänt innan. Gå i stället igenom en **följd** av ingångar, ett steg i taget, och skriv upp tillståndet efter varje steg. I CircuitVerse gör du samma sak: ändra en ingång, titta på utgången, anteckna, nästa. Kontrolluppgiften är övning 6.4. Lösningsförslagen finns i bilaga I.

Varje övning är märkt med sin sort.

Övning 6.1 Kombinatoriskt eller sekventiellt? FÖRSTÅELSE

Avgör för var och en av kretsarna nedan om den är kombinatorisk eller sekventiell, och motivera med en mening.

a) Tankens larm i kapitel 4, som går när minst två av tre givare ger 1.

b) Start- och stoppkretsen med självhållning i kapitel 2.

c) En avkodare som tänder rätt segment i en sifferdisplay för en BCD-siffra.

d) Ett trafikljus som växlar mellan rött, gult och grönt.

e) Ett kodlås som öppnar när siffrorna 3, 7 och 1 har tryckts i den ordningen.

f) Bromsassistenten i kapitel 2.

Övning 6.2 SR-låset RÄKNA FÖR HAND

Ett SR-lås av två NOR-grindar (avsnitt 6.1.3) börjar med `Q = 0`. Ingångarna ändras i den här ordningen. Fyll i `Q` efter varje steg.

Steg	S	R	Q
start	0	0	0
1	1	0	
2	0	0	
3	0	0	
4	0	1	
5	0	0	
6	1	0	
7	0	0	

a) Fyll i tabellen.

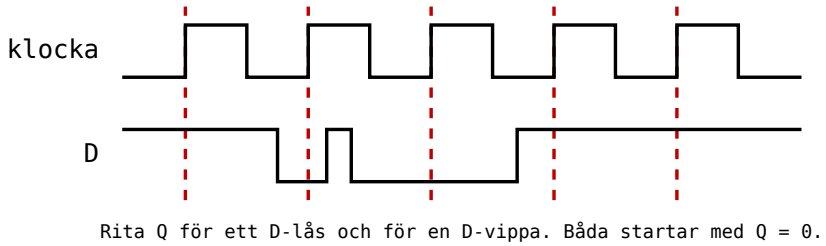
b) Steg 2, 3, 5 och 7 har samma ingångar. Har de samma `Q`? Varför är det här skälet till att nätet kallas sekventiellt?

c) Varför är kombinationen `S = R = 1` förbjuden i ett SR-lås av NOR-grindar?

Övning 6.3 Lås och vippa

RÄKNA FÖR HAND

Ett D-lås och en D-vippa får samma klocka och samma `D`. Låset är genomsläppligt medan klockan är hög; vippan läser `D` vid varje stigande flank. Båda börjar med `Q = 0`.



Figur 6.13 Klockan och `D` för övning 6.3.

a) Rita `Q` för låset.

b) Rita `Q` för vippan.

c) `D` har en kort puls mellan den andra och den tredje stigande flanken. Syns den på låsets utgång? På vippans? Förklara.

Övning 6.4 Kontroll: bygg ett SR-lås

KONTROLL

Förutsäg, bygg i CircuitVerse, jämför.

a) **Förutsäg.** Använd din tabell från övning 6.2.

b) **Bygg** SR-låset av två NOR-grindar i CircuitVerse (`https://circuitverse.org/simulator`), med två ingångar `S` och `R` och två utgångar `Q` och `Q'`, korskopplade som i avsnitt 6.1.3.

c) **Kör följden** från övning 6.2, ett steg i taget. Anteckna `Q` och `Q'` efter varje steg, och jämför med din förutsägelse.

d) När du precis har byggt kretsen, innan du tryckt på något, vad visar `Q`? Är det samma sak varje gång du bygger om den? Vad säger det om ett minne som slås på?

e) Ställ in `S = R = 1` och sedan båda till 0 samtidigt. Vad händer, och stämmer det med vad avsnitt 6.1.3 säger?

Övning 6.5 Klockan

RÄKNA FÖR HAND

a) En Arduino Uno har klockfrekvensen 16 MHz. Hur lång är en klockcykel?

b) Hur många klockcykler går det på en millisekund? På en sekund?

c) Ett program utför 1000 instruktioner som tar en klockcykel var. Hur lång tid tar det?

d) Samma program på en krets som går på 8 MHz. Hur lång tid då?

Övning 6.6 Räknare

RÄKNA FÖR HAND

a) En 3-bitars räknare står på `000`. Vad står den på efter 11 stigande klockflanker?

b) Hur många vippor behövs för en räknare som ska kunna räkna från 0 till 999?

c) En 4-bitars räknare får en klocka på 16 Hz. Med vilken frekvens växlar den mest signifikanta biten, `Q3`?

d) Ett 8-bitars register står på 255, och räknas upp ett steg. Vad står det på efter? Varför?

Övning 6.7 Skiftregister

RÄKNA FÖR HAND

Ett 4-bitars skiftregister som i avsnitt 6.1.8 börjar med `Q0 Q1 Q2 Q3 = 0000`.

a) Bitarna 1, 1, 0 och 1 skickas in på `in`, en per klockflank. Skriv upp `Q0`–`Q3` efter varje flank.

b) Läs `Q3 Q2 Q1 Q0` som ett binärt tal. Registret innehåller `0011` (talet 3), och ett steg till skiftas in med `in = 0`. Vilket tal står det nu? Vilken räkneoperation motsvarar ett steg?

Övning 6.8 Blockschemat

FÖRSTÅELSE

Vilket block i mikrodatorn (avsnitt 6.2.2) beskrivs?

a) Håller adressen till nästa instruktion.

b) Adderar två tal, eller jämför dem.

c) Håller programmet, även när strömmen är avslagen.

d) Talar om att det senaste resultatet blev noll.

e) Tolkar instruktionen och talar om för de andra blocken vad de ska göra.

f) Ett register vars bitar är kopplade till kretsens stift.

g) Håller variablerna medan programmet kör.

h) Ger takten som alla vippor uppdateras i.

Övning 6.9 Instruktionscykeln

RÄKNA FÖR HAND

Programminnet innehåller:

Adress	Instruktion	Betyder
0	`ldi r16, 10`	lägg 10 i r16
1	`ldi r17, 20`	lägg 20 i r17
2	`add r16, r17`	r16 = r16 + r17
3	`rjmp 3`	hoppa till adress 3

a) Fyll i en tabell som den i avsnitt 6.2.4 för de sex första instruktionerna som utförs: PC före, instruktionen, vad som händer, r16 och r17 efter, PC efter.

b) Vad gör programmet efter steg 4, och hur länge? Varför behövs en sådan instruktion sist i ett program för en mikrokontroller?

Övning 6.10 Tre minnen

FÖRSTÅELSE

a) I vilket av ATmega328P:s tre minnen hamnar: programmet; en variabel som räknar hur många gånger en knapp tryckts; ett kalibreringsvärde som ska finnas kvar efter ett strömavbrott?

b) Varför finns programmet kvar när strömmen slås av, men inte variablerna?

c) Vad betyder det att ATmega328P har en Harvardarkitektur?

Övning 6.11 Bussar och minnesstorlek

RÄKNA FÖR HAND

a) ATmega328P har 2 kB SRAM, alltså 2048 byte. Hur många adressbitar behövs för att peka ut varje byte?

b) Flashminnet är 32 kB och varje instruktion är 16 bitar (2 byte). Hur många instruktioner ryms som mest, och hur många bitar behöver programräknaren?

c) Hur många olika värden kan databussen i en 8-bitars dator bära?

Övning 6.12 Självhållning och SR-lås

KONSTRUKTION FÖRDJUPNING

Titta på självhållningskretsen i avsnitt 6.1.2.

a) Vilken knapp motsvarar SR-låsets `S`, och vilken `R`?

b) Vad händer i reläkretsen om båda knapparna trycks samtidigt? Jämför med SR-låset av NOR-grindar.

c) I en del maskiner är det tvärtom önskvärt att start vinner om båda trycks, kallat **tillslagsdominerande** självhållning. Rita om kretsen så att `S1` alltid kan dra `K1`, även när `S2` är nedtryckt. Varför är den varianten sällsynt för maskiner där stopp är en säkerhetsfunktion?

KAPITEL 7

Mikrodatorn och Microchip Studio

ATmega328P på nära håll, och det första programmet, sexton bitar i taget.

I DET HÄR KAPITLET

- ATmega328P: mikrodatorn från blockschemat i kapitel 6, som en verklig krets på ett Arduino Uno-kort.
- De 32 registren, och varför bara hälften av dem kan laddas med en konstant.
- Statusregistret, programräknaren och de tre minnena.
- Vad en instruktion är: `ldi r16, 0x2A` isärplockad till de sexton bitar den blir.
- Assemblerspråkets byggstenar: instruktioner, operander, etiketter, kommentarer och direktiv.
- Talformat: samma tal skrivet decimalt, binärt och hexadecimalt.
- Microchip Studio: projekt, assemblering, felmeddelanden och simulering.
- **Labb 3 börjar**: del 1.1, 1.2 och 2.

I kapitel 6 byggdes mikrodatorn upp block för block. Här blir den en verklig krets, ATmega328P, med namn och siffror på varje block, och här skrivs det första programmet till den. Du lär dig vad en instruktion är, ända ned till de sexton bitar den blir i programminnet, hur ett program skrivs i assembler, och hur du bygger det och stegar igenom det i simulatorn i Microchip Studio.

Microchip Studio ska vara installerat innan du börjar, enligt avsnitt D.1 i bilaga D. Läs sedan avsnitt 7.1 och avsnitt 7.2, och repetera mikrodatorns blockschema i avsnitt 6.2 och omvandlingen mellan talsystem i kapitel 1. Avsnitt 7.3 om hur simulatorn används läser du när du sätter dig vid Microchip Studio. Övningarna finns i avsnitt 7.5, och lösningsförslagen till dem som inte är program i bilaga I.

Med det här kapitlet börjar också **Labb 3** (bilaga C): gör del 1.1, 1.2 och 2 när du har läst kapitlet.

7.1 Mikrodatorn ATmega328P

7.1.1 Från blockschema till en verklig krets

I kapitel 6 byggdes mikrodatorn upp på blockschemanivå: en CPU med ALU, register och styrenhet, ett programminne, ett dataminne, in- och utportar, och en klocka som driver alltihop. Från och med nu arbetar vi med en verklig sådan krets: **ATmega328P**, som sitter på ett Arduino Uno-kort.

ATmega328P är en **mikrokontroller**: en hel mikrodator på ett enda chip. Allt i blockschemat från kapitel 6 finns inuti den:

Del	I ATmega328P
CPU	En 8-bitars processor av typen AVR. Den räknar med 8 bitar, en byte, i taget.
Register	32 st, `r0`–`r31`, med 8 bitar vardera.
Programminne	32 kB flashminne. Programmet ligger kvar när strömmen bryts.
Dataminne	2 kB SRAM, 2048 byte. Innehållet försvinner när strömmen bryts.
In- och utportar	Port B, C och D, som styr kretsens stift.
Klocka	16 MHz på ett Arduino Uno-kort: 16 miljoner klockcykler i sekunden.

Det finns inget operativsystem, inga fönster och ingen skärm. När kretsen får ström börjar den köra det program som ligger i programminnet, instruktion för instruktion, och det fortsätter den med så länge strömmen är på. Det är **ditt** program, och ingenting annat.

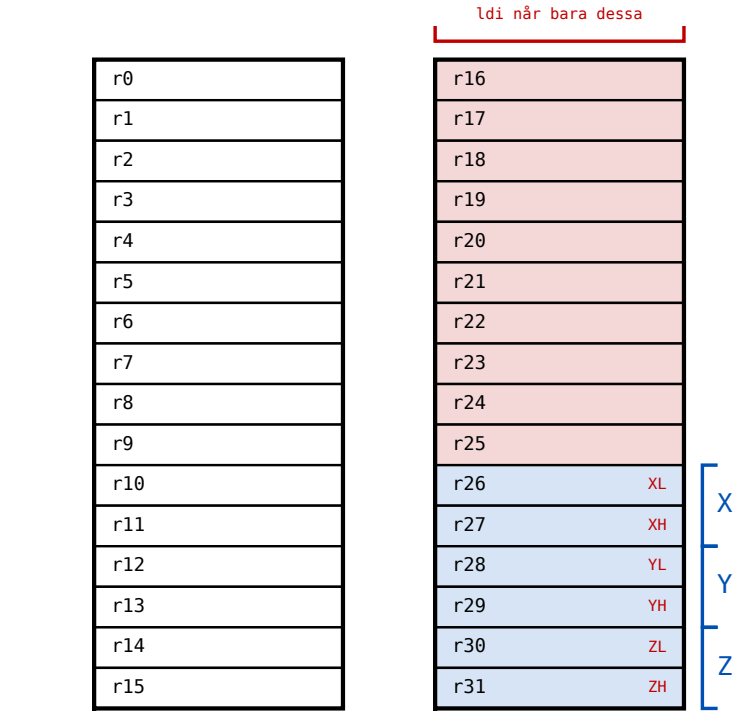
Två egenskaper hos maskinen är värda att känna till från början, eftersom de styr hur alla program i boken ser ut:

All räkning sker i registren. Det finns ingen instruktion som adderar två byte i minnet med varandra. Värdena hämtas till register, räknas där, och resultatet lämnas i ett register. Det är därför bokens program handlar så mycket om `r16`, `r17` och deras grannar.

Program och data ligger i olika minnen. Programmet ligger i flashminnet och variablerna i SRAM, och de två är skilda minnen med egna adresser (avsnitt 7.1.5). Den uppdelningen kallas **Harvardarkitektur**, och den känns igen från kapitel 6.

7.1.2 Registren

Registren är processorns arbetsbänk: 32 st, vart och ett med plats för en byte, och med namnen `r0`–`r31`. Allt processorn räknar på passerar genom dem.



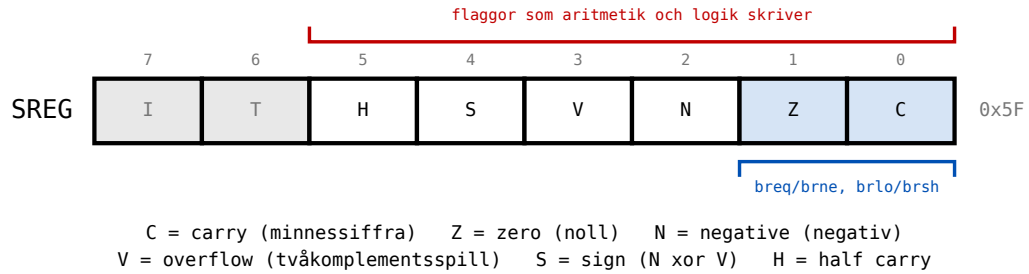
Figur 7.1 De 32 registren `r0`–`r31`.

De flesta instruktioner fungerar med vilket register som helst. Men en viktig grupp gör det inte: **bara `r16`–`r31` kan laddas med en konstant.** Instruktionen `ldi r16, 25` (*load immediate*, ladda ett fast värde) är tillåten; `ldi r5, 25` är det inte, och assemblern säger ifrån med ett felmeddelande. Samma begränsning gäller de andra instruktionerna som tar en konstant, som `subi`, `andi`, `ori` och `cpi`, som du möter i kapitel 8 och 9.

Förklaringen finns i hur instruktionen är kodad, och den står i avsnitt 7.1.6. Den praktiska regeln är enkel: **använd `r16`–`r25` som arbetsregister.** Det gör alla program i boken, och då behöver du aldrig fundera på begränsningen. Registren `r26`–`r31` bildar tre 16-bitars par, `X`, `Y` och `Z`, som används för att peka ut adresser i minnet; dem låter vi vara i den här boken.

7.1.3 Statusregistret SREG

Utöver de 32 registren finns ett register som inte räknar själv, utan som **minns något om den senaste uträkningen**: statusregistret `SREG`.



Figur 7.2 Statusregistret SREG och dess åtta flaggor.

Varje bit i SREG kallas en **flagga**. Efter en addition kan processorn till exempel komma ihåg om resultatet blev noll (flaggan Z, *zero*) eller om det blev en minnessiffra ut ur bit 7 (flaggan C, *carry*). Flaggorna är det som gör att ett program kan fatta beslut: ”om resultatet blev noll, hoppa hit”. Hur varje flagga sätts är ämnet för kapitel 8, och hur hoppen läser dem är ämnet för kapitel 9.

7.1.4 Programräknaren

Processorn måste hela tiden veta vilken instruktion som står på tur. Det håller ett särskilt register reda på: **programräknaren**, *program counter* eller PC. Den innehåller adressen i programminnet till nästa instruktion.

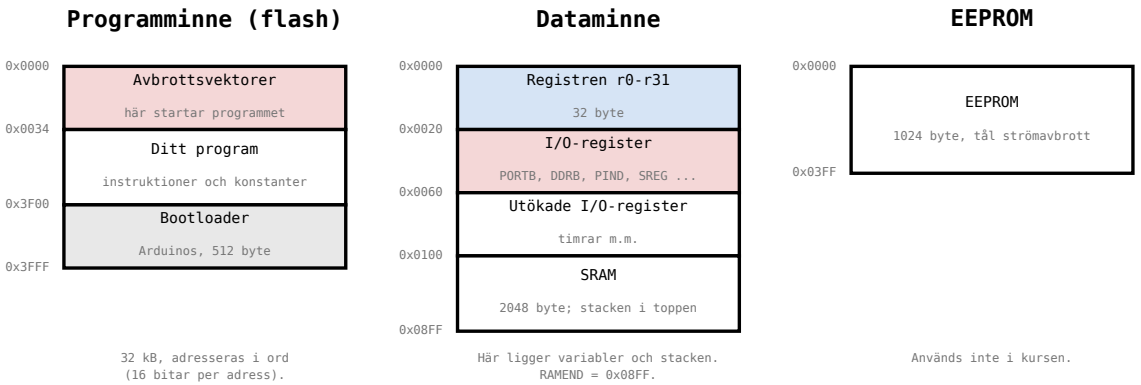
Efter varje instruktion räknas PC upp till nästa instruktion, precis som räknaren i avsnitt 6.1.7. Det är därför ett program körs uppifrån och ned. Den enda gången programräknaren gör något annat är när en instruktion **hoppar**: då skrivs en ny adress in i PC, och programmet fortsätter där.

När kretsen startar, eller återställs med resetknappen, sätts PC till 0x0000. Det är därför första raden i varje program i boken ligger på adress 0 (se avsnitt 7.2.6).

I simulatorn syns programräknaren som den gula pilen i editorn, och som värdet *Program Counter* i fönstret Processor Status.

7.1.5 Tre minnen

ATmega328P har tre minnen, och de har var sina adresser. Adress 0x0100 betyder alltså olika saker beroende på vilket minne det gäller.



Figur 7.3 ATmega328P:s tre minnen, med sina adresser.

Programminnet (flash, 32 kB) innehåller programmet: instruktionerna, en efter en. Det behåller innehållet utan ström, vilket är varför ett Arduino-kort kör sitt program direkt när det ansluts. Programminnet adresseras i **ord** om 16 bitar, eftersom nästan varje instruktion är 16 bitar lång.

Dataminnet (SRAM, 2 kB) innehåller det som ändras medan programmet kör. Det börjar på adress `0x0100` och slutar på `0x08FF`, en adress som har ett eget namn, `RAMEND`. Längst upp i dataminnet ligger **stacken**, som kapitel 10 handlar om. Under SRAM, på de lägsta adresserna, ligger två andra saker som också räknas till dataminnet: de 32 registren och **I/O-registren**.

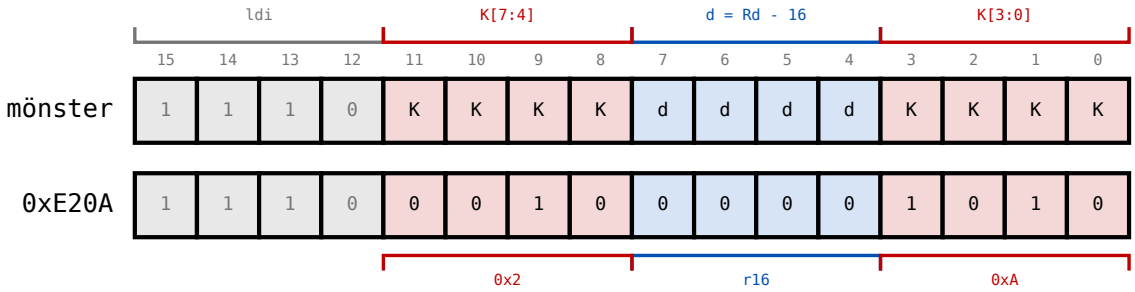
I/O-registren är de register som styr kretsens delar utanför processorn. `DDRB`, `PORTB` och `PINB` styr port B, och `SREG` är också ett I/O-register. Ett I/O-register läses och skrivs med egna instruktioner, `in` och `out`, som du möter i kapitel 8. Namnen, som `PORTB`, får du från filen `m328Pdef.inc` (se avsnitt 7.2.3).

EEPROM (1 kB) är ett litet minne som, liksom flash, tål att strömmen bryts, men som programmet självt kan skriva i. Boken använder det inte.

7.1.6 Vad en instruktion är

En instruktion är, för processorn, bara ett tal: 16 bitar i programminnet. Det tal som står i programminnet kallas **maskinkod**. Assemblerspråket, som du skriver, är ett sätt för människor att skriva maskinkoden utan att räkna ut bitarna själva.

Ta instruktionen `ldi r16, 0x2A`. Den blir 16 bitar, och varje bit har en uppgift:



`ldi r16, 0x2A` blir maskinkoden `0xE20A`. Konstanten delas i två halvor, och registerfältet har bara fyra bitar, därför når `ldi` bara `r16`-`r31`.

Figur 7.4 Instruktionen `ldi r16, 0x2A` som sexton bitar.

- De fyra första bitarna, `1110`, säger att det är en `ldi`. De kallas **operationskoden**.
- De åtta bitarna märkta `K` är konstanten, `0x2A` = `0010 1010`, delad i två halvor.
- De fyra bitarna märkta `d` säger vilket register som ska laddas.

Här ser du också **varför `ldi` bara når `r16`–`r31`**. Fyra bitar kan bara räkna från 0 till 15, alltså sexton olika register. Processorn lägger till 16 till talet i fältet, så `0000` betyder `r16` och `1111` betyder `r31`. Det finns helt enkelt ingen plats i instruktionen för att skriva `r5`.

Så `ldi r16, 0x2A` blir maskinkoden:

```
1110  0010  0000  1010   =   0xE20A
 ldi  K7-4   r16  K3-0
```

Du kan kontrollera det själv i listfilen som Microchip Studio skriver när programmet byggs, se avsnitt 7.2.7. Nästan alla instruktioner i boken är ett ord, 16 bitar, långa.

7.1.7 En instruktion per klockcykel

Klockan på ett Arduino Uno-kort går med **16 MHz**. Det betyder 16 miljoner klockcykler per sekund, och en klockcykel tar

$$T = \frac{1}{16\,000\,000\ \text{Hz}} = 62{,}5\ \text{ns}$$

De flesta instruktioner tar **en klockcykel**. Några tar två, till exempel hopp; ett par tar fler. Ett program med tio enkla instruktioner tar alltså bara 0,625 mikrosekunder. Det är så snabbt att ingen människa kan se ett program köra, vilket är skälet till att vi stegar i simulatorn, och till att programmen i kapitel 9 måste lägga in tidsfördröjningar för att en lysdiod ska hinna synas.

Att antalet klockcykler är fast för varje instruktion är en ovanlig egenskap. Den gör att du kan räkna ut exakt hur lång tid ett program tar, med papper och penna, innan du har kört det. Det gör vi i kapitel 9.

7.1.8 Vad maskinen inte har

En mikrokontroller är enkel, och det är värt att veta vad som saknas innan man letar efter det.

Inget operativsystem att återvända till. Ett program på en dator avslutas och lämnar tillbaka kontrollen till operativsystemet. Här finns ingenting att lämna tillbaka till. Ett program som ”tar slut” fortsätter att köra det som råkar ligga efter det i programminnet. Därför slutar varje program i boken med en loop som hoppar till sig själv, se avsnitt 7.2.6.

Inget skydd. Ingenting hindrar ett program från att skriva i fel register eller hoppa till fel ställe. Det blir inget felmeddelande när programmet körs, bara fel resultat. Därför är det så viktigt att förutsäga vad varje instruktion ska göra, och kontrollera det i simulatorn.

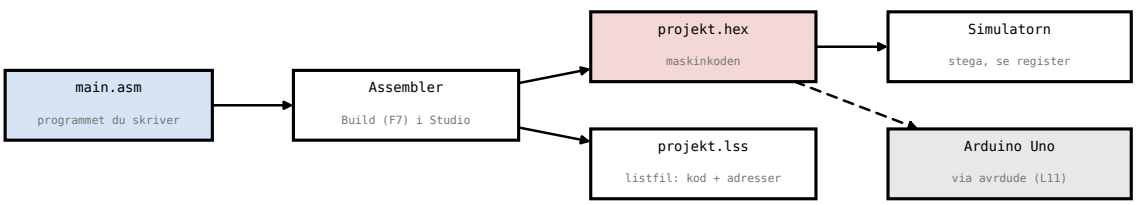
Ingen division. Processorn kan addera, subtrahera och multiplicera, men inte dividera. Division med 2, 4, 8 och så vidare görs genom att skifta bitarna (kapitel 8).

7.2 Assemblerspråk

7.2.1 Från assembler till maskinkod

Processorn förstår bara maskinkod: tal i programminnet, som i avsnitt 7.1.6. **Assemblerspråk** är maskinkoden skriven med ord i stället för tal. Varje rad med en instruktion blir en instruktion i maskinkod, och ingenting annat: `inc r16` blir exakt en instruktion, som ökar `r16` med ett.

Programmet som översätter kallas **assemblern**, och översättningen kallas att **assemblera**. I Microchip Studio sker det när du bygger projektet med **F7**.



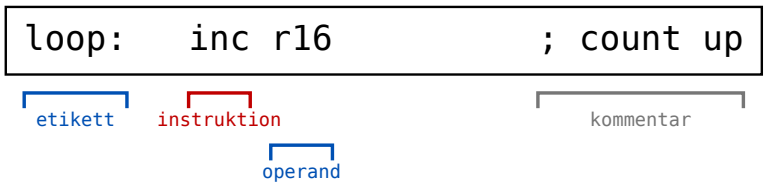
Figur 7.5 Verktygskedjan: från `main.asm` till hexfil och listfil.

Assemblern är det enklaste verktyget i kedjan. Den ändrar inte ordning på något, den fyller inte i något du glömt, och den kontrollerar inte att programmet gör något vettigt. Den

kontrollerar bara att varje rad går att översätta. Allt programmet gör är alltså sådant du själv har skrivit. Det är både det svåra och det fina med assembler: det finns ingenting dolt.

7.2.2 En rad assembler

En rad i ett assemblerprogram kan ha fyra delar, och alla fyra är valfria:




```
Etiketten namnger adressen, instruktionen säger vad processorn ska göra, operanden
säger med vad, och kommentaren är till för människor: assemblern hoppar över den.
```

Figur 7.6 En rad assembler med etikett, instruktion, operand och kommentar.

- **Etikett**: ett namn följt av kolon, först på raden. Etiketten namnger adressen där nästa instruktion hamnar, så att ett hopp kan hoppa dit med namn i stället för med en adress. Bokens etiketter skrivs med små bokstäver och understreck: `main`, `loop`, `delay_loop`.
- **Instruktion**: vad processorn ska göra, skrivet som en kort förkortning. `inc` betyder *increment*, öka med ett. Förkortningen kallas också instruktionens **mnemonic**.
- **Operander**: vad instruktionen ska arbeta med. `inc` har en operand, registret som ska ökas. `ldi r16, 25` har två, åtskilda med kommatecken: registret och värdet. När det finns två operander är den första nästan alltid **målet**, det som ändras.
- **Kommentar**: allt efter ett semikolon. Assemblern hoppar över kommentarer; de är till för den som läser programmet, vilket oftast är du själv en vecka senare.

Indrag är inte nödvändigt, men bokens program drar in instruktionerna fyra blanksteg och låter etiketterna stå längst till vänster. Då syns programmets struktur direkt.

7.2.3 Direktiv

Vissa rader börjar med en punkt. De är **direktiv**: instruktioner till assemblern, inte till processorn. Ett direktiv blir ingen maskinkod.

Direktiv	Exempel	Betydelse
`.include`	`.include "m328Pdef.inc"`	Läs in en annan fil här.
`.org`	`.org 0x0000`	Lägg nästa instruktion på den här adressen i programminnet.
`.def`	`.def counter = r16`	Ge ett register ett eget namn.
`.equ`	`.equ LIMIT = 10`	Ge en konstant ett eget namn.

`.include "m328Pdef.inc"` står först i varje program. Filen `m328Pdef.inc` följer med Microchip Studio och innehåller namnen på alla ATmega328P:s register och konstanter: `PORTB`, `DDRB`, `PIND`, `RAMEND` och flera hundra till. Utan den vet assemblern inte vad `PORTB` betyder.

`.def` gör ett program lättare att läsa. Efter `.def counter = r16` kan du skriva `inc counter`, och assemblern byter ut `counter` mot `r16`. Det blir samma maskinkod, men programmet säger nu vad registret används till. Du använder `.def` i del 6 av Labb 3 (bilaga C).

`.equ` gör samma sak för tal. Efter `.equ LIMIT = 10` betyder `LIMIT` talet 10 överallt i programmet. Konstanter skrivs med stora bokstäver, så att de syns.

7.2.4 Talformat

Ett tal kan skrivas på flera sätt i ett assemblerprogram. Det blir samma bitar i maskinkoden oavsett hur du skriver det. Talsystemen går tillbaka till kapitel 1.

Skrivsätt	Exempel	Värde
Decimalt	`ldi r16, 200`	200
Binärt, prefix `0b`	`ldi r16, 0b11001000`	$128 + 64 + 8 = 200$
Hexadecimalt, prefix `0x`	`ldi r16, 0xC8`	$12 \cdot 16 + 8 = 200$
Hexadecimalt, prefix `$`	`ldi r16, $C8`	200; äldre skrivsätt, vanligt i gamla exempel
Tecken, inom apostrofer	`ldi r16, 'A'`	$65 =$ `0x41`, tecknets ASCII-kod

Välj skrivsätt efter vad talet **betyder**. Ett antal, till exempel hur många varv en loop ska gå, skrivs decimalt. Ett bitmönster, till exempel vilka lysdioder som ska lysa, skrivs binärt, så att man ser varje bit. Hexadecimalt är det kompakta sättet att skriva ett bitmönster, och det är så simulatorn visar registren.

Exempelprogrammet `number_formats.asm` laddar samma tal på fyra sätt i fyra register. Stega igenom det och se att alla fyra blir `0xC8`.

lectures/L07/examples/number_formats.asm

```
;*******************************************************************************
; L07: One number, four ways of writing it.
;
; r16, r17, r18 and r19 all end up holding 200, written in decimal, binary and two
; hexadecimal notations. r20 holds the ASCII code of the character 'A'.
;*******************************************************************************
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi r16, 200                    ; Decimal.
    ldi r17, 0b11001000             ; Binary: 128 + 64 + 8 = 200.
```

```
    ldi r18, 0xC8                   ; Hexadecimal: 12 * 16 + 8 = 200.
    ldi r19, $C8                    ; Hexadecimal again, in the older $ notation.
    ldi r20, 'A'                    ; A character: its ASCII code, 65 = 0x41.

end:
    rjmp end
```

7.2.5 De första instruktionerna

Åtta instruktioner räcker för att skriva bokens första program. Antal klockcykler står inom parentes; varför några tar två cykler tas upp i kapitel 9.

Instruktion	Exempel	Gör
`ldi`	`ldi r16, 5`	Laddar ett fast värde i ett register, `r16`–`r31`. (1)
`mov`	`mov r18, r16`	Kopierar ett register till ett annat. `r16` ändras inte. (1)
`clr`	`clr r19`	Nollställer ett register: alla bitar blir 0. (1)
`ser`	`ser r20`	Ettställer ett register: alla bitar blir 1, `0xFF`. (1)
`inc`	`inc r18`	Ökar ett register med ett. (1)
`dec`	`dec r17`	Minskar ett register med ett. (1)
`rjmp`	`rjmp end`	Hoppar till en etikett. (2)
`nop`	`nop`	Gör ingenting, i en klockcykel. (1)

Lägg märke till ordningen i `mov r18, r16`: målet först, sedan källan. Det läses ”flytta till `r18` från `r16`”, och det är samma ordning som i `ldi r16, 5`, där `r16` är det som ändras.

Genomarbetat exempel

Här är `first_program.asm`, bokens första program. Tabellen visar registren **efter** varje instruktion. I simulatorn är alla register 0 när programmet startar.

```
main:
    ldi r16, 5                      ; r16 = 5
    ldi r17, 3                      ; r17 = 3
    mov r18, r16                    ; r18 = r16, so r18 = 5 (r16 is unchanged)
    inc r18                         ; r18 = r18 + 1 = 6
    dec r17                         ; r17 = r17 - 1 = 2
    clr r19                         ; r19 = 0
    ser r20                         ; r20 = 0xFF (all ones)

end:
    rjmp end                        ; Stay here forever.
```

Efter	r16	r17	r18	r19	r20
`ldi r16, 5`	5	0	0	0	0
`ldi r17, 3`	5	3	0	0	0
`mov r18, r16`	5	3	5	0	0
`inc r18`	5	3	6	0	0
`dec r17`	5	2	6	0	0
`clr r19`	5	2	6	0	0
`ser r20`	5	2	6	0	0xFF

`clr r19` ändrar ingenting synligt, eftersom `r19` redan var 0. Instruktionen körs ändå, och det är en god vana att nollställa ett register innan man använder det, i stället för att lita på att det redan är noll.

7.2.6 Programmets skelett

Alla program i Labb 3 utgår från samma mall, `labs/lab3/code/template.asm` (sidan 286). Så här ser dess delar ut, rad för rad:

```
.include "m328Pdef.inc"     ; Register names for the ATmega328P: PORTB, DDRB ...

.org 0x0000                 ; The processor starts here after reset.
    rjmp main               ; Jump over everything else, to main.

main:
    ; Write your program here.

end:
    rjmp end                ; Stay here forever: there is nothing to return to.
```

- **`.include`** ger registernamnen, se avsnitt 7.2.3.
- **`.org 0x0000`** säger att nästa instruktion ska ligga på adress 0 i programminnet. Det är där processorn börjar efter en reset (avsnitt 7.1.4).
- **`rjmp main`** hoppar till etiketten `main`. Det kan verka onödigt, eftersom `main` ändå kommer direkt efter. Men adresserna närmast 0 är reserverade för avbrottsvektorer, som boken inte använder, och vanan att börja med ett hopp gör att programmet fungerar även den dag något läggs där. Så börjar nästan alla AVR-program.
- **`main:`** är där ditt program börjar.
- **`end: rjmp end`** hoppar till sig självt, om och om igen. Processorn stannar alltså aldrig; den står och stampar på samma ställe. Utan den raden skulle processorn fortsätta in i det som råkar ligga efter programmet i flashminnet, vilket inte är något du har skrivit (avsnitt 7.1.8).

Från kapitel 8 och framåt byts `end` ofta ut mot en **huvudloop**: ett program som styr något gör samma sak om och om igen, till exempel läser en knapp och tänder en lysdiod.

7.2.7 Listfilen

När projektet byggs skriver assemblern, utöver hexfilen, en **listfil**: programmet med adressen och maskinkoden bredvid varje rad. I Microchip Studio har den filändelsen `.lss` och ligger bland projektets utdatafiler, i katalogen `Debug`. Listfilen för `first_program.asm` innehåller bland annat de här raderna (formatet skiljer sig lite mellan versioner, men innehållet är detsamma):

```
000000 c000      rjmp main
000001 e005      ldi r16, 5
000002 e013      ldi r17, 3
000003 2f20      mov r18, r16
000004 9523      inc r18
000005 951a      dec r17
000006 2733      clr r19
000007 ef4f      ser r20
000008 cfff      rjmp end
```

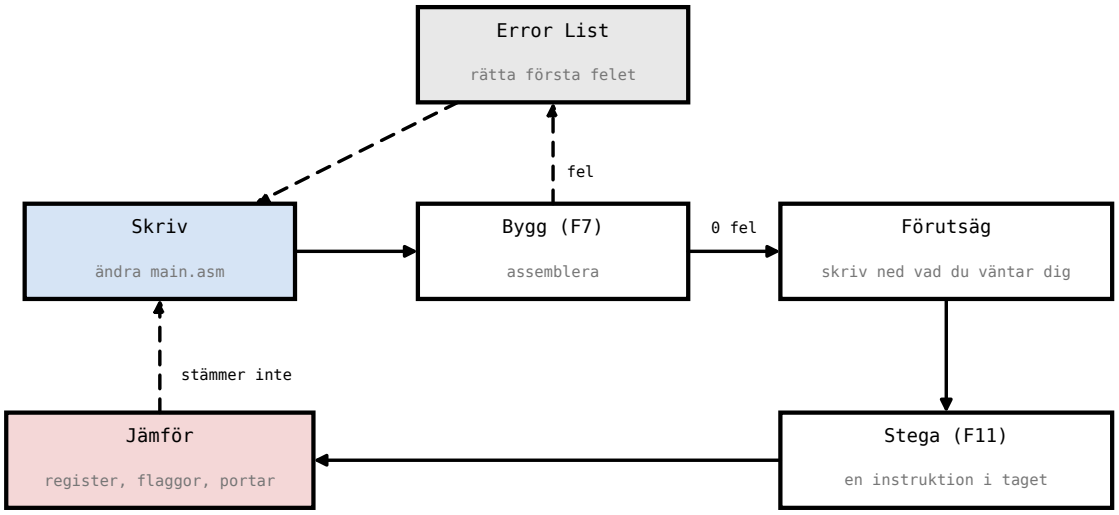
Första kolumnen är adressen i programminnet, räknad i ord, och andra kolumnen är maskinkoden, hexadecimalt. Tre saker är värda att lägga märke till:

- **`ldi r16, 5` blir `0xE005`.** Samma mönster som i avsnitt 7.1.6: `1110`, konstantens höga halva `0000`, registerfältet `0000` för `r16`, konstantens låga halva `0101`.
- **`clr r19` blir en helt annan instruktion än `ldi r19, 0`.** Den är i själva verket `eor r19, r19`, en XOR av registret med sig självt, vilket alltid blir noll. `clr` är ett bekvämt namn på den.
- **Adresserna ökar med ett för varje rad**, eftersom varje instruktion här är ett ord lång. Hela programmet ryms i nio ord, 18 byte, av programminnets 32 768.

7.3 Simulering i Microchip Studio

7.3.1 Arbetsgången

Allt arbete i Labb 3 följer samma slinga: skriv, bygg, förutsäg, stega och jämför.



Figur 7.7 Arbetsgången: skriv, bygg, förutsäg, stega och jämför.

Hur du skapar ett projekt, bygger och startar simulatorn står i bilaga D, avsnitt D.2–D.4. Det här avsnittet handlar om hur du **använder** simulatorn för att lära dig vad instruktionerna gör.

Varför simulatorn? Ett riktigt kort kör upp till 16 miljoner instruktioner i sekunden, och du ser bara slutresultatet: en lysdiod som lyser eller inte. Simulatorn låter dig köra en instruktion i taget och se varje register, varje flagga och varje port ändras. Det är det bästa sättet att förstå vad ett program gör, och att hitta felet när det inte gör det du tänkt. Nästan hela Labb 3 görs i simulatorn; kortet används först i kapitel 11.

7.3.2 Att stega och läsa registren

Starta simuleringen med **Debug → Start Debugging and Break** (**Alt+F5**). Programmet stannar på första instruktionen, och en gul pil visar var.

Den gula pilen pekar på den instruktion som står på tur, inte på den som just kördes. Det är den vanligaste källan till förvirring första gången. När pilen står på `inc r18` har `inc r18` ännu inte körts; tryck **F11** (*Step Into*), så körs den, och pilen flyttar till nästa rad.

Öppna fönstret **Processor Status** (**Debug → Windows → Processor Status**). Där finns:

- **`R00`–`R31`**: de 32 registren. Ett register som ändrades i det senaste steget visas i rött.
- **Program Counter**: adressen till nästa instruktion. Jämför den med adresserna i listfilen (avsnitt 7.2.7).

- **Status Register**: SREG, med en ruta per flagga. Flaggorna tas upp i kapitel 8.
- **Cycle Counter** och **Stop Watch**: hur många klockcykler, och hur lång tid, programmet har kört. Se avsnitt 7.3.5.

Registren visas hexadecimalt, så `r18 = 6` visas som `0x06` och `r16 = 200` som `0xC8`. Vänj dig vid det: det är så datablad, listfiler och instruktionslistor skriver, och omvandlingen är den från kapitel 1.

7.3.3 Förutsäg före varje steg

Att trycka F11 och titta på vad som händer lär dig mindre än man kan tro. Du ser att något ändrades, men inte om det var det du väntade dig. Därför en regel för hela Labb 3:

Skriv ned vad du väntar dig innan du trycker F11.

Det enklaste sättet är en tabell med en rad per instruktion och en kolumn per register du bryr dig om, som tabellen i avsnitt 7.2.5. Fyll i raden, tryck F11, och jämför med Processor Status:

- **Stämmer det** vet du att du förstår instruktionen.
- **Stämmer det inte** har du hittat något att lära dig: antingen gör instruktionen något annat än du trodde, eller så står programmet inte där du trodde. Båda är värda att reda ut innan du går vidare.

Samma vana gäller när programmen blir längre. Då förutsäger du inte varje steg, utan värdet vid en viss rad, och kör fram till den med en brytpunkt eller **Run To Cursor** (**Ctrl+F10**).

7.3.4 Felmeddelanden

Ett program som inte går att assemblera ger felmeddelanden i fönstret **Error List** när du bygger. Varje meddelande har ett radnummer; dubbelklicka på det så hoppar editorn dit. Tre fel är vanligast i början:

Felmeddelandet handlar om	Exempel på rad	Rättelse
ett ogiltigt register	`ldi r5, 3`	`ldi` når bara `r16`–`r31`: använd `ldi r20, 3`.
en okänd symbol	`rjmp mian`	Etiketten heter `main`. Stavningen måste stämma exakt.
syntax	`ldi r16 7`	Kommatecknet saknas: `ldi r16, 7`.

Två råd som sparar mycket tid:

- **Rätta det första felet först, och bygg om.** Ett enda fel ger ibland flera följdfel längre ned, som försvinner av sig själva.
- **Läs raden, inte bara meddelandet.** Meddelandet säger vad assemblern inte förstod, men felet kan sitta en bit tidigare på raden än där assemblern gav upp.

Ett program som assembleras utan fel kan fortfarande vara fel. Assemblern kontrollerar bara att varje rad går att översätta, inte att programmet gör det du vill. Det är simulatorns uppgift, och din förutsägelse är det du kontrollerar mot.

7.3.5 Cykelräknaren

Cycle Counter i Processor Status räknar hur många klockcykler programmet har kört sedan start, och **Stop Watch** visar samma sak som tid. Kontrollera först att fältet **Frequency** står på **16 MHz**, så att tiden stämmer med ett Arduino Uno-kort (bilaga D, avsnitt D.4.4).

Kör `first_program.asm` fram till raden `end:`. Cykelräknaren visar då **9**:

- `rjmp main` tar 2 cykler;
- de sju instruktionerna i `main` tar 1 cykel var, 7 cykler;
- totalt $2 + 7 = 9$ cykler, vilket är $9 \cdot 62{,}5\ \text{ns} = 562{,}5\ \text{ns}$.

Det är värt att räkna så en gång nu, för i kapitel 9 blir cykelräknaren det verktyg som mäter hur lång tid en tidsfördröjning tar. Då räknar du ut antalet cykler för hand först, och kontrollerar med stoppuret.

Ett tips till sist. Medan simuleringen står stilla kan du ändra värdet i ett register direkt i Processor Status, genom att dubbelklicka på värdet och skriva in ett nytt. Det är ett snabbt sätt att prova ett program med olika startvärden utan att bygga om det, och det används i del 12 av Labb 3.

7.4 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Beskriva ATmega328P:s delar, register, statusregister, programräknare och minnen, och koppla dem till mikrodatorns blockschema.
- Säga vilka register `ldi` kan ladda, och förklara varför utifrån instruktionens kodning.
- Förklara skillnaden mellan programminnet och dataminnet.
- Läsa en rad assembler och peka ut etikett, instruktion, operander och kommentar.
- Skriva samma tal decimalt, binärt och hexadecimalt i ett program.
- Skapa ett projekt i Microchip Studio, bygga det, rätta felen, och stega programmet i simulatorn.
- Förutsäga registrens värden efter varje instruktion i ett kort program, och kontrollera förutsägelsen i simulatorn.

Frågor att testa dig själv med

1. Varför kan `ldi` ladda `r16` men inte `r15`? Hur löser du det om du ändå behöver en konstant i `r5`?
2. Den gula pilen i simulatorn står på `inc r18`. Har `inc r18` körts?
3. Vad händer om du tar bort raden `rjmp end` sist i programmet?
4. `ldi r16, 200`, `ldi r16, 0b11001000` och `ldi r16, 0xC8`: vad skiljer maskinkoden åt?
5. Vad är skillnaden mellan en instruktion och ett direktiv? Blir `.def counter = r16` någon maskinkod?
6. Ett program assembleras utan fel men ger fel resultat. Vad säger det om vad assemblern kontrollerar?

Nästa kapitel

Kapitel 8 handlar om aritmetik, flaggor och logik i registren:

- Addition och subtraktion i registren, och vad som händer när resultatet inte får plats.
- Tvåkomplement: hur samma byte kan vara 255 eller −1.
- Flaggorna i SREG: vad varje instruktion lämnar efter sig.
- Logiska operationer, skift och rotation.
- Utporten: de första lysdioderna, i simulatorns I/O-fönster.

7.5 Övningar

> **Så kontrollerar du ditt arbete.** Övningarna märkta **Program** och **Kontroll** görs i Microchip Studio: skapa ett projekt enligt bilaga D, skriv programmet, bygg med **F7** och stega med **F11**. Förutsäg alltid resultatet innan du stegar (avsnitt 7.3.3).
> Övriga övningar görs med papper och penna. Lösningsförslag finns i bilaga I. Gör övningen innan du läser lösningen: det du nästan svarade lär dig mer än det rätta svaret gör.

Varje övning är märkt med sin sort. Det finns en **Kontroll** i det här kapitlet, övning 7.7.

Övning 7.1 Arbetsbänken FÖRSTÅELSE

a) Hur många register har ATmega328P, och hur många bitar rymmer vart och ett?

b) Vilka register kan laddas med `ldi`?

c) Förklara, med hjälp av hur instruktionen `ldi` är kodad, varför de övriga registren inte kan laddas med `ldi`. Svaret ska innehålla ett antal bitar.

d) Du vill ha talet 7 i `r5`. Skriv två instruktioner som gör det.

Övning 7.2 Tre minnen

FÖRSTÅELSE

a) I vilket minne ligger programmet?

b) I vilket minne ligger värden som ändras medan programmet kör?

c) Vilket av de två minnena behåller sitt innehåll när strömmen bryts? Varför är det viktigt för ett Arduino-kort?

d) Vad heter adressen `0x08FF` i programmen, och vad ligger där?

Övning 7.3 Samma tal, olika skrivsätt

RÄKNA FÖR HAND

a) Skriv tre `ldi`-instruktioner som var och en laddar talet 45 i `r16`: decimalt, binärt och hexadecimalt.

b) Gör samma sak för talen 255, 16 och 100.

c) Vilket decimalt värde laddas av `ldi r16, 0b01010101`, `ldi r17, 0x3F`, `ldi r18, $80` och `ldi r19, 'a'`? Tecknet `a` har ASCII-koden `0x61`.

d) En lysdiod ska lysa för varje etta i ett register. Vilket skrivsätt väljer du när du laddar mönstret, och varför?

Övning 7.4 Förutsäg registren

RÄKNA FÖR HAND

Fyll i tabellen med värdet i varje register **efter** varje instruktion, hexadecimalt. Alla register är 0 när programmet startar.

```
main:
    ldi r16, 0x10
    ldi r17, 7
    mov r18, r17
    inc r18
    inc r18
    dec r16
    mov r19, r16
    clr r17
    ser r20
    dec r20
end:
    rjmp end
```

Efter	r16	r17	r18	r19	r20
`ldi r16, 0x10`					
`ldi r17, 7`					
`mov r18, r17`					
`inc r18`					
`inc r18`					
`dec r16`					
`mov r19, r16`					
`clr r17`					
`ser r20`					
`dec r20`					

Kontrollera sedan ditt svar genom att stega programmet i simulatorn.

Övning 7.5 Hitta felen

FÖRSTÅELSE

Programmet nedan går inte att assemblera. Hitta alla fel, förklara vart och ett, och skriv det rättade programmet.

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main
    ldi r16 5
    ldi r10, 3
    mov r17 r16
    inc r20, 1
    rjmp mian
```

Övning 7.6 Ditt första egna program

PROGRAM

Skriv ett program, utifrån mallen (sidan 286), som:

1. laddar talet 12 i `r16`;
2. kopierar `r16` till `r17`;
3. ökar `r17` med 3, med `inc`;
4. laddar bitmönstret `1010 1010` i `r18`, skrivet binärt;
5. kopierar `r18` till `r19`;
6. nollställer `r18`.

a) Skriv ned vilka värden `r16`–`r19` ska ha när programmet når `end`, både decimalt och hexadecimalt.

b) Bygg och stega programmet. Stämmer värdena?

c) Hur många klockcykler har programmet kört när det når `end`? Räkna först, kontrollera sedan med Cycle Counter.

Övning 7.7 Maskinkod för hand

KONTROLL

Räkna ut för hand, kontrollera i listfilen, förklara.

Använd kodningen av `ldi` i avsnitt 7.1.6: `1110 KKKK dddd KKKK`, där `dddd` är registrets nummer minus 16.

a) Räkna ut maskinkoden, hexadecimalt, för:

- `ldi r16, 0x2A`
- `ldi r24, 0x2A`
- `ldi r31, 0xFF`

- `ldi r20, 0x03`

Visa de fyra fälten för åtminstone den första.

b) Skriv de fyra instruktionerna i ett program, bygg det, och öppna listfilen (`.lss`) i projektets katalog `Debug`. Jämför maskinkoden där med dina svar.

c) Stämde alla fyra? Om inte: vilket fält hade du fel på? Det vanligaste felet är att skriva konstanten i ett stycke i stället för att dela den i två halvor.

d) Varför finns det ingen maskinkod för `ldi r15, 0x2A`?

Övning 7.8 Programmet som inte tog slut

FÖRSTÅELSE

a) Varför slutar varje program i boken med raderna `end:` och `rjmp end`?

b) Ta bort de två raderna ur programmet i övning 7.4. Programmet går fortfarande att assemblera. Vad händer när processorn har kört den sista instruktionen?

c) Varför börjar varje program med `rjmp main`, trots att `main` kommer direkt efter?

Övning 7.9 Tillåtet eller inte?

FÖRSTÅELSE

Vilka av raderna är tillåtna? Förklara varför de otillåtna inte är det, och skriv en rad som gör det som troligen var meningen.

a) `mov r5, r16`

b) `ldi r5, 16`

c) `mov r16, 5`

d) `ldi r16, r5`

e) `inc r3`

Övning 7.10 Hur lång tid tar programmet?

RÄKNA FÖR HAND

a) Hur många klockcykler tar programmet i övning 7.4, från start tills det når `end`? Programmet står i mallen, så det börjar med `rjmp main` på adress 0.

b) Hur lång tid är det vid 16 MHz?

c) Ungefär hur många gånger per sekund skulle processorn hinna köra programmet, om det startade om efter varje varv?

Övning 7.11 Maskinkod baklänges

RÄKNA FÖR HAND FÖRDJUPNING

Tre instruktioner i en listfil har maskinkoden nedan. Alla tre är `ldi`-instruktioner. Vilka?

a) `0xE3F5`

b) `0xE0A0`

c) `0xEF0F`. Vilken annan instruktion ur avsnitt 7.2.5 gör exakt samma sak?

KAPITEL 8

Aritmetik, logik och flaggor

Vad processorn gör med åtta bitar, och vad den skriver upp om varje resultat.

I DET HÄR KAPITLET

- Addition och subtraktion i registren, och varför det inte finns någon `addi`.
- Aritmetisk rundgång: efter 255 kommer 0, och `inc` säger inte till.
- Tvåkomplement: hur samma byte kan vara 255 eller −1.
- Flaggorna i SREG: vad `C`, `Z`, `N`, `V`, `S` och `H` betyder, och vilka instruktioner som ändrar dem.
- Logiska operationer och masker: att ettställa, nollställa, invertera och testa enskilda bitar.
- Skift och rotation: multiplikation och division med 2, och ett rinnande ljus.
- Utporten: `DDRB`, `PORTB`, `out`, `sbi` och `cbi`, och portarna i simulatorns I/O-fönster.
- Labb 3, del 3–10 (bilaga C).

I kapitel 7 laddade programmen tal i registren. Det här kapitlet handlar om vad processorn gör med dem: adderar och subtraherar, ändrar enskilda bitar och flyttar dem, och vad den skriver upp om varje resultat i statusregistrets flaggor. Flaggorna är vad kapitel 9 bygger på, när programmen börjar välja väg. Kapitlet slutar med utporten, där bitarna i ett register blir spänningar på kretsens stift.

Läs avsnitt 8.1 först, och repetera gärna binär addition i avsnitt 1.1.7 och grindarnas sanningstabeller i avsnitt 2.1. Gör två förutsägelser innan du kör något i simulatorn: vad `r16` och `C` blir efter `add` när `r16` = 200 och `r17` = 100, och vilka flaggor som sätts av `0x7F + 0x01`, särskilt om `V` blir 0 eller 1. Läs sedan avsnitt 8.2 om logik och skift och avsnitt 8.3 om utporten.

Övningarna finns i avsnitt 8.5, och lösningsförslagen till dem i bilaga I. Till kapitlet hör Labb 3, del 3–10 (avsnitt C.8). Gör klart del 1.1, 1.2 och 2 från kapitel 7 först, om de inte redan är gjorda.

8.1 Aritmetik och flaggor

8.1.1 Addition i ett register

Instruktionen `add` adderar två register och lägger summan i det första:

```
ldi r16, 25
ldi r17, 17
add r16, r17                    ; r16 = r16 + r17 = 42. r17 is unchanged.
```

Processorn räknar binärt, precis som i avsnitt 1.1: bit för bit från höger, med en minnessiffra, **carry**, när en kolumn blir 2 eller mer.

```
  carry     0 0 1 0 0 0 1 0  (in to each column)
  25    =   0 0 0 1 1 0 0 1
+ 17    =   0 0 0 1 0 0 0 1
  --------------------------
  42    =   0 0 1 0 1 0 1 0  =  0x2A
```

Bit 0: 1 + 1 = 10 binärt, alltså 0 och en etta i minne till bit 1. Bit 4: 1 + 1 = 10 igen, en etta i minne till bit 5. Resten av kolumnerna blir 0 eller 1 utan minne.

Två instruktioner till adderar:

- **`inc r16`** adderar 1. Den har redan dykt upp i kapitel 7.
- **`adc r16, r17`** adderar `r17` **och carryflaggan** till `r16`. Den behövs för tal större än en byte, och tas upp i kapitel 10.

Det finns **ingen instruktion som adderar en konstant**, ingen `addi`. Hur man ändå gör det står i avsnitt 8.1.4.

8.1.2 När summan inte får plats

En byte har åtta bitar och rymmer talen 0 till 255. Vad händer med 200 + 100 = 300?

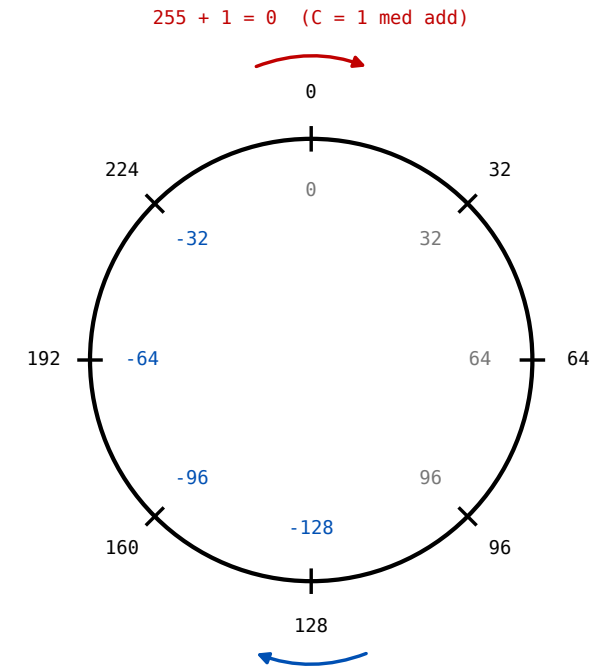
```
  200   =     1 1 0 0 1 0 0 0
+ 100   =     0 1 1 0 0 1 0 0
  ----------------------------
  300   =   1 0 0 1 0 1 1 0 0
            ^
            nionde biten: hamnar i carryflaggan C
```

Summan behöver nio bitar. Registret behåller de åtta lägsta, `0010 1100` = **44**, och den nionde biten, minnessiffran ut ur bit 7, hamnar i **carryflaggan `C`** i statusregistret. Så `C = 1` efter additionen betyder: ”resultatet fick inte plats, det riktiga svaret är 256 större”. 44 + 256 = 300.

Processorn stannar inte, och säger inte ifrån. Det är programmets sak att titta på `C` om det spelar roll, och det är just det flaggorna är till för (avsnitt 8.1.6).

8.1.3 Aritmetisk rundgång

En byte fungerar som mätaren i en bil: efter 999 999 kommer 000 000. Efter 255 kommer 0, och före 0 kommer 255. Det kallas **aritmetisk rundgång**.



Figur 8.1 De 256 värdena i en byte som en urtavla, utan tecken utanför och med tecken innanför.

Tänk dig talen på en urtavla. `inc` flyttar ett steg medurs, `dec` ett steg moturs. Från 255 tar `inc` dig till 0, och från 0 tar `dec` dig till 255.

Här finns en fälla. `inc` och `dec` ändrar **inte** carryflaggan. När `inc` slår om från 255 till 0 sätts `Z`, eftersom resultatet blev noll, men `C` lämnas som den var. Vill man veta om en ökning slog om måste man antingen använda `add` med ett register som innehåller 1, som sätter `C`, eller titta på `Z` efter `inc`.

Instruktion	Från	Till	Z	C
`inc r16`	255	0	1	oförändrad
`add r16, r17` med `r17` = 1	255	0	1	1
`dec r16`	0	255	0	oförändrad

Varför är `inc` och `dec` gjorda så? Därför att de oftast används som räknare i loopar, och då vill man kunna räkna utan att förstöra `C`, som kanske håller något viktigt. Det blir tydligt i kapitel 10, där `C` bär minnessiffran mellan två byte.

8.1.4 Subtraktion

Subtraktion finns i tre varianter:

Instruktion	Exempel	Gör
`sub`	`sub r16, r17`	`r16 = r16 - r17`
`subi`	`subi r16, 5`	`r16 = r16 - 5`, med en konstant. Bara `r16`–`r31`.
`dec`	`dec r16`	`r16 = r16 - 1`

Om det man drar ifrån är större än det man drar ifrån det från, får man **låna** från en tänkt nionde bit. Då sätts **`C = 1`**, som här betyder *lån* (*borrow*) i stället för minnessiffra:

```
ldi r16, 20
ldi r17, 50
sub r16, r17                ; 20 - 50: r16 = 226 = 0xE2, and C = 1 (a borrow)
```

226 är 256 + 20 − 50. Rundgången gäller åt båda hållen.

Att addera en konstant med `subi`

Eftersom det inte finns någon `addi` adderar man en konstant genom att **subtrahera dess negativa värde**:

```
subi r16, -5                    ; r16 = r16 - (-5) = r16 + 5
```

Det fungerar på grund av rundgången: −5 lagras som 251 (avsnitt 8.1.5), och att dra ifrån 251 är detsamma som att lägga till 5 och slå om ett varv. Tänk på att flaggorna då blir en subtraktions flaggor: `C` betyder lån, inte minnessiffra, så `C` blir 1 när summan **inte** slår om, tvärtom mot `add`. Använd `subi` med negativ konstant för att räkna, inte för att testa carry.

8.1.5 Tvåkomplement: negativa tal

En byte är bara åtta bitar. Om de betyder ett tal utan tecken, 0 till 255, eller ett tal med tecken, −128 till 127, bestämmer **programmet**, inte processorn. Samma bitar kan läsas på två sätt:

Bitar	Hex	Utan tecken	Med tecken
`0000 0000`	0x00	0	0
`0000 0001`	0x01	1	1
`0111 1111`	0x7F	127	127
`1000 0000`	0x80	128	−128
`1111 1011`	0xFB	251	−5
`1111 1111`	0xFF	255	−1

Sättet att skriva negativa tal kallas **tvåkomplement**. Det har tre egenskaper som gör det till det alla processorer använder:

- **Bit 7 är teckenbiten.** Är den 1 är talet negativt, när byten läses med tecken.
- **$-x$ är $256 - x$.** -1 är 255, -5 är 251 och -128 är 128. Det är rundgången igen: ett steg moturs från 0 är både 255 och -1.
- **Addition och subtraktion fungerar likadant för båda tolkningarna.** `add` behöver inte veta om talen har tecken: 0xFB + 0x07 = 0x02 är både $251 + 7 = 258 - 256 = 2$ och $-5 + 7 = 2$.

Att bilda ett negativt tal

Så räknar du ut tvåkomplementet, det negativa talet, för hand: **invertera alla bitar och lägg till 1.**

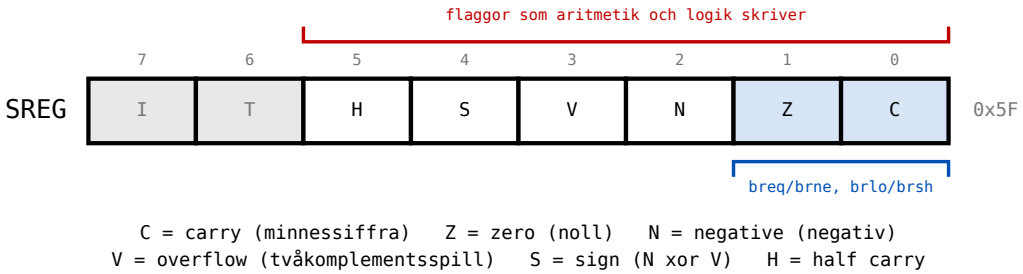
```
5           =  0000 0101
invertera   =  1111 1010
lägg till 1 =  1111 1011  =  0xFB  =  -5
```

Instruktionen **`neg r16`** gör exakt detta: `r16 = 0 - r16`.

Det är samma sak Windows miniräknare visar i läget Programmerare när den visar ett negativt tal som `FFFF FFFF FFFF FFFB`: det är -5 i tvåkomplement med 64 bitar i stället för 8.

8.1.6 Flaggorna

Efter varje aritmetisk eller logisk instruktion skriver processorn in några fakta om resultatet i statusregistret `SREG`, en bit per faktum. Bitarna kallas **flaggor**.



Figur 8.2 Statusregistret SREG och dess flaggor.

Flagga	Namn	Sätts till 1 när
`C`	carry	det blev en minnessiffra ut ur bit 7 vid addition, eller ett lån vid subtraktion
`Z`	zero	resultatet blev noll
`N`	negative	bit 7 i resultatet är 1, alltså negativt om talet har tecken
`V`	overflow	resultatet blev fel **med tecken**: två positiva tal gav ett negativt, eller tvärtom
`S`	sign	`N` xor `V`: det rätta tecknet, även när `V` säger att resultatet slog om
`H`	half carry	det blev en minnessiffra ut ur bit 3; används sällan

De två sista bitarna, `I` och `T`, är inte resultatflaggor och används inte i boken.

`C` och `V` svarar på samma fråga för de två tolkningarna. `C = 1` betyder att resultatet inte fick plats om talen är **utan** tecken. `V = 1` betyder att resultatet inte fick plats om talen är **med** tecken. Processorn vet inte vilken tolkning programmet använder, så den räknar ut båda, och programmet tittar på den flagga som passar.

Inte alla instruktioner ändrar alla flaggor. `ldi`, `mov`, `out` och hoppen ändrar inga flaggor alls. `inc` och `dec` ändrar `Z`, `N`, `V` och `S`, men inte `C`. Vilka flaggor varje instruktion ändrar står i kolumnen *Flaggor* på referensbladet i bilaga E. Det är värt att titta där när ett program beter sig konstigt: en flagga som man trodde var satt kan ha skrivits över av instruktionen före.

8.1.7 Fyra genomarbetade exempel

Programmet `flags_demo.asm` gör de fyra uträkningarna nedan. Stega det och jämför SREG i Processor Status efter varje uträkning.

lectures/L08/examples/flags_demo.asm

```
;***************************************************************************
; L08: Three additions and a subtraction, and the flags each one leaves behind.
;
```

```
; Set a breakpoint on each of the labels check1 to check4 (or step with F11) and
; read SREG in Processor Status. The expected flags are written beside each
; instruction.
;***********************************************************************************
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi r16, 0x7F                   ; 127, the largest positive signed byte.
    ldi r17, 0x01
    add r16, r17                    ; 0x80: C=0 Z=0 N=1 V=1 S=0 H=1
check1:
    ldi r16, 0xFF                   ; 255, the largest unsigned byte.
    add r16, r17                    ; 0x00: C=1 Z=1 N=0 V=0 S=0 H=1
check2:
    ldi r16, 0x80                   ; -128 as a signed byte.
    ldi r17, 0x80
    add r16, r17                    ; 0x00: C=1 Z=1 N=0 V=1 S=1 H=0
check3:
    ldi r16, 5
    ldi r17, 7
    sub r16, r17                    ; 0xFE: C=1 Z=0 N=1 V=0 S=1 H=1
check4:
    rjmp check4
```

0x7F + 0x01 = 0x80

```
   0111 1111    127  eller  +127
 + 0000 0001      1  eller    +1
   ---------
   1000 0000    128  eller  -128
```

- `C = 0`: ingen minnessiffra ut ur bit 7. Utan tecken är 128 rätt.
- `Z = 0`: resultatet är inte noll.
- `N = 1`: bit 7 är 1.
- `V = 1`: två positiva tal gav ett negativt. Med tecken är svaret fel, +127 + 1 får inte plats.
- `S = N xor V = 0`: det rätta svaret, 128, är positivt.
- `H = 1`: bit 3 + bit 3 gav en minnessiffra, 1111 + 0001 i den låga halvan.

0xFF + 0x01 = 0x00

- `C = 1`: 255 + 1 = 256 får inte plats utan tecken.

- `Z = 1`: resultatet är noll.
- `N = 0`, `V = 0`: med tecken är det $-1 + 1 = 0$, vilket är rätt.
- `S = 0`, `H = 1`.

0x80 + 0x80 = 0x00

- `C = 1`: $128 + 128 = 256$ får inte plats utan tecken.
- `Z = 1`: resultatet är noll.
- `V = 1`: med tecken är det $-128 + (-128) = -256$, som inte heller får plats. Två negativa tal gav noll, och det är fel.
- `N = 0`, `S = N xor V = 1`: det rätta svaret, -256, är negativt. `H = 0`.

0x05 – 0x07 = 0xFE

- `C = 1`: $5 - 7$ kräver ett lån. Utan tecken blir det 254.
- `Z = 0`, `N = 1`: bit 7 är 1.
- `V = 0`: med tecken är $5 - 7 = -2$, och 0xFE **är** -2. Svaret är rätt.
- `S = 1`: resultatet är negativt. `H = 1`: den låga halvan, $5 - 7$, behövde också låna.

Lägg märke till det sista exemplet: `C = 1` men `V = 0`. Utan tecken är svaret ”fel” (det skulle ha varit -2, som inte finns bland talen 0–255), men med tecken är det rätt. Samma bitar, två tolkningar, två flaggor.

8.2 Logiska operationer, skift och rotation

8.2.1 Logiska operationer, bit för bit

Grindarna från kapitel 2 finns som instruktioner. De arbetar på alla åtta bitar i ett register samtidigt, men varje bit för sig: bit 0 i resultatet beror bara på bit 0 i de två operanderna, bit 1 bara på bit 1, och så vidare. Det är åtta grindar bredvid varandra.

Instruktion	Exempel	Gör, för varje bit
`and`	`and r16, r17`	1 om båda bitarna är 1
`andi`	`andi r16, 0x0F`	samma, med en konstant
`or`	`or r16, r17`	1 om minst en av bitarna är 1
`ori`	`ori r16, 0x80`	samma, med en konstant
`eor`	`eor r16, r17`	1 om bitarna är olika (XOR)
`com`	`com r16`	inverterar, NOT

Ett exempel med `and`:

```
    r16  =  1011 0110
    r17  =  0110 0011
    ----------------- and
    r16  =  0010 0010
```

Bara i bit 5 och bit 1 är båda operanderna 1, så bara de bitarna blir 1 i resultatet.

Det finns ingen `eori`. Vill man göra XOR med en konstant laddar man först konstanten i ett register och använder `eor`:

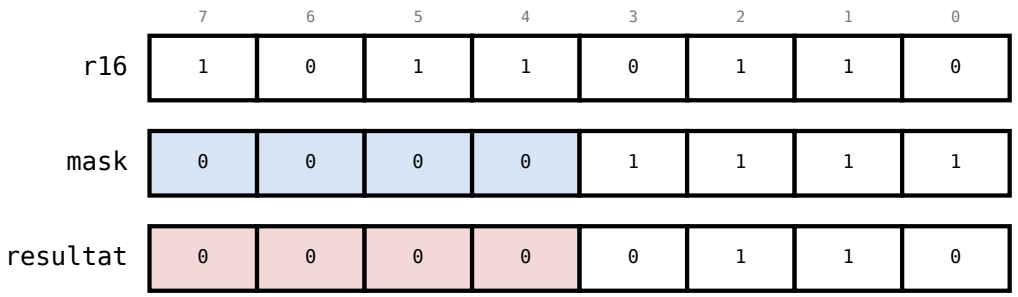
```
ldi r24, 0b11000000
eor r16, r24                    ; Toggle bits 7 and 6 of r16.
```

8.2.2 Masker

Den vanligaste användningen av de logiska instruktionerna är att ändra **vissa** bitar i ett register och lämna de andra i fred. Konstanten som väljer vilka bitar kallas en **mask**. Det behövs hela tiden när man styr en port: en port har åtta stift, och man vill ofta ändra ett av dem utan att röra de andra sju.

Nollställa bitar: `andi`

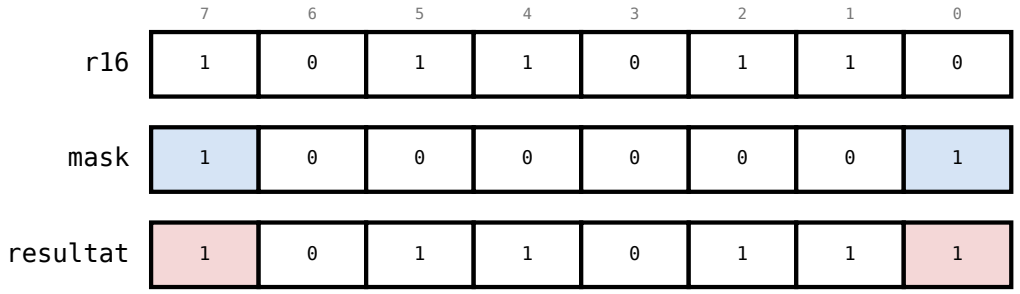
En nolla i masken tvingar biten till 0, en etta låter den vara som den var.



Figur 8.3 Att nollställa bitar med `andi`.

Ettställa bitar: `ori`

En etta i masken tvingar biten till 1, en nolla låter den vara.

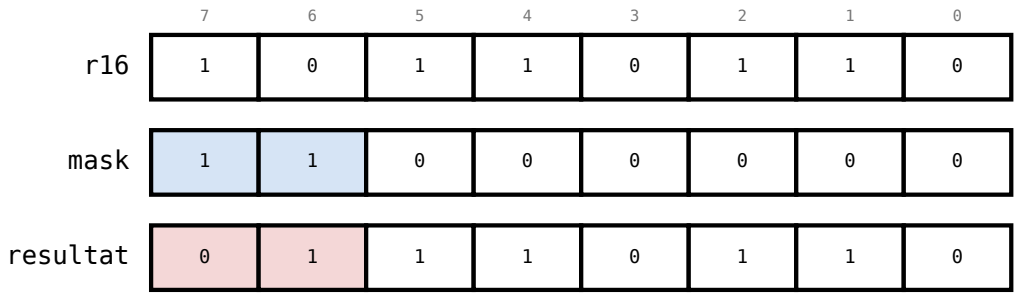



```
OR med en etta i masken ettställer biten; en nolla låter den vara.
ori r16, 0b10000001 ettställer bit 7 och bit 0.
```

Figur 8.4 Att ettställa bitar med `ori`.

Invertera bitar: `eor`

En etta i masken vänder på biten, en nolla låter den vara.




```
XOR med en etta i masken inverterar biten; en nolla låter den vara.
eor med 0b11000000 inverterar bit 7 och bit 6.
```

Figur 8.5 Att invertera bitar med `eor`.

Testa en bit: `andi` och `Z`

Vill man veta om en viss bit är 1 kan man göra AND med en mask som bara har den biten satt. Blir resultatet noll var biten 0, annars 1, och flaggan `Z` säger vilket:

```
    andi r16, 0b00000100            ; Keep only bit 2. Z = 1 if bit 2 was 0.
```

Nackdelen är att `r16` ändras. Hur man testar en bit utan att förstöra registret, och vad man gör med `Z` efteråt, kommer i kapitel 9.

Sammanfattning

Vill du	Använd	Masken har
nollställa bitar	`andi`	0 i de bitar som ska bli 0, 1 i resten
ettställa bitar	`ori`	1 i de bitar som ska bli 1, 0 i resten
invertera bitar	`eor`	1 i de bitar som ska vändas, 0 i resten
testa en bit	`andi`, sedan `Z`	1 i den bit som testas, 0 i resten

Alla exemplen finns i programmet `masks.asm`:

lectures/L08/examples/masks.asm

```
;*******************************************************************************
; L08: Setting, clearing, toggling and testing bits with masks.
;
; Every result goes to its own register, so they can all be read at the end.
;*******************************************************************************
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

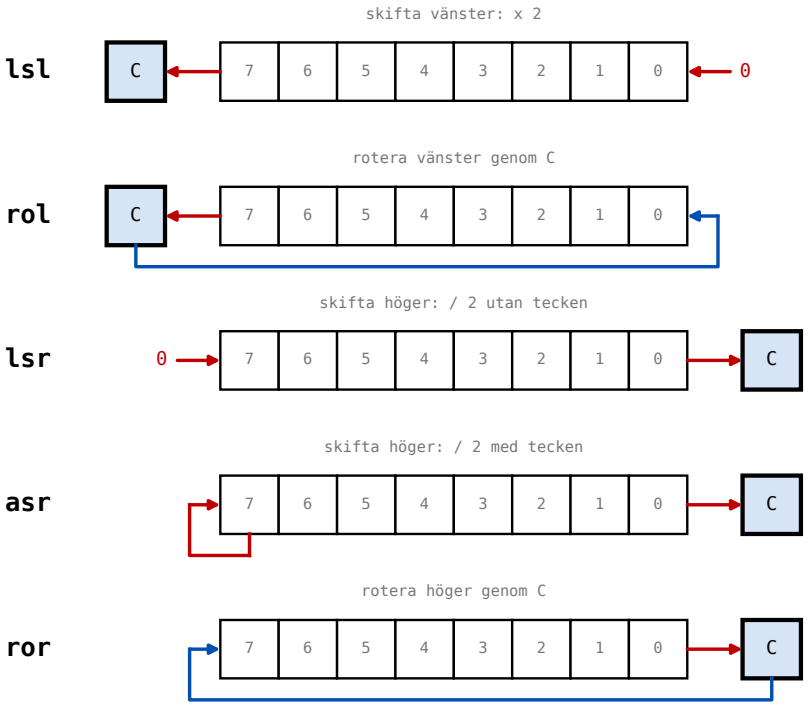
main:
    ldi r16, 0b10110110             ; The starting pattern, used by every example.

    mov r17, r16
    andi r17, 0b00001111            ; Clear the upper four bits: 0b00000110
    mov r18, r16
    ori r18, 0b00000001             ; Set bit 0: 0b10110111
    mov r19, r16
    ldi r24, 0b11000000             ; No eori, so the mask goes in a register.
    eor r19, r24                    ; Toggle bits 7 and 6: 0b01110110
    mov r20, r16
    com r20                         ; Invert every bit: 0b01001001
    mov r21, r16
    andi r21, 0b00000100            ; Test bit 2: the result is not zero, so Z = 0.
```

```
end:
    rjmp end
```

8.2.3 Skift

Ett **skift** flyttar alla bitar ett steg åt vänster eller höger. Biten som ramlar ut i ena änden hamnar i carryflaggan `C`, och i andra änden kommer en ny bit in.



I alla fem flyttas varje bit ett steg. Det som skiljer är ändarna: vad som skiftas in, och att biten som skiftas ut alltid hamnar i carryflaggan C.

Figur 8.6 Skiften och rotationerna `lsl`, `rol`, `lsr`, `asr` och `ror`.

`lsl`, *logical shift left*: alla bitar ett steg åt vänster, en nolla in i bit 0, bit 7 till `C`. Att flytta ett binärt tal ett steg åt vänster är att **multiplicera med 2**, precis som att lägga till en nolla sist i ett decimalt tal multiplicerar med 10:

```
3    =  0000 0011
lsl  =  0000 0110  =  6
lsl  =  0000 1100  =  12
```

`lsr`, *logical shift right*: alla bitar ett steg åt höger, en nolla in i bit 7, bit 0 till `C`. Det är att **dividera med 2**, och biten i `C` är resten: `lsr` på 7 ger 3 och `C = 1`.

`asr`, *arithmetic shift right*: som `lsr`, men bit 7 behåller sitt värde i stället för att bli 0. Det gör att division med 2 blir rätt även för negativa tal med tecken:

```
-16  =  1111 0000
asr  =  1111 1000  =  -8     (rätt, med tecken)
lsr  =  0111 1000  =  120    (fel, med tecken: teckenbiten försvann)
```

Använd `lsr` för tal utan tecken och `asr` för tal med tecken. Processorn kan inte dividera på något annat sätt (avsnitt 7.1.8), så division med 2, 4, 8 och så vidare görs alltid med skift.

8.2.4 Rotation

En **rotation** är ett skift där biten som skiftas in inte är en nolla, utan carryflaggans gamla värde. Biten som skiftas ut hamnar som vanligt i `C`. Man kan tänka sig att de åtta bitarna och `C` sitter i en ring om nio platser.

- **`rol`**, *rotate left*: bit 7 till `C`, och `C` in i bit 0.
- **`ror`**, *rotate right*: bit 0 till `C`, och `C` in i bit 7.

Ett exempel med `rol`, som börjar med `C = 0`:

```
r16 = 1001 0011, C = 0
rol   0010 0110, C = 1     (bit 7 till C, gamla C = 0 in i bit 0)
rol   0100 1101, C = 0     (bit 7 till C, gamla C = 1 in i bit 0)
```

Ettan som ramlade ut ur bit 7 kom tillbaka in i bit 0 ett steg senare. Inget försvinner i en rotation.

Ett rinnande ljus

Programmet `running_light.asm` visar en etta som vandrar över port B med `rol`:

```
    clc                         ; C = 0, so no stray one is rotated in.
    ldi r16, 0b00000001         ; Start with one LED lit.

loop:
    out PORTB, r16              ; Show the pattern.
    rol r16                     ; Move it one place to the left, through C.
    rjmp loop
```

`clc` (*clear carry*) nollställer `C` innan första rotationen. Stega programmet och titta på PORTB i I/O-fönstret: ettan går från bit 0 till bit 7, sedan är **alla** lysdioder släckta ett steg, och sedan börjar ettan om i bit 0. Förklaringen är ringen om nio platser: det steg då alla är släckta ligger ettan i `C`.

8.2.5 swap

`swap r16` byter plats på de två halvorna av en byte, de fyra övre och de fyra nedre bitarna. En halv byte kallas en **nibble**, och motsvarar en hexadecimal siffra:

```
r16  =  0x3A  =  0011 1010
swap =  0xA3  =  1010 0011
```

Den används när man vill arbeta med en hexadecimal siffra i taget, till exempel för att visa en byte som två tecken. Det gör du i Labb 3 del 18.

8.2.6 Flaggorna efter logik och skift

De logiska instruktionerna och skiften ändrar också flaggorna, men inte på samma sätt som aritmetiken:

- **`and`, `andi`, `or`, `ori`, `eor`** sätter `Z` och `N` efter resultatet, nollställer alltid `V`, och rör inte `C`.
- **`com`** gör samma sak och sätter dessutom alltid `C = 1`.
- **Skiften och rotationerna** lägger den utskiftade biten i `C`, och sätter `Z` och `N` efter resultatet.

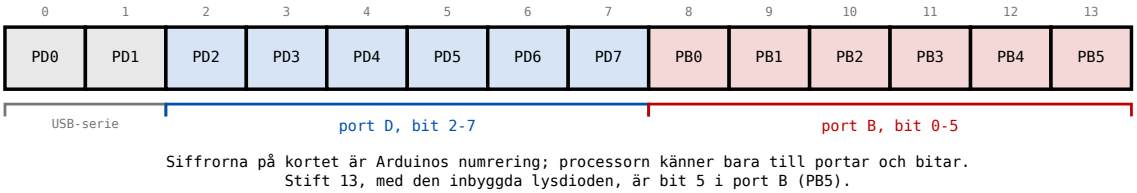
Att `and` lämnar `C` orörd kommer till nytta i kapitel 9: man kan testa en bit med `andi` och sedan hoppa på `Z` utan att förstöra en carry man vill spara.

8.3 Utporten

8.3.1 Portar och stift

Hittills har programmen bara ändrat register inuti processorn. För att styra något utanför, en lysdiod, ett relä, en motor, behövs en **utport**: ett register vars bitar är kopplade till kretsens stift. En etta i biten ger 5 V på stiftet, en nolla ger 0 V.

ATmega328P har tre portar, B, C och D, med upp till åtta stift var. På ett Arduino Uno-kort är stiften numrerade på Arduinos eget sätt, och de motsvarar portbitar så här:



Figur 8.7 Arduinokortets digitala stift 0–13 och portbitarna de motsvarar.

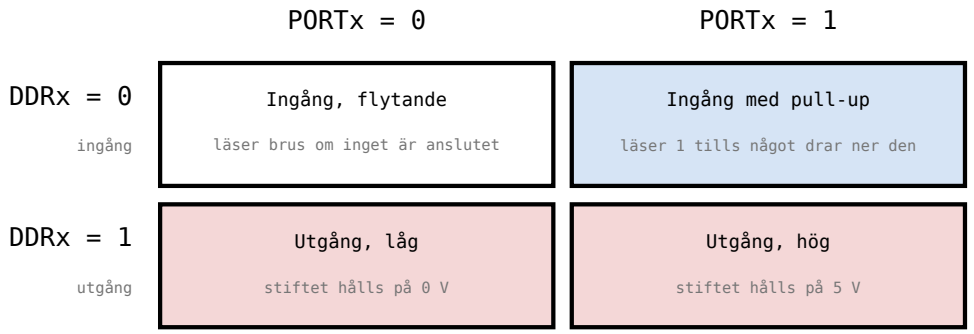
I boken är **port B utporten**. I simulatorn syns alla åtta bitar i port B. På Arduino Uno-kortet går bara bit 0–5 ut till stiften 8–13; bit 6 och 7 används av klockan. Port D används som **inport** i kapitel 10.

8.3.2 Tre register per port

Varje port styrs av tre I/O-register. För port B heter de `DDRB`, `PORTB` och `PINB`:

Register	Betydelse
`DDRB`	*Data Direction Register*. En bit per stift: 1 = utgång, 0 = ingång.
`PORTB`	För en utgång: nivån som drivs ut, 1 = 5 V, 0 = 0 V.
`PINB`	Stiftets verkliga nivå, att läsa. Används för ingångar, i kapitel 10.

Två bitar per stift, `DDRB` och `PORTB`, ger fyra kombinationer:



Figur 8.8 De fyra lägena som `DDRx` och `PORTx` ger ett stift.

För en utport räcker den nedre raden: **`DDRB` avgör att stiftet är en utgång, och `PORTB` avgör om det är högt eller lågt.** Alla bitar i `DDRB` är 0 när processorn startar, så varje stift är en ingång tills programmet säger något annat.

8.3.3 Att skriva till en port: `out`

I/O-register skrivs med instruktionen **`out`**, och läses med **`in`**:

```
    ser r16                         ; r16 = 0xFF
    out DDRB, r16                   ; Every pin of port B is an output.
    ldi r16, 0b00100001
    out PORTB, r16                  ; PB5 and PB0 high, the other six low.
```

Tre saker att lägga märke till:

- **`out` tar ett register, inte en konstant.** Det finns ingen `out PORTB, 0x21`. Värdet laddas först i ett register med `ldi`.
- **Ordningen är I/O-registret först**, sedan registret: `out PORTB, r16`, ”ut till `PORTB` från `r16`”. För `in` är det tvärtom: `in r16, PIND`, ”in till `r16` från `PIND`”. Målet står alltid först, precis som i `ldi` och `mov`.
- **`out` skriver alla åtta bitarna.** Stift som var höga och får en nolla blir låga. Vill man bara ändra ett stift används `sbi` eller `cbi` (avsnitt 8.3.4), eller en mask (avsnitt 8.3.5).

Namnen `DDRB` och `PORTB` kommer från `m328Pdef.inc` (avsnitt 7.2.3). De är adresserna till I/O-registren, och `out` och `in` tar just den sortens adress.

8.3.4 Ett stift i taget: `sbi` och `cbi`

Två instruktioner ändrar en enda bit i ett I/O-register och lämnar resten orört:

Instruktion	Exempel	Gör
`sbi`	`sbi PORTB, 3`	*set bit in I/O*: bit 3 i `PORTB` blir 1
`cbi`	`cbi PORTB, 0`	*clear bit in I/O*: bit 0 i `PORTB` blir 0

De tar en konstant bitposition, 0–7, och behöver inget register. Båda används i programmet `leds_on_portb.asm`:

```
    ser r16
    out DDRB, r16               ; Every pin of port B is an output.
    ldi r16, 0b00100001         ; LEDs on PB5 and PB0.
    out PORTB, r16              ; Write all eight bits at once.
    sbi PORTB, 3                ; Also light PB3, leaving the others as they were.
    cbi PORTB, 0                ; Put out PB0, leaving the others as they were.
```

Efter programmet är `PORTB` = `0b00101000`: bit 5 och bit 3.

`sbi` och `cbi` tar två klockcykler, mot en för `out`. De når bara de lägsta I/O-registren, men dit hör alla portregistren i boken.

8.3.5 Ändra några stift med en mask

När flera stift ska ändras, men inte alla, kombineras `in`, en maskoperation från avsnitt 8.2.2 och `out`:

```
    in r16, PORTB                   ; Read what port B drives now.
    ori r16, 0b00000110             ; Set PB2 and PB1, leave the rest.
    out PORTB, r16                  ; Write it back.
```

Mönstret kallas **läs, ändra, skriv**. Man kan läsa `PORTB` tillbaka med `in` och få det värde man senast skrev dit.

8.3.6 Namn på register med `.def`

I ett program som räknar på en port blir det snabbt svårt att komma ihåg vad `r16` och `r17` används till. Direktivet `.def` ger ett register ett namn:

```
.def counter = r16                  ; The value shown on port B.
.def temp = r17                     ; A scratch register.
```

Sedan kan man skriva `inc counter` och `out PORTB, counter`, och assemblern byter ut namnen mot `r16`. Maskinkoden blir densamma, men programmet går att läsa. Namn skrivs med små bokstäver, och `.def` placeras överst i programmet, efter `.include`.

8.3.7 I/O-simulering

I simulatorn kan du följa portarna i fönstret **I/O** (**Debug → Windows → I/O**). Välj port B i listan över enheter, där den heter ungefär *I/O Port* (*PORTB*), så visas `PINB`, `DDRB` och `PORTB` med en ruta per bit. En fylld ruta är en etta.

Stega ett program som räknar på port B, som i Labb 3 del 6:

```
.def counter = r16
.def temp = r17

main:
    ser temp
    out DDRB, temp                  ; All eight pins of port B are outputs.
    clr counter

loop:
    out PORTB, counter              ; Show the count on port B.
    inc counter                     ; Count up.
    rjmp loop                       ; And again, forever.
```

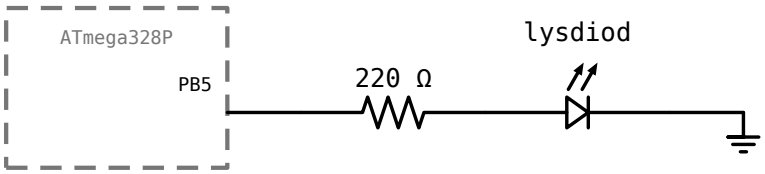
För varje varv i loopen visar rutorna i `PORTB` nästa binära tal: 0000 0001, 0000 0010, 0000 0011 och så vidare. Det är binär räkning, bit för bit, och varje ruta är en lysdiod på ett riktigt kort.

Lägg märke till att även **`PINB`** ändras, fast programmet aldrig skriver till den. `PINB` visar stiftens verkliga nivå, och ett utgångsstift har den nivå porten driver ut.

Om du i stället kör programmet fritt, med **Continue** (**F5**), och sedan stoppar med **Break All**, har räknaren hunnit många varv, och värdet i `PORTB` ser ut som ett slumpmässigt tal. Varje varv tar fyra klockcykler, alltså 0,25 mikrosekunder. På ett riktigt kort skulle lysdioderna byta så fort att alla åtta bara skulle se ut att lysa svagt, hela tiden. För att ett program ska gå att följa med ögat behövs tidsfördröjningar, och det är ämnet för kapitel 9.

8.3.8 En lysdiod på riktigt

En utgång på ATmega328P kan driva en lysdiod direkt, men alltid genom ett **förkopplingsmotstånd** som begränsar strömmen:




```
En etta i PORTB bit 5 tänder lysdioden, en nolla släcker den. Motståndet begränsar
          strömmen; utan det kan både lysdioden och stiftet förstöras.
```

Figur 8.9 En lysdiod med förkopplingsmotstånd på stiftet PB5.

Utan motståndet begränsas strömmen bara av vad stiftet orkar leverera, vilket är mer än både lysdioden och stiftet tål. Arduino Uno-kortet har en lysdiod med motstånd redan monterad på stift 13, alltså PB5. Att koppla egna lysdioder och räkna ut motståndet gör du i kapitel 11.

8.4 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Addera och subtrahera binärt för hand, och säga vad ett register och `C` innehåller efteråt.
- Förklara aritmetisk rundgång, och varför `inc` och `dec` inte ändrar `C`.
- Omvandla mellan ett negativt tal och dess tvåkomplement, och läsa samma byte med och utan tecken.
- Förutsäga `C`, `Z`, `N` och `V` efter en addition eller subtraktion, och förklara skillnaden mellan `C` och `V`.
- Välja mask och instruktion för att ettställa, nollställa, invertera eller testa enskilda bitar.
- Använda skift för att multiplicera och dividera med 2, och förklara skillnaden mellan `lsr` och `asr`.
- Göra en port till utport, och tända och släcka enskilda stift med `out`, `sbi` och `cbi`.

Frågor att testa dig själv med

1. `r16` är 255 och `C` är 0. Vad är `r16` och `C` efter `inc r16`? Och efter `add r16, r17` med `r17` = 1?
2. Varför finns det ingen `addi`, och hur adderar man 5 till `r16` ändå?
3. Byten `0xF6`: vilket tal är den utan tecken, och vilket med tecken?
4. `0x80 - 0x01` ger `C = 0` men `V = 1`. Vad säger det om svaret, utan tecken och med tecken?
5. Vilken instruktion och mask nollställer bit 7 i `r16` utan att röra de andra bitarna?
6. Varför är ett av nio steg mörkt i det rinnande ljuset med `rol`?
7. `out PORTB, r16` och `sbi PORTB, 3`: vad skiljer dem åt, när det gäller de andra sju stiften?

Referensbladet i bilaga E visar vilka flaggor varje instruktion ändrar.

Nästa kapitel

Kapitel 9 handlar om programflöde, loopar och tidsfördröjningar:

- Flödesplaner: att rita ett program innan det skrivs.
- Jämförelser och villkorliga hopp: det flaggorna är till för.
- Loopar som går ett bestämt antal varv.
- Tidsfördröjningar, beräknade exakt från klockcykler och kontrollerade med simulatorns stoppur.

8.5 Övningar

Så kontrollerar du ditt arbete. Övningarna märkta **Program** och **Kontroll** görs i Microchip Studio: skriv programmet, bygg och stega, och läs registren i Processor Status och portarna i I/O-fönstret. Skriv ned din förutsägelse innan du stegar.
Övriga övningar görs med papper och penna, utan miniräknare. Lösningsförslag finns i bilaga I. Varje övning är märkt med sin sort. Det finns en **Kontroll** i det här kapitlet, övning 8.5.

Övning 8.1 Binär addition för hand

RÄKNA FÖR HAND

Addera binärt, kolumn för kolumn med minnessiffror, och ange resultatet i en byte, hexadecimalt, och värdet på `C` efteråt.

a) `0b00101101 + 0b00010111`

b) `0x9C + 0x7A`

c) `0x0F + 0x01`. Vilken flagga, utöver `Z`, `N` och `C`, sätts här, och varför?

Övning 8.2 Mätaren slår om

FÖRSTÅELSE

`r16` innehåller 253 och `C` är 0. Programmet kör `inc r16` fem gånger i rad.

a) Vilket värde har `r16` efter varje `inc`?

b) Efter vilken `inc` är `Z = 1`? Vad är `Z` när alla fem är körda?

c) Vad är `C` efter den femte `inc`? Varför?

d) Hur skulle du ändra programmet så att `C` blir 1 när räknaren slår om från 255 till 0?

Övning 8.3 Negativa tal

RÄKNA FÖR HAND

a) Skriv −1, −10, −100 och −128 som byte i tvåkomplement, hexadecimalt. Visa metoden ”invertera och lägg till 1” för −10.

b) Vilka tal med tecken är `0xF0`, `0x81` och `0x7F`?

c) Vad blir `neg r16` om `r16` är `0x01`, `0x80` respektive `0x00`? Ett av svaren är överraskande. Vilket, och varför?

Övning 8.4 Addera en konstant

PROGRAM

a) Skriv ett program som laddar 40 i `r16` och sedan adderar 25 till det, utan att använda ett andra register.

b) Lägg till instruktioner som laddar 30 i `r17` och drar ifrån 100.

c) Förutsäg `r16`, `r17` och `C` när programmet är klart. Simulera och kontrollera.

Övning 8.5 Flaggorna

KONTROLL

Förutsäg för hand, kontrollera i simulatorn, förklara skillnaderna.

a) Fyll i tabellen **för hand**: resultatet hexadecimalt och flaggorna `C`, `Z`, `N`, `V` och `S` efter varje operation. Operationerna görs var för sig, inte efter varandra.

Operation	Resultat	C	Z	N	V	S
`0x50 + 0x50` (`add`)						
`0xF0 + 0x20` (`add`)						
`0x30 - 0x30` (`sub`)						
`0x00 - 0x01` (`sub`)						
`0x80 - 0x01` (`sub`)						
`0xFF + 1` med `inc`, när `C` är 1 före						

b) Skriv ett program som gör de sex operationerna i ordning, med en etikett efter varje, som i `flags_demo.asm` (avsnitt 8.1.7). Sätt `C` till 1 före den sista med instruktionen `sec` (*set carry*).

c) Kör fram till varje etikett och jämför SREG med tabellen. Stämde allt? Förklara varje ruta där du hade fel, och säg vilken regel i avsnitt 8.1 du hade missat.

Övning 8.6 Masker

RÄKNA FÖR HAND

`r16` innehåller `0b11010011`. Vad blir `r16` efter var och en av instruktionerna nedan? Varje instruktion utgår från `0b11010011`.

a) `andi r16, 0xF0`

b) `ori r16, 0x0C`

c) `eor r16, r24`, där `r24` innehåller `0xFF`

d) `com r16`. Jämför med **c**).

Övning 8.7 Välj rätt mask

KONSTRUKTION

Skriv en instruktion, eller två om det behövs, som gör följande med `r16` och lämnar alla andra bitar orörda:

a) ettställer bit 0 och bit 1;

b) nollställer bit 7;

c) inverterar bit 4–7;

d) nollställer alla bitar utom bit 3;

e) nollställer bit 0–3 och ettställer bit 7.

Övning 8.8 Skift och rotation

RÄKNA FÖR HAND

a) `r16` innehåller `0b00000110`. Vad blir den efter två `lsl`? Vilket tal har den multiplicerats med?

b) `r17` innehåller `0b10000001`. Vad blir `r17` och `C` efter en `lsr`?

c) `r18` innehåller `0xF8`, det vill säga −8. Vad blir den efter en `asr`, och efter en `lsr`? Vilket av svaren är −8/2?

d) `r19` innehåller `0b11000000` och `C` är 0. Vad blir `r19` och `C` efter var och en av tre `rol` i rad?

Övning 8.9 Lysdioder på port B

PROGRAM

a) Skriv ett program som gör port B till utport och tänder lysdioderna på PB0, PB2 och PB4 med en enda `out`.

b) Skriv om programmet så att samma tre lysdioder tänds med `sbi`, en i taget, i stället.

c) Lägg till instruktioner som släcker PB2 med `cbi`, och sedan tänder PB7 med mönstret läs, ändra, skriv (avsnitt 8.3.5).

d) Förutsäg `PORTB` efter programmet, binärt. Kontrollera i I/O-fönstret.

Övning 8.10 Läs programmet

FÖRSTÅELSE

Förutsäg `PORTB`, binärt, efter var och en av de markerade raderna.

```
.def leds = r16
.def temp = r17

main:
    ser temp
    out DDRB, temp
    ldi leds, 0b00001111
    out PORTB, leds                 ; (1)
    lsl leds
    lsl leds
    out PORTB, leds                 ; (2)
    sbi PORTB, 0                    ; (3)
    com leds
    out PORTB, leds                 ; (4)
    cbi PORTB, 7                    ; (5)
end:
    rjmp end
```

Övning 8.11 Multiplicera med tio

PROGRAM FÖRDJUPNING

Processorn har ingen instruktion som multiplicerar med 10, men $10x = 8x + 2x$, och multiplikation med 2 och 8 är skift.

a) Skriv ett program som laddar ett tal x i `r16` och räknar ut $10x$ i `r16`, med `mov`, `lsl` och `add`.

b) Prova med $x = 7$. Vad blir resultatet, decimalt och hexadecimalt?

c) Upp till vilket x ger programmet rätt svar? Vad händer för större x?

Övning 8.12 När är det overflow?

FÖRSTÅELSE FÖRDJUPNING

a) `0x40 + 0x40` ger `V = 1`, men `0xC0 + 0xC0` ger `V = 0`. Förklara båda genom att läsa talen med tecken.

b) Vad blir `C` i de två fallen? Förklara genom att läsa talen utan tecken.

c) Formulera med egna ord en regel för när en addition ger `V = 1`.

KAPITEL 9

Programflöde, loopar och tidsfördröjningar

Hur ett program väljer väg, går varv efter varv och väntar exakt så länge det ska.

I DET HÄR KAPITLET

- Flödesplaner: att rita ett program innan det skrivs, med fem symboler.
- Jämförelser: `cp`, `cpi` och `tst`, och varför en jämförelse är en subtraktion utan resultat.
- Villkorliga hopp: `breq`, `brne`, `brlo`, `brsh` och de andra, och om-annars i assembler.
- Tal med och utan tecken: när `brlo` och när `brlt`.
- Loopar med ett bestämt antal varv, med räknaren i ett register.
- Tidsfördröjningar, räknade exakt från klockcykler och kontrollerade med simulatorns stoppur.
- Flerval, och att arbeta om en flödesplan så att programmet får en väg in och en väg ut.
- Labb 3, del 11, 12 och 14–17 (bilaga C).

Programmen i kapitel 8 gick rakt uppifrån och ned. Det här kapitlet ger dem det som riktiga program behöver: beslut, loopar och väntan. Besluten bygger på flaggorna från kapitel 8, och väntan räknas ut exakt, klockcykel för klockcykel, och kontrolleras sedan med simulatorns stoppur.

Läs avsnitt 9.1 om flödesplaner och villkorliga hopp först, och repetera gärna flaggorna i avsnitt 8.1.6. Läs sedan avsnitt 9.2 om loopar och tidsfördröjningar. Gör två förutsägelser innan du kör något: hur många gånger `brne` körs, och hur många av dem den hoppar, i en loop som går 200 varv; och hur många klockcykler fördröjningen i `short_delay.asm` (avsnitt 9.2.4) tar, och hur lång tid det är vid 16 MHz.

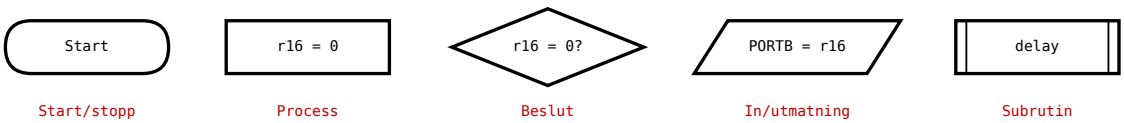
Övningarna finns i avsnitt 9.4, och lösningsförslagen till dem i bilaga I. Till kapitlet hör Labb 3, del 11, 12 och 14–17 (avsnitt C.9). Gör klart del 3–10 från kapitel 8 först, om de inte redan är gjorda.

9.1 Flödesplaner och villkorliga hopp

9.1.1 Flödesplaner

Programmen hittills har gått rakt uppifrån och ned, en instruktion efter en annan. Riktiga program måste **välja**: tänd värmen om det är kallt, annars inte. Innan ett sådant program skrivs är det värt att rita det, som en **flödesplan**.

En flödesplan beskriver vad programmet ska göra, i vilken ordning, och var det väljer väg, utan att bry sig om vilka instruktioner som behövs. Den ritas med fem symboler:



Figur 9.1 De fem symbolerna i en flödesplan.

Symbol	Betyder	Exempel
Start/stopp	Var programmet börjar och slutar	Start
Process	Något programmet gör	`summa = summa + 1`
Beslut	En fråga med två utgångar, Ja och Nej	`r16 = 0?`
In/utmatning	Läsa en ingång eller skriva en utgång	`PORTB = mönster`
Subrutin	Ett delprogram som anropas; tas upp i kapitel 10	`delay`

Fyra regler gör en flödesplan lätt att läsa, och lätt att översätta till ett program:

- **En start**, och flödet går uppifrån och ned. Pilar som går uppåt är loopar.
- **Varje beslut har exakt två utgångar**, märkta Ja och Nej.
- **Skriv vad, inte hur.** `Värme på` är bättre än `sbi PORTB, 0`; vilken instruktion det blir bestäms när programmet skrivs.
- **Grenar som delar sig ska mötas igen.** Mer om det i avsnitt 9.1.8.

9.1.2 Från flödesplan till program

När flödesplanen är klar översätts den ruta för ruta:

- en **process** eller en **in/utmatning** blir en eller några få instruktioner;
- ett **beslut** blir en jämförelse följd av ett villkorligt hopp (avsnitt 9.1.3 och 9.1.4);
- varje ställe dit en **pil pekar** från mer än ett håll får en **etikett**, så att ett hopp kan hamna där.

Det sista är nyckeln. Ett program kör uppifrån och ned tills en hoppinstruktion skickar det någon annanstans, och dit måste det finnas en etikett att hoppa till.

9.1.3 Jämförelse: `cp`, `cpi` och `tst`

Ett beslut i en flödesplan är en fråga om två tal: är de lika, är det ena mindre? På AVR ställs frågan med en **jämförelse**:

Instruktion	Exempel	Gör
`cp`	`cp r16, r17`	Beräknar `r16 - r17` och sparar bara flaggorna.
`cpi`	`cpi r16, 20`	Beräknar `r16 - 20` och sparar bara flaggorna. Bara `r16`–`r31`.
`tst`	`tst r16`	Sätter `Z` om `r16` är noll och `N` om bit 7 är satt.

En jämförelse är en subtraktion vars resultat kastas bort. `cp r16, r17` räknar ut `r16 - r17` precis som `sub` gör, men lägger inte svaret någonstans: `r16` är oförändrat efteråt. Det enda som blir kvar är flaggorna, och de säger allt man behöver veta om de två talen:

Om `r16` och `r17` är	blir `r16 - r17`	och flaggorna
lika	noll	`Z = 1, C = 0`
`r16` större, utan tecken	positivt, inget lån	`Z = 0, C = 0`
`r16` mindre, utan tecken	ett lån behövs	`Z = 0, C = 1`

Flaggorna och deras betydelse kommer från avsnitt 8.1.6. Det är det flaggorna är till för.

Är du osäker på vilken väg en jämförelse läses, skriv ut subtraktionen. `cp r16, r17` är `r16 - r17`, alltid i den ordningen.

9.1.4 Villkorliga hopp

Ett **villkorligt hopp** hoppar till en etikett om en flagga har ett visst värde, och fortsätter annars med nästa instruktion. Tillsammans med en jämförelse blir det ett beslut. De vanligaste:

Hopp	Efter `cp a, b`: hoppar om	Läser
`breq`	$a = b$ (*branch if equal*)	`Z = 1`
`brne`	$a \neq b$ (*branch if not equal*)	`Z = 0`
`brlo`	$a < b$, utan tecken (*lower*)	`C = 1`
`brsh`	$a \geq b$, utan tecken (*same or higher*)	`C = 0`
`brlt`	$a < b$, med tecken (*less than*)	`S = 1`
`brge`	$a \geq b$, med tecken (*greater or equal*)	`S = 0`
`brmi`	resultatet negativt (*minus*)	`N = 1`
`brpl`	resultatet positivt eller noll (*plus*)	`N = 0`

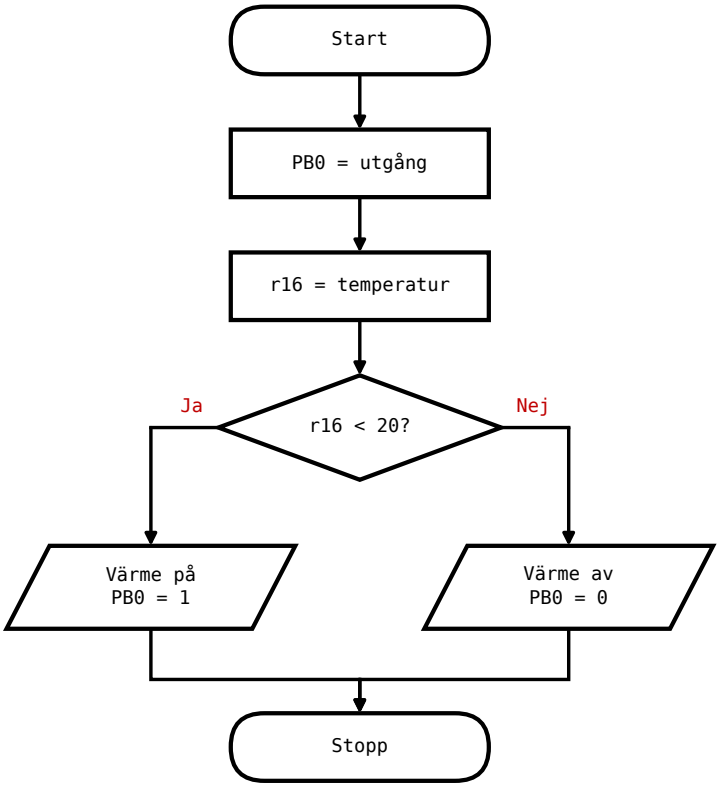
Hela listan finns på referensbladet i bilaga E. Två saker till är värda att veta:

- **`rjmp`** är det **ovillkorliga** hoppet: det hoppar alltid.
- **Ett villkorligt hopp tar 1 klockcykel om det inte hoppar och 2 om det hoppar**, eftersom processorn då måste hämta en annan instruktion än den som stod på tur. Det spelar roll för tidsfördröjningarna i avsnitt 9.2.

Det finns inga hopp för ”större än” och ”mindre än eller lika med”. De behövs inte: $a > b$ är detsamma som $b < a$, så man byter plats på operanderna i `cp`, eller jämför med ett tal ett steg högre: `r16 > 24` är `r16 ≥ 25`.

9.1.5 Om, annars

Det vanligaste beslutet är **om-annars** (*if-else*): om villkoret gäller, gör en sak, annars en annan. Här är värmaren i `compare_branch.asm`, först som flödesplan:




```
Ett beslut med två grenar som möts igen: if-else. I programmet blir beslutet
cpi och brlo, och grenen som inte hoppar slutar med ett rjmp förbi den andra.
```

Figur 9.2 Flödesplan för värmaren: ett om-annars.

och sedan som program:

```
    cpi r16, LIMIT              ; Compare: computes r16 - 20, keeps only the flags.
    brlo heater_on              ; Branch if lower (C = 1): temperature < 20.
    cbi PORTB, 0                ; Otherwise: heater off,
    rjmp done                   ; and jump past the other branch.
heater_on:
    sbi PORTB, 0                ; Heater on.
done:
    rjmp done
```

Följ de två vägarna:

- **Temperaturen är 18.** `cpi` räknar $18 - 20$, som kräver ett lån, så `C = 1`. `brlo` hoppar till `heater_on`, och värmen slås på.
- **Temperaturen är 22.** $22 - 20$ kräver inget lån, `C = 0`. `brlo` hoppar inte, och programmet fortsätter med `cbi`, som slår av värmen. Sedan hoppar `rjmp done` **förbi** `heater_on`.

Det sista `rjmp` är lätt att glömma, och då blir programmet fel. Utan det skulle programmet efter `cbi` fortsätta rakt ned in i `heater_on` och slå på värmen igen. I flödesplanen ser det ut som att grenarna är åtskilda; i programmet ligger de efter varandra, och det är hoppen som håller isär dem.

Bara om

Utan annars-gren blir det enklare: hoppa **förbi** det som bara ska göras ibland. Då hoppar man på det **motsatta** villkoret:

```
    cpi r16, 30
    brlo not_too_hot            ; Below 30: skip the alarm.
    sbi PORTB, 7                ; 30 or above: sound the alarm.
not_too_hot:
```

Flödesplanen säger ”om r16 ≥ 30, larma”, och programmet säger ”om r16 < 30, hoppa över larmet”. Det är samma sak, och det är det vanligaste mönstret i assembler.

9.1.6 Utan tecken eller med tecken

Samma jämförelse kan ge olika svar beroende på om talen läses med eller utan tecken (avsnitt 8.1.5). Jämför -5 med 3:

```
    ldi r16, -5                 ; 0xFB: 251 without sign, -5 with.
    ldi r17, 3
    cp r16, r17                 ; Unsigned 251 > 3: C = 0. Signed -5 < 3: S = 1.
```

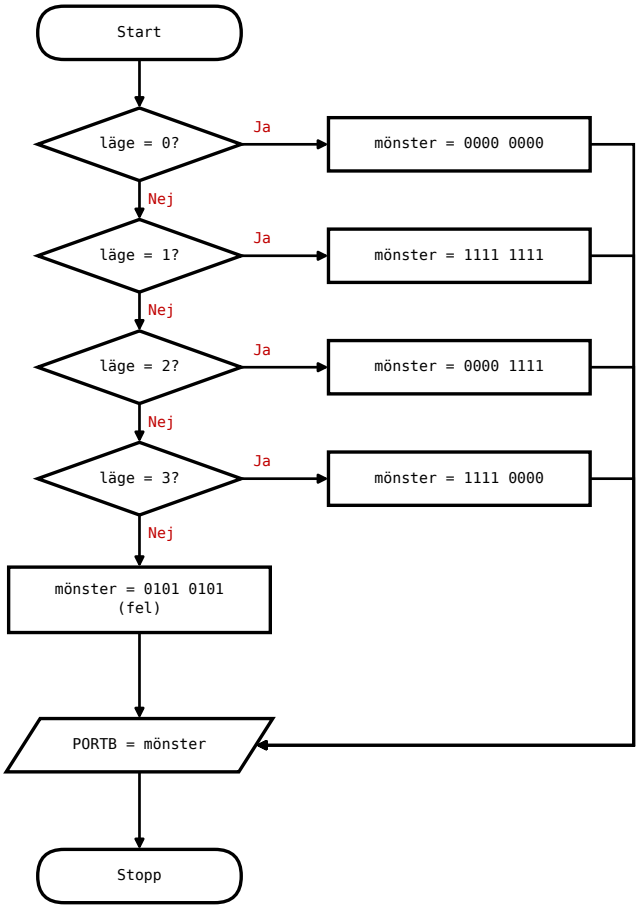
Efter `cp` är `C = 0`, så `brlo` hoppar **inte**: utan tecken är 251 inte mindre än 3. Men `S = 1`, så `brlt` hoppar: med tecken är −5 mindre än 3. Välj hopp efter vad talen betyder:

- **utan tecken**, räknare, antal, portvärden: `brlo` och `brsh`;
- **med tecken**, tal som kan bli negativa, som en utomhustemperatur: `brlt` och `brge`.

Värmaren i avsnitt 9.1.5 använder `brlo`. Det fungerar så länge temperaturen aldrig är negativ, vilket gäller inomhus. Vid −5 grader läser `brlo` talet som 251, bedömer att det är varmt nog, och slår av värmen. En termostat för minusgrader behöver `brlt`; se övning 9.12.

9.1.7 Flerval

Ibland finns fler än två alternativ: ett läge 0, 1, 2 eller 3, som var och ett ska ge ett eget mönster på lysdioderna. Det byggs som en **kedja av beslut**, där varje beslut testar ett alternativ:



Ett flerval: en kedja av beslut, där varje ja-gren gör sitt och alla vägar möts i samma ruta. Programmet får en väg in och en väg ut.

Figur 9.3 Flödesplan för ett flerval med en gemensam utgång.

Programmet `menu.asm` följer flödesplanen rad för rad:

```
    cpi r16, 0
    breq mode_0
    cpi r16, 1
    breq mode_1
    cpi r16, 2
    breq mode_2
    cpi r16, 3
```

```
    breq mode_3
    ldi r17, 0b01010101             ; None of them: the error pattern.
    rjmp show
mode_0:
    ldi r17, 0b00000000
    rjmp show
    ...
mode_3:
    ldi r17, 0b11110000
show:
    out PORTB, r17                  ; The one place every alternative ends up.
```

Varje alternativ gör bara en sak, lägger sitt mönster i `r17`, och hoppar sedan till samma ställe, `show`. Det sista alternativet behöver inget `rjmp show`, eftersom `show` kommer direkt efter.

Tänk på **alternativet som inte finns med**: vad ska hända om läget är 7? En flödesplan utan svar på den frågan ger ett program som gör något ingen har bestämt. Här visar det ett felmönster.

9.1.8 Att arbeta om en flödesplan

Den första flödesplanen man ritar blir sällan den bästa. Ett vanligt första försök för flervalet ser ut så här: varje ja-gren får en egen `PORTB = mönster` och en egen stopp-ruta. Det fungerar, men flödesplanen får fyra slut, och programmet får fyra kopior av `out` och fyra slutloopar. Den dag något ska ändras i slutet, en fördröjning, en loop tillbaka till början, måste det göras på fyra ställen, och ett av dem glöms.

Att **arbeta om** flödesplanen betyder att flytta det som är gemensamt till ett ställe:

1. **Leta efter rutor som gör samma sak** i flera grenar. Här är det `PORTB = mönster` och stopp.
2. **Låt grenarna bara göra det som skiljer dem åt**, att välja mönster, och låt dem mötas.
3. **Rita den gemensamma delen en gång**, efter mötespunkten.

Resultatet är flödesplanen i avsnitt 9.1.7: en väg in, en väg ut, och varje alternativ en enda ruta. Det är den form som blir ett kort och läsbart program, och den form som går att bygga vidare på. I Labb 3 del 17 gör du samma omarbetning själv.

9.1.9 Vanliga fel

- **Flaggorna skrivs över mellan jämförelsen och hoppet.** Nästan alla räkneinstruktioner ändrar flaggorna. Lägg hoppet direkt efter `cp` eller `cpi`. `ldi`, `mov` och `out` ändrar inga flaggor och är ofarliga emellan; `inc`, `dec` och `add` är det inte.
- **Fel sorts hopp.** `brlo` för tal med tecken, eller `brlt` för tal utan, fungerar för de flesta värden och blir fel för några. Se avsnitt 9.1.6.

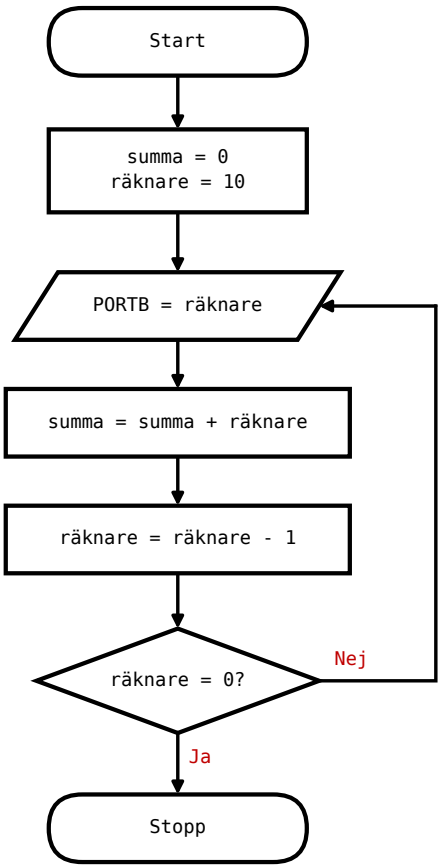
- **Det saknade `rjmp`** efter den första grenen i ett om-annars, så att programmet ramlar in i den andra grenen. Se avsnitt 9.1.5.
- **Omvänd jämförelse.** `cp r16, r17` följt av `brlo` hoppar om `r16 < r17`, inte tvärtom. Skriv ut subtraktionen om du är osäker.
- **Hoppet når inte fram.** Ett villkorligt hopp når bara ungefär 64 instruktioner bort. Längre bort ger ett felmeddelande när programmet byggs, och löses genom att hoppa på det motsatta villkoret förbi ett `rjmp`, som når ungefär 2 000 instruktioner åt vardera hållet.

9.2 Loopar och tidsfördröjningar

9.2.1 En loop med ett bestämt antal varv

En **loop** är en del av programmet som körs om och om igen. Den byggs med ett hopp bakåt, till en etikett före det som ska upprepas. Oftast ska loopen gå ett bestämt antal varv, och då behövs en **räknare**: ett register som räknas ned för varje varv, tills det blir noll.

Programmet `count_loop.asm` räknar ut summan $1 + 2 + \cdots + 10$:



En loop med ett bestämt antal varv. Räknaren räknas ned, och beslutet sist i loopen avgör om den ska gå ett varv till: i programmet dec och brne.

Figur 9.4 Flödesplan för en räknarloop.

```
    clr r17                         ; r17 = the sum so far = 0
    ldi r16, 10                     ; r16 = the counter: laps left to run

loop:
    out PORTB, r16                  ; Show the counter.
    add r17, r16                    ; sum = sum + counter
    dec r16                         ; One lap fewer left. Sets Z when it reaches 0.
    brne loop                       ; Not zero yet: go round again.
```

Nyckeln är de två sista raderna. `dec r16` minskar räknaren och sätter `Z = 1` när den når noll. `brne loop` hoppar tillbaka så länge `Z = 0`, alltså så länge räknaren inte är noll. Ingen `cpi` behövs: `dec` sätter själv den flagga hoppet läser.

Följ räknaren: den börjar på 10, och `brne` hoppar tillbaka när den är 9, 8, ..., 1. När `dec` gör den till 0 hoppar `brne` inte, och programmet fortsätter efter loopen. Loopen går alltså **10 varv**, lika många som räknarens startvärde, och summan blir $10 + 9 + \cdots + 1 = 55$.

En fälla: startvärdet 0. Om räknaren börjar på 0 gör den första `dec` den till 255, och loopen går **256 varv**, inte noll. En loop med testet sist körs alltid minst en gång.

9.2.2 Testet först eller sist

Loopen i avsnitt 9.2.1 har testet **sist**: först görs arbetet, sedan frågar `brne` om det ska göras igen. Det är den kortaste formen, och den som används mest i assembler.

Om loopen ska kunna gå **noll** varv måste testet stå **först**:

```
    tst r16                         ; Is the counter already zero?
loop:
    breq done                       ; Yes: leave before doing any work.
    add r17, r16
    dec r16                         ; Sets Z for the breq at the top.
    rjmp loop
done:
```

Nu hoppar `breq` ut direkt om räknaren är noll från början. Priset är en instruktion till per varv, `rjmp`, och en klockcykel till: `breq` som inte hoppar och `rjmp` tar 1 + 2 cykler, mot 2 för `brne` som hoppar. Välj formen efter om noll varv kan förekomma.

9.2.3 Vad en instruktion kostar i tid

Ett program som tänder en lysdiod och sedan släcker den gör det på en bråkdel av en mikrosekund. För att ögat ska hinna se något, eller för att ett relä ska hinna dra, eller för att ett trafikljus ska lysa grönt i några sekunder, måste programmet **vänta**. En **tidsfördröjning** är en loop som inte gör något annat än att ta tid.

Hur mycket tid den tar går att räkna ut exakt, eftersom varje instruktion tar ett fast antal klockcykler (avsnitt 7.1.7):

Klockcykler	Instruktioner
1	`ldi mov add sub subi inc dec and or eor com cp cpi lsl lsr rol ror in out nop`
2	`rjmp sbi cbi`
1 eller 2	`breq brne brlo brsh` och de andra villkorliga hoppen: 1 om de inte hoppar, 2 om de hoppar

Vid 16 MHz tar en klockcykel 62,5 ns, och 16 cykler tar en mikrosekund. Hela tabellen, med fler instruktioner, finns på referensbladet i bilaga E.

9.2.4 En kort tidsfördröjning

Den enklaste fördröjningen är en loop som räknar ned ett register, som i `short_delay.asm`:

```
delay_start:
    ldi r18, 200                    ; 1 cycle
delay_loop:
    dec r18                         ; 1 cycle per lap
    brne delay_loop                 ; 2 cycles per lap, 1 on the last
delay_end:
```

Räkna ut kostnaden med en rad per instruktion: vad den kostar, hur många gånger den körs, och produkten. `brne` får två rader, eftersom den kostar olika mycket när den hoppar och när den inte gör det.

Instruktion	Cykler	Gånger	Summa
`ldi r18, 200`	1	1	1
`dec r18`	1	200	200
`brne`, hoppar	2	199	398
`brne`, hoppar inte	1	1	1
Totalt			**600**

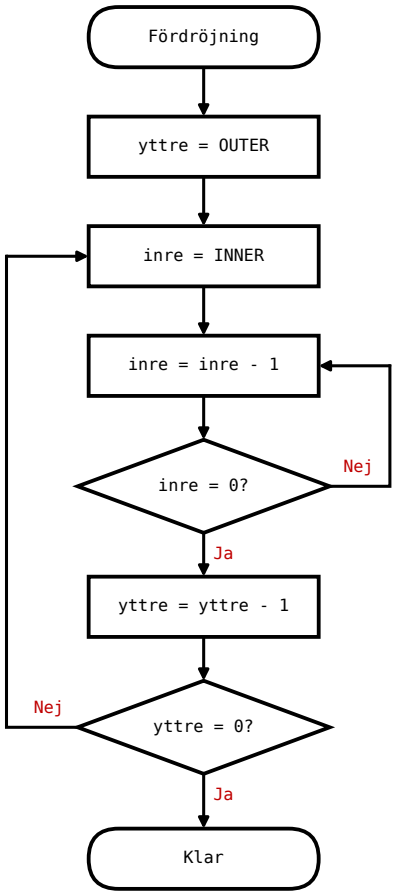
600 cykler är $600 \cdot 62{,}5$ ns = **37,5 µs**. Det svåra i tabellen är kolumnen *Gånger*: `brne` körs 200 gånger, men hoppar bara 199 av dem, eftersom den sista gången, när räknaren har blivit noll, går programmet vidare.

Med ett annat antal varv, n, blir summan $1 + n + 2(n-1) + 1 = \mathbf{3n}$ klockcykler. Det är en formel värd att komma ihåg:

- **Önskad tid → antal varv.** 30 µs är $30 \cdot 16 = 480$ cykler, och $n = 480/3 = 160$.
- **Längsta möjliga.** Ett register rymmer högst 255, men startvärdet 0 ger 256 varv (avsnitt 9.2.1): 768 cykler, 48 µs. Längre än så räcker inte en loop.

9.2.5 En längre tidsfördröjning

För att vänta längre läggs en loop **inuti** en annan. Den inre loopen är fördröjningen från avsnitt 9.2.4. Den yttre upprepar den ett antal gånger:



Två loopar i varandra. Den inre loopen körs från början, INNER varv, för varje varv i den yttre. Tiden blir OUTER · (3 · INNER + 3) klockcykler.

Figur 9.5 Flödesplan för två loopar i varandra.

Programmet `long_delay.asm`:

```
delay_start:
    ldi r19, OUTER                  ; 1 cycle
delay_outer:
    ldi r18, INNER                  ; 1 cycle per outer lap
delay_inner:
    dec r18                         ; 1 cycle per inner lap
    brne delay_inner                ; 2 per inner lap, 1 on the last
```

```
    dec r19                         ; 1 cycle per outer lap
    brne delay_outer                ; 2 per outer lap, 1 on the last
delay_end:
```

Räkna ut det i två steg:

1. **Ett varv i den yttre loopen.** `ldi r18` (1), den inre loopen utan sin `ldi` ($3 \cdot \text{INNER} - 1$), `dec r19` (1) och `brne` som hoppar (2): $3 \cdot \text{INNER} + 3$ cykler. Det sista yttre varvet är en cykel kortare, eftersom `brne` då inte hoppar.
2. **Hela fördröjningen.** `ldi r19` (1), plus OUTER varv, minus den cykel det sista varvet sparar:

$$\text{cykler} = \text{OUTER} \cdot (3 \cdot \text{INNER} + 3)$$

Med OUTER = 208 och INNER = 255 blir det $208 \cdot 768 =$ **159 744 cykler**, vilket är 159 744 · 62,5 ns = **9,984 ms**. Nästan 10 ms.

Att träffa exakt

Formeln ger bara vissa tider. 10 ms är 160 000 cykler, och 160 000 är inte delbart med 3, så ingen kombination av OUTER och INNER ger exakt 10 ms. Lösningen är att lägga till en `nop`, som inte gör något annat än att ta en cykel, i den yttre loopen. Då kostar varje yttre varv en cykel mer:

$$\text{cykler} = \text{OUTER} \cdot (3 \cdot \text{INNER} + 4)$$

och OUTER = 250, INNER = 212 ger $250 \cdot 640 =$ **160 000 cykler, exakt 10 ms**. Det gör du i Labb 3 del 16.

Så väljer du konstanterna för en önskad tid:

1. Räkna om tiden till cykler: tid i µs · 16.
2. Välj INNER så stort som möjligt, 255, och räkna ut OUTER = cykler / 768. Avrunda.
3. Räkna ut vad det faktiskt blir. Behövs mer precision, prova andra kombinationer, eller lägg till `nop`.

Med två loopar räcker det till ungefär $256 \cdot 771$ cykler, knappt 12,4 ms. För längre tider, en halv sekund för en blinkande lysdiod, behövs en loop till eller en räknare med 16 bitar, som i kapitel 10.

9.2.6 Mät med stoppuret

Ett räknat värde är en förutsägelse. Kontrollera det med simulatorns stoppur, enligt avsnitt D.4.4:

1. Kontrollera att **Frequency** i Processor Status står på **16 MHz**.
2. Sätt en brytpunkt på raden `delay_start` och en på raden `delay_end`.
3. Kör till den första med **F5**, nollställ stoppuret, och kör till den andra.
4. Läs av **Cycle Counter** eller **Stop Watch**.

För `short_delay.asm` ska stoppuret visa exakt 600 cykler, 37,5 µs, och för `long_delay.asm` 159 744 cykler. Instruktionen där en brytpunkt sitter har ännu inte körts när programmet stannar, så mätningen från `delay_start` till `delay_end` omfattar precis fördröjningen: `ldi` men inte instruktionen efter.

Om mätningen inte stämmer med räkningen är det nästan alltid räkningen som är fel. Kontrollera först kolumnen *Gånger* för hoppen, och sedan var brytpunkterna sitter: en brytpunkt en rad för tidigt eller för sent tar med eller utelämnar en instruktion. Att stämma av en räkning mot en mätning och förklara skillnaden är det kontrolluppgiften, övning 9.9, handlar om.

9.3 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Rita en flödesplan för ett program med beslut och loopar, och översätta den till assembler.
- Förklara vad `cp` och `cpi` gör med flaggorna, och välja rätt villkorligt hopp för ett villkor.
- Skriva om-annars, och förklara varför den första grenen slutar med ett `rjmp`.
- Välja mellan hopp för tal utan tecken och tal med tecken.
- Skriva en loop som går ett bestämt antal varv, och säga hur många varv den går för startvärdet 0.
- Räkna ut hur många klockcykler en fördröjning tar, och välja konstanter för en önskad tid.
- Mäta en tid i simulatorn med brytpunkter och stoppuret, och förklara en skillnad mot räkningen.
- Skriva ett flerval med en gemensam utgång.

Frågor att testa dig själv med

1. `cp r16, r17` följt av `brlo`: hoppar programmet om `r16` är mindre än `r17`, eller tvärtom?
2. Varför behövs ingen `cpi` före `brne` i en loop som räknar ned med `dec`?
3. Vad händer om `rjmp` efter den första grenen i ett om-annars saknas?
4. En loop räknar ned från startvärdet 0. Hur många varv går den?
5. Fördröjningen `ldi r18, 200` / `dec r18` / `brne` tar 600 cykler, inte 599 och inte 800. Varför?
6. Varför går det inte att göra en fördröjning på en sekund med två loopar i varandra?
7. Hur mäter du i simulatorn hur lång tid en del av programmet tar?

Referensbladet i bilaga E har alla hopp och varje instruktions antal klockcykler, och avsnitt D.4.4 visar hur brytpunkterna och stoppuret används i Microchip Studio.

Nästa kapitel

Kapitel 10 handlar om inporten, subrutiner och stacken:

- ASCII-koder: att göra om ett tal till tecken.
- Tal med 16 bitar, och en fördröjning på en hel sekund.
- Inporten: att läsa knappar med `PIND`, och varför en nedtryckt knapp läses som noll.
- Subrutiner: att skriva fördröjningen en gång och anropa den många gånger.
- Stacken: var returadressen hamnar, och hur register sparas med `push` och `pop`.

9.4 Övningar

Så kontrollerar du ditt arbete. Rita flödesplanen innan du skriver programmet, och förutsäg resultatet innan du simulerar. Övningarna märkta **Program** och **Kontroll** görs i Microchip Studio; tidsfördröjningar mäts med stoppuret enligt avsnitt 9.2.6.
Övriga övningar görs med papper och penna. Lösningsförslag finns i bilaga I.
Varje övning är märkt med sin sort. Det finns en **Kontroll** i det här kapitlet, övning 9.9.

Övning 9.1 Rita en flödesplan

KONSTRUKTION

Ett program ska läsa ett tal i `r16` och tända PB0 om talet är jämnt, annars PB1. Ett tal är jämnt om bit 0 är 0.

a) Rita flödesplanen.

b) Vilken instruktion och mask använder du för att ta reda på bit 0? Tips: avsnitt 8.2.2.

c) Vilket villkorligt hopp passar efter den instruktionen?

Övning 9.2 Vilket hopp?

FÖRSTÅELSE

Skriv en jämförelse och ett villkorligt hopp till etiketten `yes` för varje villkor. Talen är utan tecken om inget annat sägs.

a) `r16` är lika med `r17`.

b) `r16` är inte noll.

c) `r16` är mindre än 100.

d) `r16` är 100 eller större.

e) `r16` är större än 100.

f) `r16` är negativt, när det läses med tecken.

g) `r16` är mindre än `r17`, när båda läses med tecken.

Övning 9.3 Följ programmet

RÄKNA FÖR HAND

Vilket värde får `PORTB` när `r16` börjar med 5, 10, 99, 100 respektive 200?

```
    ldi r17, 0
    cpi r16, 10
    brlo small
    cpi r16, 100
    brsh big
    ldi r17, 2
    rjmp done
small:
    ldi r17, 1
    rjmp done
big:
    ldi r17, 3
done:
    out PORTB, r17
```

Rita också programmets flödesplan. Vad är programmets uppgift, med en mening?

Övning 9.4 Om, annars

PROGRAM

a) Skriv ett program som tänder alla lysdioder på port B om `r16` är 42, och släcker alla annars.

b) Testa programmet med `r16` = 42 och med `r16` = 41. Byt värde antingen i `ldi`-raden eller i Processor Status.

c) Ta bort `rjmp`-instruktionen efter den första grenen. Vad händer nu för `r16` = 41, och för `r16` = 42? Förklara.

Övning 9.5 Hur många varv?

RÄKNA FÖR HAND

```
    ldi r16, N
loop:
    inc r17
    dec r16
    brne loop
```

`r17` är 0 från början. Vad är `r17` efter loopen när

a) N = 5?

b) N = 1?

c) N = 0? Förklara svaret.

d) Hur många gånger körs `brne` när N = 5, och hur många av dem hoppar den?

Övning 9.6 Summera

PROGRAM

a) Skriv ett program som räknar ut $1 + 2 + \cdots + n$, med n i `r16` och summan i `r17`.

b) Prova med $n = 15$. Vilket svar ska du få?

c) Vilket är det största n som ger rätt svar i en byte? Vad får du för $n = 23$, och varför?

Övning 9.7 Räkna cykler

RÄKNA FÖR HAND

a) Hur många klockcykler tar fördröjningen i avsnitt 9.2.4 om startvärdet är 100 i stället för 200? Hur lång tid är det vid 16 MHz?

b) Samma fråga för startvärdet 1.

c) Samma fråga för startvärdet 0.

Övning 9.8 Välj konstanter

KONSTRUKTION

a) En fördröjning ska vara 40 µs. Hur många varv ska loopen i avsnitt 9.2.4 gå? Hur nära 40 µs kommer du, och hur kan du komma exakt?

b) En fördröjning ska vara 5 ms. Välj OUTER och INNER för loopen i avsnitt 9.2.5, först utan och sedan med en `nop` i den yttre loopen. Hur nära 5 ms kommer du i de två fallen?

c) Varför går det inte att göra en fördröjning på en sekund med två loopar?

Övning 9.9 Räkna, mät och förklara

KONTROLL

Räkna ut för hand, mät med stoppuret, förklara skillnaden.

Använd `long_delay.asm`, med OUTER = 208 och INNER = 255:

lectures/L09/examples/long_delay.asm

```
;*****************************************************************************
; L09: A longer delay: a loop inside a loop.
;
; The inner loop, its ldi included, takes 3 * INNER cycles, as in short_delay.asm.
; Around it, every outer lap adds dec (1) and brne (2, or 1 on the last lap), and
; ldi r19 adds back the one cycle the last lap saves, so the whole delay takes
; OUTER * (3 * INNER + 3) cycles. With 208 and 255: 208 * 768 = 159 744 cycles,
; which is 9.984 ms at 16 MHz.
;*****************************************************************************
.include "m328Pdef.inc"

.equ OUTER = 208                    ; Outer laps, 1-255.
.equ INNER = 255                    ; Inner laps per outer lap, 1-255.
```

```
.org 0x0000
    rjmp main

main:
    sbi DDRB, 0
    sbi PORTB, 0                    ; A pulse starts.

delay_start:
    ldi r19, OUTER                  ; 1 cycle
delay_outer:
    ldi r18, INNER                  ; 1 cycle per outer lap
delay_inner:
    dec r18                         ; 1 cycle per inner lap
    brne delay_inner                ; 2 per inner lap, 1 on the last
    dec r19                         ; 1 cycle per outer lap
    brne delay_outer                ; 2 per outer lap, 1 on the last
delay_end:
    cbi PORTB, 0                    ; The pulse ends, about 10 ms later.

end:
    rjmp end
```

a) Räkna ut antalet klockcykler från raden `delay_start` till raden `delay_end`, med en tabell som den i avsnitt 9.2.4: en rad per instruktion, cykler, antal gånger och summa. Räkna om till millisekunder.

b) Mät samma sak i simulatorn med brytpunkter och stoppuret. Stämmer det?

c) Flytta den första brytpunkten till raden `sbi PORTB, 0` och den andra till raden `cbi PORTB, 0`. Förutsäg först hur mycket mätningen ändras, mät sedan. Förklara skillnaden.

d) Ändra OUTER till 100 och förutsäg den nya tiden med formeln. Mät och jämför.

e) En kamrat har räknat ut att fördröjningen tar OUTER · INNER · 3 = 159 120 cykler. Vad har kamraten glömt, och hur stort blir felet i procent?

Övning 9.10 Flerval

PROGRAM

Ett program ska visa ett mönster på port B beroende på läget i `r16`:

- läge 0: alla släckta;
- läge 1: bara PB0;
- läge 2: PB0 och PB1;
- alla andra lägen: alla tända, som en felindikering.

a) Rita flödesplanen, med en gemensam ruta för `PORTB = mönster`.

b) Skriv programmet och testa alla fyra fallen.

Övning 9.11 Arbeta om

KONSTRUKTION

Programmet nedan fungerar, men det har tre slut och tre kopior av `out`. Rita dess flödesplan, arbeta om den enligt avsnitt 9.1.8, och skriv det omarbetade programmet.

```
    cpi r16, 1
    breq one
    cpi r16, 2
    breq two
    ldi r17, 0
    out PORTB, r17
end_1:
    rjmp end_1
one:
    ldi r17, 1
    out PORTB, r17
end_2:
    rjmp end_2
two:
    ldi r17, 3
    out PORTB, r17
end_3:
    rjmp end_3
```

Övning 9.12 Minusgrader

FÖRSTÅELSE FÖRDJUPNING

Värmaren i avsnitt 9.1.5 ska användas utomhus, där temperaturen kan bli negativ. Den lagras som ett tal med tecken.

a) Vad gör programmet när temperaturen är −5 grader? Följ `cpi` och `brlo` steg för steg.

b) Vilket hopp ska användas i stället, och vilken flagga läser det?

c) Fungerar det rättade programmet även för temperaturen 25 grader? För 100 grader?

KAPITEL 10

Inporten, subrutiner och stacken

Hur ett program läser en knapp, räknar med tal större än 255 och gör samma sak från flera ställen.

I DET HÄR KAPITLET

- Tecken är tal: ASCII-koder, och hur en byte blir två tecken med `swap` och `andi`.
- Tal större än 255: registerpar, `add`/`adc`, `sub`/`sbc`, `adiw`, `sbiw` och `movw`.
- En lång tidsfördröjning, exakt en sekund, med en 16-bitars loop.
- Inporten: `DDRD`, pull-up-motståndet och `PIND`, och varför en nedtryckt knapp läses som noll.
- Att testa en enda bit med `sbic` och `sbis`, och att vänta på en knapp.
- Subrutiner: `rcall`, `ret` och returadressen på stacken.
- Stacken som lagringsplats: `push`, `pop`, och kursens regel om att spara det man ändrar.
- **Labb 3**, del 18–23 (bilaga C).

Hittills har programmen bara skrivit till en port, räknat med tal som ryms i en byte och gått rakt igenom sin kod, med loopar och villkorliga hopp. Det här kapitlet ger dem tre saker till: de kan göra om tal till text, räkna med 16 bitar och vänta i en hel sekund, de kan läsa en knapp på inporten, och de kan dela upp sig i subrutiner som anropas från flera ställen, med stacken som håller reda på vart de ska återvända.

Kapitlet förutsätter att du har gjort Labb 3 till och med del 17, och det använder looparna och de villkorliga hoppen från kapitel 9 hela tiden. Repetera ASCII i avsnitt 1.2.4 och utporten i avsnitt 8.3 om det behövs. Läs sedan avsnitt 10.1 och avsnitt 10.2 innan du börjar på del 18–23 i Labb 3, och avsnitt 10.3, om subrutiner och stacken, innan del 22. Övningarna i avsnitt 10.5 kontrollerar du mot bilaga I.

Gör två förutsägelser innan du kör exempelprogrammen i simulatorn: vilken text byten `0x3C` blir, och vilken byte varje tecken är; och vad `SP` är mitt i `delay_ms`, när returadressen och tre register ligger på stacken. Stega sedan och jämför.

10.1 ASCII-koder och 16-bitarstal

10.1.1 Tecken är också tal

En mikrodator känner bara till tal. Ett tecken, en bokstav eller en siffra på en display, är ett tal som alla har kommit överens om att tolka som ett visst tecken. Den överenskommelse som nästan all elektronik använder heter **ASCII**, och den gicks igenom i avsnitt 1.2. Här behövs bara en liten del av tabellen:

Tecken	ASCII-kod	Tecken	ASCII-kod
`'0'`	`0x30`	`'A'`	`0x41`
`'1'`	`0x31`	`'B'`	`0x42`
`'2'`	`0x32`	`'C'`	`0x43`
...	...	`'D'`	`0x44`
`'8'`	`0x38`	`'E'`	`0x45`
`'9'`	`0x39`	`'F'`	`0x46`

Två saker i tabellen gör resten av det här avsnittet möjligt:

- **Siffrorna ligger i ordning.** Tecknet för siffran `n` har koden `0x30 + n`. Tecknet `'7'` är alltså `0x37`, och siffran 7 blir ett tecken genom att man adderar `0x30`.
- **Bokstäverna ligger också i ordning, men inte direkt efter siffrorna.** Efter `'9'` (`0x39`) kommer sju andra tecken (`:`, `;`, `<` och så vidare) innan `'A'` (`0x41`). Den luckan på sju kommer att dyka upp i koden.

Assemblern känner till tecken. `ldi r16, 'A'` betyder precis samma sak som `ldi r16, 0x41`, och det är ofta det tydligaste sättet att skriva en konstant som är tänkt som ett tecken.

Varför behövs det här? Så fort ett tal ska visas för en människa, på en display, i ett terminalfönster eller i en logg, måste det göras om till tecken. Talet `0x3C` i ett register är en byte; texten ”3C” på en skärm är två byte, `0x33` och `0x43`.

10.1.2 En siffra blir ett tecken

Ett värde 0–15 ska bli tecknet för sin hexadecimala siffra, `'0'`–`'9'` eller `'A'`–`'F'`.

Värdet 0–9: addera `0x30`. Värdet 7 blir `0x37`, som är `'7'`.

Värdet 10–15: addera `0x30` och sju till. Värdet 10 blir `0x0A + 0x07 + 0x30 = 0x41`, som är `'A'`. De sju extra är luckan mellan `'9'` och `'A'` från avsnitt 10.1.1.

AVR saknar en instruktion som adderar en konstant till ett enskilt register. Men att subtrahera ett negativt tal är samma sak som att addera, så `subi r24, -0x30` adderar `0x30` (se avsnitt 8.1). Hela omvandlingen blir fyra instruktioner och en etikett:

```
    cpi r24, 10                 ; A digit 0-9, or a letter A-F?
    brlo digit                  ; 0-9: skip the extra step.
    subi r24, -7                ; A-F: add 7 more, so that 10 lands on 'A'.
digit:
    subi r24, -0x30             ; Add 0x30, the ASCII code for '0'.
```

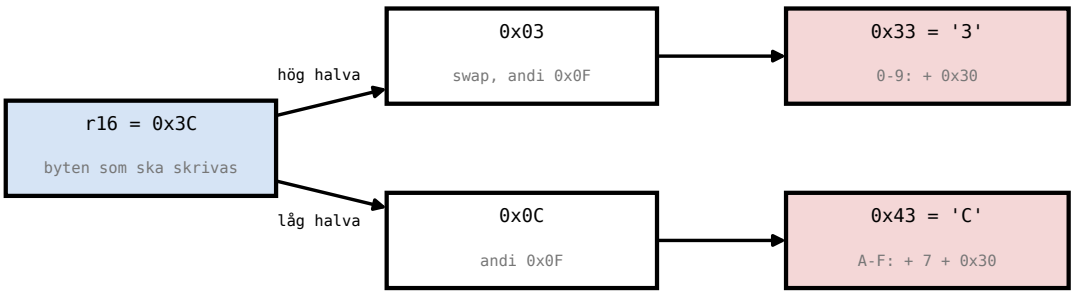
Följ två värden genom koden:

`r24` före	`cpi r24, 10`	Hopp?	Efter `subi r24, -7`	Efter `subi r24, -0x30`
`0x03`	$3 < 10$, C = 1	ja, `brlo` hoppar	(körs inte)	`0x33` = `'3'`
`0x0C`	$12 \geq 10$, C = 0	nej	`0x13`	`0x43` = `'C'`

Flaggorna efter `subi` med ett negativt tal beter sig som vid en subtraktion, inte som vid en addition: `C` blir 1 när det *inte* blev någon minnessiffra i additionen. Här spelar det ingen roll, eftersom ingenting läser flaggorna efteråt. Men bygg aldrig ett villkorligt hopp på flaggorna efter ett sådant `subi`.

10.1.3 En byte blir två tecken

En byte är två hexadecimala siffror, och varje siffra är fyra bitar, en **nibble** (avsnitt 1.1). För att skriva ut byten `0x3C` ska den höga nibblen `3` och den låga nibblen `C` göras om var för sig.




```
Varje hexadecimal siffra är fyra bitar. En byte blir två tecken: först delas
        den i två halvor, sedan blir varje halva sitt ASCII-tecken.
```

Figur 10.1 Byten `0x3C` delas i sin höga och sin låga halva, som blir var sitt tecken.

Den låga nibblen får man genom att nollställa de fyra höga bitarna med en mask: `andi r24, 0x0F` behåller bit 0–3 och nollställer bit 4–7. `0x3C` blir `0x0C`.

Den höga nibblen får man med en ny instruktion, **`swap`**, som byter plats på ett registers två halvor: `0x3C` blir `0xC3`. Därefter gör samma mask som nyss att `0x03` blir kvar. `swap` tar en cykel och påverkar inga flaggor.

Hela programmet, `hex_to_ascii.asm`, gör om `0x3C` i `r16` till tecknen `'3'` i `r20` och `'C'` i `r21`:

```
main:
    ldi r16, 0x3C                   ; The byte to convert.

    ; The high nibble: 0x3C -> 0x03 -> '3'.
    mov r24, r16                    ; Work on a copy, so r16 keeps its value.
    swap r24                        ; Swap the two nibbles: 0x3C -> 0xC3.
    andi r24, 0x0F                  ; Keep only the low nibble: 0xC3 -> 0x03.
    cpi r24, 10                     ; A digit 0-9, or a letter A-F?
    brlo high_digit                 ; 0-9: skip the extra step.
    subi r24, -7                    ; A-F: add 7 more, so that 10 lands on 'A'.
high_digit:
    subi r24, -0x30                 ; Add 0x30, the ASCII code for '0'.
    mov r20, r24                    ; r20 = '3' = 0x33.

    ; The low nibble: 0x3C -> 0x0C -> 'C'.
    mov r24, r16
    andi r24, 0x0F                  ; Keep only the low nibble: 0x3C -> 0x0C.
    cpi r24, 10
    brlo low_digit
    subi r24, -7
low_digit:
    subi r24, -0x30
    mov r21, r24                    ; r21 = 'C' = 0x43.
```

Lägg märke till att samma fem rader står två gånger, bara med olika etiketter. Det är ett tecken på att de borde vara en **subrutin**, något man skriver en gång och anropar två gånger. Det är ämnet för avsnitt 10.3, och i avsnitt 10.3.9 skrivs det här programmet om så.

10.1.4 Tillbaka från tecken till värde

Omvänt: ett tecken `'0'`–`'9'` eller `'A'`–`'F'` ska bli sitt värde 0–15. Det är samma steg i omvänd ordning:

1. Subtrahera `0x30`. En siffra är nu klar: `'7'` (`0x37`) blir 7.
2. En bokstav blev 17 eller mer: `'A'` (`0x41`) blir `0x11` = 17. Subtrahera sju till.

```
    subi r22, 0x30                  ; '0'-'9' -> 0-9, 'A'-'F' -> 17-22.
    cpi r22, 10
    brlo done                       ; A digit: done.
    subi r22, 7                     ; A letter: 17-22 -> 10-15.
done:
```

Det är så en mikrodator tolkar ett tal som någon har skrivit in på ett tangentbord: varje tangent skickar ett tecken, och programmet gör om tecknen till tal.

10.1.5 Att lägga text i minnet

Hittills har varje värde legat i ett register. Text brukar i stället ligga i **dataminnet**, SRAM, en byte per tecken efter varandra (avsnitt 7.1). En ny instruktion skriver ett register till en adress i dataminnet:

```
    sts 0x0100, r20                 ; Store r20 at data address 0x0100.
    sts 0x0101, r21                 ; ... and r21 right after it.
```

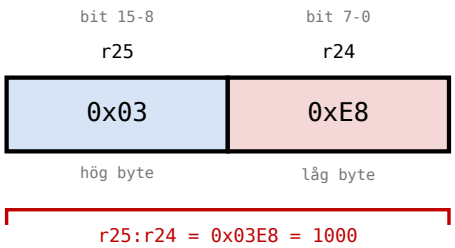
`sts` (*store direct to data space*) tar en adress och ett register, och tar två cykler. Motsatsen, **`lds r16, 0x0100`**, läser tillbaka byten till ett register. SRAM börjar på adress `0x0100`, så de adresserna är lediga att använda för egna data.

Lägg till de två raderna sist i programmet i avsnitt 10.1.3, så hamnar tecknen `'3'` och `'C'` efter varandra i minnet. I Microchip Studio syns de i fönstret **Memory** med **data IRAM** valt (avsnitt D.4.3 i bilaga D). Till höger om bytena `33 43` visar fönstret samma byte tolkade som ASCII, och där står texten `3C`.

10.1.6 Tal större än 255

Ett register är åtta bitar, och åtta bitar räcker till talen 0–255. Det räcker långt, men inte till allt: ett räknevärde på 1000, en fördröjning på en sekund, eller en adress i ett minne på 2048 byte.

Lösningen är att använda **två register tillsammans**, som ett 16-bitars tal. Det ena håller den **höga byten** (bit 15–8) och det andra den **låga byten** (bit 7–0). Ett sådant **registerpar** skrivs med det höga registret först, till exempel `r25:r24`.



Registerparet r25:r24 är ett 16-bitars tal. r25 är den höga byten och r24 den låga: 0x03 x 256 + 0xE8 = 768 + 232 = 1000.

Figur 10.2 Registerparet `r25:r24` som håller talet 1000.

Talet 1000 är `0x03E8`. Den höga byten är `0x03` och den låga `0xE8`, och värdet är `0x03 · 256 + 0xE8 = 768 + 232 = 1000`. En etta i den höga byten är alltså värd 256.

Assemblern delar upp ett 16-bitars tal åt dig, med `low()` och `high()`:

```
    ldi r24, low(1000)              ; r24 = 0xE8, the low byte.
    ldi r25, high(1000)             ; r25 = 0x03, the high byte.
```

Med 16 bitar går det att räkna till `2^16 - 1 = 65 535`. Paren `r25:r24`, `r27:r26`, `r29:r28` och `r31:r30` används oftast, av skäl som avsnitt 10.1.8 visar.

10.1.7 Addition och subtraktion med 16 bitar

Processorn räknar fortfarande åtta bitar i taget. En 16-bitars addition görs därför i två steg, på samma sätt som när man adderar för hand med minnessiffra (avsnitt 1.1.7): först de låga bytena, sedan de höga bytena **plus minnessiffran** från det första steget.

Minnessiffran hamnar i flaggan `C`, och det finns en instruktion som tar med den: **`adc`** (*add with carry*) beräknar `Rd = Rd + Rr + C`.

Exempel: 1000 + 2000 = 3000, eller `0x03E8 + 0x07D0`.

```
                r25    r24
               0x03   0xE8      1000
             +  0x07   0xD0      2000
             ---------------
add r24:              0x1B8  -> r24 = 0xB8, C = 1
                                (0xE8 + 0xD0 = 0x1B8 får inte plats)
adc r25:     0x03 + 0x07 + 1 = 0x0B
             ---------------
                0x0B   0xB8      3000
```

```
    ldi r24, low(1000)              ; r25:r24 = 1000 = 0x03E8.
    ldi r25, high(1000)
    ldi r22, low(2000)              ; r23:r22 = 2000 = 0x07D0.
    ldi r23, high(2000)
    add r24, r22                    ; Low bytes: 0xE8 + 0xD0 = 0x1B8. r24 = 0xB8, ↩
    ↪ C = 1.
    adc r25, r23                    ; High bytes and the carry: 0x03 + 0x07 + 1 = ↩
    ↪ 0x0B.
```

Utan `adc`, med `add` även för de höga bytena, hade svaret blivit `0x0AB8` = 2744: minnessiffran hade försvunnit, och felet hade varit exakt 256. Ett fel på 256 i ett 16-bitars resultat är nästan alltid en glömd minnessiffra.

Subtraktion fungerar likadant. **`sub`** tar de låga bytena, och **`sbc`** (*subtract with carry*) tar de höga bytena minus lånet i `C`: `Rd = Rd - Rr - C`.

```
    sub r20, r22                    ; Low bytes first; a borrow lands in C ...
    sbc r21, r23                    ; ... and sbc subtracts it from the high bytes.
```

Ordningen spelar roll: de låga bytena först, alltid. Det är den operationen som ger `C`, och den höga behöver det.

10.1.8 `adiw`, `sbiw` och `movw`

Tre instruktioner arbetar på ett helt registerpar på en gång:

Instruktion	Gör	Cykler
`adiw r24, 1`	`r25:r24 = r25:r24 + 1`. Konstanten får vara 0–63.	2
`sbiw r24, 1`	`r25:r24 = r25:r24 - 1`. Konstanten får vara 0–63.	2
`movw r18, r24`	Kopierar paret: `r19:r18 = r25:r24`.	1

Man skriver bara det låga registret i paret; det höga följer med. `adiw` och `sbiw` fungerar bara på de fyra paren `r25:r24`, `r27:r26`, `r29:r28` och `r31:r30`, vilket är varför de paren används mest. `movw` fungerar på alla par vars låga register har jämnt nummer.

Det viktigaste med `sbiw` är flaggan **Z**: den blir 1 när **hela** 16-bitarstalet blev noll, inte bara den ena byten. Därför kan `sbiw` följt av `brne` räkna ned ett 16-bitars tal till noll, vilket är precis vad en lång fördröjning behöver (avsnitt 10.1.10).

Programmet `add16.asm` visar alla tre: efter additionen i avsnitt 10.1.7 kopieras 3000 till `r19:r18` med `movw`, och `r25:r24` blir sedan 3001 med `adiw` och 2999 med `sbiw`.

10.1.9 Att jämföra 16-bitarstal

En jämförelse görs också byte för byte. `cpi` jämför de låga bytena, och **`cpc`** (*compare with carry*) jämför de höga bytena och tar hänsyn till resultatet av den första jämförelsen. Efter paret säger `Z` om talen var lika, och `C` om det första var mindre, precis som för 8-bitarstal.

```
    ldi r16, high(1000)             ; cpc compares with a register, not a constant.
count:
    adiw r24, 1                     ; r25:r24 = r25:r24 + 1.
    cpi r24, low(1000)              ; Compare the low bytes ...
    cpc r25, r16                    ; ... then the high bytes.
    brne count                      ; Not 1000 yet: again.
```

`cpc` finns bara i en form som jämför två register. Det finns ingen `cpci`, så den höga byten av konstanten måste först laddas i ett eget register, här `r16`.

Loopen räknar `r25:r24` från 0 till 1000. Varje varv tar `2 + 1 + 1 + 2 = 6` cykler, utom det sista, där `brne` inte hoppar och tar en cykel: `1000 · 6 - 1 = 5999` cykler.

10.1.10 En lång tidsfördröjning

Fördröjningarna i avsnitt 9.2 räknade med 8-bitars register. En loop med `dec` och `brne` kan gå högst 256 varv, och med 3 cykler per varv blir det ungefär 768 cykler, 48 µs. Två sådana loopar i varandra räcker till ungefär 200 000 cykler, drygt 12 ms. För en sekund, 16 000 000 cykler vid 16 MHz, behövs mer.

Med en 16-bitars loop kan den inre loopen gå upp till 65 535 varv. En sådan loop med `sbiw` och `brne` tar 4 cykler per varv:

```
    ldi r24, low(INNER_TURNS)       ; 1 cycle.
    ldi r25, high(INNER_TURNS)      ; 1 cycle.
inner:
    sbiw r24, 1                     ; 2 cycles.
    brne inner                      ; 2 cycles while not 0, 1 the last time.
```

Med `N` varv tar loopen `4N - 1` cykler: `N` varv om 4 cykler, minus en eftersom den sista `brne` inte hoppar. Det räcker till ungefär 262 000 cykler, 16 ms. För en hel sekund läggs ytterligare en loop utanför, som kör den inre loopen `M` gånger. Hela fördröjningen i `one_second_delay.asm`:

```
.equ OUTER_TURNS = 80                ; Outer loop: 80 turns.
.equ INNER_TURNS = 49999             ; Inner loop: 49 999 turns of 4 cycles each.

delay_begin:
    ldi r20, OUTER_TURNS             ; 1 cycle.
outer:
    ldi r24, low(INNER_TURNS)        ; 1 cycle.
    ldi r25, high(INNER_TURNS)       ; 1 cycle.
inner:
    sbiw r24, 1                      ; 2 cycles: r25:r24 = r25:r24 - 1.
    brne inner                       ; 2 cycles while r25:r24 is not 0, 1 the last ↪
    ↪ time.
    dec r20                          ; 1 cycle.
    brne outer                       ; 2 cycles while r20 is not 0, 1 the last time.
delay_end:
```

Räkna rad för rad, som i kapitel 9:

Instruktion	Cykler	Gånger	Summa
`ldi r20`	1	1	1
`ldi r24`, `ldi r25`	1 + 1	`M`	`2M`
inre loopen	`4N - 1`	`M`	`M(4N - 1)`
`dec r20`	1	`M`	`M`
`brne outer`, hoppar	2	`M - 1`	`2M - 2`
`brne outer`, hoppar inte	1	1	1

Summan är `M(4N + 4)`. Ett varv i den yttre loopen tar alltså `4N + 4` cykler, och med `N = 49 999` blir det exakt 200 000 cykler, 12,5 ms. Då behövs `M = 16 000 000 / 200 000 = 80` varv för en sekund:

```
M(4N + 4) = 80 · (4 · 49 999 + 4) = 80 · 200 000 = 16 000 000 cykler = 1,000000 s
```

Simulatorn mäter också 16 000 000 cykler mellan `delay_begin` och `delay_end`. Så väljer man konstanterna för en önskad tid:

1. Räkna om tiden till cykler: tid · 16 000 000.
2. Välj ett jämnt varv för den yttre loopen, till exempel 200 000 cykler, och räkna ut `N` ur `4N + 4 = 200 000`.
3. Dela antalet cykler med varvtiden, så får du `M`. `M` måste vara 1–255.

Namnen på konstanter och etiketter får inte krocka. Assemblern skiljer inte på stora och små bokstäver, så `.equ OUTER = 80` och etiketten `outer:` är samma namn, och programmet assembleras inte. Därför heter konstanterna `OUTER_TURNS` och `INNER_TURNS`.

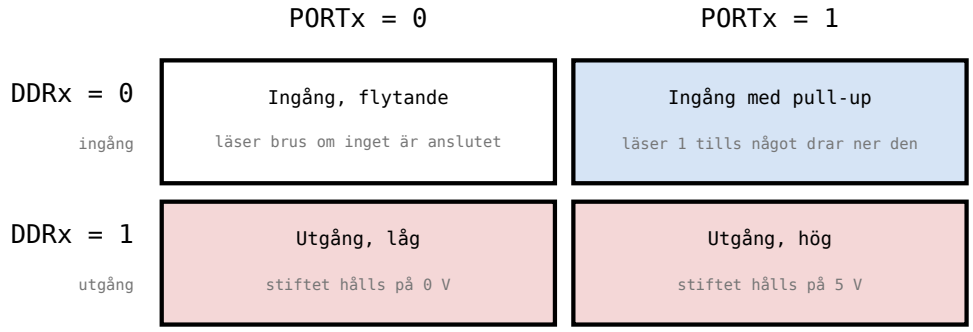
10.2 Inporten

10.2.1 Från utport till inport

I avsnitt 8.3 styrde programmet lysdioder genom utporten: `DDRB` gjorde stiften till utgångar och `PORTB` bestämde vad de skulle driva. Nu går informationen åt andra hållet. En knapp, en brytare eller en givare ändrar nivån på ett stift, och programmet läser av den.

Samma tre register per port används, men på ett annat sätt:

Register	Som utgång (kapitel 8)	Som ingång (nu)
`DDRD`	biten är 1	biten är **0**
`PORTD`	nivån som drivs ut	**1 = pull-up påslagen**, 0 = avslagen
`PIND`	(används inte)	**stiftets verkliga nivå**, att läsa



Figur 10.3 De fyra tillstånd ett stift kan ha, beroende på `DDRx` och `PORTx`.

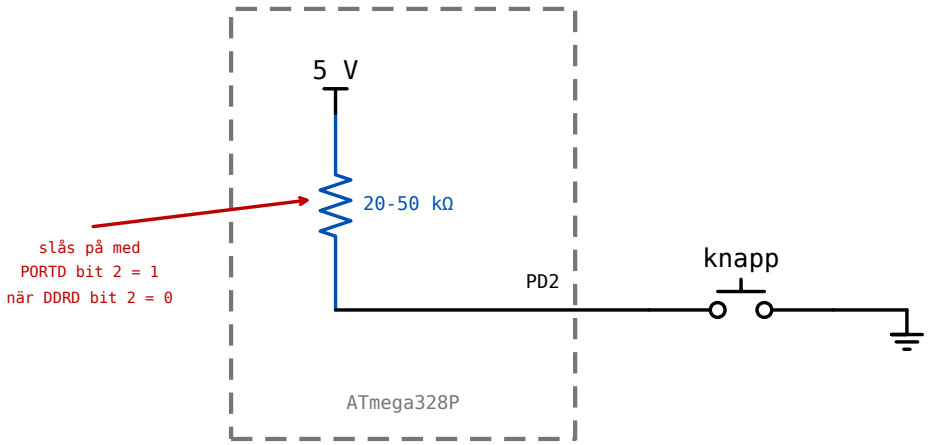
Efter reset är alla bitar i `DDRx` noll, så varje stift börjar som ingång. Men en ingång som inte är kopplad till något är **flytande**: den har ingen bestämd nivå, och `PINx` läser det brus som råkar finnas på ledningen. Den kan visa 1 när du håller handen nära kortet och 0 när du tar bort den. En flytande ingång är alltid ett fel, aldrig ett förval att lita på.

I kursen ligger knapparna på port D, Arduino-stift 2–7, och lysdioderna på port B, stift 8–13.

10.2.2 Knappen och pull-up-motståndet

Det enklaste sättet att ansluta en knapp är mellan stiftet och jord, tillsammans med ett **pull-up-motstånd** som håller stiftet högt när knappen är släppt. Motståndet behöver inte

kopplas in utanför kretsen: ATmega328P har ett inbyggt i varje stift, på 20–50 kΩ, som slås på genom att man skriver en etta till biten i `PORTD` medan biten i `DDRD` är noll.




```
Släppt knapp: pull-upen håller stiftet på 5 V och PIND bit 2 läses som 1.
      Nedtryckt knapp: stiftet kortsluts till jord och läses som 0.
         En nedtryckt knapp läses alltså som noll (aktivt låg).
```

Figur 10.4 En knapp från PD2 till jord, med det inbyggda pull-up-motståndet till 5 V inuti kretsen.

- **Knappen släppt:** ingenting drar ned stiftet, pull-upen håller det på 5 V, och `PIND` bit 2 läses som **1**.
- **Knappen nedtryckt:** stiftet kortsluts till jord, och `PIND` bit 2 läses som **0**.

En nedtryckt knapp läses alltså som noll. Det kallas att knappen är **aktivt låg**. Det är tvärtemot vad man väntar sig, och ett program som glömmer det gör exakt motsatsen till vad det ska, varje gång. Det är åtminstone lätt att upptäcka.

Något liknande fanns redan i kapitel 2: en brytande kontakt och en slutande kontakt ger motsatt logik. Här är det kopplingen, knapp mot jord med pull-up, som vänder på den.

Så här ställs stift PD2 in som ingång med pull-up:

```
    cbi DDRD, 2                     ; PD2 is an input (it already is after reset) ↩
    ↪ ...
    sbi PORTD, 2                    ; ... with its pull-up switched on.
```

10.2.3 Att läsa hela porten

`in r16, PIND` läser alla åtta stiften på port D på en gång, en bit per stift. Programmet `buttons_to_leds.asm` visar varje knapp på lysdioden med samma bitnummer:

```
main:
    ldi r16, 0x00
    out DDRD, r16                   ; Port D: all eight pins are inputs.
    ldi r16, 0xFF
    out PORTD, r16                  ; ... with their pull-ups switched on.
    out DDRB, r16                   ; Port B: all eight pins are outputs.

loop:
    in r16, PIND                    ; Read the level of all eight pins at once.
    com r16                         ; Invert: a pressed button (0) becomes a lit ↪
    ↪ LED (1).
    out PORTB, r16                  ; Show the result.
    rjmp loop                       ; And again, for ever.
```

`com` inverterar varje bit (avsnitt 8.2), så en nedtryckt knapp, som läses som 0, blir en tänd lysdiod. Utan `com` hade alla lysdioder lyst när ingen knapp var nedtryckt.

Följ en läsning där knapparna på PD2 och PD3 är nedtryckta:

Steg	r16 binärt	Förklaring
`in r16, PIND`	`1111 0011`	Bit 2 och 3 är 0: de två knapparna är nedtryckta.
`com r16`	`0000 1100`	Nu är de nedtryckta knapparna ettor.
`out PORTB, r16`	`0000 1100`	Lysdioderna på PB2 och PB3 lyser.

Programmet läser porten om och om igen, flera miljoner gånger i sekunden. Det kallas **pollning**: programmet frågar hela tiden i stället för att bli meddelat. För en människa som trycker på en knapp ser det ut som att lysdioden tänds i samma ögonblick.

Ofta är bara några av bitarna knappar. Då maskas de andra bort med `andi`, precis som i kapitel 8:

```
    in r16, PIND
    com r16                         ; Pressed = 1.
    andi r16, 0b00001100            ; Keep PD2 and PD3; clear the other six bits.
```

Stift 0 och 1 på Arduino-kortet (PD0 och PD1) är kopplade till USB-kretsen och används när kortet programmeras. Använd dem inte till knappar. I simulatorn spelar det ingen roll, men på kortet gör det det.

10.2.4 Att testa en enda bit: `sbic` och `sbis`

När programmet bara bryr sig om en knapp behöver det inte läsa hela porten. Två instruktioner testar en enda bit i ett I/O-register och **hoppar över nästa instruktion** beroende på vad biten är:

Instruktion	Hoppar över nästa instruktion om	Cykler
`sbic PIND, 2`	bit 2 i `PIND` är **0** (*skip if bit clear*)	1 om den inte hoppar, 2 om den hoppar
`sbis PIND, 2`	bit 2 i `PIND` är **1** (*skip if bit set*)	1 om den inte hoppar, 2 om den hoppar

Den instruktion som hoppas över är nästan alltid ett `rjmp`. Paret `sbic` + `rjmp` blir då ett villkorligt hopp: ”hoppa hit om biten är 1”. `led_follows_button.asm` tänder lysdioden på PB5 så länge knappen på PD2 hålls nedtryckt:

```
loop:
    sbic PIND, 2                    ; Skip the next instruction if PD2 is 0 ↩
    ↪ (pressed).
    rjmp released                   ; PD2 is 1: the button is released.
pressed:
    sbi PORTB, 5                    ; Light the LED.
    rjmp loop
released:
    cbi PORTB, 5                    ; Turn the LED off.
    rjmp loop
```

Läs det som två fall:

- **Knappen släppt**, bit 2 är 1: `sbic` hoppar inte över något, `rjmp released` körs, och lysdioden släcks.
- **Knappen nedtryckt**, bit 2 är 0: `sbic` hoppar över `rjmp released`, programmet fortsätter på `pressed`, och lysdioden tänds.

Namnen är lätta att blanda ihop. Ett sätt att komma ihåg dem: *sbic* hoppar när biten är **c**leared, nollställd, och *sbis* när den är **s**et, satt. Och eftersom knappen är aktivt låg betyder ”nollställd” här ”nedtryckt”.

`sbic` och `sbis` fungerar bara på de 32 första I/O-registren, vilket är de som portarna ligger i. För en bit i ett vanligt register finns motsvarande `sbrc` och `sbrs`.

10.2.5 Att vänta på en knapp

Ett vanligt behov är att programmet ska stå still tills någon trycker. Det är en loop som bara består av testet:

```
wait_press:
    sbic PIND, 2                    ; Skip the jump when the button is pressed (0).
    rjmp wait_press                 ; Released: keep waiting.
    ; The button is pressed; the program continues here.
```

Och omvänt, för att vänta tills knappen släpps:

```
wait_release:
    sbis PIND, 2                    ; Skip the jump when the button is released ↵
    ↪ (1).
    rjmp wait_release               ; Still pressed: keep waiting.
```

En väntan på ett tryck följd av en väntan på att knappen släpps är precis ett tryck. Men en riktig knapp **studsar**: kontakterna sluts och öppnas flera gånger under några millisekunder innan de ligger still, och ett program som räknar tryck kan räkna ett tryck som flera. Hur man hanterar det tas upp i avsnitt 11.1.4. I simulatorn studsar ingenting.

10.2.6 Logik i mjukvara

I Labb 1 (bilaga A) byggdes logiska funktioner av kontakter: två knappar i serie tände lampan bara när båda var nedtryckta, AND, och två parallellt när minst en var det, OR. Samma funktioner kan en mikrodator räkna ut i ett program, och då är det ett program och inte en koppling som ändras när funktionen ska ändras.

Lampan på PB0 ska lysa när **båda** knapparna på PD2 och PD3 är nedtryckta:

```
loop:
    in r16, PIND                    ; Read the buttons.
    com r16                         ; Pressed = 1.
    andi r16, 0b00001100            ; Keep PD2 and PD3.
    cpi r16, 0b00001100             ; Both pressed?
    brne not_both
    sbi PORTB, 0                    ; Yes: PB0 on.
    rjmp loop
not_both:
    cbi PORTB, 0                    ; No: PB0 off.
    rjmp loop
```

Efter `andi` finns fyra möjliga värden, ett per kombination av de två knapparna. Bara ett av dem, `0b00001100`, betyder att båda är nedtryckta, och det är det `cpi` letar efter. OR-funktionen, och fler, är del 21 i Labb 3.

Det här är skälet till att mikrodatorer har ersatt så mycket reläteknik: logiken ligger i ett program, och samma kort kan göra en ny sak genom att programmeras om. En PLC är i grunden en mikrodator med robusta in- och utgångar för 24 V.

10.2.7 Att simulera en ingång

I simulatorn finns ingen knapp att trycka på. I stället ändrar du ingångens nivå för hand, i fönstret **I/O** under **PORTD**, genom att klicka på biten i `PIND` (avsnitt D.4.5 i bilaga D).

Tänk på tre saker när du gör det:

- **Stoppa programmet först**, med en brytpunkt eller **Break All**, och ändra sedan biten. Programmet läser den nya nivån nästa gång det kör `in r16, PIND` eller `sbic PIND, 2`.
- **Kontrollera vad `PIND` visar efter att programmet har slagit på pull-uparna.** En släppt knapp ska läsas som 1. Om biten ändå visar 0, fyll i rutan för hand; det är så en släppt knapp ser ut.
- **En tom ruta är en nedtryckt knapp.** Det är aktivt låg logik även i simulatorn.

Förutsäg alltid först vad som ska hända, stega sedan, och jämför. En knapp i simulatorn är ett ypperligt tillfälle att öva på det: du bestämmer själv exakt när den trycks ned.

10.3 Subrutiner och stacken

10.3.1 Samma kod på flera ställen

Programmen har vuxit. Blinkprogrammet i avsnitt 10.1.10 har en fördröjning på nio rader, och ett trafikljus behöver fyra fördröjningar av olika längd. Att skriva av samma nio rader fyra gånger fungerar, men det är fyra ställen att rätta när något är fel, och fyra uppsättningar etiketter som måste heta olika.

Hex-till-ASCII-programmet i avsnitt 10.1.3 har samma problem i liten skala: samma fem rader två gånger.

Lösningen är en **subrutin**: en bit kod som skrivs en gång, får ett namn, och **anropas** från så många ställen som behövs. När subrutinen är klar fortsätter programmet där anropet gjordes. Andra programspråk kallar samma sak en funktion eller en metod.

10.3.2 `rcall` och `ret`

Två instruktioner räcker:

Instruktion	Gör	Cykler
`rcall delay_ms`	Sparar adressen till instruktionen efter `rcall`, och hoppar till `delay_ms`.	3
`ret`	Hoppar tillbaka till den sparade adressen.	4

Adressen som sparas kallas **returadressen**: den visar vart subrutinen ska återvända. Men var sparas den? Inte i något av de 32 registren, utan på **stacken**, ett område i toppen av dataminnet.

```
loop:
    ...
    rcall delay_ms              ; Save the address of rjmp loop, and jump.
    rjmp loop                   ; ret comes back here.

delay_ms:
    ...                         ; The subroutine's work.
    ret                         ; Jump back to the saved address.
```

Ett anrop kostar alltså $3 + 4 = 7$ cykler innan subrutinen har gjort något alls. Det är lite, men det är skälet till att en fördröjning i en subrutin blir några cykler längre än samma loop skriven direkt i programmet.

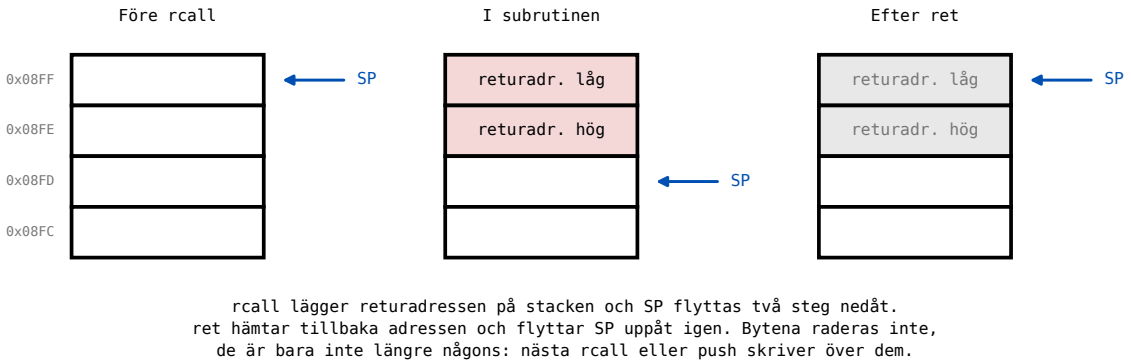
10.3.3 Stacken och stackpekaren

Stacken är en del av SRAM som används som en trave: det som läggs dit sist tas bort först. Den börjar i toppen av dataminnet, på adress `RAMEND` = `0x08FF`, och växer **nedåt**, mot lägre adresser.

Stackpekaren, `SP`, är ett 16-bitars register som pekar på den första lediga byten i stacken. Den består av två I/O-register, `SPH` (hög byte) och `SPL` (låg byte).

Så här går ett anrop till, byte för byte:

1. Före `rcall` pekar `SP` på `0x08FF`, och stacken är tom.
2. `rcall` skriver returadressens låga byte på `0x08FF` och den höga på `0x08FE`, och flyttar `SP` två steg nedåt, till `0x08FD`. Sedan hoppar den.
3. `ret` flyttar `SP` två steg uppåt, läser returadressen, och hoppar dit.



Figur 10.5 Stacken före `rcall`, i subrutinen och efter `ret`.

Lägg märke till det sista: `ret` **raderar ingenting**. Bytena ligger kvar i minnet, men `SP` pekar förbi dem, och nästa anrop skriver över dem.

Att initiera stackpekaren

Ett program som använder subrutiner börjar med att peka `SP` på toppen av SRAM:

```
main:
    ldi r16, high(RAMEND)           ; Point the stack pointer at the top of SRAM,
    out SPH, r16                    ; high byte first ...
    ldi r16, low(RAMEND)
    out SPL, r16                    ; ... then the low byte. SP = 0x08FF.
```

På ATmega328P gör hårdvaran faktiskt redan detta vid varje reset, så i våra program ändrar de fyra raderna ingenting. De skrivs ändå, av två skäl: många andra AVR-kretsar startar med `SP` = 0, och ett program som hoppar in i ditt program utan reset, till exempel en bootloader, kan ha lämnat `SP` var som helst. Fyra rader är billig försäkring, och de gör programmet begripligt för den som läser det.

10.3.4 En subrutin med parameter: `delay_ms`

En fördröjning som alltid är lika lång är till begränsad nytta. Bättre är en subrutin som får veta hur länge den ska vänta. Värdet den får kallas en **parameter**, och i kursen lämnas den i ett register före anropet, här `r24`:

```
    ldi r24, 250                    ; Wait 250 ms.
    rcall delay_ms
```

Hela subrutinen, från `blink_subroutine.asm`:

```
;-------------------------------------------------------------------------------
; delay_ms: Waits r24 milliseconds (1-255) at 16 MHz.
;
; Each turn of the outer loop takes exactly 16 000 cycles, 1 ms. With the rcall, ↩
    ↪ the
; pushes, the pops and the ret, a call takes 16 000 x r24 + 18 cycles.
;-------------------------------------------------------------------------------
delay_ms:
    push r24                        ; Save the registers this subroutine changes.
    push r26
    push r27
delay_ms_outer:
    ldi r26, low(3999)              ; X = r27:r26 = 3999.
    ldi r27, high(3999)
delay_ms_inner:
    sbiw r26, 1                     ; 4 cycles per turn, 3 the last time.
    brne delay_ms_inner
    dec r24                         ; One more millisecond done.
    brne delay_ms_outer
    pop r27                         ; Restore them, in the opposite order.
    pop r26
    pop r24
    ret                             ; Back to the instruction after the rcall.
```

Uppbyggnaden är samma som sekundfördröjningen i avsnitt 10.1.10: en 16-bitars inre loop i en yttre loop. Den yttre loopen räknar ned `r24`, en millisekund per varv. Ett varv tar `4 · 3999 + 4 = 16 000` cykler, precis 1 ms vid 16 MHz.

Resten av cyklerna är subrutinens egna:

Del	Cykler
`rcall`	3
tre `push`	6
`r24` varv om 16 000 cykler, minus 1 för sista `brne`	$16\,000 \cdot$ `r24` $- 1$
tre `pop`	6
`ret`	4

Summan är **16 000 · r24 + 18** cykler. Med `r24` = 250 blir det 4 000 018 cykler, och det är också vad simulatorn mäter mellan `rcall` och instruktionen efter. De 18 extra cyklerna tar drygt en mikrosekund, en miljondels sekund. Ingen människa ser skillnad, men det är bra att veta var de kommer ifrån.

Varför börjar och slutar subrutinen med `push` och `pop`? Det är ämnet för de två nästa avsnitten.

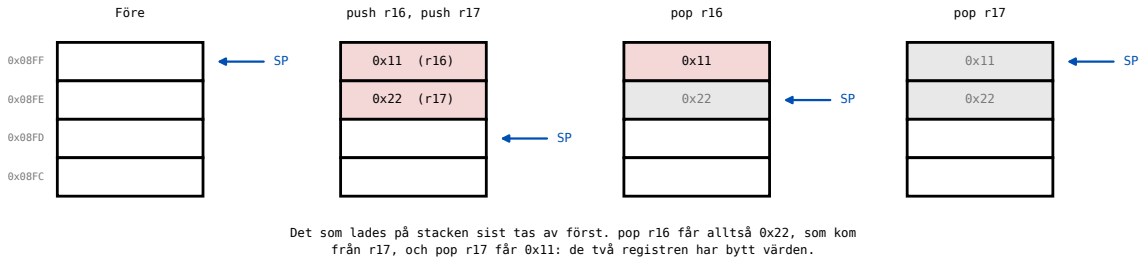
10.3.5 push och pop

Stacken kan användas till mer än returadresser. Två instruktioner lägger dit och hämtar godtyckliga register:

Instruktion	Gör	Cykler
`push r16`	Skriver `r16` där `SP` pekar, och flyttar `SP` ett steg nedåt.	2
`pop r16`	Flyttar `SP` ett steg uppåt, och läser byten där till `r16`.	2

Stacken är **sist in, först ut**. Programmet `push_pop.asm` visar vad det betyder:

```
ldi r16, 0x11
ldi r17, 0x22
push r16                        ; 0x08FF = 0x11, SP = 0x08FE.
push r17                        ; 0x08FE = 0x22, SP = 0x08FD.
pop r16                         ; r16 = 0x22 (the last one pushed), SP = ↩
↪ 0x08FE.
pop r17                         ; r17 = 0x11, SP = 0x08FF.
```



Figur 10.6 Stacken när `r16` och `r17` läggs dit och hämtas tillbaka i samma ordning.

Registren har bytt värden, eftersom `pop r16` får det som lades dit sist. Vill man ha tillbaka samma värden i samma register ska de hämtas i **omvänd ordning**: `push r16`, `push r17`, sedan `pop r17`, `pop r16`.

10.3.6 Kursens regel: spara det du ändrar

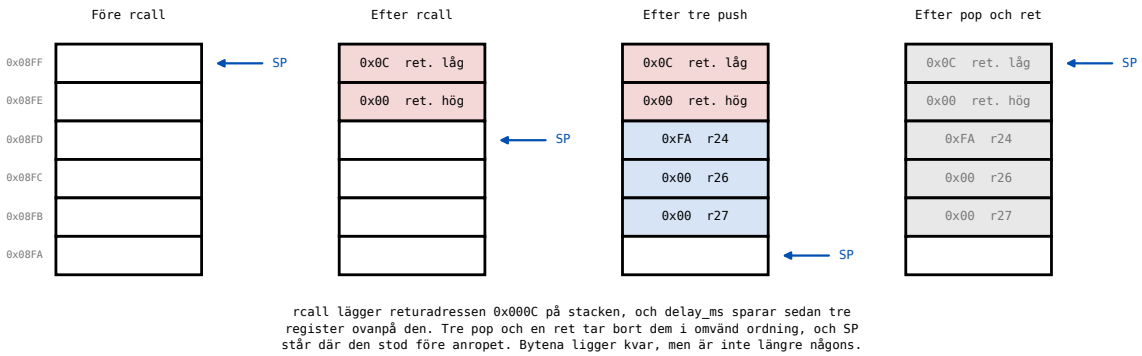
En subrutin använder register för sitt arbete. `delay_ms` räknar med `r24`, `r26` och `r27`. Men programmet som anropar den kan ha något viktigt i just de registren, och det vet inte subrutinen.

Kursens regel är därför enkel:

> **En subrutin sparar varje register den ändrar med `push` när den börjar, och återställer dem med `pop` i omvänd ordning innan `ret`.** Undantaget är ett register som subrutinen är till för att lämna ett svar i; det ska stå i kommentaren ovanför subrutinen.

Med regeln behöver den som anropar en subrutin aldrig undra om ett register har ändrats. Efter `rcall delay_ms` innehåller `r24` fortfarande 250, och därför kan samma parameter användas igen utan att laddas om.

Så här ser stacken ut mitt i `delay_ms`, efter anropet och de tre `push`-instruktionerna, i `blink_subroutine.asm`. `rcall` ligger på adress `0x000B` i programminnet, så returadressen är `0x000C`:



Figur 10.7 Stacken i `blink_subroutine.asm`: före `rcall`, efter `rcall`, efter tre `push` och efter `pop` och `ret`.

Fem byte är upptagna: två för returadressen och tre för de sparade registren. `SP` pekar på `0x08FA`. Tre `pop` tar bort registren i omvänd ordning, `ret` tar returadressen, och `SP` är tillbaka på `0x08FF`.

Vad som händer om man slarvar

Anta att `delay_ms` gör tre `push` men bara två `pop`. Då ligger en byte kvar ovanpå returadressen när `ret` körs, och `ret` läser den byten som en del av adressen. Programmet hoppar till en adress som ingen har bestämt, någonstans i programminnet, och fortsätter därifrån. Ingenting varnar. Ofta hamnar programmet i tomt flashminne, kör igenom det, och börjar om från adress 0, så att det ser ut som om kortet startar om av sig självt.

Det är därför regeln säger **varje** `push` har sin `pop`, i omvänd ordning. Ett fel här syns inte när man läser koden rad för rad, men det syns direkt i simulatorn: stega fram till `ret` och titta på `SP`. Står den inte där den stod när subrutinen började, stämmer inte stacken.

10.3.7 Subrutiner som anropar subrutiner

En subrutin kan anropa en annan. Varje anrop lägger en ny returadress på stacken, och varje `ret` tar bort den senaste, så programmet hittar alltid tillbaka i rätt ordning.

```
delay_500ms:
    push r24                        ; Save the register this subroutine changes.
    ldi r24, 250
    rcall delay_ms                  ; 250 ms. delay_ms saves r24, so it is still ↪
    ↪ 250 ...
    rcall delay_ms                  ; ... and this is another 250 ms.
    pop r24
    ret
```

När huvudprogrammet anropar `delay_500ms`, som anropar `delay_ms`, ligger det på stacken:

- returadressen till huvudprogrammet, 2 byte;
- `r24`, sparat av `delay_500ms`, 1 byte;
- returadressen till `delay_500ms`, 2 byte;
- `r24`, `r26` och `r27`, sparade av `delay_ms`, 3 byte.

Åtta byte, och `SP` står på `0x08FF - 8 = 0x08F7`. Det är den djupaste punkten i programmet.

Lägg också märke till att det andra anropet av `delay_ms` fungerar utan att `r24` laddas om. Det är regeln i avsnitt 10.3.6 som gör det möjligt.

10.3.8 Hur mycket stack finns det?

SRAM är 2048 byte. Programmen i kursen använder som mest ett tiotal byte stack, så det finns gott om plats. Men det finns ingen spärr: om stacken skulle växa ned i området där programmet har egna data, till exempel texten i avsnitt 10.1.5, skriver den över dem utan varning.

Därför är det värt att räkna efter, på samma sätt som i avsnitt 10.3.7: två byte per anrop som pågår samtidigt, och en byte per sparat register. Summan är stackens största djup.

10.3.9 Hex-till-ASCII som subrutin

Nu kan programmet i avsnitt 10.1.3 skrivas utan de dubbla raderna. Omvandlingen av en nibble blir en subrutin som tar värdet 0–15 i `r24` och lämnar tecknet i samma register. Det är just det undantag regeln i avsnitt 10.3.6 tillåter: `r24` är till för svaret, och kommentaren säger det.

```
main:
    ldi r16, high(RAMEND)           ; SP = RAMEND.
    out SPH, r16
    ldi r16, low(RAMEND)
    out SPL, r16

    ldi r16, 0x3C                   ; The byte to convert.
    mov r24, r16
    swap r24
    andi r24, 0x0F                  ; The high nibble, 0x03.
    rcall nibble_to_ascii
    mov r20, r24                    ; r20 = '3'.
    mov r24, r16
    andi r24, 0x0F                  ; The low nibble, 0x0C.
    rcall nibble_to_ascii
    mov r21, r24                    ; r21 = 'C'.
end:
    rjmp end

;-------------------------------------------------------------------------------
; nibble_to_ascii: Turns the value 0-15 in r24 into its hexadecimal character.
; Returns the character in r24; changes no other register.
;-------------------------------------------------------------------------------
nibble_to_ascii:
    cpi r24, 10                     ; A digit 0-9, or a letter A-F?
    brlo nibble_digit
    subi r24, -7                    ; A-F: 7 more, so that 10 lands on 'A'.
nibble_digit:
    subi r24, -0x30                 ; Add the code for '0'.
    ret
```

Subrutinen ändrar flaggorna, med `cpi` och `subi`. Det gör nästan varje subrutin, och det är därför huvudprogrammet aldrig ska räkna med att flaggorna är orörda efter ett anrop. Ett villkorligt hopp ska alltid komma direkt efter den jämförelse det bygger på.

10.4 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Göra om ett värde 0–15 till sitt hexadecimala tecken och tillbaka, och en byte till två tecken.
- Skriva ett 16-bitars tal som två byte, och addera, subtrahera och jämföra 16-bitarstal med `add`/`adc`, `sub`/`sbc` och `cpi`/`cpc`.
- Räkna ut konstanterna för en fördröjning på en given tid, och kontrollera resultatet med stoppuret i simulatorn.
- Ställa in ett stift som ingång med pull-up, läsa det med `in` eller `sbic`/`sbis`, och förklara varför en nedtryckt knapp läses som noll.
- Skriva en subrutin, anropa den med `rcall`, och förklara vad som händer med stacken och `SP` vid `rcall`, `push`, `pop` och `ret`.
- Följa kursens regel: en subrutin sparar varje register den ändrar, och återställer dem i omvänd ordning.

Frågor att testa dig själv med

1. Varför behövs sju extra när värdet 12 ska bli tecknet `'C'`, men inte när 7 ska bli `'7'`?
2. Ett 16-bitars resultat är exakt 256 för litet. Vilken instruktion har troligen bytts ut, och mot vilken?
3. Varför fungerar `sbiw r24, 1` men inte `sbiw r20, 1`?
4. En knapp mot jord läses som 1 när den är släppt. Vad är det som håller stiftet högt?
5. `sbic PIND, 2` följt av `rjmp released`: vilket läge har knappen när `rjmp` hoppas över?
6. `rcall` körs när `SP` är `0x08FF`. Var ligger returadressens två byte, och vad är `SP` sedan?
7. En subrutin gör `push r16`, `push r17`, `pop r16`, `pop r17`. Vad händer med registren?
8. Varför kan `delay_500ms` anropa `delay_ms` två gånger i rad utan att ladda om `r24` emellan?

Nästa kapitel

Kapitel 11 flyttar programmen från simulatorn till ett riktigt kort:

- Styrobjekt i verkligheten: lysdioder, knappar som studsar, och ett relä som styr en lampa på 24 V.
- Programmet flyttas från simulatorn till ett riktigt Arduino Uno-kort.
- Från flödesplan till program: ett trafikljus, steg för steg.
- Labb 3 slutförs med slutuppgiften, trafikljuset och övergångsstället.

10.5 Övningar

> **Så kontrollerar du ditt arbete.** Övningarna märkta **Program** skrivs i Microchip Studio och kontrolleras i simulatorn: förutsäg vad som ska hända, stega, och jämför. Knappar simuleras genom att biten i `PIND` ändras för hand i fönstret I/O (avsnitt D.4.5 i bilaga D).
> Övningarna märkta **Räkna för hand** och **Förståelse** görs på papper, utan miniräknare, och kontrolleras mot bilaga I. Gör varje övning innan du läser lösningen. Där en övning har ett troligt felsvar tar lösningen upp det också, och säger varför det är fel.
> Instruktionerna finns på referensbladet i bilaga E.
> Varje övning är märkt med sin sort. Exakt en övning är en **Kontroll**, och i det här kapitlet är det övning 10.11.

Övning 10.1 Tecken och koder

RÄKNA FÖR HAND

Använd tabellen i avsnitt 10.1.1.

a) Vilka ASCII-koder har tecknen `'5'`, `'9'`, `'E'` och `'F'`? Svara hexadecimalt.

b) Vilket tal blir kvar när ASCII-koden för `'8'` minskas med ASCII-koden för `'0'`?

c) Förklara med en mening varför omvandlingen av värdena 10–15 behöver sju extra, men inte omvandlingen av 0–9.

d) Raden `ldi r16, 'E'` och raden `ldi r16, 0x45` assembleras till exakt samma maskinkod. Varför skriver man ändå ibland den första?

Övning 10.2 En byte blir text

RÄKNA FÖR HAND

Programmet i avsnitt 10.1.3 körs med `ldi r16, 0x9E` i stället för `0x3C`.

a) Fyll i värdet i `r24` efter varje instruktion för den höga nibblen: `mov`, `swap`, `andi`, och sedan det som `cpi/brlo` bestämmer. Vilket värde får `r20`?

b) Gör samma sak för den låga nibblen. Vilket värde får `r21`?

c) Vilken text står i minnet om programmet avslutas med `sts 0x0100, r20` och `sts 0x0101, r21`?

d) Gör om a)–c) för `ldi r16, 0x5A`.

Övning 10.3 Från tecken till värde

RÄKNA FÖR HAND

Använd koden i avsnitt 10.1.4.

a) Följ tecknet `'B'` (`0x42`) genom koden. Vilket värde får `r22`?

b) Följ tecknet `'4'`.

c) (*fördjupning*) Någon skriver ett litet `b` på tangentbordet, ASCII `0x62`. Vilket värde lämnar koden i `r22`, och varför är det fel? Föreslå en ändring som hanterar små bokstäver.

Övning 10.4 Registerpar

RÄKNA FÖR HAND

a) Skriv talen 2500, 256, 255 och 40 000 som 16-bitars tal i hexadecimal form, och ange vad `low()` och `high()` blir för vart och ett.

b) `r25` innehåller `0x12` och `r24` innehåller `0x34`. Vilket decimalt tal är `r25:r24`?

c) Vilket är det största tal ett registerpar kan hålla? Varför just det?

Övning 10.5 Addition och subtraktion med 16 bitar

RÄKNA FÖR HAND

a) `r25:r24` = `0x12F0` och `r23:r22` = `0x0A20`. Följ `add r24, r22` och `adc r25, r23` steg för steg, som i avsnitt 10.1.7: vad blir `r24`, `C` och `r25`? Kontrollera svaret genom att räkna om båda talen och summan decimalt.

b) Någon skriver `add r25, r23` i stället för `adc r25, r23`. Vilket svar blir det då, och med hur mycket är det fel?

c) `r21:r20` = `0x0100`, alltså 256. Följ `ldi r22, 1`, `ldi r23, 0`, `sub r20, r22` och `sbc r21, r23`. Vad blir `r20`, `C` och `r21`, och vilket tal är resultatet?

d) Någon vill göra samma sak kortare, med `sbiw r20, 1`. Varför går det inte att assemblera, och vilket registerpar hade fungerat?

Övning 10.6 En fördröjning på 10 ms

RÄKNA FÖR HAND

a) Hur många klockcykler är 10 ms vid 16 MHz?

b) En enda 16-bitars loop med `sbiw` och `brne`, som i avsnitt 10.1.10, plus de två `ldi` före den. Hur många varv `N` ska loopen gå för att hela fördröjningen ska bli så nära 10 ms som möjligt, och hur många cykler blir det exakt?

c) Samma fördröjning med subrutinen `delay_ms` från avsnitt 10.3.4: vilket värde ska `r24` ha, och hur många cykler tar anropet?

d) Vilken av de två är närmast 10 ms? Vilken skulle du välja i ett program, och varför?

Övning 10.7 Knappen som läses baklänges

FÖRSTÅELSE

En knapp är kopplad mellan PD2 och jord, som i avsnitt 10.2.2.

a) Förklara, med pull-up-motståndet, varför `PIND` bit 2 läses som 0 när knappen är nedtryckt.

b) Programmet glömmer att slå på pull-upen (`sbi PORTD, 2`). Vad läser programmet när knappen är släppt? Varför är det svårt att hitta det felet genom att prova?

c) Varför innehåller `buttons_to_leds.asm` instruktionen `com`? Vad hade lysdioderna visat utan den, när ingen knapp är nedtryckt?

Övning 10.8 Följ programmet

RÄKNA FÖR HAND

Programmet i avsnitt 10.2.3 läser `PIND` och visar det inverterade värdet på `PORTB`.

a) `PIND` läses som `0b11111011`. Vilka knappar är nedtryckta, och vad blir `PORTB`?

b) Programmet ändras så att `andi r16, 0xF0` läggs in mellan `com` och `out`. Vad blir `PORTB` nu för samma `PIND`? Och för `PIND` = `0b01101111`?

c) Hur många gånger per sekund ungefär läser loopen porten vid 16 MHz? Räkna cyklerna för de fyra instruktionerna i loopen.

Övning 10.9 Start och stopp med självhållning

PROGRAM

I kapitel 2 byggdes en start/stopp-krets med ett relä som håller sig självt: ett tryck på startknappen drar reläet, och det förblir draget när knappen släpps, tills stoppknappen trycks. Samma funktion ska nu skrivas som ett program.

- Startknappen sitter på PD2 och stoppknappen på PD3, båda mot jord med pull-up.
- En ”motor”, en lysdiod på PB0, startar när startknappen trycks, och fortsätter gå när knappen släpps.
- Motorn stannar när stoppknappen trycks.
- Om båda knapparna hålls nedtryckta samtidigt ska motorn **stå still**: stopp går före start, av säkerhetsskäl.

a) Rita en flödesplan innan du skriver någon kod.

b) Skriv programmet, med `sbic` eller `sbis`.

c) Testa i simulatorn alla fyra fallen: start, släpp start, stopp, och båda samtidigt.

d) Var i programmet finns ”minnet”, det som motsvarar reläets självhållning?

Övning 10.10 `rcall`, `ret` och stacken

FÖRSTÅELSE

a) Beskriv de två saker `rcall` gör, i den ordning den gör dem.

b) Varför behöver `ret` stacken? Kunde returadressen ha sparats i ett av de 32 registren i stället?

c) `SP` är `0x08FF` när ett `rcall` körs. Vad är `SP` när subrutinen börjar? Och efter `ret`?

d) Efter `ret` ligger returadressen kvar i minnet. Varför är det inget problem?

Övning 10.11 Kontroll: förutsäg stacken

KONTROLL

Förutsäg för hand, kontrollera i simulatorn, förklara skillnaden.

Programmet nedan anropar en subrutin som lägger två register på stacken. Adresserna i programminnet kommer från listfilen och står i kommentarerna.

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main                       ; 0x0000

main:
    ldi r16, high(RAMEND)           ; 0x0001
    out SPH, r16                    ; 0x0002
    ldi r16, low(RAMEND)            ; 0x0003
    out SPL, r16                    ; 0x0004
    ldi r16, 0xA5                   ; 0x0005
    ldi r17, 0x3C                   ; 0x0006
    rcall save_two                  ; 0x0007
end:
    rjmp end                        ; 0x0008

save_two:
    push r16                        ; 0x0009
    push r17                        ; 0x000A
here:
    pop r17                         ; 0x000B
    pop r16
    ret
```

a) För hand

När programmet står på etiketten `here`, alltså efter de två `push`: vilket värde har `SP`, och vilka byte ligger på adresserna `0x08FF`, `0x08FE`, `0x08FD` och `0x08FC`? Skriv upp svaret innan du går vidare.

b) I simulatorn

Skriv in programmet, sätt en brytpunkt på `pop r17` och kör dit. Läs `SP` i Processor Status och bytena i fönstret Memory med **data IRAM** valt.

c) Jämför

Stämmer din förutsägelse? Den vanligaste avvikelsen är att returadressens två byte står i fel ordning, eller att `SP` är en byte fel. Om ditt svar skiljer sig: vilket av de två felen var det, och varför?

d)

Stega vidare till instruktionen efter `ret`. Vilket värde har `SP` nu, och vilka av de fyra bytena har ändrats?

Övning 10.12 Hitta felet

FÖRSTÅELSE FÖRDJUPNING

a) En subrutin börjar med `push r16` och `push r17`, och slutar med `pop r16`, `pop r17` och `ret`. Den assembleras utan fel och återvänder rätt. Vad är fel, och hur märks det i programmet som anropar den?

b) En annan subrutin börjar med `push r16` och `push r17`, men slutar med bara `pop r17` och `ret`. Vad händer när `ret` körs? Varför märks felet mycket tydligare än i a)?

c) Hur kan du i simulatorn, med ett enda värde, se att en subrutin inte lämnar stacken som den fann den?

Övning 10.13 Hela byten som text

PROGRAM FÖRDJUPNING

Skriv en subrutin `byte_to_ascii` som gör om byten i `r24` till två tecken: det för den höga nibblen i `r24` och det för den låga i `r25`. Den ska använda `nibble_to_ascii` från avsnitt 10.3.9 och följa kursens regel om register.

a) Vilka register ändrar `byte_to_ascii`, och vilka av dem måste sparas?

b) Skriv subrutinen och ett huvudprogram som anropar den med `0xB7` och lägger tecknen på adress `0x0100` och `0x0101`.

c) Kontrollera i fönstret Memory att texten `B7` står där. Hur djup blir stacken som mest?

KAPITEL 11

Styrobjekt på riktig hårdvara

Från rutor i simulatorn till lysdioder, knappar och reläer på ett riktigt kort.

I DET HÄR KAPITLET

- Styrobjekt: ingångar, beslut och utgångar, och varför mikrodatorn har tagit över efter reläer.
- Lysdioden och förkopplingsmotståndet, räknat med Ohms lag.
- Större laster: en transistor som drar ett relä, som tänder lampan på 24 V från Labb 1.
- Knappar som studsar, och avstudsning med en kort väntan.
- Arduino Uno-kortet, och hur programmet förs över från Microchip Studio.
- Från flödesplan till program: vägarbetsljuset som genomarbetat exempel.
- **Labb 3** slutförs: slutuppgiften, trafikljuset och övergångsstället, på kortet.

Hittills har programmen körts i simulatorn, där en utgång är en ruta i ett fönster. Det här kapitlet flyttar dem till ett riktigt Arduino Uno-kort, med riktiga lysdioder, knappar och reläer, och visar vad som tillkommer då: strömmar som måste begränsas, knappar som studsar och kopplingar som kan vara fel. Sedan går det, i ett genomarbetat exempel, från en tillståndstabell via en flödesplan till ett färdigt program, med samma metod som Labb 3:s slutuppgift använder.

Kapitlet förutsätter att Labb 3 är klar till och med del 22: slutuppgiften bygger på subrutiner. Läs avsnitt 11.1 och avsnitt 11.2 innan du börjar på slutuppgiften i bilaga C, och rita flödesplanen för dess del A innan du skriver programmet; den ska redovisas först. Installera Arduino IDE och gör inställningen i avsnitt D.5 i bilaga D innan du ska programmera kortet, så går labbtiden till att programmera och inte till att installera. Övningarna i avsnitt 11.4 kontrollerar du mot bilaga I.

Gör två förutsägelser innan något körs: hur stor strömmen genom en röd lysdiod med 220 Ω blir, och vad multimetern kommer att visa; och hur många tryck `press_counter.asm` räknar för fem tryck när väntan är borttagen.

11.1 Styrobjekt: in- och utgångar i verkligheten

11.1.1 Vad ett styrobjekt är

Ett **styrobjekt** är det som ett styrsystem styr: en lampa, en motor, en ventil, en port, ett trafikljus. Styrsystemet får veta hur det står till genom **ingångar**, knappar, brytare och givare, och påverkar styrobjektet genom **utgångar**. Däremellan fattas besluten.

```
  ingångar                  beslut                    utgångar
knappar, givare   --->   mikrodatorns program   --->   lampor, reläer, motorer
```

Det är samma bild som boken har ritat hela tiden, bara med olika saker i mitten:

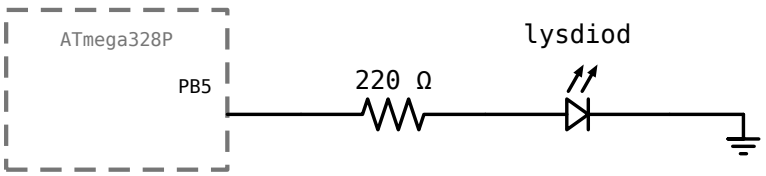
- i Labb 1 (bilaga A) var besluten kontakternas koppling, och de ändrades med sladdarna;
- i Labb 2 (bilaga B) var de grindar i IC-kretsar, och de ändrades på kopplingsdäcket;
- nu är de ett program, och de ändras genom att programmet skrivs om.

Det sista är skälet till att nästan all styrning i dag görs med mikrodatorer. En PLC, den dator som styr maskiner i industrin, är i grunden en mikrodator som läser ingångar, kör ett program och sätter utgångar, om och om igen, med in- och utgångar byggda för 24 V och en elektriskt stökig omgivning.

I Labb 3 har programmen hittills körts i simulatorn, där en utgång är en ruta i ett fönster. Det här avsnittet handlar om vad som tillkommer när utgången är en riktig lysdiod och ingången en riktig knapp.

11.1.2 Lysdioden och förkopplingsmotståndet

En lysdiod kopplas från ett stift, genom ett motstånd, till jord. När stiftet är högt, 5 V, går det en ström genom lysdioden och den lyser; när stiftet är lågt, 0 V, går ingen ström.




```
En etta i PORTB bit 5 tänder lysdioden, en nolla släcker den. Motståndet begränsar
strömmen; utan det kan både lysdioden och stiftet förstöras.
```

Figur 11.1 Stiftet PB5 via ett motstånd på 220 Ω och en lysdiod till jord.

Motståndet är inte valfritt. En lysdiod är inte ett motstånd: spänningen över den är nästan densamma oavsett ström, ungefär 2 V för en röd eller gul lysdiod och 2–3 V för en grön. Utan något som begränsar strömmen blir den så stor som stiftet orkar leverera, och det räcker för att förstöra lysdioden, och ibland stiftet.

Med motståndet i serie ligger resten av spänningen över motståndet, och strömmen följer av Ohms lag:

```
I = (5 V - 2 V) / 220 Ω = 3 V / 220 Ω ≈ 13,6 mA
```

Så väljer man motstånd:

- ATmega328P tål **40 mA per stift** som absolut maximum, och bör hållas kring **20 mA** eller mindre i drift. Stiften i en grupp delar dessutom på en gemensam gräns, 100–150 mA beroende på kapsel och grupp; databladet anger exakt vilka stift som hör ihop. Räkna med 100 mA, så finns det marginal.
- En vanlig lysdiod lyser bra vid 5–15 mA. **220–330 Ω** ger 9–14 mA, och är vad kursen använder.
- Ett större motstånd ger en svagare lysdiod men är aldrig farligt. Ett för litet är det.

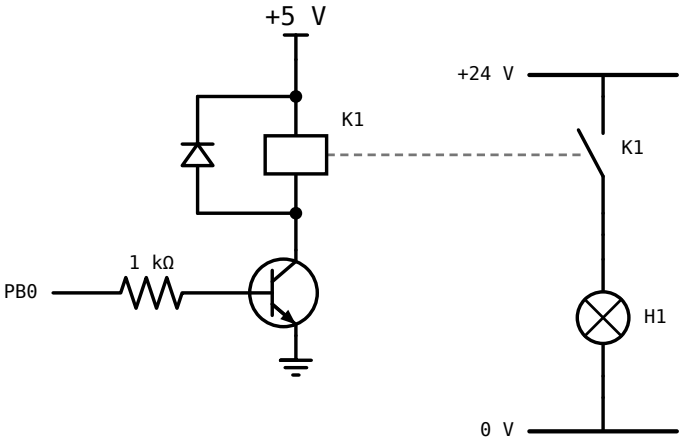
Lysdioden måste vändas rätt. Det långa benet, anoden, ska mot stiftet och motståndet, och det korta, katoden, mot jord. Vänd fel lyser den inte alls, men den går inte sönder av det.

Den andra kopplingen fungerar också. En lysdiod kan kopplas från 5 V genom motståndet till stiftet. Då lyser den när stiftet är **lågt**, och programmet måste skriva en nolla för att tända. Det är inget fel, men det vänder på varje `sbi` och `cbi` i programmet, och ett program som skrivits för den ena kopplingen gör exakt fel på den andra.

11.1.3 Större laster: transistor och relä

Ett stift klarar en lysdiod, men inte en motor, en magnetventil eller glödlampan på 24 V från Labb 1 (bilaga A). Då behövs något emellan, som låter en liten ström från stiftet styra en stor ström i en annan krets.

Det vanligaste är en **transistor** som styr ett **relä**:



Stiftet orkar inte driva reläspolen själv. Transistorn gör det: en etta på PB0 slår på den, spolen drar, och reläkontakten K1 sluter 24 V-kretsen till lampan H1. Dioden tar hand om spänningstoppen när spolen släpps; utan den förstörs transistorn.

Figur 11.2 PB0 styr via en transistor reläspolen K1, vars kontakt tänder lampan H1 i en krets på 24 V.

Så fungerar kopplingen:

1. En etta på PB0 ger en liten ström, under 5 mA, genom motståndet in i transistorns bas.
2. Transistorn leder, och en större ström går från 5 V genom reläets spole.
3. Spolen drar, och reläkontakten K1 sluter. Kontakten sitter i en helt annan krets, på 24 V, där den tänder lampan H1.

Reläkontakten är samma sorts slutande kontakt som i Labb 1, bara manövrerad av en spole i stället för av ett finger. Mikrodatorn och 24 V-kretsen har inte ens gemensam jord: de hålls isär av reläet, vilket är ett skäl att använda ett.

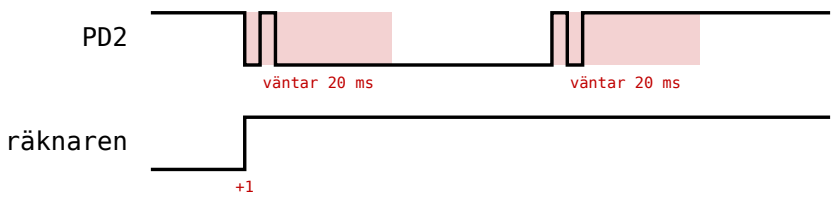
Dioden över spolen är inte valfri. En spole vill hålla sin ström igång. När transistorn slår av blir det en kort spänningstopp på många tiotals volt över spolen, och utan dioden går den genom transistorn och förstör den. Dioden ger strömmen en väg runt spolen tills den har klingat av.

I praktiken köper man ofta ett färdigt **reläkort** för Arduino, där transistor, diod och relä redan sitter monterade, och kopplar det med tre sladdar: 5 V, jord och en signal från stiftet.

> **Koppla aldrig 24 V till Arduino-kortet eller kopplingsdäcket.** Kortet tål högst 5,5 V på sina stift. 24 V-kretsen hör hemma på reläkontaktens sida, och bara där.

11.1.4 Knappar som studsar

En knapp i simulatorn går från 1 till 0 i ett enda ögonblick. En riktig knapp gör inte det. När två metallkontakter slår ihop studsar de isär och ihop igen, flera gånger, under några millisekunder, och samma sak händer när knappen släpps. Stiftet rapporterar troget varje studs:




```
En knapp studsar några millisekunder när den sluts och när den öppnas.
Programmet räknar vid första nollan och tittar sedan inte på knappen på
20 ms. Utan väntan hade det här trycket räknats tre gånger.
```

Figur 11.3 En knapp som studsar när den trycks och när den släpps, och programmet som väntar 20 ms efter varje ändring.

För ett program som tänder en lampa så länge knappen hålls ned spelar det ingen roll: lampan flimrar under några millisekunder, vilket ingen ser. Men för ett program som **räknar tryck**, eller växlar något för varje tryck, är varje studs ett nytt tryck.

Programmet `press_counter.asm` räknar tryck på knappen på PD2 och visar antalet binärt på PB0–PB3. I simulatorn prövades det med två tryck, där kontakten studsar en gång både när den sluts och när den släpps, så att stiftet går 1-0-1-0 vid varje tryck och 0-1-0-1 vid varje släpp:

- **med** väntan efter varje ändring räknar programmet **2**;
- **utan** väntan räknar samma program **6**, tre för varje tryck.

Lösningen är att låta kontakten sätta sig. Efter varje ändring som programmet reagerar på väntar det **20 ms**, längre än knappen studsar men kortare än ett mänskligt tryck, innan det tittar på knappen igen:

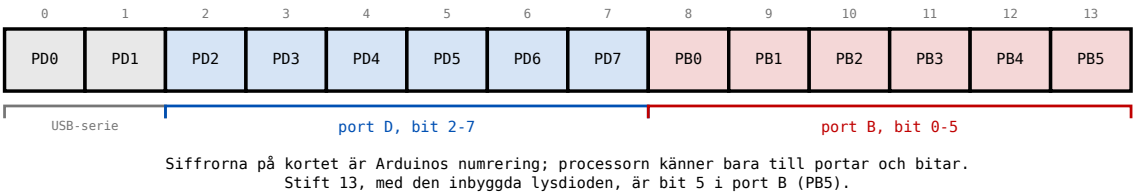
```
loop:
wait_press:
    sbic PIND, BUTTON               ; Skip the jump when the button is pressed (0).
    rjmp wait_press
    inc r17                         ; One more press.
    andi r17, 0x0F                  ; Four LEDs: count 0-15 and round again.
    out PORTB, r17
    ldi r24, SETTLE_MS              ; Let the contact settle.
    rcall delay_ms

wait_release:
    sbis PIND, BUTTON               ; Skip the jump when the button is released ↩
    ↪ (1).
    rjmp wait_release
    ldi r24, SETTLE_MS              ; Let the contact settle again.
    rcall delay_ms
    rjmp loop
```

Det kallas **avstudsning**. Programmet räknar i samma ögonblick som den första nollan kommer, så användaren märker ingen fördröjning. Det som försvinner är bara studsarna efter.

11.1.5 Arduino Uno-kortet

Kortet som används i Labb 3 är ett **Arduino Uno**, med en ATmega328P som klockas med 16 MHz. Kortet gör tre saker åt processorn: det ger den ström, via USB-kabeln; det har en USB-krets som gör att den kan programmeras från datorn; och det för ut processorns stift till hylslister med numrerade stift.



Figur 11.4 Arduino-stift 0–13 och den port och bit varje stift är.

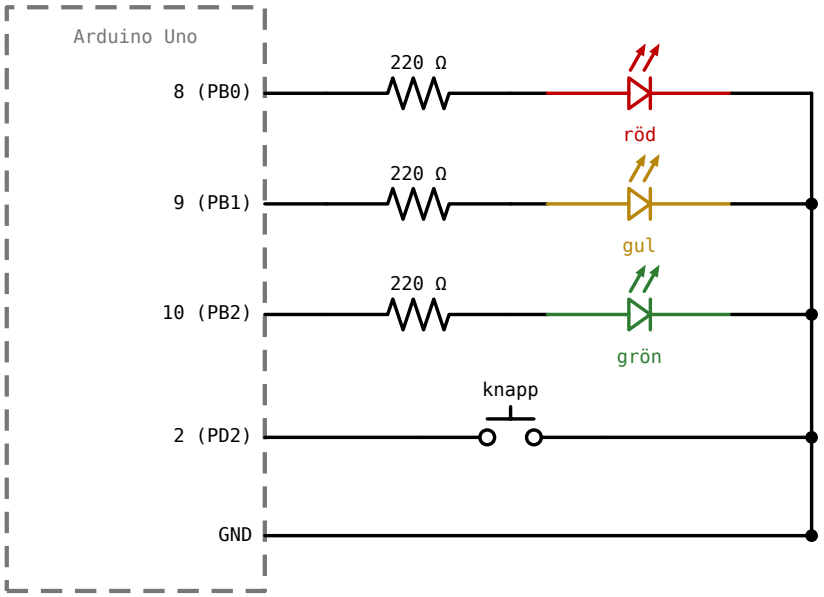
Att tänka på:

- **Stift 0 och 1** (PD0 och PD1) är kopplade till USB-kretsen, och används när kortet programmeras. Lämna dem lediga.

- **Stift 13** (PB5) har redan en lysdiod med motstånd på kortet. Den är användbar som signal: ett program kan tända den för att visa att det har kommit till en viss punkt.
- **5V** och **GND** på kortet ger matning och jord till kopplingsdäcket. Använd kortets GND som gemensam jord för alla lysdioder och knappar.
- PB6 och PB7 syns i simulatorn men sitter inte på något stift; på kortet används de av klockan.

En knapp kopplas mellan ett stift och GND, och stiftets inbyggda pull-up håller det högt när knappen är släppt, precis som i avsnitt 10.2.2 och figur 10.4.

Kopplingen till trafikljuset i slutuppgiften:



Varje lysdiod har ett eget motstånd på 220-330 Ω och går till jord (GND). Knappen kopplar stift 2 till jord; den interna pull-upen håller stiftet högt när knappen är släppt. Stift 0 och 1 används av USB-kabeln och lämnas lediga.

Figur 11.5 Kopplingen till trafikljuset: tre lysdioder på stift 8, 9 och 10, och en knapp på stift 2.

11.1.6 Från simulatorn till kortet

Hur programmet förs över beskrivs i avsnitt D.5 i bilaga D. Det är en engångsinställning; därefter är det ett menyval efter varje bygge.

Tre saker skiljer kortet från simulatorn, och alla tre ger fel som inte fanns där:

- **Tiden är verklig.** En sekund i programmet är en sekund på kortet. I simulatorn tar en sekund mycket längre tid att köra, och därför används konstanten `QUARTER_MS` i slutuppgiften: sätt den till 1 i simulatorn och tillbaka till 250 innan programmet förs över.
- **Knappar studsar**, som i avsnitt 11.1.4.
- **Kopplingen kan vara fel.** En lysdiod som är vänd fel, en sladd i fel rad på kopplingsdäcket, eller ett motstånd som inte har kontakt, ser ut precis som ett programfel.

När något inte fungerar på kortet, gör i den här ordningen:

1. **Fungerar programmet i simulatorn?** Om inte, är det ett programfel, och kortet är oskyldigt.
2. **Är programmet verkligen överfört?** Läs utskriften från `avrdude`: står det att bytena har skrivits?
3. **Stämmer kopplingen?** Följ varje ledning från stiftet till jord, och kontrollera lysdiodernas riktning. Byt plats på två lysdioder: flyttar felet med dem, sitter felet i kopplingen.
4. **Kommer programmet dit du tror?** Tänd lysdioden på stift 13 på det ställe i programmet du vill veta om det når.

11.1.7 Hur noggrann är klockan?

Fördröjningarna i kursen är räknade på cykeln när. En sekund med `delay_s` är 16 000 102 cykler, och simulatorn mäter exakt det. Men en cykel är bara 62,5 ns om klockan verkligen går på 16 MHz.

På ett Arduino Uno klockas ATmega328P av en keramisk **resonator**, som typiskt är noggrann på ungefär $\pm 0{,}5$ %. Ett program som räknar till exakt en sekund kan alltså i verkligheten ta mellan 0,995 och 1,005 s, och under en timme kan det skilja 18 sekunder. Felet i själva programmet, några mikrosekunder per sekund, är nästan tusen gånger mindre.

Det är en viktig skillnad att hålla isär: **programmet** kan vara exakt, men **tiden** blir aldrig noggrannare än klockan som driver det. För ett trafikljus spelar en halv procent ingen roll. För en klocka som ska visa rätt tid efter en månad gör det det, och då används en noggrannare kristall eller en separat klockkrets.

11.2 Genomarbetat exempel: från flödesplan till program

11.2.1 Uppgiften: vägarbetsljuset

Vid ett vägarbete är vägen smal nog för ett körfält, och trafiken från de två hållen turas om. Vid varje ände står ett ljus med en röd och en grön lampa. Det här exemplet programmerar det ena ljuset: **rött i 4 sekunder, grönt i 4 sekunder, och om igen, för alltid.**

Exemplet är medvetet enklare än trafikljuset i slutuppgiften: två lampor och två tillstånd i stället för tre lampor och fyra. Metoden är densamma, och det är metoden avsnittet handlar om:

1. Skriv upp **tillstånden**, vilka lampor som lyser i vart och ett, och hur länge.

2. Rita **flödesplanen**.
3. Välj **stift och bitmönster**.
4. Skriv **programmet**, ett tillstånd i taget, med subrutiner för det som upprepas.
5. **Testa** i simulatorn, och sedan på kortet.

Lamporna är lysdioder på kortet: röd på stift 8 (PB0) och grön på stift 10 (PB2), kopplade som i slutuppgiften.

11.2.2 Tillstånden

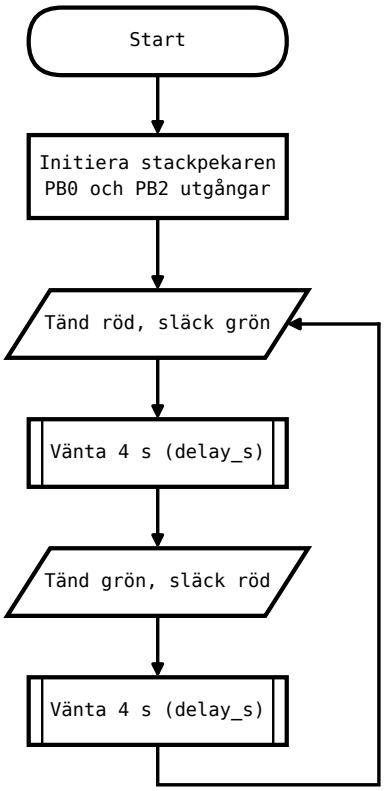
Ett styrobjekt som går igenom samma steg om och om igen beskrivs bäst som en följd av **tillstånd**. Ett tillstånd säger vilka utgångar som är på, och hur länge systemet stannar där:

Tillstånd	Röd (PB0)	Grön (PB2)	Tid	Därefter
Stopp	tänd	släckt	4 s	Kör
Kör	släckt	tänd	4 s	Stopp

Det är samma idé som tillståndsföljden för en räknare i kapitel 6: systemet är i ett tillstånd i taget, och går till nästa vid en bestämd händelse. För en räknare var händelsen en klockflank. Här är det att en viss tid har gått.

Tabellen är värd att skriva även när uppgiften är så här liten. Den är specifikationen, och det färdiga programmet ska kunna kontrolleras mot den rad för rad.

11.2.3 Flödesplanen




```
Vägarbetsljuset: två tillstånd som avlöser varandra för
      alltid. Loopen tillbaka har inget villkor.
```

Figur 11.6 Flödesplanen för vägarbetsljuset.

Varje tillstånd blir två symboler: en utmatning som tänder rätt lampor, och ett subrutinanrop som väntar. Loopen tillbaka har inget villkor, eftersom ljuset aldrig slutar. Det enda som händer före loopen är initieringen: stackpekaren, eftersom programmet anropar subrutiner, och de två stiften som utgångar.

11.2.4 Subrutinen `delay_s`

Programmet behöver vänta hela sekunder, men `delay_ms` från avsnitt 10.3.4 räcker bara till 255 ms. En ny subrutin, `delay_s`, tar antalet sekunder i `r24` och anropar `delay_ms` med 250 ms fyra gånger per sekund:

```
;-------------------------------------------------------------------------------
; delay_s: Waits r24 seconds (1-63), as 4 x r24 calls of delay_ms with 250 ms each.
; A call takes 16 000 084 x r24 + 18 cycles: about 5 microseconds too long per ↪
    ↪ second.
;-------------------------------------------------------------------------------
delay_s:
    push r24                        ; Save the registers this subroutine changes.
    push r25
    mov r25, r24
    lsl r25                         ; r25 = 2 x r24 ...
    lsl r25                         ; ... = 4 x r24: the number of quarter seconds.
    ldi r24, QUARTER_MS             ; A quarter of a second is 250 ms.
delay_s_loop:
    rcall delay_ms
    dec r25
    brne delay_s_loop
    pop r25                         ; Restore them, in the opposite order.
    pop r24
    ret
```

Tre saker i den är värda att lägga märke till:

- **`lsl` två gånger multiplicerar med fyra**, som i kapitel 8: varje skift åt vänster dubblar talet. Tre sekunder blir tolv kvartar.
- **Den följer kursens regel.** Den ändrar `r24` och `r25`, och sparar båda. Efter `ldi r24, 4` och `rcall delay_s` är `r24` fortfarande 4, och kan användas igen.
- **Den fungerar upp till 63 sekunder.** Fyra gånger 64 är 256, som inte får plats i `r25`.

Konstanten `QUARTER_MS` står överst i programmet och är normalt 250:

```
.equ QUARTER_MS = 250               ; 250 ms. Set it to 1 to run about 250 times
                                    ; faster in the simulator; set it back before
                                    ; flashing.
```

Den finns för simulatorns skull, se avsnitt 11.2.6.

Hur exakt är den? Varje kvart är ett anrop av `delay_ms` med 250, alltså `16 000 · 250 + 18` cykler, plus 3 cykler för `dec` och `brne` i loopen. Fyra kvartar blir 16 000 084 cykler, och hela anropet `16 000 084 · r24 + 18` cykler. En sekund blir alltså ungefär 5 µs för lång. Det är långt mindre än felet i kortets klocka (avsnitt 11.1.7).

11.2.5 Programmet, rad för rad

Hela programmet finns i `roadwork_light.asm`. Konstanter och initiering först:

```
.equ RED = 0                        ; PB0, Arduino pin 8.
.equ GREEN = 2                      ; PB2, Arduino pin 10.

main:
    ldi r16, high(RAMEND)           ; SP = RAMEND = 0x08FF.
    out SPH, r16
    ldi r16, low(RAMEND)
    out SPL, r16

    ldi r16, (1 << RED) | (1 << GREEN)
    out DDRB, r16                   ; The two lights are outputs.
```

`(1 << RED)` är ett uttryck som assemblern räknar ut: en etta skiftad `RED` steg åt vänster, alltså `0b00000001`, och `(1 << GREEN)` blir `0b00000100`. `|` är OR, bit för bit, så båda tillsammans blir `0b00000101`. Skrivsättet gör att raden säger *vilka* lampor det gäller, i stället för bara ett tal, och om grön flyttas till ett annat stift behöver bara `.equ`-raden ändras.

Sedan de två tillstånden, i en loop:

```
loop:
    ldi r16, (1 << RED)             ; Red, 4 s.
    out PORTB, r16
    ldi r24, 4
    rcall delay_s

    ldi r16, (1 << GREEN)           ; Green, 4 s.
    out PORTB, r16
    ldi r24, 4
    rcall delay_s

    rjmp loop                       ; And round again.
```

Varje tillstånd är fyra rader: bitmönstret, ut på porten, tiden, vänta. Programmet läses som tabellen i avsnitt 11.2.2, rad för rad, och det är ingen slump: tabellen skrevs först.

Lägg märke till att `out PORTB, r16` skriver **hela** porten. Den tänder en lampa och släcker den andra i samma instruktion, och det är därför bitmönstret för ”Kör” är `0b00000100` och inte bara ”tänd grön”. Med `sbi` och `cbi` hade samma sak krävt två instruktioner per tillstånd.

11.2.6 Testa i simulatorn

Ett tillstånd på 4 sekunder är 64 miljoner cykler, och simulatorn behöver en lång stund för att köra dem. Därför finns `QUARTER_MS`: sätt den till **1** medan programmet testas i simulatorn. Då tar varje ”sekund” ungefär 4 ms, och hela cykeln syns på ett ögonblick, i samma ordning och med samma förhållande mellan tiderna.

Så testar du:

1. Sätt `QUARTER_MS` till 1 och bygg.
2. Sätt en brytpunkt på varje `out PORTB, r16`.
3. Kör med **Continue** (**F5**). Vid varje stopp: stega ett steg med **F11** och kontrollera i I/O-fönstret att rätt bitar i `PORTB` är ettor.
4. Nollställ stoppuret vid ett stopp och kör till nästa. Med `QUARTER_MS` = 1 ska två stopp i rad ligga ungefär 16 ms isär, 4 ”sekunder” gånger 4 ms.

I kursens kontroll av programmet, med `QUARTER_MS` = 250, tar en hel cykel **128 000 716** cykler, 8,0000448 s. Med `QUARTER_MS` = 1 tar den 512 716 cykler, 32 ms: ungefär 250 gånger fortare.

11.2.7 Till kortet

1. **Sätt tillbaka `QUARTER_MS` till 250.** Det är det vanligaste felet i det här steget. Med 1 går hela cykeln runt ungefär 30 gånger i sekunden, och båda lamporna ser ut att lysa svagt på en gång.
2. Koppla lysdioderna enligt avsnitt 11.1.5, med USB-kabeln urkopplad.
3. Bygg, anslut kortet och för över programmet enligt avsnitt D.5 i bilaga D.
4. Ta tid på några cykler och jämför med tabellen.

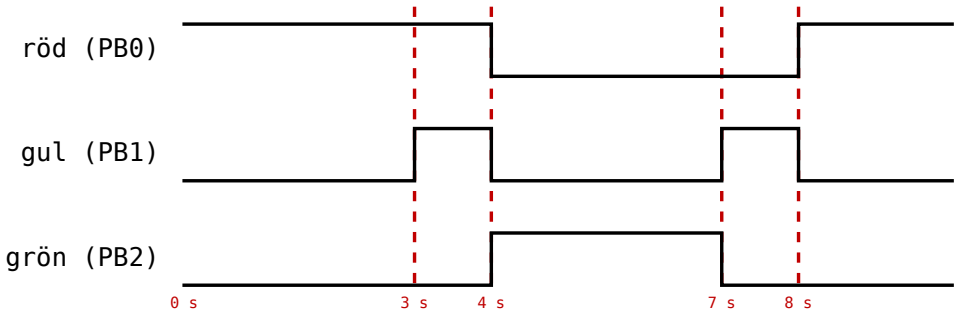
Om lamporna gör något annat än i simulatorn, följ felsökningsordningen i avsnitt 11.1.6.

11.2.8 Från vägarbetsljus till trafikljus

Slutuppgiftens del A, trafikljuset i bilaga C, använder precis samma metod med fler tillstånd:

- **Tabellen** får fyra rader i stället för två, och en tredje lampa, gul på PB1.
- **Flödesplanen** får fyra par av symboler, ett per tillstånd.
- **Bitmönstren** blir fyra, och ett av dem har två lampor tända samtidigt.
- **Subrutinerna** är desamma, oförändrade.

Ljuscykeln som del A ska ge, ritad som ett tidsdiagram:



Röd 3 s, röd och gul 1 s, grön 3 s, gul 1 s. Efter 8 s börjar cykeln om från början.

Figur 11.7 Tidsdiagram för trafikljuset i slutuppgiftens del A: en cykel på 8 s.

Del B, övergångsstället, lägger till något nytt: ett **villkor**. Ljuset står still i ett tillstånd tills knappen trycks, och det blir en beslutssymbol i flödesplanen och en väntan med `sbic` i programmet (avsnitt 10.2.5).

11.3 Sammanfattning

Det här ska du kunna nu

Efter det här kapitlet ska du kunna:

- Beskriva ett styrsystem som ingångar, beslut och utgångar, och peka ut dem i ett exempel.
- Räkna ut förkopplingsmotståndet för en lysdiod, och förklara varför det behövs.
- Förklara hur en transistor och ett relä låter mikrodatorn styra en last på 24 V, och vad dioden över spolen gör.
- Förklara varför en knapp studsar, när det spelar roll, och hur en kort väntan löser det.
- Gå från en tillståndstabell till en flödesplan och ett program, och testa programmet först i simulatorn och sedan på ett riktigt kort.
- Felsöka ett program på kortet: skilja ett programfel från ett kopplingsfel.

Frågor att testa dig själv med

1. Varför får en lysdiod aldrig kopplas direkt mellan ett stift och jord?
2. Ett stift tål 40 mA. Varför styrs ett relä ändå genom en transistor?
3. Ett program som räknar knapptryck räknar för många, men bara på kortet, aldrig i simulatorn. Varför?
4. Vad händer om `QUARTER_MS` står kvar på 1 när programmet förs över till kortet?
5. Programmet räknar en sekund exakt på cykeln. Varför kan trafikljuset ändå gå fel med en halv procent?
6. Varför skriver `out PORTB, r16` hela porten på en gång, och varför är det en fördel i ett trafikljus?

Nästa kapitel

Kapitel 12 förbereder det skriftliga provet:

- Repetition av hela kursen, lärandemål för lärandemål.
- Det skriftliga provet: två timmar, utan miniräknare, med referensbladet för AVR-instruktionerna (bilaga E).

Förbered provet med övningsprovet i bilaga G.

11.4 Övningar

Så kontrollerar du ditt arbete. Övningarna märkta **Program** skrivs i Microchip Studio och testas först i simulatorn och sedan, om ni har kortet, på ett Arduino Uno. Övning 11.9, kontrollen, kräver kortet och en multimeter, och görs lämpligen i samband med Labb 3.
Övningarna märkta **Räkna för hand**, **Förståelse** och **Konstruktion** görs på papper, utan miniräknare, och kontrolleras mot bilaga I. Gör varje övning innan du läser lösningen.
Varje övning är märkt med sin sort. Exakt en övning är en **Kontroll**, och i det här kapitlet är det övning 11.9.

Övning 11.1 Ingångar, beslut och utgångar FÖRSTÅELSE

Beskriv för vart och ett av styrsystemen nedan vilka **ingångar** det har, vilka **utgångar**, och vilket **beslut** det fattar, som i avsnitt 11.1.1.

a) En garageport med en knapp, en motor som kan köra upp och ned, och två gränslägesbrytare, en som känner att porten är helt öppen och en att den är helt stängd.

b) En handtork som blåser när en IR-givare ser en hand, och slutar tio sekunder efter att handen försvunnit.

c) Övergångsstället i slutuppgiften, del B i Labb 3.

Övning 11.2 Förkopplingsmotståndet

RÄKNA FÖR HAND

Stiftet ger 5 V. Räkna med 2 V över en röd lysdiod och 3 V över en grön.

a) Hur stor blir strömmen genom en röd lysdiod med 220 Ω, 330 Ω och 1 kΩ?

b) Hur stor blir strömmen genom en grön lysdiod med 220 Ω? Varför lyser den kanske svagare än den röda med samma motstånd?

c) Vilket motstånd behövs för 10 mA genom en röd lysdiod?

d) Någon tar ett motstånd på 47 Ω. Hur stor blir strömmen, och vad säger databladets gräns om det?

Övning 11.3 Hela porten

RÄKNA FÖR HAND

Sex röda lysdioder sitter på PB0–PB5, var och en med 220 Ω, och alla lyser samtidigt.

a) Hur stor är den sammanlagda strömmen ut ur porten?

b) Jämför med gränsen på ungefär 100 mA för en grupp av stift, från avsnitt 11.1.2. Hur mycket marginal finns det?

c) Varför är 330 Ω ett säkrare val än 220 Ω om många lysdioder kan lysa samtidigt?

Övning 11.4 Reläet

FÖRSTÅELSE

Titta på kopplingen i avsnitt 11.1.3.

a) Varför kopplas reläspolen inte direkt till stiftet?

b) Vad gör dioden, och vad händer om den saknas?

c) Mikrodatorn och 24 V-kretsen har ingen gemensam ledning. Varför är det en fördel?

d) Någon kopplar lampan på 24 V direkt mellan stiftet och jord, utan relä. Vad händer, med lampan och med kortet?

Övning 11.5 Knappar som studsar

FÖRSTÅELSE

a) Förklara med egna ord varför programmet i avsnitt 11.1.4 räknar 6 i stället för 2 när väntan tas bort.

b) Varför väntar programmet 20 ms, och inte 1 ms eller 500 ms? Vad går fel med var och en av de två andra?

c) Lysdioden i `led_follows_button.asm` i kapitel 10 (avsnitt 10.2.4) tänds så länge knappen hålls ned, och har ingen avstudsning alls. Varför behövs den inte där?

Övning 11.6 Hur lång tid tar ett varv?

RÄKNA FÖR HAND

Använd formeln för `delay_s` i avsnitt 11.2.4.

a) Hur många cykler tar ett anrop av `delay_s` med `r24` = 4?

b) Vägarbetsljusets loop har, utöver de två anropen, tre instruktioner per tillstånd (`ldi`, `out`, `ldi`) och ett `rjmp`. Hur många cykler tar ett helt varv? Jämför med de 128 000 716 cykler som kursens kontroll mätte.

c) Hur mycket för långt blir ett varv, i mikrosekunder? Hur mycket blir det på en timme?

d) Jämför med resonatorns noggrannhet i avsnitt 11.1.7. Vilken av de två felkällorna är värd att bry sig om?

Övning 11.7 Nattläge

KONSTRUKTION

På natten stängs trafikljus ofta av, och bara den gula lampan blinkar: en halv sekund tänd, en halv sekund släckt.

a) Skriv tillståndstabellen, som i avsnitt 11.2.2.

b) Rita flödesplanen.

c) Skriv programmet, med den gula lysdioden på PB1 och subrutinen `delay_ms`. Kan `r24` laddas en enda gång, före loopen? Varför, eller varför inte?

d) Hur lång blir en hel blinkperiod, räknat i cykler?

Övning 11.8 En knapp som växlar

PROGRAM

Skriv ett program där varje tryck på knappen på PD2 **växlar** lysdioden på PB5: ett tryck tänder, nästa släcker, och så vidare. Använd avstudsning som i avsnitt 11.1.4.

a) Rita flödesplanen.

b) Skriv programmet och testa det i simulatorn.

c) Vad händer på kortet om väntan efter trycket tas bort? Prova, om ni har kortet.

Övning 11.9 Kontroll: strömmen genom lysdioden

KONTROLL

Räkna för hand, mät på kortet, förklara skillnaden.

En röd lysdiod med 220 Ω sitter på stift 8 (PB0), och ett program tänder den.

a) För hand

Räkna ut strömmen genom lysdioden, med 2 V över den.

b)

Mät spänningen över motståndet med en multimeter, när lysdioden lyser. Räkna ut strömmen med Ohms lag. Mät också spänningen mellan stiftet och jord.

c) Jämför
Stämmer strömmarna i a) och b)? Förklara skillnaden med det du mätte: är spänningen på stiftet verkligen 5 V, och är spänningen över lysdioden verkligen 2 V?

d)
Byt till en grön lysdiod och gör om b). Vad ändras, och varför?

Övning 11.10 Felsökning på kortet

FÖRSTÅELSE

Programmet för trafikljuset fungerar i simulatorn. Förklara för vart och ett av symptomen nedan vad det troligaste felet är, och hur du kontrollerar det.

a) Den gröna lampan lyser aldrig. De andra två fungerar.

b) Alla tre lamporna ser ut att lysa svagt, hela tiden.

c) Ingenting händer alls, och utskriften efter överföringen slutar med `ser_open(): can't open device`.

d) Den röda och den gula lampan har bytt plats: gul lyser när röd ska lysa.

Övning 11.11 En dag på ett trafikljus

RÄKNA FÖR HAND FÖRDJUPNING

a) Kortets resonator går 0,5 % för fort. Hur mycket fel har en klocka byggd på programmets fördröjningar efter ett dygn?

b) Hur många hela ljuscykler, om 8 s var, hinner trafikljuset i slutuppgiften med på ett dygn? Räkna med exakt 8 s.

KAPITEL 12

Repetition inför det skriftliga provet

Hela kursen en gång till, ordnad efter det du ska kunna när den är slut.

I DET HÄR KAPITLET

- Repetition av kursen, lärandemål för lärandemål: det viktigaste, en lista att bocka av och de vanligaste felen.
- Tips inför det skriftliga provet, som skrivs under två timmar, individuellt och utan miniräknare.
- Repetitionsövningar i provets stil, från talomvandlingar till stacken.

Det här kapitlet tar inte upp något nytt. Det går igenom kursen en gång till, i den ordning kursplanens lärandemål står, så att du kan se vad du kan och vad du behöver läsa om före det skriftliga provet. Provets form, poäng och betygsgränser står i bilaga F.

Börja med övningsprovet i bilaga G. Gör det under två timmar, utan miniräknare och med referensbladet i bilaga E bredvid, och rätta det sedan mot lösningsförslagen i bilaga H. Gå därefter igenom avsnitt 12.1 och bocka av listorna ”Kan du …?”. Läs om det du inte kan, i det kapitel avsnittet pekar på. Övningarna i avsnitt 12.3 är i provets stil, och deras lösningsförslag finns i bilaga I.

Skriv upp det som fortfarande är oklart, och fråga läraren om det. En fråga som ställs före provet är värd mer än en som ställs efter.

12.1 Repetition

12.1.1 Så används det här avsnittet

Kursen har gått fort: fyra ämnen på elva kapitel. Det här avsnittet går igenom dem igen, ordnat efter kursplanens lärandemål, så att du kan se vad du kan och vad du behöver läsa om.

Varje avsnitt har tre delar:

- **Det viktigaste**, kort, med de metoder som provet bygger på.
- **Kan du ...?**, en lista att bocka av. Svara på varje fråga på papper, utan att titta, och kontrollera sedan mot avsnittet som hänvisningen pekar på.
- **Vanliga fel**, de misstag som oftast kostar poäng.

Provets form, poäng och betygsgränser står i bilaga F. Där finns också ett övningsprov, i bilaga G, med lösningsförslag i bilaga H. Gör det under två timmar, utan miniräknare, med referensbladet i bilaga E bredvid, precis som på det riktiga provet.

12.1.2 Talsystem och binära koder

Lärandemål: beskriva binära, hexadecimala och decimala talsystem, och omvandla talvärden mellan dem. (Kapitel 1.)

Det viktigaste

- I ett **positionssystem** är varje siffra värd siffran gånger talbasen upphöjd till positionen, räknat från 0 längst till höger. Binärt har basen 2, hexadecimalt 16.
- **Binärt till decimalt**: addera vikterna för ettorna. `1010 1110` = 128 + 32 + 8 + 4 + 2 = 174.
- **Decimalt till binärt**: dra av den största vikt som får plats, om och om igen, eller dela med 2 och läs resterna nedifrån (avsnitt 1.1.4).
- **Binärt och hexadecimalt**: fyra bitar är en hexadecimal siffra. `1011 0110` = `0xB6`. Omvandla alltid mellan decimalt och hexadecimalt **via binärt** om du är osäker (avsnitt 1.1.6).
- *n* **bitar** räcker till `2^n` värden, 0 till `2^n - 1`: en byte till 0–255.
- **Binär addition** görs som för hand, med minnessiffra. En summa som inte får plats i åtta bitar lämnar en minnessiffra, carry (avsnitt 1.1.7).
- **Koder**: BCD skriver varje decimal siffra med fyra bitar, Graykod ändrar en bit i taget, och ASCII ger varje tecken ett tal: `'0'` = `0x30`, `'A'` = `0x41` (avsnitt 1.2).

Kan du ...?

- ... omvandla 1100 1011, 142 och `0x5E` till de två andra talsystemen, utan miniräknare?
- ... förklara varför en hexadecimal siffra motsvarar exakt fyra bitar?
- ... addera två 8-bitars tal binärt och säga om det blir en minnessiffra?
- ... skriva 59 i BCD, och säga varför BCD inte är samma sak som 59 binärt?
- ... ange ASCII-koderna för tecknen `'4'` och `'C'`?

Vanliga fel

- Att räkna positionerna från 1 i stället för från 0, så att varje vikt blir dubbelt för stor.
- Att gruppera bitar i fyror från vänster. Grupperna räknas **från höger**: `1 0110` är `0x16`.
- Att tro att BCD för 59 är 59 binärt. BCD är `0101 1001`, binärt är `0011 1011`.

12.1.3 Grindar, boolesk algebra och kontaktnät

Lärandemål: förklara grindar och logiska funktioner; beskriva logisk algebra och förenklingar av booleska uttryck. (Kapitel 2 och 3.)

Det viktigaste

- Sju grindar: NOT, AND, OR, NAND, NOR, XOR och XNOR. NAND, NOR och XNOR är inverserna av AND, OR och XOR, och XOR är 1 när ingångarna är olika.
- **Notation**: `A'` för NOT, `AB` för AND, `A + B` för OR. NOT binder hårdast, sedan AND, sist OR.
- **Från sanningstabell till uttryck**: en AND-term per etta, med varje variabel direkt eller inverterad; OR:a ihop termerna. Det ger ett uttryck på **SP-form** (avsnitt 2.1.6).
- **Förenkla** genom att bryta ut det gemensamma och använda `A + A' = 1`. Tre lagar räcker långt: `A + AB = A`, `A + A'B = A + B` och De Morgan (avsnitt 2.1.4).
- **De Morgan**: `(A + B)' = A'B'` och `(AB)' = A' + B'`. Bryt strecket, byt tecknet (avsnitt 2.1.5).
- **Analys av ett grindnät**: skriv uttrycket vid varje grinds utgång, från ingångarna mot utgången, och räkna sedan ut sanningstabellen rad för rad (avsnitt 2.1.8).
- **Kontaktnät**: kontakter i serie är AND, parallellt OR, och en brytande kontakt är NOT. Lampan lyser när det finns en sluten väg från +24 V till 0 V (avsnitt 2.2 och avsnitt 3.1).

Kan du ...?

- ... rita symbolen och skriva sanningstabellen för var och en av de sju grindarna?
- ... läsa av ett uttryck på SP-form ur en sanningstabell med tre variabler?
- ... förenkla `AB + AB' + A'B` till två variabler, och säga vilken lag varje steg använder?
- ... skriva `(A + B)'` utan parentes, och bygga OR av enbart NAND-grindar?
- ... skriva uttrycket och sanningstabellen för ett grindnät eller ett kontaktnät du inte sett förut?
- ... rita ett kontaktnät för `S1(S2 + S3')`?

Vanliga fel

- Att glömma att NOT binder hårdare än AND: `AB'` är `A(B')`, inte `(AB)'`.
- Att använda De Morgan halvt: invertera variablerna men glömma att byta AND mot OR.
- Att tro att en brytande kontakt är öppen: den är **sluten** tills knappen trycks.

12.1.4 Karnaughdiagram och IC-kretsar

Lärandemål: använda Karnaughdiagram för analys och minimering av booleska uttryck; syntetisera digitala kombinatoriska nät med IC-kretsar och kontakter; analysera digitala kombinatoriska nät. (Kapitel 4 och 5.)

Det viktigaste

- Ett **Karnaughdiagram** är sanningstabellen ritad så att grannrutor skiljer sig i exakt en variabel. Raderna och kolumnerna står i **Graykod**: 00, 01, 11, 10.
- **Gruppera** ettorna i rektanglar om 1, 2, 4 eller 8 rutor, så stora som möjligt och så få som möjligt. Grupper får överlappa och gå runt kanterna; de fyra hörnen är grannar (avsnitt 4.1.3 och 4.1.5).
- **Läs av** varje grupp som de variabler som är konstanta inom den. En grupp om fyra i ett diagram med fyra variabler ger en term med två variabler.
- **Don't care**, X, får räknas som 1 om det ger en större grupp, annars som 0.
- **NAND-nät**: ett uttryck på SP-form blir två nivåer NAND-grindar, eftersom `AB + CD` = `((AB)'(CD)')'`.
- **74-serien**: fyra grindar per kapsel (sex inverterare i 74HC04), VCC på ben 14 och GND på ben 7. Oanvända ingångar får aldrig flyta (avsnitt 4.2).
- **Felsökning**: matningen på varje krets först, sedan ingångarna, sedan signalen grind för grind (avsnitt 5.1).

Kan du ...?

- ... rita ett tomt Karnaughdiagram för fyra variabler, med Graykoden rätt på båda axlarna?
- ... fylla i det från en lista med mintermer, och gruppera det minimalt?
- ... förklara varför hörnen i ett diagram med fyra variabler kan bilda en grupp?
- ... räkna ut hur många 74HC-kretsar ett nät behöver?
- ... förklara vad som händer med en flytande ingång på en 74HC-krets?

Vanliga fel

- Att skriva axlarna i vanlig binär ordning, 00, 01, 10, 11. Då är grannrutorna inte grannar.
- Att göra en grupp om tre, eller en grupp som inte är en rektangel.
- Att behålla en grupp vars alla ettor redan täcks av andra grupper.

12.1.5 Mikrodatorns uppbyggnad

Lärandemål: redogöra för mikrodatorns uppbyggnad på blockschemanivå. (Kapitel 6 och 7.)

Det viktigaste

- En **mikrodator** består av en **CPU**, **minne** och **in- och utportar**, sammankopplade med **bussar** och drivna av en **klocka** (avsnitt 6.2.2).
- **CPU:n** har en **ALU** som räknar, **register** som håller värdena den räknar med, och en **styrenhet** som avkodar instruktionerna och styr resten.
- **Programräknaren**, PC, pekar på nästa instruktion. Instruktionscykeln är **hämta, avkoda, utföra**, en gång per instruktion (avsnitt 6.2.4).

- ATmega328P har **tre minnen**: programmet i flash, som finns kvar utan ström; variablerna och stacken i SRAM, som töms när strömmen bryts; och EEPROM (avsnitt 7.1.5).
- Varje block är byggt av det kursen redan har gått igenom: ALU:n av grindar, register och PC av vippor, minnet av tusentals lås med en adressavkodare (avsnitt 6.2.10).

Kan du ...?

- ... rita mikrodatorns blockschema och säga vad varje block gör?
- ... beskriva vad som händer i hämta, avkoda och utföra för en instruktion som `inc r16`?
- ... säga vilket minne programmet ligger i, och vilket stacken ligger i, och vilket av dem som finns kvar när strömmen bryts?
- ... förklara vad en buss är, och nämna två?

Vanliga fel

- Att blanda ihop registren med dataminnet. Registren är 32 byte inuti CPU:n; SRAM är 2048 byte utanför den.
- Att tro att programmet ligger i SRAM. Det ligger i flash.

12.1.6 Assemblerprogrammering

Lärandemål: använda assemblerspråk för att programmera en mikrodator. (Kapitel 7–10.)

Det viktigaste

- En rad assembler är en **etikett**, en **instruktion** med **operander**, och en **kommentar**. **Direktiv** som `.include`, `.org`, `.equ` och `.def` blir ingen maskinkod.
- `ldi` laddar en konstant, men bara i `r16`–`r31`. Tal skrivs decimalt, `0b...` eller `0x...`.
- **Aritmetik och flaggor**: `add`, `sub`, `inc`, `dec` och resten skriver flaggorna i SREG. `C` är minnessiffra eller lån, `Z` att resultatet blev noll, `N` att bit 7 är satt (avsnitt 8.1.6). Åtta bitar går runt: 255 + 1 blir 0.
- **Logik**: `and`/`andi` nollställer bitar, `or`/`ori` ettställer, `eor` inverterar, med en mask (avsnitt 8.2.2). `lsl` multiplicerar med 2 och `lsr` delar med 2.
- **Hopp**: `cp`/`cpi` är en subtraktion som bara sparar flaggorna, och det villkorliga hoppet läser dem: `breq`, `brne`, `brlo`, `brsh` (avsnitt 9.1.4).
- **Loopar och fördröjningar**: en loop med `dec` och `brne` tar 3 cykler per varv, utom det sista, som tar 2. Vid 16 MHz är en cykel 62,5 ns (avsnitt 9.2.3).
- **Portar**: `DDRx` väljer riktning, `PORTx` sätter en utgång eller pull-upen på en ingång, `PINx` läser (avsnitt 8.3.2 och avsnitt 10.2.2). En knapp mot jord med pull-up läses som 0 när den är nedtryckt.
- **Subrutiner och stacken**: `rcall` lägger returadressen på stacken och `ret` hämtar den. En subrutin sparar varje register den ändrar med `push`, och återställer dem med `pop` i omvänd ordning (avsnitt 10.3).

Kan du ...?

- ... följa ett kort program instruktion för instruktion och ange registren och flaggorna `C`, `Z` och `N` efter varje rad?
- ... räkna ut hur många cykler en loop tar, och hur lång tid det är vid 16 MHz?
- ... skriva ett program som läser en knapp och tänder en lysdiod?
- ... skriva en jämförelse och ett villkorligt hopp för ”om `r16` är minst 100”?
- ... säga vad `SP` är efter ett `rcall` och två `push`, när den var `0x08FF` från början?

Vanliga fel

- Att tro att den gula pilen i simulatorn visar instruktionen som just kördes. Den visar nästa.
- Att lägga en instruktion som ändrar flaggorna mellan `cpi` och hoppet.
- Att glömma att `inc` och `dec` inte ändrar `C`, och att `andi` inte heller gör det.
- Att räkna en loops sista varv som alla andra. Det sista villkorliga hoppet hoppar inte, och tar en cykel mindre.
- Att glömma `com` när knappar mot jord läses, så att programmet gör precis tvärtom.

12.1.7 Styrobjekt

Lärandemål: programmera mikrodatorn till att kontrollera styrobjekt. (Kapitel 11.)

Det viktigaste

- Ett styrsystem läser **ingångar**, fattar **beslut** och sätter **utgångar**. En tillståndstabell och en flödesplan före koden gör programmet enkelt att skriva och att kontrollera (avsnitt 11.2).
- En lysdiod behöver ett **förkopplingsmotstånd**: `(5 V - 2 V) / 220 Ω ≈ 14 mA`. Större laster styrs genom en transistor och ett relä (avsnitt 11.1).
- En knapp **studsar**; ett program som räknar tryck väntar en kort stund efter varje ändring.

Kan du ...?

- ... räkna ut strömmen genom en lysdiod med ett givet motstånd?
- ... rita flödesplanen för ett trafikljus, och översätta den till ett program?
- ... förklara varför en knapp kan räknas flera gånger, och hur det förhindras?

12.1.8 Tips inför provet

- **Visa hur du räknar.** Metoden ger poäng även när svaret blir fel på grund av ett slarvfel, och ett fel som förs vidare till nästa deluppgift kostar bara en gång.
- **Kontrollera med en sanningstabell** när du har förenklat ett uttryck. Tre variabler är åtta rader och tar en minut.
- **Skriv alltid ut talsystemet**: `0x2C`, `0b0010 1100` eller 44. Ett tal utan skrivsätt kan läsas på tre sätt.

- **Följ ett program med en tabell**: en rad per instruktion och en kolumn per register och flagga.
- **Läs frågan två gånger.** Frågar den efter ett uttryck, en sanningstabell eller ett grindnät? Frågar den efter `C` efter en viss instruktion, eller efter hela programmet?

12.2 Sammanfattning

Det här ska du kunna nu

Kursplanens lärandemål. Efter kursen ska du kunna:

- förklara grindar och logiska funktioner;
- beskriva binära, hexadecimala och decimala talsystem, och omvandla talvärden mellan dem;
- beskriva logisk algebra och förenklingar av booleska uttryck;
- redogöra för mikrodatorns uppbyggnad på blockschemanivå;
- syntetisera digitala kombinatoriska nät med IC-kretsar och kontakter;
- använda Karnaughdiagram för analys och minimering av booleska uttryck;
- använda assemblerspråk för att programmera en mikrodator, och programmera den att styra styrobjekt;
- analysera digitala kombinatoriska nät.

Frågor att testa dig själv med

1. Vilket tal är `0xB6` decimalt, och hur ser det ut binärt?
2. Vad säger De Morgans lagar, och varför kan varje logisk funktion byggas av enbart NAND-grindar?
3. Varför står raderna i ett Karnaughdiagram i ordningen 00, 01, 11, 10?
4. Vad gör programräknaren, och vad händer med den vid ett `rcall`?
5. Efter `ldi r16, 200` och `subi r16, 201`: vad är `r16`, och vad är `C`?
6. En loop med `dec` och `brne` går 100 varv. Hur många cykler tar den, och hur lång tid är det vid 16 MHz?
7. Varför läses en nedtryckt knapp som noll, och vad behövs för att den ska läsas som något alls när den är släppt?

Efter kursen

Kursen har gått från ettor och nollor till en mikrodator som styr ett trafikljus. Varje steg på vägen finns kvar i det du har byggt: grindarna finns i ALU:n, vipporna i registren och programräknaren, kontaktnätens logik i programmets villkorliga hopp.

Vill du vidare finns några naturliga nästa steg:

- **PLC-programmering.** Kontaktnät och stegdiagram från kapitel 2, med samma idé om ingångar, beslut och utgångar som i kapitel 11, i den form industrin använder.
- **C för mikrokontroller.** Samma ATmega328P och samma register, `DDRB`, `PORTB` och `PIND`, men i ett språk där kompilatorn skriver assemblern åt dig. Den som har skrivit assembler vet vad kompilatorn gör.
- **Programmerbar logik.** Grindnät och vippor beskrivna i ett hårdvarubeskrivande språk, och körda på en FPGA.

Spara projekten från Labb 3. Ett program som fungerar är den bästa referensen när nästa program ska skrivas.

12.3 Övningar

Så kontrollerar du ditt arbete. Övningarna är repetition av hela kursen, i samma stil som provets frågor. Gör dem på papper, utan miniräknare, med referensbladet i bilaga E bredvid, och kontrollera mot lösningsförslagen i bilaga I. Övning 12.10, kontrollen, görs först för hand och sedan i Microchip Studio.
Varje övning säger vilken del av avsnitt 12.1 den repeterar. Går en övning dåligt, läs den delen och det kapitel den pekar på innan du går vidare.
Varje övning är märkt med sin sort. Exakt en övning är en **kontrolluppgift**, och i det här kapitlet är det övning 12.10.

Övning 12.1 Talomvandlingar

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.2.

Fyll i tabellen. Visa hur du räknar i minst en omvandling åt varje håll.

Binärt	Decimalt	Hexadecimalt
`1011 0101`		
	200	
		`0x7E`
	64	

Övning 12.2 Binär addition och flaggor

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.2 och avsnitt 12.1.6.

a) Addera `0110 1101` och `0101 1011` binärt, med minnessiffror. Kontrollera svaret decimalt.

b) Additionen görs med `add` i ett 8-bitars register. Blir flaggan `C` satt? Blir `N` satt?

c) Läs resultatet som ett tal **med tecken**, i tvåkomplement. Vilket tal är det? Varför är det ett konstigt svar på en addition av två positiva tal?

Övning 12.3 Koder

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.2.

a) Skriv 47 i BCD.

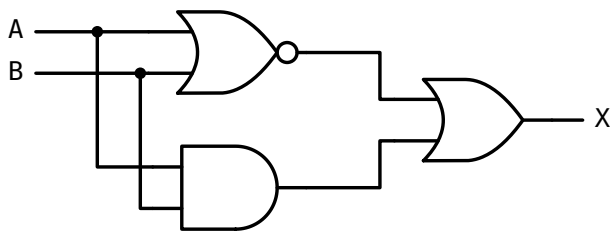
b) Skriv de åtta 3-bitars Graykoderna i ordning, från `000`. Hur många bitar ändras mellan två koder som står bredvid varandra?

c) Vilka ASCII-koder har tecknen i texten `OK`? Bokstäverna ligger i alfabetisk ordning från `'A'` = `0x41`.

Övning 12.4 Vilken grind är det?

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.3.




```
Analysera nätet: skriv uttrycket för X, gör sanningstabellen och
           säg vilken enda grind som gör samma sak.
```

Figur 12.1 Ett grindnät med en NOR-, en AND- och en OR-grind.

a) Skriv uttrycket vid varje grinds utgång, och uttrycket för X.

b) Gör sanningstabellen för X.

c) Vilken enda grind gör samma sak som hela nätet?

Övning 12.5 Förenkla

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.3.

Förenkla algebraiskt, och ange vilken lag varje steg använder. Kontrollera svaret med en sanningstabell.

a) `X = ABC + ABC' + AB'C`

b) `X = (A + B)(A + B')`

Övning 12.6 Två NAND-grindar

KONSTRUKTION

Repeterar avsnitt 12.1.3 och avsnitt 12.1.4.

a) Visa med De Morgan att `X = AB + C'` kan skrivas `X = ((AB)' · C)'`.

b) Rita nätet med enbart NAND-grindar med två ingångar. Hur många behövs?

c) Hur många kapslar 74HC00 behövs, och hur många grindar blir över?

Övning 12.7 Ett kontaktnät

KONSTRUKTION

Repeterar avsnitt 12.1.3.

Lampan H1 ska styras av tre tryckknappar enligt `H1 = S1(S2 + S3')`.

a) Rita kontaktnätet mellan +24 V och 0 V. Vilka kontakter är slutande, och vilka är brytande?

b) Gör sanningstabellen. För vilka kombinationer av nedtryckta knappar lyser lampan?

Övning 12.8 Karnaughdiagram

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.4.

En funktion av A, B, C och D är 1 för mintermerna 0, 1, 4, 5, 10, 11, 14 och 15, och 0 annars.

a) Rita Karnaughdiagrammet, med `AB` på raderna och `CD` på kolumnerna, och fyll i ettorna.

b) Gruppera minimalt, och skriv det minimerade uttrycket.

c) Uttrycket är en känd grind med två av variablerna som ingångar. Vilken?

Övning 12.9 Mikrodatorn

FÖRSTÅELSE

Repeterar avsnitt 12.1.5.

a) Rita mikrodatorns blockschema, med CPU (ALU, register, styrenhet, programräknare), programminne, dataminne, in- och utportar och bussar.

b) Beskriv vad programräknaren gör under hämta-avkoda-utföra för en instruktion.

c) ATmega328P startas om efter ett strömavbrott. Vad finns kvar i flashminnet, i SRAM och i registren?

Övning 12.10 Kontroll: följ programmet

KONTROLL

Repeterar avsnitt 12.1.6. Förutsäg för hand, kontrollera i simulatorn, förklara skillnaden.

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi r16, 0x9C
    ldi r17, 0x64
    add r16, r17
    ldi r18, 5
    sub r18, r17
    mov r19, r18
    lsr r19
    andi r19, 0x0F
end:
    rjmp end
```

a) För hand

Fyll i tabellen med värdena **efter** varje instruktion, hexadecimalt. Fyll i flaggorna på varje rad, men lämna en registerruta tom om registret inte har ändrats sedan raden ovanför.

Instruktion	r16	r17	r18	r19	C	Z	N
`add r16, r17`							
`sub r18, r17`							
`lsr r19`							
`andi r19, 0x0F`							

b) I simulatorn

Skriv in programmet, stega det, och jämför med tabellen efter varje steg.

c) Jämför

Vilka rutor hade du fel i? Den vanligaste avvikelsen gäller `C` efter `andi`: vad visar simulatorn, och varför?

Övning 12.11 En loop

RÄKNA FÖR HAND

Repeterar avsnitt 12.1.6.

```
    ldi r20, 50
loop:
    nop
    dec r20
    brne loop
```

a) Hur många cykler tar koden, från `ldi` till och med den sista `brne`?

b) Hur lång tid är det vid 16 MHz?

c) Vilket värde ska laddas i `r20` för att koden ska ta exakt 25 µs?

Övning 12.12 Om, annars

PROGRAM

Repeterar avsnitt 12.1.6.

Lysdioden på PB0 ska lysa om värdet i `r16` är **minst 100**, och lysdioden på PB1 annars.

a) Rita flödesplanen.

b) Skriv programmet. Vilket villkorligt hopp används, och varför just det?

c) Testa i simulatorn med `r16` = 150, 100 och 99. Vilket av de tre värdena är viktigast att testa, och varför?

Övning 12.13 Stacken

RÄKNA FÖR HAND FÖRDJUPNING

Repeterar avsnitt 12.1.6.

`SP` är `0x08FF`. Programmet kör `rcall work`, och subrutinen `work` börjar med `push r16` och `push r17`.

a) Vad är `SP` efter de två `push`?

b) Vilka byte ligger på adresserna `0x08FF`–`0x08FC`, i ord?

c) Subrutinen slutar med `pop r16`, `pop r17` och `ret`. Programmet återvänder rätt, men något annat är fel. Vad?

BILAGA A

Labb 1: Kopplingsnät

I den här laborationen bygger du logiska funktioner av strömkontakter: två tryckknappar, en lampa och ett aggregat på 24 V. Samma funktioner som i kapitel 2 byggdes av grindar blir här serie- och parallellkopplingar av slutande och brytande kontakter, och lampans läge är utgången.

Laborationen görs i par, och hör till kapitel 3. Teorin finns i avsnitt 2.2 och, för analys och felsökning, i avsnitt 3.1.

A.1 Mål

Efter laborationen ska du kunna:

- avgöra vilka anslutningar på en tryckknapp som är slutande och vilka som är brytande;
- koppla upp ett kontaktnät efter ett strömvägsschema;
- realisera AND, OR, NOT, NAND, NOR, XOR och XNOR med kontakter;
- syntetisera ett kontaktnät ur en sanningstabell, via ett förenklat uttryck;
- analysera ett kontaktnät och ersätta det med ett mindre, likvärdigt nät;
- kontrollera ett nät genom att jämföra förutsagda och uppmätta sanningstabeller.

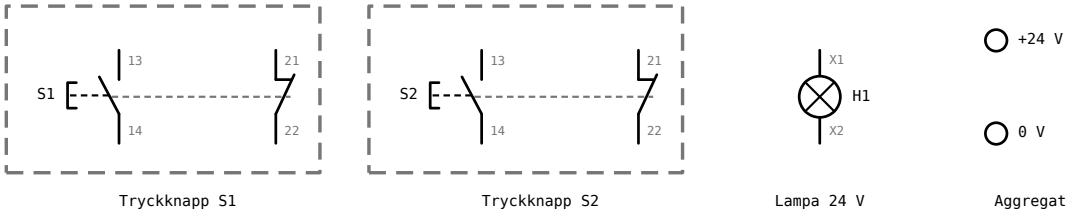
A.2 Utrustning

Varje station har:

- ett **DC-aggregat** inställt på 24 V;
- två **tryckknappar**, S1 och S2, med både slutande (NO) och brytande (NC) kontaktelement;
- en **glödlampa** för 24 V, H1;
- **kopplingssladdar**;
- en **multimeter**, för felsökning.

Varje kontaktelement har två anslutningar, märkta med ett nummer. Enligt standarden slutar numret på **3 och 4 för ett slutande element** och på **1 och 2 för ett brytande**, medan

den första siffran anger elementets plats i knappen. Figur A.1 visar ett exempel; märkningen kan skilja sig mellan stationerna, och uppgift 1 går ut på att ta reda på hur just din station ser ut.



Figur A.1 Stationens delar: tryckknapparna S1 och S2, lampan H1 och aggregatet.

Alla uppgifter i laborationen går att göra med **ett slutande och ett brytande element per knapp**. Har din station fler, använd dem du behöver.

A.3 Säkerhet

- **Koppla alltid med aggregatet frånslaget**, och slå på det först när kopplingen är kontrollerad mot schemat. 24 V är inte farligt att röra, men en kortslutning ger stora strömmar, varma sladdar och ett aggregat som slår ifrån.
- **Varje strömväg ska gå genom lampan.** Koppla aldrig en kontakt direkt mellan +24 V och 0 V: när kontakten sluts blir det en kortslutning.
- Glödlampan blir varm när den har lyst en stund. Rör inte glaset.
- Se också de gemensamma säkerhetsreglerna för laborationerna i förordet.

A.4 Förberedelser

Förberedelseuppgifterna ska vara gjorda **före** passet. Läraren tittar på dem innan ni börjar koppla, och utan dem hinner man inte klart.

F1. Skriv sanningstabellen för H1 med ingångarna S1 och S2 för var och en av funktionerna AND, OR, NAND, NOR, XOR och XNOR, och tabellen för NOT med S1 som enda ingång. En nedtryckt knapp är `1` och en tänd lampa är `1`.

F2. Rita ett strömvägsschema mellan +24 V och 0 V för var och en av funktionerna i F1, med tryckknappar och lampan H1. Ange för varje kontakt om den är slutande eller brytande. Lämna plats att fylla i anslutningarnas nummer när du har gjort uppgift 1.

F3. Läs uppgift 7. Ta fram uttrycket för H1 på SP-form ur sanningstabellen, förenkla det, och rita det minsta kontaktnätet.

F4. Läs uppgift 8. Skriv uttrycket för nätet i figuren, ta fram dess sanningstabell, förenkla uttrycket, och rita det minsta likvärdiga nätet.

A.5 Arbetsgång

Varje uppgift görs på samma sätt, enligt avsnitt 3.1.4:

1. Ta fram schemat från förberedelserna, och fyll i anslutningarnas nummer.
2. Fyll i kolumnen **förutsagt** i uppgiftens tabell.
3. Koppla med aggregatet frånslaget. Den ena i paret kopplar, den andra kontrollerar mot schemat.
4. Slå på aggregatet, pröva varje kombination av knapparna, och fyll i kolumnen **uppmätt**.
5. Stämmer kolumnerna? Om inte: felsök enligt avsnitt 3.1.5.
6. REDOVISA för läraren: schemat, tabellen och ett prov med knapparna.

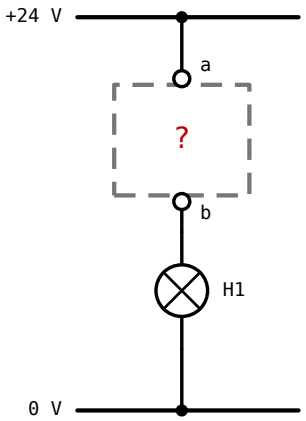
   Byt roll mellan uppgifterna, så att båda i paret har kopplat.

A.6 Uppgifterna

Uppgift 1 Identifiera kontakterna

Innan något kan kopplas måste du veta vilka anslutningar som hör till vilket kontaktelement, och om elementet är slutande eller brytande. Märkningen berättar det, men den ska kontrolleras, eftersom en felaktigt kopplad kontakt ger en krets som gör precis tvärtom.

Använd lampan som provare: koppla +24 V till en anslutning a, och lampan mellan en annan anslutning b och 0 V. Om det som sitter mellan a och b leder, lyser lampan.



Lampan lyser om det som sitter mellan a och b leder.
Koppla ett kontaktelement i taget mellan a och b, och tryck på knappen.

Figur A.2 Provkopplingen: lampan lyser om det som sitter mellan a och b leder.

a) Koppla in ett kontaktelement i taget mellan a och b. Pröva med knappen släppt och nedtryckt, och fyll i tabellen, med en rad per element. Har knapparna fler än två element, lägg till rader:

Knapp	Anslutningar	Lampan, knappen släppt	Lampan, knappen nedtryckt	Slutande eller brytande?
S1				
S1				
S2				
S2				

b) Stämmer resultatet med märkningen, enligt regeln att 3–4 är slutande och 1–2 brytande?

c) Rita av din station som i figur A.1, med rätt nummer, och fyll i numren i dina scheman från F2.

REDOVISA tabellen och ritningen.

Uppgift 2 AND

Bygg AND-funktionen, `H1 = S1 · S2`, efter ditt schema från F2.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

Nämn ett ställe där en AND-funktion med två knappar används i verkligheten, och förklara varför.

REDOVISA schemat, tabellen och kopplingen.

Uppgift 3 OR

Bygg OR-funktionen, `H1 = S1 + S2`.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

REDOVISA schemat, tabellen och kopplingen.

Uppgift 4 NOT

Bygg NOT-funktionen, `H1 = S1'`, med S1 som enda knapp.

S1	H1 förutsagt	H1 uppmätt
0		
1		

Förklara varför en nödstoppsknapp alltid kopplas med sitt brytande element.

REDOVISA schemat, tabellen och kopplingen.

Uppgift 5 NAND och NOR

a) Bygg NAND-funktionen, `H1 = (S1 · S2)'`. Använd De Morgans lag för att skriva om uttrycket så att det bara innehåller enskilda, eventuellt inverterade, variabler, och bygg det uttrycket.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

b) Bygg NOR-funktionen, `H1 = (S1 + S2)'`, på samma sätt.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

c) Jämför dina NAND- och NOR-kopplingar med AND- och OR-kopplingarna i uppgift 2 och 3. Vad har ändrats, och hur hänger det ihop med De Morgans lagar?

REDOVISA båda kopplingarna och svaret på c).

Uppgift 6 XOR och XNOR

a) Bygg XOR-funktionen, `H1 = S1 · S2' + S1' · S2`. Varje knapp används med både sitt slutande och sitt brytande element.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

b) Gör om kopplingen till XNOR, `H1 = S1 · S2 + S1' · S2'`, genom att **flytta så få sladdar som möjligt**. Vilka flyttade du, och varför räcker det?

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

c) XOR-kopplingen kallas ibland trappkoppling. Förklara varför, och vad som skiljer den från en riktig trappbelysning med två vippströmbrytare.

REDOVISA båda kopplingarna och svaren på b) och c).

Uppgift 7 Syntes från sanningstabell

H1 är driftlampan till en maskin. Den ska lysa när maskinens skyddslucka är stängd, eller när servicenyckeln är vriden, oavsett luckan. I laborationen motsvaras servicenyckeln av S1 (nedtryckt betyder vriden) och luckans givare av S2, som påverkas, alltså är nedtryckt, när luckan är **öppen**.

S1	S2	H1
0	0	1
0	1	0
1	0	1
1	1	1

a) Ta fram uttrycket för H1 på SP-form ur tabellen, och förenkla det (F3). Vilken räknelag använder du?

b) Bygg det minsta kontaktnätet för det förenklade uttrycket, och kontrollera det mot tabellen.

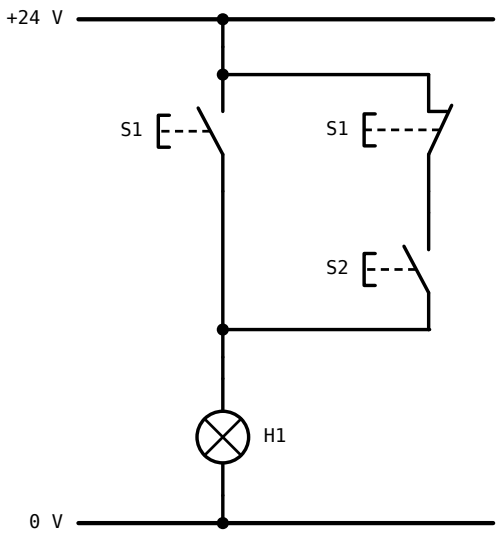
S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

c) Hur många kontakter hade nätet behövt om du hade byggt det oförenklade uttrycket, med en strömväg per minterm?

REDOVISA härledningen, kopplingen och svaret på c).

Uppgift 8 Analys av ett kontaktnät

Nätet i figur A.3 har konstruerats av någon annan. Din uppgift är att ta reda på vad det gör, och om det går att göra enklare.



Nätet i uppgift 8. Kontakterna med samma beteckning sitter i samma knapp.

Figur A.3 Nätet i uppgift 8.

a) Skriv uttrycket för H1 (F4).

b) Förutsäg sanningstabellen, bygg nätet precis som det är ritat, och mät.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

c) Förenkla uttrycket, och bygg det minsta likvärdiga nätet. Mät igen, och kontrollera att tabellen är densamma.

S1	S2	H1 förutsagt	H1 uppmätt
0	0		
0	1		
1	0		
1	1		

d) En av kontakterna i det ursprungliga nätet gick att ta bort. Förklara med ord, utan räknelagar, varför den aldrig påverkar lampan fast den påverkar strömmen.

REDOVISA båda näten, tabellerna och förklaringen i d).

Uppgift 9 Trappomkopplare

Bygg en trappomkopplare, där lampan kan tändas och släckas med vilken som helst av de två knapparna.

Varje gång en knapp ändrar läge ska lampan ändra läge. Om lampan lyser när båda knapparna är släppta, ska den alltså slockna när S1 eller S2 trycks ned. När båda knapparna är nedtryckta ska lampan vara i samma läge som när båda är släppta.

Rita kontaktnätet och koppla det.

REDOVISA kontaktnätet och kopplingen.

Uppgift 10 Fler funktioner EXTRA

Frivillig uppgift, för den som blir klar i tid.

Med två knappar finns det 16 olika funktioner, eftersom varje rad i en tabell med fyra rader kan vara `0` eller `1`.

a) Skriv upp alla 16 sanningstabellerna, och ange för var och en ett så enkelt uttryck som möjligt. Två av dem behöver inga kontakter alls.

b) Välj fyra funktioner som du inte redan har byggt, bygg dem, och kontrollera dem mot tabellerna.

c) Vilka av de 16 funktionerna behöver både det slutande och det brytande elementet i båda knapparna? Vilka klarar sig med ett element per knapp?

A.7 Redovisning

Läraren prickar av varje uppgift när den är redovisad. För godkänd laboration krävs uppgift 1–9; uppgift 10 är frivillig. Båda i paret ska kunna förklara varje koppling.

Uppgift	Innehåll	Redovisad
F1–F4	Förberedelser	
1	Identifiera kontakterna	
2	AND	
3	OR	
4	NOT	
5	NAND och NOR	
6	XOR och XNOR	
7	Syntes från sanningstabell	
8	Analys av ett kontaktnät	
9	Trappomkopplare	
10	Extra (frivillig)	

Återställ stationen när ni är klara: aggregatet av, sladdarna tillbaka på sin plats.

BILAGA B

Labb 2: Grindnät med IC-kretsar

I Labb 1 byggde du logiska funktioner av kontakter. Nu gör du samma sak med **IC-kretsar**: små kretsar ur 74-serien, med fyra eller sex grindar i varje, uppkopplade på ett kopplingsdäck. Labben hör till kapitel 5 och har två delar som görs i samma pass:

- **Labb 2-1** (avsnitt B.5): grindnät med AND-, OR- och inverterarkretsar, från ett minimerat uttryck till en fungerande krets.
- **Labb 2-2** (avsnitt B.6): samma sorts nät med enbart NAND-grindar, och en titt på NOR.

Teorin finns i kapitel 4: Karnaughdiagram i avsnitt 4.1 och IC-kretsar, kopplingsdäck och planering i avsnitt 4.2. Hur du hittar felet när något inte fungerar står i avsnitt 5.1.

B.1 Mål

Efter labben ska du kunna:

- ta fram ett minimerat uttryck med ett Karnaughdiagram och realisera det med IC-kretsar;
- läsa en benplacering och koppla upp en IC-krets med matning, ingångar och utgång;
- planera en uppkoppling med en grindtilldelning och en kopplingslista, och följa planen;
- bygga om ett nät till enbart NAND-grindar, och förklara varför det kan spara kretsar;
- verifiera ett nät mot dess sanningstabell, och felsöka det systematiskt när det inte stämmer.

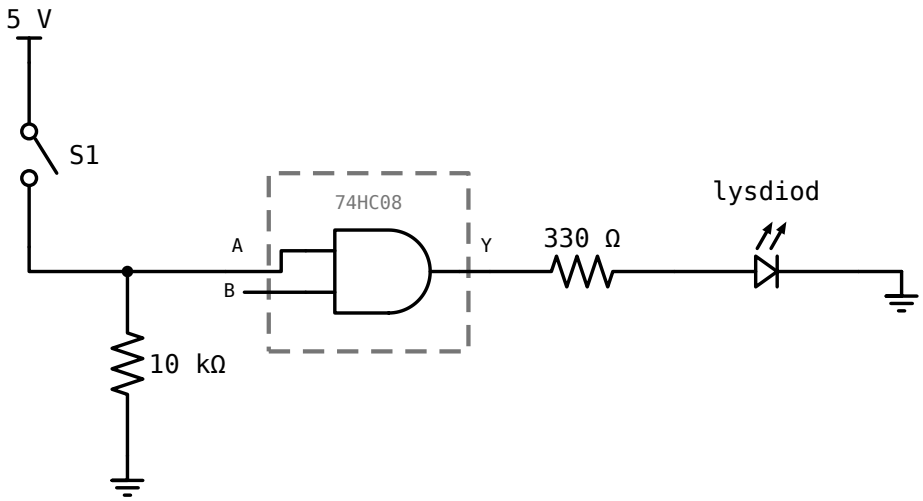
B.2 Utrustning

Varje station har:

- ett kopplingsdäck med 5 V-matning, strömbrytare för ingångarna och lysdioder för utgångarna;
- kretsarna **74HC08** (AND), **74HC32** (OR) och **74HC04** (inverterare) till Labb 2-1;
- kretsarna **74HC00** (NAND) och **74HC02** (NOR) till Labb 2-2;
- kopplingstråd, och en multimeter.

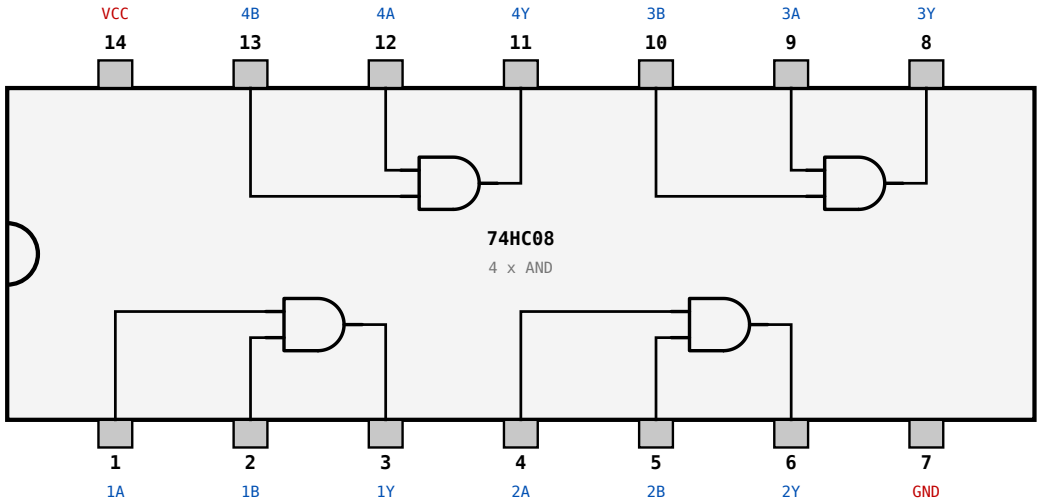
Titta efter på din station hur strömbrytarna och lysdioderna är anslutna. Om stationen saknar egna lysdioder kopplar du en lysdiod med ett förkopplingsmotstånd på 330 Ω från

utgången till jord. Om du använder egna tryckknappar i stället för stationens strömbrytare behöver varje ingång ett pull-down-motstånd. Båda beskrivs i avsnitt 4.2.4 och 4.2.5, och figur B.1 visar dem.

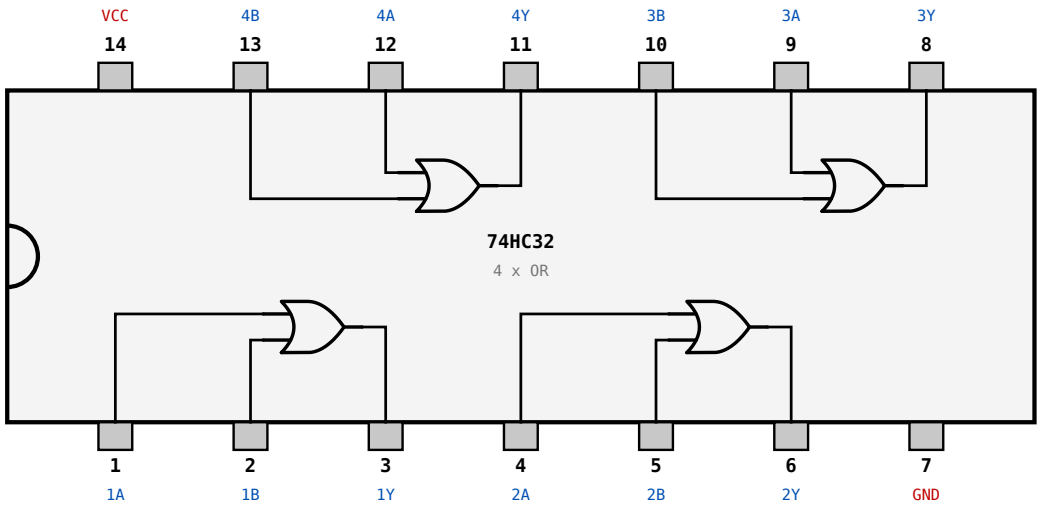


Figur B.1 Tryckknapp med pull-down-motstånd på en grindingång, och lysdiod med förkopplingsmotstånd på utgången.

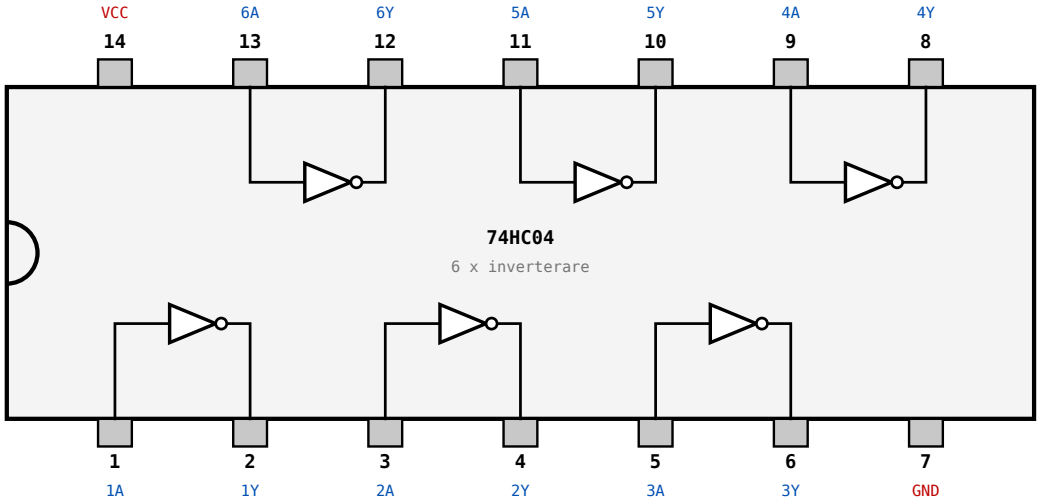
Benplaceringarna för kretsarna, sedda ovanifrån med hacket åt vänster, finns i figur B.2–B.6.



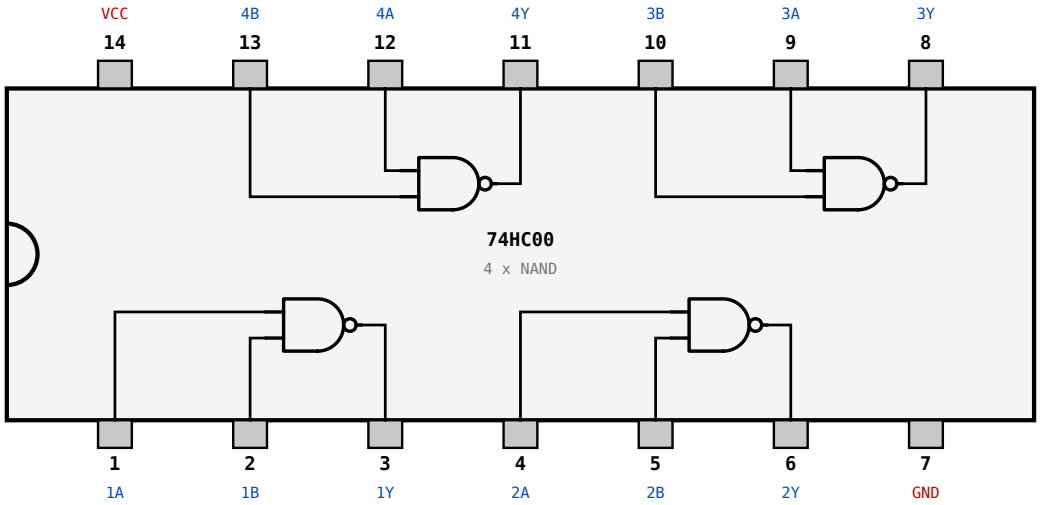
Figur B.2 Benplacering för 74HC08, fyra AND-grindar.



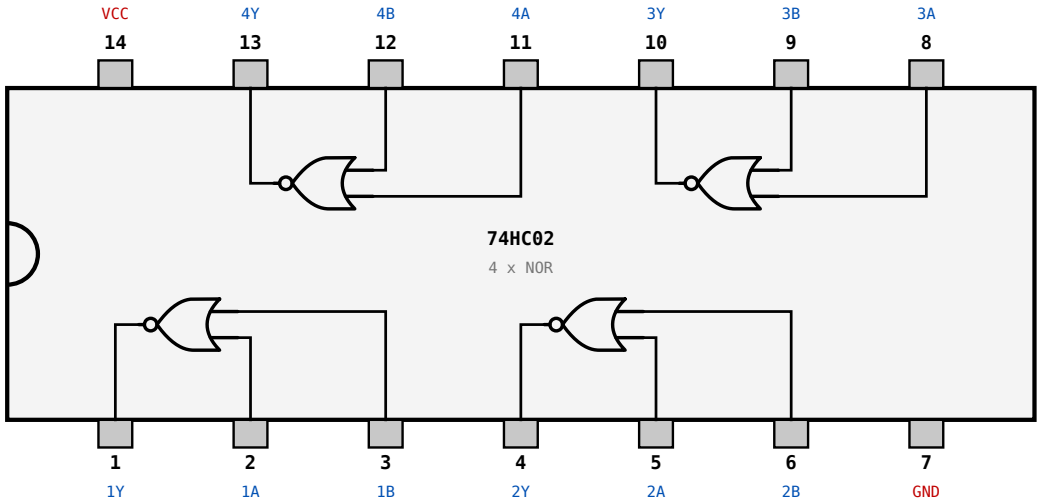
Figur B.3 Benplacering för 74HC32, fyra OR-grindar.



Figur B.4 Benplacering för 74HC04, sex inverterare.

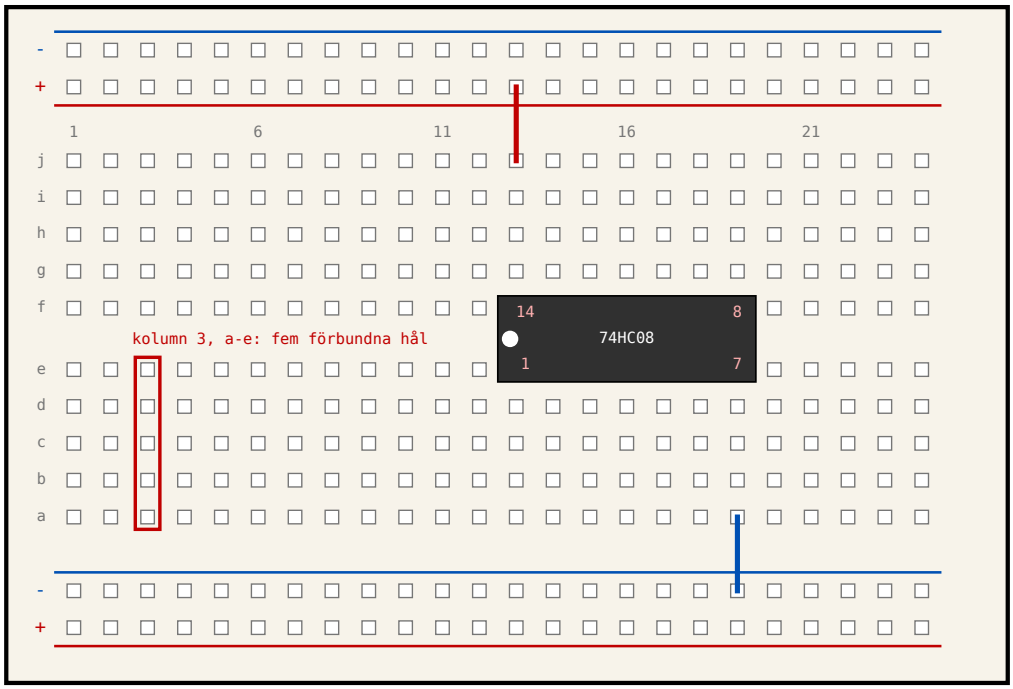


Figur B.5 Benplacering för 74HC00, fyra NAND-grindar.



Figur B.6 Benplacering för 74HC02, fyra NOR-grindar. Här är ben 1 en utgång.

Figur B.7 visar kopplingsdäcket, med kretsen över mittspåret och matningen dragen.



Matningsskenorna går längs hela kanten; hålen i mitten är förbundna fem och fem i kolumner. Kretsen sitter över mittspåret, så att varje ben får en egen kolumn. Ben 14 går till +5 V och ben 7 till jord.

Figur B.7 Kopplingsdäcket, med en IC-krets över mittspåret.

B.3 Säkerhet

- Koppla med matningen **frånslagen**, och slå på först när kopplingen är kontrollerad mot din kopplingslista.
- **Kontrollera kretsens riktning** innan du slår på: hacket åt vänster, ben 1 nere till vänster. En felvänd krets får plus och jord omkastade.
- Känn efter med ett finger på kretsen strax efter att du slagit på. **En krets som blir varm är felkopplad**: bryt matningen direkt.
- Koppla aldrig 24 V från Labb 1 till kopplingsdäcket. Kretsarna tål högst 7 V.

B.4 Förberedelser

Görs **före** passet, hemma eller i samband med kapitel 4. Utan dem hinner du inte klart, och de ingår i redovisningen.

F1 – Sanningstabeller. Skriv den förväntade sanningstabellen för en AND-, en OR- och en inverterargrind, och för näten i Labb 2-1 uppgift 2 och 3.

F2 – Karnaughdiagram. Ta fram ett minimerat uttryck för Labb 2-1 uppgift 4 med ett Karnaughdiagram, och rita grindnätet.

F3 – NAND-nät. Rita NOT, AND och OR byggda av enbart NAND-grindar (Labb 2-2 uppgift 2), och nätet för `X = AB + C` med enbart NAND (Labb 2-2 uppgift 3). Kontrollera varje nät med De Morgans lagar eller genom att gå igenom alla rader.

F4 – Simulera. Bygg varje nät i CircuitVerse, `https://circuitverse.org/simulator`, och kontrollera det mot sanningstabellen, rad för rad.

F5 – Planera. Gör en **grindtilldelning** och en **kopplingslista** för varje uppgift där du bygger ett nät: Labb 2-1 uppgift 2, 3 och 4 och Labb 2-2 uppgift 3. Använd formen i avsnitt 4.2.7, med bennummer på varje rad.

B.5 Labb 2-1

AND-, OR- och inverterarkretsar: 74HC08, 74HC32 och 74HC04.

Arbetsgången är densamma i varje uppgift, och det är värt att hålla fast vid den:

1. Koppla matningen till varje krets först, ben 14 till +5 V och ben 7 till jord.
2. Koppla resten enligt kopplingslistan, och bocka av varje rad.
3. Koppla ingångarna på oanvända grindar till jord.
4. Slå på, och gå igenom **alla** kombinationer av ingångarna. Fyll i tabellen: förväntat värde från förberedelserna, och uppmätt värde.
5. Om en rad inte stämmer: felsök enligt avsnitt 5.1, inte genom att flytta sladdar på måfå.

Uppgift 1 Verifiera AND, OR och inverterare

Innan du bygger något med en krets ska du veta att den fungerar och att du läser benplaceringen rätt.

a) Sätt 74HC08 på kopplingsdäcket och koppla matningen. Koppla grind 1 (ben 1, 2 och 3) till två strömbrytare och en lysdiod. Koppla ingångarna på grind 2–4 till jord.

b) Gå igenom alla fyra kombinationerna och fyll i tabellen.

c) Byt 74HC08 mot 74HC32 **utan att flytta en enda sladd**, och upprepa. Förklara varför det fungerar.

d) Koppla en inverterare i 74HC04 (ben 1 in, ben 2 ut) och kontrollera båda raderna.

e) Mät med multimetern spänningen på en utgång som är 1 och en som är 0, med lysdioden ansluten. Skriv upp värdena; de används i kontrolluppgiften, övning 5.10.

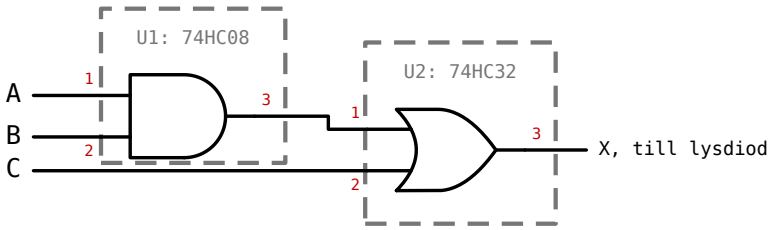
A	B	AND, förväntat	AND, mätt	OR, förväntat	OR, mätt
0	0				
0	1				
1	0				
1	1				

A	NOT, förväntat	NOT, mätt
0		
1		

REDOVISA tabellerna och de uppmätta spänningarna.

Uppgift 2 X = AB + C

Nätet från avsnitt 4.1.3, två grindar i två kretsar. Figur B.8 visar det med kretsbeteckningar och bennummer, så som kopplingslistan läser det.




```
Röda siffror är bennummer. Båda kretsarna har dessutom VCC på ben 14
och GND på ben 7, och ingångarna på de oanvända grindarna kopplade till GND.
```

Figur B.8 Grindnätet `X = AB + C` med 74HC08 och 74HC32, och bennumren utskrivna.

a) Koppla upp nätet enligt din kopplingslista från F5.

b) Gå igenom alla åtta kombinationerna och fyll i tabellen.

c) Mät spänningen på U1 ben 3 för raden `ABC = 110`. Vilken logisk nivå är det, och stämmer det med vad `AB` ska vara?

A	B	C	X, förväntat	X, mätt
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

REDOVISA den fungerande kretsen och tabellen.

Uppgift 3 Bromsassistenten

Bromsassistenten från avsnitt 2.1.10 har fyra ingångar och en utgång, med samma beteckningar som där:

- `D`: föraren står på bromspedalen;
- `S` och `R`: avståndssensorn respektive radarn, två detektorer som var och en kan se ett hinder;
- `F`: assistanssystemet rapporterar ett fel;
- `B`: bilen bromsar.

Bilen ska bromsa om föraren bromsar, eller om någon av detektorerna ser ett hinder och systemet inte rapporterar något fel. Föraren ska alltid kunna bromsa, även när systemet är trasigt.

a) Skriv uttrycket för `B` (du tog fram det i kapitel 2; kontrollera det mot kravet igen), och rita grindnätet med AND, OR och NOT.

b) Gör en grindtilldelning. Hur många grindar av varje sort behövs, och vilka kretsar?

c) Koppla upp nätet och gå igenom alla sexton kombinationerna.

d) Säkerhetskravet: kontrollera särskilt de åtta raderna där `D = 1`. Vad ska `B` vara på alla dem, oavsett de andra ingångarna?

D	S	R	F	B, förväntat	B, mätt
0	0	0	0		
0	0	0	1		
0	0	1	0		
0	0	1	1		
0	1	0	0		
0	1	0	1		
0	1	1	0		
0	1	1	1		
1	0	0	0		
1	0	0	1		
1	0	1	0		
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

REDOVISA den fungerande kretsen, tabellen, och svaret på d).

Uppgift 4 Syntes med Karnaughdiagram

Ett grindnät med fyra ingångar `ABCD` och en utgång `X` har sanningstabellen nedan. Du har tagit fram det minimerade uttrycket i F2.

ABCD	X	ABCD	X
0000	0	1000	0
0001	1	1001	1
0010	0	1010	0
0011	1	1011	1
0100	0	1100	1
0101	0	1101	0
0110	0	1110	1
0111	0	1111	0

a) Visa ditt Karnaughdiagram och ditt minimerade uttryck.

b) Hur många kretsar behövs? Jämför med vad den direkta avläsningen ur tabellen hade krävt.

c) Koppla upp nätet och kontrollera alla sexton raderna. Skriv ned förväntat och uppmätt värde för varje rad, i en tabell som ovan.

REDOVISA diagrammet, uttrycket, den fungerande kretsen och tabellen.

B.6 Labb 2-2

NAND och NOR: 74HC00 och 74HC02.

I kapitel 2 såg du att NAND ensam räcker för att bygga vilken funktion som helst. Här bevisar du det med riktiga kretsar, och ser vad det kostar och vad det sparar.

Uppgift 1 Verifiera NAND och NOR

a) Koppla grind 1 i 74HC00 (ingångar ben 1 och 2, utgång ben 3) och kontrollera alla fyra raderna.

b) Byt till 74HC02. **Titta på benplaceringen först**: i 74HC02 är ben 1 en utgång, och ingångarna på grind 1 är ben 2 och 3. Flytta sladdarna därefter, och kontrollera alla fyra raderna.

c) Vad hade hänt om du bara bytt kretsen, som i Labb 2-1 uppgift 1 c)? Två utgångar hade kopplats mot varandra, eller en strömbrytare mot en utgång. Förklara varför det inte är bra.

A	B	NAND, förväntat	NAND, mätt	NOR, förväntat	NOR, mätt
0	0				
0	1				
1	0				
1	1				

REDOVISA tabellen.

Uppgift 2 NOT, AND och OR av NAND

Bygg de tre grundgrindarna av enbart NAND-grindar ur en 74HC00, en i taget, med näten från F3:

- **NOT**: en NAND med båda ingångarna ihopkopplade.
- **AND**: en NAND följd av en NOT.
- **OR**: en NOT på varje ingång, följda av en NAND. Kontrollera med De Morgan: `(A'B')' = A + B`.

a) Hur många NAND-grindar kräver var och en? Ryms alla tre i en 74HC00 samtidigt?

b) Koppla upp var och en och kontrollera den mot sin sanningstabell.

REDOVISA varje grind när den fungerar.

Uppgift 3 X = AB + C med enbart NAND

Samma funktion som i Labb 2-1 uppgift 2, nu enligt avsnitt 4.1.9:

```
X = ((AB)' · C')'
```

a) Koppla upp nätet i en enda 74HC00.

b) Gå igenom alla åtta raderna. Tabellen ska vara identisk med den i Labb 2-1 uppgift 2.

c) Jämför de två lösningarna: antal grindar, antal kretsar och antal sladdar. Vilken hade du valt om du skulle bygga hundra exemplar?

REDOVISA den fungerande kretsen, tabellen och jämförelsen.

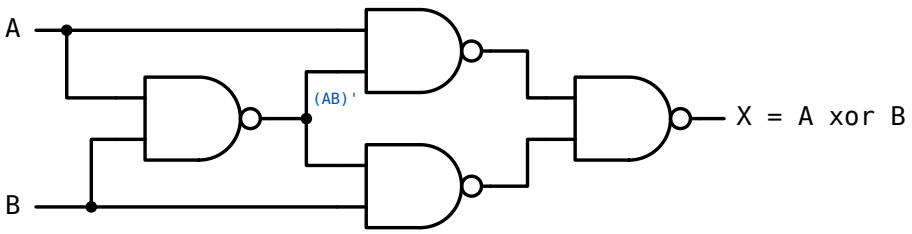
Uppgift 4 Enbart NOR, och XOR av fyra NAND

EXTRA

Frivillig, för den som hinner. Den krävs inte för godkänt.

a) Bygg `X = AB + C` med enbart NOR-grindar i en 74HC02, med uttrycket `X = ((A + C)' + (B + C)')'`. Tänk på benplaceringen.

b) Bygg en XOR av fyra NAND-grindar i en 74HC00, som i figur B.9.



Figur B.9 XOR byggd av fyra NAND-grindar.

Kontrollera alla fyra raderna, och jämför med sanningstabellen för XOR.

c) Bromsassistenten från Labb 2-1 uppgift 3 med enbart NAND-grindar. Hur många kretsar behövs?

REDOVISA det du hunnit.

B.7 Redovisning

Bocka av varje uppgift när den är redovisad för läraren. Båda i paret ska kunna förklara lösningen.

Del	Uppgift	Krävs för G	Redovisad
Förberedelser	F1–F5	ja	
Labb 2-1	1 – Verifiera AND, OR och inverterare	ja	
Labb 2-1	2 – X = AB + C	ja	
Labb 2-1	3 – Bromsassistenten	ja	
Labb 2-1	4 – Syntes med Karnaughdiagram	ja	
Labb 2-2	1 – Verifiera NAND och NOR	ja	
Labb 2-2	2 – NOT, AND och OR av NAND	ja	
Labb 2-2	3 – X = AB + C med enbart NAND	ja	
Labb 2-2	4 – Extra	nej	

BILAGA C

Labb 3: Mikrodatorn

C.1 Mål

I Labb 3 programmerar du mikrodatorn ATmega328P i assembler. Du börjar med att ladda ett tal i ett register och slutar med ett trafikljus som körs på ett riktigt Arduino Uno-kort. På vägen använder du allt kursens mikrodatordel tar upp: talformat, aritmetik och flaggor, logiska operationer, skift, villkorliga hopp, loopar, tidsfördröjningar, in- och utportar, subrutiner och stacken.

Efter laborationen ska du kunna:

- skapa, bygga, rätta och simulera ett assemblerprogram i Microchip Studio;
- förutsäga vad ett kort program gör med register, flaggor och portar, och kontrollera det i simulatorn;
- skriva program med beslut, loopar och tidsfördröjningar utifrån en flödesplan;
- programmera mikrodatorn att styra ett styrobjekt: lysdioder som tänds och släcks, och en knapp som läses av.

C.2 Upplägg

Labb 3 löper över de fem mikrodatorpassen, som hör till kapitel 7–11. Varje pass börjar med en genomgång, och resten av passet är laborationstid. Laborationen består av många små delar som görs **i ordning, så långt ni hinner**. Delarna bygger på varandra, så hoppa inte över någon.

- **Arbeta i par**, vid en dator med Microchip Studio installerat (bilaga D). Byt den som skriver vid varje ny del.
- **Ett projekt per del.** Döp projektet efter delen, till exempel `lab3_06` för del 6, så att ni kan öppna det igen vid redovisningen.
- **Förutsäg innan ni simulerar.** Varje del ber er skriva ned vad ni väntar er innan ni stegar. Gör det på riktigt, på papper eller i en kommentar i programmet. Det är förutsägelsen som visar om ni har förstått, och en avvikelse är det snabbaste sättet att hitta ett fel (avsnitt 7.3.3).

- **Redovisa** för läraren vid varje ställe som är markerat REDOVISA. Visa programmet i simulatorn och förklara det; båda i paret ska kunna svara på frågor.

  Nästan allt görs i simulatorn. Arduino-kortet behövs först i slutuppgiften, i kapitel 11.

C.3 Programmallen

Varje program utgår från mallen, `labs/lab3/code/template.asm`. Kopiera in den i `main.asm` när projektet är skapat, och skriv programmet mellan `main:` och `end:`.

labs/lab3/code/template.asm

```
;*****************************************************************************
; Lab 3, part N: <title>
;
; <What the program does, in one or two sentences.>
;
; Author: <your names>
;*****************************************************************************
.include "m328Pdef.inc"             ; Register names for the ATmega328P: PORTB, ↪
    ↪ DDRB ...

.org 0x0000                         ; The processor starts here after reset.
    rjmp main                       ; Jump over everything else, to main.

;-----------------------------------------------------------------------------
; main: The program starts here.
;-----------------------------------------------------------------------------
main:
    ; Write your program here.

end:
    rjmp end                        ; Stay here forever: there is nothing to ↪
    ↪ return to.
```

Vad varje rad gör förklaras i avsnitt 7.2.6. Instruktionerna finns samlade på referensbladet i bilaga E.

C.4 Delarna

Numreringen följer den laborationssamling kursen bygger på. Del 13 ingår inte, så numreringen hoppar från 12 till 14.

Del	Innehåll	Kapitel	Avsnitt
1.1	Microchip Studio: skapa, skriva, spara, assemblera, simulera	7	C.7
1.2	Öppna, ändra, rätta, assemblera, simulera	7	C.7
2	Decimalt, binärt eller hexadecimalt	7	C.7
3	Addition	8	C.8
4	Subtraktion	8	C.8
5	Addition och subtraktion decimalt, binärt och hexadecimalt	8	C.8
6	Utporten PORTB, inc, I/O-simulering och .def	8	C.8
7	Logiska operationer	8	C.8
8	Rotationer och skift	8	C.8
9	Aritmetisk rundgång	8	C.8
10	Flaggorna i SREG	8	C.8
11	Vitsen med flaggor: jämförelse med cp	9	C.9
12	Flödesplan och villkorliga hopp	9	C.9
14	Loop med visst antal varv	9	C.9
15	Kort tidsfördröjning	9	C.9
16	Längre tidsfördröjning	9	C.9
17	Flervalssituationer och omarbetning av flödesplaner	9	C.9
18	ASCII-koder för hexadecimala siffertecken	10	C.10
19	16-bitarsoperationer	10	C.10
20	Lång tidsfördröjning	10	C.10
21	Inporten PORTD (PIND)	10	C.10
22	Subrutiner	10	C.10
23	Använda stacken som lagringsplats	10	C.10
A	Slutuppgift del A: trafikljuset	11	C.11
B	Slutuppgift del B: övergångsställe	11	C.11

Varje pass hör ihop med en del av laborationen, men ni behöver inte bli klara med alla delar på samma pass. Fortsätt där ni slutade nästa gång.

C.5 Redovisning och bedömning

Laboration	Krav för G	Krav för VG
Labb 3	Del 1.1–12, 14–16, 21 och slutuppgiften del A	Samtliga delar 1.1–23 och slutuppgiften del B

En del är godkänd när programmet är redovisat för läraren i simulatorn, eller på kortet i slutuppgiften, och båda i paret kan förklara det. Skriv upp vilka delar som är redovisade, och spara projekten tills kursen är slut.

C.6 När något inte fungerar

- **Bygg om och läs det första felet** i Error List (avsnitt 7.3.4).

- **Stega fram till felet.** Leta upp den första instruktion där registren inte blir det ni förutsade. Felet sitter där eller strax före.
- **Ändra en sak i taget**, och bygg om efter varje ändring.
- **Fråga.** Läraren hjälper hellre till tidigt än ser er fastna i en halvtimme.

Vanliga fel samlas i tabellen i avsnitt D.6.

C.7 Del 1.1, 1.2 och 2: Första stegen

Delarna hör till kapitel 7. Läs avsnitt 7.2 och avsnitt 7.3 innan ni börjar, och ha bilaga D om Microchip Studio öppen bredvid.

Del 1.1 Microchip Studio: skapa, skriva, spara, assemblera, simulera

Mål

Skapa ett assemblerprojekt, skriva ett program, bygga det och stega igenom det i simulatorn.

Bakgrund

Arbetsgången, skriv, bygg, förutsäg, stega och jämför, beskrivs i avsnitt 7.3.1. Instruktionerna i programmet står i avsnitt 7.2.5.

Programmet

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi r16, 10                    ; r16 = 10
    ldi r17, 20                    ; r17 = 20
    mov r18, r16                   ; r18 = r16
    inc r18                        ; r18 = r18 + 1
    inc r18                        ; r18 = r18 + 1
    dec r17                        ; r17 = r17 - 1

end:
    rjmp end
```

Uppgifter

1. Skapa ett projekt med namnet `lab3_01_1` enligt avsnitt D.2.
2. Ersätt innehållet i `main.asm` med programmet ovan. **Skriv av det** i stället för att klistra in: det är så ni lär er hur en rad är uppbyggd. Kommentarerna kan ni hoppa över.
3. Spara med **Ctrl+S**.
4. Bygg med **F7**. Läs utskriften i fönstret Output: står det `0 errors`? Om inte, rätta raden Error List pekar på och bygg om.
5. Välj simulatorn som verktyg (avsnitt D.4.1) och starta med **Debug → Start Debugging and Break** (**Alt+F5**). Öppna fönstret **Processor Status**.
6. **Innan ni stegar:** fyll i tabellen nedan med registrens värden efter varje instruktion, decimalt och hexadecimalt.

Efter	r16	r17	r18
`ldi r16, 10`			
`ldi r17, 20`			
`mov r18, r16`			
`inc r18`			
`inc r18`			
`dec r17`			

7. Stega med **F11**, en instruktion i taget, och jämför Processor Status med tabellen efter varje steg. Lägg märke till att registret som just ändrades visas i rött.
8. Läs av **Program Counter** vid varje steg. Med hur mycket ökar den för varje instruktion?
9. När den gula pilen står på `rjmp end`: vad visar **Cycle Counter**? Räkna ut värdet för hand först, med hjälp av avsnitt 7.3.5.

Frågor

- Den gula pilen står på `inc r18`. Har `inc r18` körts ännu?
- Ändras `r16` av instruktionen `mov r18, r16`?

REDOVISA tabellen och simuleringen för läraren.

Del 1.2 Öppna, ändra, rätta, assemblera, simulera

Mål

Öppna ett befintligt projekt, ändra programmet, och rätta fel med hjälp av felmeddelandena.

Bakgrund

De vanligaste felmeddelandena, och hur de rättas, står i avsnitt 7.3.4.

Uppgifter

1. Stäng Microchip Studio och starta det igen. Öppna projektet från del 1.1, antingen från listan över senaste projekt på startsidan eller med **File → Open → Project/Solution...**
2. Ändra programmet:
   - `r16` ska laddas med 25 i stället för 10;
   - lägg till raden `mov r19, r17` efter `dec r17`.
3. **Förutsäg** värdena i `r16`–`r19` när programmet når `end`. Bygg, simulera och kontrollera.
4. Lägg nu till de här fyra raderna direkt efter `mov r19, r17`. De innehåller fyra fel med flit:

   ```
   ldi r16 7
   ldi r5, 3
   inc r19, 2
   rjmp ennd
   ```

   Meningen med raderna är: ladda 7 i `r16`, ladda 3 i något register, öka `r19` med två, och hoppa till `end`.
5. Bygg. Läs det första felet i **Error List**, dubbelklicka på det, rätta raden och bygg om. Fortsätt tills programmet byggs utan fel. Skriv för varje fel ned vad meddelandet sa och hur ni rättade det.
6. **Förutsäg** värdena i `r16`–`r20` när programmet når `end`. Simulera och kontrollera.

Frågor

- Hur många gånger behövde ni bygga om innan alla fel var borta? Fick ni alla fyra felen på en gång?
- Varför kan `inc` inte öka med två på en gång?

REDOVISA det rättade programmet och de fyra felmeddelandena.

Del 2 Decimalt, binärt eller hexadecimalt

Mål

Skriva tal i ett program decimalt, binärt och hexadecimalt, och läsa dem i simulatorn.

Bakgrund

Talformaten står i avsnitt 7.2.4 och omvandlingen mellan talsystemen i kapitel 1.

Uppgifter

1. Skriv ett program som laddar talet 200 i `r16` decimalt, i `r17` binärt och i `r18` hexadecimalt. Räkna ut det binära och det hexadecimala skrivsättet för hand. **Förutsäg** hur Processor Status visar de tre registren. Simulera och kontrollera.
2. Fyll i de tomma rutorna i tabellen **för hand**:

Register	Decimalt	Binärt	Hexadecimalt
`r19`	9		
`r20`		`0b00001111`	
`r21`			`0x10`
`r22`	100		
`r23`		`0b01111111`	
`r24`			`0x80`
`r25`	255		

3. Lägg till en `ldi` per rad i programmet, som laddar registret med talet **i det skrivsätt tabellen ger det**: `ldi r19, 9`, `ldi r20, 0b00001111`, och så vidare. Simulera och kontrollera att Processor Status visar det hexadecimala värde ni räknade ut på varje rad.
4. Lägg till `ldi r26, '0'` och `ldi r27, 'A'`. Vilka värden får registren? Ett tecken lagras som ett tal, dess ASCII-kod; det återkommer i del 18.
5. Välj för varje rad i tabellen vilket skrivsätt som passar bäst om talet är
   - ett antal varv i en loop;
   - ett mönster av lysdioder som ska lysa.

Frågor

- Blir maskinkoden olika för `ldi r16, 200` och `ldi r16, 0xC8`? Titta i listfilen om ni är osäkra (avsnitt 7.2.7).

REDOVISA tabellen och simuleringen.

C.8 Del 3–10: Aritmetik, logik och flaggor

Delarna hör till kapitel 8. Varje del är ett eget projekt som utgår från mallen (avsnitt C.3). Ha referensbladet i bilaga E till hands.

Förutsäg alltid först. Varje del har uppgifter som börjar med *Förutsäg*. Skriv svaret på papper eller som en kommentar i programmet innan ni stegar, och jämför sedan.

Del 3 Addition

Mål

Addera med `add` och `inc`, och se vad som händer när summan inte får plats i en byte.

Bakgrund

Avsnitt 8.1.1 och 8.1.2.

Uppgifter

1. Skriv ett program som laddar 25 i `r16` och 17 i `r17`, och adderar dem med `add r16, r17`. **Förutsäg** `r16` decimalt, hexadecimalt och binärt. Simulera och kontrollera.
2. Gör samma addition för hand på papper, binärt, kolumn för kolumn med minnessiffror.
3. Lägg till en addition av två hexadecimala tal: `0x3A` i `r18` plus `0x15` i `r19`. **Förutsäg** `r18`, både hexadecimalt och decimalt.
4. Lägg till instruktioner som räknar ut 10 + 20 + 30 i `r20`. Hur många `add` behövs?
5. Lägg till en addition av 200 i `r23` och 100 i `r24`. **Förutsäg** `r23`. Titta sedan på flaggan `C` i Processor Status efter additionen. Vad betyder den här?

Frågor

- Vilket av de två registren i `add r16, r17` ändras?
- Summan 200 + 100 = 300 får inte plats. Var finns ”resten” av svaret efter additionen?

REDOVISA programmet, uträkningen från uppgift 2 och svaret på frågorna.

Del 4 Subtraktion

Mål
Subtrahera med `sub`, `subi` och `dec`, och addera en konstant utan en `addi`.

Bakgrund
Avsnitt 8.1.4.

Uppgifter

1. Räkna ut 50 – 20 med `sub`, med 50 i `r16` och 20 i `r17`. **Förutsäg** resultatet.
2. Ladda `0x20` i `r18` och dra ifrån `0x05` med `subi`. **Förutsäg** `r18` hexadecimalt och decimalt.
3. Ladda 3 i `r19` och kör `dec r19` tre gånger. **Förutsäg** `r19` och flaggan `Z` efter varje `dec`. Sätt en etikett efter den tredje, så kan ni köra dit med **Run To Cursor**.
4. Det finns ingen `addi`. Ladda 100 i `r20` och addera 28 med en `subi`. Vilken konstant ska `subi` ha? **Förutsäg** `r20`.
5. Räkna ut 20 – 50 med `sub`, med 20 i `r21` och 50 i `r22`. **Förutsäg** `r21` och `C`.

Frågor

- Vad betyder `C = 1` efter en subtraktion?
- `r21` blev 226 i uppgift 5. Vilket tal är det om man läser det med tecken? Jämför med avsnitt 8.1.5.

REDOVISA programmet och förutsägelserna.

Del 5 Addition och subtraktion decimalt, binärt och hexadecimalt

Mål
Räkna med tal som är skrivna i olika talsystem i samma uttryck.

Bakgrund
Avsnitt 7.2.4 och kapitel 1.

Uppgifter

Skriv ett program som räknar ut de tre uttrycken, med talen skrivna **precis som i uttrycket**. **Förutsäg** först varje resultat decimalt, hexadecimalt och binärt, i en tabell.

1. `r20` = `0x3C` + `0b00001010` – 25
2. `r21` = `0b11110000` – `0x0F` + 1, där den sista additionen görs med `inc`
3. `r22` = `0b10101010` + `0x55`

Register	Decimalt	Hexadecimalt	Binärt
`r20`			
`r21`			
`r22`			

Simulera och jämför med tabellen.

Frågor

- Uttryck 3 ger ett särskilt bitmönster. Vilket, och varför blev det så? Titta på de två talen binärt.

REDOVISA tabellen och simuleringen.

Del 6 Utporten PORTB, inc, I/O-simulering och .def

Mål

Göra port B till utport, räkna på den, och följa bitarna i simulatorns I/O-fönster.

Bakgrund

Avsnitt 8.3, särskilt 8.3.3, 8.3.6 och 8.3.7.

Programmet

```
.include "m328Pdef.inc"

.def counter = r16                  ; The value shown on port B.
.def temp = r17                     ; A scratch register.

.org 0x0000
    rjmp main

main:
    ser temp                        ; temp = 0xFF
```

```
    out DDRB, temp                  ; All eight pins of port B are outputs.
    clr counter                     ; Start counting at zero.

loop:
    out PORTB, counter              ; Show the count on port B.
    inc counter                     ; Count up.
    rjmp loop                       ; And again, forever.
```

Uppgifter

1. Skriv av programmet, bygg och starta simuleringen. Öppna fönstret **I/O** (**Debug → Windows → I/O**) och välj port B.
2. Stega och följ rutorna i `DDRB` och `PORTB`. När blir alla rutor i `DDRB` fyllda?
3. **Förutsäg** hur rutorna i `PORTB` ser ut efter fem varv i loopen. Stega och kontrollera.
4. Varför ändras även `PINB`, fast programmet aldrig skriver dit?
5. Ändra programmet så att det räknar **nedåt**. **Förutsäg** vilket värde som visas direkt efter 0.
6. Ändra programmet så att det räknar uppåt i steg om **två**, med en enda instruktion i stället för `inc`.
7. Kör programmet fritt med **F5**, vänta några sekunder och stoppa med **Break All**. Går det att förutsäga värdet i `PORTB`? Hur lång tid tar ett varv i loopen, vid 16 MHz?

Frågor

- Vad gör `.def`, och blir det någon skillnad i maskinkoden?
- Hur skulle lysdioderna se ut på ett riktigt kort om programmet kördes fritt?

REDOVISA programmet med nedräkning och med steg om två, och svaren på frågorna.

Del 7 Logiska operationer

Mål

Använda `andi`, `ori`, `eor` och `com` med masker för att ändra enskilda bitar.

Bakgrund

Avsnitt 8.2.1 och 8.2.2.

Uppgifter

Gör port B till utport först i programmet, och skriv varje resultat till `PORTB` med `out`, så att det syns bit för bit i I/O-fönstret. **Förutsäg** varje resultat binärt innan ni stegar.

1. Ladda `0b10101010` i `r16` och behåll bara de fyra nedre bitarna.
2. Ladda `0b00110000` i `r17` och ettställ bit 7 och bit 0.
3. Ladda `0b00001111` i `r18` och invertera de fyra övre bitarna. Det finns ingen `eori`: hur gör ni?

4. Ladda `0b00110011` i `r19` och invertera alla bitar med en enda instruktion.
5. Ladda `0b01010101` i `r20`. Ettställ bit 1, nollställ bit 6 och invertera bit 7, med en instruktion per ändring. Vilket värde får `r20`, och vilka stift på port B lyser?

Frågor

- Vilken instruktion och vilken sorts mask använder man för att nollställa bitar? För att ettställa?
- Varför ger `com` och `eor` med `0xFF` samma resultat?

REDOVISA programmet och förutsägelserna.

Del 8 Rotationer och skift

Mål

Skifta och rotera bitar, och använda skift för att multiplicera och dividera med 2.

Bakgrund

Avsnitt 8.2.3 och 8.2.4.

Uppgifter

Förutsäg varje resultat, och `C`, innan ni stegar.

1. Ladda 3 i `r16` och skifta vänster två gånger med `lsl`. Vilket tal har `r16` multiplicerats med?
2. Ladda 200 i `r17` och skifta höger två gånger med `lsr`.
3. Ladda `0b11110000` i `r18`, som är −16 med tecken, och kopiera det till `r19`. Skifta `r18` med `asr` och `r19` med `lsr`. Vilket av resultaten är −16/2?
4. Ladda `0b10010011` i `r20`, nollställ `C` med `clc`, och rotera vänster två gånger med `rol`. **Förutsäg** `r20` och `C` efter varje `rol`.
5. Skriv ett rinnande ljus på port B med `rol r21`, som i avsnitt 8.2.4. Stega igenom ett helt varv, och följ `PORTB` och `C`. Hur många steg går det på ett varv, och varför är ett av dem mörkt?

Frågor

- Varför används `asr` och inte `lsr` för att dividera ett tal med tecken med 2?
- (*Fördjupning*) Byt ut `rol r21` mot de två instruktionerna `lsl r21` och `adc r21, r25`, där `r25` har nollställts med `clr` före loopen. Hur många steg går ljuset nu på ett varv, och varför?

REDOVISA det rinnande ljuset och förutsägelserna.

Del 9 Aritmetisk rundgång

Mål
Se vad som händer när ett register räknas förbi 255 eller under 0, och vilka flaggor som visar det.

Bakgrund
Avsnitt 8.1.3.

Uppgifter
Sätt en etikett efter varje uppgift, så att ni kan köra fram till den och läsa SREG. **Förutsäg** registret och flaggorna `Z` och `C` innan ni kör.

1. Nollställ `C` med `clc`. Ladda 250 i `r16` och kör `inc r16` sex gånger.
2. Ladda 255 i `r17` och 1 i `r18`, och addera dem med `add`.
3. Ladda 0 i `r19` och kör `dec r19`. Titta även på `N`.
4. Ladda 3 i `r20` och 5 i `r21`, och räkna ut `r20 - r21` med `sub`.
5. Ladda 200 i `r22` och 100 i `r23`, och addera dem.

Frågor

- Efter uppgift 1 är `r16` $= 0$ och `Z` $= 1$. Vad är `C`, och varför?
- Efter uppgift 3 är `C` $= 1$. Är det `dec` som har satt den? Vilken instruktion satte den?
- Vilket tal ska man lägga till 44 för att få det ”riktiga” svaret på uppgift 5?

REDOVISA förutsägelserna och svaren på frågorna.

Del 10 Flaggorna i SREG

Mål
Förutsäga flaggorna `C`, `Z`, `N`, `V`, `S` och `H` efter olika operationer, och kontrollera dem.

Bakgrund
Avsnitt 8.1.6 och 8.1.7.

Uppgifter

1. Fyll i tabellen **för hand**. Varje rad görs för sig, med `add`, `sub`, `andi` eller `com` som anges.

Rad	Operation	Resultat	C	Z	N	V	S	H
a	`0x7F + 0x01`							
b	`0xFF + 0x01`							
c	`0x80 + 0x80`							
d	`0x05 - 0x07`							
e	`0x40 + 0x40`							
f	`0x0F + 0x01`							
g	`0x0F` AND `0xF0` (`andi`)							
h	NOT `0x00` (`com`)							

2. Skriv ett program med en rad per operation och en etikett efter varje, till exempel `row_a`, `row_b` och så vidare.
3. Kör fram till varje etikett och jämför SREG med tabellen. Markera varje ruta ni hade fel på.
4. För rad g och h: vilka flaggor ändrades inte av operationen? Jämför med kolumnen *Flaggor* på referensbladet i bilaga E.

Frågor

- Rad a och rad e ger båda `V = 1`. Vad har de gemensamt?
- Rad d ger `C = 1` men `V = 0`. Är svaret rätt eller fel? Svara både utan och med tecken.

REDOVISA tabellen, med era förutsägelser och de rätta värdena bredvid varandra.

C.9 Del 11, 12 och 14–17: Programflöde, loopar och tidsfördröjningar

Delarna hör till kapitel 9. Varje del är ett eget projekt som utgår från mallen (avsnitt C.3). Från och med nu gäller två vanor till:

- **Rita flödesplanen först**, på papper, innan ni skriver en enda instruktion (avsnitt 9.1.1).
- **Mät tider med stoppuret**, med Frequency inställd på 16 MHz (avsnitt 9.2.6).

Del 13 ingår inte i laborationen; numreringen hoppar från 12 till 14.

Del 11 Vitsen med flaggor: jämförelse med cp

Mål

Se vad `cp` och `cpi` gör med flaggorna, och hur flaggorna säger om två tal är lika, större eller mindre.

Bakgrund

Avsnitt 9.1.3 och 9.1.6.

Uppgifter

Skriv ett program med de fem jämförelserna nedan, med en etikett efter var och en. **Förutsäg** `Z`, `C` och `S` för varje jämförelse i en tabell innan ni kör fram till etiketterna.

Jämförelse	r16	r17	Z	C	S	r16 är … r17
1. `cp r16, r17`	50	30				
2. `cp r16, r17`	50	50				
3. `cp r16, r17`	50	70				
4. `cpi r16, 50`	50	–				
5. `cp r16, r17`	−5	3				

1. Fyll i tabellen, och fyll i sista kolumnen med *större än*, *lika med* eller *mindre än*.
2. Kör fram till varje etikett och kontrollera flaggorna.
3. Kontrollera efter jämförelse 1–3 att `r16` fortfarande är 50. Varför ändras det inte, fast `cp` räknar ut en subtraktion?
4. Titta särskilt på jämförelse 5. Är −5 mindre eller större än 3? Svara både **utan** tecken, när `r16` läses som 251, och **med** tecken. Vilken flagga svarar på vilken fråga?

Frågor

- Vilka två flaggor räcker för att avgöra om två tal utan tecken är lika, om det första är större, eller om det är mindre?
- Varför behövs både `brlo` och `brlt`?

REDOVISA tabellen.

Del 12 Flödesplan och villkorliga hopp

Mål
Rita en flödesplan med beslut, och skriva det som ett program med jämförelser och villkorliga hopp.

Bakgrund
Avsnitt 9.1.4 och 9.1.5.

Specifikation
En termostat styr ett rum. Temperaturen i grader finns i `r16`. Port B styr två saker:

- **PB0**, en värmare, ska vara på när temperaturen är **under 20** grader;
- **PB1**, en fläkt, ska vara på när temperaturen är **25 grader eller mer**;
- mellan 20 och 24 grader ska båda vara av.

Programmet ska gå i en loop: det läser `r16`, sätter utgångarna, och börjar om. Då kan ni stoppa simuleringen, ändra värdet i `r16` i Processor Status, och köra vidare för att se utgångarna ändras.

Uppgifter
1. Rita flödesplanen. Den har två beslut. Märk varje beslut med Ja och Nej.
2. Skriv programmet. Ladda startvärdet 18 i `r16` **före** loopen, så att ett värde ni skriver in i Processor Status inte skrivs över.
3. **Förutsäg** `PORTB` för temperaturerna 18, 20, 24, 25 och 30. Testa alla fem i simulatorn.
4. Vilka av värdena är **gränsfall**, och varför är det de som är viktigast att testa?

Frågor
- Villkoret ”25 grader eller mer” används för fläkten. Hur skriver man ”mer än 24” med de hopp som finns?
- Vad händer med termostaten vid −5 grader? Se avsnitt 9.1.6.

REDOVISA flödesplanen och programmet, och visa minst tre temperaturer i simulatorn.

Del 14 Loop med visst antal varv

Mål
Skriva en loop som går ett bestämt antal varv med en räknare, och veta exakt hur många gånger varje instruktion i den körs.

Bakgrund
Avsnitt 9.2.1.

Specifikation

Processorn har ingen multiplikation i den här laborationen. Räkna ut $7 \cdot 3$ genom att addera 3 sju gånger, med en loop. Visa antalet varv som hittills gått på port B, så att det syns i I/O-fönstret.

Uppgifter

1. Rita flödesplanen. Använd ett register för resultatet, ett för talet som adderas, ett för varven som är kvar och ett för varven som har gått.
2. Skriv programmet med `.equ` för konstanterna, till exempel `.equ LAPS = 7` och `.equ TERM = 3`.
3. **Förutsäg** resultatet och `PORTB` när programmet är klart. Stega igenom minst två varv och se `PORTB` räkna, kör sedan till slutet med **Run To Cursor**.
4. Hur många gånger körs `brne`, och hur många av dem hoppar den?
5. Ändra `LAPS` till 0. **Förutsäg** resultatet innan ni kör. Förklara det ni ser.
6. Räkna ut $12 \cdot 25$ med samma program. Stämmer resultatet? Varför, eller varför inte?

Frågor

- Varför behövs ingen `cpi` före `brne`?

REDOVISA programmet och svaren på uppgift 4–6.

Del 15 Kort tidsfördröjning

Mål

Göra en tidsfördröjning med en loop, räkna ut dess längd i klockcykler, och kontrollera den med stoppuret.

Bakgrund

Avsnitt 9.2.3 och 9.2.4.

Programmet

```
main:
    sbi DDRB, 0                     ; PB0 is an output.
    sbi PORTB, 0                    ; The pulse starts.

delay_start:
    ldi r18, 200
delay_loop:
    dec r18
    brne delay_loop
delay_end:
    cbi PORTB, 0                    ; The pulse ends.
```

```
end:
    rjmp end
```

Programmet ger en puls på PB0: stiftet går högt, fördröjningen körs, och stiftet går lågt.

Uppgifter

1. Räkna ut fördröjningens längd, från `delay_start` till `delay_end`, i klockcykler och i mikrosekunder. Använd en tabell med en rad per instruktion, som i avsnitt 9.2.4.
2. Mät samma sak i simulatorn: brytpunkt på `delay_start` och `delay_end`, Frequency på 16 MHz, och stoppuret nollställt vid den första brytpunkten. Stämmer det?
3. Pulsen ska vara **30 µs**. Räkna ut hur många varv loopen ska gå, ändra programmet, och mät.
4. Vilken är den längsta fördröjning loopen kan ge? Räkna först, mät sedan.
5. Flytta den första brytpunkten till `sbi PORTB, 0`. **Förutsäg** hur mycket mätningen ändras.

Frågor

- Varför blir det exakt $3 \cdot n$ cykler och inte $3 \cdot n + 1$?

REDOVISA uträkningen och mätningen för 30 µs.

Del 16 Längre tidsfördröjning

Mål

Göra en längre fördröjning med två loopar i varandra, och träffa en önskad tid exakt.

Bakgrund

Avsnitt 9.2.5.

Specifikation

PB0 ska byta läge var **10:e millisekund**: hög i 10 ms, låg i 10 ms, och så vidare, för alltid. Byt läge genom att läsa `PORTB`, invertera bit 0 med `eor` och en mask, och skriva tillbaka. Fördröjningen mellan bytena görs med två loopar i varandra.

Uppgifter

1. Rita flödesplanen: en loop som byter läge på PB0 och sedan väntar, med fördröjningen som två loopar i varandra.
2. Välj OUTER och INNER med formeln $\text{OUTER} \cdot (3 \cdot \text{INNER} + 3)$ så att fördröjningen blir så nära 10 ms som möjligt. Skriv programmet och mät fördröjningen.
3. Lägg till en `nop` i den yttre loopen. Hur ändras formeln? Välj nya konstanter så att fördröjningen blir **exakt** 10 ms, 160 000 cykler. Mät.

4. Mät nu en hel halvperiod: från ett byte av PB0 till nästa. Den blir några cykler längre än fördröjningen. Hur många, och vilka instruktioner är det?
5. Kör programmet fritt med **F5** i några sekunder och stoppa. Hur lång tid har gått enligt stoppuret? Ungefär hur många gånger har PB0 bytt läge?

Frågor

- Skulle en lysdiod på PB0 synas blinka med den här fördröjningen på ett riktigt kort? Varför inte?

REDOVISA konstanterna för exakt 10 ms och mätningarna.

Del 17 Flervalssituationer och omarbetning av flödesplaner

Mål

Skriva ett program som väljer mellan flera alternativ, och arbeta om en flödesplan så att programmet får en väg in och en väg ut.

Bakgrund

Avsnitt 9.1.7 och 9.1.8.

Specifikation

Ett läge i `r16` väljer ett mönster på port B:

Läge	Mönster på port B
0	alla släckta
1	alla tända
2	de fyra nedre, PB0–PB3
3	de fyra övre, PB4–PB7
annat	varannan, `0101 0101`, som felindikering

Programmet går i en loop, så att läget kan ändras i Processor Status medan programmet står stilla.

Uppgifter

1. Rita en **första** flödesplan, hur ni vill. Skriv programmet efter den, och testa alla lägen, inklusive ett felläge.
2. Räkna i er flödesplan och i programmet: hur många rutor eller instruktioner skriver till port B? Hur många ställen går programmet tillbaka till loopens början från?
3. **Arbeta om** flödesplanen enligt avsnitt 9.1.8: låt varje alternativ bara välja mönster, och låt alla alternativ mötas i **en** ruta som skriver till port B.
4. Skriv om programmet efter den nya flödesplanen. Testa alla lägen igen. Programmet ska göra exakt samma sak som förut.

5. Lägg till läge 4, PB0 och PB7 tända. Hur många ställen i programmet behövde ändras, i den första versionen och i den omarbetade?

Frågor

- Varför är det viktigt att ha med alternativet ”annat” i flödesplanen?

REDOVISA båda flödesplanerna och det omarbetade programmet.

C.10 Del 18–23: Text, 16 bitar, inporten och subrutiner

Delarna hör till kapitel 10. Läs avsnitt 10.1 före del 18–20, avsnitt 10.2 före del 21 och avsnitt 10.3 före del 22–23. Instruktionerna finns på referensbladet i bilaga E.

Del 21 krävs för godkänt. Del 18–20 och 22–23 krävs för VG, och del 22 behövs dessutom för slutuppgiften i kapitel 11, så gör den även om ni inte siktar på VG.

Del 18 ASCII-koder för hexadecimala siffertecken

Mål

Göra om värden till de tecken som skriver ut dem, och tillbaka, och lägga text i minnet.

Bakgrund

ASCII-tabellen och luckan på sju mellan `'9'` och `'A'` finns i avsnitt 10.1.1–10.1.2. En byte blir två tecken med `swap` och `andi` enligt avsnitt 10.1.3, och `sts` beskrivs i avsnitt 10.1.5. Kom ihåg att `subi r17, -0x30` adderar `0x30`.

Uppgifter

1. **En siffra blir ett tecken.** Ladda `r17` med 7 och gör om det till tecknet `'7'` med en enda instruktion. *Förutsäg* `r17` hexadecimalt innan ni stegar.
2. **En hexadecimal siffra blir ett tecken.** Ladda `r18` med ett värde 0–15, kopiera det till `r19` och gör om `r19` till tecknet för siffran, med `cpi`, `brlo` och två `subi`. Fyll i tabellen **innan** ni kör programmet, och kontrollera sedan varje rad genom att byta värdet i `ldi`:

r18	r19, förutsagt	r19, i simulatorn
0x00		
0x09		
0x0A		
0x0B		
0x0F		

3. **En byte blir text.** Ladda `r16` med `0xA7`. Gör om den höga nibblen till ett tecken i `r20` och den låga till ett tecken i `r21`, och lägg dem på adress `0x0100` och `0x0101` med `sts`. Öppna fönstret **Memory**, välj **data IRAM** och gå till adress `0x0100`. Står texten `A7` i ASCII-kolumnen till höger? Prova också `0x3C`, `0xFF` och `0x00`.
4. **Tillbaka från tecken till värde.** Ladda `r22` med `'F'` och gör om det till värdet 15, enligt avsnitt 10.1.4. Prova också `'0'`, `'9'` och `'A'`.

Frågor

- Varför måste den höga nibblen flyttas ned med `swap` innan den kan göras om till ett tecken?
- Vad blir tecknet om man glömmer `andi r20, 0x0F` efter `swap`, för byten `0xA7`?

REDOVISA uppgift 3 och 4 i simulatorn, med texten synlig i Memory-fönstret.

Del 19 16-bitarsoperationer

Mål

Räkna med tal som inte får plats i en byte: addera, subtrahera, räkna och jämföra 16-bitarstal.

Bakgrund

Registerpar och `low()`/`high()` finns i avsnitt 10.1.6, addition och subtraktion med minnessiffra i avsnitt 10.1.7, `adiw`, `sbiw` och `movw` i avsnitt 10.1.8 och jämförelsen med `cpi`/`cpc` i avsnitt 10.1.9.

Uppgifter

1. **Addition.** Beräkna 1000 + 2000 med `r25:r24` och `r23:r22`, med `add` och `adc`. Kopiera resultatet till `r19:r18` med `movw`.
   Förutsäg först, med additionen uppställd för hand: vad blir `r24` och flaggan `C` efter `add`, och vad blir `r25` efter `adc`? Stega sedan och jämför i Processor Status.
2. **Subtraktion.** Beräkna 3000 – 1234 med `sub` och `sbc`, och lägg resultatet i `r21:r20`. *Förutsäg* båda bytena hexadecimalt. Blir det ett lån från den låga byten?
3. **En 16-bitars räknare.** Nollställ `r25:r24` och räkna upp det med `adiw` tills det är 1000, med `cpi` och `cpc` i loopen. Sätt en etikett direkt efter loopen.
   Förutsäg hur många cykler loopen tar, med samma metod som i kapitel 9. Mät sedan med brytpunkter före och efter loopen och fönstret Processor Status (avsnitt D.4.4). Stämmer siffrorna? Om inte: vilket varv räknade ni fel på?
4. **När 16 bitar inte räcker.** Ladda `r27:r26` med `0xFFFF` och addera 1 med `adiw`. *Förutsäg* resultatet och flaggorna `C` och `Z`.

Frågor

- Varför måste de låga bytena adderas först?
- Varför behövs registret med `high(1000)` i uppgift 3, när den låga byten kan jämföras direkt med `cpi`?

REDOVISA uppgift 1–4, och er förutsägelse och mätning i uppgift 3.

Del 20 Lång tidsfördröjning

Mål

Konstruera en fördröjning på en halv sekund, exakt på cykeln, och få en lysdiod att blinka en gång per sekund.

Bakgrund

Metoden, en 16-bitars inre loop i en yttre 8-bitars loop, och formeln `M(4N + 4)` finns i avsnitt 10.1.10. Exemplet där blinkar med en sekunds halvperiod; här ska det gå dubbelt så fort.

Programmet

Fyll i de två konstanterna.

```
.include "m328Pdef.inc"

.equ OUTER_TURNS = ?                ; Work these two out before you run anything.
.equ INNER_TURNS = ?

.org 0x0000
    rjmp main

main:
    sbi DDRB, 5                     ; PB5, Arduino pin 13, is an output.
    ldi r17, 0b00100000             ; The mask that selects bit 5.

loop:
    in r16, PORTB                   ; Toggle PB5.
    eor r16, r17
    out PORTB, r16

delay_begin:
    ldi r20, OUTER_TURNS
delay_outer:
    ldi r24, low(INNER_TURNS)
    ldi r25, high(INNER_TURNS)
delay_inner:
    sbiw r24, 1
    brne delay_inner
    dec r20
    brne delay_outer
delay_end:
    rjmp loop
```

Uppgifter

1. **Räkna först.** Hur många cykler är en halv sekund vid 16 MHz? Välj `INNER_TURNS` så att ett varv i den yttre loopen tar 200 000 cykler, och räkna ut `OUTER_TURNS`. Skriv upp uträkningen.
2. **Mät.** Sätt stoppurets frekvens till 16 MHz, sätt en brytpunkt på `delay_begin` och en på `delay_end`, och mät tiden mellan dem. *Förutsäg* stoppurets värde innan ni kör.
3. **Hela perioden.** Hur lång är en hel blinkperiod, från en tändning till nästa, räknat i cykler? Varför är den inte exakt en sekund? Hur mycket fel är den, i mikrosekunder?
4. **En annan tid.** Ändra till en fjärdedels sekund. Vilken av de två konstanterna ändrar ni, och till vad?

> **Tips.** Simulatorn behöver en stund för att köra åtta miljoner cykler. Använd **Continue** (**F5**) mellan brytpunkterna, inte **Step Into**.

Frågor

- Varför är det enklare att välja ett ”jämnt” antal cykler för den yttre loopen först, och räkna ut antalet varv sedan?
- Vad händer om `OUTER_TURNS` sätts till 0?

REDOVISA uträkningen och mätningen.

Del 21 Inporten PORTD (PIND)

Mål

Läsa knappar på port D och styra lysdioder på port B utifrån dem, och bygga logiska funktioner i mjukvara.

Bakgrund

Pull-up-motståndet och varför en nedtryckt knapp läses som 0 finns i avsnitt 10.2.2, att läsa hela porten i avsnitt 10.2.3 och `sbic`/`sbis` i avsnitt 10.2.4. Så simuleras en knapp: avsnitt D.4.5.

I hela delen sitter knapparna mellan stiftet och jord, och de inbyggda pull-uparna används. En **tom** ruta i `PIND` är alltså en **nedtryckt** knapp.

Uppgifter

1. **Knapparna på lysdioderna.** Skriv av programmet `buttons_to_leds.asm` från avsnitt 10.2.3. Kör till loopen, stoppa, och ändra `PIND` för hand. *Förutsäg* `PORTB` för varje rad innan ni stegar:

`PIND`	Nedtryckta knappar	`PORTB`, förutsagt	`PORTB`, i simulatorn
`0b11111111`			
`0b11111011`			
`0b11110011`			
`0b01111110`			

2. **En knapp, en lysdiod.** Skriv ett program där lysdioden på PB5 lyser så länge knappen på PD2 hålls nedtryckt, med `sbic` eller `sbis` och utan att läsa hela porten.
3. **AND och OR i mjukvara.** Två knappar, på PD2 och PD3. Lysdioden på PB0 ska lysa när **båda** är nedtryckta, och lysdioden på PB1 när **minst en** är det. Det är samma två funktioner som seriekopplingen och parallellkopplingen i Labb 1 (bilaga A).

Rita en flödesplan först. Fyll sedan i sanningstabellen, med förutsägelsen före simuleringen:

PD2	PD3	PB0 (AND)	PB1 (OR)
släppt	släppt		
släppt	nedtryckt		
nedtryckt	släppt		
nedtryckt	nedtryckt		

4. **Extra.** Lägg till lysdioden på PB2, som ska lysa när **exakt en** av knapparna är nedtryckt. Vilken grind från kapitel 2 är det? Vilken koppling i Labb 1 gjorde samma sak?

Frågor

- Vad läser programmet på en ingång utan pull-up, när knappen är släppt?
- I uppgift 3 behövs `com` och `andi` innan jämförelsen. Vad gör var och en av dem, och vad går fel om man hoppar över `andi`?
- Varför är det enklare att ändra funktionen i uppgift 3 än i Labb 1?

REDOVISA uppgift 3, med sanningstabellen, i simulatorn.

Del 22 Subrutiner

Mål

Flytta en fördröjning till en subrutin, se returadressen på stacken, och skriva en subrutin som tar en parameter.

Bakgrund

`rcall`, `ret`, stacken och stackpekaren finns i avsnitt 10.3.2–10.3.3, subrutinen `delay_ms` med parameter i avsnitt 10.3.4, och en subrutin som anropar en annan i avsnitt 10.3.7.

Från och med den här delen ska varje program som använder `rcall` börja med att initiera stackpekaren, med de fyra raderna i avsnitt 10.3.3.

Uppgifter

1. **Fördröjningen blir en subrutin.** Ta programmet från del 20. Flytta fördröjningen, från `delay_begin` till och med `brne delay_outer`, till en subrutin `delay_500ms` som slutar med `ret`, och anropa den med `rcall` där fördröjningen stod. Lägg till initieringen av `SP`. Programmet ska blinka precis som förut.
2. **Returadressen.** *Förutsäg* `SP` före `rcall`, i början av subrutinen och efter `ret`. Leta upp adressen till instruktionen efter `rcall` i listfilen (`.lss`, avsnitt 7.2.7). Stega in i subrutinen

med **F11** och läs de två översta bytena i stacken i fönstret Memory (**data IRAM**, adress `0x08FE–0x08FF`). Är det returadressen? Vilken byte ligger var?

3. **En parameter.** Skriv av `delay_ms` från avsnitt 10.3.4. Skriv om `delay_500ms` så att den laddar `r24` med 250 och anropar `delay_ms` två gånger. Glöm inte att `delay_500ms` själv ändrar `r24`, och alltså ska spara det.
4. **Järnvägsövergången.** Ändra huvudprogrammet så att lysdioderna på PB0 och PB1 blinkar växelvis, en halv sekund var, som vid en järnvägsövergång.
5. **Mät ett anrop.** *Förutsäg* hur många cykler ett anrop av `delay_500ms` tar, från `rcall` till instruktionen efter, med formeln för `delay_ms` och cyklerna för `rcall`, `push`, `ldi`, `pop` och `ret`. Mät sedan med stoppuret.
6. **Den djupaste punkten.** Stoppa programmet mitt i `delay_ms`, när den har anropats från `delay_500ms`. *Förutsäg* `SP` först, och kontrollera. Hur många byte ligger på stacken, och vad är var och en?

Frågor

- Vad skulle hända om `delay_ms` inte sparade `r24`?
- `rcall` kostar 3 cykler och `ret` 4. Är det ett argument mot att använda subrutiner?

REDOVISA uppgift 2, 4 och 5.

Del 23 Använda stacken som lagringsplats

Mål

Använda `push` och `pop` för att spara register, förstå varför ordningen spelar roll, och se vad som händer när stacken inte stämmer.

Bakgrund

`push`, `pop` och sist in, först ut finns i avsnitt 10.3.5, och kursens regel om att spara det man ändrar i avsnitt 10.3.6.

Programmet

Huvudprogrammet räknar i `r16` hur många gånger det har anropat `flash_all`, och slutar efter tre gånger. Subrutinen `flash_all` tänder alla lysdioder på port B i 100 ms och släcker dem i 100 ms, och använder själv `r16` till något helt annat. Den saknar `push` och `pop`, med flit.

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi r16, high(RAMEND)
```

```
    out SPH, r16
    ldi r16, low(RAMEND)
    out SPL, r16

    ldi r16, 0xFF
    out DDRB, r16                   ; Port B: all outputs.

    clr r16                         ; r16 counts the flashes.
loop:
    inc r16
    rcall flash_all
    cpi r16, 3
    brne loop
    out PORTB, r16                  ; Show the count.
end:
    rjmp end

flash_all:
    ldi r16, 0xFF                   ; All LEDs on ...
    out PORTB, r16
    ldi r24, 100
    rcall delay_ms                  ; ... for 100 ms ...
    clr r16                         ; ... and off ...
    out PORTB, r16
    ldi r24, 100
    rcall delay_ms                  ; ... for 100 ms.
    ret

; Copy delay_ms from part 22 here.
```

Uppgifter

1. **Byt plats med stacken.** Ladda `r20` med `0x55` och `r21` med `0xAA` i ett eget litet program. Lägg dem på stacken med `push r20`, `push r21`, och hämta dem med `pop r20`, `pop r21`. *Förutsäg* värdena i `r20` och `r21` efteråt, och `SP` efter varje instruktion.
2. **Felet.** Kör programmet ovan. *Förutsäg* först vad det gör. Hur många gånger blinkar det? Stega igenom ett varv i loopen och följ `r16`. Förklara vad som går fel.
3. **Rättningen.** Lägg till `push` och `pop` i `flash_all` så att subrutinen följer kursens regel. Vilka register måste sparas? Programmet ska nu blinka tre gånger och sedan visa talet 3 på lysdioderna.
4. **Djupet.** Stoppa programmet mitt i den första `delay_ms`. *Förutsäg* `SP`, och kontrollera. Hur många byte ligger på stacken, och vad är var och en?
5. **Fel ordning.** Byt plats på de två `pop` i `flash_all`, så att de inte längre står i omvänd ordning mot `push`. Vad händer? Förklara med stacken.

6. **En pop för lite.** Ta bort en av `pop`-instruktionerna. Stega fram till `ret` i `flash_all` och läs `SP`. Stega en gång till: vart hoppar programmet? Förklara med stacken.

Frågor

- Varför måste `pop` göras i omvänd ordning mot `push`?
- Felet i uppgift 6 syns tydligare än felet i uppgift 5. Varför är det i själva verket en fördel?

REDOVISA uppgift 2, 3 och 4.

C.11 Slutuppgift: trafikljuset och övergångsstället

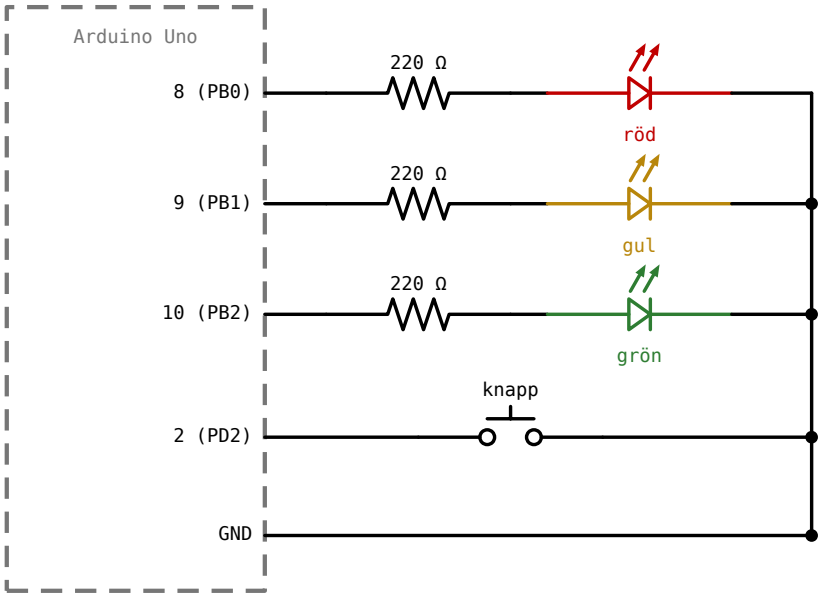
Slutuppgiften hör till kapitel 11, och är den del av Labb 3 där programmet till sist lämnar simulatorn och styr något på riktigt: tre lysdioder och en knapp på ett Arduino Uno-kort. Del A krävs för godkänt, och del B för VG.

Läs avsnitt 11.1 om lysdioder, knappar och kopplingen, och avsnitt 11.2, där ett enklare ljus, ett vägarbetsljus, tas hela vägen från flödesplan till program. Hur programmet förs över till kortet står i avsnitt D.5.

C.11.1 Kopplingen

Samma koppling används i båda delarna. Varje lysdiod har ett eget förkopplingsmotstånd, och knappen kopplas mellan stift 2 och jord.

Funktion	Arduino-stift	Port och bit
Röd lysdiod	8	PB0
Gul lysdiod	9	PB1
Grön lysdiod	10	PB2
Knapp, till jord	2	PD2
Jord	GND	–




```
Varje lysdiod har ett eget motstånd på 220-330 Ω och går till jord (GND). Knappen
kopplar stift 2 till jord; den interna pull-upen håller stiftet högt när knappen är
släppt. Stift 0 och 1 används av USB-kabeln och lämnas lediga.
```

Figur C.1 Kopplingsschemat: tre lysdioder med förkopplingsmotstånd på stift 8, 9 och 10, och en knapp från stift 2 till jord.

Lysdiodens långa ben, anoden, ska vara vänt mot motståndet och stiftet, och det korta, katoden, mot jord. Koppla med USB-kabeln urkopplad, och kontrollera kopplingen mot tabellen innan kabeln sätts i.

C.11.2 De färdiga subrutinerna

Båda delarna använder fördröjningar på hela sekunder. Ni behöver inte skriva dem själva: kopiera in de två subrutinerna nedan sist i programmet, och konstanten `QUARTER_MS` överst. De förklaras i avsnitt 11.2.4, och `delay_ms` är samma subrutin som i del 22.

```
.equ QUARTER_MS = 250                ; 250 ms. Set it to 1 to run about 250 times
                                     ; faster in the simulator; set it back before
                                     ; flashing.
```

```
;-------------------------------------------------------------------------------
; delay_s: Waits r24 seconds (1-63), as 4 x r24 calls of delay_ms with 250 ms each.
; A call takes 16 000 084 x r24 + 18 cycles: about 5 microseconds too long per ↩
    ↪ second.
;-------------------------------------------------------------------------------
delay_s:
    push r24                        ; Save the registers this subroutine changes.
    push r25
    mov r25, r24
    lsl r25                         ; r25 = 2 x r24 ...
    lsl r25                         ; ... = 4 x r24: the number of quarter seconds.
    ldi r24, QUARTER_MS             ; A quarter of a second is 250 ms.
delay_s_loop:
    rcall delay_ms
    dec r25
    brne delay_s_loop
    pop r25                         ; Restore them, in the opposite order.
    pop r24
    ret

;-------------------------------------------------------------------------------
; delay_ms: Waits r24 milliseconds (1-255) at 16 MHz.
; A call takes 16 000 x r24 + 18 cycles, the rcall and the ret included.
;-------------------------------------------------------------------------------
delay_ms:
    push r24                        ; Save the registers this subroutine changes.
    push r26
    push r27
delay_ms_outer:
    ldi r26, low(3999)              ; X = r27:r26 = 3999.
    ldi r27, high(3999)
delay_ms_inner:
    sbiw r26, 1                     ; 4 cycles per turn, 3 the last time.
    brne delay_ms_inner
    dec r24                         ; One more millisecond done.
    brne delay_ms_outer
    pop r27                         ; Restore them, in the opposite order.
    pop r26
    pop r24
    ret
```

Så används de: ladda `r24` med antalet sekunder och anropa `delay_s`. `ldi r24, 3` följt av `rcall delay_s` väntar tre sekunder, och `r24` är fortfarande 3 efteråt.

I simulatorn tar varje simulerad sekund lång tid att köra. Sätt `QUARTER_MS` till 1 medan ni testar programmet där: då går varje ”sekund” på ungefär 4 ms, och hela ljuscykeln syns på några ögonblick. **Sätt tillbaka den till 250 innan programmet förs över till kortet.**

Del A Trafikljuset

Mål

Styra ett trafikljus med tre lysdioder genom en fast ljuscykel, i simulatorn och på kortet.

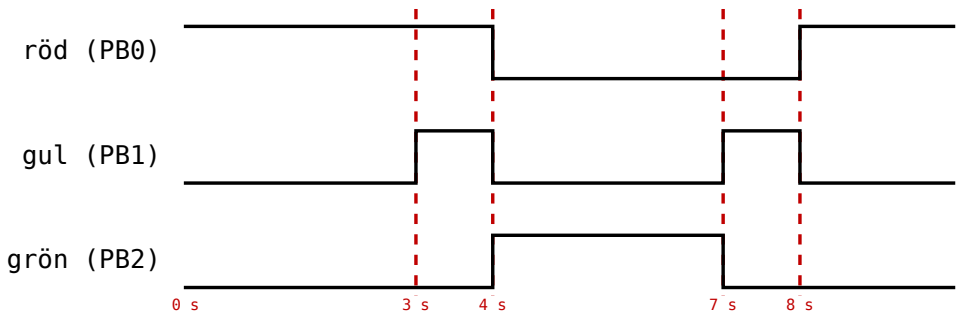
Bakgrund

Vägarbetsljuset i avsnitt 11.2 har två tillstånd; trafikljuset har fyra. Metoden är densamma: skriv upp tillstånden och deras tider, rita flödesplanen, och skriv sedan programmet ett tillstånd i taget.

Specifikation

Trafikljuset går igenom fyra tillstånd, i den här ordningen, och börjar sedan om:

Tillstånd	Röd (PB0)	Gul (PB1)	Grön (PB2)	Tid
Stopp	tänd	släckt	släckt	3 s
Klart att köra	tänd	tänd	släckt	1 s
Kör	släckt	släckt	tänd	3 s
Stopp strax	släckt	tänd	släckt	1 s



Röd 3 s, röd och gul 1 s, grön 3 s, gul 1 s. Efter 8 s börjar cykeln om från början.

Figur C.2 Tidsdiagram för trafikljuset: en ljuscykel är 8 s.

Uppgifter

1. **Flödesplan.** Rita flödesplanen för programmet, med ett symbolpar per tillstånd: tänd rätt lampor, vänta rätt tid. REDOVISA den innan ni börjar skriva kod.
2. **Bitmönstren.** Skriv upp vilket värde `PORTB` ska ha i vart och ett av de fyra tillstånden, binärt och hexadecimalt. Använd gärna `.equ RED = 0` och skrivsättet `(1 << RED)`, så att programmet säger vad det gör.
3. **Programmet.** Skriv programmet utifrån programmallen, med initiering av `SP`, `DDRB`, och de färdiga subrutinerna.
4. **Simulera.** Sätt `QUARTER_MS` till 1 och kör i simulatorn. Sätt en brytpunkt på varje `out PORTB, r16` och kontrollera i I/O-fönstret att rätt lampor tänds, i rätt ordning. Kontrollera också med stoppuret att tiderna står i rätt förhållande till varandra: 3, 1, 3, 1.
5. **På kortet.** Sätt tillbaka `QUARTER_MS` till 250, bygg, och för över programmet till kortet enligt avsnitt D.5.
6. **Mät.** Ta tid på tio hela cykler med stoppuret i en mobiltelefon. *Förutsäg* tiden först, med formeln för `delay_s`. Hur nära kommer ni? Vad begränsar noggrannheten i er mätning, och vad begränsar noggrannheten i programmet?

Frågor

- Vad händer om `delay_s` inte sparade `r24`? Hade programmet märkt det?
- Varför tänds röd och gul samtidigt i det andra tillståndet? Vad säger det till en bilist?

REDOVISA flödesplanen, simuleringen och trafikljuset på kortet.

Del B Övergångsstället

Mål

Låta en knapp styra när ljuscykeln körs: ett övergångsställe där bilarna har grönt tills en fotgängare trycker på knappen.

Bakgrund

Att vänta på en knapp med `sbic` finns i avsnitt 10.2.5, och varför en knapp studsar i avsnitt 11.1.4. Subrutiner och kursens regel om register finns i avsnitt 10.3.

Specifikation

- Bilarna har **grönt**, så länge ingen trycker på knappen.
- När knappen trycks går ljuset igenom:

Tillstånd	Röd (PB0)	Gul (PB1)	Grön (PB2)	Tid
Stopp strax	släckt	tänd	släckt	1 s
Stopp: fotgängarna går	tänd	släckt	släckt	4 s
Klart att köra	tänd	tänd	släckt	1 s

- Därefter blir det grönt igen, och programmet väntar på nästa tryck.
- Knappen är aktivt låg, med den inbyggda pull-upen: PD2 som ingång, med pull-up.
- Programmet ska använda minst en egen subrutin utöver de färdiga, och varje subrutin ska följa kursens regel: spara varje register den ändrar.

Uppgifter

1. **Flödesplan.** Rita flödesplanen, med beslutet ”knappen nedtryckt?” och de tre tillstånden. REDOVISA den innan ni skriver kod.
2. **En egen subrutin.** De tre tillstånden gör samma sak med olika värden: tänd vissa lampor, vänta en viss tid. Skriv en subrutin `show` som tänder lamporna i `r16` och väntar det antal sekunder som står i `r24`. Vilka register ändrar den? Måste något av dem sparas?
3. **Programmet.** Skriv programmet. Initiera `SP`, `DDRB`, `DDRD` och pull-upen på PD2.
4. **Simulera.** Sätt `QUARTER_MS` till 1. Kör programmet, stoppa det medan det väntar på knappen, och ”tryck” genom att tömma rutan för bit 2 i `PIND`. *Förutsäg* vilka lampor som ska tändas i vilken ordning, och kontrollera med brytpunkter.
5. **På kortet.** Sätt tillbaka `QUARTER_MS` till 250 och för över programmet till kortet. Prova ett kort tryck, ett långt tryck, och ett tryck medan ljuset redan är rött.

Frågor

- Knappen studsar när den trycks. Varför ställer det inte till något här, när det gjorde det för knapptryckräknaren i avsnitt 11.1.4?
- Vad händer om någon håller knappen nedtryckt hela tiden? Är det ett problem för bilarna?
- Ett tryck medan ljuset är rött gör ingenting. Varför inte? Hur skulle programmet behöva ändras för att komma ihåg ett sådant tryck?

Extra

Frivilliga utökningar, för den som vill vidare:

- **Minsta tid på grönt.** Riktiga övergångsställen ger bilarna grönt i minst några sekunder innan ett nytt tryck får verkan. Låt bilarna alltid ha grönt i minst 5 s efter varje fotgängarfas.
- **Fotgängarnas ljus.** Lägg till en röd och en grön lysdiod för fotgängarna på PB3 och PB4 (stift 11 och 12). Fotgängarnas gröna ska lysa bara under de 4 sekunderna med rött för bilarna.
- **Vänta-lampan.** Tänd en lampa, till exempel PB5 (stift 13, lysdioden på kortet), när knappen har tryckts, och släck den när fotgängarna får grönt.

REDOVISA flödesplanen, simuleringen och övergångsstället på kortet.

BILAGA D

Microchip Studio

Microchip Studio är den utvecklingsmiljö där programmen i Labb 3 (bilaga C) skrivs, assembleras och körs. Den här bilagan är referensen för allt verktygsarbete i boken:

- avsnitt D.1–D.4 behövs från kapitel 7: installation, projekt, assemblering och simulering;
- avsnitt D.5 behövs först i kapitel 11, när programmet ska köras på ett riktigt Arduino Uno-kort.

Microchip Studio finns bara för Windows. Menyernas namn nedan är de engelska, eftersom programmet bara finns på engelska.

D.1 Installera Microchip Studio

1. Ladda ned installationsprogrammet från Microchips webbplats, `https://www.microchip.com/en-us/tools-resources/develop/microchip-studio`. Välj webbinstallationen (*web installer*) om du inte vet vilken du ska ta.
2. Kör installationen. När du får välja arkitekturer räcker **AVR**; SAM och UC3 behövs inte.
3. Godkänn installationen av drivrutiner om du tillfrågas, och starta om datorn när installationen är klar.

Installationen tar en stund och kräver flera gigabyte. Gör den hemma före det första mikrodatorpasset, inte i början av passet.

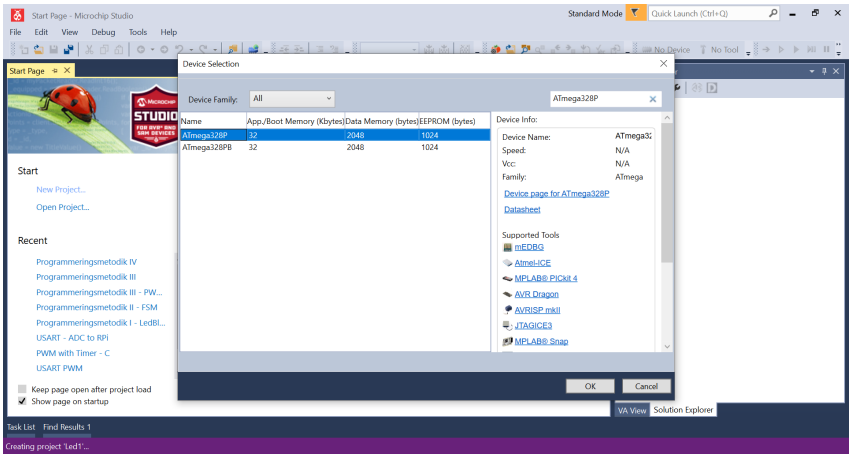
> **Om installationsfilen inte går att öppna.** Windows 11 kan blockera okända installationsprogram med funktionen *Smart App Control*. Sök efter Smart App Control i Windows sökruta, stäng av den, och starta om datorn.

D.2 Skapa ett assemblerprojekt

Varje program i Labb 3 är ett eget projekt. Ett projekt är en katalog med en projektfil och en källfil, `main.asm`.

1. Välj **File → New → Project...**

2. Välj **Assembler** i listan till vänster, och sedan **AVR Assembler Project**.
3. Ge projektet ett namn som säger vilken del av labben det hör till, till exempel `lab3_06`, och välj var det ska sparas. Klicka **OK**.
4. I dialogen **Device Selection**, sök efter och välj **ATmega328P**. Klicka **OK**.



Figur D.1 Dialogen Device Selection med ATmega328P markerad.

Studio skapar projektet och öppnar `main.asm` med ett litet exempelprogram. Ersätt det med kursens programmall (avsnitt C.3), eller med det program uppgiften anger.

Varje program i kursen börjar med raden:

```
.include "m328Pdef.inc"
```

Den läser in definitionsfilen för ATmega328P, som ger namnen `PORTB`, `DDRB`, `PIND`, `RAMEND` och alla andra registernamn. Studio läser ofta in filen automatiskt, men raden gör att programmet fungerar oavsett inställningar, och det gör ingen skada att filen läses in två gånger.

D.3 Assemblera

Välj **Build → Build Solution**, eller tryck **F7**.

Assemblern översätter programmet till maskinkod. Resultatet visas i fönstret **Output** längst ned:

```
Assembly complete, 0 errors. 0 warnings
...
========== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped ==========
```

Om något är fel visas felen i fönstret **Error List** (**View → Error List** om det inte syns). Varje fel anger fil och radnummer. Dubbelklicka på felet så hoppar editorn till raden.

Tre fel som alla gör i början:

- **`Invalid register`** på en rad med `ldi`: `ldi` kan bara ladda `r16`–`r31` (avsnitt 7.1).
- **`Undefined symbol`**: en etikett eller ett registernamn är felstavat, eller raden `.include` saknas.
- **`Syntax error`**: ofta ett kommatecken som saknas mellan operanderna, `ldi r16 5` i stället för `ldi r16, 5`.

Rätta **det första felet först** och bygg om. Ett fel följs ofta av flera följdfel som försvinner av sig själva.

D.4 Simulera

Simulatorn kör ditt program i en modell av ATmega328P, instruktion för instruktion, och visar register, flaggor och portar medan det går. Nästan hela Labb 3 görs i simulatorn.

D.4.1 Välj simulatorn

Första gången i varje projekt:

1. Välj **Project → Properties** (eller **Alt+F7**).
2. Välj fliken **Tool**.
3. Välj **Simulator** under *Selected debugger/programmer*.
4. Spara med **Ctrl+S** och stäng fliken.

D.4.2 Stega genom programmet

Välj **Debug → Start Debugging and Break** (**Alt+F5**). Programmet assembleras, laddas in i simulatorn och stannar på första instruktionen, markerad med en gul pil. Den gula pilen pekar på den instruktion som står på tur, **inte** den som just har körts.

Kommando	Tangent	Gör
Step Into	F11	Kör en instruktion. Följer med in i en subrutin.
Step Over	F10	Kör en instruktion. Kör en hel subrutin som ett steg.
Run To Cursor	Ctrl+F10	Kör tills programmet når raden där markören står.
Continue	F5	Kör fritt, tills en brytpunkt nås.
Break All	Ctrl+F5	Stoppar ett program som kör fritt.
Stop Debugging	Ctrl+Shift+F5	Avslutar simuleringen.

D.4.3 Fönstren du behöver

Öppnas under **Debug → Windows** medan simuleringen pågår.

- **Processor Status**: de 32 registren `R00`–`R31`, statusregistret `SREG` med en ruta per flagga, programräknaren (*Program Counter*), stackpekaren (*Stack Pointer*), och två räknare som mäter tid: *Cycle Counter* och *Stop Watch*. Värden som ändrades i det senaste steget visas i rött. Medan programmet står still kan du ändra ett registers värde genom att dubbelklicka på det och skriva in ett nytt, vilket är ett snabbt sätt att prova samma program med andra indata.
- **I/O**: alla I/O-register, grupperade per enhet. Under **PORTB** finns `DDRB`, `PORTB` och `PINB`, med en ruta per bit. En fylld ruta är en etta.
- **Memory**: minnet byte för byte. Välj **data IRAM** för att se SRAM, där stacken ligger, och **prog FLASH** för att se programmet som maskinkod.
- **Watch**: värden du själv väljer att bevaka.

Värdena visas som standard hexadecimalt. Högerklicka i fönstret för att växla till decimalt om det finns, men vänj dig vid hexadecimal: det är så datablad och instruktionslistor skriver.

D.4.4 Brytpunkter och cykelräknaren

En **brytpunkt** stoppar ett program som kör fritt. Klicka i den grå marginalen till vänster om en rad, eller ställ markören på raden och tryck **F9**. En röd prick markerar brytpunkten.

Cycle Counter räknar klockcykler sedan programmet startade, och **Stop Watch** visar samma sak som tid. Stoppuret räknar tiden utifrån fältet **Frequency**, som ska stå på **16 MHz**, samma klockfrekvens som ett Arduino Uno-kort har. Dubbelklicka på värdet för att ändra det.

Så mäter du hur lång tid en del av programmet tar, till exempel en fördröjning i kapitel 9:

1. Sätt en brytpunkt före och en efter den del du vill mäta.
2. Kör till den första brytpunkten.
3. Högerklicka på **Stop Watch** och välj **Reset Stopwatch**, eller skriv upp värdet.
4. Kör vidare till den andra brytpunkten, och läs av.

D.4.5 Simulera en ingång

I simulatorn finns ingen knapp att trycka på. I stället ändrar du ingångens värde för hand, i fönstret **I/O**:

1. Stoppa programmet, med en brytpunkt eller **Break All**.
2. Välj **PORTD** i I/O-fönstret.
3. Klicka på den bit i **PIND** du vill ändra. En fylld ruta är en etta, en tom en nolla.
4. Stega eller kör vidare. Programmet läser det nya värdet nästa gång det läser `PIND`.

En knapp kopplad mot jord med pull-up läses som `1` när den är släppt och `0` när den är nedtryckt, se avsnitt 10.2. I simulatorn betyder det: töm rutan för att ”trycka ned” knappen.

D.5 Programmera ett Arduino-kort från Microchip Studio

Hittills har programmet bara körts i simulatorn. I kapitel 11 ska det köras på ett riktigt Arduino Uno-kort. Det här avsnittet är en engångsinställning som gör det till ett menyval i Studio.

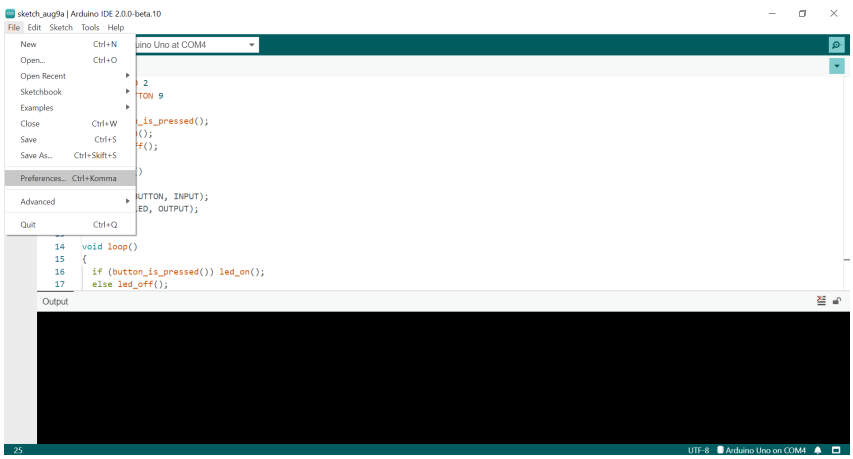
D.5.1 Varför det här arbetsflödet

Arduino Uno-kortet har en **bootloader**, ett litet program i toppen av flashminnet som tar emot ett nytt program över USB-kabeln. Programmet som skickar det heter `avrdude`, och det följer med Arduino IDE. Arduino IDE vet dessutom exakt vilket kommando som behövs för just ditt kort och din COM-port.

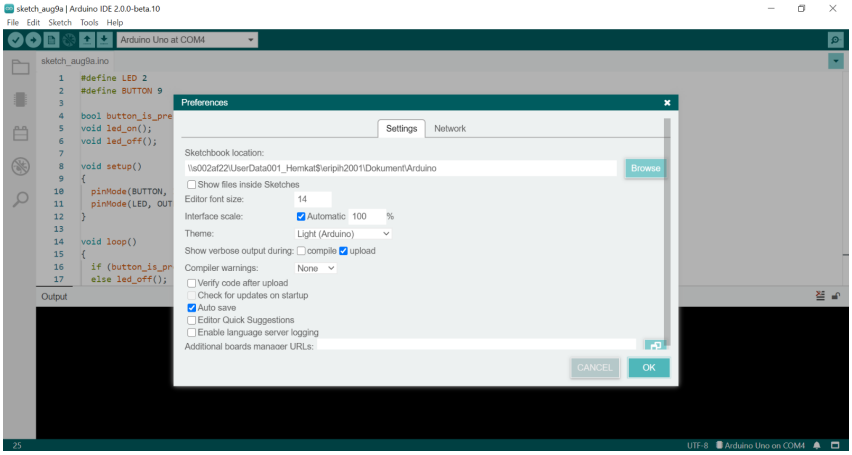
Så tricket är att låna det kommandot från Arduino IDE, och peka det på den hexfil Microchip Studio skapar i stället för på Arduinos egen. Ingen extra programmerare behövs.

D.5.2 Hämta avrdude-kommandot ur Arduino IDE

1. Installera Arduino IDE, `https://www.arduino.cc/en/software`, och starta den.
2. Slå på utförlig utskrift vid uppladdning: **File → Preferences**, och kryssa i **Show verbose output during: Upload**.

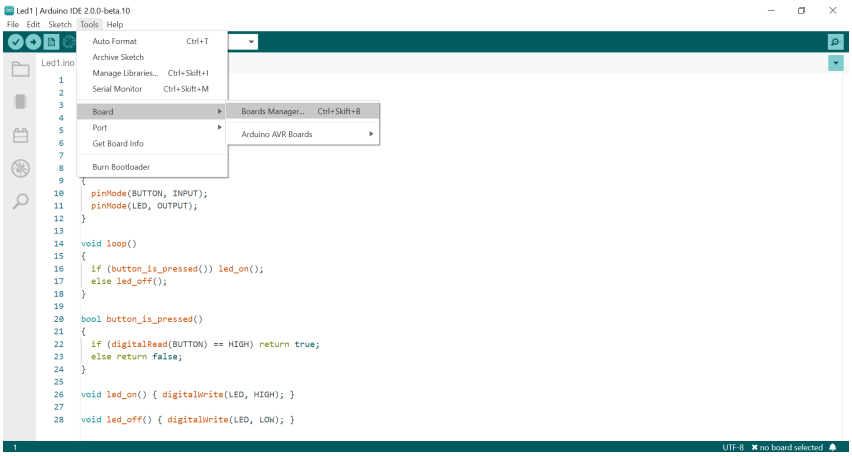


Figur D.2 Arduino IDE:s meny File med Preferences markerat.

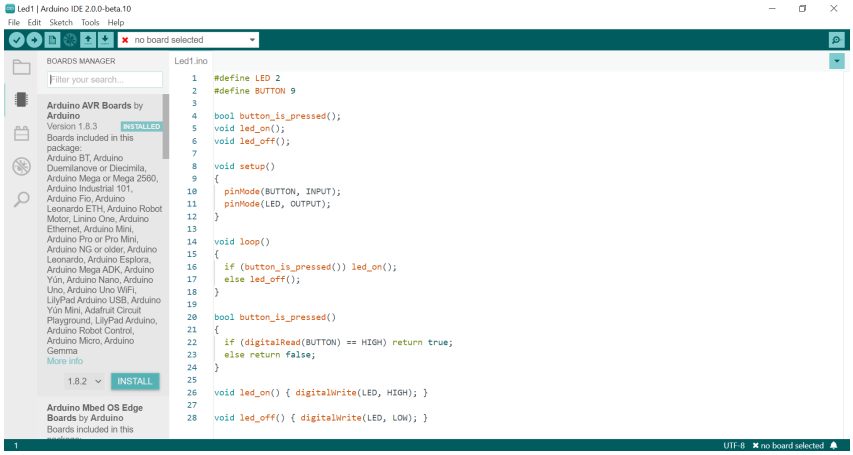


Figur D.3 Arduino IDE:s inställningar, med rutan för utförlig utskrift vid uppladdning ikryssad.

3. Installera stödet för AVR-korten om det inte redan finns: **Tools → Board → Boards Manager**, och installera **Arduino AVR Boards**.

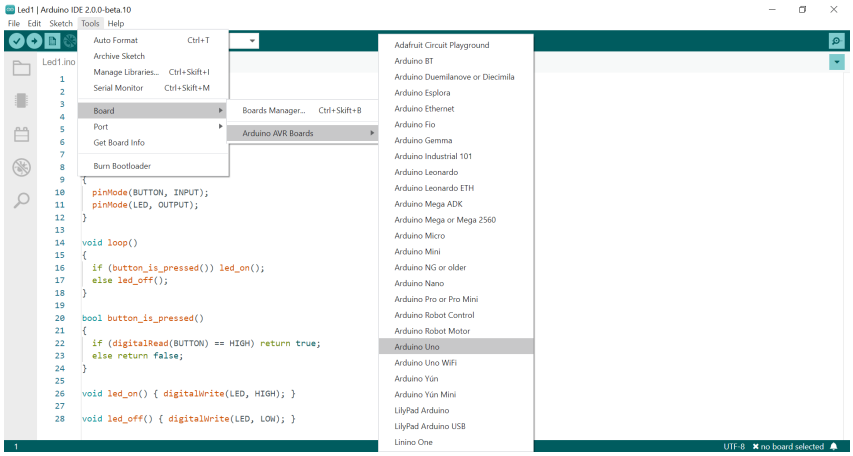


Figur D.4 Arduino IDE:s meny Tools → Board med Boards Manager markerat.

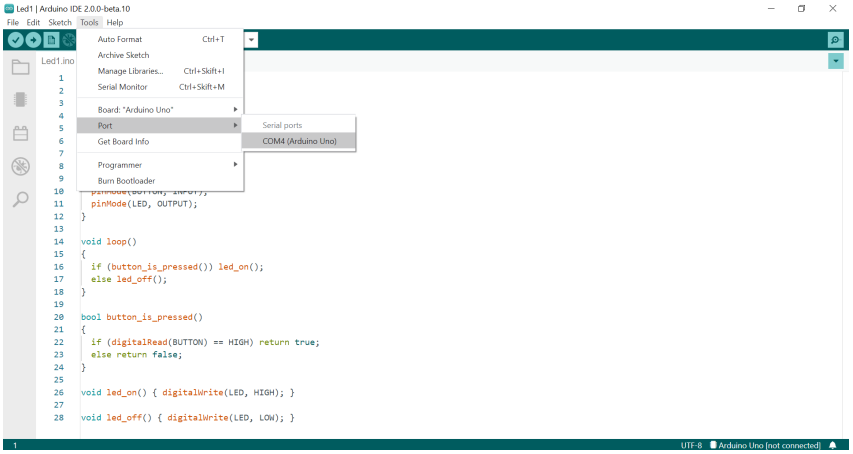


Figur D.5 Boards Manager, där Arduino AVR Boards redan är installerat.

4. Anslut kortet med USB-kabeln. Välj kortet under **Tools → Board → Arduino AVR Boards → Arduino Uno**, och dess port under **Tools → Port**, till exempel COM4. Numret spelar ingen roll, men skriv upp det.

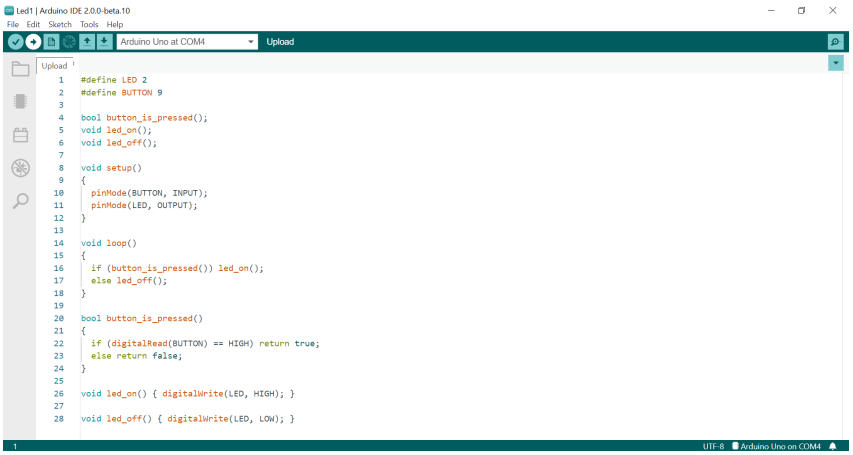


Figur D.6 Arduino IDE:s meny Tools → Board med Arduino Uno vald.



Figur D.7 Arduino IDE:s meny Tools → Port med COM4 vald.

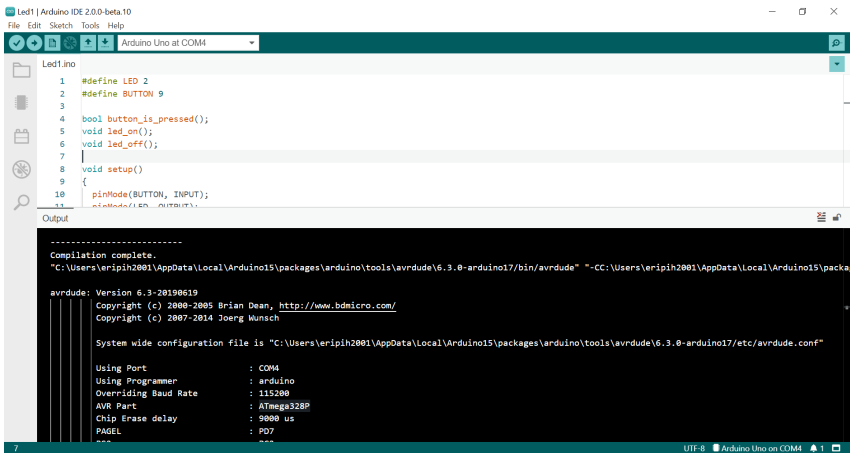
5. Öppna exemplet **File → Examples → 01.Basics → Blink** och klicka på **Upload**, pilen uppe till vänster. Lysdioden på kortet börjar blinka.



Figur D.8 Arduino IDE:s knapp Upload, en pil högerut uppe till vänster.

6. Leta upp raden som startar `avrdude` i utskriften längst ned, och kopiera den. Den ser ut ungefär så här (med ditt användarnamn och din COM-port):

```
"C:\Users\...\Arduino15\packages\arduino\tools\avrdude\6.3.0-arduino17/bin/avrdude"
"-CC:\Users\...\Arduino15\packages\arduino\tools\avrdude\6.3.0-arduino17/etc/
    avrdude.conf"
-v -V -patmega328p -carduino "-PCOM4" -b115200 -D
"-Uflash:w:C:\Users\...\Temp\arduino-sketch-.../Blink.ino.hex:i"
```

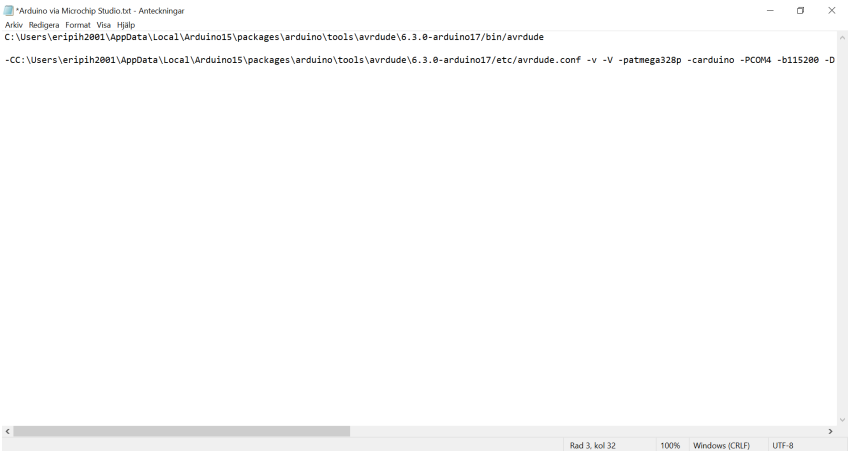


Figur D.9 Arduino IDE:s utskrift efter uppladdningen, med avrdude-kommandot.

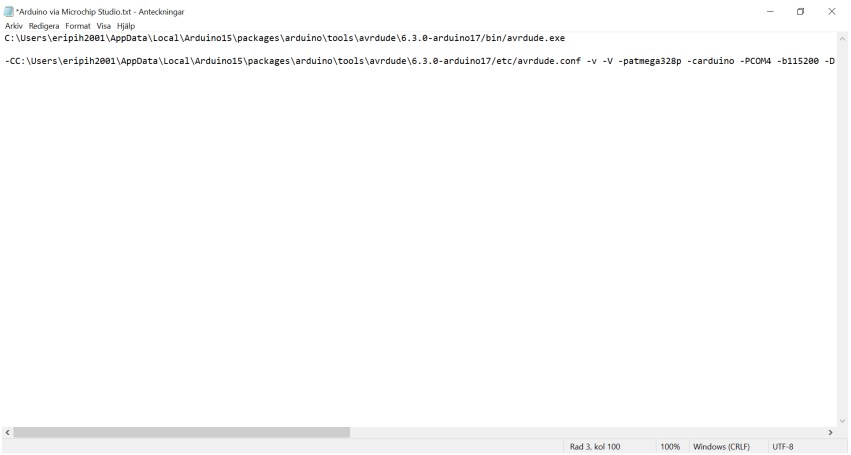
D.5.3 Gör om kommandot till ett återanvändbart verktyg

Kommandot ska delas upp i två delar: själva programmet (**Command**) och allt det får med sig (**Arguments**). Gör det i Anteckningar (Notepad):

1. Klistra in raden och ta bort alla citattecken.
2. Sätt en radbrytning direkt efter `bin/avrdude`, och lägg till `.exe`, så att första raden slutar på `bin/avrdude.exe`.

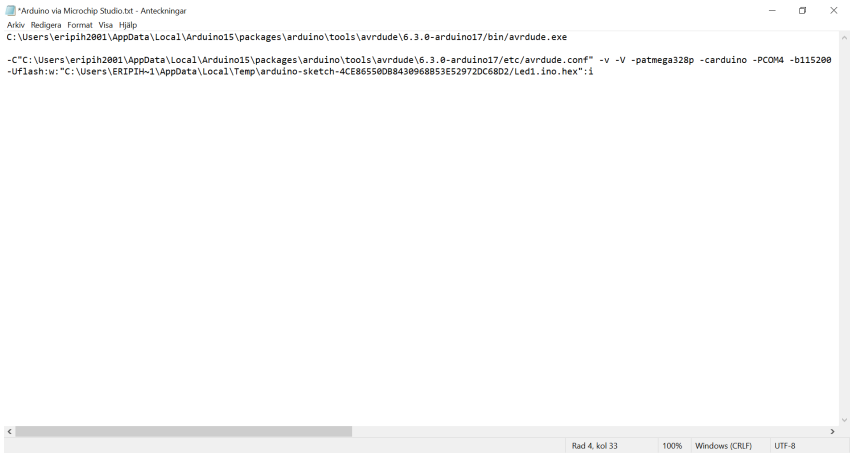


Figur D.10 Anteckningar med avrdude-kommandot inklistrat utan citattecken.



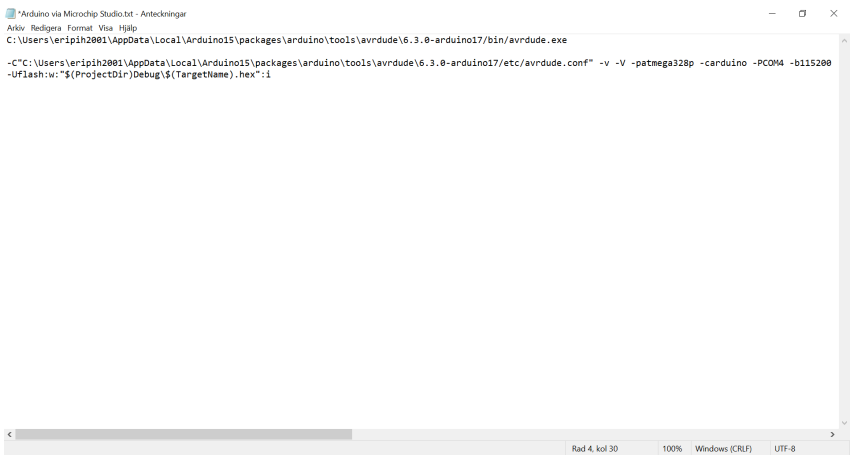
Figur D.11 Anteckningar där första raden nu slutar på avrdude.exe.

3. Sätt en radbrytning till före `-Uflash`, så att resten hamnar på en tredje rad.
4. Sätt tillbaka citattecken runt de två filsökvägarna: sökvägen till `avrdude.conf` på andra raden, och sökvägen till hexfilen på tredje.



Figur D.12 Kommandot på tre rader, med citattecken runt de två filsökvägarna och Arduinos hexfil kvar på tredje raden.

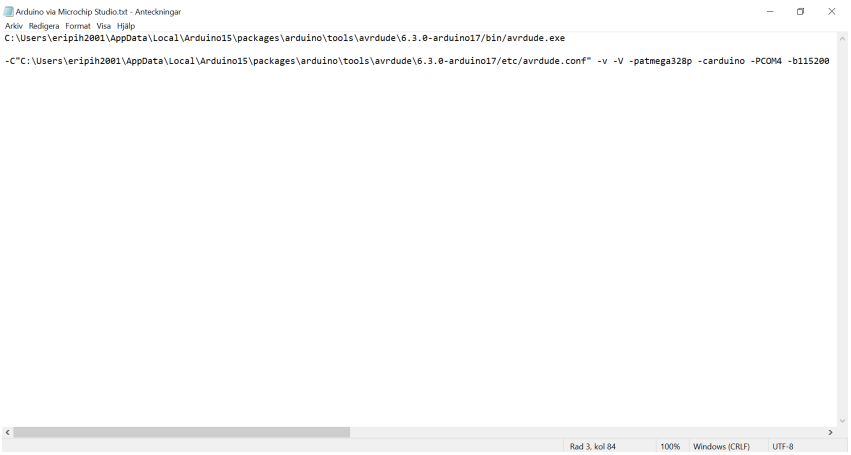
5. Ersätt sökvägen till Arduinos hexfil, allt mellan citattecknen på tredje raden, med `$(ProjectDir)Debug\$(TargetName).hex`. Microchip Studio byter ut de två variablerna mot det aktuella projektets katalog och namn, så samma kommando fungerar för alla dina projekt.
6. Slå ihop de två sista raderna till en, så att det bara finns två rader kvar. Spara filen.



Figur D.13 Efter bytet i steg 5, med sökvägen till projektets hexfil i Debug-katalogen på tredje raden.



Figur D.14 Samma tre rader utan den tomma raden, och med citattecken också runt -PCOM4, vilket fungerar lika bra.

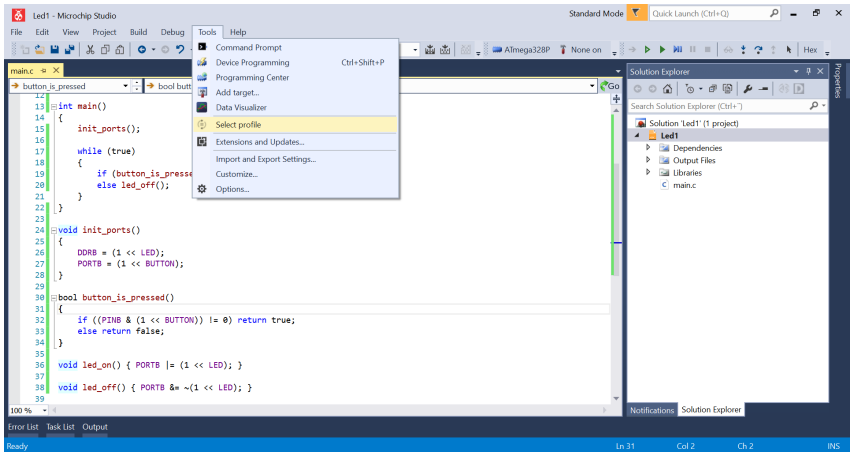


Figur D.15 Efter steg 6, med två rader kvar och filen sparad.

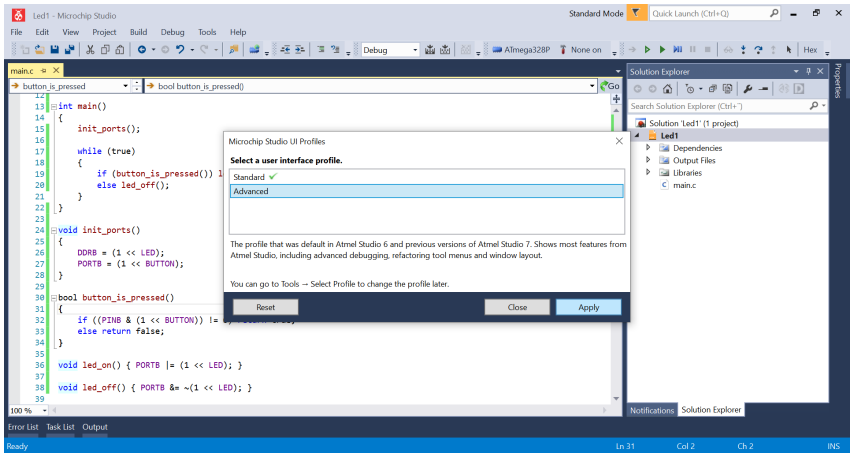
Första raden är nu kommandot, andra raden argumenten.

D.5.4 Lägg till verktyget i Microchip Studio

1. Externa verktyg kräver den avancerade profilen: **Tools → Select Profile**, välj **Advanced** och klicka **Apply**.

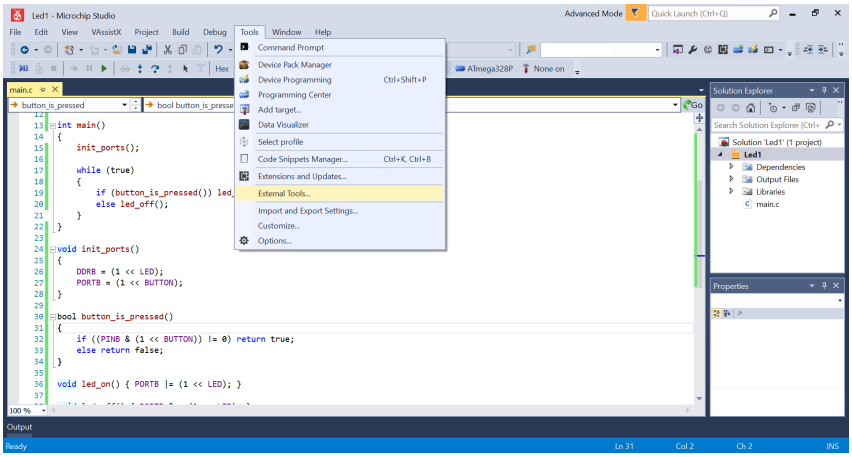


Figur D.16 Microchip Studios meny Tools med Select Profile markerat.

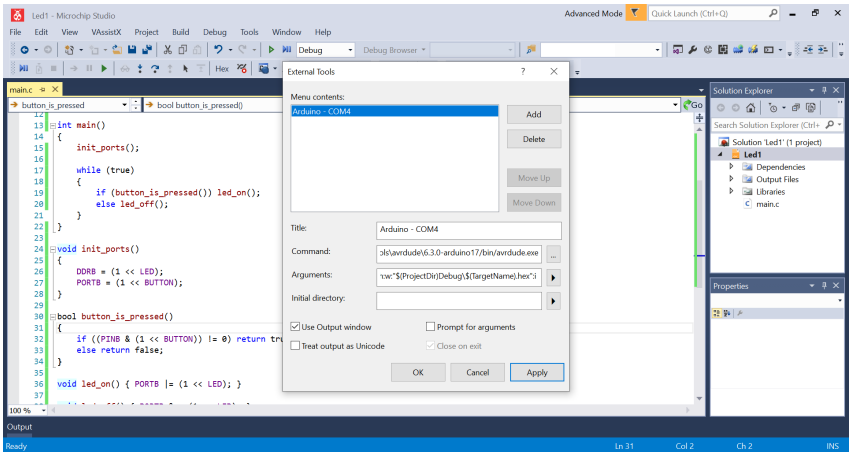


Figur D.17 Dialogen Select Profile med profilen Advanced vald.

2. Välj **Tools → External Tools** och klicka **Add**.
3. Döp verktyget till `Arduino` följt av COM-porten, till exempel `Arduino - COM4`.
4. Klistra in första raden från Anteckningar i **Command** och andra raden i **Arguments**.
5. Kryssa i **Use Output window** och klicka **OK**.



Figur D.18 Microchip Studios meny Tools med External Tools markerat.

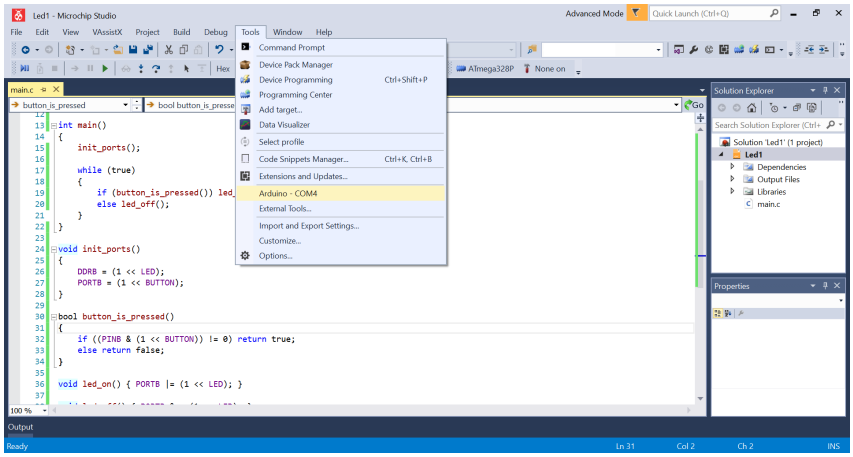


Figur D.19 Dialogen External Tools, ifylld med titel, kommando och argument för Arduino på COM4.

Skärmbilderna är tagna i ett C-projekt. För ett assemblerprojekt är varje steg detsamma.

D.5.5 Programmera och kontrollera

1. Bygg projektet med **F7**. Hexfilen skapas i projektets katalog `Debug`; kontrollera i Output-fönstret var den hamnade om programmeringen inte hittar den.
2. Välj **Tools → Arduino - COM4**.

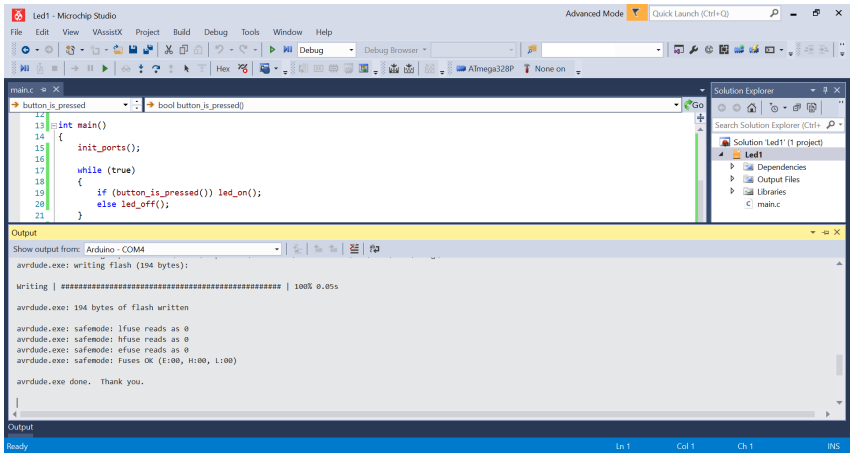


Figur D.20 Microchip Studios meny Tools med det nya verktyget Arduino - COM4.

Om allt fungerar slutar utskriften med antalet byte som skrevs:

```
Writing | ################################################## | 100% 0.05s

avrdude.exe: 194 bytes of flash written
...
avrdude.exe done. Thank you.
```



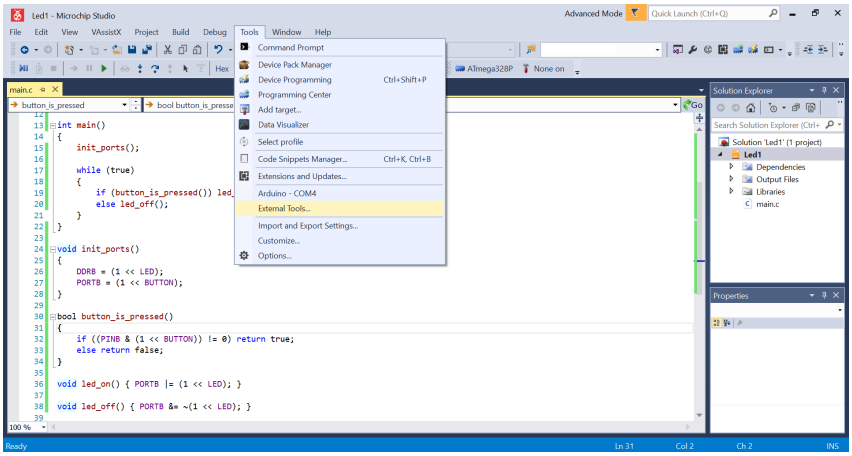
Figur D.21 Microchip Studios utskrift efter en lyckad programmering av kortet.

D.5.6 Om COM-porten ändras

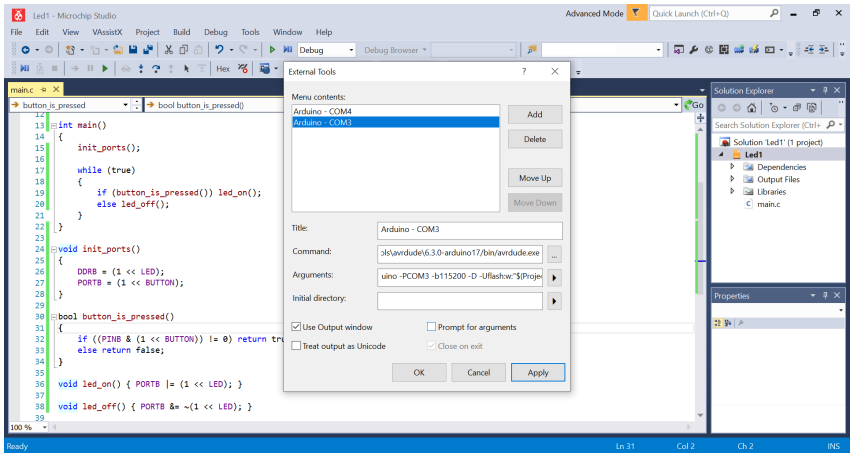
Om kortet ansluts till en annan USB-port kan det få en annan COM-port, och programmeringen misslyckas med ett fel i stil med:

```
avrdude.exe: ser_open(): can't open device "\\.\COM4": Det går inte att hitta filen.
```

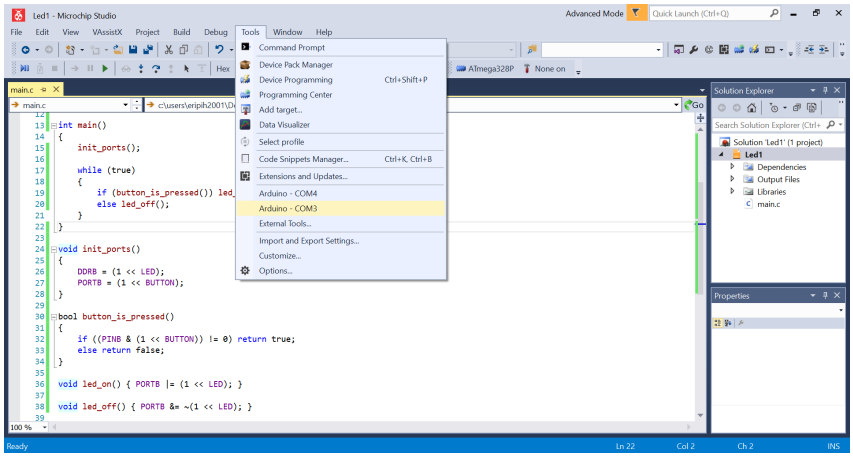
Kontrollera kortets aktuella port i Arduino IDE (**Tools → Port**). Lägg sedan till ett andra verktyg i **Tools → External Tools**, likadant som det första men med den nya porten i **Arguments**, till exempel `-PCOM3` i stället för `-PCOM4`.



Figur D.22 Microchip Studios meny Tools med verktyget Arduino - COM4 och External Tools markerat.



Figur D.23 Dialogen External Tools, ifylld för Arduino på COM3.



Figur D.24 Microchip Studios meny Tools med verktygen för både COM3 och COM4.

D.6 Vanliga fel

Symptom	Trolig orsak
Start Debugging frågar efter ett verktyg	Simulatorn är inte vald, se avsnitt D.4.1.
Den gula pilen står kvar på samma rad	Programmet väntar i en loop, till exempel på en ingång som aldrig ändras.
Ett register ändras inte som väntat	Den gula pilen visar nästa instruktion, inte den som just kördes.
Stoppuret visar orimliga tider	**Frequency** står inte på 16 MHz.
En port ändras inte i I/O-fönstret	Fel register (`PINB` i stället för `PORTB`), eller `DDRB` är inte satt.
`avrdude` hittar inte hexfilen	Projektet är inte byggt, eller hexfilen ligger i en annan katalog än `Debug`.
`ser_open(): can't open device`	Fel COM-port, eller kortet är inte anslutet, se avsnitt D.5.6.

BILAGA E

Referensblad: AVR-instruktionerna

De instruktioner och direktiv som används i kursen, för ATmega328P. Bladet delas ut tillsammans med det skriftliga provet (bilaga F), och är tänkt att ligga bredvid tangentbordet under Labb 3.

`Rd` och `Rr` är vilket som helst av registren `r0`–`r31`. `K` är en konstant, 0–255. Instruktioner med `K` fungerar **bara med `r16`–`r31`**. Klockfrekvensen på ett Arduino Uno-kort är 16 MHz, så en klockcykel tar 62,5 ns.

Flaggorna i SREG: **C** carry (minnessiffra, eller lån vid subtraktion), **Z** zero (resultatet blev noll), **N** negative (bit 7 i resultatet), **V** overflow (spill i tvåkomplement), **S** sign (`N` xor `V`), **H** half carry (minnessiffra från bit 3). I kolumnen *Flaggor* står de flaggor instruktionen kan ändra; övriga flaggor lämnas orörda.

Direktiv

Direktiv är instruktioner till assemblern, inte till processorn. De blir ingen maskinkod.

Direktiv	Exempel	Betydelse
`.include`	`.include "m328Pdef.inc"`	Läs in en annan fil, här registernamnen för ATmega328P.
`.org`	`.org 0x0000`	Lägg nästa instruktion på den här adressen i programminnet.
`.def`	`.def counter = r16`	Ge ett register ett namn.
`.equ`	`.equ LIMIT = 10`	Ge en konstant ett namn.
`low()`, `high()`	`ldi r16, low(RAMEND)`	Den låga respektive höga byten av ett 16-bitars värde.

Talformat: decimalt `25`, binärt `0b00011001`, hexadecimalt `0x19` eller `$19`, tecken `'A'`.

I/O-registren i kursen

Register	Betydelse
`DDRB`, `DDRD`	Riktning: 1 = utgång, 0 = ingång, en bit per stift.
`PORTB`, `PORTD`	Utgång: nivån som drivs ut. Ingång: 1 = pull-up påslagen.
`PINB`, `PIND`	Stiftens verkliga nivå, att läsa.
`SPH`, `SPL`	Stackpekaren, hög och låg byte. `RAMEND` = `0x08FF`.
`SREG`	Statusregistret, med flaggorna.

Dataöverföring

Instruktion	Exempel	Gör	Flaggor	Cykler
`ldi Rd, K`	`ldi r16, 0x2A`	Rd = K	–	1
`mov Rd, Rr`	`mov r17, r16`	Rd = Rr	–	1
`movw Rd, Rr`	`movw r24, r22`	Rd+1:Rd = Rr+1:Rr (ett registerpar; Rd och Rr jämna)	–	1
`clr Rd`	`clr r16`	Rd = 0	Z N V S	1
`ser Rd`	`ser r16`	Rd = 0xFF (bara `r16`–`r31`, som `ldi`)	–	1
`in Rd, A`	`in r16, PIND`	Rd = I/O-registret A	–	1
`out A, Rr`	`out PORTB, r16`	I/O-registret A = Rr	–	1
`lds Rd, k`	`lds r16, 0x0100`	Rd = byten på adress k i dataminnet (SRAM)	–	2
`sts k, Rr`	`sts 0x0100, r16`	Byten på adress k i dataminnet = Rr	–	2
`push Rr`	`push r16`	Lägg Rr på stacken	–	2
`pop Rd`	`pop r16`	Hämta Rd från stacken	–	2

Aritmetik

Instruktion	Exempel	Gör	Flaggor	Cykler
`add Rd, Rr`	`add r16, r17`	Rd = Rd + Rr	H S V N Z C	1
`adc Rd, Rr`	`adc r17, r19`	Rd = Rd + Rr + C	H S V N Z C	1
`sub Rd, Rr`	`sub r16, r17`	Rd = Rd – Rr	H S V N Z C	1
`subi Rd, K`	`subi r16, 5`	Rd = Rd – K	H S V N Z C	1
`sbc Rd, Rr`	`sbc r17, r19`	Rd = Rd – Rr – C	H S V N Z C	1
`inc Rd`	`inc r16`	Rd = Rd + 1	S V N Z	1
`dec Rd`	`dec r16`	Rd = Rd – 1	S V N Z	1
`neg Rd`	`neg r16`	Rd = 0 – Rd (tvåkomplement)	H S V N Z C	1
`adiw Rd, K`	`adiw r24, 1`	Rd+1:Rd = Rd+1:Rd + K, K = 0–63	S V N Z C	2
`sbiw Rd, K`	`sbiw r24, 1`	Rd+1:Rd = Rd+1:Rd – K, K = 0–63	S V N Z C	2

Det finns ingen `addi`. Addera en konstant genom att subtrahera dess negativa värde: `subi r16, -5` adderar 5. `inc` och `dec` ändrar **inte** C. `adiw` och `sbiw` fungerar bara med registerparen `r25:r24`, `r27:r26`, `r29:r28` och `r31:r30`.

Logiska operationer

Instruktion	Exempel	Gör	Flaggor	Cykler
`and Rd, Rr`	`and r16, r17`	Rd = Rd AND Rr, bit för bit	S V N Z	1
`andi Rd, K`	`andi r16, 0x0F`	Rd = Rd AND K	S V N Z	1
`or Rd, Rr`	`or r16, r17`	Rd = Rd OR Rr	S V N Z	1
`ori Rd, K`	`ori r16, 0x80`	Rd = Rd OR K	S V N Z	1
`eor Rd, Rr`	`eor r16, r17`	Rd = Rd XOR Rr	S V N Z	1
`com Rd`	`com r16`	Rd = NOT Rd (inverterar alla bitar)	S V N Z C	1
`tst Rd`	`tst r16`	Rd AND Rd: sätter Z och N, ändrar inte Rd	S V N Z	1
`swap Rd`	`swap r16`	Byter plats på de två halvorna (nibblarna)	–	1

Efter `and`, `andi`, `or`, `ori`, `eor`, `com` och `tst` är V alltid 0. Efter `com` är C alltid 1.

Skift och rotation

Instruktion	Exempel	Gör	Flaggor	Cykler
`lsl Rd`	`lsl r16`	Skifta vänster; bit 7 till C, 0 in i bit 0 (Rd × 2)	H S V N Z C	1
`lsr Rd`	`lsr r16`	Skifta höger; bit 0 till C, 0 in i bit 7 (Rd / 2, utan tecken)	S V N Z C	1
`asr Rd`	`asr r16`	Skifta höger; bit 7 behålls (Rd / 2, med tecken)	S V N Z C	1
`rol Rd`	`rol r16`	Rotera vänster genom C: C in i bit 0, bit 7 till C	H S V N Z C	1
`ror Rd`	`ror r16`	Rotera höger genom C: C in i bit 7, bit 0 till C	S V N Z C	1

Jämförelse

Instruktion	Exempel	Gör	Flaggor	Cykler
`cp Rd, Rr`	`cp r16, r17`	Beräknar Rd – Rr, sparar bara flaggorna	H S V N Z C	1
`cpc Rd, Rr`	`cpc r17, r19`	Beräknar Rd – Rr – C, sparar bara flaggorna	H S V N Z C	1
`cpi Rd, K`	`cpi r16, 100`	Beräknar Rd – K, sparar bara flaggorna	H S V N Z C	1

Hopp

Instruktion	Hoppar om	Flagga	Cykler
`rjmp k`	alltid	–	2
`breq k`	lika (resultatet var noll)	Z = 1	1 / 2
`brne k`	inte lika	Z = 0	1 / 2
`brlo k`	lägre, utan tecken	C = 1	1 / 2
`brsh k`	samma eller högre, utan tecken	C = 0	1 / 2
`brlt k`	mindre än, med tecken	S = 1	1 / 2
`brge k`	större än eller lika, med tecken	S = 0	1 / 2
`brmi k`	negativt (bit 7 satt)	N = 1	1 / 2
`brpl k`	positivt (bit 7 noll)	N = 0	1 / 2
`brcs k`	carry satt	C = 1	1 / 2
`brcc k`	carry noll	C = 0	1 / 2

`k` är en etikett. Ett villkorligt hopp tar **1 cykel om det inte hoppar och 2 om det hoppar**. Det når högst 64 instruktioner bort; längre bort når man med ett `rjmp`.

Subrutiner

Instruktion	Exempel	Gör	Flaggor	Cykler
`rcall k`	`rcall delay`	Lägg returadressen på stacken och hoppa till k	–	3
`ret`	`ret`	Hämta returadressen från stacken och hoppa tillbaka	–	4

Bitinstruktioner

Instruktion	Exempel	Gör	Flaggor	Cykler
`sbi A, b`	`sbi PORTB, 5`	Sätt bit b i I/O-registret A till 1	–	2
`cbi A, b`	`cbi PORTB, 5`	Nollställ bit b i I/O-registret A	–	2
`sbic A, b`	`sbic PIND, 2`	Hoppa över nästa instruktion om bit b i A är 0	–	1 / 2
`sbis A, b`	`sbis PIND, 2`	Hoppa över nästa instruktion om bit b i A är 1	–	1 / 2
`sbrc Rr, b`	`sbrc r16, 0`	Hoppa över nästa instruktion om bit b i Rr är 0	–	1 / 2
`sbrs Rr, b`	`sbrs r16, 7`	Hoppa över nästa instruktion om bit b i Rr är 1	–	1 / 2
`clc`	`clc`	Nollställ carryflaggan: C = 0	C	1
`sec`	`sec`	Ettställ carryflaggan: C = 1	C	1
`nop`	`nop`	Ingenting, en cykel	–	1

`sbi`, `cbi`, `sbic` och `sbis` når bara I/O-registren på adress 0–31, bland annat `PINB`, `DDRB`, `PORTB`, `PIND`, `DDRD` och `PORTD`. En instruktion som hoppar över nästa tar 1 cykel om den inte hoppar och 2 om den hoppar, eller 3 om instruktionen den hoppar över är två ord lång, som `lds` och `sts`.

BILAGA F

Det skriftliga provet

Kursen avslutas med ett skriftligt prov, efter kapitel 12. Här står hur provet är upplagt, vad det omfattar och hur det bedöms, och hur det tillsammans med laborationerna ger kursens betyg.

Ett övningsprov finns att träna på: frågorna står i bilaga G, och lösningsförslagen, med poäng per deluppgift, i bilaga H. Gör det under två timmar, innan du tittar i lösningarna. Själva provet och dess lösningar är lärarens och finns inte i boken.

Provets form

- **Två timmar**, individuellt, i slutet av kursen.
- **Hjälpmedel:** penna, suddgummi och linjal. Ingen miniräknare. Ett referensblad med AVR-instruktionerna, bilaga E, delas ut tillsammans med provet; det är samma blad som använts under kursen.
- **40 poäng**, fördelade på sex frågor. Varje fråga har deluppgifter, och poängen står vid varje deluppgift.
- Svaren skrivs på separata papper, med en ny fråga på ett nytt papper.

Betygsgränser

Betyg	Poäng
U	under 20
G	20–29
VG	30–40

Slutbetyget på kursen bygger på provet och laborationerna tillsammans; se ”Provet och laborationerna” nedan.

Vad provet omfattar

Provet omfattar hela kursen utom sekvensnät och vippor, som förekommer bara som en del av mikrodatorns uppbyggnad. Frågorna är desamma till form och poäng på varje prov:

Fråga	Område	Kapitel	Lärandemål	Poäng
1	Talsystem och koder	1	beskriva talsystem; omvandla talvärden	8
2	Grindar och boolesk algebra	2	förklara grindar; beskriva logisk algebra och förenkling	8
3	Karnaughdiagram	4	använda Karnaughdiagram för minimering	6
4	Analys av ett kombinatoriskt nät, grindnät eller kontaktnät	2–5	analysera digitala kombinatoriska nät	6
5	Mikrodatorns uppbyggnad	6–7	redogöra för mikrodatorns uppbyggnad	4
6	Assemblerprogrammering	7–10	använda assemblerspråk för att programmera en mikrodator	8
				40

Lärandemålen om att syntetisera nät med IC-kretsar och kontakter, och att styra styrobjekt, examineras i laborationerna, där de hör hemma.

Assemblerfrågorna gäller ATmega328P med klockfrekvensen 16 MHz, och de instruktioner som står på referensbladet. Ingen fråga kräver att en instruktionskod eller en cykeltid kan utantill; det som behövs står på bladet.

Hur provet bedöms

- **Metoden ger poäng.** En korrekt uträkning med ett slarvfel är värd mer än ett rätt svar utan uträkning. Ett svar utan uträkning kan få full poäng bara där frågan inte ber om någon.
- **Följdfel kostar en gång.** Om en deluppgift bygger på svaret i en tidigare, och det svaret var fel, bedöms den senare deluppgiften utifrån det felaktiga svaret. Ett fel i en sanningstabell som förs vidare till ett Karnaughdiagram kostar poäng i tabellen, inte i diagrammet.
- **Talsystemet ska synas.** Ett svar som `2C` utan skrivsätt är tvetydigt och ger inte full poäng. Skriv `0x2C`, `0010 1100` eller 44.
- **Program bedöms efter vad de gör, inte efter stavningen.** Ett program som skulle fungera men har ett felstavat registernamn eller ett saknat kommatecken förlorar ingenting. Ett program som läser fel bit, eller glömmer att en knapp är aktivt låg, förlorar de poäng den delen gäller.
- **Namngivna fällor.** Några deluppgifter finns till för att se om ett vanligt misstag undviks, till exempel en flagga som en instruktion inte ändrar, eller en Karnaughaxel i binär ordning

i stället för Graykod. Lösningsförslagen säger var fällorna finns, och ett svar som går i en fälla förlorar de poäng som fällan gäller, inte hela deluppgiftens. Lösningsförslagen till övningsprovet, i bilaga H, visar hur det ser ut.

Provet och laborationerna

- **Labb 1** och **Labb 2** bedöms med **U/G**.
- **Labb 3** bedöms med **U/G/VG**.
- Det **skriftliga provet**, två timmar i slutet av kursen, bedöms med **U/G/VG**. **G** kräver 20 poäng av 40, och **VG** kräver 30.

Laborationerna

En laboration är godkänd när samtliga uppgifter för godkänt är redovisade för läraren, och båda i paret kan förklara lösningen. Redovisning sker under laborationspassen. Handledningarna finns i bilaga A, B och C.

Laboration	Krav för G	Krav för VG
Labb 1	Uppgift 1–8	–
Labb 2	Labb 2-1 uppgift 1–4 och Labb 2-2 uppgift 1–3	–
Labb 3	Del 1.1–12, 14–16, 21 och slutuppgiften del A	Samtliga delar 1.1–23 och slutuppgiften del B

Slutbetyg

Betyg	Krav
G	Samtliga laborationer godkända och godkänt skriftligt prov.
VG	Samtliga laborationer godkända, VG på Labb 3 och VG på det skriftliga provet.

För godkänt betyg krävs att alla kursens lärandemål är uppnådda. Omprov och restlaborationer genomförs enligt skolans rutiner.

Lärandemålen

Var varje lärandemål i kursplanen undervisas, och var det examineras.

Lärandemål	Kapitel	Examineras
Förklara grindar och logiska funktioner	2, 4	Labb 1, Labb 2, prov
Beskriva binära, hexadecimala och decimala talsystem	1	Prov
Beskriva logisk algebra och förenklingar av booleska uttryck	2, 4	Prov
Redogöra för mikrodatorns uppbyggnad på blockschemanivå	6, 7	Prov
Omvandla talvärden mellan olika talsystem	1, 7, 8	Prov, Labb 3
Syntetisera digitala kombinatoriska nät med IC-kretsar och kontakter	2–5	Labb 1, Labb 2
Använda Karnaughdiagram för analys och minimering av booleska uttryck	4	Labb 2, prov
Använda assemblerspråk för att programmera en mikrodator	7–10	Labb 3, prov
Programmera mikrodatorn till att kontrollera styrobjekt	8, 10, 11	Labb 3
Analysera digitala kombinatoriska nät	2–5	Labb 1, Labb 2, prov

Sekvensnät och vippor (kapitel 6) ingår inte i kursplanens lärandemål. De tas med för att mikrodatorn annars blir en svart låda: register, programräknare och minne är vippor, och det är den kopplingen som gör blockschemat begripligt. På provet förekommer vippor bara som en del av mikrodatorns uppbyggnad.

BILAGA G

Övningsprov

Övningsprovet har samma form och samma poäng som det riktiga provet, som bilaga F beskriver. Gör det under två timmar, som på provet, och rätta det sedan mot lösningsförslagen i bilaga H.

- **Tid:** 2 timmar.
- **Hjälpmedel:** penna, suddgummi och linjal, och det utdelade referensbladet med AVR-instruktionerna, bilaga E. Ingen miniräknare.
- **Poäng:** 40. **G** kräver 20 poäng och **VG** 30 poäng.

Instruktioner:

- Börja varje fråga på ett nytt papper, och skriv ditt namn på varje papper.
- Visa hur du räknar. Metoden ger poäng även när svaret blir fel.
- Skriv alltid ut talsystemet: `0x2C`, `0010 1100` eller 44.
- Assemblerfrågorna gäller ATmega328P med klockfrekvensen 16 MHz.

Fråga 1 Talsystem och koder — 8 p

a) Omvandla det binära talet `1101 0110` till decimalt och hexadecimalt. (2 *p*)

b) Omvandla det decimala talet 173 till binärt och hexadecimalt. (2 *p*)

c) Talen `0xB4` och `0x6A` adderas i ett 8-bitars register. Ställ upp additionen binärt, och ange resultatet i registret hexadecimalt. Blir det en minnessiffra ut ur registret? (2 *p*)

d) Skriv talet 59 i BCD. Ange också de två ASCII-koder som texten `59` består av. (2 *p*)

Fråga 2 Grindar och boolesk algebra — 8 p

a) Gör sanningstabellen för `X = A'B + BC'`. (2 *p*)

b) Förenkla uttrycket algebraiskt så långt det går, och ange vilken lag du använder i varje steg. Vilken enda grind beräknar det förenklade uttrycket? (*3 p*)

```
X = AB + AB' + A'B
```

c)

1. Skriv `(A + B)'` utan parentes. (*1 p*)
2. Visa hur funktionen `A + B` kan byggas av enbart NAND-grindar med två ingångar. Rita nätet, och visa med boolesk algebra att det stämmer. (*2 p*)

Fråga 3 Karnaughdiagram

6 P

En funktion `X` av de fyra variablerna A, B, C och D är 1 för mintermerna 0, 2, 5, 7, 8, 10, 13 och 15, och 0 för alla andra.

a) Rita Karnaughdiagrammet, med `AB` på raderna och `CD` på kolumnerna, och fyll i ettorna. (*2 p*)

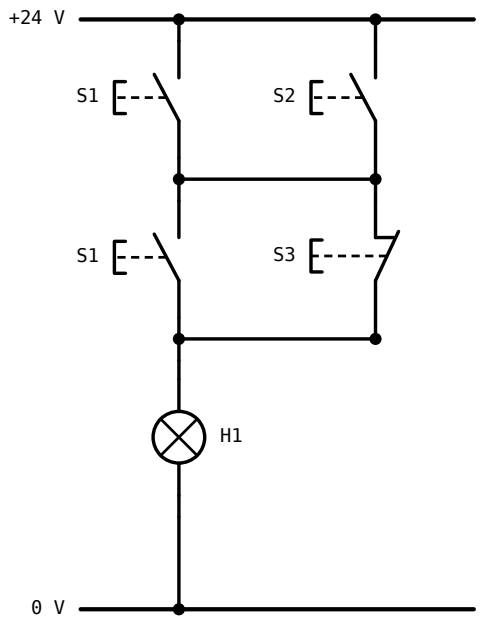
b) Gruppera ettorna minimalt, och skriv det minimerade uttrycket för `X` på SP-form. (*3 p*)

c) Uttrycket beror bara på två av variablerna. Vilka två, och vilken grind med två ingångar är `X`? (*1 p*)

Fråga 4 Analys av ett kontaktnät

6 p

Kontaktnätet i figur G.1 styr lampan H1 med tre tryckknappar. Tryckknappen S1 har två kontaktblock, som båda manövreras av samma knapp. S1 och S2 är slutande, S3 är brytande.



Figur G.1 Kontaktnätet som styr lampan H1.

a) Skriv det booleska uttrycket för H1. (*2 p*)

b) Gör sanningstabellen för H1, med S1, S2 och S3 som ingångar. (*2 p*)

c) Förenkla uttrycket, och rita ett kontaktnät som gör samma sak med färre kontakter. (*2 p*)

Fråga 5 Mikrodatorns uppbyggnad

4 p

a) Rita en mikrodators blockschema, med CPU (ALU, register, styrenhet och programräknare), programminne, dataminne, in- och utportar och bussar. Skriv en mening om vad varje block gör. (*2 p*)

b) Vad innehåller programräknaren, och vad händer med den när ett `rcall` utförs? (*1 p*)

c) I vilket minne i ATmega328P ligger programmet, och i vilket ligger stacken? Vilket av dem behåller sitt innehåll när strömmen bryts? (*1 p*)

Fråga 6 Assemblerprogrammering 8 p

a) Programmet nedan körs från början. Ange värdena i `r16`, `r17` och `r18` hexadecimalt när det har kommit till `end`, och värdet på flaggan `C` efter `add`. Förklara varför `r18` inte blev 300. (*3 p*)

```
    ldi r16, 0xB4
    ldi r17, 0x0F
    and r17, r16
    lsl r16
    ldi r18, 200
    ldi r19, 100
    add r18, r19
end:
    rjmp end
```

b) Programmet nedan körs från början.

```
    ldi r16, 0xFF
    out DDRB, r16
    ldi r16, 1
    ldi r17, 4
loop:
    out PORTB, r16
    lsl r16
    dec r17
    brne loop
```

1. Hur många gånger körs loopen, och vilket värde har `PORTB` när loopen är klar? (*1 p*)
2. Hur många klockcykler tar loopen, från den första `out` till och med den sista `brne`, och hur lång tid är det? (*1 p*)

c) En knapp är kopplad mellan PD3 och jord, och en lysdiod med förkopplingsmotstånd mellan PB0 och jord. Skriv ett komplett program, med initiering, där lysdioden lyser så länge knappen hålls nedtryckt och är släckt annars. Använd stiftets inbyggda pull-up-motstånd. (*3 p*)

BILAGA H

Lösningsförslag till övningsprovet

Lösningarna följer övningsprovet i bilaga G. Vid varje deluppgift står poängen och vad som ger delpoäng. Bedömningsprinciperna, med metod och följdfel, står i bilaga F. Programmen i fråga 6 a) och b) har körts i simulatorn.

Fråga 1 Talsystem och koder 8 p

a) 2 p

Ettorna har vikterna 128, 64, 16, 4 och 2: `128 + 64 + 16 + 4 + 2 = 214`. Grupperna `1101` och `0110` är `D` och `6`: **214** och **`0xD6`**.

1 p per rätt omvandling med uträkning.

b) 2 p

173: 128 ryms, 45 kvar; 32 ryms, 13 kvar; 8 ryms, 5 kvar; 4 ryms, 1 kvar; 1 ryms. Ettor på vikterna 128, 32, 8, 4 och 1: **`1010 1101`**. Grupperna `1010` och `1101` är `A` och `D`: **`0xAD`**.

1 p per rätt omvandling. Division med 2 och avläsning av resterna är lika bra.

c) 2 p

```
minnessiffror: 1 11
                 1011 0100      0xB4 = 180
               + 0110 1010    + 0x6A = 106
               -----------
               1 0001 1110      286
```

Bit 5 ger en minnessiffra in i bit 6, bit 6 en in i bit 7, och bit 7 en som går ut ur registret.

Registret får de åtta låga bitarna, **`0x1E`**, och **en minnessiffra går ut** ur registret, eftersom `180 + 106 = 286` inte ryms i åtta bitar: `286 - 256 = 30 = 0x1E`.

1 p för en korrekt uppställd addition, 1 p för `0x1E` och minnessiffran.

d) 2 p

BCD skriver varje siffra för sig med fyra bitar: **`0101 1001`**. ASCII-koderna för tecknen `'5'` och `'9'` är **`0x35`** och **`0x39`**.

1 p för BCD, 1 p för ASCII-koderna.

Fälla: 59 binärt, `0011 1011`, är inte BCD och ger 0 p på BCD-delen.

Fråga 2 Grindar och boolesk algebra 8 p

a) 2 p

A	B	C	A'B	BC'	X
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	1	1
1	1	1	0	0	0

2 p för en helt rätt tabell, 1 p för högst ett fel.

b) 3 p

```
X = AB + AB' + A'B
  = A(B + B') + A'B        bryt ut A
  = A · 1 + A'B            komplement: B + B' = 1
  = A + A'B                identitet
  = A + B                  förenklingslagen: A + A'B = A + B
```

Det förenklade uttrycket är en **OR-grind**.

1 p för att slå ihop de två första termerna till A, 1 p för förenklingslagen, 1 p för OR.
Ett svar som stannar vid `A + A'B` får 1 p: det saknar förenklingslagen, och pekar inte ut någon enda grind.

c) 3 p

1. `A'B'`, med De Morgan. *(1 p)*
2. Med De Morgan är `A + B = (A'B')'`. Inversen av en variabel är en NAND-grind med båda ingångarna kopplade till variabeln: `(AA)' = A'`. Nätet blir tre NAND-grindar: en som bildar `A'`, en som bildar `B'`, och en tredje med `A'` och `B'` in, som bildar `(A'B')' = A + B`. *(2 p)*

   1 p för ett korrekt nät, 1 p för att visa varför det stämmer.

Fråga 3 Karnaughdiagram 6 p

a) 2 p

AB \ CD	00	01	11	10
00	1			1
01		1	1	
11		1	1	
10	1			1

1 p för rätt Graykod på båda axlarna, 1 p för rätt placerade ettor.

Fälla: axlarna i binär ordning, `00, 01, 10, 11`, ger 0 p på axlarna, men följdfelet bedöms i b).

b) 3 p

Två grupper om fyra:

- **De fyra hörnen**, mintermerna 0, 2, 8 och 10. Hörnen är grannar eftersom diagrammet går runt både uppifrån och ned och från vänster till höger. Där är `B` = 0 och `D` = 0: termen **`B'D'`**.
- **De fyra mittrutorna**, mintermerna 5, 7, 13 och 15. Där är `B` = 1 och `D` = 1: termen **`BD`**.

```
X = B'D' + BD
```

1 p för varje rätt grupp med rätt term, 1 p för det samlade uttrycket.
Den som missar att hörnen är en grupp och skriver hörnen som två par, `A'B'D' + AB'D' + BD`, får 1 p.

c) 1 p

`X` beror bara på **B och D**, och är 1 när de är lika: en **XNOR**-grind. A och C påverkar inte `X` alls.

Fråga 4 Analys av ett kontaktnät 6 p

a) 2 p

Två parallellkopplingar i serie. Den första är `S1 + S2`, den andra `S1 + S3'`, eftersom S3 är brytande:

```
H1 = (S1 + S2)(S1 + S3')
```

1 p per rätt parallellkoppling, inklusive `S3'`.

b) 2 p

S1	S2	S3	`S1 + S2`	`S1 + S3'`	H1
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

2 p för en helt rätt tabell, 1 p för högst ett fel. Följdfel från a) bedöms här.

c) 2 p

Med den distributiva lagen `A + BC = (A + B)(A + C)`, läst baklänges:

```
H1 = (S1 + S2)(S1 + S3') = S1 + S2S3'
```

Nätet: den slutande kontakten S1 **parallellt med** en seriekoppling av den slutande kontakten S2 och den brytande kontakten S3, och sedan lampan. Tre kontakter i stället för fyra, och bara ett kontaktblock på S1.

1 p för det förenklade uttrycket, 1 p för ett rätt ritat nät.
Kontrollera med tabellen: `S1 + S2S3'` är 1 på raderna 010, 100, 101, 110 och 111, samma som i b).

Fråga 5 Mikrodatorns uppbyggnad 4 p

a) 2 p

CPU:n med ALU (räknar och jämför), register (håller värdena som räknas med), styrenhet (avkodar instruktionerna och styr resten) och programräknare (pekar på nästa instruktion). Programminnet håller programmet, dataminnet variablerna och stacken, och in- och utportarna förbinder mikrodatorn med omvärlden. Bussarna, adressbussen och databussen, förbinder CPU:n med minnena och portarna.

1 p för ett blockschema med alla delar på plats, 1 p för att beskriva vad de gör.

b) 1 p

Programräknaren innehåller **adressen till nästa instruktion** som ska hämtas. Vid `rcall` sparas adressen till instruktionen efter `rcall` på stacken, och programräknaren får subrutinens adress, så att nästa instruktion hämtas därifrån.

c) 1 p

Programmet ligger i **flashminnet**, som behåller sitt innehåll utan ström. Stacken ligger i **SRAM**, som töms när strömmen bryts.

Fråga 6 Assemblerprogrammering 8 p

a) 3 p

Uppmätt i simulatorn: **`r16 = 0x68`**, **`r17 = 0x04`**, **`r18 = 0x2C`**, och **`C = 1`** efter `add`.

- `and r17, r16`: `0000 1111 AND 1011 0100 = 0000 0100` = `0x04`.
- `lsl r16`: `1011 0100` skiftas ett steg åt vänster och blir `0110 1000` = `0x68`. Bit 7 hamnar i `C`, men `C` skrivs över senare av `add`.
- `add r18, r19`: `200 + 100 = 300`, som inte ryms i åtta bitar. Registret får `300 - 256 = 44` = `0x2C`, och minnessiffran hamnar i `C`.

1 p för `r16` och `r17`, 1 p för `r18`, 1 p för `C` och förklaringen: åtta bitar räcker till 255.

b) 2 p

1. Loopen körs **4 gånger**, en gång för varje värde på `r17`: 4, 3, 2 och 1. `PORTB` får värdena 1, 2, 4 och 8, och är **`0x08`** när loopen är klar. `r16` är då `0x10`, men det skrivs aldrig till porten.
2. Ett varv är `out` 1, `lsl` 1, `dec` 1 och `brne` 2 cykler, alltså 5; det sista varvet 4. Loopen tar `4 · 5 - 1 = 19` cykler, och `19 · 62,5 ns ≈ 1,19 µs`. Simulatorn mäter 19 cykler.

1 p per deluppgift.

Fälla: `PORTB` = `0x10`, genom att räkna med `lsl` efter den sista `out`, ger 0 p på den delen.

c) 3 p

Programmet står inte i boken. Så här bedöms det:

1 p för initieringen (utgång, ingång och pull-up), 1 p för att läsa rätt bit och ta hänsyn till att knappen är aktivt låg, 1 p för en loop som tänder och släcker rätt. Ett program som läser hela porten med `in`*, inverterar med* `com` *och maskar med* `andi` *är lika bra.*

Fälla: ett program som tänder lysdioden när biten är 1 lyser när knappen är släppt, och förlorar poängen för aktivt låg.

BILAGA I

Lösningsförslag till övningarna

Den här bilagan innehåller lösningsförslagen till övningarna i kapitel 1 till 12, med samma numrering som övningarna. Varje lösning står under sin övning, med sidan där övningen finns, och delarna följer övningens bokstäver.

Lösningarna visar uträkningen och inte bara svaret, eftersom det är uträkningen du ska kunna. Där en övning brukar ge ett visst felaktigt svar står det också, och varför det är fel.

Programmen har inga lösningsförslag här. En övning av sorten PROGRAM ber dig skriva assembler och köra det i simulatorn, och simulatorn visar om programmet gör vad det ska. Det som räknas eller förklaras i en sådan övning, en förutsägelse av registren, en cykelräkning eller en förklaring av vad som hände, har däremot sitt svar här. En övning eller en del som inte står med här är ett program.

Försök alltid själv först. Ett svar du har läst är mycket mindre värt än ett du har räknat fram och sedan kontrollerat.

KAPITEL 1

Talsystem och binära koder

Övning 1.1 Positionssystem

sidan 16

a) Positionerna räknas från höger med början på noll:

Siffra	Position	Vikt	Bidrag
4	3	$10^3 = 1000$	4000
0	2	$10^2 = 100$	0
7	1	$10^1 = 10$	70
2	0	$10^0 = 1$	2

Summan är $4000 + 0 + 70 + 2 = 4072$. Nollan bidrar inte med något, men den behövs ändå: utan den skulle sjuan hamna på position 2 och vara värd 700.

b) Position 2 har vikten $5^2 = 25$. Då är

$$243_5 = 2 \cdot 25 + 4 \cdot 5 + 3 \cdot 1 = 50 + 20 + 3 = 73_{10}$$

c) Vikten för position n är B^n, och alla tal upphöjda till noll är 1. Ett mer vardagligt sätt att se det: den högra positionen räknar ental, hur många ”hela” det finns, och det är samma sak i alla talsystem. Först när entalen tar slut, vid talbasen, börjar nästa position räkna.

Övning 1.2 Från binärt till decimalt

sidan 16

Summera vikterna för de positioner där det står en etta.

a) `1101` = 8 + 4 + 1 = **13**.

b) `1 0000` = **16**. En enda etta på position 4, alltså 2^4.

c) `0110 0100` = 64 + 32 + 4 = **100**.

d) `1000 0001` = 128 + 1 = **129**.

e) `1111 1111` = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = **255**, det största 8-bitars talet.

f) `1101 0110` = 128 + 64 + 16 + 4 + 2 = **214**.

Vanligt fel: Att börja viktraden från vänster med 1, alltså läsa talet baklänges. `1101` blir då 1 + 2 + 8 = 11 i stället för 13. Viktraden börjar alltid med 1 längst till **höger**.

Övning 1.3 Från decimalt till binärt

sidan 16

a) 13 med vikttabellen: 8 ryms (kvar 5), 4 ryms (kvar 1), 2 ryms inte, 1 ryms (kvar 0). Svar: **`1101`**. Kontroll: 8 + 4 + 1 = 13.

b) 37: 32 ryms (kvar 5), 16 nej, 8 nej, 4 ja (kvar 1), 2 nej, 1 ja. Svar: **`10 0101`**, i en byte `0010 0101`. Kontroll: 32 + 4 + 1 = 37.

c) 100: 64 ja (kvar 36), 32 ja (kvar 4), 16 nej, 8 nej, 4 ja (kvar 0), 2 nej, 1 nej. Svar: **`110 0100`**, i en byte `0110 0100`. Jämför med övning 1.2 c).

d) 200 med division med 2:

Division	Kvot	Rest
200 / 2	100	0
100 / 2	50	0
50 / 2	25	0
25 / 2	12	1
12 / 2	6	0
6 / 2	3	0
3 / 2	1	1
1 / 2	0	1

Nedifrån och upp: **`1100 1000`**. Kontroll: 128 + 64 + 8 = 200.

Lägg märke till att 200 är dubbelt så mycket som 100, och att `1100 1000` är `0110 0100` flyttat ett steg åt vänster. Det är ingen slump, se övning 1.14.

e) 255: divisionerna ger rest 1 varje gång (255, 127, 63, 31, 15, 7, 3, 1), så svaret är **`1111 1111`**. Det går också att se direkt: $255 = 256 - 1$, och $2^8 - 1$ är åtta ettor.

f) 1000:

Division	Kvot	Rest
1000 / 2	500	0
500 / 2	250	0
250 / 2	125	0
125 / 2	62	1
62 / 2	31	0
31 / 2	15	1
15 / 2	7	1
7 / 2	3	1
3 / 2	1	1
1 / 2	0	1

Nedifrån och upp: **`11 1110 1000`**, tio bitar. Kontroll: $512 + 256 + 128 + 64 + 32 + 8 = 1000$. Talet ryms alltså inte i en byte.

Övning 1.4 Binärt och hexadecimalt

sidan 16

Gruppera i fyror från höger, och byt varje grupp mot sin siffra.

a) `1010 1111` = `1010` `1111` = A F = **`0xAF`**.

b) `0011 1100` = 3 C = **`0x3C`**.

c) `1 1110` fylls ut till `0001 1110` = 1 E = **`0x1E`**. Den som grupperar från vänster får `1111 0` och svaret F0, som är ett helt annat tal: 240 i stället för 30.

d) `11 1110 1000` fylls ut till `0011 1110 1000` = 3 E 8 = **`0x3E8`**. Jämför med övning 1.3 f): det är 1000.

e) `0x7F` = 7 F = **`0111 1111`**.

f) `0xC3` = C 3 = **`1100 0011`**.

g) `0x2A5` = 2 A 5 = **`0010 1010 0101`**, eller utan de inledande nollorna `10 1010 0101`.

Övning 1.5 Hexadecimalt och decimalt

sidan 17

a) `0x2A` = $2 \cdot 16 + 10$ = **42**.

b) `0xFF` = $15 \cdot 16 + 15 = 240 + 15$ = **255**.

c) `0x100` = $1 \cdot 256 + 0 + 0$ = **256**. Ett mer än `0xFF`, och det första talet som behöver tre hexadecimala siffror och nio bitar.

d) `0x3E8` = $3 \cdot 256 + 14 \cdot 16 + 8 = 768 + 224 + 8$ = **1000**.

e) $75 = 4 \cdot 16 + 11$, så siffrorna är 4 och B: **`0x4B`**. Via binärt: $75 = 64 + 8 + 2 + 1 =$ `0100 1011` $= 4B$.

f) $160 = 10 \cdot 16 + 0$: **`0xA0`**.

g) $4095 = 4096 - 1 = 2^{12} - 1$, alltså tolv ettor: `1111 1111 1111` = **`0xFFF`**. Med division: 4095 / 16 = 255 rest 15, 255 / 16 = 15 rest 15, 15 / 16 = 0 rest 15. Tre femton, FFF.

Övning 1.6 Bitar och talområden

sidan 17

a)

Bitar	Antal värden	Största värde
4	$2^4 = 16$	15
8	$2^8 = 256$	255
12	$2^{12} = 4096$	4095
16	$2^{16} = 65\,536$	65 535

b) $2^9 = 512$ räcker inte, men $2^{10} = 1024$ gör det. Alltså **10 bitar**.

c) Nivåerna 0 till 200 är **201** olika värden, inte 200: noll räknas också. Sju bitar ger 128 värden, vilket inte räcker; åtta bitar ger 256. Alltså **8 bitar**, och $256 - 201 =$ **55** kombinationer blir oanvända.

Det är lätt att svara 200 värden, och i det här fallet ändrar det inte antalet bitar. Men i en tank som ska mäta 0 till 128 gör det skillnad: det är 129 värden, som kräver åtta bitar, medan den som räknar till 128 värden drar slutsatsen att sju bitar räcker.

d) LSB är `1`, så talet är **udda**: alla andra vikter är jämna. MSB är `1`, så talet är **minst 128**: MSB ensam är värd 128. (Talet är 183.)

Övning 1.7 Binär addition

sidan 17

a) 58 + 22:

```
minne:   0111 1100
           0011 1010     (58)
       + 0001 0110     (22)
       -----------
           0101 0000     (80)
```

`0101 0000` $= 64 + 16 = 80$. Stämmer.

b) 127 + 1:

```
minne:   1111 1110
           0111 1111     (127)
       + 0000 0001     (1)
```

```
          -----------
            1000 0000     (128)
```

Minnessiffran vandrar genom alla sju ettorna, precis som 999 + 1 i decimalt.

c) 200 + 100:

```
  minne: 1 1000 0000
           1100 1000     (200)
         + 0110 0100     (100)
         -----------
         1 0010 1100
```

d) Summan i c) är 300 och ryms inte i åtta bitar. I registret blir `0010 1100` kvar, alltså **44**, och den nionde biten, minnessiffran ut ur bit 7, hamnar utanför. 300 – 256 = 44: det som försvann var exakt 256, värdet av den nionde biten. I en mikrodator sparas den biten i flaggan *carry*, så att programmet kan upptäcka att summan inte fick plats.

Övning 1.8 BCD

sidan 18

a) 47 i BCD: 4 = `0100` och 7 = `0111`, alltså **`0100 0111`**. Som binärt tal är 47 = 32 + 8 + 4 + 2 + 1 = **`0010 1111`**. Två olika bitmönster för samma tal.

b) 2026 i BCD: **`0010 0000 0010 0110`**, fyra bitar per siffra, 16 bitar. Som binärt tal behövs bara 11 bitar (2026 = `111 1110 1010`), så BCD kostar fler bitar.

c) `1001 0011` i BCD är 9 och 3, alltså **93**. Som binärt tal är samma mönster 128 + 16 + 2 + 1 = **147**.

d) **Nej.** Den högra nibblen `1010` är 10, och i BCD finns bara siffrorna 0–9. En BCD-krets som får `1010` har fått något som inte är en siffra, och vad den gör med det är odefinierat.

Övning 1.9 Graykod

sidan 18

a) Tvåbitarskoden och dess spegelbild, med 0 respektive 1 framför:

```
0 00   0 01   0 11   0 10   1 10   1 11   1 01   1 00
```

Alltså `000, 001, 011, 010, 110, 111, 101, 100`, samma som tabellen i avsnitt 1.2.3.

b) Binärt, från `111` till `000`: **alla tre** bitarna ändras. Graykod, från `100` till `000`: **en** bit. Graykoden håller egenskapen hela vägen runt, också när räknaren slår om från det sista talet till det första.

c) Binärt är läge 1 `001` och läge 2 `010`: både bit 1 och bit 0 ändras. Hinner bit 0 ändras först läser styrsystemet ett ögonblick `000`, alltså läge 0; hinner bit 1 ändras först läser det `011`, alltså läge 3. Givaren kan alltså tillfälligt rapportera ett läge som axeln aldrig var i.

Med Graykod är läge 1 `001` och läge 2 `011`: bara bit 1 ändras. Under övergången läses antingen `001` eller `011`, alltså läge 1 eller läge 2, och båda är rimliga svar på var axeln är just då.

Övning 1.10 ASCII

sidan 18

a) `I` = `0x49` (`A` är `0x41`, så `I`, åtta bokstäver senare, är `0x49`), `C` = `0x43`, `7` = `0x37`, `4` = `0x34`. Texten är **`0x49 0x43 0x37 0x34`**.

Siffertecknen 7 och 4 blir `0x37` och `0x34`, inte `0x07` och `0x04`. Det är det vanligaste felet i den här uppgiften.

b) `A` är `0x41`, så `0x48` är sju bokstäver senare: B, C, D, E, F, G, **H**. `0x65` är **e**, och `0x6A` är fem bokstäver efter e, **j**. Texten är **`Hej`**.

c) `a` är `0x61` och `A` är `0x41`, en skillnad på **`0x20`** = 32. Binärt är `A` = `0100 0001` och `a` = `0110 0001`: de skiljer sig bara i **bit 5**, som har vikten 32.

d) Sifferstecknet `7` är `0x37`. Programmet drar bort `0x30` och får **7**. Åt andra hållet lägger det till `0x30`: 3 + `0x30` = `0x33`, som är tecknet **3**. Det fungerar bara för en siffra 0–9; tal med flera siffror måste delas upp siffra för siffra först.

Övning 1.11 7-segmentkod

sidan 18

a) `E` ritas med a, d, e, f och g:

g	f	e	d	c	b	a
1	1	1	1	0	0	1

Binärt `111 1001`, i en byte `0111 1001`, hexadecimalt **`0x79`**.

b) `0x6D` = `0110 1101`. Med g f e d c b a för bit 6 till 0: g = 1, f = 1, e = 0, d = 1, c = 1, b = 0, a = 1. Segmenten a, c, d, f och g lyser, vilket är siffran **5**. Jämför med figur 1.3.

c) Siffror som inte använder g påverkas inte: **0, 1 och 7**. Den allvarliga förväxlingen är **8**: utan g lyser a, b, c, d, e och f, vilket är exakt samma segment som **0**. En 8 och en 0 går alltså inte att skilja åt.

Övriga siffror som använder g, 2, 3, 4, 5, 6 och 9, blir ofullständiga figurer som inte liknar någon siffra. De är fel, men felet syns. Det är 8:an som är farlig, eftersom den ser rimlig ut: en display som visar 0 grader när det är 8 grader väcker ingen misstanke.

Övning 1.12 Paritetsbit

sidan 19

a)

Tecken	ASCII (7 bitar)	Antal ettor	Paritetsbit	Byte som skickas
`B`	`100 0010`	2	0	`0100 0010`
`C`	`100 0011`	3	1	`1100 0011`
`G`	`100 0111`	4	0	`0100 0111`

b) `1100 0001` har tre ettor, ett udda antal, så **ett fel har inträffat**. Men mottagaren kan inte veta vilken bit som vändes. Det kan ha varit paritetsbiten (då var tecknet `A`, `0100 0001`), eller någon av de andra bitarna. Allt mottagaren kan göra är att be om tecknet igen.

c) Ett exempel: `A` skickas som `0100 0001`. Två störningar vänder bit 0 och bit 1, och mottagaren får `0100 0010`. Det har två ettor, ett jämnt antal, så paritetskontrollen godkänner byten. Mottagaren tar emot `B` i stället för `A` utan att märka något. Varje dubbelfel ändrar antalet ettor med två, noll eller minus två, och antalet förblir jämnt.

Övning 1.13 Kontroll: för hand, och sedan med kalkylatorn

sidan 19

a)

- 173 = 128 + 32 + 8 + 4 + 1 = **`1010 1101`** = **`0xAD`**.
- `0x3E8` = 3 · 256 + 14 · 16 + 8 = **1000**, binärt **`11 1110 1000`**.
- `0b1100 1010` = 128 + 64 + 8 + 2 = **202** = **`0xCA`**.

b) Kalkylatorn ska visa exakt dessa värden. Om den inte gör det är det nästan alltid ett av tre fel: en vikt som hoppats över i summeringen, resterna i divisionen lästa i fel ordning, eller bitarna grupperade från vänster i stället för från höger. Hitta vilket, så att du inte gör det igen.

c) I läget `QWORD` (64 bitar) visar kalkylatorn `-1` som `FFFF FFFF FFFF FFFF`, och binärt som 64 ettor. I läget `BYTE` visar den `FF` och `1111 1111`.

d) `1111 1111` är som positivt tal **255**. Ett skäl till att datorn skriver −1 så: lägg till 1, och vad händer? Enligt avsnitt 1.1.7 blir 255 + 1 = 256, som i åtta bitar är `0000 0000`, alltså 0. Ett tal som blir noll när man lägger till ett beter sig precis som −1. Det är idén bakom **tvåkomplement**, som är ämnet för kapitel 8. Om du gissade något i den riktningen har du redan förstått det viktigaste.

Övning 1.14 Att lägga till en nolla

sidan 20

a) `0b10110` = 16 + 4 + 2 = **22**, och `0b101100` = 32 + 8 + 4 = **44**. Varje nolla som läggs till fördubblar talet.

b) När alla bitar flyttas ett steg åt vänster hamnar varje etta på en position som är värd dubbelt så mycket. **Värdet fördubblas**, precis som ett decimalt tal blir tio gånger större när man lägger till en nolla. (I ett register med åtta bitar gäller det bara så länge ingen etta trillar ut till vänster, se övning 1.7 d).)

c) Värdet **halveras, och en eventuell rest försvinner**: `0b1011` (11) blir `0b101` (5), och `0b1010` (10) blir också `0b101` (5). Biten som tas bort är LSB, som avgör om talet är udda, så den försvunna halvan är just resten vid division med 2.

d) Talet blir **16 gånger större**, eftersom talbasen är 16: `0x2A` (42) blir `0x2A0` = $2 \cdot 256 + 10 \cdot 16 = 672 = 42 \cdot 16$. En hexadecimal siffra är fyra bitar, så att lägga till en hexadecimal nolla är detsamma som att flytta alla bitar fyra steg åt vänster: $2^4 = 16$.

KAPITEL 2

Logiska grindar, boolesk algebra och kontaktnät

Övning 2.1 Grindar med tre ingångar

sidan 44

a)

A	B	C	AND	OR	XOR
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	0	1	1
1	0	1	0	1	0
1	1	0	0	1	0
1	1	1	1	1	1

b)

- AND är 1 när **alla** tre ingångarna är 1.
- OR är 1 när **minst en** ingång är 1.
- XOR är 1 när ett **udda** antal ingångar är 1, alltså en eller tre.

Den sista raden är den många får fel: XOR med tre ingångar är 1 på raden 111, eftersom tre är udda. Den som tänker ”XOR är 1 när ingångarna är olika” missar det, för den regeln gäller bara med två ingångar.

c) AND är `1` på **en** rad och OR på **sju**. AND kräver att alla villkor är uppfyllda, och det är de bara i en kombination; OR kräver bara ett, och det är uppfyllt i alla kombinationer utom den där alla är `0`. AND och OR är varandras spegelbilder, vilket är precis vad De Morgan säger.

Övning 2.2 Från ord till uttryck

sidan 45

a) Med D = ”dörren är öppen”, A = ”larmet är aktiverat” och L = ”larmet ljuder”: **`L = DA`**.

b) Med N = ”nivån är låg”, M = ”manuellt stopp är intryckt” och P = ”pumpen går”: **`P = NM'`**. Ordet *ingen* i beskrivningen är ett NOT.

c) Med D1, D2 och D3 för de tre dörrarna och H för lampan: **`H = D1 + D2 + D3`**.

d) X är `1` om maskinen är påslagen och skyddet inte är på plats, eller om nödlarmet är utlöst. Uttrycket beskriver alltså något som bör leda till ett larm, till exempel ett signalhorn.

e) `AB + C = 0 · 0 + 1 = 1`, men `A(B + C) = 0 · (0 + 1) = 0`. De två uttrycken ger olika svar, så `AB + C` får inte läsas som `A(B + C)`. AND binder hårdare än OR, precis som multiplikation binder hårdare än addition.

Övning 2.3 Räknelagarna

sidan 45

a) `A + A = A` (idempotens)

b) `A · A' = 0` (komplement)

c) `A + 1 = 1` (dominans)

d) `(A')' = A` (dubbel invertering)

e) `A + AB = A` (absorption)

f) `A(A + B) = A` (absorption). Eller: `A(A + B) = AA + AB = A + AB = A`.

g) `A + A'B = A + B` (förenkling)

h) `AB + A'B = B(A + A') = B · 1 = B` (distributiv, komplement, identitet)

Övning 2.4 De Morgan

sidan 45

a) `(AB)' = A' + B'`

b) `(A + B + C)' = A'B'C'`

c) `(A'B)' = (A')' + B' = A + B'`

d) `(A + B')' = A' · (B')' = A'B`

Vanligt fel: Att i c) och d) bara byta tecknet men glömma att invertera varje variabel, eller att invertera men glömma att en variabel som redan var inverterad blir oinverterad.

e)

A	B	AB	(AB)′	A′	B′	A′ + B′
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Kolumnerna `(AB)'` och `A' + B'` är lika på varje rad.

Övning 2.5 Från sanningstabell till uttryck

sidan 46

a) X är 1 på raderna 000, 010, 110 och 111:

```
X = A'B'C' + A'BC' + ABC' + ABC
```

b) Para ihop de två första termerna (de delar `A'C'`) och de två sista (de delar `AB`):

```
X = A'C'(B' + B) + AB(C' + C) = A'C' + AB
```

c) `A'C'` är 1 när A = 0 och C = 0: raderna 000 och 010. `AB` är 1 när A = 1 och B = 1: raderna 110 och 111. Tillsammans exakt de fyra ettorna i tabellen, och inga andra.

d) Oförenklat: tre NOT-grindar (för `A'`, `B'` och `C'`), fyra AND-grindar med tre ingångar och en OR-grind med fyra, alltså **8 grindar**. Förenklat: två NOT-grindar (`A'` och `C'`), två AND-grindar med två ingångar och en OR-grind med två, alltså **5 grindar**.

Övning 2.6 Algebraisk förenkling

sidan 46

a) Använd termen `AB` två gånger (idempotens):

```
AB + AB' + A'B = (AB + AB') + (AB + A'B) = A(B + B') + B(A + A') = A + B
```

b)

```
(A + B)(A + B') = AA + AB' + AB + BB' = A + A(B' + B) + 0 = A + A = A
```

Eller kortare med den andra distributiva lagen baklänges:

```
(A + B)(A + B') = A + BB' = A + 0 = A
```

c) Samma knep som i a), med `SR` två gånger:

```
S'R + SR' + SR = (S'R + SR) + (SR' + SR) = R(S' + S) + S(R' + R) = R + S
```

d)

```
A'B'C + A'BC + ABC = A'C(B' + B) + ABC
                   = A'C + ABC
                   = C(A' + AB)
                   = C(A' + B)
                   = A'C + BC
```

Steget `A' + AB = A' + B` är förenklingslagen med A och `A'` i omvända roller. Kontroll: X är 1 på raderna 001, 011 och 111, och `A'C + BC` är 1 på exakt de raderna.

Övning 2.7 Kontroll: analys av ett grindnät

sidan 46

a)

```
P = (A + B)'
Q = BC
X = P + Q = (A + B)' + BC = A'B' + BC
```

b)

A	B	C	P = (A + B)′	Q = BC	X
0	0	0	1	0	1
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	0	0
1	1	1	0	1	1

c) Simuleringen ska ge exakt den här tabellen. De vanligaste orsakerna till en avvikelse är att NOR-grinden har byggts som en OR (då blir P fel på varje rad), eller att B inte har kopplats till båda grindarna, eftersom den förgrenar sig i ritningen.

d) En avvikelse på en enstaka rad beror oftast på ett räknefel i tabellen, eftersom ett fel i bygget brukar synas på flera rader. Räkna om P och Q för raden, och följ sedan ledningen från varje ingång i bygget.

e) X är 1 när **både A och B är `0`**, eller när **både B och C är `1`**.

Övning 2.8 Från uttryck till grindnät

sidan 47

a) Två NOT-grindar (`A'` och `C'`), två AND-grindar (`A'B` och `BC'`) och en OR-grind: **5 grindar**.

b)

```
X = A'B + BC' = B(A' + C') = B(AC)'
```

Det sista steget är De Morgan: `A' + C' = (AC)'`. Nätet blir en **NAND-grind** med ingångarna A och C, vars utgång går till en **AND-grind** tillsammans med B. **2 grindar** i stället för 5.

c) En OR-grind för `A + B`, en NOT-grind för `C'` och en AND-grind: **3 grindar**.

d) Båda näten ska vara `1` på raderna ABC = 010, 011 och 110, och `0` på alla andra.

Övning 2.9 Allt av NAND

sidan 47

a) Med båda ingångarna kopplade till A blir utgången `(A · A)' = (A)' = A'`, eftersom `AA = A`. **En NAND-grind med ihopkopplade ingångar är en NOT.**

b) En NAND ger `(AB)'`. Inverteras det en gång till blir det `AB`. **Två NAND-grindar**: en som NAND och en, med ihopkopplade ingångar, som NOT.

c)

```
A + B = ((A + B)')' = (A'B')'
```

`(A'B')'` är en NAND av `A'` och `B'`, och `A'` och `B'` är var sin NAND med ihopkopplade ingångar. Alltså **tre NAND-grindar**: två som inverterar A och B, och en tredje som tar emot deras utgångar.

d) NOT ska ge 1, 0 för A = 0, 1. AND ska bara ge `1` för AB = 11. OR ska ge `1` för allt utom AB = 00.

Övning 2.10 Från uttryck till kontaktnät

sidan 48

a) En strömväg: S1 **slutande** i serie med S2 **brytande**, i serie med H1. Lampan lyser när S1 är nedtryckt och S2 inte är det.

b) Två parallella grenar mellan +24 V och lampan: den ena med S1 **slutande**, den andra med S2 **slutande** och S3 **slutande** i serie. Därefter H1.

c) S1 och S2 **slutande** parallellt, och den parallellkopplingen i serie med S3 **brytande** och H1. Samma form som bromsassistentens högra gren i avsnitt 2.2.6.

d) I a)–c) behöver varje knapp **ett** kontaktelement, eftersom varje variabel förekommer en gång. I XOR-kopplingen förekommer varje variabel två gånger, en gång rak och en gång inverterad, så varje knapp behöver **två**: ett slutande och ett brytande.

Övning 2.11 Analys av kontaktnät

sidan 48

a) Nät a) är en parallellkoppling av S1 (slutande) och S2 (brytande), i serie med S3:

```
H1 = (S1 + S2') · S3
```

S1	S2	S3	H1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

b) Nät b) är en seriekoppling av S1 och S2, parallellt med S3 (brytande):

```
H2 = S1 · S2 + S3'
```

S1	S2	S3	H2
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

c) H2 lyser utan S1 och S2 när **S3 är släppt**: raden 000. S3 är brytande och sitter i en egen parallell gren, så när S3 inte påverkas finns alltid en sluten väg genom den grenen, vad S1 och S2 än gör. Faktum är att H2 lyser i alla fyra kombinationer där S3 är släppt.

Övning 2.12 Självhållning

sidan 49

a) När S1 trycks sluts strömväg 1 genom S2 och S1, och **spolen K1 drar**. Båda K1-kontakterna sluts: **H1 tänds**, och kontakten K1 parallellt med S1 sluts. När S1 släpps går strömmen genom K1:s egen kontakt i stället, så spolen **förblir strömsatt** och H1 fortsätter lysa.

b) När S2 trycks bryts strömväg 1, **spolen släpper**, och båda K1-kontakterna öppnar. **H1 släcks.** När S2 släpps sluts S2 igen, men S1 och K1 är nu öppna, så spolen förblir strömlös.

c) Med S2 nedtryckt är strömväg 1 bruten, vad S1 än gör. **Stopp vinner.** Kopplingen kallas därför stoppdominant, och det är så en maskins start- och stoppkrets ska vara.

d) Med S2 slutande är strömväg 1 bara sluten medan S2 **hålls nedtryckt**. Att trycka på start gör ingenting. Trycker man på båda knapparna startar maskinen, och den fortsätter så länge S2 hålls in, men den stannar när man **släpper** stoppknappen. Stoppknappen fungerar alltså som en startknapp som måste hållas in, och ingen går att använda för att stoppa maskinen på det sätt en operatör förväntar sig. Felet är farligt även om det i det här fallet råkar leda till att maskinen stannar, eftersom ingen kommer att förstå vad som händer.

e) Med båda knapparna släppta kan H1 vara både tänd och släckt, beroende på om start eller stopp trycktes senast. Utgången beror alltså inte bara på ingångarna nu, utan på vad som har hänt tidigare. Kretsen har **minne**, genom att K1:s kontakt återkopplar spolens läge till dess egen strömväg, och ett nät med minne är ett sekvensnät, inte ett kombinatoriskt nät.

Övning 2.13 Bromsassistenten, felkopplad

sidan 49

a) `B = (D + S + R)F'` är 1 när F = 0 och minst en av D, S och R är 1. Jämfört med avsnitt 2.1.10 skiljer sig tabellerna på fyra rader:

D	S	R	F	B enligt avsnitt 2.1.10	B felkopplad
1	0	0	1	1	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	1	0

b) På alla fyra raderna är **D = 1 och F = 1**: föraren trampar på bromspedalen samtidigt som assistanssystemet rapporterar ett fel. Den felkopplade kretsen **bromsar inte**. Föraren ser ett hinder, trampar på bromsen, och ingenting händer, precis i den situation där assistanssystemet redan har visat att det inte går att lita på.

c) Sanningstabellen i avsnitt 2.1.10 skrevs ur kravet, där säkerhetskravet ”föraren kan alltid bromsa” blev hela den nedre halvan med ettor. En tabell som läses av ur en krets man redan har byggt visar bara vad kretsen gör, inte vad den borde göra, och skulle ha visat nollor på de fyra raderna utan att någon reagerade. **En tabell härledd ur kravet är ett oberoende facit; en tabell avläst ur kretsen är bara en beskrivning av den.** Det är skälet till att man alltid förutsäger först och kontrollerar sedan.

KAPITEL 3

Analys och felsökning av kontaktnät

Övning 3.1 Läs av uttrycket

sidan 60

Nät a) Den minsta gruppen är parallellkopplingen av S2 slutande och S2 brytande, `S2 + S2'`. Den sitter i serie med S1:

```
H1 = S1 · (S2 + S2')
```

S1	S2	H1
0	0	0
0	1	0
1	0	1
1	1	1

Med ord: lampan lyser när S1 är nedtryckt. S2 spelar ingen roll.

Nät b) Den minsta gruppen är parallellkopplingen av S1 slutande och S2 slutande, `S1 + S2`. Den sitter i serie med S1 brytande:

```
H2 = S1' · (S1 + S2)
```

S1	S2	H2
0	0	0
0	1	1
1	0	0
1	1	0

Med ord: lampan lyser när S2 är nedtryckt och S1 inte är det.

Nät c) Två seriekopplingar, `S1 · S2` och `S1 · S3`, som sitter parallellt:

```
H3 = S1 · S2 + S1 · S3
```

S1	S2	S3	H3
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Med ord: lampan lyser när S1 är nedtryckt tillsammans med minst en av S2 och S3.

Övning 3.2 Det minsta nätet

sidan 60

a)

- `S1 · (S2 + S2') = S1 · 1 = S1` (komplement och identitet).
- `S1' · (S1 + S2) = S1'·S1 + S1'·S2 = 0 + S1'·S2 = S1' · S2` (distributiv lag och komplement; det är förenklingslagen `A(A' + B) = AB` med S1′ i A:s roll).
- `S1 · S2 + S1 · S3 = S1 · (S2 + S3)` (distributiva lagen, S1 bruten ut).

b)

- Nät a): **S1 slutande ensam**, i serie med lampan.
- Nät b): **S1 brytande i serie med S2 slutande**.
- Nät c): **S1 slutande i serie med parallellkopplingen av S2 och S3**.

c)

- Nät a): tre kontakter blir en; **två sparade**. S2 behövde både sitt slutande och sitt brytande element, och behövs efteråt inte alls.
- Nät b): tre kontakter blir två; **en sparad**. S1 behövde två element, slutande och brytande, och behöver efteråt bara det brytande.
- Nät c): fyra kontakter blir tre; **en sparad**. S1 behövde två slutande element, och behöver efteråt ett.

Övning 3.3 Kontroll: förutsäg och bygg

sidan 61

a) Två parallellkopplingar i serie:

```
H1 = (S1 + S2') · (S1' + S2)
```

S1	S2	S1 + S2′	S1′ + S2	H1
0	0	1	1	1
0	1	0	1	0
1	0	1	0	0
1	1	1	1	1

b)

```
(S1 + S2')(S1' + S2) = S1·S1' + S1·S2 + S2'·S1' + S2'·S2
                     = 0 + S1·S2 + S1'·S2' + 0
                     = S1·S2 + S1'·S2'
```

Lampan lyser när knapparna är i **samma** läge: **XNOR**.

c) Lampan ska lysa när ingen knapp och när båda knapparna är nedtryckta, och vara släckt när bara en av dem är det. Stämmer det inte: kontrollera först att varje knapp sitter med sitt slutande element i den ena parallellkopplingen och sitt brytande i den andra, precis som ritningen visar. Det vanligaste felet är att båda elementen i en knapp har hamnat i samma parallellkoppling.

d) Båda näten har **fyra kontakter**, och båda använder varje knapps slutande och brytande element en gång. Nätet i Labb 1 är skrivet som en summa av produkter, `S1·S2 + S1'·S2'`: två seriekopplingar parallellt. Nätet här är en produkt av summor: två parallellkopplingar i serie. Uttrycken ser olika ut, men uträkningen i b) visar att de är samma funktion, och det är allt som räknas för den som trycker på knapparna.

Övning 3.4 Syntes

sidan 62

a) Lampan är släckt på en enda rad, S1 = 1 och S2 = 0. Uttrycket på SP-form är

```
H1 = S1'·S2' + S1'·S2 + S1·S2 = S1'(S2' + S2) + S1·S2 = S1' + S1·S2 = S1' + S2
```

där det sista steget är förenklingslagen med S1′ i A:s roll. Det minsta nätet är **S1 brytande parallellt med S2 slutande**, två kontakter.

Ett snabbare sätt att se det: lampan är släckt bara när S1 är nedtryckt och S2 inte är det, alltså `H1' = S1 · S2'`. Enligt De Morgan är då `H1 = (S1 · S2')' = S1' + S2`.

b)

```
K1 = (S1 + S2) · S3
```

En strömväg: S1 och S2 **slutande** parallellt, i serie med givaren S3 **slutande**, och därefter reläspolen K1. S3 är slutande eftersom den ska vara påverkad, alltså sluten, när grinden är stängd.

Övning 3.5 Felsökning

sidan 62

a) Spänningen finns före S3 men inte efter, så **S3 leder inte**, fast den är släppt och ska vara sluten. Den troligaste orsaken är att sladdarna sitter på S3:s **slutande** kontaktelement i stället för det brytande. Pröva: tänds lampan när du trycker på S3? Då är det så. Tänds den inte alls, är det i stället en lös sladd eller en trasig kontakt vid S3.

b) Lampan slocknar inte när S3 trycks, så S3 bryter inte strömvägen. **S3 är förbikopplad**: en extra sladd går förbi den, eller båda sladdarna har hamnat på samma anslutning. Strömmen tar då en väg runt S3, och lampan realiserar bara `S1 + S2`.

c) Lampan lyser bara när S3 är nedtryckt: S3 fungerar som en **slutande** kontakt. Sladdarna sitter på S3:s slutande element i stället för det brytande, samma fel som var den troligaste orsaken i a). Nätet realiserar då `(S1 + S2) · S3` i stället för `(S1 + S2) · S3'`.

Övning 3.6 Stoppknappen

sidan 63

a) En avbruten ledning bryter strömvägen precis som ett tryck på stoppknappen gör. **Maskinen stannar**, och går inte att starta igen förrän felet är åtgärdat.

b) En avbruten ledning till en slutande stoppknapp gör att knappen aldrig kan sluta kretsen. Maskinen fortsätter gå, och **stoppknappen fungerar inte**. Ingen märker felet förrän någon behöver stoppa maskinen, och då är det för sent.

c) En stoppfunktion ska kopplas så att **ett fel leder till det säkra läget**: en bruten ledning, en lossnad sladd eller en trasig kontakt ska stoppa maskinen, inte hindra den från att stoppas. Därför används brytande kontakter för stopp och nödstopp.

Övning 3.7 Alla funktioner av två knappar

sidan 63

a) Varje rad i tabellen kan ha utgången `0` eller `1`, oberoende av de andra. Med fyra rader finns $2^4 = \mathbf{16}$ olika tabeller, alltså 16 olika funktioner av två variabler.

b) **Konstant 0**, lampan lyser aldrig, realiseras genom att inte ansluta lampan till +24 V alls. **Konstant 1**, lampan lyser alltid, realiseras med en ledning direkt från +24 V till lampan.

c) **XOR** och **XNOR**. Båda är `S1·S2' + S1'·S2` respektive `S1·S2 + S1'·S2'` på minsta form, och i båda förekommer varje variabel både rak och inverterad. Alla andra funktioner av två variabler klarar sig med högst ett kontaktelement per knapp.

Övning 3.8 En väljare av kontakter

sidan 63

a)

S1	S2	S3	S1 + S2	S1′ + S3	H1
0	0	0	0	1	0
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	1	0	0
1	0	1	1	1	1
1	1	0	1	0	0
1	1	1	1	1	1

b) I den övre halvan, S1 = 0, är H1 samma som **S2**. I den nedre, S1 = 1, är H1 samma som **S3**. S1 väljer alltså vilken av de andra två knapparna som styr lampan.

c)

```
(S1 + S2)(S1' + S3) = S1·S1' + S1·S3 + S2·S1' + S2·S3
                    = S1·S3 + S1'·S2 + S2·S3
```

Termen `S2·S3` är onödig. Den är 1 bara när S2 = S3 = 1, och då är antingen S1 = 0, så att `S1'·S2` redan är 1, eller S1 = 1, så att `S1·S3` redan är 1. Den tillför alltså ingen etta som de andra termerna inte redan har, och kan strykas:

```
H1 = S1'·S2 + S1·S3
```

(Regeln bakom kallas konsensusregeln, och Karnaughdiagrammen i kapitel 4 hittar sådana onödiga termer utan att man behöver leta efter dem.)

d) Båda är **väljare**, multiplexrar: en signal bestämmer vilken av två andra som släpps fram till utgången. I avsnitt 2.1.8 var det A som valde mellan B och C; här är det S1 som väljer mellan S2 och S3. Samma funktion kan alltså byggas både med grindar och med kontakter.

KAPITEL 4

Karnaughdiagram och IC-kretsar

Varje uttryck nedan är kontrollerat mot sin sanningstabell, rad för rad. Gör samma sak med dina egna svar: det är den enda kontroll som säger att en gruppering är rätt och inte bara ser rätt ut.

Grindantal räknas med grindar som de ritas i nätet. Där kretsantal efterfrågas räknas med 74-seriens grindar, som har två ingångar (en AND med tre ingångar blir då två AND-grindar i rad).

Övning 4.1 Graykod och grannar

sidan 91

a) Två rutor som ligger bredvid varandra ska skilja sig i **exakt en** ingång. Då betyder två ettor bredvid varandra att utgången inte beror på den ingången, och den kan strykas. I ordningen `00, 01, 10, 11` ändras båda bitarna mellan `01` och `10`, så de två mittkolumnerna skulle se ut som grannar utan att vara det.

b) Ruta 5 är `0101`. Byt en bit i taget:

Bit som byts	Ny kombination	Minterm
A	1101	13
B	0001	1
C	0111	7
D	0100	4

Grannarna är **1, 4, 7 och 13**. I diagrammet ligger de ovanför, till vänster, till höger och nedanför ruta 5.

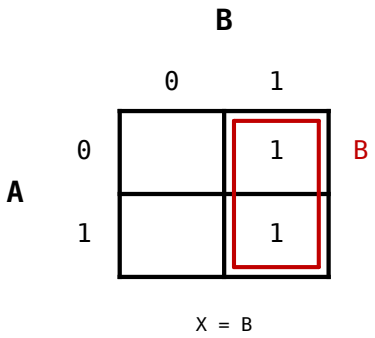
c) Ruta 0 är `0000`, och grannarna är 8 (`1000`), 4 (`0100`), 2 (`0010`) och 1 (`0001`). **Ruta 2** ligger i kolumnen `CD = 10` längst till höger och **ruta 8** på raden `AB = 10` längst ned: båda är grannar över en kant.

d) En grupp om 2^k rutor är precis de rutor där k ingångar tar alla sina kombinationer medan resten är fasta. Tre rutor kan inte vara det: två ingångar ger fyra kombinationer, en ingång ger två. Tre ettor på rad ringas in som två överlappande par.

Övning 4.2 Två ingångar

sidan 91

a) och b)



Figur I.1 Övning 4.2: kolumnen `B = 1` är en grupp.

Båda ettorna ligger i kolumnen `B = 1`. `A` är 0 i den ena och 1 i den andra, och stryks:

```
X = B
```

`X` beror **inte alls** på `A`. Kretsen är en ledning från `B` till `X`, och `A` behöver inte ens kopplas in.

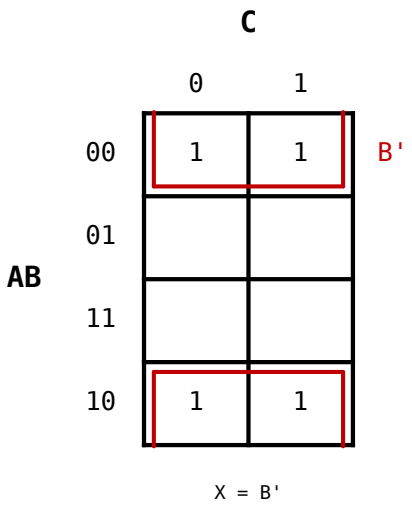
c) Direkt ur tabellen: raderna `01` och `11` ger `X = A'B + AB`. Bryt ut `B`: `X = B(A' + A) = B · 1 = B`. Samma svar. Karnaughdiagrammet gör precis den förenklingen: `A' + A = 1` är det som händer när två grannrutor slås ihop.

Övning 4.3 Tre ingångar

sidan 92

a) Ettorna ligger i rutorna 0, 1, 4 och 5, alltså på raderna `AB = 00` och `AB = 10`. På papperet ligger de överst och nederst, men de **är** grannar: de skiljer sig bara i `A`, och diagrammets översta och nedersta rad hänger ihop runt kanten (avsnitt 4.1.5).

b)



Figur I.2 Övning 4.3: översta och nedersta raden är en grupp över kanten.

En grupp om fyra, över kanten. `A` varierar, `C` varierar, och `B = 0` i alla fyra rutorna:

```
X = B'
```

c) Det minimerade nätet är **en** inverterare. Den direkta avläsningen, `X = A'B'C' + A'B'C + AB'C' + AB'C`, hade krävt fyra AND-grindar med tre ingångar, en OR-grind med fyra ingångar och tre inverterare, åtta grindar i allt.

Vanligt fel: Att ringa in raderna `00` och `10` som två separata par, `A'B'` och `AB'`. Svaret `A'B' + AB'` är korrekt men inte minimalt, och går att förenkla vidare till `B'`. Glöm inte kanterna.

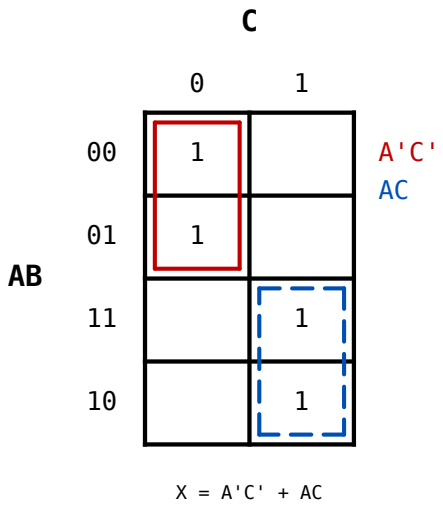
Övning 4.4 Från mintermer

sidan 92

a) Mintermerna 0, 2, 5 och 7 är `000`, `010`, `101` och `111`:

ABC	X
000	1
001	0
010	1
011	0
100	0
101	1
110	0
111	1

b)



Figur I.3 Övning 4.4: grupperna `A'C'` och `AC`.

Rutorna 0 och 2 ligger på raderna `00` och `01` i kolumnen `C = 0`: `A = 0` och `C = 0`, `B` varierar, termen blir `A'C'`. Rutorna 5 och 7 ligger på raderna `10` och `11` i kolumnen `C = 1`: `A = 1` och `C = 1`, termen blir `AC`.

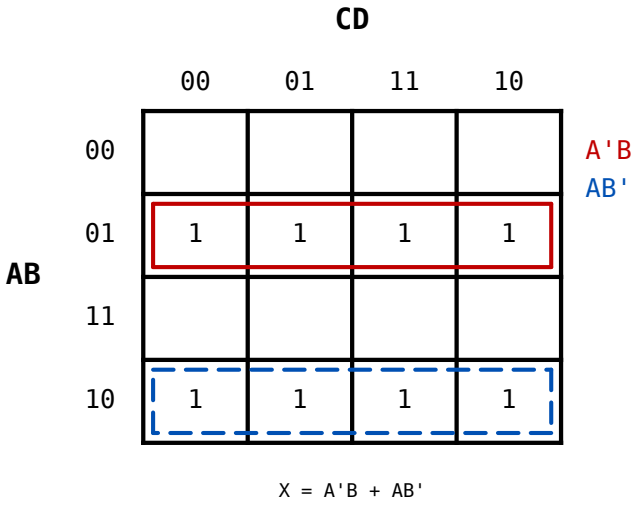
```
X = A'C' + AC
```

Lägg märke till att raderna `11` och `10` ligger bredvid varandra i diagrammet, trots att de kommer i omvänd ordning mot vanlig binär räkning. Det är Graykoden igen.

c) `X` är 1 när `A` och `C` är lika. Det är **XNOR**: `X = (A xor C)'`, en enda grind. `B` påverkar inte utgången.

Övning 4.5 **Två utgångar** sidan 93

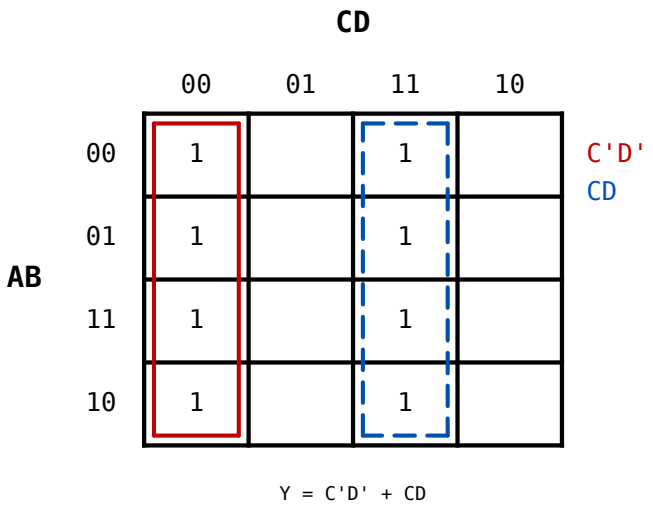
a) och b)



Figur I.4 Övning 4.5, X: raderna `AB = 01` och `AB = 10` är två grupper om fyra.

`X` har ettor på hela raden `AB = 01` och hela raden `AB = 10`. Två grupper om fyra, där `C` och `D` varierar över hela raden:

```
X = A'B + AB'
```



Figur I.5 Övning 4.5, `Y`: kolumnerna `CD = 00` och `CD = 11` är två grupper om fyra.

`Y` har ettor i hela kolumnen `CD = 00` och hela kolumnen `CD = 11`:

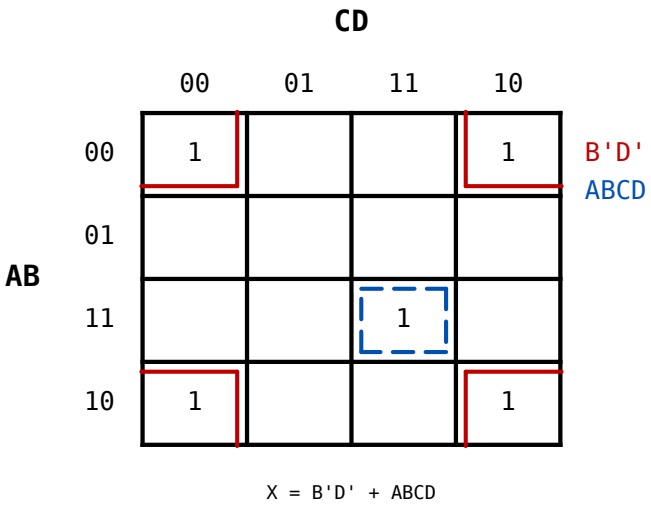
```
Y = C'D' + CD
```

`X` påverkas inte av `C` och `D`: dess grupper är hela **rader**, där `C` och `D` tar alla värden. `Y` påverkas inte av `A` och `B`: dess grupper är hela **kolumner**. När en grupp sträcker sig över hela diagrammet i en riktning betyder det att variablerna i den riktningen saknar betydelse.

c) `X = A'B + AB'` är **XOR** av `A` och `B`. `Y = C'D' + CD` är **XNOR** av `C` och `D`. Två grindar i stället för fyra AND, två OR och fyra inverterare.

Övning 4.6 Hörnen sidan 93

a)



Figur I.6 Övning 4.6: de fyra hörnen är en grupp, och ruta 15 står ensam.

Hörnen 0, 2, 8 och 10 är en grupp om fyra, `B'D'` (se avsnitt 4.1.5). Ruta 15 står ensam:

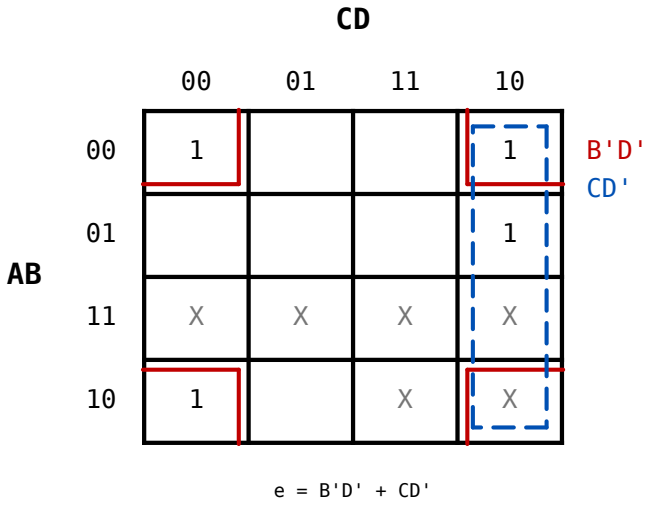
```
X = B'D' + ABCD
```

b) Nej. Ruta 15:s grannar är 7, 11, 13 och 14, och ingen av dem är en etta. En grupp får bara innehålla ettor, så ruta 15 blir en grupp om en, med alla fyra ingångarna kvar i termen. En ensam etta är ingen felgruppering, bara en funktion som inte går att förenkla mer just där.

Övning 4.7 Segment e i en sifferdisplay

sidan 93

a) och b)



Figur I.7 Övning 4.7: segment e, med don't care i rutorna 10–15 och två grupper.

Ettor i 0, 2, 6 och 8. Utan X-rutorna kan 0, 2 och 8 bara bilda par. Med X i ruta 10 blir hörnen en grupp om fyra, `B'D'`. Ruta 6 går ihop med 2, 14 och 10 i kolumnen `CD = 10`, där 14 och 10 är X: `CD'`.

```
e = B'D' + CD'
```

c)

Siffra	ABCD	`B'D'`	`CD'`	e	Ska vara
0	0000	1	0	1	tänd
1	0001	0	0	0	släckt
2	0010	1	1	1	tänd
3	0011	0	0	0	släckt
4	0100	0	0	0	släckt
5	0101	0	0	0	släckt
6	0110	0	1	1	tänd
7	0111	0	0	0	släckt
8	1000	1	0	1	tänd
9	1001	0	0	0	släckt

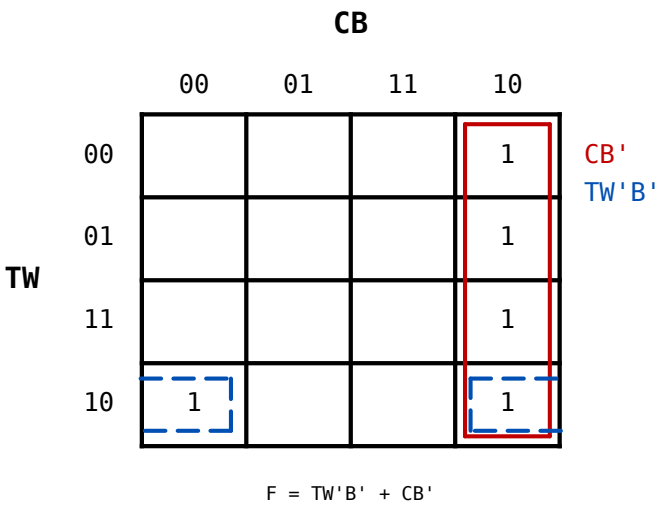
Alla tio stämmer.

d) `1010` ger `B'D' = 1`, så segment e lyser. Displayen visar något som inte är en siffra. Det är inget problem så länge kretsen framför bara skickar BCD, och det var just det antagandet som gav oss rätt att använda X-rutorna. Om kombinationen **kan** komma, är den inte ”don’t care”, och då måste diagrammet ha nollor där.

Övning 4.8 Fläkten

sidan 94

a) och b) `F = 1` när `(T = 1 och W = 0) eller C = 1`, och `B = 0`. Ettorna blir mintermerna 2, 6, 8, 10 och 14 (`TWCB` = 0010, 0110, 1000, 1010, 1110):



Figur I.8 Övning 4.8: grupperna `CB'` och `TW'B'`, den senare över kanten.

Kolumnen `CB = 10` är full: `C = 1` och `B = 0`, termen `CB'`. Raden `TW = 10` har ettor i kolumnerna `00` och `10`, som är grannar över kanten: `T = 1`, `W = 0`, `B = 0`, termen `TW'B'`.

```
F = TW'B' + CB'
```

Läs det tillbaka mot specifikationen: ”varmt och fönstret stängt, och inget brandlarm; eller hög koldioxidhalt, och inget brandlarm”. Det är precis det som stod.

Vanligt fel: Att glömma brandlarmet i en av termerna, till exempel `F = TW'B' + C`. Då går fläkten när koldioxidhalten är hög, även under en brand. En sanningstabell som är härledd ur kravet, och inte ur en skiss av nätet, fångar det direkt.

c) Två inverterare (`W'` och `B'`), en AND-grind med tre ingångar för `TW'B'`, en AND med två för `CB'` och en OR med två, fem grindar. Med tvåingångsgrindar blir det sex, eftersom AND-grinden med tre ingångar blir två (se d). Bryter du ut `B'`, `F = B'(TW' + C)`, räcker två AND-grindar med två ingångar, en OR och samma två inverterare: fem grindar, ingen med mer än två ingångar.

d) Med `F = TW'B' + CB'` och tvåingångsgrindar: två inverterare ur **en 74HC04**, tre AND-grindar (två för `TW'B'`, en för `CB'`) ur **en 74HC08**, och en OR-grind ur **en 74HC32**. Tre kretsar, med fyra inverterare, en AND och tre OR över till annat.

Övning 4.9 Enbart NAND

sidan 94

a) En inverterare för `B'`, en AND-grind för `AB'` och en OR-grind som lägger till `C`.

b) Varje AND blir en NAND, OR-grinden blir en NAND, och `C`, som går direkt till OR-grinden, måste inverteras. Inverterare görs av NAND med ingångarna ihopkopplade:

Grind	Ingångar	Utgång
N1	B, B	`B'`
N2	A, N1	`(AB')'`
N3	C, C	`C'`
N4	N2, N3	X

c) `X = ((AB')' · C')'`. Med De Morgan, `(PQ)' = P' + Q'`: `X = (AB')'' + C'' = AB' + C`. Stämmer.

d) Fyra NAND-grindar, precis **en 74HC00**. AND-OR-NOT-versionen hade krävt tre kretsar: en 74HC04, en 74HC08 och en 74HC32.

Övning 4.10 Enbart NOR

sidan 94

a) `(A + B)C` är en **produkt av summor** (PS-form): en OR-term `(A + B)` och en term med en enda variabel, `C`, ihop med AND.

b) Varje OR blir en NOR, AND-grinden blir en NOR, och `C`, som går direkt till AND-grinden, inverteras med en NOR med ingångarna ihopkopplade:

Grind	Ingångar	Utgång
N1	A, B	`(A + B)'`
N2	C, C	`C'`
N3	N1, N2	X

`X = ((A + B)' + C')'`, och med De Morgan, `(P + Q)' = P'Q'`: `X = (A + B)'' · C'' = (A + B)C`.

c)

A	B	C	N1	N2	X	`(A + B)C`
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	1	0	0	1	0	0
0	1	1	0	0	1	1
1	0	0	0	1	0	0
1	0	1	0	0	1	1
1	1	0	0	1	0	0
1	1	1	0	0	1	1

Tre NOR-grindar, en 74HC02.

Övning 4.11 Kretsar och ben

sidan 95

a) 74HC00, grind 3: ingångar på **ben 9 och 10**, utgång på **ben 8**. 74HC02, grind 3: ingångar på **ben 8 och 9**, utgång på **ben 10**. Samma grindnummer, olika ben: 74HC02 har utgången först i varje grupp.

b) Sex AND-grindar kräver **två 74HC08** (åtta grindar, två över), och inverteraren **en 74HC04**. Tre kretsar.

c)

$$R = \frac{5 - 2{,}1}{0{,}005} = 580\ \Omega$$

Närmaste standardvärde uppåt är **680 Ω**, som ger `I = 2,9 / 680 ≈ 4,3 mA`. Det är gott nog: en lysdiod lyser tydligt vid några milliampere. 560 Ω, närmast nedåt, ger 5,2 mA och går också bra.

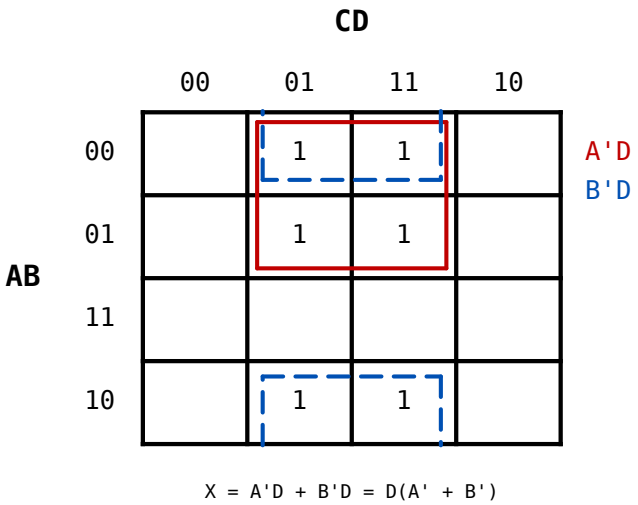
d) En CMOS-ingång som inte är ansluten har ingen bestämd nivå (avsnitt 4.2.4). Den kan läsas som 0 eller 1, växla av störningar, eller hamna mitt emellan, där kretsen drar onödigt mycket ström och kan bli varm. Att det ”fungerar” betyder bara att de oanvända grindarnas utgångar inte är kopplade till något du tittar på. Men störningen påverkar hela kretsen genom den gemensamma matningen, och felet kommer och går med handens närhet och luftfuktigheten. Koppla oanvända ingångar till GND.

Övning 4.12 Kontroll: minimerat och ominimerat nät

sidan 95

a) **För hand.** Mintermerna 1, 3, 5, 7, 9 och 11 är `0001`, `0011`, `0101`, `0111`, `1001` och `1011`:

```
X = A'B'C'D + A'B'CD + A'BC'D + A'BCD + AB'C'D + AB'CD
```



Figur I.9 Övning 4.12: grupperna `A'D` och `B'D`, den senare över kanten.

Alla ettor ligger i kolumnerna `CD = 01` och `CD = 11`, där `D = 1`. Raderna `00` och `01` ger en grupp om fyra, `A'D`. Raderna `00` och `10` ger en grupp om fyra över kanten, `B'D`.

```
X = A'D + B'D
```

b) **Förutsäg.** Den direkta avläsningen: sex AND med fyra ingångar, en OR med sex, och tre inverterare (`A'`, `B'` och `C'`; `D'` behövs aldrig, eftersom `D` står oinverterad i alla sex termerna). Med tvåingångsgrindar blir varje fyraingångs-AND tre grindar, alltså 18 AND, och OR-grinden fem, totalt 26 grindar: fem 74HC08, två 74HC32 och en 74HC04, **åtta kretsar**.

Det minimerade uttrycket: två inverterare, två AND och en OR, fem grindar i **tre kretsar** (74HC04, 74HC08, 74HC32).

c) och d) De två utgångarna ska vara lika på alla sexton raderna: 1 för raderna 1, 3, 5, 7, 9 och 11, och 0 för övriga. De två uttrycken beskriver samma funktion, så varje avvikelse är ett fel i ett av näten, oftast en felkopplad ledning i CircuitVerse, inte i minimeringen. Om de skiljer sig på en rad: kontrollera den raden i båda uttrycken för hand. Det uttryck som ger fel värde där är det felaktiga, och den AND-term eller grupp som borde ha täckt raden är den som ska granskas.

e) `X = D(A' + B') = D · (AB)'`: en NAND-grind och en AND-grind, i en 74HC00 och en 74HC08, **två kretsar**. Eller med enbart NAND: `N1 = (AB)'`, `N2 = (N1 · D)'`, `X = (N2 · N2)' = N1 · D`, tre NAND-grindar i **en** 74HC00. Från åtta kretsar till en, för samma sanningstabell. Det är hela poängen med att minimera, och med att ibland tänka ett steg längre än diagrammet.

Övning 4.13 Multiplexern

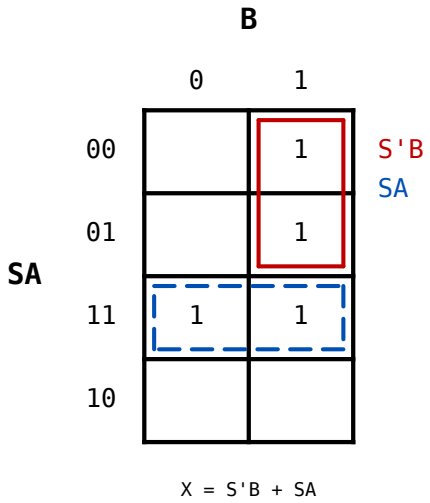
sidan 96

a)

S	A	B	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

När `S = 0` är `X` en kopia av `B`, när `S = 1` en kopia av `A`.

b)



Figur I.10 Övning 4.13: grupperna `S'B` och `SA`.

```
X = S'B + SA
```

c) En inverterare för `S'`, två AND-grindar och en OR-grind. Den övre AND-grinden får `S'` och `B`, den nedre `S` och `A`. Eftersom `S` och `S'` aldrig är 1 samtidigt kan högst en av AND-grindarna släppa igenom något: `S` fungerar som en strömbrytare som öppnar den ena vägen och stänger den andra. Samma idé, med fler väljarbitar, sitter inuti mikrokontrollern och väljer vilken av dess analoga ingångar som ska mätas.

KAPITEL 5

Felsökning av grindnät

I felsökningsfallen är det **resonemanget** som är svaret: vilken mätning som görs härnäst, och vad den kan visa. Den som gissade rätt orsak utan att kunna säga var hen skulle mäta har gissat.

Övning 5.1 Matningen först

sidan 104

a) Röd mätsladd på **ben 14** (VCC), svart på **ben 7** (GND). Du ska se ungefär **5 V**.

b) Skenan kan ha 5 V medan kretsen inte har det: sladden till ben 14 kan sitta i kolumnen bredvid, eller skenan kan vara delad på mitten av kopplingsdäcket. Det som räknas är spänningen **på kretsens ben**, så det är där du mäter.

c) Varje ingång på en CMOS-krets har skyddsdioder mot VCC. När en ingång får 5 V från en strömbrytare leder dioden, och kretsen får en del av sin matning **genom ingången**. Kretsen kan då ge utsignaler som ibland är rätt, men bara när någon ingång råkar vara hög, och med fel nivåer. Det är därför ett matningsfel sällan ser ut som ”ingenting fungerar”, och därför matningen ska mätas först även när något verkar fungera.

Övning 5.2 Vad betyder mätvärdet?

sidan 104

a) 4,7 V är över 3,5 V: logisk **1**, en normal hög utgång.

b) 0,1 V är under 1,5 V: logisk **0**, en normal låg utgång.

c) 2,3 V ligger i det förbjudna området mellan 1,5 och 3,5 V: **ingen giltig nivå**. Troligen en flytande ingång, som inte är ansluten till något.

d) 2,5 V på en utgång som är kopplad till en annan utgång: **två utgångar mot varandra**. Den ena försöker driva hög, den andra låg, och spänningen hamnar någonstans mitt emellan medan en onödigt stor ström går mellan dem. Det är ett fel, och kretsarna blir varma. Två utgångar får aldrig kopplas ihop.

Övning 5.3 En grind som ljuger

sidan 104

a) En AND-grind med ingångarna 1 och 0 ska ge **0**, alltså under 0,3 V på ben 3.

b) Grinden ger 1 för ingångarna 1 och 0: den beter sig som en **OR-grind**. Den troligaste förklaringen är att det sitter en **74HC32** i stället för en 74HC08. De har samma benplacering, så nätet går att koppla upp och fungerar på alla rader där AND och OR råkar ge samma svar (00 och 11), vilket gör felet lätt att missa. Kontrollera texten på kretsen. Ett alternativ är att ben 3 är kortslutet mot 5 V; det skiljer du ut genom att ställa ingångarna på 00: en OR ger då 0, en kortslutning ger fortfarande 5 V.

Övning 5.4 Den varma kretsen

sidan 105

a) **Bryt matningen direkt.** En varm krets håller på att skadas, och det går fort.

b) De två vanligaste orsakerna:

- **kretsen sitter felvänd**, så att den får plus på ben 7 och jord på ben 14;
- **en utgång är kortsluten**: mot matningen, mot jord, mot en annan utgång eller mot en strömbrytare som driver den i motsatt riktning.

c) Kontrollera först riktningen: hacket åt vänster, ben 1 längst ned till vänster, och ben 14 i kolumnen med sladden till +. Om riktningen är rätt, gå igenom kopplingslistan rad för rad och leta efter en ledning som går till ett utgångsben (ben 3, 6, 8 och 11 på en 74HC08) från något annat än en ingång. Multimeterns resistansläge (Ω), med matningen **frånslagen**, visar om ett utgångsben har direkt förbindelse med en skena.

Övning 5.5 Läs felet ur tabellen

sidan 105

a) Förväntat är `X = 1` på raderna 001, 011, 101, 110 och 111. Raderna **001, 011 och 101** är fel: där blir `X = 0`.

b) Alla tre har `C = 1` och `AB = 0`. Det är de rader där `X` bara kan bli 1 **tack vare C**. Raderna där `AB = 1` stämmer. Alltså når `AB` fram genom OR-grinden, men `C` gör det inte: den uppmätta tabellen är precis `X = AB`.

c) Ställ in raden `ABC = 001` och mät på **OR-grindens andra ingång**, U2 ben 2. Förväntat 1. Om den visar 0 sitter felet i ledningen från strömbrytaren C: fel kolumn, eller kanske kopplad till jord tillsammans med de oanvända ingångarna. Om den visar 1 är det OR-grinden eller dess utgång som ska undersökas.

Övning 5.6 Bromsassistenten som inte bromsar

sidan 105

a) Kravet att **föraren alltid ska kunna bromsa**, även när assistanssystemet är trasigt. Nätet släpper inte igenom förarens pedal när `F = 1`, alltså precis när systemet rapporterar ett fel. Just då, när föraren inte kan lita på systemet och bromsar själv, bromsar bilen inte. Det är den farligaste raden i hela tabellen.

b) Pedalen är kopplad **in i OR-grinden tillsammans med detektorerna**, så att den passerar AND-grinden med `F'`. Nätet de byggt är:

```
B = (D + S + R) · F'
```

i stället för det rätta:

```
B = D + (S + R) · F'
```

Kontrollera: med `D = 1` och `F = 1` ger det felaktiga uttrycket `1 · 0 = 0`, och det rätta `1 + ... = 1`. Med `F = 0` ger båda samma svar, vilket är varför tolv rader stämmer.

c) Följ ledningen från strömbrytaren `D`. Den ska gå till **den sista OR-grindens** ingång, den vars utgång är `B`. Ställ in en rad med `D = 1, F = 1` och mät på ingångarna till grinden som driver `B`. I det rätta nätet är det OR-grinden, med `D = 1` på ena ingången. I det felaktiga nätet är det AND-grinden med `F'`, och dess `F'`-ingång ligger på 0: pedalens etta kommer aldrig förbi den.

Övning 5.7 Halvering

sidan 106

a) **Efter grind 8**, i mitten. Då vet du efter en enda mätning i vilken halva felet sitter, och halva nätet är avskrivet.

b) Sexton grindar halveras till åtta, fyra, två och en: högst **fyra mätningar**.

c) Från ingången och grind för grind: i värsta fall **15 mätningar** (efter grind 1 till 15; om alla är rätt är det grind 16 som felar).

Övning 5.8 Den flimrande lysdioden

sidan 106

a) En **flytande ingång** någonstans i nätet: en ingång som inte är ansluten till något, och som därför påverkas av laddning från omgivningen, till exempel från din hand.

b) Mät spänningen på varje ingång i nätet, också på oanvända grindar. En ansluten ingång visar stadigt nära 0 V eller 5 V; den flytande visar ett värde som vandrar, eller ligger mitt emellan, och ändras när du rör handen. Jämför sedan med kopplingslistan: vilken ingång saknar en rad?

c) En HC-krets är byggd i CMOS, och dess ingångar har extremt hög resistans: nästan ingen ström flyter in eller ut, så ingenting bestämmer spänningen. En LS-krets (TTL) har ingångar där en liten ström flyter **ut** ur ingången, och den strömmen drar en okopplad ingång mot hög nivå. En flytande LS-ingång läses därför nästan alltid som 1. Det gör felet mindre synligt på LS, men det är fel där också.

Övning 5.9 Planera fläkten

sidan 106

a) Grindtilldelning för `F = TW'B' + CB'`:

Grind i nätet	Krets	Grind	Ingångar (ben)	Utgång (ben)
NOT: `W'`	U1 = 74HC04	1	W → 1	2
NOT: `B'`	U1 = 74HC04	2	B → 3	4
AND: `TW'`	U2 = 74HC08	1	T → 1, U1 ben 2 → 2	3
AND: `TW'B'`	U2 = 74HC08	2	U2 ben 3 → 4, U1 ben 4 → 5	6
AND: `CB'`	U2 = 74HC08	3	C → 9, U1 ben 4 → 10	8
OR: `F`	U3 = 74HC32	1	U2 ben 6 → 1, U2 ben 8 → 2	3

b) Kopplingslista:

Nr	Från	Till
1	U1, U2 och U3 ben 14	+5 V
2	U1, U2 och U3 ben 7	GND
3	strömbrytare W	U1 ben 1
4	strömbrytare B	U1 ben 3
5	strömbrytare T	U2 ben 1
6	U1 ben 2	U2 ben 2
7	U2 ben 3	U2 ben 4
8	U1 ben 4	U2 ben 5
9	U1 ben 4	U2 ben 10
10	strömbrytare C	U2 ben 9
11	U2 ben 6	U3 ben 1
12	U2 ben 8	U3 ben 2
13	U3 ben 3	lysdiod F
14	U1 ben 5, 9, 11, 13	GND
15	U2 ben 12, 13	GND
16	U3 ben 4, 5, 9, 10, 12, 13	GND

`B'` från U1 ben 4 används av två grindar, så två rader utgår från samma ben. På kopplingsdäcket sätter du båda sladdarna i kolumnen för U1 ben 4.

c) Fläkten ska starta på hög temperatur när `T = 1`, `W = 0` och `B = 0`. Om `C = 1` samtidigt startar den ändå, tack vare `CB'`. Den felaktiga raden är alltså **`TWCB = 1000`**, där bara termen `TW'B'` kan göra `F = 1`. Ställ in den och mät längs den termen: U2 ben 3 (`TW'`, ska vara 1) och U2 ben 6 (`TW'B'`, ska vara 1), och sedan U3 ben 1.

Övning 5.10 Kontroll: en utgång under last

sidan 106

a) Med 5 V på utgången och 2 V över lysdioden ligger 3 V över motståndet:

$$I = \frac{5 - 2}{330} \approx 9{,}1 \text{ mA}$$

b) Typiska mätvärden på en station är ungefär: hög utgång **4,4–4,7 V**, lysdiod **1,9–2,0 V**, låg utgång **0,0–0,1 V**. Dina egna värden är svaret; skriv upp dem.

c) Med till exempel 4,5 V på utgången och 1,95 V över lysdioden:

$$I = \frac{4{,}5 - 1{,}95}{330} \approx 7{,}7 \text{ mA}$$

d) Antagandet att utgången ger **exakt 5 V** var fel. En hög utgång har en viss inre resistans, och när den levererar ström tappar den en del av spänningen: med siffrorna ovan ungefär

0,5 V vid 7,7 mA, alltså runt 65 Ω. Lysdiodens spänning kan också skilja sig lite från 2 V, beroende på färg och exemplar.

Den uppmätta höga nivån ligger väl över databladets V_{OH} på 3,84 V vid 4 mA, trots att strömmen här är dubbelt så stor. Den låga nivån ligger nästan på 0 V, långt under V_{OL} på 0,33 V, eftersom lysdioden är släckt när utgången är låg och ingen ström går genom utgången alls. Slutsatsen: skillnaden mellan kalkyl och mätning är några tiondels volt och ett par milliampere, och den kommer från utgångens inre resistans. Den spelar ingen roll för lysdioden, men den visar varför en utgång inte ska belastas mer än nödvändigt.

Övning 5.11 Fel krets i sockeln

sidan 107

a) Ben 1 på en 74HC02 är **utgången** på grind 1, `1Y`.

b) När NOR-grindens ingångar (ben 2 och 3) båda är 0 försöker grinden ge en etta på ben 1. Om strömbrytaren samtidigt håller ben 1 på 0 V kortsluts utgången mot jord genom strömbrytaren. Strömmen begränsas bara av utgångstransistorns inre resistans, några tiotal ohm, och blir tiotals milliampere: över kretsens gränsvärde på 25 mA. Kretsen blir varm och kan förstöras. Samma sak händer åt andra hållet när strömbrytaren ger 5 V och grinden försöker ge 0.

c) Ben 3 är en **ingång** på en 74HC02 (`1B`). En ingång driver ingenting, så lysdioden lyser aldrig, vad strömbrytarna än står på. Lysdioden och motståndet håller dessutom ben 3 nära jord, så NOR-grinden ser ingången som 0. Kombinationen, en lysdiod som aldrig lyser och en krets som blir varm, är kännetecknet för en krets med fel benplacering.

KAPITEL 6

Sekvensnät, vippor och mikrodatorns uppbyggnad

Övning 6.1 Kombinatoriskt eller sekventiellt?

sidan 128

a) **Kombinatoriskt.** Larmet beror bara på vad givarna visar just nu.

b) **Sekventiellt.** Med båda knapparna släppta kan reläet vara draget eller inte, beroende på vilken knapp som trycktes sist. Kretsen minns.

c) **Kombinatoriskt.** Samma siffra in ger alltid samma segment tända.

d) **Sekventiellt.** Vilken färg som tänds härnäst beror på vilken som lyser nu. Trafikljuset måste minnas sitt tillstånd.

e) **Sekventiellt.** Siffran 1 öppnar bara om 3 och 7 kom före. Låset måste minnas hur långt i koden det har kommit.

f) **Kombinatoriskt.** Bromsbeslutet beror bara på de fyra ingångarna i det ögonblicket.

Frågan att ställa är alltid: **kan samma ingångar ge olika utgång?** Om ja, finns ett minne.

Övning 6.2 SR-låset

sidan 128

a)

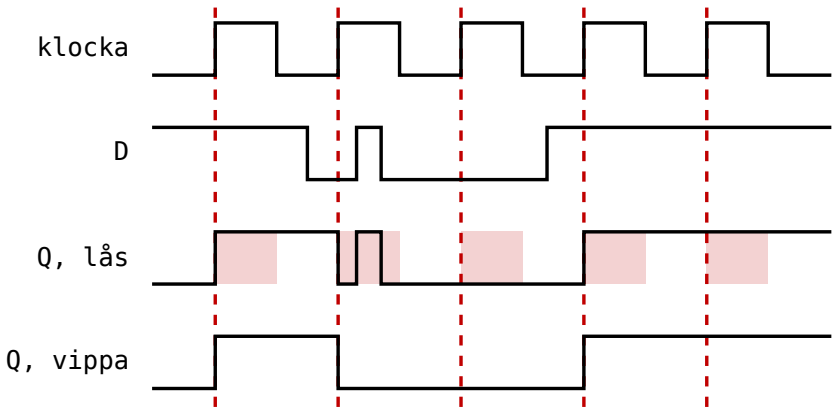
Steg	S	R	Q	Varför
start	0	0	0	givet
1	1	0	1	ettställ
2	0	0	1	minns
3	0	0	1	minns
4	0	1	0	nollställ
5	0	0	0	minns
6	1	0	1	ettställ
7	0	0	1	minns

b) Nej: steg 2, 3 och 7 har `Q = 1` och steg 5 har `Q = 0`, trots samma ingångar. Utgången beror på vad som hände tidigare, och det är definitionen av ett sekvensnät. Ingen sanningstabell med bara `S` och `R` som ingångar kan beskriva låset; tabellen måste också ha med det tidigare `Q`.

c) Med `S = R = 1` har båda NOR-grindarna en etta på en ingång, så båda utgångarna blir 0. `Q` och `Q'` är då inte längre varandras invers. Värre är vad som händer när båda släpps samtidigt: båda grindarna ser då två nollor och vill bli 1, och den grind som hinner först vinner. Resultatet går inte att förutsäga. En kombination som ger ett oförutsägbart minne är värdelös, därför är den förbjuden.

Övning 6.3 Lås och vippa

sidan 129



Figur I.11 Klockan, `D` och utgångarna för lås och vippa.

a) Låset följer `D` medan klockan är hög (de skuggade fönstren) och håller medan den är låg: det blir 1 vid den första flanken, eftersom `D` redan är 1 då; 0 vid den andra, eftersom `D` då är 0; 1 och sedan 0 igen under den korta pulsen, som kommer medan klockan fortfarande är hög; oförändrat 0 genom det tredje fönstret; 1 vid den fjärde flanken, och kvar på 1.

b) Vippan läser `D` vid varje stigande flank: 1, 0, 0, 1, 1. Den enda skillnaden mot låset är pulsen efter den andra flanken.

c) Pulsen kommer medan klockan är **hög**, strax efter den andra flanken. Då är låset genomsläppligt, och pulsen går rakt igenom till låsets `Q`. Vippan ser den inte: den läste `D` vid den andra flanken, innan pulsen kom, och läser nästa gång vid den tredje, när pulsen redan är över. Hade pulsen kommit medan klockan var låg hade inte heller låset släppt igenom den. Det är skillnaden mellan nivåkänsligt och flanktriggat, i ett enda exempel.

Vanligt fel: Att låta låset ändra sig bara vid flankerna, som vippan. Låset ändrar sig när som helst medan klockan är hög, inte bara i början av det fönstret.

Övning 6.4 Kontroll: bygg ett SR-lås

sidan 129

a) och c) Tabellen från övning 6.2 ska stämma steg för steg. `Q'` ska vara inversen av `Q` i varje steg, eftersom `S = R = 1` aldrig förekommer i följden. Om ett steg skiljer sig är den vanligaste orsaken en felkopplad återkoppling: `Q` ska gå till den **andra** grindens ingång, inte tillbaka till sin egen.

d) Innan någon ingång ändrats finns inget ”rätt” värde. Kretsen har två stabila lägen, `Q = 0` och `Q = 1`, och vilket den hamnar i beror på i vilken ordning simulatorn räknar ut grindarna. En riktig krets hamnar på samma sätt i ett okänt läge när strömmen slås på, beroende på små skillnader mellan transistorerna. Därför behöver varje verkligt system en **reset**: ett sätt att ställa alla minnen i ett känt läge vid start. En mikrokontroller har en resetingång av just det skälet.

e) Med `S = R = 1` blir både `Q` och `Q'` 0. När båda släpps samtidigt ser båda grindarna två nollor, och i en simulator som räknar båda grindarna i samma steg kan de börja **svänga**: båda blir 1, sedan båda 0, och så vidare. Andra gånger lägger sig kretsen i ett av lägena. Båda utfallen bekräftar avsnitt 6.1.3: kombinationen ger inget förutsägbart minne.

Övning 6.5 Klockan

sidan 130

a)

$$T = \frac{1}{16\,000\,000\ \text{Hz}} = 62{,}5\ \text{ns}$$

b) 16 000 000 cykler per sekund, alltså **16 000 cykler per millisekund**.

c) $1000 \cdot 62{,}5\ \text{ns} = 62\,500\ \text{ns} =$ **62,5 µs**.

d) Halva frekvensen ger dubbla cykeltiden, 125 ns, och **125 µs**.

Övning 6.6 Räknare

sidan 130

a) En 3-bitars räknare börjar om efter 8 steg. $11 = 8 + 3$, så den står på 3, **`011`**.

b) 999 kräver **10 bitar**: $2^9 = 512$ räcker inte, $2^{10} = 1024$ gör det. Tio vippor.

c) Varje bit har halva frekvensen av biten före: `Q0` 8 Hz, `Q1` 4 Hz, `Q2` 2 Hz och **`Q3` 1 Hz**. `Q3` är en fyrkantvåg som är hög en halv sekund och låg en halv sekund.

d) Det står på **0**. 255 är `1111 1111`, och ett till ger `1 0000 0000`, men den nionde biten finns inte i ett 8-bitars register. Den tappas bort, och kvar blir `0000 0000`. Det kallas rundgång. När mikrodatorn adderar hamnar den bortfallna biten i carry-flaggan, men inte när den räknar upp med `inc`: `inc` lämnar carry-flaggan orörd (avsnitt 8.1.3).

Övning 6.7 Skiftregister

sidan 130

a)

Efter flank	in	Q0	Q1	Q2	Q3
start		0	0	0	0
1	1	1	0	0	0
2	1	1	1	0	0
3	0	0	1	1	0
4	1	1	0	1	1

b) Före: `Q3 Q2 Q1 Q0 = 0011`. Vid flanken tar varje vippa värdet från vippan före, och `Q0` får `in = 0`: `Q0 = 0`, `Q1 = 1`, `Q2 = 1`, `Q3 = 0`. Registret innehåller **`0110`, talet 6**. Ett steg mot de mer signifikanta bitarna är **multiplikation med 2**, precis som att lägga till en nolla sist i ett decimalt tal multiplicerar med 10. Mikrodatorns instruktion `lsl` gör just det (kapitel 8).

Övning 6.8 Blockschemat

sidan 130

a) Programräknaren, PC.

b) ALU:n.

c) Programminnet (flash).

d) Statusregistret, SREG, med noll-flaggan.

e) Styrenheten.

f) En utport, till exempel PORTB.

g) Dataminnet (SRAM).

h) Klockan.

Övning 6.9 Instruktionscykeln

sidan 131

a)

Steg	PC före	Instruktion	Händer	r16	r17	PC efter
1	0	`ldi r16, 10`	r16 = 10	10	?	1
2	1	`ldi r17, 20`	r17 = 20	10	20	2
3	2	`add r16, r17`	r16 = 10 + 20	30	20	3
4	3	`rjmp 3`	PC = 3	30	20	3
5	3	`rjmp 3`	PC = 3	30	20	3
6	3	`rjmp 3`	PC = 3	30	20	3

`r17` har inget känt värde före steg 2; programmet har inte skrivit något där än.

b) Programmet hoppar till sig självt, om och om igen, så länge strömmen är på. En mikrokontroller har inget operativsystem att återvända till när programmet är slut. Utan hoppet skulle PC fortsätta till adress 4, 5, 6 och vidare, och CPU:n skulle utföra det som råkar ligga i flashminnet där, som inte är ett program. En oändlig loop är hur ett program för en mikrokontroller slutar.

Övning 6.10 Tre minnen

sidan 131

a) Programmet i **flashminnet**. Knapptryckningsräknaren i **SRAM**. Kalibreringsvärdet i **EEPROM**, som bevaras utan ström och kan skrivas en byte i taget.

b) Flash och EEPROM lagrar laddning på ett sätt som finns kvar utan matning. SRAM är i princip lås (avsnitt 6.1.4), och ett lås minns bara så länge dess grindar har ström: när matningen försvinner försvinner återkopplingen, och därmed minnet.

c) Programminnet och dataminnet är **separata minnen med var sin buss**. CPU:n kan därför hämta nästa instruktion samtidigt som den läser eller skriver data, och ett tal i programminnet nås med andra instruktioner än ett tal i dataminnet.

Övning 6.11 Bussar och minnesstorlek

sidan 131

a) $2048 = 2^{11}$, så **11 adressbitar**.

b) 32 kB = 32 768 byte, och med 2 byte per instruktion blir det **16 384 instruktioner**. $16\,384 = 2^{14}$, så programräknaren behöver **14 bitar**. (Några få AVR-instruktioner är dubbelt så långa, 32 bitar, och tar två platser; det ändrar inte hur många platser som finns.)

c) 2^8 = **256** olika värden, 0–255.

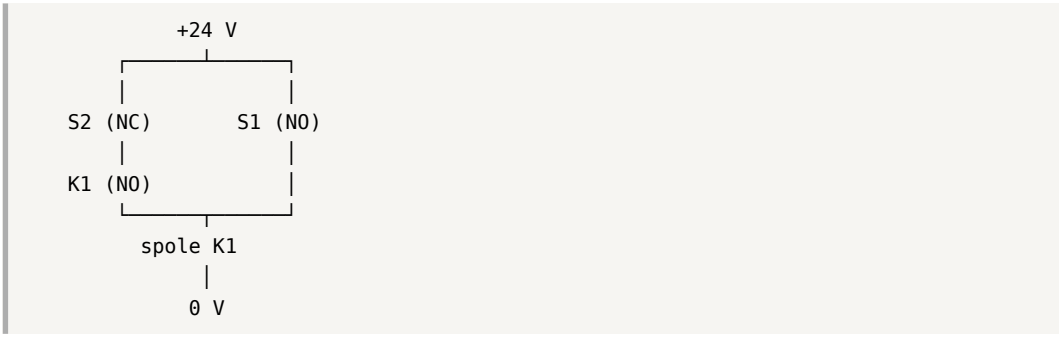
Övning 6.12 Självhållning och SR-lås

sidan 131

a) Startknappen `S1` ettställer, som **S**. Stoppknappen `S2` nollställer, som **R**.

b) I reläkretsen sitter `S2` i serie med allt annat i spolens strömväg. När den trycks bryts strömmen, vad `S1` än gör: **stopp vinner**. Kretsen kallas därför **frånslagsdominerande**. I NOR-låset ger `S = R = 1` i stället två nollor på utgångarna och ett oförutsägbart läge efteråt. Reläkretsen har alltså ingen förbjuden kombination: den har en regel för vem som vinner, och regeln är vald för säkerhetens skull.

c) Låt `S1` gå förbi `S2`: koppla `S1` direkt från +24 V till spolen, parallellt med seriekopplingen av `S2` och `K1`:s kontakt. Då drar `S1` alltid reläet, även när `S2` är nedtryckt, och när `S1` släpps håller självhållningen genom `K1` bara om `S2` inte är nedtryckt.



Den varianten är ovanlig för maskiner, eftersom **stopp ska alltid vinna**: den som trycker på stopp ska kunna lita på att maskinen stannar, även om någon annan samtidigt håller in start eller en startknapp har fastnat. Nödstopp kopplas därför alltid så att de bryter, och de bryter allt.

KAPITEL 7

Mikrodatorn och Microchip Studio

Övning 7.1 Arbetsbänken

sidan 148

a) 32 register, `r0`–`r31`, med 8 bitar, en byte, vardera.

b) `r16`–`r31`.

c) `ldi` är 16 bitar lång. Fyra av bitarna är operationskoden och åtta är konstanten, så det blir bara **fyra bitar** kvar till registret. Fyra bitar kan räkna från 0 till 15, alltså skilja på sexton register, och processorn lägger till 16: `0000` betyder `r16` och `1111` betyder `r31`. Det finns inget bitmönster som betyder `r5`.

d) Ladda konstanten i ett övre register och kopiera den:

```
ldi r16, 7                    ; r16 = 7
mov r5, r16                   ; r5 = r16 = 7
```

`mov` fungerar med alla 32 register, eftersom den inte har någon konstant att få plats med.

Övning 7.2 Tre minnen

sidan 149

a) I programminnet, flashminnet.

b) I dataminnet, SRAM. Under SRAM, på dataminnets lägsta adresser, ligger också registren och I/O-registren.

c) Flashminnet behåller innehållet utan ström. Därför börjar ett Arduino-kort köra sitt program så fort det får ström, utan att programmet behöver laddas in igen. SRAM töms när strömmen bryts.

d) `RAMEND`, den sista adressen i SRAM. Där börjar stacken, som kapitel 10 handlar om.

Övning 7.3 Samma tal, olika skrivsätt

sidan 149

a) $45 = 32 + 8 + 4 + 1 =$ `0b00101101`, och `0010 1101` = `0x2D`:

```
ldi r16, 45
ldi r16, 0b00101101
ldi r16, 0x2D
```

b)

Decimalt	Binärt	Hexadecimalt
255	`0b11111111`	`0xFF`
16	`0b00010000`	`0x10`
100	`0b01100100`	`0x64`

$100 = 64 + 32 + 4$, alltså bitarna 6, 5 och 2.

c)

- `0b01010101` $= 64 + 16 + 4 + 1 =$ **85**.
- `0x3F` $= 3 \cdot 16 + 15 =$ **63**.
- `$80` $= 8 \cdot 16 =$ **128**. `$` är ett annat sätt att skriva `0x`.
- `'a'` = `0x61` $= 6 \cdot 16 + 1 =$ **97**.

d) Binärt. Då står varje bit, och därmed varje lysdiod, för sig i programmet: `0b10000001` tänder de två yttersta lysdioderna, vilket inte syns lika lätt i `0x81` och inte alls i `129`.

Övning 7.4 Förutsäg registren

sidan 150

Efter	r16	r17	r18	r19	r20
`ldi r16, 0x10`	0x10	0x00	0x00	0x00	0x00
`ldi r17, 7`	0x10	0x07	0x00	0x00	0x00
`mov r18, r17`	0x10	0x07	0x07	0x00	0x00
`inc r18`	0x10	0x07	0x08	0x00	0x00
`inc r18`	0x10	0x07	0x09	0x00	0x00
`dec r16`	0x0F	0x07	0x09	0x00	0x00
`mov r19, r16`	0x0F	0x07	0x09	0x0F	0x00
`clr r17`	0x0F	0x00	0x09	0x0F	0x00
`ser r20`	0x0F	0x00	0x09	0x0F	0xFF
`dec r20`	0x0F	0x00	0x09	0x0F	0xFE

Två rader brukar ge fel svar:

- **`dec r16` från `0x10`** blir `0x0F`, inte `0x09`. Hexadecimalt räknar man ned från `10` till `0F`, precis som decimalt från 10 till 9. Den som svarar `0x09` har räknat hexadecimalt som om det vore decimalt.
- **`mov r19, r16`** kopierar det värde `r16` har **då**, `0x0F`, inte det den fick i början.

Övning 7.5 Hitta felen

sidan 151

Sex fel:

1. **`main`** saknar kolon. Utan kolon läser assemblern ordet som en instruktion, och det finns ingen instruktion som heter `main`.
2. **`ldi r16 5`** saknar kommatecken mellan operanderna.
3. **`ldi r10, 3`**: `ldi` når bara `r16`–`r31`. Använd till exempel `r20`.
4. **`mov r17 r16`** saknar kommatecken.
5. **`inc r20, 1`**: `inc` har bara en operand. Den ökar alltid med ett.
6. **`rjmp mian`**: etiketten heter `main`. Assemblern känner inte till `mian`.

Det rättade programmet:

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi r16, 5
    ldi r20, 3
    mov r17, r16
    inc r20
    rjmp main
```

I Microchip Studio får du inte alltid alla sex felen på en gång. Rätta det första, bygg om, och fortsätt med nästa, som i avsnitt 7.3.4.

Övning 7.6 Ditt första egna program

sidan 151

Programmet självt har inget lösningsförslag här; simulatorn visar om det gör vad det ska.

a)

Register	Decimalt	Hexadecimalt
`r16`	12	0x0C
`r17`	15	0x0F
`r18`	0	0x00
`r19`	170	0xAA

b) Värdena ska stämma. Om `r19` är 0 har `clr r18` hamnat före `mov r19, r18`: ordningen spelar roll, eftersom `mov` kopierar det värde registret har just då.

c) Programmet har åtta instruktioner i `main`: `ldi`, `mov`, tre `inc`, `ldi`, `mov` och `clr`. `rjmp main` tar 2 cykler och de åtta instruktionerna 1 cykel var: $2 + 8 = \mathbf{10}$ cykler. Cycle Counter ska visa 10 när den gula pilen står på `rjmp end`.

Övning 7.7 Maskinkod för hand

sidan 151

a) Mönstret är `1110 KKKK dddd KKKK`, med konstantens höga halva i de första `K` och den låga halvan i de sista.

`ldi r16, 0x2A`: konstanten `0x2A` = `0010 1010`, registret `r16` ger `dddd` = 16 − 16 = 0 = `0000`:

```
1110  0010  0000  1010  =  0xE20A
```

De övriga på samma sätt:

Instruktion	K höga halvan	dddd	K låga halvan	Maskinkod
`ldi r16, 0x2A`	`0010`	`0000` (r16)	`1010`	`0xE20A`
`ldi r24, 0x2A`	`0010`	`1000` (r24)	`1010`	`0xE28A`
`ldi r31, 0xFF`	`1111`	`1111` (r31)	`1111`	`0xEFFF`
`ldi r20, 0x03`	`0000`	`0100` (r20)	`0011`	`0xE043`

En genväg: i hexadecimal form är maskinkoden `E`, konstantens första siffra, registrets nummer minus 16, och konstantens andra siffra. `ldi r24, 0x2A` blir `E 2 8 A`.

b) Listfilen visar samma fyra ord, `e20a`, `e28a`, `efff` och `e043`, på adresserna där instruktionerna hamnade.

c) De ska stämma. Det vanligaste felet är att skriva konstanten i ett stycke, till exempel `1110 0000 0010 1010` = `0xE02A` för `ldi r16, 0x2A`. Det är fel ordning på fälten: konstanten är delad, med registerfältet emellan. Ett annat vanligt fel är att skriva registrets nummer utan att dra ifrån 16, `1 0000`, vilket inte ens får plats i fyra bitar.

d) Registerfältet har fyra bitar och betyder `r16`–`r31`. Det finns inget värde i fältet som betyder `r15`, så det finns ingen maskinkod att skriva, och assemblern ger ett fel.

Övning 7.8 Programmet som inte tog slut

sidan 152

a) Processorn slutar aldrig köra. Det finns inget operativsystem att återvända till, så när programmet har gjort sitt måste det stanna någonstans, och `rjmp end` hoppar till sig självt för evigt.

b) Processorn fortsätter med det som ligger efter programmet i flashminnet. Där har du inte skrivit något: ett tomt flashminne innehåller bara ettor, `0xFF`, och processorn försöker tolka dem som instruktioner. I praktiken löper programräknaren då igenom resten av minnet, slår runt till adress 0 och kör programmet från början igen, om och om igen. Programmet ser ut att starta om av sig självt, vilket är ett förvirrande fel att leta efter om man inte vet om det.

c) Adresserna närmast 0 är reserverade för avbrottsvektorer. Boken använder inga avbrott, men vanan att börja med ett hopp gör att programmet fortsätter att fungera om något läggs där, och det är så nästan alla AVR-program börjar (avsnitt 7.2.6).

Övning 7.9 Tillåtet eller inte?

sidan 152

a) **Tillåtet.** `mov` kopierar mellan två valfria register, `r0`–`r31`.

b) **Inte tillåtet.** `ldi` når bara `r16`–`r31`. Skriv `ldi r16, 16` och `mov r5, r16`.

c) **Inte tillåtet.** `mov` kopierar mellan register; den har ingen konstant. Skriv `ldi r16, 5`.

d) **Inte tillåtet.** `ldi` laddar en konstant, inte ett register. Skriv `mov r16, r5`.

e) **Tillåtet.** `inc` fungerar med alla 32 register.

Övning 7.10 Hur lång tid tar programmet? sidan 152

a) `rjmp main` tar 2 cykler och de tio instruktionerna i `main` 1 cykel var: 2 + 10 = **12** cykler.

b) 12 · 62,5 ns = **750 ns**, det vill säga 0,75 mikrosekunder.

c) 1 s / 750 ns ≈ 1,3 miljoner gånger per sekund. Så snabbt går det; det är därför programmet måste stegas i simulatorn för att man ska se något.

Övning 7.11 Maskinkod baklänges sidan 153

Dela ordet i fyra hexadecimala siffror: `E`, konstantens höga halva, registrets nummer minus 16, och konstantens låga halva.

a) `0xE3F5`: konstanten `0x35`, registerfältet `F` = 15, 15 + 16 = 31. **`ldi r31, 0x35`**.

b) `0xE0A0`: konstanten `0x00`, registerfältet `A` = 10, 10 + 16 = 26. **`ldi r26, 0x00`**.

c) `0xEF0F`: konstanten `0xFF`, registerfältet `0` = 0, alltså `r16`. **`ldi r16, 0xFF`**. Det är exakt vad `ser r16` gör, och `ser` är i själva verket bara ett annat namn på `ldi` med konstanten `0xFF`: assemblern gör samma maskinkod av båda. Det syns i listfilen i avsnitt 7.2.7, där `ser r20` blev `ef4f`.

KAPITEL 8

Aritmetik, logik och flaggor

Övning 8.1 Binär addition för hand sidan 174

a) 45 + 23 = 68:

```
   minne     0 1 1 1 1 1 1 0
             0 0 1 0 1 1 0 1     (45)
          +  0 0 0 1 0 1 1 1     (23)
             ---------------
             0 1 0 0 0 1 0 0     (68 = 0x44)
```

Resultatet är `0x44`, och `C = 0`: summan får plats.

b) `0x9C` = 156 och `0x7A` = 122, summan är 278:

```
             1 0 0 1 1 1 0 0     (0x9C)
          +  0 1 1 1 1 0 1 0     (0x7A)
             ---------------
          1  0 0 0 1 0 1 1 0     (0x16, och en nionde bit)
```

Registret får `0x16` = 22, och den nionde biten hamnar i `C`: **`C = 1`**. Kontroll: 22 + 256 = 278.

c) `0x0F + 0x01 = 0x10`, `C = 0`, `Z = 0`, `N = 0`. Dessutom **`H = 1`**, *half carry*, eftersom den låga halvan, `1111 + 0001`, gav en minnessiffra ut ur bit 3 och in i bit 4.

Övning 8.2 Mätaren slår om

sidan 174

a) 254, 255, 0, 1, 2.

b) Efter den **tredje** `inc`, när `r16` slår om från 255 till 0. När alla fem är körda är `Z = 0`, eftersom den sista `inc` gav 2. Flaggan beskriver bara den senaste instruktionen.

c) `C = 0`, som före. `inc` ändrar aldrig `C`, inte ens när värdet slår om.

d) Använd `add` i stället för `inc`, med ett register som innehåller 1:

```
ldi r17, 1
add r16, r17                    ; Sets C = 1 when r16 wraps from 255 to 0.
```

Man kan också behålla `inc` och titta på `Z` direkt efter den: `Z = 1` efter `inc` betyder precis att räknaren just slog om till 0.

Övning 8.3 Negativa tal

sidan 175

a)

Tal	Tvåkomplement
−1	`0xFF`
−10	`0xF6`
−100	`0x9C`
−128	`0x80`

Metoden för −10:

```
10           =  0000 1010
invertera    =  1111 0101
lägg till 1  =  1111 0110  =  0xF6
```

Kontroll: 256 − 10 = 246 = `0xF6`.

b) `0xF0` = 240, och 240 − 256 = **−16**. `0x81` = 129, och 129 − 256 = **−127**. `0x7F` = 127 har bit 7 = 0, så talet är positivt: **127**.

c)

- `neg` av `0x01` blir `0xFF`, alltså −1.
- `neg` av `0x00` blir `0x00`. −0 är 0.
- `neg` av `0x80` blir **`0x80`** igen. `0x80` är −128, och +128 finns inte bland talen −128 till 127, så resultatet slår om och blir −128 igen. Det är det överraskande svaret, och processorn sätter `V = 1` för att tala om att resultatet är fel med tecken.

Övning 8.4 Addera en konstant

sidan 175

c) `r16` = 65 = `0x41`. `r17` = 256 + 30 − 100 = **186** = `0xBA`, och eftersom 100 är större än 30 har subtraktionen lånat: **`C = 1`**. Med tecken är `0xBA` = −70, vilket är rätt svar på 30 − 100.

Den som svarar `r17` = −70 och tänker sig att registret ”innehåller ett minustecken” har rätt i sak men fel om vad registret innehåller: det innehåller bitarna `1011 1010`, och om de betyder 186 eller −70 bestämmer programmet.

Övning 8.5 Flaggorna

sidan 175

a) och c) Simulatorn visar följande.

Operation	Resultat	C	Z	N	V	S
`0x50 + 0x50` (`add`)	`0xA0`	0	0	1	1	0
`0xF0 + 0x20` (`add`)	`0x10`	1	0	0	0	0
`0x30 - 0x30` (`sub`)	`0x00`	0	1	0	0	0
`0x00 - 0x01` (`sub`)	`0xFF`	1	0	1	0	1
`0x80 - 0x01` (`sub`)	`0x7F`	0	0	0	1	1
`0xFF + 1` med `inc`, när `C` är 1 före	`0x00`	1	1	0	0	0

Rad för rad:

- **`0x50 + 0x50`**: 80 + 80 = 160 får plats utan tecken, så `C = 0`. Med tecken är det två positiva tal som gav ett negativt, `0xA0` = −96: **`V = 1`**. `S = N xor V = 0`, det rätta svaret +160 är positivt.
- **`0xF0 + 0x20`**: 240 + 32 = 272 får inte plats: **`C = 1`**, resultatet 272 − 256 = 16. Med tecken är det −16 + 32 = 16, som är rätt: `V = 0`.
- **`0x30 - 0x30`**: noll, `Z = 1`. Inget lån, `C = 0`.
- **`0x00 - 0x01`**: 0 − 1 kräver ett lån: `C = 1`, och resultatet är 255. Med tecken är det −1, vilket är rätt: `V = 0`, `N = 1`, `S = 1`.
- **`0x80 - 0x01`**: utan tecken 128 − 1 = 127, inget lån, `C = 0`. Med tecken är det −128 − 1 = −129, som inte finns: resultatet slår om till +127, **`V = 1`**. `S = N xor V = 1`: det rätta svaret, −129, är negativt.
- **`inc` från `0xFF`**: resultatet blir 0 och `Z = 1`, men **`C` förblir 1**, eftersom `inc` aldrig ändrar `C`. Den som svarar `C = 0` här har glömt regeln i avsnitt 8.1.3.

Vanligaste felen: **V-kolumnen**, där man tänker ”fick det plats?” utan tecken i stället för med tecken, och **sista raden**, där man tror att `inc` sätter `C`.

Övning 8.6 Masker

sidan 176

a) `0b11010000`. De fyra nedre bitarna nollställs, de övre behålls.

b) `0b11011111`. Masken `0x0C` = `0b00001100` ettställer bit 3 och bit 2, som båda var 0. Övriga bitar behålls.

c) `0b00101100`. Varje bit inverteras, eftersom masken har en etta överallt.

d) `0b00101100`, samma som **c**). `com` är detsamma som `eor` med `0xFF`, men behöver inget extra register.

Övning 8.7 Välj rätt mask

sidan 176

a) `ori r16, 0b00000011`

b) `andi r16, 0b01111111`

c) Det finns ingen `eori`, så masken läggs i ett register först:

```
    ldi r24, 0b11110000
    eor r16, r24
```

d) `andi r16, 0b00001000`

e) Två instruktioner, en per sorts ändring:

```
    andi r16, 0b11110000            ; Clear bits 0-3.
    ori r16, 0b10000000             ; Set bit 7.
```

Ordningen spelar ingen roll här, eftersom de två maskerna rör olika bitar.

Övning 8.8 Skift och rotation

sidan 176

a) `0b00000110` = 6. Efter två `lsl`: `0b00011000` = **24**. Talet har multiplicerats med 4.

b) `lsr` flyttar allt ett steg åt höger och skiftar in en nolla: `r17` = **`0b01000000`**, och bit 0, som var 1, hamnar i **`C = 1`**.

c) `asr` behåller teckenbiten: `0xF8` = `1111 1000` blir `1111 1100` = **`0xFC`** = **−4**, vilket är −8/2. `lsr` skiftar in en nolla: `0111 1100` = `0x7C` = 124, som är fel om talet har tecken.

d)

Efter	r19	C
start	1100 0000	0
rol	1000 0000	1
rol	0000 0001	1
rol	0000 0011	0

Vid varje `rol` flyttas bit 7 till `C` och det gamla `C` in i bit 0. Ettorna ”kommer tillbaka” in från höger ett steg efter att de lämnat vänster ände.

Övning 8.9 Lysdioder på port B

sidan 176

b) `sbi` behöver inget register för mönstret, men port B måste fortfarande göras till utport.

d) `PORTB` = **`0b10010001`**: PB7, PB4 och PB0.

Övning 8.10 Läs programmet

sidan 177

Rad	PORTB	Varför
(1)	`0b00001111`	`leds` skrivs ut som den är.
(2)	`0b00111100`	Två `lsl` flyttar mönstret två steg åt vänster.
(3)	`0b00111101`	`sbi` ettställer bit 0 och lämnar resten.
(4)	`0b11000011`	`com` inverterar `leds`, som fortfarande är `0b00111100`, och `out` skriver alla åtta bitar.
(5)	`0b01000011`	`cbi` nollställer bit 7.

Rad (4) är den som brukar bli fel. `sbi PORTB, 0` på rad (3) ändrade porten, men inte registret `leds`, så `com leds` utgår från `0b00111100`. Och `out` skriver hela byten, så ettan som `sbi` satte i bit 0 skrivs över och ersätts med den inverterade bitens värde, som råkar vara 1 också.

Övning 8.11 Multiplicera med tio

sidan 177

b) $70 =$ `0x46`.

c) Upp till **x = 25**: $10 \cdot 25 = 250$ får plats i en byte, men $10 \cdot 26 = 260$ gör det inte och slår om till 4. Redan mellanresultatet $8x$ slår om för $x = 32$ och större.

Övning 8.12 När är det overflow?

sidan 177

a) Med tecken är `0x40` $= +64$, och $+64 + 64 = +128$. Det största talet med tecken är $+127$, så resultatet `0x80` betyder -128: två positiva tal gav ett negativt, och **`V = 1`**.

`0xC0` är -64, och $-64 + (-64) = -128$, som **får plats**. Resultatet `0x80` är -128 och är rätt, så **`V = 0`**.

b) Utan tecken är `0x40 + 0x40` $= 64 + 64 = 128$, som får plats: `C = 0`. `0xC0 + 0xC0` $= 192 + 192 = 384$, som inte får plats: `C = 1`. Det är alltså tvärtom mot `V`, vilket visar att de två flaggorna svarar på olika frågor.

c) En addition ger `V = 1` när **två tal med samma tecken ger ett resultat med motsatt tecken**: två positiva som blir negativt, eller två negativa som blir positivt. Två tal med olika tecken kan aldrig ge overflow, eftersom summan då alltid ligger mellan dem.

KAPITEL 9

Programflöde, loopar och tidsfördröjningar

Övning 9.1 Rita en flödesplan

sidan 194

a) Start, en process ”läs talet i r16”, ett beslut ”bit 0 = 0?”, en utmatning ”PB0 = 1” på Ja-grenen och ”PB1 = 1” på Nej-grenen, grenarna möts, stopp.

b) `andi r16, 0b00000001` behåller bara bit 0. Resultatet är noll precis när talet är jämnt, och då sätts `Z = 1`. Vill man inte förstöra `r16` kopierar man det först till ett annat register.

c) `breq` hoppar när `Z = 1`, alltså när talet är jämnt.

Övning 9.2 Vilket hopp?

sidan 194

Villkor	Instruktioner
a) `r16` = `r17`	`cp r16, r17` / `breq yes`
b) `r16` ≠ 0	`tst r16` / `brne yes`, eller `cpi r16, 0` / `brne yes`
c) `r16` < 100	`cpi r16, 100` / `brlo yes`
d) `r16` ≥ 100	`cpi r16, 100` / `brsh yes`
e) `r16` > 100	`cpi r16, 101` / `brsh yes`
f) `r16` negativt	`tst r16` / `brmi yes`
g) `r16` < `r17`, med tecken	`cp r16, r17` / `brlt yes`

e) är den som brukar bli fel. Det finns inget hopp för ”större än”, men `r16 > 100` är samma sak som `r16 ≥ 101`. Ett annat sätt är att byta ordning i jämförelsen: `ldi r17, 100` / `cp r17, r16` / `brlo yes`, som frågar om 100 < `r16`.

Övning 9.3 Följ programmet

sidan 195

`r16`	Väg	`PORTB`
5	`brlo small` hoppar	1
10	`brlo` hoppar inte (10 är inte < 10), `brsh big` hoppar inte	2
99	som för 10	2
100	`brsh big` hoppar (100 ≥ 100)	3
200	`brsh big` hoppar	3

Programmet delar in talet i tre klasser: **ensiffrigt** (1), **tvåsiffrigt** (2) och **tresiffrigt** (3). Flödesplanen har två beslut i rad, ”r16 < 10?” och ”r16 ≥ 100?”, tre grenar som var och en sätter `r17`, och en gemensam utmatning `PORTB = r17`.

Gränsfallen 10 och 100 är de man ska testa. `brlo` betyder strikt mindre än, `brsh` betyder större än eller lika med.

Övning 9.4 Om, annars

sidan 195

b) `PORTB` blir `0xFF` för 42 och `0x00` för 41.

c) Utan `rjmp` efter den första grenen fortsätter programmet, när det har tänt alla lysdioderna, rakt ned i annars-grenen och släcker dem igen. För 42 blir `PORTB` då `0x00`, alltså fel. För 41 blir det fortfarande rätt, eftersom hoppet (`brne`) går förbi den första grenen. Felet syns alltså bara för det ena värdet, och det är därför båda grenarna måste testas.

Övning 9.5 Hur många varv?

sidan 195

a) 5. Loopen går lika många varv som startvärdet.

b) 1. `dec` gör räknaren till 0, och `brne` hoppar inte.

c) **256**, och eftersom `r17` är en byte blir den $256 - 256 = \mathbf{0}$. Den första `dec` gör 0 till 255, och sedan går loopen 255 varv till. En loop med testet sist körs alltid minst en gång (avsnitt 9.2.1).

d) `brne` körs 5 gånger och hoppar 4 av dem. Den femte gången är räknaren 0, och då fortsätter programmet efter loopen.

Övning 9.6 Summera

sidan 196

b) $1 + 2 + \cdots + 15 = 15 \cdot 16/2 = \mathbf{120}$.

c) Summan $1 + \cdots + n$ är $n(n+1)/2$. För $n = 22$ är den 253, som får plats; för $n = 23$ är den 276, som inte gör det. Det största n är alltså **22**. För $n = 23$ blir `r17` $= 276 - 256 = \mathbf{20}$: summan slår om (avsnitt 8.1.3), och programmet märker ingenting.

Övning 9.7 Räkna cykler

sidan 196

Fördröjningen tar $3n$ klockcykler, och en cykel tar 62,5 ns.

a) $3 \cdot 100 =$ **300 cykler**, $300 \cdot 62{,}5$ ns = **18,75 µs**.

b) $3 \cdot 1 =$ **3 cykler**: `ldi` (1), `dec` (1) och `brne` som inte hoppar (1). 187,5 ns.

c) Startvärdet 0 ger 256 varv: $3 \cdot 256 =$ **768 cykler**, **48 µs**. Den längsta fördröjning en loop kan ge.

Övning 9.8 Välj konstanter

sidan 196

a) 40 µs $= 40 \cdot 16 = 640$ cykler. $n = 640/3 = 213{,}3$. Med **n = 213** blir det 639 cykler, 39,94 µs, en cykel för kort. Lägg till en `nop` före eller efter loopen, så blir det exakt 640 cykler, 40 µs.

b) 5 ms $= 5000 \cdot 16 = 80\,000$ cykler.

- **Utan `nop`**: cykler = OUTER $\cdot$ (3 $\cdot$ INNER + 3). Med INNER = 255 är ett yttre varv 768 cykler, och OUTER $= 80\,000/768 = 104{,}2$. **OUTER = 104** ger $104 \cdot 768 = 79\,872$ cykler, **4,992 ms**.

- **Med `nop`**: cykler = OUTER · (3 · INNER + 4). **OUTER = 125, INNER = 212** ger 125 · 640 = 80 000 cykler, **exakt 5 ms**. Det är samma INNER som för 10 ms i avsnitt 9.2.5, med halva antalet yttre varv.

c) En sekund är 16 000 000 cykler. Två loopar ger högst 256 · 771 = 197 376 cykler, ungefär 12,3 ms. Det behövs ytterligare en loop runt de två, som går ungefär 81 varv, eller en räknare med 16 bitar, som i kapitel 10.

Övning 9.9 Räkna, mät och förklara

sidan 196

a)

Instruktion	Cykler	Gånger	Summa
`ldi r19, 208`	1	1	1
`ldi r18, 255`	1	208	208
`dec r18`	1	208 · 255 = 53 040	53 040
`brne delay_inner`, hoppar	2	208 · 254 = 52 832	105 664
`brne delay_inner`, hoppar inte	1	208	208
`dec r19`	1	208	208
`brne delay_outer`, hoppar	2	207	414
`brne delay_outer`, hoppar inte	1	1	1
Totalt			**159 744**

159 744 · 62,5 ns = **9,984 ms**. Samma som formeln OUTER · (3 · INNER + 3) = 208 · 768.

b) Stoppuret ska visa 159 744 cykler och 9 984 µs. Stämmer det inte, kontrollera först var brytpunkterna sitter, och sedan att Frequency är 16 MHz om det är tiden som skiljer.

c) Mätningen blir **2 cykler längre**, 159 746. Brytpunkten på `sbi PORTB, 0` stoppar innan `sbi` har körts, så nu räknas `sbi` med, och den tar 2 cykler. `cbi` räknas fortfarande inte, eftersom brytpunkten på den stoppar innan den körs. Det här är en viktig vana: en mätning mäter exakt det som ligger mellan brytpunkterna, och man måste veta vad det är.

d) 100 · 768 = **76 800 cykler**, 4,8 ms. Mätningen ska stämma exakt.

e) Kamraten har räknat tre cykler per varv i den inre loopen och ingenting annat. Ett helt yttre varv kostar 3 · INNER + 3 cykler (avsnitt 9.2.5): den yttre loopens egna instruktioner, `ldi r18`, `dec r19` och `brne delay_outer`, kostar 4 cykler, och den inre loopens sista `brne` sparar 1. Det är 3 cykler per yttre varv som saknas, 3 · 208 = **624 cykler**, eller **0,39 %**. Litet här, men felet växer ju fler loopar som ligger i varandra, och det är precis den sortens fel en mätning avslöjar.

Övning 9.10 Flerval

sidan 197

b) Testa alla fyra fallen: läge 0, 1, 2 och ett felläge, till exempel 9. Glöm inte felläget: det är det alternativ som oftast saknas.

Övning 9.11 Arbeta om

sidan 198

Flödesplanen har två beslut, tre grenar, och tre utmatningar med var sitt stopp. Det som skiljer grenarna åt är bara värdet i `r17`; `out` och slutet är gemensamma. I den omarbetade flödesplanen lägger varje gren bara sitt värde i `r17`, och alla tre möts i en enda utmatning, `PORTB = r17`, följd av ett enda stopp.

Det omarbetade programmet gör exakt samma sak, men har en `out` och ett slut. Ska programmet senare gå i en loop, räcker det att ändra slutet till ett hopp tillbaka till början, på ett ställe i stället för tre.

Övning 9.12 Minusgrader

sidan 198

a) -5 lagras som `0xFB` $= 251$ (avsnitt 8.1.5). `cpi r16, 20` räknar $251 - 20 = 231$ utan lån, så `C = 0`. `brlo` hoppar inte, och programmet **slår av** värmen, vid -5 grader. Programmet läser talet utan tecken och tror att det är 251 grader varmt.

b) `brlt`, som läser `S`. Efter `cpi r16, 20` med `r16` $= -5$ är `S = 1`, eftersom $-5 - 20 = -25$ är negativt, och `brlt` hoppar till `heater_on`.

c) Ja. För 25 är $25 - 20 = 5$ positivt, `S = 0`, och `brlt` hoppar inte: värmen slås av, som den ska. För 100 gäller samma sak. Med tecken går temperaturerna att jämföra från -128 till 127 grader, vilket räcker för en termostat.

KAPITEL 10

Inporten, subrutiner och stacken

Varje antal cykler här är uppmätt i simulatorn, inte bara uträknat.

Övning 10.1 Tecken och koder

sidan 222

a) `'5'` $=$ `0x35`, `'9'` $=$ `0x39`, `'E'` $=$ `0x45`, `'F'` $=$ `0x46`.

b) `0x38 - 0x30 = 0x08`, alltså 8. Det är skälet till att `subi r16, 0x30` gör om ett siffertecken till sitt värde.

c) Siffrorna `'0'`–`'9'` ligger direkt efter varandra från `0x30`, men mellan `'9'` (`0x39`) och `'A'` (`0x41`) ligger sju andra tecken, och de måste hoppas över.

d) För läsbarheten. `'E'` säger att konstanten är tänkt som ett tecken; `0x45` säger bara att den är ett tal. Maskinen ser ingen skillnad, men den som läser programmet gör det.

Övning 10.2 En byte blir text

sidan 222

a) Den höga nibblen av `0x9E`:

Instruktion	r24	Kommentar
`mov r24, r16`	`0x9E`	
`swap r24`	`0xE9`	halvorna byter plats
`andi r24, 0x0F`	`0x09`	den höga nibblen
`cpi r24, 10`	`0x09`	$9 < 10$, så `brlo` hoppar över `subi r24, -7`
`subi r24, -0x30`	`0x39`	

`r20` = `0x39` = `'9'`.

b) Den låga nibblen: `mov` ger `0x9E`, `andi` ger `0x0E`. 14 är inte mindre än 10, så `brlo` hoppar inte, och `subi r24, -7` ger `0x15`. Sista `subi` ger `0x45`. `r21` = `0x45` = `'E'`.

c) `0x39` på adress `0x0100` och `0x45` på `0x0101`: texten `9E`.

d) För `0x5A`: den höga nibblen blir `0x05`, som är mindre än 10, och blir `0x35` = `'5'`. Den låga blir `0x0A`, som inte är mindre än 10: `0x0A + 7 = 0x11`, och `0x11 + 0x30 = 0x41` = `'A'`. Texten blir `5A`.

Övning 10.3 Från tecken till värde

sidan 223

a) `0x42 - 0x30 = 0x12` = 18. 18 är inte mindre än 10, så koden subtraherar 7 till: `r22` = 11 = `0x0B`. Rätt: B är 11.

b) `0x34 - 0x30 = 4`. 4 är mindre än 10, och koden är klar: `r22` = 4.

c) `0x62 - 0x30 = 0x32` = 50, och `50 - 7 = 43` = `0x2B`. Det är inget värde 0–15 alls; koden förutsätter stora bokstäver. En liten bokstav har en kod som är `0x20` högre än motsvarande stora: `'a'` = `0x61` och `'A'` = `0x41`. Gör om små bokstäver till stora först:

```
    cpi r22, 'a'                    ; A lowercase letter?
    brlo not_lower
    subi r22, 0x20                  ; 'a'-'f' -> 'A'-'F'.
not_lower:
```

Det frestande alternativet, att nollställa bit 5 med `andi r22, 0xDF`, fungerar för bokstäverna men förstör siffrorna: `'0'` = `0x30` har också bit 5 satt, och skulle bli `0x10`.

Övning 10.4 Registerpar

sidan 223

a)

Tal	Hexadecimalt	`high()`	`low()`
2500	`0x09C4`	`0x09`	`0xC4`
256	`0x0100`	`0x01`	`0x00`
255	`0x00FF`	`0x00`	`0xFF`
40 000	`0x9C40`	`0x9C`	`0x40`

Uträkningen för 2500: `2500 / 256 = 9` med resten `196`, och `196 = 0xC4`. För 40 000: `40 000 / 256 = 156` med resten `64`, och `156 = 0x9C`, `64 = 0x40`.

Lägg märke till 255 och 256: talen skiljer sig med ett, men bytena är helt olika. 255 är den största låga byten, och 256 är den första ettan i den höga.

b) `0x12 · 256 + 0x34 = 18 · 256 + 52 = 4608 + 52 = 4660`.

c) `0xFFFF` = 65 535 = `2^16 - 1`. Sexton bitar ger `2^16` = 65 536 olika värden, och det första är 0.

Övning 10.5 Addition och subtraktion med 16 bitar

sidan 223

a)

```
                r25    r24
               0x12   0xF0      4848
             +  0x0A   0x20      2592
             ---------------
add r24:            0x110  -> r24 = 0x10, C = 1
adc r25:     0x12 + 0x0A + 1 = 0x1D
             ---------------
               0x1D   0x10      7440
```

Kontroll decimalt: `0x12F0 = 18 · 256 + 240 = 4848`, `0x0A20 = 10 · 256 + 32 = 2592`, och `4848 + 2592 = 7440`. Resultatet `0x1D10 = 29 · 256 + 16 = 7440`. Det stämmer.

b) `add` tar inte med minnessiffran: `0x12 + 0x0A = 0x1C`, och svaret blir `0x1C10` = 7184. Det är 256 för lite, precis värdet av den minnessiffra som tappades.

c) `sub r20, r22`: `0x00 - 0x01` går inte utan att låna, så `r20` = `0xFF` och `C` = 1. `sbc r21, r23`: `0x01 - 0x00 - 1 = 0x00`, så `r21` = `0x00`. Resultatet är `0x00FF` = 255, som det ska.

d) `sbiw` fungerar bara på paren `r25:r24`, `r27:r26`, `r29:r28` och `r31:r30` (avsnitt 10.1.8), och assemblern godtar inte `r20`. Med talet i `r25:r24` hade `sbiw r24, 1` fungerat.

Övning 10.6 En fördröjning på 10 ms

sidan 223

a) `0,010 s · 16 000 000 cykler/s = 160 000 cykler`.

b) De två `ldi` tar 2 cykler och loopen `4N - 1`, totalt `4N + 1`. `4N + 1 = 160 000` ger `N = 39 999,75`, som inte är ett heltal. `N = 40 000` ger `160 001` cykler, en cykel för mycket, och `N = 39 999` ger `159 997`, tre för lite. **`N = 40 000`, 160 001 cykler**, vilket också är vad simulatorn mäter.

c) `r24` = 10, och anropet tar `16 000 · 10 + 18 = 160 018` cykler (avsnitt 10.3.4).

d) Loopen är närmast: en cykel, 62,5 ns, fel, mot 18 cykler, drygt 1 µs, för subrutinen. Men båda felen är långt under vad något i ett styrsystem märker. Välj ändå subrutinen: den skrivs en gång, tar tiden som parameter, och lämnar alla register som den fann dem. Loopen måste skrivas av på varje ställe den behövs, och förstör `r25:r24` där.

Övning 10.7 Knappen som läses baklänges

sidan 224

a) När knappen är nedtryckt är stiftet direkt förbundet med jord, 0 V. Pull-up-motståndet på 20–50 kΩ ligger mellan 5 V och stiftet, och hela spänningen hamnar över motståndet, med en mycket liten ström. Stiftet ligger på 0 V och läses som 0.

b) Utan pull-up är stiftet **flytande** när knappen är släppt: ingenting bestämmer dess nivå. Programmet läser 0 eller 1 beroende på brus, luftfuktighet och var händerna är. Felet är svårt att hitta eftersom programmet ofta *verkar* fungera en stund, och slutar fungera när man tar i kortet eller flyttar det.

c) En nedtryckt knapp läses som 0, men en lysdiod tänds av en etta. `com` vänder på varje bit. Utan `com` hade alla lysdioder lyst när ingen knapp var nedtryckt, och en lysdiod hade slocknat när dess knapp trycktes.

Övning 10.8 Följ programmet

sidan 224

a) Bit 2 är 0, så knappen på PD2 är nedtryckt. `com` ger `0b00000100`, och lysdioden på PB2 lyser: `PORTB` = `0b00000100`.

b) Med `andi r16, 0xF0` behålls bara bit 4–7. För `0b11111011` blir det `0b00000100 AND 0xF0 = 0b00000000`: ingenting lyser, trots att en knapp är nedtryckt, eftersom den ligger i den bortmaskade halvan.

För `0b01101111`: `com` ger `0b10010000`, och masken ändrar ingenting. `PORTB` = `0b10010000`, så PB7 och PB4 lyser. PB7 syns i simulatorn men inte på Arduino-kortet, där PB6 och PB7 används av klockan.

c) `in` 1, `com` 1, `out` 1 och `rjmp` 2: 5 cykler per varv. `16 000 000 / 5 = 3 200 000` läsningar per sekund.

Övning 10.9 Start och stopp med självhållning

sidan 224

a) Flödesplanen, i ord:

1. Start. Gör PB0 till utgång, och slå på pull-uparna på PD2 och PD3.
2. Är stoppknappen nedtryckt? **Ja:** släck PB0 och gå till 2. **Nej:** fortsätt.
3. Är startknappen nedtryckt? **Ja:** tänd PB0. **Nej:** gör ingenting.
4. Gå till 2.

Frågan om stopp ställs **först**, så att stopp vinner när båda är nedtryckta.

c) Uppmätt i simulatorn:

Händelse	`PORTB` bit 0
Ingen knapp	0
Start trycks och släpps	1, och förblir 1
Stopp trycks	0
Båda nedtryckta	0

d) Minnet är utgångsregistret `PORTB` självt. När ingen knapp är nedtryckt gör programmet ingenting med PB0, och biten behåller det värde den senast fick. Det motsvarar reläets självhållningskontakt: den håller kvar tillståndet tills något aktivt ändrar det. Det är samma idé som vippan i kapitel 6, ett minne som ändras bara när det tillåts.

Övning 10.10 `rcall`, `ret` och stacken

sidan 225

a) Först läggs returadressen, adressen till instruktionen efter `rcall`, på stacken, och `SP` flyttas två steg nedåt. Sedan hoppar programmet till subrutinen.

b) Returadressen måste finnas kvar tills subrutinen är klar, hur många register subrutinen än använder och hur många anrop den själv gör. Ett fast register hade skrivits över av nästa anrop i en subrutin som anropar en annan. Stacken ger varje pågående anrop en egen plats, och den växer och krymper med anropen.

c) `0x08FD` i subrutinen, eftersom två byte har lagts dit. `0x08FF` igen efter `ret`.

d) Allt som ligger under `SP` räknas som ledigt. Ingen läser det, och nästa `rcall` eller `push` skriver över det.

Övning 10.11 Kontroll: förutsäg stacken

sidan 225

a) `rcall` ligger på `0x0007`, så returadressen är `0x0008`. `rcall` skriver den låga byten först, på `0x08FF`, och den höga på `0x08FE`. Därefter lägger `push r16` värdet `0xA5` på `0x08FD`, och `push r17` värdet `0x3C` på `0x08FC`.

Adress	Byte	Vad det är
`0x08FF`	`0x08`	returadressens låga byte
`0x08FE`	`0x00`	returadressens höga byte
`0x08FD`	`0xA5`	`r16`
`0x08FC`	`0x3C`	`r17`

Fyra byte är upptagna, och `SP` = `0x08FF - 4` = **`0x08FB`**.

b) Simulatorn visar `SP` = `0x08FB`, och i fönstret Memory står, från adress `0x08FC` och uppåt, bytena `3C A5 00 08`: precis tabellen ovan, läst nedifrån och upp.

c) De två vanligaste avvikelserna:

- **Returadressen i fel ordning**, `0x00` på `0x08FF` och `0x08` på `0x08FE`. Den låga byten läggs dit först och hamnar därför på den högre adressen; stacken växer nedåt.
- **`SP` en byte fel**, `0x08FC`. `SP` pekar på den första **lediga** byten, inte på den senast använda. Den senast använda är `0x08FC`, och den första lediga är en under.

d) Efter `pop r17`, `pop r16` och `ret` är `SP` tillbaka på `0x08FF`. Inget av de fyra bytena har ändrats: `pop` och `ret` läser och flyttar `SP`, men raderar ingenting. `r16` och `r17` har sina gamla värden, `0xA5` och `0x3C`.

Övning 10.12 Hitta felet

sidan 226

a) Registren hämtas i samma ordning som de lades dit, i stället för i omvänd ordning. `pop r16` får det som lades dit sist, `r17`:s värde, och tvärtom. Subrutinen återvänder rätt, men programmet som anropar den finner `r16` och `r17` med bytta värden. Felet märks först långt senare, som ett konstigt resultat, och ser inte alls ut att ha med subrutinen att göra.

b) En byte, `r16`:s sparade värde, ligger kvar ovanpå returadressen. `ret` läser den som en del av adressen och hoppar någonstans där ingen har bestämt att programmet ska vara. Därifrån kör processorn vad som helst som råkar ligga i programminnet, ofta tomt flashminne, tills den kommer runt till adress 0 och börjar om. Det märks direkt, men felet syns inte på raden där det sitter.

c) Läs `SP` i Processor Status när programmet står på `rcall`, och igen när det står på instruktionen efter. Är värdena olika, lämnar subrutinen stacken i ett annat skick än den fann den.

Övning 10.13 Hela byten som text

sidan 226

a) `r24` och `r25` får svaren och behöver inte sparas: det står i kommentaren att de är subrutinens resultat. Subrutinen behöver också ett register för att spara byten under tiden, här `r16`, och det måste sparas med `push` och `pop`.

Den låga nibblen görs först, eftersom `nibble_to_ascii` lämnar sitt svar i `r24`, och det är där den höga nibblens tecken ska hamna till sist.

c) I simulatorn står `42 37` på adress `0x0100`, och fönstret visar `B7`. Stacken är som djupast inne i `nibble_to_ascii`: två byte returadress till huvudprogrammet, en byte för `r16` och två byte returadress till `byte_to_ascii`, fem byte. `SP` är då `0x08FA`.

KAPITEL 11

Styrobjekt på riktig hårdvara

De cykeltal som anges här är uppmätta i simulatorn.

Övning 11.1 Ingångar, beslut och utgångar

sidan 241

a) Ingångar: knappen och de två gränslägesbrytarna. Utgångar: motor upp och motor ned. Beslut: ett tryck startar porten mot det läge den inte står i, och motorn stannar när gränslägesbrytaren i det läget slår till. Ett bra program ser också till att upp och ned aldrig är på samtidigt.

b) Ingång: IR-givaren. Utgång: fläkten. Beslut: fläkten går så länge givaren ser en hand, och tio sekunder till efter att den försvunnit. Beslutet innehåller alltså en tid, precis som trafikljuset.

c) Ingång: knappen. Utgångar: den röda, den gula och den gröna lampan. Beslut: grönt tills knappen trycks, sedan en fast följd av tillstånd med bestämda tider, och grönt igen.

Övning 11.2 Förkopplingsmotståndet

sidan 242

a) Spänningen över motståndet är `5 - 2 = 3 V`.

- 220 Ω: `3 / 220 ≈ 0,0136 A = 13,6 mA`
- 330 Ω: `3 / 330 ≈ 9,1 mA`
- 1 kΩ: `3 / 1000 = 3 mA`

b) `(5 - 3) / 220 ≈ 9,1 mA`. Den gröna lysdioden tar mer av spänningen själv, så det blir mindre kvar över motståndet och därmed mindre ström. Dessutom ger olika färger olika mycket ljus för samma ström, så gröna lysdioder upplevs ofta som svagare.

c) `R = 3 V / 0,010 A = 300 Ω`. Närmaste vanliga värde är 330 Ω, som ger 9,1 mA, eller 270 Ω, som ger 11 mA. Båda duger.

d) `3 / 47 ≈ 64 mA`, långt över 40 mA, som är stiftets absoluta maximum. Både lysdioden och stiftet riskerar att förstöras. I praktiken sjunker spänningen på stiftet vid så stor ström, så det blir något mindre, men det är ändå långt utanför vad kretsen är specificerad för.

Övning 11.3 Hela porten

sidan 242

a) `6 · 13,6 mA ≈ 82 mA`.

b) Marginalen är ungefär 18 mA, drygt en lysdiod till. Det fungerar, men det är inte mycket att ha om någon senare lägger till fler utgångar på samma grupp.

c) Med 330 Ω blir det `6 · 9,1 ≈ 55 mA`, med god marginal, och lysdioderna lyser nästan lika starkt. Att välja det lite större motståndet kostar nästan ingenting och tar bort risken.

Övning 11.4 Reläet

sidan 242

a) En reläspole behöver typiskt mer ström än ett stift kan ge, ofta 70 mA eller mer för ett relä på 5 V. Transistorn låter några milliampere från stiftet styra den större strömmen.

b) När transistorn slår av vill spolen hålla sin ström igång, och spänningen över den slår upp till många tiotals volt. Dioden leder då och låter strömmen klinga av genom spolen. Utan dioden går spänningstoppen över transistorn, som kan förstöras redan första gången.

c) Störningar och fel på 24 V-sidan, en kortslutning, en spänningstopp från en motor, kan inte nå mikrodatorn. De två kretsarna kan dessutom ha olika jord.

d) Stiftet ger bara 5 V, så lampan kommer knappt att glöda. Men en kall glödtråd har lågt motstånd, och strömmen kan bli långt över vad stiftet tål. Stiftet, och i värsta fall kretsen, riskerar att förstöras. Rätt sätt är ett relä, som i avsnitt 11.1.3.

Övning 11.5 Knappar som studsar

sidan 242

a) Programmet räknar varje gång stiftet går från 1 till 0. I testet slöts knappen två gånger vid varje tryck, sluts, studs, sluts igen, och en gång till när den släpptes. Varje sådan nolla ser för programmet ut som ett nytt tryck: tre per tryck, sex för två tryck.

b) Väntan ska vara längre än studsarna varar och kortare än den tid en människa behöver mellan två tryck.

- **1 ms** är kortare än studsarna, som kan hålla på i flera millisekunder. Programmet tittar på knappen igen medan den fortfarande studsar, och räknar för mycket.
- **500 ms** tar bort studsarna, men programmet blir trögt: ett andra tryck inom en halv sekund missas, och det märks för den som trycker snabbt.

c) Programmet räknar ingenting. Under några millisekunder av studs flimrar lysdioden, vilket ingen människa ser, och därefter följer den knappen.

Övning 11.6 Hur lång tid tar ett varv? sidan 243

a) `16 000 084 · 4 + 18 = 64 000 354` cykler.

b) Ett tillstånd tar `1 + 1 + 1 + 64 000 354 = 64 000 357` cykler. Två tillstånd och ett `rjmp`: `2 · 64 000 357 + 2 = 128 000 716` cykler. Precis vad kontrollen mätte.

c) 8 s är 128 000 000 cykler, så varvet är 716 cykler för långt. En cykel är 62,5 ns, så `716 · 62,5 ns ≈ 44,8 µs`. En timme är `3600 / 8 = 450` varv, och `450 · 44,8 µs ≈ 20 ms`.

d) Resonatorn kan ge 0,5 %, alltså 18 sekunder på en timme. Det är nästan tusen gånger mer än programmets 20 ms. Det är resonatorn som är värd att bry sig om, om noggrannheten alls spelar roll, och för ett trafikljus gör den inte det.

Övning 11.7 Nattläge sidan 243

a)

Tillstånd	Gul (PB1)	Tid	Därefter
Tänd	tänd	0,5 s	Släckt
Släckt	släckt	0,5 s	Tänd

b) Start, initiering av `SP` och PB1, och sedan en loop utan villkor: tänd gul, vänta 0,5 s, släck gul, vänta 0,5 s, tillbaka till ”tänd gul”. Samma form som vägarbetsljusets flödesplan.

c) Ja, `r24` kan laddas en enda gång, eftersom `delay_ms` följer kursens regel och lämnar `r24` oförändrat. Med en subrutin som inte gjorde det hade `r24` behövt laddas om före varje anrop.

d) Räkna med ett program där `r24` = 250 laddas före loopen, och loopen är `sbi`, två anrop av `delay_ms`, `cbi`, två anrop till och ett `rjmp` tillbaka. Den tända halvan är `sbi` (2 cykler) och två anrop om `4 000 018` cykler: `8 000 038`. Den släckta är `cbi` (2), två anrop och `rjmp` (2): `8 000 040`. En hel period är **16 000 078** cykler, en sekund och knappt 5 µs. Simulatorn mäter samma sak.

Övning 11.8 En knapp som växlar sidan 243

a) Start och initiering, sedan en loop: vänta tills knappen är nedtryckt, växla lysdioden, vänta 20 ms, vänta tills knappen är släppt, vänta 20 ms, och tillbaka.

b) I simulatorn, med samma studsande tryck som `press_counter.asm`, ska lysdioden lysa efter det första trycket och vara släckt efter det andra.

c) Utan väntan växlar lysdioden en gång för varje studs. Är antalet studsar jämnt ser det ut som att trycket inte gjorde någonting, och är det udda fungerar det. Resultatet blir slumpmässigt, och det är just det som gör felet så förvirrande att hitta.

Övning 11.9 Kontroll: strömmen genom lysdioden

sidan 243

a) `(5 - 2) / 220 ≈ 13,6 mA`.

b) Ett typiskt resultat är ungefär 2,8 V över motståndet, alltså `2,8 / 220 ≈ 12,7 mA`, och ungefär 4,8 V mellan stiftet och jord. Era värden beror på lysdioden och på USB-porten.

c) Strömmen blir något mindre än uträknat, av två skäl som mätningen visar:

- **Stiftet ger inte riktigt 5 V under last.** Utgången har ett litet inre motstånd, så spänningen sjunker när en ström går ut ur den. Dessutom är matningen från USB inte alltid exakt 5 V.
- **Lysdiodens spänning är inte exakt 2 V.** Den beror på typ och färg, och varierar något med strömmen.

Skillnaden är några procent, i exemplet ovan ungefär 7 %, och den visar hur mycket en förenklad modell räcker till: tillräckligt för att välja motstånd, inte för att förutsäga strömmen på milliampere när.

d) Den gröna lysdioden har högre spänning över sig, kanske 2,2–3 V beroende på typ. Då blir det mindre kvar över motståndet, och strömmen blir mindre. Mät spänningen över lysdioden, så ser ni hur stor skillnaden är.

Övning 11.10 Felsökning på kortet

sidan 244

a) Troligast är den gröna lysdioden vänd fel, eller kopplad till fel stift. Kontrollera att det långa benet sitter mot motståndet, och att sladden går till stift 10. Byt plats på den gröna och den röda lysdioden: följer felet med lysdioden, är det den eller dess koppling.

b) `QUARTER_MS` står kvar på 1. Hela ljuscykeln går då runt ungefär 30 gånger i sekunden, och ögat ser alla lamporna lysa svagt. Sätt den till 250, bygg om och för över programmet igen.

c) Programmet har aldrig kommit fram till kortet: fel COM-port, eller kortet är inte anslutet. Kontrollera porten i Arduino IDE och i verktygets argument, se avsnitt D.5.6 i bilaga D.

d) Sladdarna till stift 8 och 9 har bytt plats, eller så har bitnumren i `.equ RED` och `.equ YELLOW` bytt plats i programmet. Jämför kopplingen med tabellen i slutuppgiften.

Övning 11.11 En dag på ett trafikljus

sidan 244

a) `0,005 · 86 400 s = 432 s`, alltså 7 minuter och 12 sekunder på ett dygn.

b) `86 400 / 8 = 10 800` ljuscykler.

KAPITEL 12

Repetition inför det skriftliga provet

Övning 12.1 Talomvandlingar

sidan 252

Binärt	Decimalt	Hexadecimalt
`1011 0101`	181	`0xB5`
`1100 1000`	200	`0xC8`
`0111 1110`	126	`0x7E`
`0100 0000`	64	`0x40`

- `1011 0101`: ettorna har vikterna 128, 32, 16, 4 och 1, och `128 + 32 + 16 + 4 + 1 = 181`. Grupperna `1011` och `0101` är `B` och `5`.
- 200: 128 ryms, 72 kvar; 64 ryms, 8 kvar; 8 ryms, 0 kvar. Ettor på vikterna 128, 64 och 8 ger `1100 1000`, och grupperna `1100` och `1000` är `C` och `8`.
- `0x7E`: 7 är `0111` och `E` är `1110`. Vikterna 64, 32, 16, 8, 4 och 2 ger 126.
- 64 är en enda etta, på vikten 64: `0100 0000` = `0x40`.

Övning 12.2 Binär addition och flaggor

sidan 253

a)

```
minnessiffror: 1111 111
                 0110 1101     109
               + 0101 1011   +  91
               -----------   -----
                 1100 1000     200
```

b) Summan 200 får plats i åtta bitar, så det blir ingen minnessiffra ut ur bit 7: **`C = 0`**. Bit 7 i resultatet är 1, så **`N = 1`**.

c) Med tecken är `1100 1000` lika med `200 - 256 = -56`. Två positiva tal har gett ett negativt resultat, eftersom den riktiga summan, 200, är större än 127, det största tal med tecken som ryms i en byte. Det kallas *overflow*, och det är det flaggan `V` visar: här är `V` = 1. Utan tecken är svaret rätt, 200; med tecken är det fel. Samma bitar, två tolkningar.

Övning 12.3 Koder

sidan 253

a) 4 är `0100` och 7 är `0111`: 47 i BCD är **`0100 0111`**. Binärt är 47 `0010 1111`, ett helt annat bitmönster.

b) `000`, `001`, `011`, `010`, `110`, `111`, `101`, `100`. Mellan två grannar ändras **exakt en bit**, även från den sista, `100`, tillbaka till den första, `000`.

c) `O` är den femtonde bokstaven: `0x41 + 14 = 0x4F`. `K` är den elfte: `0x41 + 10 = 0x4B`. Texten `OK` är alltså de två bytena **`0x4F 0x4B`**.

Övning 12.4 Vilken grind är det?

sidan 253

a) NOR-grinden ger `(A + B)'`, AND-grinden ger `AB`, och OR-grinden slår ihop dem:

```
X = (A + B)' + AB
```

Med De Morgan är `(A + B)' = A'B'`, så `X = A'B' + AB`.

b)

A	B	`(A + B)'`	`AB`	X
0	0	1	0	1
0	1	0	0	0
1	0	0	0	0
1	1	0	1	1

c) X är 1 när A och B är lika: det är en **XNOR**-grind. Tre grindar gör det en enda grind hade klarat.

Övning 12.5 Förenkla

sidan 254

a)

```
X = ABC + ABC' + AB'C
  = AB(C + C') + AB'C       bryt ut AB
  = AB + AB'C               komplement: C + C' = 1
  = A(B + B'C)              bryt ut A
  = A(B + C)                förenklingslagen: B + B'C = B + C
  = AB + AC
```

Kontroll: X ska vara 1 på raderna 111, 110 och 101 och 0 på de andra. `AB + AC` är 1 precis när `A` = 1 och minst en av `B` och `C` är 1, alltså just de tre raderna.

b)

```
X = (A + B)(A + B')
  = A + BB'                  distributiva lagen: (A + B)(A + C) = A + BC
  = A + 0                    komplement: BB' = 0
  = A
```

Det går också att multiplicera ut: `AA + AB' + AB + BB' = A + AB' + AB + 0`, och absorption ger `A`. Båda vägarna ger samma svar, och en tabell med fyra rader bekräftar det.

Övning 12.6 Två NAND-grindar

sidan 254

a) De Morgan på hela uttrycket:

```
((AB)' · C)' = ((AB)')' + C'      De Morgan: (PQ)' = P' + Q'
             = AB + C'            dubbel invertering
```

b) Den inre parentesen `(AB)'` är en NAND-grind med A och B. Den yttre är en NAND-grind med den första grindens utgång och C. **Två NAND-grindar** räcker, trots att uttrycket innehåller en AND, en OR och en NOT.

c) En 74HC00 har fyra NAND-grindar, så **en kapsel** räcker, och **två grindar** blir över. Deras ingångar ska kopplas till jord eller matning, inte lämnas flytande.

Övning 12.7 Ett kontaktnät

sidan 254

a) Från +24 V: den slutande kontakten S1 i serie med en parallellkoppling av den slutande kontakten S2 och den **brytande** kontakten S3, och sedan lampan H1 till 0 V. S3 är brytande eftersom uttrycket innehåller `S3'`.

b)

S1	S2	S3	`S2 + S3'`	H1
0	0	0	1	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	0
1	0	0	1	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1

Lampan lyser när S1 är nedtryckt och S3 inte är det (S2 spelar då ingen roll), och när S1 och S2 är nedtryckta (S3 spelar då ingen roll).

Övning 12.8 Karnaughdiagram

sidan 254

a)

AB \ CD	00	01	11	10
00	1	1		
01	1	1		
11			1	1
10			1	1

b) Två grupper om fyra, två block om 2 × 2:

- Uppe till vänster, raderna `00` och `01` och kolumnerna `00` och `01`: där är `A` = 0 och `C` = 0 hela tiden, medan `B` och `D` varierar. Termen är `A'C'`.
- Nere till höger, raderna `11` och `10` och kolumnerna `11` och `10`: `A` = 1 och `C` = 1. Termen är `AC`.

```
X = A'C' + AC
```

c) X är 1 när A och C är lika: en **XNOR** av A och C. B och D påverkar inte X alls.

Vanligt fel: Att skriva axlarna i binär ordning, `00, 01, 10, 11`. Här råkar det inte ändra svaret, eftersom ettorna ligger i paren `00`–`01` och `10`–`11`, som är grannar i båda ordningarna. Men en grupp över kolumnerna `01` och `11` hamnar då i två kolumner som inte ligger bredvid varandra, och uttrycket får fler termer än det behöver. Graykoden är rätt även när det råkar gå bra utan den.

Övning 12.9 Mikrodatorn

sidan 254

a) CPU:n i mitten, med ALU, register, styrenhet och programräknare. Programminnet (flash), dataminnet (SRAM) och in- och utportarna runt om, kopplade till CPU:n med adressbuss och databuss. En klocka som driver alltihop.

b) Programräknaren innehåller adressen till nästa instruktion. I **hämta** läses instruktionen på den adressen ur programminnet, och programräknaren räknas upp till nästa instruktion. I **avkoda** tolkar styrenheten instruktionen. I **utföra** görs arbetet, till exempel en addition i ALU:n. Ett hopp utförs genom att ett nytt värde skrivs till programräknaren.

c) Programmet i **flash** finns kvar; det är därför programmet startar igen. Innehållet i **SRAM** och i **registren** är borta: de har okända värden tills programmet skriver dem. Därför börjar varje program med att ställa in det det behöver, som stackpekaren och portarnas riktning.

Övning 12.10 Kontroll: följ programmet

sidan 255

a) **och b**) Simulatorn visar:

Instruktion	r16	r17	r18	r19	C	Z	N
`add r16, r17`	`0x00`	`0x64`			1	1	0
`sub r18, r17`			`0xA1`		1	0	1
`lsr r19`				`0x50`	1	0	0
`andi r19, 0x0F`				`0x00`	1	1	0

Så räknas raderna ut:

- `0x9C + 0x64 = 0x100`, som inte får plats: `r16` = `0x00`, `C` = 1, och resultatet är noll, så `Z` = 1.
- `5 - 100 = -95`. Det kräver ett lån, så `C` = 1, och resultatet är `256 - 95 = 161` = `0xA1`, med bit 7 satt, så `N` = 1.
- `mov` kopierar `0xA1` till `r19`. `lsr` skiftar åt höger: `1010 0001` blir `0101 0000` = `0x50`, och biten som skiftas ut, bit 0, var 1: `C` = 1.
- `andi r19, 0x0F` behåller den låga halvan av `0x50`, som är 0: `r19` = `0x00` och `Z` = 1.

c) Den vanligaste avvikelsen är att `C` förutsägs bli 0 efter `andi`. Men `andi` ändrar inte `C` alls (referensbladet i bilaga E visar vilka flaggor varje instruktion påverkar), så `C` behåller värdet från `lsr`, alltså 1. En flagga som en instruktion inte skriver ligger kvar från den förra som gjorde det.

Övning 12.11 En loop

sidan 256

a) Ett varv är `nop` 1, `dec` 1 och `brne` 2 cykler, alltså 4. Det sista varvet tar 3, eftersom `brne` inte hoppar. Med `ldi` först:

```
1 + 50 · 4 - 1 = 200 cykler
```

Simulatorn mäter också 200 cykler.

b) `200 · 62,5 ns = 12 500 ns = 12,5 µs`.

c) 25 µs är `25 / 0,0625 = 400` cykler. Koden tar `1 + 4n - 1 = 4n` cykler, så `n` = **100**.

Övning 12.12 Om, annars

sidan 256

a) Start; initiering av PB0 och PB1 som utgångar; beslutet ”`r16` ≥ 100?”; **ja**: tänd PB0; **nej**: tänd PB1; slut.

b) Hoppet är `brsh`, *branch if same or higher*, efter `cpi r16, 100`. Det hoppar när `r16 - 100` inte behövde låna, alltså när `r16` är 100 eller mer, räknat utan tecken. `brlo` hade gett det omvända villkoret, och `brge` hade räknat med tecken, så att till exempel 150 hade tolkats som ett negativt tal.

Programmet självt står inte i boken.

c) I simulatorn tänds PB0 för 150 och för 100, och PB1 för 99. **100** är viktigast att testa: det är gränsen, och det är där ett fel villkor, till exempel ”större än” i stället för ”minst”, syns. Värden långt från gränsen fungerar med båda.

Övning 12.13 Stacken

sidan 256

a) `rcall` lägger två byte, och varje `push` en: `0x08FF - 4 = 0x08FB`.

b) `0x08FF`: returadressens låga byte. `0x08FE`: returadressens höga byte. `0x08FD`: `r16`. `0x08FC`: `r17`.

c) Registren hämtas i samma ordning som de lades dit, inte i omvänd ordning. `pop r16` får `r17`:s gamla värde och `pop r17` får `r16`:s, så programmet som anropade `work` finner `r16` och `r17` med bytta värden.